

Clean Architecture

Bản dịch song ngữ Anh–Việt · A Craftsman's Guide to Software Structure and
Design

Robert C. Martin (nguyên tác)

2026

Mục lục

Foreword — Lời nói đầu	7
Preface — Lời tựa	12
Acknowledgments — Lời cảm ơn	16
About the Author — Về tác giả	18
Introduction — Giới thiệu	19
What Is Design and Architecture? — Thiết kế và kiến trúc là gì?	21
The Goal? — Mục tiêu?	22
Case Study — Nghiên cứu tình huống	22
Conclusion — Kết luận	30
A Tale of Two Values — Câu chuyện về hai giá trị	32
Behavior — Hành vi	32
Architecture — Kiến trúc	33
The Greater Value — Giá trị nào lớn hơn	34
Eisenhower's Matrix — Ma trận Eisenhower	35
Fight for the Architecture — Hãy chiến đấu vì kiến trúc	36
Starting with the Bricks: Programming Paradigms — Bắt đầu từ những viên gạch: Các mô thức lập trình	38
Paradigm Overview — Tổng quan về các mô thức	40
Structured Programming — Lập trình có cấu trúc	40
Object-Oriented Programming — Lập trình hướng đối tượng	40
Functional Programming — Lập trình hàm	41
Food for Thought — Đòi điều để ngẫm nghĩ	42
Conclusion — Kết luận	42
Structured Programming — Lập trình có cấu trúc	43
Proof — Chứng minh	44
A Harmful Proclamation — Một tuyên ngôn tai hại	46
Functional Decomposition — Phân rã theo hàm	47
No Formal Proofs — Không có chứng minh hình thức nào cả	47
Science to the Rescue — Khoa học ra tay giải cứu	48
Tests — Kiểm thử	49
Conclusion — Kết luận	49

Object-Oriented Programming — Lập trình hướng đối tượng (OO)	51
Encapsulation? — Đóng gói?	52
Inheritance? — Kế thừa?	54
Polymorphism? — Đa hình?	56
Conclusion — Kết luận	63
Functional Programming — Lập trình hàm	64
Squares of Integers — Bình phương của các số nguyên	64
Immutability and Architecture — Tính bất biến và kiến trúc	66
Segregation of Mutability — Tách riêng phần khả biến	67
Event Sourcing — Event Sourcing	69
Conclusion — Kết luận	70
Design Principles — Các nguyên tắc thiết kế	72
SRP: The Single Responsibility Principle — SRP: Nguyên tắc Trách nhiệm Đơn nhất	75
Symptom 1: Accidental Duplication — Triệu chứng 1: Trùng lặp tình cờ	76
Symptom 2: Merges — Triệu chứng 2: Merge	79
Solutions — Các giải pháp	80
Conclusion — Kết luận	81
OCP: The Open- Closed Principle — OCP — Nguyên tắc Đóng-Mở	83
A Thought Experiment — Một thí nghiệm tư duy	83
Directional Control — Kiểm soát chiều phụ thuộc	88
Information Hiding — Che giấu thông tin	89
Conclusion — Kết luận	89
LSP: The Liskov Substitution Principle — LSP: Nguyên tắc Thay thế Liskov	91
Guiding the Use of Inheritance — Định hướng cho việc dùng kế thừa	91
The Square/Rectangle Problem — Bài toán hình vuông/hình chữ nhật	92
LSP and Architecture — LSP và kiến trúc	93
Example LSP Violation — Ví dụ về một vi phạm LSP	94
Conclusion — Kết luận	96
ISP: The Interface Segregation Principle — ISP: Nguyên tắc Phân tách Interface	97
ISP and Language — ISP và ngôn ngữ	98
ISP and Architecture — ISP và kiến trúc	99
Conclusion — Kết luận	100
DIP: The Dependency Inversion Principle — DIP: Nguyên tắc Đảo ngược Phụ thuộc	101
Stable Abstractions — Những trừu tượng ổn định	102
Factories — Factory	103
Concrete Components — Các component cụ thể	105
Conclusion — Kết luận	105
Component Principles — Các nguyên tắc về component	106
Components — Components	107
A Brief History of Components — Lược sử của component	107
Relocatability — Khả năng tái định vị	111
Linkers — Linker	112
Conclusion — Kết luận	114

Component Cohesion — Độ gắn kết của component	116
The Reuse/Release Equivalence Principle — Nguyên tắc Tương đương Tái dùng/Phát hành .	116
The Common Closure Principle — Nguyên tắc Đóng chung	118
The Common Reuse Principle — Nguyên tắc Tái dùng chung	119
The Tension Diagram for Component Cohesion — Sơ đồ căng kéo cho độ gắn kết của component	121
Conclusion — Kết luận	123
Component Coupling — Độ kết dính giữa các component	124
The Acyclic Dependencies Principle — Nguyên tắc Phụ thuộc Phi chu trình (The Acyclic Dependencies Principle)	124
Top-Down Design — Thiết kế từ trên xuống	132
The Stable Dependencies Principle — Nguyên tắc Phụ thuộc Ổn định (The Stable Dependencies Principle)	133
AbstractComponents — Các component trừu tượng	141
The Stable Abstractions Principle — Nguyên tắc Trừu tượng Ổn định (The Stable Abstractions Principle)	141
TheZoneofPain — Vùng đau đớn	146
TheZoneofUselessness — Vùng vô dụng	148
Conclusion — Kết luận	151
Architecture — Kiến trúc	152
What Is Architecture? — Kiến trúc là gì?	153
Development — Phát triển	154
Deployment — Triển khai	155
Operation — Vận hành	156
Maintenance — Bảo trì	157
Keeping Options Open — Để ngỏ các lựa chọn	157
Device Independence — Độc lập với thiết bị	160
Junk Mail — Thư rác quảng cáo	162
Physical Addressing — Đánh địa chỉ vật lý	163
Conclusion — Kết luận	165
Independence — Tính độc lập	166
Use Cases — Use case	166
Operation — Vận hành	167
Development — Phát triển	168
Deployment — Triển khai	168
Leaving Options Open — Để ngỏ các lựa chọn	169
Decoupling Layers — Tách rời các tầng	169
Decoupling Use Cases — Tách rời các use case	170
Decoupling Mode — Chế độ tách rời	171
Independent Develop-ability — Khả năng phát triển độc lập	172
Independent Deployability — Khả năng triển khai độc lập	173
Duplication — Trùng lặp	173
Decoupling Modes (Again) — Chế độ tách rời (lần nữa)	174
Conclusion — Kết luận	177
Boundaries: Drawing Lines — Ranh giới: Vạch những đường kẻ	178

A Couple of Sad Stories — Vài câu chuyện buồn	179
FitNesse — FitNesse	182
Which Lines Do You Draw, and When Do You Draw Them? — Bạn vạch những đường nào, và vạch khi nào?	185
What About Input and Output? — Còn đầu vào và đầu ra thì sao?	189
Plugin Architecture — Kiến trúc plugin	190
The Plugin Argument — Lập luận về plugin	192
Conclusion — Kết luận	193
Boundary Anatomy — Giải phẫu ranh giới	195
Boundary Crossing — Vượt ranh giới	195
The Dreaded Monolith — Con quái vật monolith đáng sợ	195
Deployment Components — Component triển khai	198
Threads — Thread	199
Local Processes — Tiến trình cục bộ	199
Services — Dịch vụ	200
Conclusion — Kết luận	201
Policy and Level — Chính sách và cấp	202
Level — Cấp	203
Conclusion — Kết luận	207
Business Rules — Quy tắc nghiệp vụ	208
Entities — Entity	209
Use Cases — Use case	210
Request and Response Models — Mô hình request và response	212
Conclusion — Kết luận	213
Screaming Architecture — Kiến trúc biết “gào lên” (Screaming Architecture)	214
The Theme of an Architecture — Chủ đề của một kiến trúc	215
The Purpose of an Architecture — Mục đích của một kiến trúc	215
But What About the Web? — Thế còn web thì sao?	216
Frameworks Are Tools, Not Ways of Life — Framework là công cụ, không phải lễ sống	216
Testable Architectures — Kiến trúc kiểm thử được	217
Conclusion — Kết luận	217
The Clean Architecture — The Clean Architecture	219
The Dependency Rule — Quy tắc Phụ thuộc (the Dependency Rule)	221
A Typical Scenario — Một kịch bản điển hình	225
Conclusion — Kết luận	227
Presenters and Humble Objects — Presenter và Humble Object	228
The Humble Object Pattern — Humble Object pattern	228
Presenters and Views — Presenter và View	228
Testing and Architecture — Kiểm thử và kiến trúc	230
Database Gateways — Database gateway	230
Data Mappers — Data mapper	231
Service Listeners — Service listener	231
Conclusion — Kết luận	232
Partial Boundaries — Ranh giới bán phần (partial boundary)	233
Skip the Last Step — Bỏ qua bước cuối	233

One-Dimensional Boundaries — Ranh giới một chiều	234
Facades — Facade	235
Conclusion — Kết luận	236
Layers and Boundaries — Tầng và ranh giới	237
Hunt the Wumpus — Hunt the Wumpus	237
Clean Architecture? — Clean Architecture?	239
Crossing the Streams — Bắt chéo các dòng	242
Splitting the Streams — Tách các dòng	243
Conclusion — Kết luận	245
The Main Component — Component Main	247
The Ultimate Detail — Chi tiết tận cùng	247
Conclusion — Kết luận	250
Services: Great and Small — Dịch vụ: Lớn và nhỏ	252
Service Architecture? — Kiến trúc dịch vụ?	252
Service Benefits? — Lợi ích của dịch vụ?	253
The Kitty Problem — Bài toán “mèo cưng”	255
Objects to the Rescue — Đối tượng ra tay cứu nguy	257
Component-Based Services — Dịch vụ dựa trên component	259
Cross-Cutting Concerns — Mối quan tâm xuyên suốt	261
Conclusion — Kết luận	261
The Test Boundary — Ranh giới kiểm thử (test boundary)	263
Tests as System Components — Test như những component của hệ thống	263
Design for Testability — Thiết kế để kiểm thử được	264
The Testing API — API dành cho kiểm thử	265
Conclusion — Kết luận	266
Clean Embedded Architecture — Kiến trúc nhúng sạch (Clean Embedded Architecture)	268
App-titude Test — Bài kiểm tra App-titude	271
The Target-Hardware Bottleneck — Nút thắt phần cứng đích	274
Layers — Các tầng	276
TheHardwareIsaDetail — Phần cứng là một chi tiết	278
TheProcessorIsaDetail — Bộ xử lý là một chi tiết	281
TheOperatingSystemIsaDetail — Hệ điều hành là một chi tiết	285
Conclusion — Kết luận	289
Details — Chi tiết	291
The Database Is a Detail — Database là một chi tiết	292
Relational Databases — Database quan hệ	292
Why Are Database Systems So Prevalent? — Vì sao các hệ database lại phổ biến đến thế?	293
What If There Were No Disk? — Nếu không còn đĩa thì sao?	295
Details — Chi tiết	295
But What about Performance? — Nhưng còn hiệu năng thì sao?	296
Anecdote — Giai thoại	296
Conclusion — Kết luận	298

The Web Is a Detail — Web là một chi tiết	299
The Endless Pendulum — Con lắc bất tận	300
The Upshot — Rốt cuộc là	302
Conclusion — Kết luận	303
Frameworks Are Details — Framework là chi tiết	304
Framework Authors — Tác giả của framework	304
Asymmetric Marriage — Cuộc hôn nhân bất đối xứng	305
The Risks — Những rủi ro	305
The Solution — Giải pháp	306
I Now Pronounce You ... — Giờ tôi tuyên bố hai người là...	307
Conclusion — Kết luận	308
Case Study: Video Sales — Nghiên cứu tình huống: Bán video	309
The Product — Sản phẩm	309
Use Case Analysis — Phân tích use case	310
Component Architecture — Kiến trúc component	312
Dependency Management — Quản lý phụ thuộc	314
Conclusion — Kết luận	315
The Missing Chapter — Chương còn thiếu	316
Package by Layer — Package by Layer	316
Package by Feature — Package by Feature	319
Ports and Adapters — Ports and Adapters	322
Package by Component — Package by Component	325
The Devil Is in the Implementation Details — Ma quỷ nằm ở chi tiết cài đặt	332
Organization versus Encapsulation — Tổ chức so với đóng gói	333
Other Decoupling Modes — Các chế độ tách rời khác	337
Conclusion: The Missing Advice — Kết luận: Lời khuyên còn thiếu	340
Appendix — Phụ lục	341
Architecture Archaeology — Khảo cổ học kiến trúc	342
Union Accounting System — Hệ thống kế toán công đoàn	342
Laser Trim — Laser Trim	352
Aluminum Die-Cast Monitoring — Giám sát đúc khuôn nhôm	356
4-TEL — 4-TEL	357
The Service Area Computer — Service Area Computer	364
C Language — Ngôn ngữ C	369
BOSS — BOSS	371
pCCU — pCCU	372
DLU/DRU — DLU/DRU	375
VRS — VRS	377
The Electronic Receptionist — The Electronic Receptionist	380
Craft Dispatch System — Craft Dispatch System	383
Clear Communications — Clear Communications	386
ROSE — ROSE	390
Architects Registry Exam — Architects Registry Exam	393
Conclusion — Kết luận	396

Foreword — Lời nói đầu

What do we talk about when we talk about **architecture**?

Chúng ta đang nói về điều gì khi nói về kiến trúc (architecture)?

As with any metaphor, describing software through the lens of architecture can hide as much as it can reveal. It can both promise more than it can deliver and deliver more than it promises.

Như mọi phép ẩn dụ khác, việc mô tả phần mềm qua lăng kính kiến trúc có thể che giấu nhiều chẳng kém gì nó phơi bày. Nó vừa có thể hứa hẹn nhiều hơn những gì làm được, vừa có thể làm được nhiều hơn những gì đã hứa.

The obvious appeal of architecture is **structure**, and structure is something that dominates the paradigms and discussions of software development— components, classes, functions, modules, layers, and services, micro or macro. But the gross structure of so many software systems often defies either belief or understanding—Enterprise Soviet schemes destined for legacy, improbable Jenga towers reaching toward the cloud, archaeological layers buried in a big- ball-of-mud slide. It's not obvious that **software structure** obeys our intuition the way building structure does.

Sức hấp dẫn hiển nhiên của kiến trúc nằm ở cấu trúc (structure), và cấu trúc là thứ thống trị mọi mô thức (paradigm) lẫn mọi cuộc tranh luận trong phát triển phần mềm — component, class, hàm, module, tầng, và dịch vụ (service), dù micro hay macro. Nhưng cấu trúc tổng thể của rất nhiều hệ thống phần mềm thường bất chấp cả niềm tin lẫn khả năng thấu hiểu của ta: những sơ đồ kiểu Xô Viết cấp doanh nghiệp sinh ra đã mang phận legacy, những tháp Jenga chông chênh vươn lên tận mây, những tầng địa chất khảo cổ bị chôn vùi trong một trận lở “quả cầu bùn khổng lồ” (big ball of mud). Chẳng có gì hiển nhiên cho thấy cấu trúc phần mềm tuân theo trực giác của ta như cách cấu trúc một toà nhà vẫn tuân theo.

Buildings have an obvious physical structure, whether rooted in stone or concrete, whether arching high or sprawling wide, whether large or small, whether magnificent or mundane. Their structures have little choice but to respect the physics of gravity and their materials. On the other hand—except in its sense of seriousness—software has little time for gravity. And what is software made of? Unlike buildings, which may be made of bricks, concrete,

Toà nhà có cấu trúc vật lý rõ ràng, dù bám rễ vào đá hay bê tông, dù vươn cao hay trải rộng, dù lớn hay nhỏ, dù tráng lệ hay tầm thường. Cấu trúc của chúng chẳng còn lựa chọn nào khác ngoài việc tôn trọng lực hấp dẫn và vật liệu tạo nên chúng. Ngược lại — trừ theo nghĩa “sức nặng” của sự nghiêm túc — phần mềm chẳng bận tâm mấy đến trọng lực. Vậy phần mềm được làm từ gì? Khác với toà nhà có thể được làm từ gạch, bê tông,

wood, steel, and glass, software is made of software. Large software constructs are made from smaller software components, which are in turn made of smaller software components still, and so on. It's coding turtles all the way down.

gỗ, thép và kính, phần mềm lại được làm từ chính phần mềm. Những cấu trúc phần mềm lớn được tạo nên từ các component phần mềm nhỏ hơn, mà các component nhỏ hơn ấy lại được tạo nên từ những component nhỏ hơn nữa, và cứ thế. Cứ code chồng lên code, xuống mãi không thấy đáy.

When we talk about software architecture, software is recursive and fractal in nature, etched and sketched in code. Everything is details. Interlocking levels of **detail** also contribute to a building's architecture, but it doesn't make sense to talk about physical scale in software. Software has structure—many structures and many kinds of structures—but its variety eclipses the range of physical structure found in buildings. You can even argue quite convincingly that there is more **design** activity and focus in software than in building architecture—in this sense, it's not unreasonable to consider software architecture more architectural than building architecture!

Khi ta bàn về kiến trúc phần mềm, phần mềm mang bản chất đệ quy và fractal, được khắc và phác ra bằng mã nguồn. Mọi thứ đều là chi tiết (detail). Các cấp độ chi tiết lồng vào nhau cũng góp phần tạo nên kiến trúc của một toà nhà, nhưng nói về quy mô vật lý trong phần mềm thì lại vô nghĩa. Phần mềm có cấu trúc — rất nhiều cấu trúc và rất nhiều loại cấu trúc — nhưng sự đa dạng ấy làm lu mờ cả dải cấu trúc vật lý ta thấy ở các toà nhà. Bạn thậm chí có thể lập luận khá thuyết phục rằng hoạt động thiết kế (design) và mức độ tập trung vào thiết kế trong phần mềm còn nhiều hơn trong kiến trúc xây dựng — theo nghĩa đó, coi kiến trúc phần mềm “kiến trúc” hơn cả kiến trúc xây dựng cũng chẳng phải điều vô lý!

But physical scale is something humans understand and look for in the world. Although appealing and visually obvious, the boxes on a PowerPoint diagram are not a software system's architecture. There's no doubt they represent a particular view of an architecture, but to mistake boxes for the big picture—for the architecture—is to miss the big picture and the architecture: Software architecture doesn't look like anything. A particular visualization is a choice, not a given. It is a choice founded on a further set of choices: what to include; what to exclude; what to emphasize by shape or color; what to de-emphasize through uniformity or omission. There is nothing natural or intrinsic about one view over another.

Nhưng quy mô vật lý là thứ con người hiểu được và luôn tìm kiếm trong thế giới này. Dù bắt mắt và trực quan đến đâu, mấy cái hộp trên sơ đồ PowerPoint vẫn không phải là kiến trúc của một hệ thống phần mềm. Chắc chắn chúng thể hiện một góc nhìn cụ thể về kiến trúc, nhưng nhằm mấy cái hộp đó với bức tranh toàn cảnh — với chính kiến trúc — là đánh mất cả bức tranh toàn cảnh lẫn kiến trúc: kiến trúc phần mềm chẳng trông giống thứ gì cả. Một cách hình dung cụ thể là một lựa chọn, không phải điều mặc định. Đó là lựa chọn dựa trên hàng loạt lựa chọn khác: đưa gì vào; bỏ gì ra; nhấn mạnh cái gì bằng hình khối hay màu sắc; làm mờ cái gì bằng sự đồng nhất hoặc bằng cách lược bỏ. Chẳng có gì tự nhiên hay cố hữu khiến góc nhìn này đúng hơn góc nhìn kia.

Although it might not make sense to talk about physics and physical scale in software architecture, we do appreciate and care about certain physical constraints. Processor speed and network bandwidth can deliver a harsh verdict on a system's performance. Memory and storage can limit the ambitions of any code base. Software may be such stuff as dreams are made on, but it runs in the physical world.

Dù bàn về vật lý và quy mô vật lý trong kiến trúc phần mềm có thể là vô nghĩa, ta vẫn trân trọng và quan tâm tới một số ràng buộc vật lý nhất định. Tốc độ bộ xử lý và băng thông mạng có thể tuyên một bản án nghiệt ngã cho hiệu năng của hệ thống. Bộ nhớ và dung lượng lưu trữ có thể giới hạn tham vọng của bất kỳ code base nào. Phần mềm có thể được dệt nên từ chất liệu của những giấc mơ, nhưng nó chạy trong thế giới vật lý.

This is the monstrosity in love, lady, that the will is infinite, and the execution confined; that the desire is boundless, and the act a slave to limit.

Đó chính là nghịch lý quái đản của tình yêu, thưa nàng: ý chí thì vô hạn, mà sức thực thi lại hữu hạn; khát khao thì không bờ bến, còn hành động lại làm nô lệ cho giới hạn.

—William Shakespeare

—William Shakespeare

The physical world is where we and our companies and our economies live. This gives us another calibration we can understand software architecture by, other less physical forces and quantities through which we can talk and reason.

Thế giới vật lý là nơi chúng ta, các công ty của chúng ta và nền kinh tế của chúng ta đang sống. Điều đó cho ta thêm một thước đo để hiểu kiến trúc phần mềm, thêm những lực và những đại lượng ít vật lý hơn để ta bàn luận và lập luận.

Architecture represents the significant design decisions that shape a system, where significant is measured by cost of change.

Kiến trúc thể hiện những quyết định thiết kế trọng yếu định hình một hệ thống, trong đó “trọng yếu” được đo bằng chi phí thay đổi.

—Grady Booch

—Grady Booch

Time, money, and **effort** give us a sense of scale to sort between the large and the small, to distinguish the architectural stuff from the rest. This measure also tells us how we can determine whether an architecture is good or not: Not only does a good architecture meet the needs of its users, developers, and owners at a given point in time, but it also meets them over time.

Thời gian, tiền bạc và công sức cho ta một cảm nhận về quy mô để phân định cái lớn với cái nhỏ, để tách phần thuộc về kiến trúc ra khỏi phần còn lại. Thước đo này cũng cho ta biết cách xác định một kiến trúc là tốt hay không: một kiến trúc tốt không chỉ đáp ứng nhu cầu của người dùng, lập trình viên và chủ sở hữu tại một thời điểm nhất định, mà còn đáp ứng được những nhu cầu ấy theo thời gian.

If you think good architecture is expensive, try bad architecture.

Nếu bạn thấy kiến trúc tốt là đắt đỏ, hãy thử kiến trúc tồi xem sao.

—Brian Foote and Joseph Yoder

—Brian Foote và Joseph Yoder

The kinds of changes a system's development typically experiences should not be the changes that are costly, that are hard to make, that take managed projects of their own rather than being folded into the daily and weekly flow of work.

Những loại thay đổi mà quá trình phát triển của một hệ thống thường gặp phải lẽ ra không được là những thay đổi tốn kém, khó thực hiện, đòi hỏi hẳn một dự án riêng để quản lý thay vì hoà vào dòng công việc hằng ngày, hằng tuần.

That point leads us to a not-so-small physics-related problem: time travel. How do we know what those typical changes will be so that we can shape those significant decisions around them? How do we reduce future development effort and cost without crystal balls and time machines?

Điều đó dẫn ta tới một vấn đề liên quan đến vật lý mà chẳng nhỏ chút nào: du hành thời gian. Làm sao ta biết được những thay đổi thường gặp ấy là gì để định hình các quyết định trọng yếu xoay quanh chúng? Làm sao ta giảm được công sức và chi phí phát triển trong tương lai khi không có quả cầu tiên tri lẫn cỗ máy thời gian?

Architecture is the decisions that you wish you could get right early in a project, but that you are not necessarily more likely to get them right than any other.

Kiến trúc là những quyết định mà bạn ước gì mình chọn đúng ngay từ đầu dự án, nhưng khả năng bạn chọn đúng chúng chưa chắc đã cao hơn bất kỳ quyết định nào khác.

—Ralph Johnson

—Ralph Johnson

Understanding the past is hard enough as it is; our grasp of the present is slippery at best; predicting the future is nontrivial.

Hiểu được quá khứ vốn đã đủ khó; nắm bắt hiện tại thì mong manh lắm cũng chỉ tới thế; còn dự đoán tương lai lại chẳng hề tầm thường.

This is where the road forks many ways.

Và đây là chỗ con đường rẽ ra nhiều ngã.

Down the darkest path comes the idea that strong and **stable** architecture comes from authority and rigidity. If change is expensive, change is eliminated—its causes subdued or headed off into a bureaucratic ditch. The architect's mandate is total and totalitarian, with the architecture becoming a dystopia for its developers and a constant source of frustration for all.

Xuống ngã tối tăm nhất là quan niệm rằng kiến trúc mạnh mẽ và ổn định sinh ra từ quyền uy và sự cứng nhắc. Nếu thay đổi là tốn kém thì thay đổi sẽ bị loại bỏ — nguyên nhân của nó bị dập tắt hoặc bị lừa vào một cái rãnh hành chính. Quyền lực của kiến trúc sư là tuyệt đối và toàn trị, còn kiến trúc thì trở thành một xã hội phản địa đàng với lập trình viên và là nguồn cơn bức bối triền miên cho tất cả mọi người.

Down another path comes a strong smell of speculative generality. A route filled with hard-coded guesswork, countless parameters, tombs of dead code, and more accidental complexity than you can shake a maintenance budget at.

Xuống một ngã khác là mùi nồng nặc của thói khái quát hoá theo phỏng đoán. Một lối đi đầy những phỏng đoán hard-code, vô số tham số, những năm mồ code chết, và nhiều độ phức tạp ngẫu sinh đến mức đem cả ngân sách bảo trì ra dọa cũng chẳng ăn thua.

The path we are most interested is the cleanest one. It recognizes the softness of software and aims to preserve it as a first-class property of the system. It recognizes that we operate with incomplete knowledge, but it also understands that, as humans, operating with incomplete knowledge is something we do, something we're good at. It plays more to our strengths than to our weaknesses. We create things and we discover things. We ask questions and we run experiments. A good architecture comes from understanding it more as a journey than as a destination, more as an ongoing process of enquiry than as a frozen artifact.

Ngã đường mà chúng ta quan tâm nhất là ngã sạch sẽ nhất. Nó thừa nhận tính “mềm” của phần mềm và hướng tới việc giữ gìn tính mềm đó như một thuộc tính hạng nhất của hệ thống. Nó thừa nhận rằng ta làm việc với hiểu biết chưa đầy đủ, nhưng nó cũng hiểu rằng làm việc với hiểu biết chưa đầy đủ chính là điều con người vẫn làm, và làm rất giỏi. Nó phát huy điểm mạnh của ta thay vì khoét vào điểm yếu. Ta tạo ra mọi thứ và ta khám phá mọi thứ. Ta đặt câu hỏi và ta chạy thí nghiệm. Một kiến trúc tốt đến từ việc hiểu nó như

một hành trình chứ không phải một đích đến, như một quá trình truy vấn liên tục chứ không phải một hiện vật đông cứng.

Architecture is a hypothesis, that needs to be proven by implementation and measurement.

Kiến trúc là một giả thuyết, cần được chứng minh bằng cài đặt và đo đạc.

—Tom Gilb

—Tom Gilb

To walk this path requires care and attention, thought and observation, practice and principle. This might at first sound slow, but it's all in the way that you walk.

Đi trên con đường này đòi hỏi sự cẩn trọng và chú tâm, tư duy và quan sát, thực hành và nguyên tắc. Thoạt nghe có vẻ chậm, nhưng tất cả nằm ở cách bạn bước đi.

The only way to go fast, is to go well.

Cách duy nhất để đi nhanh là đi cho chuẩn.

—Robert C. Martin

—Robert C. Martin

Enjoy the journey.

Chúc bạn tận hưởng hành trình.

—Kevlin Henney

—Kevlin Henney

May 2017

Tháng 5 năm 2017

Preface — Lời tựa

The title of this book is **Clean Architecture** . That's an audacious name. Some would even call it arrogant. So why did I choose that title, and why did I write this book?

Cuốn sách này có tên Clean Architecture . Đó là một cái tên táo bạo. Có người còn gọi nó là ngạo mạn. Vậy tại sao tôi lại chọn cái tên ấy, và tại sao tôi viết cuốn sách này?

I wrote my very first line of code in 1964, at the age of 12. The year is now 2016, so I have been writing code for more than half a century. In that time, I have learned a few things about how to structure software systems—things that I believe others would likely find valuable.

Tôi viết dòng code đầu tiên của đời mình vào năm 1964, khi 12 tuổi. Giờ là năm 2016, tức là tôi đã viết code hơn nửa thế kỷ. Trong quãng thời gian đó, tôi đã học được đôi điều về cách cấu trúc các hệ thống phần mềm — những điều mà tôi tin là người khác cũng sẽ thấy có giá trị.

I learned these things by building many systems, both large and small. I have built small **embedded** systems and large batch processing systems. I have built real-time systems and web systems. I have built console apps, GUI apps, process control apps, games, accounting systems, telecommunications systems, design tools, drawing apps, and many, many others.

Tôi học được những điều đó bằng cách xây dựng rất nhiều hệ thống, cả lớn lẫn nhỏ. Tôi đã xây các hệ thống nhúng (embedded) nhỏ và các hệ thống xử lý theo lô cỡ lớn. Tôi đã xây hệ thống thời gian thực và hệ thống web. Tôi đã xây ứng dụng console, ứng dụng GUI, ứng dụng điều khiển quy trình, trò chơi, hệ thống kế toán, hệ thống viễn thông, công cụ thiết kế, ứng dụng vẽ, và rất, rất nhiều thứ khác.

I have built single-threaded apps, multithreaded apps, apps with few heavy-weight processes, apps with many light-weight processes, multiprocessor apps, database apps, mathematical apps, computational geometry apps, and many, many others.

Tôi đã xây ứng dụng đơn luồng, ứng dụng đa luồng, ứng dụng với vài tiến trình hạng nặng, ứng dụng với nhiều tiến trình hạng nhẹ, ứng dụng đa bộ xử lý, ứng dụng cơ sở dữ liệu, ứng dụng toán học, ứng dụng hình học tính toán, và rất, rất nhiều thứ khác.

I've built a lot of apps. I've built a lot of systems. And from them all, and by taking them all into consideration, I've learned something startling.

Tôi đã xây rất nhiều ứng dụng. Tôi đã xây rất nhiều hệ thống. Và từ tất cả chúng, khi đặt tất cả chúng cạnh nhau, tôi học được một điều gây sửng sốt.

The architecture rules are the same!

Các quy tắc kiến trúc đều giống nhau!

This is startling because the systems that I have built have all been so radically different. Why should such different systems all share similar rules of architecture? My conclusion is that the rules of software architecture are independent of every other variable .

Điều này gây sửng sốt vì những hệ thống tôi đã xây khác nhau đến mức triệt để. Tại sao những hệ thống khác nhau đến vậy lại cùng chia sẻ những quy tắc kiến trúc tương tự nhau? Kết luận của tôi là các quy tắc của kiến trúc phần mềm độc lập với mọi biến số khác

This is even more startling when you consider the change that has taken place in hardware over the same half-century. I started programming on machines the size of kitchen refrigerators that had half-megahertz cycle times, 4K of core memory, 32K of disk memory, and a 10 character per second teletype **interface**. I am writing this preface on a bus while touring in South Africa. I am using a MacBook with four i7 cores running at 2.8 gigahertz each. It has 16 gigabytes of RAM, a terabyte of SSD, and a 2880 * 1800 retina display capable of showing extremely high-definition video. The difference in computational power is staggering. Any reasonable analysis will show that this MacBook is at least 10²² more powerful than those early computers that I started using half a century ago.

Điều này còn gây sửng sốt hơn nữa nếu bạn nghĩ đến những đổi thay đã diễn ra ở phần cứng trong cùng nửa thế kỷ ấy. Tôi bắt đầu lập trình trên những cỗ máy to bằng cái tủ lạnh nhà bếp, có chu kỳ nửa megahertz, 4K bộ nhớ lõi, 32K bộ nhớ đĩa, và một giao diện teletype chạy 10 ký tự mỗi giây. Tôi đang viết lời tựa này trên một chuyến xe buýt trong hành trình vòng quanh Nam Phi. Tôi dùng một chiếc MacBook với bốn nhân i7 chạy ở 2.8 gigahertz mỗi nhân. Nó có 16 gigabyte RAM, một terabyte SSD, và màn hình retina 2880 * 1800 đủ sức trình chiếu video độ nét cực cao. Chênh lệch về sức mạnh tính toán là kinh khủng. Bất kỳ phép phân tích hợp lý nào cũng sẽ cho thấy chiếc MacBook này mạnh gấp ít nhất 10²² lần những chiếc máy tính đời đầu mà tôi bắt đầu sử dụng nửa thế kỷ trước.

Twenty-two orders of magnitude is a very large number. It is the number of angstroms from Earth to Alpha-Centuri. It is the number of electrons in the change in your pocket or purse. And yet that number—that number at least—is the computational power increase that I have experienced in my own lifetime.

Hai mươi hai bậc độ lớn là một con số rất lớn. Đó là số angstrom từ Trái Đất tới Alpha-Centuri. Đó là số electron trong mớ tiền lẻ trong túi quần hay ví của bạn. Vậy mà con số ấy — ít nhất là con số ấy — chính là mức tăng sức mạnh tính toán mà tôi đã tận mắt chứng kiến trong đời mình.

And with all that vast change in computational power, what has been the effect on the software I write? It's gotten bigger certainly. I used to think 2000 lines was a big program. After all, it was a full box of cards that weighed 10 pounds. Now, however, a program isn't really big until it exceeds 100,000 lines.

Và với toàn bộ sự thay đổi khổng lồ về sức mạnh tính toán đó, tác động lên phần mềm tôi viết là gì? Chắc chắn nó đã to hơn. Tôi từng nghĩ 2000 dòng là một chương trình lớn. Suy cho cùng, đó là cả một hộp thẻ đục lỗ nặng 10 pound. Còn bây giờ, một chương trình chưa thực sự lớn cho tới khi nó vượt quá 100.000 dòng.

The software has also gotten much more performant. We can do things today that we could scarcely dream about in the 1960s. The Forbin Project , The

Phần mềm cũng đã nhanh hơn rất nhiều. Hôm nay chúng ta làm được những việc mà vào thập niên 1960 hầu như không dám mơ tới. The Forbin Project , The

Moon Is a Harsh Mistress , and 2001: A Space Odyssey all tried to imagine our current future, but missed the mark rather significantly. They all imagined huge machines that gained sentience. What we have instead are impossibly small machines that are still ... just machines.

Moon Is a Harsh Mistress , và 2001: A Space Odyssey đều cố hình dung ra chính cái tương lai mà ta đang sống, và đều trượt khá xa mục tiêu. Tất cả đều tưởng tượng ra những cỗ máy khổng lồ đạt tới tri giác. Thứ ta có được thay vào đó là những cỗ máy nhỏ đến khó tin nhưng vẫn chỉ là... máy móc.

And there is one thing more about the software we have now, compared to the software from back then: It's made of the same stuff. It's made of if statements, **assignment** statements, and while loops.

Và có thêm một điều nữa về phần mềm ta có bây giờ, so với phần mềm ngày đó: nó vẫn được làm từ cùng một thứ chất liệu. Nó được làm từ lệnh if, lệnh gán và vòng lặp while.

Oh, you might object and say that we've got much better languages and superior paradigms. After all, we program in Java, or C#, or Ruby, and we use object-oriented design. True—and yet the code is still just an assemblage of sequence, selection, and iteration, just as it was back in the 1960s and 1950s.

Ồ, bạn có thể phản đối và nói rằng chúng ta đã có những ngôn ngữ tốt hơn nhiều và những mô thức ưu việt hơn hẳn. Suy cho cùng, ta lập trình bằng Java, C# hay Ruby, và ta dùng thiết kế hướng đối tượng. Đúng vậy — nhưng code vẫn chỉ là một tổ hợp của tuần tự, chọn lựa và lặp, y như hồi thập niên 1960 và 1950.

When you really look closely at the practice of programming computers, you realize that very little has changed in 50 years. The languages have gotten a little better. The tools have gotten fantastically better. But the basic building blocks of a computer program have not changed.

Khi nhìn thật kỹ vào thực tế của việc lập trình máy tính, bạn sẽ nhận ra rằng rất ít thứ thay đổi trong 50 năm qua. Các ngôn ngữ đã tốt hơn một chút. Công cụ thì tốt hơn đến mức phi thường. Nhưng những viên gạch nền tảng của một chương trình máy tính thì chẳng đổi thay.

If I took a computer programmer from 1966 forward in time to 2016 and put her 1 in front of my MacBook running IntelliJ and showed her Java, she might need 24 hours to recover from the shock. But then she would be able to write the code. Java just isn't that different from C, or even from Fortran.

Nếu tôi đưa một lập trình viên từ năm 1966 tới năm 2016, đặt cô ấy 1 ngời trước chiếc MacBook của tôi đang chạy IntelliJ và cho cô ấy xem Java, có lẽ cô ấy cần 24 giờ để hoàn hồn sau cú sốc. Nhưng rồi cô ấy sẽ viết được code. Java chẳng khác C là mấy, thậm chí cũng chẳng khác Fortran là mấy.

And if I transported you back to 1966 and showed you how to write and edit PDP-8 code by punching paper tape on a 10 character per second teletype, you might need 24 hours to recover from the disappointment. But then you would be able to write the code. The code just hasn't changed that much.

Còn nếu tôi đưa bạn ngược về năm 1966 và chỉ cho bạn cách viết và sửa code PDP-8 bằng cách đục băng giấy trên một máy teletype 10 ký tự mỗi giây, có lẽ bạn cần 24 giờ để nguôi ngoai nỗi thất vọng. Nhưng rồi bạn sẽ viết được code. Code chẳng thay đổi nhiều đến thế đâu.

That's the secret: This changelessness of the code is the reason that the rules of software architecture are so consistent across system types. The rules of software architecture are the rules of ordering and assembling the building

Đó chính là bí mật: sự bất biến ấy của code là lý do khiến các quy tắc kiến trúc phần mềm nhất quán đến vậy giữa mọi loại hệ thống. Các quy tắc của kiến trúc phần mềm là quy tắc sắp xếp và lắp ráp những viên gạch

- 1. And she very likely would be female since, back then, women made up a large fraction of programmers.
 - 1. Và rất có khả năng người đó là nữ, bởi thời ấy phụ nữ chiếm một tỷ lệ lớn trong giới lập trình viên.

blocks of programs. And since those building blocks are universal and haven't changed, the rules for ordering them are likewise universal and changeless.

nền tảng của chương trình. Và bởi những viên gạch nền tảng đó mang tính phổ quát và chưa hề thay đổi, nên các quy tắc sắp xếp chúng cũng phổ quát và bất biến y như vậy.

Younger programmers might think this is nonsense. They might insist that everything is new and different nowadays, that the rules of the past are past and gone. If that is what they think, they are sadly mistaken. The rules have not changed. Despite all the new languages, and all the new frameworks, and all the paradigms, the rules are the same now as they were when Alan Turing wrote the first machine code in 1946.

Các lập trình viên trẻ có thể cho rằng đây là chuyện vớ vẩn. Họ có thể khẳng định rằng ngày nay mọi thứ đều mới mẻ và khác biệt, rằng những quy tắc của quá khứ đã là quá khứ và đã qua rồi. Nếu họ nghĩ vậy thì họ đã lầm to. Các quy tắc không hề thay đổi. Bất chấp mọi ngôn ngữ mới, mọi framework mới và mọi mô thức mới, các quy tắc hôm nay vẫn y như hồi Alan Turing viết dòng mã máy đầu tiên vào năm 1946.

But one thing has changed: Back then, we didn't know what the rules were. Consequently, we broke them, over and over again. Now, with half a century of experience behind us, we have a grasp of those rules.

Nhưng có một thứ đã thay đổi: hồi đó, chúng ta không biết các quy tắc ấy là gì. Hệ quả là chúng ta vi phạm chúng, hết lần này đến lần khác. Còn bây giờ, với nửa thế kỷ kinh nghiệm sau lưng, chúng ta đã nắm được những quy tắc đó.

And it is those rules—those timeless, changeless, rules—that this book is all about.

Và chính những quy tắc ấy — những quy tắc vượt thời gian, bất biến ấy — là toàn bộ nội dung của cuốn sách này.

Register your copy of Clean Architecture on the InformIT site for convenient access to updates and/or corrections as they become available. To start the registration process, go to informit.com/register and log in or create an account. Enter the product ISBN (9780134494166) and click Submit. Look on the Registered Products tab for an Access Bonus Content link next to this product, and follow that link to access the bonus materials.

Hãy đăng ký bản sách Clean Architecture của bạn trên trang InformIT để tiện truy cập các bản cập nhật và/hoặc đính chính ngay khi chúng được phát hành. Để bắt đầu quá trình đăng ký, hãy vào informit.com/register rồi đăng nhập hoặc tạo tài khoản. Nhập ISBN của sản phẩm (9780134494166) và bấm Submit. Tìm ở tab Registered Products liên kết Access Bonus Content nằm cạnh sản phẩm này, rồi theo liên kết đó để truy cập phần nội dung bổ sung.

Acknowledgments — Lời cảm ơn

The people who played a part in the creation of this book—in no particular order:

Những người đã góp phần tạo nên cuốn sách này — không theo thứ tự nào cả:

Chris Guzikowski Chris Zahn

Chris Guzikowski Chris Zahn

Matt Heuser Jeff Overbey

Matt Heuser Jeff Overbey

Micah Martin Justin Martin

Micah Martin Justin Martin

Carl Hickman James Grenning

Carl Hickman James Grenning

Simon Brown Kevlin Henney

Simon Brown Kevlin Henney

Jason Gorman Doug Bradbury

Jason Gorman Doug Bradbury

Colin Jones Grady Booch

Colin Jones Grady Booch

Kent Beck

Kent Beck

Martin Fowler

Martin Fowler

Alistair Cockburn James O. Coplien

Alistair Cockburn James O. Coplien

Tim Conrad Richard Lloyd

Tim Conrad Richard Lloyd

Ken Finder

Ken Finder

Kris Iyer (CK) Mike Carew

Kris Iyer (CK) Mike Carew

Jerry Fitzpatrick Jim Newkirk

Jerry Fitzpatrick Jim Newkirk

Ed Thelen Joe Mabel

Ed Thelen Joe Mabel

Bill Degnan

Bill Degnan

And many others too numerous to name.

Và còn rất nhiều người khác, nhiều tới mức không thể kể hết tên.

In my final review of this book, as I was reading the chapter on **Screaming Architecture**, Jim Weirich's bright-eyed smile and melodic laugh echoed through my mind. Godspeed, Jim!

Trong lần rà soát cuối cùng cuốn sách này, khi đọc chương về Kiến trúc biết “gào lên” (Screaming Architecture), nụ cười rạng rỡ và tiếng cười ngân nga của Jim Weirich lại vang vọng trong tâm trí tôi. Thượng lộ bình an, Jim!

About the Author — Về tác giả

Robert C. Martin (Uncle Bob) has been a programmer since 1970. He is the co-founder of cleancoders.com, which offers online video training for software developers, and is the founder of Uncle Bob Consulting LLC, which offers software consulting, training, and skill development services to major corporations worldwide. He served as the Master **Craftsman** at 8th Light, Inc., a Chicago-based software consulting firm. He has published dozens of articles in various trade journals and is a regular speaker at international conferences and trade shows. He served three years as the editor-in-chief of the C++ Report and served as the first chairman of the Agile Alliance.

Robert C. Martin (Uncle Bob) làm lập trình viên từ năm 1970. Ông là đồng sáng lập cleancoders.com, nơi cung cấp các khoá đào tạo video trực tuyến cho lập trình viên, và là người sáng lập Uncle Bob Consulting LLC, đơn vị cung cấp dịch vụ tư vấn, đào tạo và phát triển kỹ năng phần mềm cho các tập đoàn lớn trên toàn thế giới. Ông từng giữ vai trò Master Craftsman tại 8th Light, Inc., một công ty tư vấn phần mềm có trụ sở tại Chicago. Ông đã đăng hàng chục bài báo trên nhiều tạp chí chuyên ngành và là diễn giả thường xuyên tại các hội nghị và triển lãm quốc tế. Ông từng làm tổng biên tập tạp chí C++ Report trong ba năm và là chủ tịch đầu tiên của Agile Alliance.

Martin has authored and edited many books, including *The Clean Coder* , *Clean Code* , *UML for Java Programmers* , *Agile Software Development* , *Extreme Programming in Practice* , *More C++ Gems* , *Pattern Languages of Program Design 3* , and *Designing Object Oriented C++ Applications Using the Booch Method* .

Martin là tác giả và biên tập của nhiều cuốn sách, trong đó có *The Clean Coder* , *Clean Code* , *UML for Java Programmers* , *Agile Software Development* , *Extreme Programming in Practice* , *More C++ Gems* , *Pattern Languages of Program Design 3* , và *Designing Object Oriented C++ Applications Using the Booch Method* .

Introduction — Giới thiệu

It doesn't take a huge amount of knowledge and skill to get a program working. Kids in high school do it all the time. Young men and women in college start billion-dollar businesses based on scrabbling together a few lines of PHP or Ruby. Hoards of junior programmers in cube farms around the world slog through massive requirements documents held in huge issue tracking systems to get their systems to “work” by the sheer brute force of will. The code they produce may not be pretty; but it works. It works because getting something to work—once—just isn't that hard.

Chẳng cần bao nhiêu kiến thức và kỹ năng để làm cho một chương trình chạy được. Học sinh cấp ba vẫn làm chuyện đó suốt ngày. Những chàng trai cô gái còn ngồi ghế đại học dựng nên các doanh nghiệp tỷ đô chỉ từ dăm ba dòng PHP hay Ruby chắp vá lại với nhau. Hàng đàn lập trình viên non tay trong những văn phòng ngăn ô khắp thế giới lê lết qua các tập tài liệu yêu cầu đồ sộ nằm trong những hệ thống theo dõi vấn đề khổng lồ, chỉ để ép hệ thống của mình “chạy” bằng thuần túy sức mạnh của ý chí. Mãi họ viết ra có thể chẳng đẹp đẽ gì; nhưng nó chạy. Nó chạy bởi vì làm cho một thứ chạy được — một lần thôi — vốn chẳng khó đến thế.

Getting it right is another matter entirely. Getting software right is hard. It takes knowledge and skills that most young programmers haven't yet acquired. It requires thought and insight that most programmers don't take the time to develop. It requires a **level** of discipline and dedication that most programmers never dreamed they'd need. Mostly, it takes a passion for the craft and the desire to be a professional.

Làm cho đúng lại là chuyện hoàn toàn khác. Làm phần mềm cho đúng là việc khó. Nó đòi hỏi kiến thức và kỹ năng mà phần lớn lập trình viên trẻ chưa kịp có. Nó đòi hỏi sự suy nghĩ và chiều sâu thấu hiểu mà phần lớn lập trình viên không chịu bỏ thời gian ra rèn. Nó đòi hỏi một mức kỷ luật và tận tụy mà phần lớn lập trình viên chưa bao giờ mơ là mình sẽ cần đến. Và trên hết, nó đòi hỏi niềm đam mê với nghề cùng khát khao trở thành người chuyên nghiệp.

And when you get software right, something magical happens: You don't need hordes of programmers to keep it working. You don't need massive requirements documents and huge issue tracking systems. You don't need global cube farms and 24/7 programming.

Và khi bạn làm phần mềm cho đúng, một điều kỳ diệu sẽ xảy ra: bạn không cần cả bầy lập trình viên để giữ cho nó chạy. Bạn không cần những tập tài liệu yêu cầu đồ sộ và những hệ thống theo dõi vấn đề khổng lồ. Bạn không cần các văn phòng ngăn ô trải khắp toàn cầu và việc lập trình 24/7.

When software is done right, it requires a fraction of the **human resources** to create and maintain. Changes are simple and rapid. Defects are few and far between. **Effort** is minimized, and functionality and flexibility are maximized.

Khi phần mềm được làm cho đúng, nó chỉ tốn một phần nhỏ nhân lực để tạo ra và bảo trì. Thay đổi trở nên đơn giản và nhanh chóng. Lỗi thì ít và thưa thớt. Công sức được giảm tới mức tối thiểu, còn tính năng và độ linh hoạt thì đạt mức tối đa.

Yes, this vision sounds a bit utopian. But I've been there; I've seen it happen. I've worked in projects where the **design** and **architecture** of the system made it easy to write and easy to maintain. I've experienced projects that required a fraction of the anticipated human resources. I've worked on systems that had extremely low defect rates. I've seen the extraordinary effect that good software architecture can have on a system, a project, and a team. I've been to the promised land.

Đúng, viễn cảnh này nghe có phần không tưởng. Nhưng tôi đã ở đó; tôi đã tận mắt thấy nó xảy ra. Tôi từng làm trong những dự án mà thiết kế (design) và kiến trúc (architecture) của hệ thống khiến việc viết lẫn việc bảo trì đều trở nên dễ dàng. Tôi từng trải qua những dự án chỉ tốn một phần nhỏ nhân lực so với dự kiến. Tôi từng làm trên những hệ thống có tỷ lệ lỗi cực kỳ thấp. Tôi đã thấy tác động phi thường mà một kiến trúc phần mềm tốt có thể mang lại cho một hệ thống, một dự án và một đội ngũ. Tôi đã đặt chân tới miền đất hứa.

But don't take my word for it. Look at your own experience. Have you experienced the opposite? Have you worked on systems that are so interconnected and intricately coupled that every change, regardless of how trivial, takes weeks and involves huge risks? Have you experienced the impedance of bad code and rotten design? Has the design of the systems you've worked on had a huge negative effect on the morale of the team, the trust of the customers, and the patience of the managers? Have you seen teams, departments, and even companies that have been brought down by the rotten **structure** of their software? Have you been to programming hell?

Nhưng đừng vội tin lời tôi. Hãy nhìn vào trải nghiệm của chính bạn. Bạn đã từng nếm trải điều ngược lại chưa? Bạn đã từng làm trên những hệ thống chằng chịt và kết dính rối rắm tới mức mọi thay đổi, dù nhỏ nhất đến đâu, cũng tốn hàng tuần trời và kéo theo rủi ro khổng lồ? Bạn đã từng chịu sức cản của mã tồi và thiết kế mục ruỗng? Thiết kế của những hệ thống bạn từng làm đã bao giờ giáng một đòn nặng nề lên tinh thần của đội, lòng tin của khách hàng và sự kiên nhẫn của các nhà quản lý chưa? Bạn đã thấy những đội, những phòng ban, thậm chí cả những công ty bị hạ gục bởi cấu trúc mục ruỗng trong phần mềm của họ chưa? Bạn đã tới địa ngục lập trình bao giờ chưa?

I have—and to some extent, most of the rest of us have, too. It is far more common to fight your way through terrible software designs than it is to enjoy the pleasure of working with a good one.

Tôi thì rồi — và ở mức độ nào đó, hầu hết chúng ta cũng vậy. Chuyện phải vật lộn xoay sở giữa những thiết kế phần mềm tệ hại còn phổ biến hơn nhiều so với niềm vui được làm việc với một thiết kế tốt.

What Is Design and Architecture? — Thiết kế và kiến trúc là gì?

There has been a lot of confusion about **design** and **architecture** over the years. What is design? What is architecture? What are the differences between the two?

Suốt bao nhiêu năm qua, thiết kế (design) và kiến trúc (architecture) là hai chuyện gây lẫn lộn không ít. Thiết kế là gì? Kiến trúc là gì? Khác nhau ở chỗ nào?

One of the goals of this book is to cut through all that confusion and to define, once and for all, what design and architecture are. For starters, I'll assert that there is no difference between them. None at all.

Một trong những mục tiêu của cuốn sách này là dẹp bỏ mớ lẫn lộn đó và định nghĩa dứt điểm, một lần cho tất cả, thiết kế và kiến trúc là gì. Để mở màn, tôi khẳng định luôn: giữa chúng không có khác biệt nào. Không hề.

The word “architecture” is often used in the context of something at a high **level** that is divorced from the lower-level details, whereas “design” more often seems to imply structures and decisions at a lower level. But this usage is nonsensical when you look at what a real architect does.

Người ta thường dùng từ “kiến trúc” cho những thứ ở mức cao, tách rời khỏi các chi tiết ở mức thấp; còn “thiết kế” thì hay ngụ ý những cấu trúc và quyết định ở mức thấp hơn. Nhưng cách dùng ấy trở nên vô nghĩa khi bạn nhìn vào công việc thực sự của một kiến trúc sư.

Consider the architect who designed my new home. Does this home have an architecture? Of course it does. And what is that architecture? Well, it is the shape of the home, the outward appearance, the elevations, and the layout of the spaces and rooms. But as I look through the diagrams that my architect produced, I see an immense number of **low-level** details. I see where every outlet, light switch, and light will be placed. I see which switches control which lights. I see where the furnace is placed, and the size and placement of the water heater and the sump pump. I see detailed depictions of how the walls, roofs, and foundations will be constructed.

Hãy nghĩ về vị kiến trúc sư đã thiết kế ngôi nhà mới của tôi. Ngôi nhà đó có kiến trúc không? Tất nhiên là có. Và kiến trúc đó là gì? Ừ thì, đó là hình khối của ngôi nhà, dáng vẻ bên ngoài, các mặt đứng, cách bố trí không gian và các phòng. Nhưng khi lật giở đồ án vẽ ông ấy làm ra, tôi thấy vô số chi tiết ở mức thấp. Tôi thấy từng ổ cắm, từng công tắc, từng bóng đèn sẽ đặt ở đâu. Tôi thấy công tắc nào điều khiển đèn nào. Tôi thấy lò sưởi đặt chỗ nào, bình nước nóng và bơm hút nước hồ thu (sump pump) to cỡ nào, đặt ở đâu. Tôi thấy cả những mô tả chi tiết về cách thi công tường, mái và móng.

In short, I see all the little details that support all the **high-level** decisions. I also see that those low-level details and high-level decisions are part of the whole design of the house.

Nói ngắn gọn, tôi thấy tất cả những chi tiết nhỏ nhặt chống đỡ cho mọi quyết định ở mức cao. Tôi cũng thấy rằng những chi tiết mức thấp ấy và những quyết định mức cao kia đều là một phần của tổng thể thiết kế ngôi nhà.

And so it is with software design. The low-level details and the high-level **structure** are all part of the same whole. They form a continuous fabric that defines the shape of the system. You can't have one without the other; indeed, no clear dividing line separates them. There is simply a continuum of decisions from the highest to the lowest levels.

Thiết kế phần mềm cũng vậy. Các chi tiết mức thấp và cấu trúc mức cao đều thuộc về cùng một chỉnh thể. Chúng tạo thành một tấm vải liền mạch định hình hệ thống. Bạn không thể có cái này mà thiếu cái kia; thật ra chẳng có lằn ranh rõ ràng nào chia tách chúng cả. Chỉ đơn giản là một dải liên tục các quyết định, trải từ mức cao nhất xuống mức thấp nhất.

The Goal? — Mục tiêu?

And the goal of those decisions? The goal of good software design? That goal is nothing less than my utopian description:

Còn mục tiêu của những quyết định ấy? Mục tiêu của một thiết kế phần mềm tốt? Mục tiêu đó chẳng gì khác hơn chính là mô tả không tưởng của tôi:

The goal of software architecture is to minimize the **human resources** required to build and maintain the required system.

Mục tiêu của kiến trúc phần mềm là giảm thiểu nhân lực cần thiết để xây dựng và bảo trì hệ thống được yêu cầu.

The measure of design quality is simply the measure of the **effort** required to meet the needs of the customer. If that effort is low, and stays low throughout the lifetime of the system, the design is good. If that effort grows with each new release, the design is bad. It's as simple as that.

Thước đo chất lượng thiết kế đơn giản chỉ là thước đo công sức phải bỏ ra để đáp ứng nhu cầu của khách hàng. Nếu công sức ấy thấp, và giữ được mức thấp suốt vòng đời hệ thống, thì thiết kế là tốt. Nếu công sức ấy phình lên sau mỗi bản phát hành, thiết kế là tồi. Đơn giản vậy thôi.

Case Study — Nghiên cứu tình huống

As an example, consider the following case study. It includes real data from a real company that wishes to remain anonymous.

Lấy ví dụ, hãy xem xét tình huống sau đây. Nó dùng dữ liệu thật từ một công ty thật, công ty này muốn được giấu tên.

First, let's look at the growth of the engineering staff. I'm sure you'll agree that this trend is very encouraging. Growth like that shown in Figure 1.1 must be an indication of significant success!

Trước hết, hãy nhìn vào mức tăng trưởng của đội ngũ kỹ sư. Tôi chắc bạn sẽ đồng ý rằng xu hướng này rất đáng phấn khởi. Đà tăng trưởng như trong Figure 1.1 hẳn phải là dấu hiệu của một thành công lớn!

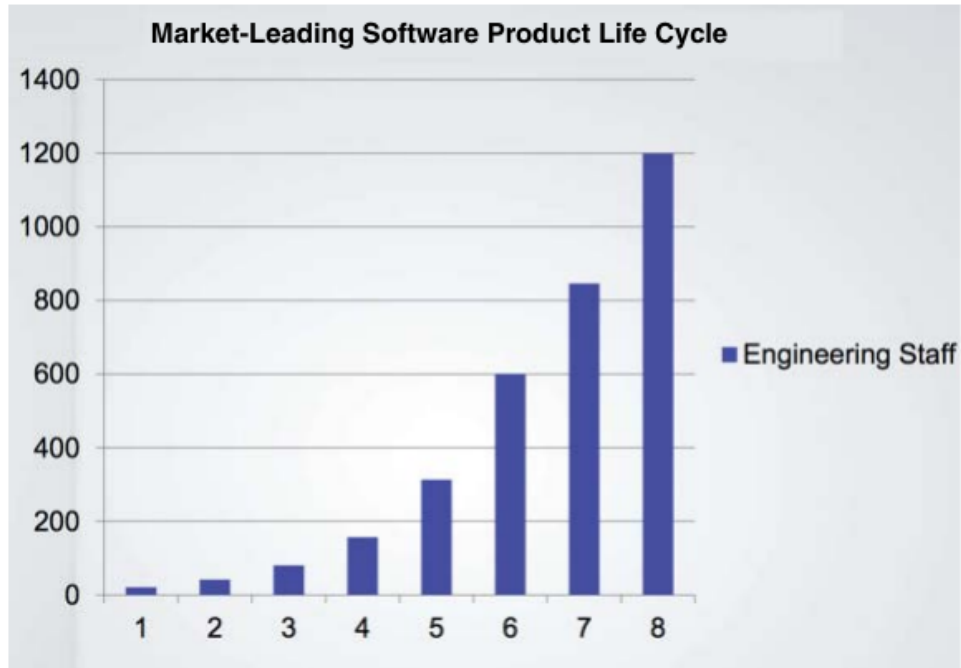


Figure 1.1 Growth of the engineering staff

Figure 1.1 Mức tăng trưởng của đội ngũ kỹ sư

Reproduced with permission from a slide presentation by Jason Gorman

Sử dụng lại với sự cho phép, từ một bài trình chiếu của Jason Gorman

Now let's look at the company's productivity over the same time period, as measured by simple lines of code (Figure 1.2).

Bây giờ hãy nhìn vào năng suất của công ty trong cùng khoảng thời gian đó, đo bằng số dòng mã đơn thuần (Figure 1.2).

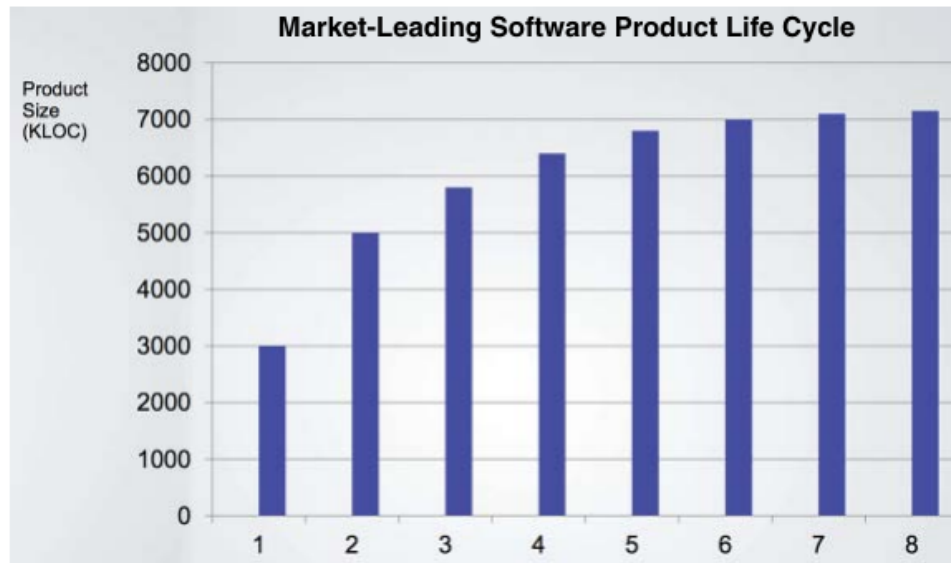


Figure 1.2 Productivity over the same period of time

Figure 1.2 Năng suất trong cùng khoảng thời gian

Clearly something is going wrong here. Even though every release is supported by an ever-increasing number of developers, the growth of the code looks like it is approaching an asymptote.

Rõ ràng có gì đó đang trục trặc. Dù mỗi bản phát hành đều được hậu thuẫn bởi số lập trình viên ngày một đông hơn, đà tăng của lượng mã trông cứ như đang tiến dần tới một đường tiệm cận.

Now here's the really scary graph: Figure 1.3 shows how the cost per line of code has changed over time.

Còn đây mới là biểu đồ thực sự đáng sợ: Figure 1.3 cho thấy chi phí trên mỗi dòng mã đã thay đổi ra sao theo thời gian.

These trends aren't sustainable. It doesn't matter how profitable the company might be at the moment: Those curves will catastrophically drain the profit from the business model and drive the company into a stall, if not into a downright collapse.

Những xu hướng này không thể duy trì được. Công ty lúc này lời lãi đến đâu cũng chẳng còn quan trọng: các đường cong kia sẽ vắt cạn lợi nhuận khỏi mô hình kinh doanh một cách thảm khốc và đẩy công ty vào cảnh chết đứng, nếu không muốn nói là sụp đổ hoàn toàn.

What caused this remarkable change in productivity? Why was the code 40 times more expensive to produce in release 8 as opposed to release 1?

Điều gì đã gây ra cú thay đổi năng suất kinh hoàng đến thế? Tại sao viết mã ở bản phát hành 8 lại đắt gấp 40 lần so với bản phát hành 1?

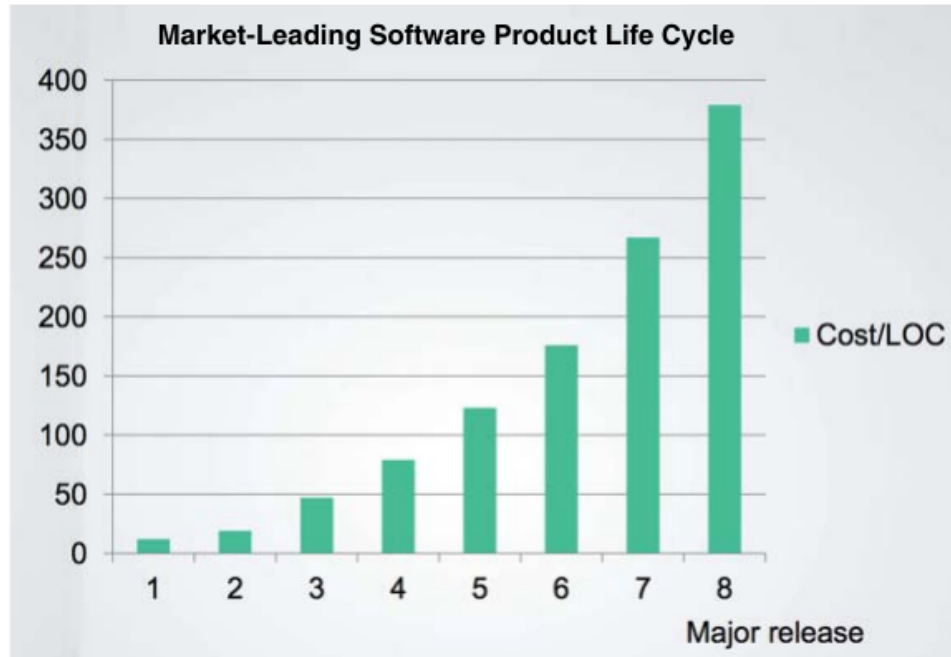


Figure 1.3 Cost per line of code over time

Figure 1.3 Chi phí trên mỗi dòng mã theo thời gian

The Signature of a Mess — Dấu hiệu nhận dạng của một mớ hỗn độn

What you are looking at is the signature of a **mess**. When systems are thrown together in a hurry, when the sheer number of programmers is the sole driver of output, and when little or no thought is given to the cleanliness of the code or the structure of the design, then you can bank on riding this curve to its ugly end.

Cái bạn đang nhìn thấy chính là dấu hiệu nhận dạng của một mớ hỗn độn. Khi các hệ thống bị chấp vá lại trong vội vã, khi số đầu người lập trình viên là động lực duy nhất tạo ra sản lượng, và khi người ta gần như chẳng bận tâm gì tới sự sạch sẽ của mã hay cấu trúc của thiết kế, thì bạn có thể đặt cược rằng mình sẽ cưỡi đường cong này tới cái kết xấu xí của nó.

Figure 1.4 shows what this curve looks like to the developers. They started out at nearly 100% productivity, but with each release their productivity declined. By the fourth release, it was clear that their productivity was going to bottom out in an asymptotic approach to zero.

Figure 1.4 cho thấy đường cong ấy trông thế nào dưới mắt các lập trình viên. Họ khởi đầu với năng suất gần 100%, nhưng sau mỗi bản phát hành, năng suất lại tụt xuống. Đến bản phát hành thứ tư, đã thấy rõ năng suất của họ sẽ chậm dần theo kiểu tiệm cận về không.

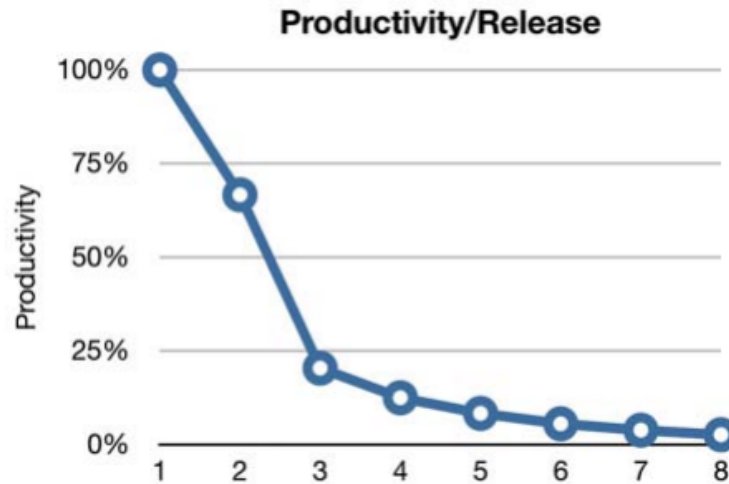


Figure 1.4 Productivity by release

Figure 1.4 Năng suất theo từng bản phát hành

From the developers' point of view, this is tremendously frustrating, because everyone is working hard . Nobody has decreased their effort.

Dưới góc nhìn của lập trình viên, chuyện này bức bối kinh khủng, bởi ai cũng đang làm việc cật lực. Chẳng ai giảm bớt công sức cả.

And yet, despite all their heroics, overtime, and dedication, they simply aren't getting much of anything done anymore. All their effort has been diverted away from features and is now consumed with managing the mess. Their job, such as it is, has changed into moving the mess from one place to the next, and the next, and the next, so that they can add one more meager little feature.

Vậy mà bất chấp mọi nỗ lực phi thường, mọi giờ tăng ca và mọi sự tận tụy, họ đơn giản là chẳng còn làm xong được bao nhiêu nữa. Toàn bộ công sức của họ đã bị rút khỏi việc làm tính năng và giờ bị ngốn sạch vào việc xoay xở với mớ hỗn độn. Công việc của họ, nếu còn gọi được là công việc, đã biến thành chuyển mớ hỗn độn từ chỗ này sang chỗ khác, rồi chỗ khác nữa, rồi chỗ khác nữa, chỉ để nhét thêm được một tính năng cỏn con.

The Executive View — Dưới mắt ban lãnh đạo

If you think that's bad, imagine what this picture looks like to the executives! Consider Figure 1.5, which depicts monthly development payroll for the same period.

Nếu bạn thấy thế đã tệ, hãy tưởng tượng bức tranh này trông ra sao dưới mắt ban lãnh đạo! Hãy xem Figure 1.5, biểu diễn quỹ lương phát triển phần mềm hằng tháng trong cùng giai đoạn.

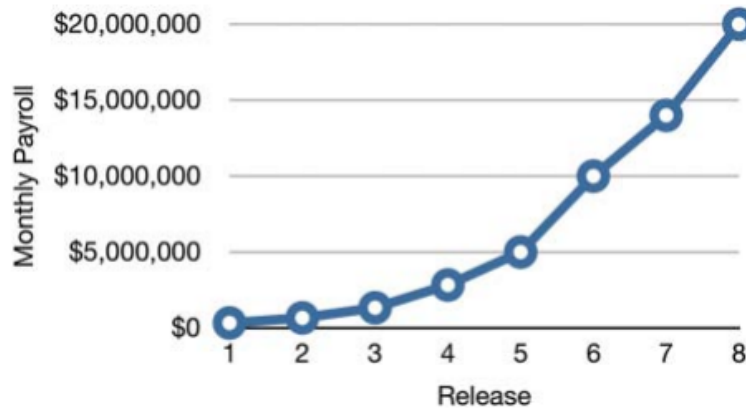


Figure 1.5 Monthly development payroll by release

Figure 1.5 Quỹ lương phát triển hàng tháng theo từng bản phát hành

Release 1 was delivered with a monthly payroll of a few hundred thousand dollars. The second release cost a few hundred thousand more. By the eighth release monthly payroll was \$20 million, and climbing.

Bản phát hành 1 được giao với quỹ lương hàng tháng vài trăm nghìn đô la. Bản thứ hai tốn thêm vài trăm nghìn nữa. Đến bản phát hành thứ tám, quỹ lương hàng tháng đã là 20 triệu đô la, và vẫn đang leo.

Just this chart alone is scary. Clearly something startling is happening. One hopes that revenues are outpacing costs and therefore justifying the expense. But no matter how you look at this curve, it's cause for concern.

Chỉ riêng biểu đồ này thôi đã đáng sợ. Rõ ràng có chuyện gì đó giật mình đang diễn ra. Ta chỉ có thể hy vọng doanh thu tăng nhanh hơn chi phí và do đó biện minh được cho khoản chi này. Nhưng nhìn đường cong ấy kiểu gì thì nó cũng đáng lo.

But now compare the curve in Figure 1.5 with the lines of code written per release in Figure 1.2. That initial few hundred thousand dollars per month bought a lot of functionality—but the final \$20 million bought almost nothing! Any CFO would look at these two graphs and know that immediate action is necessary to stave off disaster.

Giờ hãy so đường cong ở Figure 1.5 với số dòng mã viết ra mỗi bản phát hành ở Figure 1.2. Vài trăm nghìn đô mỗi tháng lúc đầu mua về được cả đống chức năng — còn 20 triệu đô cuối cùng thì gần như chẳng mua được gì! Bất kỳ CFO nào nhìn hai biểu đồ này cũng biết phải hành động ngay lập tức để chặn thảm họa.

But which action can be taken? What has gone wrong? What has caused this incredible decline in productivity? What can executives do, other than to stamp their feet and rage at the developers?

Nhưng hành động nào đây? Đã sai ở đâu? Điều gì gây ra cú sụt năng suất khó tin đến vậy? Ban lãnh đạo có thể làm gì, ngoài giậm chân đùng đùng và trút giận lên các lập trình viên?

What Went Wrong? — Đã sai ở đâu?

Nearly 2600 years ago, Aesop told the story of the Tortoise and the Hare. The moral of that story has been stated many times in many different ways:

Gần 2600 năm trước, Aesop kể câu chuyện Rùa và Thỏ. Bài học của câu chuyện ấy đã được phát biểu nhiều lần, theo nhiều cách khác nhau:

- “Slow and steady wins the race.”
 - “Chậm mà chắc thì thắng cuộc đua.”
- “The race is not to the swift, nor the battle to the strong.”
 - “Cuộc đua không dành cho kẻ nhanh, trận chiến không thuộc về kẻ mạnh.”
- “The more haste, the less speed.”
 - “Càng vội thì càng chậm.”

The story itself illustrates the foolishness of overconfidence. The Hare, so confident in its intrinsic speed, does not take the race seriously, and so naps while the Tortoise crosses the finish line.

Bản thân câu chuyện minh họa sự ngớ ngẩn của thói tự tin thái quá. Thỏ, quá tin vào tốc độ trời cho của mình, không coi cuộc đua ra gì, thế là lăn ra ngủ trong khi Rùa băng qua vạch đích.

Modern developers are in a similar race, and exhibit a similar overconfidence. Oh, they don’t sleep—far from it. Most modern developers work their butts off. But a part of their brain does sleep—the part that knows that good, clean, well-designed code matters .

Lập trình viên thời nay cũng đang ở trong một cuộc đua tương tự, và cũng bộc lộ thói tự tin thái quá tương tự. Ồ, họ không ngủ đâu — còn lâu mới ngủ. Phần lớn lập trình viên hiện đại làm việc bỏ hơi tai. Nhưng có một phần trong não họ thì ngủ thật — cái phần biết rằng mã tốt, sạch, được thiết kế đẳng hoàng là điều quan trọng.

These developers buy into a familiar lie: “We can clean it up later; we just have to get to market first!” Of course, things never do get cleaned up later, because market pressures never abate. Getting to market first simply means that you’ve now got a horde of competitors on your tail, and you have to stay ahead of them by running as fast as you can.

Những lập trình viên ấy nuốt trọn một lời dối trá quen thuộc: “Dọn sau cũng được; giờ phải ra thị trường trước đã!” Tất nhiên, chẳng bao giờ có chuyện dọn dẹp sau đó, vì áp lực thị trường có bao giờ dịu đi đâu. Ra thị trường trước chỉ đơn giản có nghĩa là giờ bạn có cả một bầy đối thủ bám sát sau lưng, và bạn phải chạy hết sức để giữ khoảng cách với họ.

And so the developers never switch modes. They can’t go back and clean things up because they’ve got to get the next feature done, and the next, and the next, and the next. And so the mess builds, and productivity continues its asymptotic approach toward zero.

Thế là các lập trình viên chẳng bao giờ chuyển chế độ. Họ không thể quay lại dọn dẹp, vì còn phải làm cho xong tính năng tiếp theo, rồi tính năng tiếp theo nữa, rồi tiếp nữa, rồi tiếp nữa. Và thế là mớ hỗn độn cứ chất chồng, còn năng suất thì tiếp tục tiệm cận về không.

Just as the Hare was overconfident in its speed, so the developers are overconfident in their ability to remain productive. But the creeping mess of code that saps their productivity never sleeps and never relents. If given its way, it will reduce productivity to zero in a matter of months.

Cũng như Thỏ tự tin thái quá vào tốc độ của mình, các lập trình viên tự tin thái quá vào khả năng duy trì năng suất của họ. Nhưng mớ mã hỗn độn đang bò lan và hút cạn năng suất của họ thì chẳng bao giờ ngủ và chẳng bao giờ nương tay. Nếu cứ để mặc, chỉ trong vài tháng nó sẽ kéo năng suất về không.

The bigger lie that developers buy into is the notion that writing messy code makes them go fast in the short term, and just slows them down in the long term. Developers who accept this lie exhibit the hare's overconfidence in their ability to switch modes from making messes to cleaning up messes sometime in the future, but they also make a simple error of fact. The fact is that making messes is always slower than staying clean, no matter which time scale you are using.

Lời dối trá lớn hơn mà các lập trình viên nuốt trọn là quan niệm rằng viết mã bừa bãi giúp họ đi nhanh trong ngắn hạn, và chỉ làm họ chậm lại trong dài hạn. Ai chấp nhận lời dối trá này đều đang bộc lộ đúng thói tự tin thái quá của Thỏ về khả năng chuyển từ chế độ bày ra mớ hỗn độn sang chế độ dọn dẹp mớ hỗn độn vào lúc nào đó trong tương lai; nhưng họ còn phạm một sai lầm đơn giản về mặt sự thật nữa. Sự thật là bày ra mớ hỗn độn luôn luôn chậm hơn giữ cho mọi thứ sạch sẽ, bất kể bạn xét trên thang thời gian nào.

Consider the results of a remarkable experiment performed by Jason Gorman depicted in Figure 1.6. Jason conducted this test over a period of six days. Each day he completed a simple program to convert integers into Roman numerals. He knew his work was complete when his predefined set of acceptance tests passed. Each day the task took a little less than 30 minutes. Jason used a well-known cleanliness discipline named test-driven development (TDD) on the first, third, and fifth days. On the other three days, he wrote the code without that discipline.

Hãy xem kết quả một thí nghiệm đáng chú ý do Jason Gorman thực hiện, mô tả trong Figure 1.6. Jason tiến hành thử nghiệm này trong sáu ngày. Mỗi ngày anh hoàn thành một chương trình đơn giản chuyển số nguyên thành số La Mã. Anh biết công việc đã xong khi bộ acceptance test định sẵn của anh chạy qua hết. Mỗi ngày công việc ngắn chưa đầy 30 phút. Vào ngày thứ nhất, thứ ba và thứ năm, Jason áp dụng một kỷ luật giữ mã sạch nổi tiếng mang tên test-driven development (TDD). Ba ngày còn lại, anh viết mã mà không dùng kỷ luật đó.

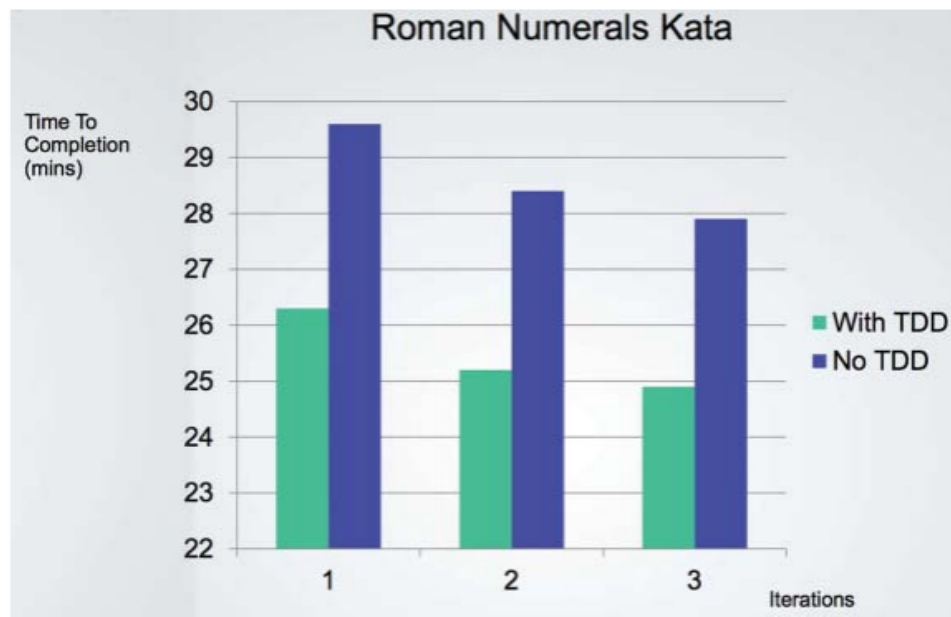


Figure 1.6 Time to completion by iterations and use/non-use of TDD

Figure 1.6 Thời gian hoàn thành theo từng lượt lặp, có và không dùng TDD

First, notice the learning curve apparent in Figure 1.6. Work on the latter days is completed more

quickly than the former days. Notice also that work on the TDD days proceeded approximately 10% faster than work on the non-TDD days, and that even the slowest TDD day was faster than the fastest non-TDD day.

Trước hết, hãy để ý đường cong học tập hiện rõ trong Figure 1.6. Công việc ở những ngày sau được hoàn thành nhanh hơn những ngày trước. Cũng hãy để ý rằng công việc trong các ngày dùng TDD diễn ra nhanh hơn khoảng 10% so với các ngày không dùng TDD, và rằng ngay cả ngày TDD chậm nhất cũng vẫn nhanh hơn ngày không TDD nhanh nhất.

Some folks might look at that result and think it's a remarkable outcome. But to those who haven't been deluded by the Hare's overconfidence, the result is expected, because they know this simple truth of software development:

Có người nhìn kết quả đó sẽ nghĩ đây là một phát hiện phi thường. Nhưng với những ai chưa bị thói tự tin thái quá của Thỏ làm cho mù mịt, kết quả này là điều dĩ nhiên, bởi họ biết một sự thật giản dị trong phát triển phần mềm:

The only way to go fast, is to go well.

Cách duy nhất để đi nhanh là đi cho chuẩn.

And that's the answer to the executive's dilemma. The only way to reverse the decline in productivity and the increase in cost is to get the developers to stop thinking like the overconfident Hare and start taking responsibility for the mess that they've made.

Và đó chính là lời giải cho thế lưỡng nan của ban lãnh đạo. Cách duy nhất để đảo ngược đà tụt năng suất và đà tăng chi phí là buộc các lập trình viên thôi nghĩ như chú Thỏ tự tin thái quá, và bắt đầu chịu trách nhiệm về mớ hỗn độn do chính họ bày ra.

The developers may think that the answer is to start over from scratch and redesign the whole system—but that's just the Hare talking again. The same overconfidence that led to the mess is now telling them that they can build it better if only they can start the race over. The reality is less rosy:

Các lập trình viên có thể nghĩ rằng câu trả lời là làm lại từ đầu và thiết kế lại toàn bộ hệ thống — nhưng đó lại chính là chú Thỏ đang lên tiếng. Đúng cái thói tự tin thái quá đã dẫn tới mớ hỗn độn giờ đang nói với họ rằng họ sẽ dựng được thứ tốt hơn, miễn là được chạy lại cuộc đua từ đầu. Thực tế thì kém tươi sáng hơn nhiều:

Their overconfidence will drive the redesign into the same mess as the original project.

Thói tự tin thái quá của họ sẽ đẩy bản thiết kế lại rơi vào đúng mớ hỗn độn như dự án ban đầu.

Conclusion — Kết luận

In every case, the best option is for the development organization to recognize and avoid its own overconfidence and to start taking the quality of its software architecture seriously.

Trong mọi trường hợp, phương án tốt nhất là tổ chức phát triển phần mềm phải nhận ra và tránh được thói tự tin thái quá của chính mình, rồi bắt đầu xem trọng chất lượng kiến trúc phần mềm của họ.

To take software architecture seriously, you need to know what good software architecture is. To build a system with a design and an architecture that minimize effort and maximize productivity, you need to know which attributes of system architecture lead to that end.

Muốn xem trọng kiến trúc phần mềm, bạn cần biết một kiến trúc phần mềm tốt là như thế nào. Muốn dựng một hệ thống có thiết kế và kiến trúc giúp giảm thiểu công sức và tối đa hóa năng suất, bạn cần biết những thuộc tính nào của kiến trúc hệ thống dẫn tới đích đó.

That's what this book is about. It describes what good clean architectures and designs look like, so that software developers can build systems that will have long profitable lifetimes.

Đó chính là điều cuốn sách này bàn tới. Nó mô tả những kiến trúc và thiết kế tốt, sạch trông ra sao, để các lập trình viên có thể dựng nên những hệ thống có vòng đời dài lâu và sinh lợi.

A Tale of Two Values — Câu chuyện về hai giá trị

Every software system provides two different values to the stakeholders: **behavior** and **structure**. Software developers are responsible for ensuring that both those values remain high. Unfortunately, they often focus on one to the exclusion of the other. Even more unfortunately, they often focus on the lesser of the two values, leaving the software system eventually valueless.

Mọi hệ thống phần mềm đều mang lại hai giá trị khác nhau cho các bên liên quan (stakeholder): hành vi (behavior) và cấu trúc (structure). Lập trình viên có trách nhiệm giữ cho cả hai giá trị đó luôn ở mức cao. Đáng tiếc là họ thường chỉ chăm chăm vào một giá trị và bỏ mặc giá trị còn lại. Đáng tiếc hơn nữa, họ thường chăm chăm vào đúng cái giá trị nhỏ hơn trong hai giá trị đó, để rồi rốt cuộc hệ thống phần mềm chẳng còn giá trị gì.

Behavior — Hành vi

The first value of software is its behavior. Programmers are hired to make machines behave in a way that makes or saves money for the stakeholders. We do this by helping the stakeholders develop a functional specification, or requirements document. Then we write the code that causes the **stakeholder's** machines to satisfy those requirements.

Giá trị thứ nhất của phần mềm là hành vi của nó. Lập trình viên được thuê để làm cho máy móc vận hành theo cách kiếm ra tiền hoặc tiết kiệm tiền cho các bên liên quan. Chúng ta làm điều đó bằng cách giúp họ soạn ra bản đặc tả chức năng, hay tài liệu yêu cầu. Sau đó chúng ta viết mã để máy móc của họ thoả mãn những yêu cầu ấy.

When the machine violates those requirements, programmers get their debuggers out and fix the problem.

Khi máy chạy sai so với các yêu cầu đó, lập trình viên lôi debugger ra và sửa.

Many programmers believe that is the entirety of their job. They believe their job is to make the machine implement the requirements and to fix any bugs. They are sadly mistaken.

Nhiều lập trình viên tin rằng đó là toàn bộ công việc của mình. Họ tin nhiệm vụ của mình là bắt máy hiện thực đúng các yêu cầu và sửa mọi lỗi. Buồn thay, họ đã lầm.

Architecture — Kiến trúc

The second value of software has to do with the word “software”—a compound word composed of “soft” and “ware.” The word “ware” means “product”; the word “soft”... Well, that’s where the second value lies.

Giá trị thứ hai của phần mềm nằm ngay trong chữ “software” — một từ ghép của “soft” và “ware”. Chữ “ware” nghĩa là “sản phẩm”; còn chữ “soft”... À, giá trị thứ hai nằm ở đó.

Software was invented to be “soft.” It was intended to be a way to easily change the behavior of machines. If we’d wanted the behavior of machines to be hard to change, we would have called it hard ware.

Phần mềm được phát minh ra để “soft” — để mềm. Nó ra đời như một cách để thay đổi hành vi của máy móc một cách dễ dàng. Nếu chúng ta muốn hành vi của máy móc khó thay đổi, hẳn chúng ta đã gọi nó là hard ware.

To fulfill its purpose, software must be soft—that is, it must be easy to change. When the stakeholders change their minds about a feature, that change should be simple and easy to make. The difficulty in making such a change should be

Để làm tròn mục đích của mình, phần mềm phải mềm — nghĩa là phải dễ thay đổi. Khi các bên liên quan đổi ý về một tính năng nào đó, việc thay đổi phải đơn giản và dễ dàng. Độ khó của một thay đổi như vậy chỉ nên

proportional only to the scope of the change, and not to the shape of the change.

tỉ lệ thuận với phạm vi (scope) của thay đổi, chứ không phải với hình dạng (shape) của nó.

It is this difference between scope and shape that often drives the growth in software development costs. It is the reason that costs grow out of proportion to the size of the requested changes. It is the reason that the first year of development is much cheaper than the second, and the second year is much cheaper than the third.

Chính khoảng cách giữa phạm vi và hình dạng này thường là thứ đẩy chi phí phát triển phần mềm phình lên. Nó là lý do khiến chi phí tăng không tương xứng với kích thước của yêu cầu thay đổi. Nó là lý do khiến năm phát triển đầu tiên rẻ hơn nhiều so với năm thứ hai, và năm thứ hai lại rẻ hơn nhiều so với năm thứ ba.

From the stakeholders’ point of view, they are simply providing a stream of changes of roughly similar scope. From the developers’ point of view, the stakeholders are giving them a stream of jigsaw puzzle pieces that they must fit into a puzzle of ever-increasing complexity. Each new **request** is harder to fit than the last, because the shape of the system does not match the shape of the request.

Dưới góc nhìn của các bên liên quan, họ chỉ đơn giản đưa ra một dòng chảy các thay đổi có phạm vi đại khái ngang nhau. Dưới góc nhìn của lập trình viên, các bên liên quan đang ném cho họ một dòng chảy những mảnh ghép hình mà họ phải nhét vừa vào một bức tranh ngày càng rối rắm. Mỗi yêu cầu mới lại khó nhét vừa hơn yêu cầu trước, bởi hình dạng của hệ thống không khớp với hình dạng của yêu cầu.

I’m using the word “shape” here in a unconventional way, but I think the metaphor is apt. Software developers often feel as if they are forced to jam square pegs into round holes.

Tôi đang dùng chữ “hình dạng” theo một nghĩa không mấy thông thường, nhưng tôi cho rằng phép ẩn dụ này rất đắt. Lập trình viên thường có cảm giác mình bị buộc phải nhét cọc vuông vào lỗ tròn.

The problem, of course, is the **architecture** of the system. The more this architecture prefers one shape over another, the more likely new features will be harder and harder to fit into that structure. Therefore architectures should be as shape agnostic as practical.

Vấn đề, dĩ nhiên, nằm ở kiến trúc (architecture) của hệ thống. Kiến trúc càng thiên vị một hình dạng nào đó, thì các tính năng mới càng có nguy cơ ngày một khó nhét vừa vào cấu trúc ấy. Bởi vậy, kiến trúc nên trung lập với hình dạng đến mức khả thi nhất.

The Greater Value — Giá trị nào lớn hơn

Function or architecture? Which of these two provides the greater value? Is it more **important** for the software system to work, or is it more important for the software system to be easy to change?

Chức năng hay kiến trúc? Cái nào mang lại giá trị lớn hơn? Hệ thống phần mềm chạy được thì quan trọng hơn, hay hệ thống phần mềm dễ thay đổi thì quan trọng hơn?

If you ask the business managers, they'll often say that it's more important for the software system to work. Developers, in turn, often go along with this attitude. But it's the wrong attitude. I can prove that it is wrong with the simple logical tool of examining the extremes.

Nếu bạn hỏi các nhà quản lý nghiệp vụ, họ thường bảo rằng chuyện hệ thống chạy được mới là quan trọng hơn. Rồi lập trình viên cũng hòa theo thái độ đó. Nhưng đó là một thái độ sai. Tôi có thể chứng minh nó sai bằng một công cụ logic đơn giản: xét các thái cực.

- If you give me a program that works perfectly but is impossible to change,
 - Nếu bạn đưa tôi một chương trình chạy hoàn hảo nhưng không thể thay đổi được,

then it won't work when the requirements change, and I won't be able to make it work. Therefore the program will become useless.

thì nó sẽ không còn chạy đúng khi yêu cầu thay đổi, và tôi cũng chẳng thể làm gì để nó chạy đúng trở lại. Vậy nên chương trình đó sẽ trở nên vô dụng.

- If you give me a program that does not work but is easy to change, then I
 - Nếu bạn đưa tôi một chương trình không chạy được nhưng dễ thay đổi, thì tôi

can make it work, and keep it working as requirements change. Therefore the program will remain continually useful.

có thể làm cho nó chạy được, và giữ cho nó chạy được khi yêu cầu thay đổi. Vậy nên chương trình đó sẽ hữu ích lâu dài.

You may not find this argument convincing. After all, there's no such thing as a program that is impossible to change. However, there are systems that are practically impossible to change, because the cost of change exceeds the benefit of change. Many systems reach that point in some of their features or configurations.

Có thể bạn thấy lập luận này chưa thuyết phục. Suy cho cùng, làm gì có chương trình nào là không thể thay đổi. Tuy nhiên, có những hệ thống mà trên thực tế là không thể thay đổi được, bởi chi phí thay đổi vượt quá lợi ích thu về. Nhiều hệ thống chạm tới ngưỡng đó ở một số tính năng hoặc một số cấu hình của chúng.

If you ask the business managers if they want to be able to make changes, they'll say that of course they do, but may then qualify their answer by noting that the current functionality is more important than any later flexibility. In contrast, if the business managers ask you for a change, and

your estimated costs for that change are unaffordably high, the business managers will likely be furious that you allowed the system to get to the point where the change was impractical.

Nếu bạn hỏi các nhà quản lý nghiệp vụ rằng họ có muốn giữ khả năng thay đổi hay không, họ sẽ nói dĩ nhiên là có, nhưng rồi rào trước đón sau rằng chức năng hiện tại quan trọng hơn bất kỳ sự linh hoạt nào sau này. Ngược lại, nếu các nhà quản lý nghiệp vụ yêu cầu bạn thực hiện một thay đổi, và bạn ước lượng chi phí cho thay đổi đó cao đến mức không kham nổi, họ sẽ nổi giận đùng đùng vì bạn đã để hệ thống trôi tới mức mà thay đổi trở nên bất khả thi.

Eisenhower's Matrix — Ma trận Eisenhower

Consider President Dwight D. Eisenhower's matrix of importance versus urgency (Figure 2.1). Of this matrix, Eisenhower said:

Hãy xét ma trận quan trọng — khẩn cấp của Tổng thống Dwight D. Eisenhower (Figure 2.1). Về ma trận này, Eisenhower nói:

I have two kinds of problems, the **urgent** and the important. The urgent are not important, and the important are never urgent. 1

Tôi có hai loại vấn đề: khẩn cấp và quan trọng. Việc khẩn cấp thì không quan trọng, còn việc quan trọng thì chẳng bao giờ khẩn cấp. 1

IMPORTANT URGENT	IMPORTANT NOT URGENT	
UNIMPORTANT URGENT	UNIMPORTANT NOT URGENT	

Figure 2.1 Eisenhower matrix

Figure 2.1 Ma trận Eisenhower

- 1. From a speech at Northwestern University in 1954.
 - 1. Trích từ một bài phát biểu tại Đại học Northwestern năm 1954.

There is a great deal of truth to this old adage. Those things that are urgent are rarely of great importance, and those things that are important are seldom of great urgency.

Có rất nhiều sự thật trong câu châm ngôn xưa cũ này. Những việc khẩn cấp hiếm khi thực sự quan trọng, còn những việc quan trọng thì ít khi thực sự khẩn cấp.

The first value of software—behavior—is urgent but not always particularly important.

Giá trị thứ nhất của phần mềm — hành vi — là khẩn cấp nhưng không phải lúc nào cũng đặc biệt quan trọng.

The second value of software—architecture—is important but never particularly urgent.

Giá trị thứ hai của phần mềm — kiến trúc — là quan trọng nhưng chẳng bao giờ đặc biệt khẩn cấp.

Of course, some things are both urgent and important. Other things are not urgent and not important. Ultimately, we can arrange these four couplets into priorities:

Dĩ nhiên, có những việc vừa khẩn cấp vừa quan trọng. Lại có những việc chẳng khẩn cấp cũng chẳng quan trọng. Rốt cuộc, ta có thể xếp bốn cặp này thành thứ tự ưu tiên:

- 1. Urgent and important
 - 1. Khẩn cấp và quan trọng
- 2. Not urgent and important
 - 2. Không khẩn cấp và quan trọng
- 3. Urgent and not important
 - 3. Khẩn cấp và không quan trọng
- 4. Not urgent and not important
 - 4. Không khẩn cấp và không quan trọng

Note that the architecture of the code—the important stuff—is in the top two positions of this list, whereas the behavior of the code occupies the first and third positions.

Hãy để ý rằng kiến trúc của mã — thứ quan trọng — chiếm hai vị trí đầu trong danh sách này, còn hành vi của mã chiếm vị trí thứ nhất và thứ ba.

The mistake that business managers and developers often make is to elevate items in position 3 to position 1. In other words, they fail to separate those features that are urgent but not important from those features that truly are urgent and important. This failure then leads to ignoring the important architecture of the system in favor of the unimportant features of the system.

Sai lầm mà các nhà quản lý nghiệp vụ lẫn lập trình viên hay mắc phải là nâng những mục ở vị trí 3 lên vị trí 1. Nói cách khác, họ không tách được những tính năng khẩn cấp nhưng không quan trọng ra khỏi những tính năng thực sự vừa khẩn cấp vừa quan trọng. Sai lầm đó dẫn tới việc bỏ mặc phần kiến trúc quan trọng của hệ thống để chạy theo những tính năng chẳng quan trọng.

The dilemma for software developers is that business managers are not equipped to evaluate the importance of architecture. That's what software developers were hired to do. Therefore it is the responsibility of the software development team to assert the importance of architecture over the urgency of features.

Thế lưỡng nan của lập trình viên là ở chỗ các nhà quản lý nghiệp vụ không đủ chuyên môn để đánh giá tầm quan trọng của kiến trúc. Đó chính là việc mà lập trình viên được thuê để làm. Bởi vậy, trách nhiệm của đội phát triển phần mềm là khẳng định tầm quan trọng của kiến trúc đứng trên tính khẩn cấp của các tính năng.

Fight for the Architecture — Hãy chiến đấu vì kiến trúc

Fulfilling this responsibility means wading into a fight—or perhaps a better word is “struggle.” Frankly, that's always the way these things are done. The development team has to struggle for what they believe to be best for the company, and so do the management team, and the marketing team, and the sales team, and the operations team. It's always a struggle .

Làm tròn trách nhiệm này nghĩa là dấn thân vào một cuộc chiến — hoặc có lẽ từ đúng hơn là một cuộc “giằng co”. Nói thẳng ra, mọi chuyện xưa nay vẫn luôn diễn ra như vậy. Đội phát triển phải giằng co vì những gì họ tin là tốt nhất cho công ty, và đội quản lý, đội marketing, đội bán hàng, đội vận hành cũng vậy. Lúc nào cũng là một cuộc giằng co.

Effective software development teams tackle that struggle head on. They unabashedly squabble with all the other stakeholders as equals. Remember, as a software developer, you are a stakeholder . You have a stake in the software that you need to safeguard. That’s part of your role, and part of your duty. And it’s a big part of why you were hired.

Những đội phát triển phần mềm hiệu quả đối diện trực tiếp với cuộc giằng co đó. Họ tranh cãi thẳng thắn, không ngại ngần, ngang hàng với mọi bên liên quan khác. Hãy nhớ: là một lập trình viên, chính bạn cũng là một bên liên quan. Bạn có phần lợi ích trong phần mềm và bạn cần bảo vệ phần lợi ích đó. Đó là một phần vai trò của bạn, một phần bổn phận của bạn. Và đó cũng là lý do lớn khiến bạn được thuê.

This challenge is doubly important if you are a software architect. Software architects are, by virtue of their job description, more focused on the structure of the system than on its features and functions. Architects create an architecture that allows those features and functions to be easily developed, easily modified, and easily extended.

Thử thách này còn quan trọng gấp đôi nếu bạn là kiến trúc sư phần mềm. Theo đúng bản mô tả công việc, kiến trúc sư phần mềm tập trung vào cấu trúc của hệ thống nhiều hơn là vào các tính năng và chức năng của nó. Kiến trúc sư tạo ra một kiến trúc giúp những tính năng và chức năng ấy dễ phát triển, dễ sửa đổi và dễ mở rộng.

Just remember: If architecture comes last, then the system will become ever more costly to develop, and eventually change will become practically impossible for part or all of the system. If that is allowed to happen, it means the software development team did not fight hard enough for what they knew was necessary.

Chỉ cần nhớ điều này: Nếu kiến trúc bị xếp sau cùng, thì hệ thống sẽ ngày càng tốn kém để phát triển, và cuối cùng việc thay đổi sẽ trở nên bất khả thi trên thực tế, với một phần hoặc toàn bộ hệ thống. Nếu chuyện đó xảy ra, điều đó có nghĩa là đội phát triển phần mềm đã không chiến đấu đủ quyết liệt cho thứ mà chính họ biết là cần thiết.

Starting with the Bricks: Programming Paradigms — Bắt đầu từ những viên gạch: Các mô thức lập trình

Software **architecture** begins with the code—and so we will begin our discussion of architecture by looking at what we’ve learned about code since code was first written.

Kiến trúc (architecture) phần mềm bắt đầu từ mã nguồn—nên chúng ta cũng sẽ bắt đầu câu chuyện về kiến trúc bằng cách nhìn lại những gì đã học được về mã nguồn kể từ khi dòng mã đầu tiên được viết ra.

In 1938, Alan Turing laid the foundations of what was to become computer programming. He was not the first to conceive of a programmable machine, but he was the first to understand that programs were simply data. By 1945, Turing was writing real programs on real computers in code that we would recognize (if we squinted enough). Those programs used loops, branches, **assignment**, subroutines, stacks, and other familiar structures. Turing’s language was binary.

Năm 1938, Alan Turing đặt nền móng cho cái sau này trở thành lập trình máy tính. Ông không phải người đầu tiên hình dung ra một cỗ máy có thể lập trình được, nhưng ông là người đầu tiên hiểu rằng chương trình chẳng qua cũng chỉ là dữ liệu. Đến năm 1945, Turing đã viết những chương trình thật trên những chiếc máy tính thật, bằng thứ mã mà ta vẫn có thể nhận ra (nếu nheo mắt đủ lâu). Những chương trình đó dùng vòng lặp, rẽ nhánh, phép gán (assignment), chương trình con, ngăn xếp và nhiều cấu trúc quen thuộc khác. Ngôn ngữ của Turing là ngôn ngữ nhị phân.

Since those days, a number of revolutions in programming have occurred. One revolution with which we are all very familiar is the revolution of languages. First, in the late 1940s, came assemblers. These “languages” relieved the programmers of the drudgery of translating their programs into binary. In 1951, Grace Hopper invented A0, the first compiler. In fact, she coined the term compiler. Fortran was invented in 1953 (the year after I was

Kể từ những ngày ấy, đã có không ít cuộc cách mạng xảy ra trong lập trình. Một cuộc cách mạng mà tất cả chúng ta đều quá quen thuộc là cuộc cách mạng về ngôn ngữ. Đầu tiên, vào cuối thập niên 1940, là các assembler. Những “ngôn ngữ” này giải phóng lập trình viên khỏi công việc nhọc nhằn là dịch chương trình của mình sang nhị phân. Năm 1951, Grace Hopper phát minh ra A0, trình biên dịch đầu tiên. Thật ra, chính bà là người đặt ra từ *compiler*. Fortran ra đời năm 1953 (một năm sau khi tôi

born). What followed was an unceasing flood of new programming languages—COBOL, PL/1, SNOBOL, C, Pascal, C++, Java, ad infinitum.

chào đời). Tiếp sau đó là một cơn lũ bất tận những ngôn ngữ lập trình mới—COBOL, PL/1, SNOBOL, C, Pascal, C++, Java, ad infinitum.

Another, probably more significant, revolution was in programming paradigms . Paradigms are ways of programming, relatively unrelated to languages. A **paradigm** tells you which programming structures to use, and when to use them. To date, there have been three such paradigms. For reasons we shall discuss later, there are unlikely to be any others.

Một cuộc cách mạng khác, có lẽ còn quan trọng hơn nhiều, diễn ra ở các *mô thức lập trình* (paradigm). Mô thức là những cách lập trình, khá độc lập với ngôn ngữ. Một mô thức cho bạn biết nên dùng những cấu trúc lập trình nào, và dùng khi nào. Cho đến nay, mới chỉ có ba mô thức như vậy. Vì những lý do sẽ bàn sau, nhiều khả năng sẽ chẳng còn mô thức nào khác nữa.

Paradigm Overview — Tổng quan về các mô thức

The three paradigms included in this overview chapter are **structured programming**, object-oriented programming, and **functional programming**.

Ba mô thức (paradigm) được điểm qua trong chương tổng quan này là lập trình có cấu trúc (structured programming), lập trình hướng đối tượng (OO) và lập trình hàm (functional programming).

Structured Programming — Lập trình có cấu trúc

The first **paradigm** to be adopted (but not the first to be invented) was structured programming, which was discovered by Edsger Wybe Dijkstra in

Mô thức đầu tiên được áp dụng (nhưng không phải mô thức đầu tiên được phát minh) là lập trình có cấu trúc, do Edsger Wybe Dijkstra khám phá ra vào năm

1968. Dijkstra showed that the use of unrestrained jumps (goto statements) is

1968. Dijkstra chỉ ra rằng việc dùng những bước nhảy tùy tiện (câu lệnh goto) là

harmful to program **structure**. As we'll see in the chapters that follow, he replaced those jumps with the more familiar if/then/else and

có hại cho cấu trúc chương trình. Như ta sẽ thấy trong các chương tiếp theo, ông thay những bước nhảy đó bằng các cấu trúc quen thuộc hơn là if/then/else và

do/while/until constructs.

We can summarize the structured programming paradigm as follows:

Có thể tóm tắt mô thức lập trình có cấu trúc như sau:

Structured programming imposes discipline on direct transfer of control.

Lập trình có cấu trúc áp đặt kỷ luật lên việc chuyển điều khiển trực tiếp.

Object-Oriented Programming — Lập trình hướng đối tượng

The second paradigm to be adopted was actually discovered two years earlier, in 1966, by Ole Johan Dahl and Kristen Nygaard. These two programmers noticed that the function call stack frame in the ALGOL language could be moved to a heap, thereby allowing local variables declared by a function

to exist long after the function returned. The function became a constructor for a class, the local variables became instance variables, and the nested functions became methods. This led inevitably to the discovery of **polymorphism** through the disciplined use of function pointers.

Mô thức thứ hai được áp dụng thực ra lại được khám phá sớm hơn hai năm, vào năm 1966, bởi Ole Johan Dahl và Kristen Nygaard. Hai lập trình viên này nhận thấy rằng khung ngăn xếp (stack frame) của lời gọi hàm trong ngôn ngữ ALGOL có thể được chuyển sang heap, nhờ đó các biến cục bộ do một hàm khai báo vẫn tồn tại rất lâu sau khi hàm ấy đã trả về. Hàm trở thành constructor của một class, các biến cục bộ trở thành biến thể hiện (instance variable), còn các hàm lồng bên trong trở thành method. Điều này tất yếu dẫn tới việc khám phá ra tính đa hình (polymorphism) thông qua cách dùng con trỏ hàm một cách có kỷ luật.

We can summarize the **object-oriented programming** paradigm as follows:

Có thể tóm tắt mô thức lập trình hướng đối tượng như sau:

Object-oriented programming imposes discipline on indirect transfer of control.

Lập trình hướng đối tượng áp đặt kỷ luật lên việc chuyển điều khiển gián tiếp.

Functional Programming — Lập trình hàm

The third paradigm, which has only recently begun to be adopted, was the first to be invented. Indeed, its invention predates computer programming

Mô thức thứ ba, mãi gần đây mới bắt đầu được áp dụng, lại chính là mô thức được phát minh đầu tiên. Thật vậy, nó ra đời trước cả bản thân việc lập trình

itself. Functional programming is the direct result of the work of Alonzo Church, who in 1936 invented λ -calculus while pursuing the same mathematical problem that was motivating Alan Turing at the same time. His λ -calculus is the foundation of the LISP language, invented in 1958 by John McCarthy. A foundational notion of λ -calculus is **immutability**—that is, the notion that the values of symbols do not change. This effectively means that a functional language has no **assignment** statement. Most functional languages do, in fact, have some means to alter the value of a variable, but only under very strict discipline.

máy tính. Lập trình hàm là kết quả trực tiếp từ công trình của Alonzo Church, người vào năm 1936 đã phát minh ra λ -calculus khi theo đuổi đúng bài toán toán học mà cùng lúc đó cũng đang thôi thúc Alan Turing. λ -calculus của ông là nền móng của ngôn ngữ LISP, do John McCarthy phát minh năm 1958. Một khái niệm nền tảng của λ -calculus là tính bất biến (immutability) — tức là ý niệm rằng giá trị của các ký hiệu không thay đổi. Điều này trên thực tế có nghĩa là một ngôn ngữ hàm không hề có câu lệnh gán. Thật ra, phần lớn các ngôn ngữ hàm đều có cách nào đó để thay đổi giá trị của một biến, nhưng chỉ dưới một kỷ luật hết sức nghiêm ngặt.

We can summarize the functional programming paradigm as follows:

Có thể tóm tắt mô thức lập trình hàm như sau:

Functional programming imposes discipline upon assignment.

Lập trình hàm áp đặt kỷ luật lên phép gán (assignment).

Food for Thought — Đôi điều để ngẫm nghĩ

Notice the pattern that I've quite deliberately set up in introducing these three programming paradigms: Each of the paradigms removes capabilities from the programmer. None of them adds new capabilities. Each imposes some kind of extra discipline that is negative in its intent. The paradigms tell us what not to do, more than they tell us what to do.

Hãy để ý cái khuôn mẫu mà tôi đã cố tình dựng lên khi giới thiệu ba mô thức lập trình này: mỗi mô thức đều tước bớt năng lực của lập trình viên. Không mô thức nào thêm vào năng lực mới. Mỗi mô thức áp đặt một thứ kỷ luật bổ sung nào đó, và ý đồ của nó là phủ định. Các mô thức nói cho ta biết điều *không* được làm, nhiều hơn là nói cho ta biết điều nên làm.

Another way to look at this issue is to recognize that each paradigm takes something away from us. The three paradigms together remove goto statements, function pointers, and assignment. Is there anything left to take away?

Một cách nhìn khác về vấn đề này là nhận ra rằng mỗi mô thức lấy đi của ta một thứ gì đó. Cả ba mô thức cộng lại đã lấy đi câu lệnh goto, con trỏ hàm và phép gán. Liệu còn thứ gì để lấy đi nữa không?

Probably not. Thus these three paradigms are likely to be the only three we will see—at least the only three that are negative. Further evidence that there are no more such paradigms is that they were all discovered within the ten years between 1958 and 1968. In the many decades that have followed, no new paradigms have been added.

Có lẽ là không. Vì vậy, ba mô thức này nhiều khả năng là ba mô thức duy nhất mà ta sẽ thấy — ít nhất là ba mô thức mang tính phủ định duy nhất. Một bằng chứng nữa cho thấy không còn mô thức nào như thế nữa là: cả ba đều được khám phá trong vòng mười năm, từ 1958 đến 1968. Suốt nhiều thập kỷ sau đó, không có mô thức mới nào được bổ sung.

Conclusion — Kết luận

What does this history lesson on paradigms have to do with **architecture**? Everything. We use polymorphism as the mechanism to cross architectural boundaries; we use functional programming to impose discipline on the location of and access to data; and we use structured programming as the algorithmic foundation of our modules.

Bài học lịch sử về các mô thức này thì liên quan gì tới kiến trúc (architecture)? Liên quan tất cả. Ta dùng đa hình làm cơ chế để vượt các ranh giới kiến trúc; ta dùng lập trình hàm để áp đặt kỷ luật lên vị trí đặt dữ liệu và cách truy cập dữ liệu; và ta dùng lập trình có cấu trúc làm nền móng thuật toán cho các module của mình.

Notice how well those three align with the three big concerns of architecture: function, separation of components, and data management.

Hãy để ý ba thứ đó khớp đến thế nào với ba mối bận tâm lớn của kiến trúc: chức năng, sự phân tách các component, và quản lý dữ liệu.

Structured Programming — Lập trình có cấu trúc

Edsger Wybe Dijkstra was born in Rotterdam in 1930. He survived the bombing of Rotterdam during World War II, along with the German occupation of the Netherlands, and in 1948 graduated from high school with the highest possible marks in math, physics, chemistry, and biology. In March 1952, at the age of 21 (and just 9 months before I was born), Dijkstra took a job with the Mathematical Center of Amsterdam as the Netherlands' very first programmer.

Edsger Wybe Dijkstra sinh năm 1930 tại Rotterdam. Ông sống sót qua trận bom dội xuống Rotterdam trong Thế chiến II cũng như thời kỳ Đức chiếm đóng Hà Lan, và năm 1948 tốt nghiệp trung học với điểm số cao nhất có thể ở cả bốn môn toán, lý, hóa và sinh. Tháng 3 năm 1952, ở tuổi 21 (và chỉ 9 tháng trước khi tôi ra đời), Dijkstra nhận việc tại Trung tâm Toán học Amsterdam với tư cách lập trình viên đầu tiên của Hà Lan.

In 1955, having been a programmer for three years, and while still a student, Dijkstra concluded that the intellectual challenge of programming was greater than the intellectual challenge of theoretical physics. As a result, he chose programming as his long-term career.

Năm 1955, sau ba năm làm lập trình viên và khi vẫn còn là sinh viên, Dijkstra đi đến kết luận rằng thách thức trí tuệ của lập trình còn lớn hơn thách thức trí tuệ của vật lý lý thuyết. Vì vậy, ông chọn lập trình làm sự nghiệp lâu dài.

In 1957, Dijkstra married Maria Debets. At the time, you had to state your profession as part of the marriage rites in the Netherlands. The Dutch authorities were unwilling to accept “programmer” as Dijkstra’s profession; they had never heard of such a profession. To satisfy them, Dijkstra settled for “theoretical physicist” as his job title.

Năm 1957, Dijkstra kết hôn với Maria Debets. Thời đó, ở Hà Lan bạn phải khai nghề nghiệp của mình như một phần của thủ tục kết hôn. Nhà chức trách Hà Lan không chịu chấp nhận “lập trình viên” là một nghề nghiệp; họ chưa từng nghe nói đến cái nghề ấy. Để làm họ hài lòng, Dijkstra đành khai chức danh là “nhà vật lý lý thuyết”.

As part of deciding to make programming his career, Dijkstra conferred with his boss, Adriaan van Wijngaarden. Dijkstra was concerned that no one had identified a discipline, or science, of programming, and that he would therefore not be taken seriously. His boss replied that Dijkstra might very well be one of the people who would discover such disciplines, thereby evolving software into a science.

Khi quyết định lấy lập trình làm nghiệp, Dijkstra có bàn bạc với sếp của mình, Adriaan van Wijngaarden. Dijkstra lo rằng chưa ai xác lập được một bộ môn, hay một ngành khoa học, về lập trình, và do đó ông sẽ không được xem trọng. Sếp ông đáp rằng rất có thể chính

Dijkstra sẽ là một trong những người khám phá ra những bộ môn ấy, qua đó biến phần mềm thành một ngành khoa học.

Dijkstra started his career in the era of vacuum tubes, when computers were huge, fragile, slow, unreliable, and (by today's standards) extremely limited. In those early years, programs were written in binary, or in very crude assembly language. Input took the physical form of paper tape or punched cards. The edit/compile/test loop was hours—if not days—long.

Dijkstra khởi nghiệp vào thời của đèn chân không, khi máy tính còn khổng lồ, mong manh, chậm chạp, kém tin cậy và (theo tiêu chuẩn ngày nay) cực kỳ hạn chế. Những năm đầu ấy, chương trình được viết bằng mã nhị phân, hoặc bằng thứ hợp ngữ rất thô sơ. Dữ liệu vào có hình hài vật lý là băng giấy hoặc phiếu đục lỗ. Vòng lặp sửa/biên dịch/kiểm thử kéo dài hàng giờ — nếu không muốn nói là hàng ngày.

It was in this primitive environment that Dijkstra made his great discoveries.

Chính trong môi trường sơ khai đó, Dijkstra đã có những khám phá lớn của mình.

Proof — Chứng minh

The problem that Dijkstra recognized, early on, was that programming is hard , and that programmers don't do it very well. A program of any complexity contains too many details for a human brain to manage without help. Overlooking just one small **detail** results in programs that may seem to work, but fail in surprising ways.

Vấn đề mà Dijkstra nhận ra từ rất sớm là lập trình *khó*, và lập trình viên thì làm việc đó không giỏi cho lắm. Một chương trình chỉ cần phức tạp ở mức vừa phải cũng đã chứa quá nhiều chi tiết để bộ não người quản nổi nếu không có trợ giúp. Bỏ sót chỉ một chi tiết nhỏ thôi là ta có một chương trình trông thì có vẻ chạy được, nhưng lại hỏng theo những cách hết sức bất ngờ.

Dijkstra's solution was to apply the mathematical discipline of proof . His vision was the construction of a Euclidian hierarchy of postulates, theorems, corollaries, and lemmas. Dijkstra thought that programmers could use that hierarchy the way mathematicians do. In other words, programmers would use proven structures, and tie them together with code that they would then prove correct themselves.

Giải pháp của Dijkstra là áp dụng bộ môn *chứng minh* của toán học. Tầm nhìn của ông là dựng lên một hệ thống phân cấp kiểu Euclid gồm các tiên đề, định lý, hệ quả và bổ đề. Dijkstra cho rằng lập trình viên có thể dùng hệ thống phân cấp ấy đúng như cách các nhà toán học vẫn làm. Nói cách khác, lập trình viên sẽ dùng những cấu trúc đã được chứng minh, rồi ghép chúng lại bằng đoạn mã mà chính họ sẽ tự chứng minh là đúng.

Of course, to get this going, Dijkstra realized that he would have to demonstrate the technique for writing basic proofs of simple algorithms. This he found to be quite challenging.

Dĩ nhiên, để khởi động chuyện này, Dijkstra hiểu rằng ông sẽ phải trình diễn được kỹ thuật viết những chứng minh cơ bản cho các thuật toán đơn giản. Và ông thấy việc đó khá hóc búa.

During his investigation, Dijkstra discovered that certain uses of goto statements prevent modules from being decomposed recursively into smaller and smaller units, thereby preventing use of the divide-and-conquer approach necessary for reasonable proofs.

Trong quá trình tìm tòi, Dijkstra phát hiện rằng một số cách dùng lệnh goto khiến các module không thể được phân rã đệ quy thành những đơn vị ngày một nhỏ hơn, và do đó chặn mất lối tiếp cận chia để trị vốn cần thiết cho những chứng minh hợp lý.

Other uses of goto , however, did not have this problem. Dijkstra realized that these “good” uses of goto corresponded to simple selection and iteration control structures such as if/then/else and do/while . Modules that used only those kinds of control structures could be recursively subdivided into provable units.

Tuy nhiên, những cách dùng goto khác lại không gặp vấn đề đó. Dijkstra nhận ra rằng các cách dùng goto “tốt” này tương ứng với những cấu trúc điều khiển chọn và lặp đơn giản như if/then/else và do/while. Những module chỉ dùng các loại cấu trúc điều khiển ấy có thể được chia nhỏ đệ quy thành các đơn vị chứng minh được.

Dijkstra knew that those control structures, when combined with sequential execution, were special. They had been identified two years before by Böhm and Jacopini, who proved that all programs can be constructed from just three structures: sequence, selection, and iteration.

Dijkstra biết rằng những cấu trúc điều khiển đó, khi kết hợp với thực thi tuần tự, là thứ rất đặc biệt. Trước đó hai năm, chúng đã được Böhm và Jacopini chỉ ra; hai ông đã chứng minh rằng mọi chương trình đều có thể được dựng nên chỉ từ ba cấu trúc: tuần tự (sequence), chọn (selection) và lặp (iteration).

This discovery was remarkable: The very control structures that made a module provable were the same minimum set of control structures from which all programs can be built. Thus **structured programming** was born.

Khám phá này thật đáng kinh ngạc: chính những cấu trúc điều khiển làm cho một module chứng minh được lại đúng là tập tối thiểu các cấu trúc điều khiển đủ để dựng nên mọi chương trình. Lập trình có cấu trúc (structured programming) ra đời từ đó.

Dijkstra showed that sequential statements could be proved correct through simple enumeration. The technique mathematically traced the inputs of the sequence to the outputs of the sequence. This approach was no different from any normal mathematical proof.

Dijkstra chỉ ra rằng các câu lệnh tuần tự có thể được chứng minh là đúng bằng phép liệt kê đơn giản. Kỹ thuật này truy vết về mặt toán học từ đầu vào của chuỗi lệnh tới đầu ra của nó. Cách làm này chẳng khác gì một chứng minh toán học thông thường.

Dijkstra tackled selection through reapplication of enumeration. Each path through the selection was enumerated. If both paths eventually produced appropriate mathematical results, then the proof was solid.

Dijkstra xử lý cấu trúc chọn bằng cách áp dụng lại phép liệt kê. Mỗi nhánh đi qua cấu trúc chọn đều được liệt kê. Nếu cả hai nhánh rốt cuộc đều cho ra kết quả toán học phù hợp, thì chứng minh là vững.

Iteration was a bit different. To prove an iteration correct, Dijkstra had to use induction . He proved the case for 1 by enumeration. Then he proved the case that if N was assumed correct, N + 1 was correct, again by enumeration. He also proved the starting and ending criteria of the iteration by enumeration.

Lặp thì hơi khác. Để chứng minh một vòng lặp là đúng, Dijkstra phải dùng *quy nạp*. Ông chứng minh trường hợp 1 bằng liệt kê. Rồi ông chứng minh rằng nếu giả sử N đúng thì N + 1 cũng đúng, cũng bằng liệt kê. Ông còn chứng minh cả tiêu chí khởi đầu và kết thúc của vòng lặp bằng liệt kê.

Such proofs were laborious and complex—but they were proofs. With their development, the idea that a Euclidean hierarchy of theorems could be constructed seemed reachable.

Những chứng minh như vậy nhọc nhằn và rắc rối — nhưng chúng là chứng minh thật sự. Với việc phát triển chúng, ý tưởng dựng nên một hệ thống phân cấp định lý kiểu Euclid trông có vẻ nằm trong tầm với.

A Harmful Proclamation — Một tuyên ngôn tai hại

In 1968, Dijkstra wrote a letter to the editor of CACM, which was published in the March issue. The title of this letter was “Go To Statement Considered Harmful.” The article outlined his position on the three control structures.

Năm 1968, Dijkstra viết một lá thư gửi ban biên tập tạp chí CACM, và nó được đăng trong số tháng Ba. Nhan đề lá thư là “Go To Statement Considered Harmful”. Bài viết trình bày quan điểm của ông về ba cấu trúc điều khiển.

And the programming world caught fire. Back then we didn’t have an Internet, so people couldn’t post nasty memes of Dijkstra, and they couldn’t flame him online. But they could, and they did, write letters to the editors of many published journals.

Và thế giới lập trình bùng cháy. Hồi đó chúng ta chưa có Internet, nên người ta không thể đăng meme xỏ xiên Dijkstra, cũng chẳng thể ném đá ông trên mạng. Nhưng họ có thể viết thư gửi ban biên tập của hàng loạt tạp chí — và họ đã làm đúng như thế.

Those letters weren’t necessarily all polite. Some were intensely negative; others voiced strong support for his position. And so the battle was joined, ultimately to last about a decade.

Những lá thư ấy không phải lúc nào cũng lịch sự. Một số gay gắt vô cùng; số khác lại ủng hộ mạnh mẽ quan điểm của ông. Thế là cuộc chiến nổ ra, và rốt cuộc kéo dài chừng một thập kỷ.

Eventually the argument petered out. The reason was simple: Dijkstra had won. As computer languages evolved, the goto statement moved ever rearward, until it all but disappeared. Most modern languages do not have a goto statement—and, of course, LISP never did.

Cuối cùng thì tranh cãi cũng lụi dần. Lý do rất đơn giản: Dijkstra đã thắng. Khi các ngôn ngữ lập trình tiến hóa, lệnh goto cứ lùi mãi về phía sau cho tới lúc gần như biến mất hẳn. Phần lớn ngôn ngữ hiện đại không có lệnh goto — và tất nhiên, LISP thì chưa bao giờ có.

Nowadays we are all structured programmers, though not necessarily by choice. It’s just that our languages don’t give us the option to use undisciplined **direct transfer of control**.

Ngày nay tất cả chúng ta đều là lập trình viên có cấu trúc, dù không hẳn vì tự nguyện. Đơn giản là ngôn ngữ của chúng ta không còn cho ta lựa chọn dùng kiểu chuyển điều khiển trực tiếp vô kỷ luật nữa.

Some may point to named break s in Java or exceptions as goto analogs. In fact, these structures are not the utterly unrestricted transfers of control that older languages like Fortran or COBOL once had. Indeed, even languages that still support the goto keyword often restrict the target to within the scope of the current function.

Có người sẽ chỉ vào break có nhãn trong Java hay cơ chế exception mà bảo rằng chúng chẳng khác gì goto. Thật ra, những cấu trúc này không phải kiểu chuyển điều khiển hoàn toàn vô hạn chế như các ngôn ngữ cũ hơn — Fortran hay COBOL — từng có. Thậm chí,

ngay cả những ngôn ngữ vẫn còn hỗ trợ từ khóa goto cũng thường giới hạn đích nhảy nằm trong phạm vi của hàm hiện tại.

Functional Decomposition — Phân rã theo hàm

Structured programming allows modules to be recursively decomposed into provable units, which in turn means that modules can be functionally decomposed. That is, you can take a large-scale problem statement and decompose it into high-**level** functions. Each of those functions can then be decomposed into lower-level functions, ad infinitum. Moreover, each of those decomposed functions can be represented using the restricted control structures of structured programming.

Lập trình có cấu trúc cho phép các module được phân rã đệ quy thành những đơn vị chứng minh được, mà điều đó cũng có nghĩa là module có thể được phân rã theo hàm. Nghĩa là bạn có thể lấy một phát biểu bài toán ở quy mô lớn rồi phân rã nó thành các hàm mức cao. Từng hàm trong số đó lại có thể được phân rã tiếp thành các hàm mức thấp hơn, cứ thế đến vô tận. Hơn nữa, mỗi hàm được phân rã ra đều có thể biểu diễn bằng đúng bộ cấu trúc điều khiển hạn chế của lập trình có cấu trúc.

Building on this foundation, disciplines such as structured analysis and structured **design** became popular in the late 1970s and throughout the 1980s. Men like Ed Yourdon, Larry Constantine, Tom DeMarco, and Meilir Page-Jones promoted and popularized these techniques throughout that period. By following these disciplines, programmers could break down large proposed systems into modules and components that could be further broken down into tiny provable functions.

Dựng trên nền tảng đó, những bộ môn như phân tích có cấu trúc và thiết kế có cấu trúc trở nên phổ biến vào cuối thập niên 1970 và suốt thập niên 1980. Những người như Ed Yourdon, Larry Constantine, Tom DeMarco và Meilir Page-Jones đã cổ vũ và phổ biến các kỹ thuật này trong suốt giai đoạn ấy. Bằng cách tuân theo những bộ môn đó, lập trình viên có thể bẻ nhỏ các hệ thống lớn được đề xuất thành module và component, rồi lại bẻ nhỏ tiếp thành những hàm tí hon chứng minh được.

No Formal Proofs — Không có chứng minh hình thức nào cả

But the proofs never came. The Euclidean hierarchy of theorems was never built. And programmers at large never saw the benefits of working through the laborious process of formally proving each and every little function correct. In the end, Dijkstra's dream faded and died. Few of today's programmers believe that formal proofs are an appropriate way to produce high-quality software.

Nhưng các chứng minh chẳng bao giờ tới. Hệ thống phân cấp định lý kiểu Euclid chưa bao giờ được dựng lên. Và giới lập trình viên nói chung chưa bao giờ thấy được ích lợi của việc lê lết qua cái quy trình nhọc nhằn là chứng minh hình thức cho từng hàm bé xíu một. Rốt cuộc, giấc mơ của Dijkstra nhạt dần rồi tắt. Rất ít lập trình viên ngày nay tin rằng chứng minh hình thức là cách phù hợp để tạo ra phần mềm chất lượng cao.

Of course, formal, Euclidian style, mathematical proofs are not the only strategy for proving something correct. Another highly successful strategy is the scientific method .

Tất nhiên, chứng minh toán học hình thức theo lối Euclid không phải chiến lược duy nhất để chứng minh một điều gì đó là đúng. Một chiến lược khác cực kỳ thành công chính là *phương pháp khoa học*.

Science to the Rescue — Khoa học ra tay giải cứu

Science is fundamentally different from mathematics, in that scientific theories and laws cannot be proven correct. I cannot prove to you that Newton's second law of motion, $F = ma$, or law of gravity, $F = Gm_1 m_2 / r^2$, are correct. I can demonstrate these laws to you, and I can make measurements that show them correct to many decimal places, but I cannot prove them in the sense of a mathematical proof. No matter how many experiments I conduct or how much empirical evidence I gather, there is always the chance that some experiment will show that those laws of motion and gravity are incorrect.

Khoa học khác biệt về căn bản so với toán học ở chỗ các lý thuyết và định luật khoa học không thể được chứng minh là đúng. Tôi không thể chứng minh cho bạn thấy rằng định luật hai của Newton về chuyển động, $F = ma$, hay định luật vạn vật hấp dẫn, $F = Gm_1 m_2 / r^2$, là đúng. Tôi có thể trình diễn các định luật ấy cho bạn xem, tôi có thể đo đạc để cho thấy chúng đúng tới nhiều chữ số thập phân, nhưng tôi không thể chứng minh chúng theo nghĩa của một chứng minh toán học. Dù tôi có tiến hành bao nhiêu thí nghiệm hay thu thập bao nhiêu bằng chứng thực nghiệm đi nữa, vẫn luôn có khả năng một thí nghiệm nào đó sẽ chỉ ra rằng những định luật về chuyển động và hấp dẫn kia là sai.

That is the nature of scientific theories and laws: They are **falsifiable** but not provable.

Đó chính là bản chất của lý thuyết và định luật khoa học: chúng khả bác bỏ (falsifiable) chứ không chứng minh được.

And yet we bet our lives on these laws every day. Every time you get into a car, you bet your life that $F = ma$ is a reliable description of the way the world works. Every time you take a step, you bet your health and safety that $F = Gm_1 m_2 / r^2$ is correct.

Vậy mà ngày nào ta cũng đánh cược mạng sống của mình vào những định luật ấy. Mỗi lần bước lên xe hơi, bạn đặt cược mạng mình rằng $F = ma$ là mô tả đáng tin cậy về cách thế giới vận hành. Mỗi lần bước một bước chân, bạn đặt cược sức khỏe và sự an toàn của mình rằng $F = Gm_1 m_2 / r^2$ là đúng.

Science does not work by proving statements true, but rather by proving statements false. Those statements that we cannot prove false, after much **effort**, we deem to be true enough for our purposes.

Khoa học không vận hành bằng cách chứng minh các phát biểu là đúng, mà bằng cách chứng minh các phát biểu là sai. Những phát biểu mà ta không chứng minh được là sai, sau rất nhiều công sức, thì ta coi là đủ đúng cho mục đích của mình.

Of course, not all statements are provable. The statement “This is a lie” is neither true nor false. It is one of the simplest examples of a statement that is not provable.

Dĩ nhiên, không phải phát biểu nào cũng chứng minh được. Phát biểu “Câu này là lời nói dối” chẳng đúng mà cũng chẳng sai. Đó là một trong những ví dụ đơn giản nhất về một phát biểu không chứng minh được.

Ultimately, we can say that mathematics is the discipline of proving provable statements true. Science, in contrast, is the discipline of proving provable statements false.

Suy cho cùng, ta có thể nói rằng toán học là bộ môn chứng minh những phát biểu chứng minh được là đúng. Còn khoa học, ngược lại, là bộ môn chứng minh những phát biểu chứng minh được là sai.

Tests — Kiểm thử

Dijkstra once said, “Testing shows the presence, not the absence, of bugs.” In other words, a program can be proven incorrect by a test, but it cannot be proven correct. All that tests can do, after sufficient testing effort, is allow us to deem a program to be correct enough for our purposes.

Dijkstra từng nói: “Kiểm thử cho thấy sự hiện diện của lỗi, chứ không phải sự vắng mặt của chúng.” Nói cách khác, một chương trình có thể bị một bài test chứng minh là sai, nhưng không thể được chứng minh là đúng. Tất cả những gì các bài test làm được, sau đủ nhiều công sức kiểm thử, là cho phép ta coi chương trình đó là đủ đúng cho mục đích của mình.

The implications of this fact are stunning. Software development is not a mathematical endeavor, even though it seems to manipulate mathematical constructs. Rather, software is like a science. We show correctness by failing to prove incorrectness, despite our best efforts.

Hệ quả của sự thật này thật choáng váng. Phát triển phần mềm không phải một nỗ lực toán học, dù nó có vẻ như đang thao tác trên các cấu trúc toán học. Đúng hơn, phần mềm giống một ngành khoa học. Ta cho thấy tính đúng đắn bằng việc thất bại trong nỗ lực chứng minh sự sai, dù đã cố hết sức.

Such proofs of incorrectness can be applied only to provable programs. A program that is not provable—due to unrestrained use of goto, for example—cannot be deemed correct no matter how many tests are applied to it.

Những chứng minh về sự sai như vậy chỉ áp dụng được cho các chương trình chứng minh được. Một chương trình không chứng minh được — chẳng hạn vì dùng goto một cách vô hạn chế — thì không thể được coi là đúng, bất kể ta ném vào nó bao nhiêu bài test.

Structured programming forces us to recursively decompose a program into a set of small provable functions. We can then use tests to try to prove those small provable functions incorrect. If such tests fail to prove incorrectness, then we deem the functions to be correct enough for our purposes.

Lập trình có cấu trúc buộc ta phải phân rã đệ quy một chương trình thành một tập các hàm nhỏ chứng minh được. Rồi ta có thể dùng các bài test để cố chứng minh những hàm nhỏ chứng minh được ấy là sai. Nếu các bài test đó không chứng minh được là sai, thì ta coi những hàm ấy là đủ đúng cho mục đích của mình.

Conclusion — Kết luận

It is this ability to create falsifiable units of programming that makes structured programming valuable today. This is the reason that modern languages do not typically support unrestrained goto statements. Moreover,

Chính khả năng tạo ra những đơn vị lập trình khả bác bỏ mới là thứ khiến lập trình có cấu trúc còn giá trị đến ngày nay. Đó cũng là lý do các ngôn ngữ hiện đại thường không hỗ trợ lệnh goto vô hạn chế. Hơn nữa,

at the architectural level, this is why we still consider **functional decomposition** to be one of our best practices.

ở cấp kiến trúc, đây chính là lý do chúng ta vẫn xem phân rã theo hàm là một trong những thực hành tốt nhất của mình.

At every level, from the smallest function to the largest **component**, software is like a science and, therefore, is driven by falsifiability. Software architects strive to define modules, components, and

services that are easily falsifiable (testable). To do so, they employ restrictive disciplines similar to structured programming, albeit at a much higher level.

Ở mọi cấp độ, từ hàm nhỏ nhất tới component lớn nhất, phần mềm giống một ngành khoa học và vì thế lấy tính khả bác bỏ làm kim chỉ nam. Kiến trúc sư phần mềm nỗ lực định nghĩa những module, component và dịch vụ để bác bỏ (để kiểm thử). Để làm được điều đó, họ áp dụng những kỷ luật ràng buộc tương tự lập trình có cấu trúc, chỉ khác là ở mức cao hơn nhiều.

It is those restrictive disciplines that we will study in some detail in the chapters to come.

Chính những kỷ luật ràng buộc ấy là thứ ta sẽ tìm hiểu khá chi tiết trong các chương tới.

Object-Oriented Programming — Lập trình hướng đối tượng (OO)

As we will see, the basis of a good **architecture** is the understanding and application of the principles of object-oriented **design** (OO). But just what is OO?

Như chúng ta sẽ thấy, nền tảng của một kiến trúc (architecture) tốt là việc hiểu và áp dụng các nguyên tắc của thiết kế hướng đối tượng (OO). Nhưng rốt cuộc OO là cái gì?

One answer to this question is “The combination of data and function.” Although often cited, this is a very unsatisfying answer because it implies that $o.f()$ is somehow different from $f(o)$. This is absurd. Programmers were passing data structures into functions long before 1966, when Dahl and Nygaard moved the function call stack frame to the heap and invented OO.

Một câu trả lời cho câu hỏi này là “sự kết hợp giữa dữ liệu và hàm”. Câu này được trích dẫn rất nhiều, nhưng lại rất kém thuyết phục, bởi nó hàm ý rằng $o.f()$ bằng cách nào đó khác với $f(o)$. Thật vô lý. Lập trình viên đã truyền cấu trúc dữ liệu vào hàm từ rất lâu trước năm 1966, cái năm mà Dahl và Nygaard chuyển stack frame của lời gọi hàm sang heap và phát minh ra OO.

Another common answer to this question is “A way to model the real world.” This is an evasive answer at best. What does “modeling the real world” actually mean, and why is it something we would want to do? Perhaps this statement is intended to imply that OO makes software easier to understand because it has a closer relationship to the real world—but even that statement is evasive and too loosely defined. It does not tell us what OO is.

Một câu trả lời phổ biến khác là “một cách mô hình hóa thế giới thực”. Nói vậy, nhẹ nhất thì cũng là né tránh. “Mô hình hóa thế giới thực” thực ra nghĩa là gì, và tại sao đó lại là điều ta muốn làm? Có lẽ phát biểu này ngụ ý rằng OO giúp phần mềm dễ hiểu hơn vì nó gần gũi hơn với thế giới thực — nhưng ngay cả phát biểu đó cũng là né tránh và quá lỏng lẻo. Nó không cho ta biết OO là gì.

Some folks fall back on three magic words to explain the nature of OO: **encapsulation**, **inheritance**, and **polymorphism**. The implication is that OO is the proper admixture of these three things, or at least that an OO language must support these three things.

Một số người quay về với ba từ thần chú để giải thích bản chất của OO: đóng gói (encapsulation), kế thừa (inheritance), và đa hình (polymorphism). Hàm ý là OO chính là hỗn hợp đúng đắn của ba thứ này, hoặc chí ít một ngôn ngữ OO phải hỗ trợ cả ba thứ đó.

Let’s examine each of these concepts in turn.

Hãy lần lượt xem xét từng khái niệm.

Encapsulation? — Đóng gói?

The reason encapsulation is cited as part of the definition of OO is that OO languages provide easy and effective encapsulation of data and function. As a result, a line can be drawn around a cohesive set of data and functions. Outside of that line, the data is hidden and only some of the functions are known. We see this concept in action as the private data members and the public member functions of a class.

Lý do đóng gói được viện dẫn như một phần trong định nghĩa của OO là vì các ngôn ngữ OO cung cấp cách đóng gói dữ liệu và hàm một cách dễ dàng và hiệu quả. Nhờ đó, ta có thể vạch một đường bao quanh một tập dữ liệu và hàm gắn kết với nhau. Bên ngoài đường đó, dữ liệu bị che giấu và chỉ một số hàm là được biết đến. Ta thấy khái niệm này vận hành dưới dạng các thành viên dữ liệu private và các hàm thành viên public của một class.

This idea is certainly not unique to OO. Indeed, we had perfect encapsulation in C. Consider this simple C program:

Ý tưởng này chắc chắn không phải là độc quyền của OO. Thực tế, chúng ta đã có đóng gói hoàn hảo trong C. Hãy xem chương trình C đơn giản sau:

```
point.h
struct Point;
struct Point* makePoint(double x, double y);
double distance (struct Point *p1, struct Point *p2);
point.c
#include "point.h"
#include <stdlib.h>
#include <math.h>
struct Point {
    double x,y;
};
struct Point* makepoint(double x, double y) {
    struct Point* p = malloc(sizeof(struct Point));
    p->x = x;
    p->y = y;
    return p;
}
double distance(struct Point* p1, struct Point* p2) {
    double dx = p1->x - p2->x;
    double dy = p1->y - p2->y;
    return sqrt(dx*dx+dy*dy);
}
```

The users of point.h have no access whatsoever to the members of struct Point . They can call the makePoint() function, and the distance() function, but they have absolutely no knowledge of the implementation of either the Point data **structure** or the functions.

Người dùng của point.h hoàn toàn không có cách nào truy cập các thành viên của struct Point . Họ có thể gọi hàm makePoint() và hàm distance() , nhưng tuyệt nhiên không biết gì về phần hiện thực của cấu trúc dữ liệu Point lẫn của các hàm đó.

This is perfect encapsulation—in a non-OO language. C programmers used to do this kind of thing all the time. We would forward declare data structures and functions in header files, and then implement them in

Đây là đóng gói hoàn hảo — trong một ngôn ngữ không phải OO. Ngày trước lập trình viên C làm kiểu này suốt. Chúng ta khai báo trước các cấu trúc dữ liệu và hàm trong file header, rồi hiện thực chúng trong

implementation files. Our users never had access to the elements in those implementation files.

file implementation. Người dùng của chúng ta không bao giờ chạm được tới các phần tử nằm trong những file implementation ấy.

But then came OO in the form of C++—and the perfect encapsulation of C was broken.

Nhưng rồi OO xuất hiện dưới hình hài C++ — và cái đóng gói hoàn hảo của C bị phá vỡ.

The C++ compiler, for technical reasons, needed the member variables of a class to be declared in the header file of that class. So our Point program changed to look like this:

Vì lý do kỹ thuật, 1 trình biên dịch C++ cần các biến thành viên của một class phải được khai báo trong file header của class đó. Thế là chương trình Point của chúng ta biến thành thế này:

```
point.h
class Point {
public:
    Point(double x, double y);
    double distance(const Point& p) const;
private:
    double x;
    double y;
};
point.cc
#include "point.h"
#include <math.h>
Point::Point(double x, double y)
: x(x), y(y)
{}
double Point::distance(const Point& p) const {
    double dx = x-p.x;
    double dy = y-p.y;
    return sqrt(dx*dx + dy*dy);
}
```

- 1. The C++ compiler needs to know the size of the instances of each class.
 - 1. Trình biên dịch C++ cần biết kích thước của các thể hiện của mỗi class.

Clients of the header file point.h know about the member variables x and y ! The compiler will prevent access to them, but the client still knows they exist. For example, if those member names are changed, the point.cc file must be recompiled! Encapsulation has been broken.

Client của file header point.h biết về các biến thành viên x và y ! Trình biên dịch sẽ ngăn không cho truy cập chúng, nhưng client vẫn biết chúng tồn tại. Chẳng hạn, nếu đổi tên các thành viên đó thì file point.cc phải được biên dịch lại! Đóng gói đã bị phá vỡ.

Indeed, the way encapsulation is partially repaired is by introducing the public , private , and protected keywords into the language. This, however, was a hack necessitated by the technical need for the compiler to see those variables in the header file.

Thật vậy, cách người ta vá víu phần nào cho đóng gói là đưa các từ khóa public , private và protected vào ngôn ngữ. Nhưng đó chỉ là một chiêu chấp vá, bắt nguồn từ nhu cầu kỹ thuật là trình biên dịch phải nhìn thấy các biến đó trong file header.

Java and C# simply abolished the header/implementation split altogether, thereby weakening encapsulation even more. In these languages, it is impossible to separate the declaration and definition of a class.

Java và C# thì dẹp luôn việc tách header/implementation, vì thế còn làm đóng gói yếu đi hơn nữa. Trong những ngôn ngữ này, không thể tách rời phần khai báo và phần định nghĩa của một class.

For these reasons, it is difficult to accept that OO depends on strong encapsulation. Indeed, many OO languages have little or no enforced encapsulation.

Vì những lý do đó, thật khó chấp nhận rằng OO phụ thuộc vào đóng gói mạnh. Thực tế, nhiều ngôn ngữ OO 2 có rất ít hoặc không hề áp đặt đóng gói.

OO certainly does depend on the idea that programmers are well-behaved enough to not circumvent encapsulated data. Even so, the languages that claim to provide OO have only weakened the once perfect encapsulation we enjoyed with C.

OO chắc chắn có dựa vào ý niệm rằng lập trình viên đủ tử tế để không lách qua dữ liệu đã được đóng gói. Dù vậy, những ngôn ngữ tự nhận là cung cấp OO chỉ làm suy yếu đi cái đóng gói từng hoàn hảo mà ta được hưởng với C.

Inheritance? — Kế thừa?

If OO languages did not give us better encapsulation, then they certainly gave us inheritance.

Nếu các ngôn ngữ OO không cho ta đóng gói tốt hơn, thì ít nhất chúng cũng cho ta kế thừa.

Well—sort of. Inheritance is simply the redeclaration of a group of variables and functions within an enclosing scope. This is something C programmers were able to do manually long before there was an OO language.

Ừm — đại loại vậy. Kế thừa chẳng qua là việc khai báo lại một nhóm biến và hàm bên trong một phạm vi bao ngoài. Đây là điều mà lập trình viên C đã có thể làm thủ công từ rất lâu trước khi có ngôn ngữ OO.

- 2. For example, Smalltalk, Python, JavaScript, Lua, and Ruby.
 - 2. Chẳng hạn Smalltalk, Python, JavaScript, Lua và Ruby.
- 3. Not just C programmers: Most languages of that era had the capability to masquerade one data structure as another.
 - 3. Không chỉ lập trình viên C: Phần lớn ngôn ngữ thời đó đều có khả năng cho một cấu trúc dữ

liệu giả dạng một cấu trúc khác.

liệu giả dạng một cấu trúc khác.

Consider this addition to our original point.h C program:

Hãy xem phần bổ sung sau đây vào chương trình C point.h ban đầu của chúng ta:

```

namedPoint.h
struct NamedPoint;
struct NamedPoint* makeNamedPoint(double x, double y, char* name);
void setName(struct NamedPoint* np, char* name);
char* getName(struct NamedPoint* np);
namedPoint.c
#include "namedPoint.h"
#include <stdlib.h>
struct NamedPoint {
    double x,y;
    char* name;
};
struct NamedPoint* makeNamedPoint(double x, double y, char* name) {
    struct NamedPoint* p = malloc(sizeof(struct NamedPoint));
    p->x = x;
    p->y = y;
    p->name = name;
    return p;
}
void setName(struct NamedPoint* np, char* name) {
    np->name = name;
}
char* getName(struct NamedPoint* np) {
    return np->name;
}

main.c
#include "point.h"
#include "namedPoint.h"
#include <stdio.h>
int main(int ac, char** av) {
    struct NamedPoint* origin = makeNamedPoint(0.0, 0.0, "origin");
    struct NamedPoint* upperRight = makeNamedPoint
        (1.0, 1.0, "upperRight");
    printf("distance=%f\n",
        distance(
            (struct Point*) origin,
            (struct Point*) upperRight));
}

```

If you look carefully at the main program, you'll see that the NamedPoint data structure acts as though it is a derivative of the Point data structure. This is because the order of the first two fields in NamedPoint is the same as Point . In short, NamedPoint can masquerade as Point because NamedPoint is a pure superset of Point and maintains the ordering of the members that correspond to Point .

Nếu nhìn kỹ chương trình chính, bạn sẽ thấy cấu trúc dữ liệu NamedPoint hành xử như thể nó là một dẫn xuất của cấu trúc dữ liệu Point . Đó là vì thứ tự của hai trường đầu tiên trong NamedPoint giống hệt trong Point . Nói ngắn gọn, NamedPoint có thể giả dạng Point vì NamedPoint là một tập cha thuần túy của Point và giữ nguyên thứ tự các thành viên tương ứng với Point .

This kind of trickery was a common practice of programmers prior to the advent of OO. In fact, such trickery is how C++ implements single inheritance.

Kiểu mảnh khoe này là chuyện thường ngày của lập trình viên trước khi OO ra đời. Thực tế, chính mảnh khoe đó là cách C++ hiện thực kế thừa đơn.

Thus we might say that we had a kind of inheritance long before OO languages were invented. That statement wouldn't quite be true, though. We had a trick, but it's not nearly as convenient as true inheritance. Moreover, multiple inheritance is a considerably more difficult to achieve by such trickery.

Vậy nên ta có thể nói rằng ta đã có một dạng kế thừa từ rất lâu trước khi các ngôn ngữ OO được phát minh. Nhưng phát biểu ấy không hoàn toàn đúng. Ta có một mảnh khoe, còn nó thì kém tiện lợi hơn kế thừa thực thụ rất nhiều. Hơn nữa, đa kế thừa khó đạt được hơn hẳn bằng kiểu mảnh khoe như vậy.

- 4. Indeed it still is.
 - 4. Thực ra đến giờ vẫn thế.

Note also that in `main.c`, I was forced to cast the `NamedPoint` arguments to `Point`. In a real OO language, such upcasting would be implicit.

Cũng lưu ý rằng trong `main.c`, tôi buộc phải ép kiểu các đối số `NamedPoint` thành `Point`. Trong một ngôn ngữ OO thực thụ, việc upcast như thế sẽ là ngầm định.

It's fair to say that while OO languages did not give us something completely brand new, it did make the masquerading of data structures significantly more convenient.

Nói cho công bằng, tuy các ngôn ngữ OO không mang lại cho ta thứ gì hoàn toàn mới mẻ, nhưng chúng đã khiến việc cho cấu trúc dữ liệu giả dạng nhau tiện lợi hơn rất nhiều.

To recap: We can award no point to OO for encapsulation, and perhaps a half-point for inheritance. So far, that's not such a great score.

Tóm lại: Ta không thể cho OO điểm nào về đóng gói, và có lẽ nửa điểm cho kế thừa. Đến giờ, đó chưa phải một bảng điểm đẹp.

But there's one more attribute to consider.

Nhưng còn một đặc tính nữa cần xem xét.

Polymorphism? — Đa hình?

Did we have polymorphic **behavior** before OO languages? Of course we did. Consider this simple C copy program.

Trước khi có các ngôn ngữ OO, chúng ta đã có hành vi đa hình chưa? Tất nhiên là rồi. Hãy xem chương trình copy đơn giản viết bằng C sau đây.

```
#include <stdio.h>
void copy() {
    int c;
    while ((c=getchar()) != EOF)
        putchar(c);
}
```

The function `getchar()` reads from `STDIN`. But which device is `STDIN`? The `putchar()` function writes to `STDOUT`. But which device is that? These functions are polymorphic—their behavior depends on the type of `STDIN` and

Hàm `getchar()` đọc từ `STDIN`. Nhưng `STDIN` là thiết bị nào? Hàm `putchar()` ghi ra `STDOUT`. Mà đó lại là thiết bị nào? Những hàm này có tính đa hình—hành vi của chúng phụ thuộc vào kiểu của `STDIN` và

`STDOUT`.

It's as though `STDIN` and `STDOUT` are Java-style interfaces that have implementations for each device. Of course, there are no interfaces in the example C program—so how does the call to `getchar()` actually get delivered to the device driver that reads the character?

Cứ như thể `STDIN` và `STDOUT` là những interface kiểu Java, có phần hiện thực riêng cho từng thiết bị. Dĩ nhiên trong chương trình C ví dụ chẳng có interface nào cả—vậy làm sao lời gọi `getchar()` thực sự được chuyển tới đúng driver thiết bị làm nhiệm vụ đọc ký tự?

The answer to that question is pretty straightforward. The UNIX operating system requires that every IO device driver provide five standard functions: `open`, `close`, `read`, `write`, and `seek`. The signatures of those functions must be identical for every IO driver.

Câu trả lời cho câu hỏi đó khá đơn giản. Hệ điều hành UNIX yêu cầu mọi driver thiết bị IO phải cung cấp năm hàm chuẩn: `open`, `close`, `read`, `write` và `seek`. Chữ ký của các hàm đó phải giống hệt nhau ở mọi driver IO.

The `FILE` data structure contains five pointers to functions. In our example, it might look like this:

Cấu trúc dữ liệu `FILE` chứa năm con trỏ tới hàm. Trong ví dụ của chúng ta, nó có thể trông như thế này:

```
struct FILE {
    void (*open)(char* name, int mode);
    void (*close)();
    int (*read)();
    void (*write)(char);
    void (*seek)(long index, int mode);
};
```

The IO driver for the console will define those functions and load up a `FILE` data structure with their addresses—something like this:

Driver IO cho console sẽ định nghĩa các hàm đó và nạp địa chỉ của chúng vào một cấu trúc dữ liệu `FILE`—đại loại như thế này:

```
#include "file.h"
void open(char* name, int mode) { /*...*/ }
void close() { /*...*/ };
int read() { int c; /*...*/ return c; }
void write(char c) { /*...*/ }
void seek(long index, int mode) { /*...*/ }
struct FILE console = { open, close, read, write, seek};
```

Now if `STDIN` is defined as a `FILE*`, and if it points to the console data structure, then `getchar()` might be implemented this way:

Bây giờ, nếu `STDIN` được định nghĩa là một `FILE*`, và nếu nó trỏ tới cấu trúc dữ liệu console, thì `getchar()` có thể được hiện thực theo cách này:

- 5. UNIX systems vary; this is just an example.
 - 5. Các hệ thống UNIX có khác nhau; đây chỉ là một ví dụ.

```
extern struct FILE* STDIN;
int getchar() {
    return STDIN->read();
}
```

In other words, `getchar()` simply calls the function pointed to by the `read` pointer of the `FILE` data structure pointed to by `STDIN`.

Nói cách khác, `getchar()` chỉ đơn giản gọi hàm được trỏ tới bởi con trỏ `read` của cấu trúc dữ liệu `FILE` mà `STDIN` đang trỏ tới.

This simple trick is the basis for all polymorphism in OO. In C++, for example, every virtual function within a class has a pointer in a table called a `vtable`, and all calls to virtual functions go through that table. Constructors of derivatives simply load their versions of those functions into the `vtable` of the object being created.

Mánh khóe đơn giản này chính là nền tảng của mọi đa hình trong OO. Chẳng hạn trong C++, mỗi hàm virtual bên trong một class đều có một con trỏ nằm trong một bảng gọi là `vtable`, và mọi lời gọi tới hàm virtual đều đi qua bảng đó. Constructor của các lớp dẫn xuất chỉ việc nạp phiên bản hàm của chúng vào `vtable` của đối tượng đang được tạo.

The bottom line is that polymorphism is an application of pointers to functions. Programmers have been using pointers to functions to achieve polymorphic behavior since Von Neumann architectures were first implemented in the late 1940s. In other words, OO has provided nothing new.

Chung quy lại, đa hình là một ứng dụng của con trỏ hàm. Lập trình viên đã dùng con trỏ hàm để đạt được hành vi đa hình kể từ khi các kiến trúc Von Neumann đầu tiên được hiện thực vào cuối thập niên 1940. Nói cách khác, OO chẳng mang lại gì mới.

Ah, but that's not quite correct. OO languages may not have given us polymorphism, but they have made it much safer and much more convenient.

À, nhưng nói vậy cũng chưa hẳn đúng. Các ngôn ngữ OO có thể không cho ta đa hình, nhưng chúng đã làm cho nó an toàn hơn nhiều và tiện lợi hơn nhiều.

The problem with explicitly using pointers to functions to create polymorphic behavior is that pointers to functions are dangerous. Such use is driven by a set of manual conventions. You have to remember to follow the convention to initialize those pointers. You have to remember to follow the convention to call all your functions through those pointers. If any programmer fails to remember these conventions, the resulting bug can be devilishly hard to track down and eliminate.

Vấn đề của việc dùng con trỏ hàm một cách tường minh để tạo hành vi đa hình là con trỏ hàm rất nguy hiểm. Cách dùng ấy được vận hành bằng một mớ quy ước thủ công. Bạn phải nhớ tuân theo quy ước để khởi tạo các con trỏ đó. Bạn phải nhớ tuân theo quy ước để gọi mọi hàm của mình thông qua các con trỏ đó. Chỉ cần một lập trình viên nào đó quên mất những quy ước này, con bug sinh ra có thể khó truy vết và diệt trừ đến phát điên.

OO languages eliminate these conventions and, therefore, these dangers. Using an OO language makes polymorphism trivial. That fact provides an enormous power that old C programmers could only dream of. On this

Các ngôn ngữ OO loại bỏ những quy ước ấy, và do đó loại bỏ luôn những hiểm họa ấy. Dùng ngôn ngữ OO khiến đa hình trở nên nhẹ tênh. Điều đó mang lại một sức mạnh khổng lồ mà các lập trình viên C đời cũ chỉ dám mơ. Trên cơ

basis, we can conclude that OO imposes discipline on **indirect transfer of control**.

sở đó, ta có thể kết luận rằng OO áp đặt kỷ luật lên việc chuyển điều khiển gián tiếp.

The Power of Polymorphism — Sức mạnh của đa hình

What's so great about polymorphism? To better appreciate its charms, let's reconsider the example copy program. What happens to that program if a new IO device is created? Suppose we want to use the copy program to copy data from a handwriting recognition device to a speech synthesizer device: How do we need to change the copy program to get it to work with those new devices?

Đa hình thì có gì ghê gớm? Để cảm nhận rõ hơn sức quyến rũ của nó, hãy quay lại chương trình copy làm ví dụ. Chuyện gì xảy ra với chương trình đó nếu có một thiết bị IO mới ra đời? Giả sử ta muốn dùng chương trình copy để chép dữ liệu từ một thiết bị nhận dạng chữ viết tay sang một thiết bị tổng hợp giọng nói: Ta phải sửa chương trình copy thế nào để nó chạy được với những thiết bị mới đó?

We don't need any changes at all! Indeed, we don't even need to recompile the copy program. Why? Because the source code of the copy program does not depend on the source code of the IO drivers. As long as those IO drivers implement the five standard functions defined by FILE, the copy program will be happy to use them.

Ta chẳng cần sửa gì cả! Thậm chí còn không cần biên dịch lại chương trình copy. Vì sao? Vì mã nguồn của chương trình copy không phụ thuộc vào mã nguồn của các driver IO. Miễn là các driver IO đó hiện thực năm hàm chuẩn do FILE định nghĩa, chương trình copy sẽ vui vẻ dùng chúng.

In short, the IO devices have become plugins to the copy program.

Nói ngắn gọn, các thiết bị IO đã trở thành plugin của chương trình copy.

Why did the UNIX operating system make IO devices plugins? Because we learned, in the late 1950s, that our programs should be device independent. Why? Because we wrote lots of programs that were device dependent, only to discover that we really wanted those programs to do the same job but use a different device.

Vì sao hệ điều hành UNIX biến các thiết bị IO thành plugin? Vì cuối thập niên 1950 chúng ta đã học được rằng chương trình của mình nên độc lập với thiết bị. Tại sao? Vì chúng ta đã viết vô số chương trình phụ thuộc vào thiết bị, để rồi phát hiện ra rằng thực ra mình muốn những chương trình đó làm đúng công việc ấy nhưng trên một thiết bị khác.

For example, we often wrote programs that read input data from decks of cards, 6 and then punched new decks of cards as output. Later, our customers stopped giving us decks of cards and started giving us reels of magnetic tape. This was very inconvenient, because it meant rewriting large portions of the original program. It would be very convenient if the same program worked interchangeably with cards or tape.

Chẳng hạn, chúng ta thường viết những chương trình đọc dữ liệu đầu vào từ các cỗ bài thể đục lỗ, 6 rồi đục ra những cỗ thẻ mới làm đầu ra. Về sau, khách hàng thôi không đưa cho chúng ta các cỗ thẻ nữa mà bắt đầu đưa các cuộn băng từ. Chuyện này rất bất tiện, vì nó đồng nghĩa với việc phải viết lại phần lớn chương trình gốc. Sẽ tiện biết bao nếu cùng một chương trình chạy được với cả thẻ đục lỗ lẫn băng từ, đổi thứ này lấy thứ kia mà không phải sửa gì.

- 6. Punched cards—IBM Hollerith cards, 80 columns wide. I'm sure many of you have never even seen one

- 6. Thẻ đục lỗ — thẻ IBM Hollerith, rộng 80 cột. Tôi chắc rằng nhiều bạn còn chưa từng nhìn thấy một

of these, but they were commonplace in the 1950s, 1960s, and even 1970s.

tấm nào, nhưng chúng là thứ phổ biến trong các thập niên 1950, 1960, và cả 1970.

The **plugin architecture** was invented to support this kind of IO device independence, and has been implemented in almost every operating system since its introduction. Even so, most programmers did not extend the idea to their own programs, because using pointers to functions was dangerous.

Kiến trúc plugin được phát minh để hỗ trợ kiểu độc lập với thiết bị IO này, và đã được hiện thực trong gần như mọi hệ điều hành kể từ khi nó ra đời. Dù vậy, phần lớn lập trình viên không mở rộng ý tưởng đó sang chương trình của chính mình, bởi dùng con trỏ hàm rất nguy hiểm.

OO allows the plugin architecture to be used anywhere, for anything.

OO cho phép dùng kiến trúc plugin ở bất cứ đâu, cho bất cứ thứ gì.

Dependency Inversion — Đảo ngược phụ thuộc

Imagine what software was like before a safe and convenient mechanism for polymorphism was available. In the typical calling tree, main functions called **high-level** functions, which called mid-level functions, which called **low-level** functions. In that calling tree, however, source code dependencies inexorably followed the **flow of control** (Figure 5.1).

Hãy hình dung phần mềm trông như thế nào trước khi có một cơ chế đa hình an toàn và tiện lợi. Trong cây gọi hàm điển hình, hàm main gọi các hàm mức cao, các hàm này gọi hàm mức trung, rồi đến lượt chúng gọi hàm mức thấp. Tuy nhiên, trong cây gọi hàm đó, phụ thuộc mã nguồn (source code dependency) luôn bám riết theo luồng điều khiển (Figure 5.1).

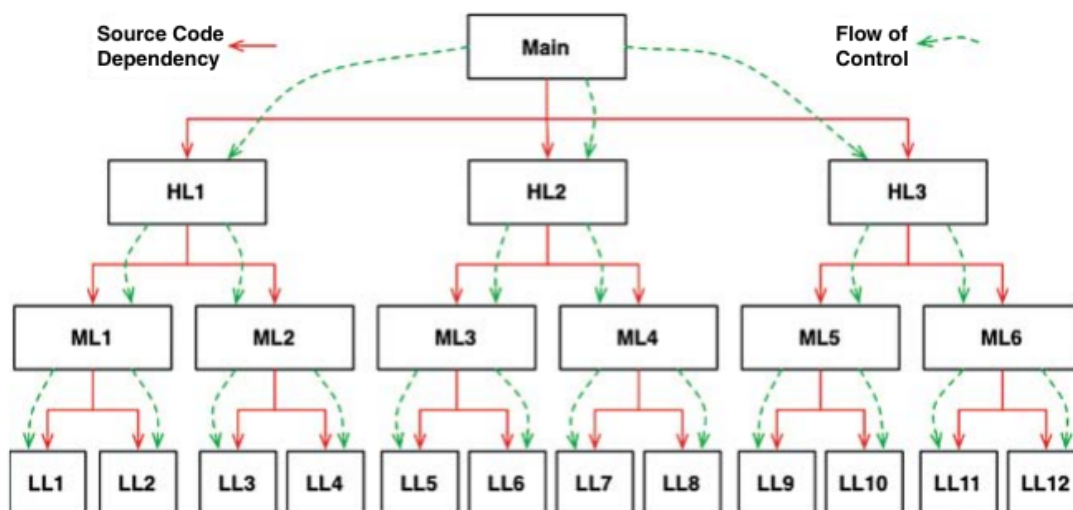


Figure 5.1 Source code dependencies versus flow of control

Figure 5.1 Phụ thuộc mã nguồn so với luồng điều khiển

For main to call one of the high-level functions, it had to mention the name of the module that contained that function. In C, this was a `#include`. In Java, it was an `import` statement. In C#, it was a `using` statement. Indeed, every caller was forced to mention the name of the module that contained the callee.

Để main gọi được một trong các hàm mức cao, nó phải nhắc đến tên của module chứa hàm đó. Trong C, đó là một `#include`. Trong Java, đó là câu lệnh `import`. Trong C#, đó là câu lệnh `using`. Thật vậy, mọi bên gọi đều buộc phải nhắc đến tên của module chứa bên được gọi.

This requirement presented the software architect with few, if any, options. The flow of control was dictated by the behavior of the system, and the source code dependencies were dictated by that flow of control.

Yêu cầu đó khiến kiến trúc sư phần mềm gần như chẳng còn lựa chọn nào. Luồng điều khiển bị quy định bởi hành vi của hệ thống, còn các phụ thuộc mã nguồn thì bị quy định bởi chính luồng điều khiển ấy.

When polymorphism is brought into play, however, something very different can happen (Figure 5.2).

Nhưng khi đa hình được đưa vào cuộc chơi, một chuyện rất khác có thể xảy ra (Figure 5.2).

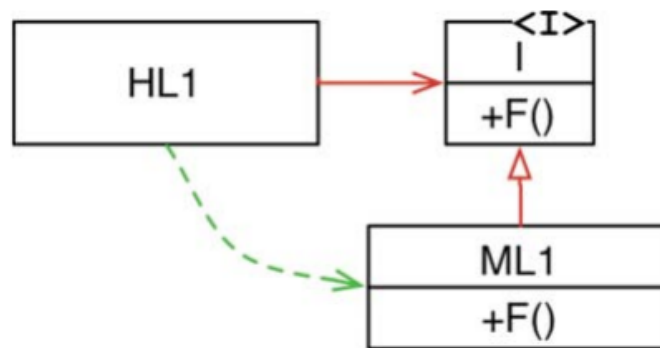


Figure 5.2 **Dependency inversion**

Figure 5.2 Đảo ngược phụ thuộc

In Figure 5.2, module HL1 calls the `F()` function in module ML1. The fact that it calls this function through an **interface** is a source code contrivance. At runtime, the interface doesn't exist. HL1 simply calls `F()` within ML1.

Trong Figure 5.2, module HL1 gọi hàm `F()` trong module ML1. Việc nó gọi hàm này thông qua một interface chỉ là một sắp đặt ở mức mã nguồn. Lúc chạy, interface đó không tồn tại. HL1 chỉ đơn giản gọi `F()` bên trong ML1.

Note, however, that the **source code dependency** (the inheritance relationship) between ML1 and the interface I points in the opposite direction compared to the flow of control. This is called **dependency inversion**, and its implications for the software architect are profound.

Tuy vậy, hãy chú ý rằng phụ thuộc mã nguồn (quan hệ kế thừa) giữa ML1 và interface I chỉ theo hướng ngược lại so với luồng điều khiển. Đây gọi là đảo ngược phụ thuộc (dependency inversion), và hệ quả của nó đối với kiến trúc sư phần mềm là vô cùng sâu sắc.

The fact that OO languages provide safe and convenient polymorphism means that any source code dependency, no matter where it is, can be inverted .

Việc các ngôn ngữ OO cung cấp đa hình an toàn và tiện lợi có nghĩa là bất kỳ phụ thuộc mã nguồn nào, dù nằm ở đâu, cũng có thể bị đảo ngược .

Now look back at that calling tree in Figure 5.1, and its many source code dependencies. Any of those source code dependencies can be turned around by inserting an interface between them.

Giờ hãy nhìn lại cây gọi hàm trong Figure 5.1 cùng vô số phụ thuộc mã nguồn của nó. Bất kỳ phụ thuộc mã nguồn nào trong đó cũng có thể bị xoay ngược lại bằng cách chèn một interface vào giữa.

- 7. Albeit indirectly.
 - 7. Dẫu là một cách gián tiếp.

With this approach, software architects working in systems written in OO languages have absolute control over the direction of all source code dependencies in the system. They are not constrained to align those dependencies with the flow of control. No matter which module does the calling and which module is called, the software architect can point the source code dependency in either direction.

Với cách tiếp cận này, kiến trúc sư phần mềm làm việc trên các hệ thống viết bằng ngôn ngữ OO nắm quyền kiểm soát tuyệt đối đối với hướng của mọi phụ thuộc mã nguồn trong hệ thống. Họ không bị ép phải xếp các phụ thuộc đó cùng chiều với luồng điều khiển. Bất kể module nào gọi và module nào bị gọi, kiến trúc sư phần mềm đều có thể chia phụ thuộc mã nguồn theo hướng nào tùy ý.

That is power! That is the power that OO provides. That's what OO is really all about—at least from the architect's point of view.

Đó mới là quyền lực! Đó chính là quyền lực mà OO mang lại. Đó mới thực sự là ý nghĩa của OO — ít nhất là dưới góc nhìn của kiến trúc sư.

What can you do with that power? As an example, you can rearrange the source code dependencies of your system so that the database and the user interface (UI) depend on the business rules (Figure 5.3), rather than the other way around.

Bạn làm được gì với quyền lực đó? Ví dụ, bạn có thể sắp xếp lại các phụ thuộc mã nguồn của hệ thống sao cho cơ sở dữ liệu và giao diện người dùng (UI) phụ thuộc vào các quy tắc nghiệp vụ (business rule) (Figure 5.3), thay vì ngược lại.



Figure 5.3 The database and the user interface depend on the business rules

Figure 5.3 Cơ sở dữ liệu và giao diện người dùng phụ thuộc vào các quy tắc nghiệp vụ

This means that the UI and the database can be plugins to the business rules. It means that the source code of the business rules never mentions the UI or the database.

Điều này có nghĩa là UI và cơ sở dữ liệu có thể trở thành plugin của các quy tắc nghiệp vụ. Nó có nghĩa là mã nguồn của các quy tắc nghiệp vụ không bao giờ nhắc tới UI hay cơ sở dữ liệu.

As a consequence, the business rules, the UI, and the database can be compiled into three separate components or deployment units (e.g., jar files, DLLs, or Gem files) that have the same dependencies as the source code. The **component** containing the business rules will not depend on the components containing the UI and database.

Hệ quả là các quy tắc nghiệp vụ, UI và cơ sở dữ liệu có thể được biên dịch thành ba component riêng biệt, hay ba đơn vị triển khai riêng biệt (chẳng hạn file jar, DLL hay file Gem), mang đúng những phụ thuộc như ở mã nguồn. Component chứa các quy tắc nghiệp vụ sẽ không phụ thuộc vào các component chứa UI và cơ sở dữ liệu.

In turn, the business rules can be deployed independently of the UI and the database. Changes to the UI or the database need not have any effect on the business rules. Those components can be deployed separately and independently.

Đến lượt mình, các quy tắc nghiệp vụ có thể được triển khai độc lập với UI và cơ sở dữ liệu. Thay đổi ở UI hay cơ sở dữ liệu không nhất thiết ảnh hưởng gì tới các quy tắc nghiệp vụ. Những component ấy có thể được triển khai tách biệt và độc lập.

In short, when the source code in a component changes, only that component needs to be redeployed. This is **independent deployability**.

Nói ngắn gọn, khi mã nguồn trong một component thay đổi, chỉ component đó cần được triển khai lại. Đây chính là khả năng triển khai độc lập (independent deployability).

If the modules in your system can be deployed independently, then they can be developed independently by different teams. That's **independent developability**.

Nếu các module trong hệ thống của bạn có thể được triển khai độc lập, thì chúng cũng có thể được phát triển độc lập bởi các đội khác nhau. Đó là khả năng phát triển độc lập (independent developability).

Conclusion — Kết luận

What is OO? There are many opinions and many answers to this question. To the software architect, however, the answer is clear: OO is the ability, through the use of polymorphism, to gain absolute control over every source code dependency in the system. It allows the architect to create a plugin architecture, in which modules that contain high-level policies are independent of modules that contain low-level details. The low-level details are relegated to plugin modules that can be deployed and developed independently from the modules that contain high-level policies.

OO là gì? Có rất nhiều quan điểm và rất nhiều câu trả lời cho câu hỏi này. Tuy nhiên, với kiến trúc sư phần mềm, câu trả lời là rõ ràng: OO là khả năng, thông qua việc sử dụng đa hình, giành quyền kiểm soát tuyệt đối đối với mọi phụ thuộc mã nguồn trong hệ thống. Nó cho phép kiến trúc sư tạo ra một kiến trúc plugin, trong đó những module chứa chính sách (policy) mức cao độc lập với những module chứa chi tiết (detail) mức thấp. Các chi tiết mức thấp bị đẩy xuống những module plugin, có thể được triển khai và phát triển độc lập với những module chứa chính sách mức cao.

Functional Programming — Lập trình hàm

In many ways, the concepts of **functional programming** predate programming itself. This **paradigm** is strongly based on the λ -calculus invented by Alonzo Church in the 1930s.

Xét trên nhiều phương diện, các khái niệm của lập trình hàm (functional programming) còn có trước cả bản thân việc lập trình. Mô thức (paradigm) này dựa chặt chẽ trên λ -calculus do Alonzo Church phát minh vào thập niên 1930.

Squares of Integers — Bình phương của các số nguyên

To explain what functional programming is, it's best to examine some examples. Let's investigate a simple problem: printing the squares of the first 25 integers.

Muốn giải thích lập trình hàm là gì, tốt nhất hãy xem vài ví dụ. Ta thử khảo sát một bài toán đơn giản: in ra bình phương của 25 số nguyên đầu tiên.

In a language like Java, we might write the following:

Với một ngôn ngữ như Java, ta có thể viết thế này:

```
public class Squint {  
    public static void main(String args[]) {  
        for (int i=0; i<25; i++)  
            System.out.println(i*i);  
    }  
}
```

In a language like Clojure, which is a derivative of Lisp, and is functional, we might implement this same program as follows:

Với một ngôn ngữ như Clojure — một biến thể của Lisp và là ngôn ngữ hàm — ta có thể cài đặt đúng chương trình đó như sau:

```
(println (take 25 (map (fn [x] (* x x)) (range))))
```

If you don't know Lisp, then this might look a little strange. So let me reformat it a bit and add some comments.

Nếu bạn không biết Lisp thì đoạn này trông có vẻ hơi lạ. Vậy để tôi trình bày lại cho dễ nhìn và thêm vài chú thích.

```
(println ; _____ Print
```

```
(take 25 ; _____ the first 25
  (map (fn [x] (* x x)) ;_ squares
    (range))) ; _____ of Integers
```

It should be clear that `println`, `take`, `map`, and `range` are all functions. In Lisp, you call a function by putting it in parentheses. For example, `(range)` calls the range function.

Dễ thấy rằng `println`, `take`, `map` và `range` đều là hàm. Trong Lisp, bạn gọi một hàm bằng cách đặt nó vào trong cặp ngoặc đơn. Chẳng hạn, `(range)` là lời gọi hàm `range`.

The expression `(fn [x] (* x x))` is an anonymous function that calls the multiply function, passing its input argument in twice. In other words, it computes the square of its input.

Biểu thức `(fn [x] (* x x))` là một hàm vô danh, nó gọi hàm nhân và truyền vào đối số đầu vào của mình hai lần. Nói cách khác, nó tính bình phương của đầu vào.

Looking at the whole thing again, it's best to start with the innermost function call.

Nhìn lại toàn bộ đoạn mã, tốt nhất là bắt đầu từ lời gọi hàm nằm trong cùng.

- The range function returns a never-ending list of integers starting with 0.
 - Hàm `range` trả về một danh sách số nguyên vô tận bắt đầu từ 0.
- This list is passed into the map function, which calls the anonymous
 - Danh sách này được truyền vào hàm `map`, hàm này gọi hàm bình phương vô danh

squaring function on each element, producing a new never-ending list of all the squares.

trên từng phần tử, sinh ra một danh sách vô tận mới chứa toàn bộ các bình phương.

- The list of squares is passed into the take function, which returns a new
 - Danh sách các bình phương được truyền vào hàm `take`, hàm này trả về một danh sách mới

list with only the first 25 elements.

chỉ gồm 25 phần tử đầu tiên.

- The `println` function prints its input, which is a list of the first 25 squares
 - Hàm `println` in ra đầu vào của nó, tức danh sách bình phương của 25 số nguyên

of integers.

đầu tiên.

If you find yourself terrified by the concept of never-ending lists, don't worry. Only the first 25 elements of those never-ending lists are actually created. That's because no element of a never-ending list is evaluated until it is accessed.

Nếu khái niệm danh sách vô tận làm bạn hoảng, đừng lo. Chỉ 25 phần tử đầu tiên của những danh sách vô tận ấy thực sự được tạo ra. Lý do là không phần tử nào của một danh sách vô tận được tính giá trị cho đến khi nó được truy cập.

If you found all of that confusing, then you can look forward to a glorious time learning all about Clojure and functional programming. It is not my goal to teach you about these topics here.

Nếu tất cả những điều trên khiến bạn rối trí thì hãy chờ đón một quãng thời gian tuyệt vời để học về Clojure và lập trình hàm. Mục tiêu của tôi ở đây không phải là dạy bạn những chủ đề đó.

Instead, my goal here is to point out something very dramatic about the difference between the Clojure and Java programs. The Java program uses a **mutable** variable—a variable that changes state during the execution of the program. That variable is `i`—the loop control variable. No such mutable variable exists in the Clojure program. In the Clojure program, variables like `x` are initialized, but they are never modified.

Thay vào đó, mục tiêu của tôi là chỉ ra một điểm khác biệt hết sức đáng chú ý giữa chương trình Clojure và chương trình Java. Chương trình Java dùng một biến khả biến (mutable)—một biến thay đổi trạng thái trong lúc chương trình chạy. Biến đó là `i`, biến điều khiển vòng lặp. Chương trình Clojure không hề có biến khả biến nào như vậy. Trong chương trình Clojure, những biến như `x` được khởi tạo, nhưng chúng không bao giờ bị sửa đổi.

This leads us to a surprising statement: Variables in functional languages do not vary .

Điều này dẫn ta tới một khẳng định đáng ngạc nhiên: *biến* trong các ngôn ngữ hàm *không* hề *biến đổi*.

Immutability and Architecture — Tính bất biến và kiến trúc

Why is this point **important** as an architectural consideration? Why would an architect be concerned with the mutability of variables? The answer is absurdly simple: All race conditions, deadlock conditions, and concurrent update problems are due to mutable variables. You cannot have a race condition or a concurrent update problem if no variable is ever updated. You cannot have deadlocks without mutable locks.

Vì sao điểm này lại quan trọng khi xét trên phương diện kiến trúc (architecture)? Vì sao một kiến trúc sư lại phải bận tâm tới chuyện biến có khả biến hay không? Câu trả lời đơn giản đến mức phi lý: mọi race condition, mọi deadlock và mọi vấn đề cập nhật đồng thời đều bắt nguồn từ biến khả biến. Bạn không thể gặp race condition hay vấn đề cập nhật đồng thời nếu chẳng có biến nào từng bị cập nhật. Bạn không thể gặp deadlock nếu không có khoá khả biến.

In other words, all the problems that we face in concurrent applications—all the problems we face in applications that require multiple threads, and multiple processors—cannot happen if there are no mutable variables.

Nói cách khác, tất cả những vấn đề ta đối mặt trong các ứng dụng chạy đồng thời — tất cả những vấn đề trong các ứng dụng đòi hỏi nhiều luồng, nhiều bộ xử lý — đều không thể xảy ra nếu không có biến khả biến.

As an architect, you should be very interested in issues of concurrency. You want to make sure that the systems you **design** will be robust in the presence of multiple threads and processors. The question you must be asking yourself, then, is whether **immutability** is practicable.

Là kiến trúc sư, bạn nên rất quan tâm đến các vấn đề đồng thời. Bạn muốn chắc chắn rằng những hệ thống mình thiết kế sẽ vững vàng khi có nhiều luồng và nhiều bộ xử lý. Vậy câu hỏi bạn phải tự đặt ra là: tính bất biến (immutability) có khả thi trên thực tế hay không?

The answer to that question is affirmative, if you have infinite storage and infinite processor speed. Lacking those infinite resources, the answer is a bit more nuanced. Yes, immutability can be practicable, if certain compromises are made.

Câu trả lời là có, nếu bạn có dung lượng lưu trữ vô hạn và tốc độ xử lý vô hạn. Thiếu những tài nguyên vô hạn đó, câu trả lời phải dè dặt hơn một chút. Đúng, tính bất biến có thể khả thi, nếu ta chấp nhận một số thỏa hiệp nhất định.

Let's look at some of those compromises.

Hãy xem vài thoả hiệp trong số đó.

Segregation of Mutability — Tách riêng phần khả biến

One of the most common compromises in regard to immutability is to segregate the application, or the services within the application, into mutable and **immutable** components. The immutable components perform their tasks in a purely functional way, without using any mutable variables. The immutable components communicate with one or more other components that are not purely functional, and allow for the state of variables to be mutated (Figure 6.1).

Một trong những thoả hiệp phổ biến nhất liên quan tới tính bất biến là tách ứng dụng, hoặc tách các dịch vụ (service) bên trong ứng dụng, thành các component khả biến và bất biến. Các component bất biến thực hiện nhiệm vụ của mình theo lối thuần hàm, không dùng bất kỳ biến khả biến nào. Chúng giao tiếp với một hoặc nhiều component khác vốn không thuần hàm và cho phép trạng thái của biến bị thay đổi (Figure 6.1).

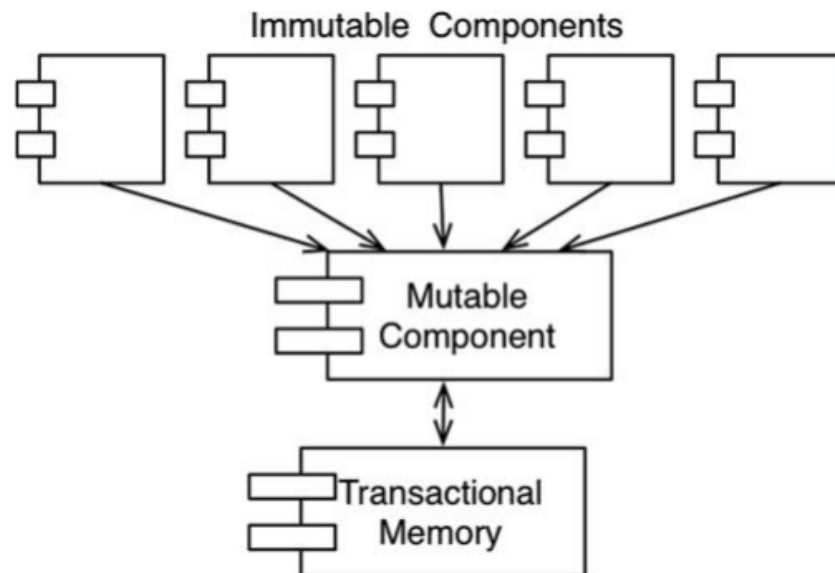


Figure 6.1 Mutating state and transactional memory

Figure 6.1 Trạng thái khả biến và bộ nhớ giao dịch

Since mutating state exposes those components to all the problems of concurrency, it is common practice to use some kind of transactional memory to protect the mutable variables from concurrent updates and race conditions.

Vì việc làm thay đổi trạng thái khiến các component đó phơi mình trước mọi vấn đề của lập trình đồng thời, thông lệ phổ biến là dùng một dạng bộ nhớ giao dịch (transactional memory) nào đó để bảo vệ biến khả biến khỏi cập nhật đồng thời và race condition.

Transactional memory simply treats variables in memory the same way a database treats records on disk. 1 It protects those variables with a transaction- or retry-based scheme.

Bộ nhớ giao dịch đơn giản là đối xử với biến trong bộ nhớ y hệt cách cơ sở dữ liệu đối xử với bản ghi trên đĩa.¹ Nó bảo vệ những biến ấy bằng một cơ chế dựa trên giao dịch hoặc dựa trên việc thử lại.

A simple example of this approach is Clojure's atom facility:

Một ví dụ đơn giản cho cách tiếp cận này là cơ chế atom của Clojure:

```
(def counter (atom 0)) ; initialize counter to 0
(swap! counter inc)    ; safely increment counter.
```

In this code, the counter variable is defined as an atom. In Clojure, an atom is a special kind of variable whose value is allowed to mutate under very disciplined conditions that are enforced by the swap! function.

Trong đoạn mã này, biến counter được định nghĩa là một atom. Trong Clojure, atom là một loại biến đặc biệt: giá trị của nó được phép thay đổi, nhưng chỉ dưới những điều kiện hết sức kỷ luật do hàm swap! áp đặt.

The swap! function, shown in the preceding code, takes two arguments: the atom to be mutated, and a function that computes the new value to be stored

Hàm swap! trong đoạn mã trên nhận hai đối số: atom cần thay đổi, và một hàm tính ra giá trị mới sẽ được lưu

- 1. I know... What's a disk?
 - 1. Tôi biết mà... Đĩa là cái gì cơ?

in the atom. In our example code, the counter atom will be changed to the value computed by the inc function, which simply increments its argument.

vào atom. Trong đoạn mã ví dụ, atom counter sẽ được đổi thành giá trị do hàm inc tính ra — hàm này chỉ đơn giản tăng đối số của nó lên một.

The strategy used by swap! is a traditional compare and swap algorithm. The value of counter is read and passed to inc. When inc returns, the value of counter is locked and compared to the value that was passed to inc. If the value is the same, then the value returned by inc is stored in counter and the lock is released. Otherwise, the lock is released, and the strategy is retried from the beginning.

Chiến lược mà swap! dùng là thuật toán compare and swap cổ điển. Giá trị của counter được đọc ra rồi truyền cho inc. Khi inc trả về, giá trị của counter bị khoá lại và đem so với giá trị đã truyền cho inc. Nếu hai giá trị bằng nhau thì giá trị do inc trả về được ghi vào counter và khoá được nhả. Ngược lại, khoá được nhả và chiến lược này được thử lại từ đầu.

The atom facility is adequate for simple applications. Unfortunately, it cannot completely safeguard against concurrent updates and deadlocks when multiple dependent variables come into play. In those instances, more elaborate facilities can be used.

Cơ chế atom là đủ dùng cho những ứng dụng đơn giản. Đáng tiếc, nó không thể bảo vệ trọn vẹn trước cập nhật đồng thời và deadlock khi có nhiều biến phụ thuộc lẫn nhau tham gia. Trong những tình huống ấy, ta có thể dùng các cơ chế công phu hơn.

The point is that well-structured applications will be segregated into those components that do not mutate variables and those that do. This kind of segregation is supported by the use of appropriate disciplines to protect those mutated variables.

Điểm mấu chốt là: một ứng dụng có cấu trúc tốt sẽ được tách thành những component không làm thay đổi biến và những component có làm thay đổi biến. Kiểu tách rời này được hỗ trợ bằng cách áp dụng các kỹ thuật thích hợp để bảo vệ những biến bị thay đổi đó.

Architects would be wise to push as much processing as possible into the immutable components, and to drive as much code as possible out of those components that must allow mutation.

Kiến trúc sư khôn ngoan sẽ đẩy càng nhiều phần xử lý càng tốt vào các component bất biến, và đẩy càng nhiều mã càng tốt ra khỏi những component buộc phải cho phép thay đổi.

Event Sourcing — Event Sourcing

The limits of storage and processing power have been rapidly receding from view. Nowadays it is common for processors to execute billions of instructions per second and to have billions of bytes of RAM. The more memory we have, and the faster our machines are, the less we need mutable state.

Giới hạn về dung lượng lưu trữ và năng lực xử lý đang lùi xa khỏi tầm mắt với tốc độ chóng mặt. Ngày nay, chuyện bộ xử lý chạy hàng tỉ lệnh mỗi giây và máy có hàng tỉ byte RAM là bình thường. Ta càng có nhiều bộ nhớ, máy càng nhanh, thì ta càng ít cần đến trạng thái khả biến.

As a simple example, imagine a banking application that maintains the account balances of its customers. It mutates those balances when deposit and withdrawal transactions are executed.

Lấy một ví dụ đơn giản, hãy hình dung một ứng dụng ngân hàng lưu số dư tài khoản của khách hàng. Nó thay đổi các số dư đó mỗi khi có giao dịch gửi tiền hay rút tiền được thực hiện.

Now imagine that instead of storing the account balances, we store only the transactions. Whenever anyone wants to know the balance of an account, we simply add up all the transactions for that account, from the beginning of time. This scheme requires no mutable variables.

Giờ hãy hình dung rằng thay vì lưu số dư tài khoản, ta chỉ lưu các giao dịch. Khi có ai đó muốn biết số dư của một tài khoản, ta chỉ việc cộng dồn mọi giao dịch của tài khoản ấy, tính từ thuở khai thiên lập địa. Cách này không cần một biến khả biến nào.

Obviously, this approach sounds absurd. Over time, the number of transactions would grow without bound, and the processing power required to compute the totals would become intolerable. To make this scheme work forever, we would need infinite storage and infinite processing power.

Rõ ràng, cách tiếp cận đó nghe thật phi lý. Theo thời gian, số giao dịch sẽ tăng lên vô hạn, và năng lực xử lý cần để tính tổng sẽ trở nên không thể chịu nổi. Muốn cơ chế này chạy được mãi mãi, ta sẽ cần dung lượng lưu trữ vô hạn và năng lực xử lý vô hạn.

But perhaps we don't have to make the scheme work forever. And perhaps we have enough storage and enough processing power to make the scheme work for the reasonable lifetime of the application.

Nhưng có lẽ ta không cần nó chạy được mãi mãi. Và có lẽ ta có đủ dung lượng lưu trữ cùng đủ năng lực xử lý để cơ chế này chạy được suốt vòng đời hợp lý của ứng dụng.

This is the idea behind **event sourcing**. 2 Event sourcing is a strategy wherein we store the transactions, but not the state. When state is required, we simply apply all the transactions from the beginning of time.

Đó chính là ý tưởng đằng sau event sourcing.² Event sourcing là chiến lược trong đó ta lưu các giao dịch chứ không lưu trạng thái. Khi cần trạng thái, ta chỉ việc áp dụng lại toàn bộ giao dịch từ thuở khai thiên lập địa.

Of course, we can take shortcuts. For example, we can compute and save the state every midnight. Then, when the state information is required, we need compute only the transactions since midnight.

Dĩ nhiên, ta có thể đi đường tắt. Chẳng hạn, ta có thể tính rồi lưu trạng thái vào mỗi nửa đêm. Sau đó, khi cần thông tin trạng thái, ta chỉ phải tính các giao dịch phát sinh từ nửa đêm tới giờ.

Now consider the data storage required for this scheme: We would need a lot of it. Realistically, offline data storage has been growing so fast that we now consider trillions of bytes to be small—so we have a lot of it.

Giờ hãy nghĩ tới dung lượng lưu trữ mà cơ chế này đòi hỏi: ta sẽ cần rất nhiều. Trên thực tế, dung lượng lưu trữ ngoại tuyến đã tăng nhanh đến mức bây giờ ta coi hàng nghìn tỉ byte là nhỏ — nên ta có rất nhiều.

More importantly, nothing ever gets deleted or updated from such a data store. As a consequence, our applications are not CRUD; they are just CR. Also, because neither updates nor deletions occur in the data store, there cannot be any concurrent update issues.

Quan trọng hơn, không có gì trong kho dữ liệu ấy bị xoá hay bị cập nhật. Hệ quả là ứng dụng của ta không phải CRUD; nó chỉ là CR. Ngoài ra, vì kho dữ liệu chẳng bao giờ có thao tác cập nhật hay xoá, nên không thể phát sinh vấn đề cập nhật đồng thời nào.

If we have enough storage and enough processor power, we can make our applications entirely immutable—and, therefore, entirely functional.

Nếu có đủ dung lượng lưu trữ và đủ năng lực xử lý, ta có thể làm cho ứng dụng của mình hoàn toàn bất biến — và do đó, *hoàn toàn theo lối hàm*.

If this still sounds absurd, it might help if you remembered that this is precisely the way your source code control system works.

Nếu điều này nghe vẫn còn phi lý, có lẽ sẽ dễ chấp nhận hơn khi bạn nhớ rằng đây chính xác là cách hệ thống quản lý mã nguồn của bạn vận hành.

- 2. Thanks to Greg Young for teaching me about this concept.
 - 2. Cảm ơn Greg Young đã dạy tôi về khái niệm này.

Conclusion — Kết luận

To summarize:

Tóm lại:

- **Structured programming** is discipline imposed upon direct transfer
 - Lập trình có cấu trúc (structured programming) là kỷ luật áp đặt lên việc chuyển điều khiển

of control.

trực tiếp.

- **Object-oriented programming** is discipline imposed upon indirect transfer

- Lập trình hướng đối tượng (OO) là kỷ luật áp đặt lên việc chuyển điều khiển of control.

gián tiếp.

- Functional programming is discipline imposed upon variable **assignment**.

- Lập trình hàm là kỷ luật áp đặt lên phép gán (assignment) cho biến.

Each of these three paradigms has taken something away from us. Each restricts some aspect of the way we write code. None of them has added to our power or our capabilities.

Mỗi mô thức trong ba mô thức này đều lấy đi của ta một thứ gì đó. Mỗi mô thức đều hạn chế một khía cạnh nào đó trong cách ta viết mã. Không mô thức nào tăng thêm quyền năng hay khả năng cho ta cả.

What we have learned over the last half-century is what not to do .

Điều ta học được suốt nửa thế kỷ qua là *những gì không nên làm*.

With that realization, we have to face an unwelcome fact: Software is not a rapidly advancing technology. The rules of software are the same today as they were in 1946, when Alan Turing wrote the very first code that would execute in an electronic computer. The tools have changed, and the hardware has changed, but the essence of software remains the same.

Với nhận thức đó, ta phải đối diện một sự thật chẳng dễ chịu: phần mềm không phải là một công nghệ tiến bộ nhanh. Các quy tắc của phần mềm ngày nay vẫn y như hồi năm 1946, khi Alan Turing viết những dòng mã đầu tiên chạy được trên một máy tính điện tử. Công cụ đã đổi, phần cứng đã đổi, nhưng bản chất của phần mềm thì vẫn thế.

Software—the stuff of computer programs—is composed of sequence, selection, iteration, and indirection. Nothing more. Nothing less.

Phần mềm — thứ làm nên các chương trình máy tính — được cấu thành từ tuần tự (sequence), chọn (selection), lặp (iteration) và gián tiếp (indirection). Không hơn. Không kém.

Design Principles — Các nguyên tắc thiết kế

Good software systems begin with clean code. On the one hand, if the bricks aren't well made, the **architecture** of the building doesn't matter much. On the other hand, you can make a substantial **mess** with well-made bricks. This is where **the SOLID principles** come in.

Hệ thống phần mềm tốt bắt đầu từ mã sạch. Một mặt, nếu từng viên gạch được nung kém thì kiến trúc của toà nhà chẳng còn mấy ý nghĩa. Mặt khác, bạn vẫn có thể tạo ra một mớ hỗn độn ra trò bằng những viên gạch tốt. Đây chính là chỗ các nguyên tắc SOLID bước vào.

The SOLID principles tell us how to arrange our functions and data structures into classes, and how those classes should be interconnected. The use of the word “class” does not imply that these principles are applicable only to object-oriented software. A class is simply a coupled grouping of functions and data. Every software system has such groupings, whether they are called classes or not. The SOLID principles apply to those groupings.

Các nguyên tắc SOLID cho ta biết cách sắp xếp hàm và cấu trúc dữ liệu vào các class, và các class ấy nên được nối với nhau ra sao. Việc dùng chữ “class” không hàm ý rằng những nguyên tắc này chỉ áp dụng cho phần mềm hướng đối tượng. Một class đơn giản là một nhóm hàm và dữ liệu gắn với nhau. Hệ thống phần mềm nào cũng có những nhóm như vậy, dù chúng có được gọi là class hay không. Các nguyên tắc SOLID áp dụng cho những nhóm đó.

The goal of the principles is the creation of mid-**level** software structures that:

Mục tiêu của các nguyên tắc này là tạo ra những cấu trúc phần mềm ở mức trung gian sao cho chúng:

- Tolerate change,
 - Chịu được thay đổi,
- Are easy to understand, and
 - Dễ hiểu, và
- Are the basis of components that can be used in many software systems.
 - Là nền tảng cho các component có thể dùng lại trong nhiều hệ thống phần mềm.

The term “mid-level” refers to the fact that these principles are applied by programmers working at the module level. They are applied just above the level of the code and help to define the kinds of software structures used within modules and components.

Cụm “mức trung gian” (mid-level) ám chỉ việc các nguyên tắc này được lập trình viên áp dụng ở cấp module. Chúng được áp dụng ngay bên trên cấp mã nguồn, và giúp định hình những kiểu cấu trúc phần mềm được dùng bên trong các module và component.

Just as it is possible to create a substantial mess with well-made bricks, so it is also possible to create a system-wide mess with well-designed mid-level components. For this reason, once we have covered the SOLID principles, we will move on to their counterparts in the **component** world, and then to the principles of **high-level** architecture.

Cũng như ta có thể tạo ra một mớ hỗn độn ra trò bằng những viên gạch tốt, ta hoàn toàn có thể tạo ra một mớ hỗn độn ở quy mô toàn hệ thống bằng những component mức trung gian được thiết kế tử tế. Vì lẽ đó, sau khi bàn xong các nguyên tắc SOLID, ta sẽ chuyển sang những nguyên tắc tương ứng trong thế giới component, rồi tới các nguyên tắc của kiến trúc mức cao.

The history of the SOLID principles is long. I began to assemble them in the late 1980s while debating software **design** principles with others on USENET (an early kind of Facebook). Over the years, the principles have shifted and changed. Some were deleted. Others were merged. Still others were added. The final grouping stabilized in the early 2000s, although I presented them in a different order.

Lịch sử của các nguyên tắc SOLID khá dài. Tôi bắt đầu gom góp chúng vào cuối thập niên 1980, trong lúc tranh luận về các nguyên tắc thiết kế phần mềm với những người khác trên USENET (một dạng Facebook thời sơ khai). Qua năm tháng, các nguyên tắc dịch chuyển và đổi thay. Có cái bị bỏ đi. Có cái được gộp lại. Lại có cái được thêm vào. Nhóm cuối cùng ổn định vào đầu những năm 2000, dù khi ấy tôi trình bày chúng theo một thứ tự khác.

In 2004 or thereabouts, Michael Feathers sent me an email saying that if I rearranged the principles, their first words would spell the word SOLID—and thus the SOLID principles were born.

Khoảng năm 2004, Michael Feathers gửi cho tôi một email nói rằng nếu tôi sắp xếp lại các nguyên tắc thì chữ cái đầu của chúng sẽ ghép thành từ SOLID — và thế là các nguyên tắc SOLID ra đời.

The chapters that follow describe each principle more thoroughly. Here is the executive summary:

Các chương tiếp theo sẽ mô tả từng nguyên tắc kỹ lưỡng hơn. Còn đây là bản tóm lược:

- SRP: The Single Responsibility Principle
 - SRP: Nguyên tắc Trách nhiệm Đơn nhất (The Single Responsibility Principle)

An active corollary to Conway’s law: The best **structure** for a software system is heavily influenced by the social structure of the organization that uses it so that each software module has one, and only one, reason to change.

Một hệ quả sống động của định luật Conway: Cấu trúc tốt nhất cho một hệ thống phần mềm chịu ảnh hưởng nặng nề từ cấu trúc xã hội của tổ chức sử dụng nó, sao cho mỗi module phần mềm có một, và chỉ một, lý do để thay đổi.

- OCP: The Open-Closed Principle
 - OCP: Nguyên tắc Đóng-Mở (The Open-Closed Principle)

Bertrand Meyer made this principle famous in the 1980s. The gist is that for software systems to be easy to change, they must be designed to allow the **behavior** of those systems to be changed by adding new code, rather than changing existing code.

Bertrand Meyer đã làm nguyên tắc này nổi tiếng vào thập niên 1980. Đại ý là: để hệ thống phần mềm dễ thay đổi, chúng phải được thiết kế sao cho hành vi (behavior) của hệ thống có thể thay đổi bằng cách thêm mã mới, chứ không phải sửa mã đang có.

- LSP: The Liskov Substitution Principle
 - LSP: Nguyên tắc Thay thế Liskov (The Liskov Substitution Principle)

Barbara Liskov's famous definition of subtypes, from 1988. In short, this principle says that to build software systems from interchangeable parts, those parts must adhere to a contract that allows those parts to be substituted one for another.

Định nghĩa nổi tiếng về kiểu con của Barbara Liskov, năm 1988. Nói ngắn gọn, nguyên tắc này bảo rằng để dựng hệ thống phần mềm từ những bộ phận thay thế lẫn nhau được, các bộ phận ấy phải tuân theo một hợp đồng cho phép chúng được thay thế cho nhau.

- ISP: The **Interface** Segregation Principle
 - ISP: Nguyên tắc Phân tách Interface (The Interface Segregation Principle)

This principle advises software designers to avoid depending on things that they don't use.

Nguyên tắc này khuyên người thiết kế phần mềm tránh phụ thuộc vào những thứ mình không dùng tới.

- DIP: The **Dependency Inversion** Principle
 - DIP: Nguyên tắc Đảo ngược Phụ thuộc (The Dependency Inversion Principle)

The code that implements high-level **policy** should not depend on the code that implements **low-level** details. Rather, details should depend on policies.

Mã hiện thực chính sách (policy) mức cao không nên phụ thuộc vào mã hiện thực chi tiết (detail) mức thấp. Ngược lại mới đúng: chi tiết phải phụ thuộc vào chính sách.

These principles have been described in **detail** in many different publications over the years. The chapters that follow will focus on the architectural implications of these principles instead of repeating those detailed discussions. If you are not already familiar with these principles, what follows is insufficient to understand them in detail and you would be well advised to study them in the footnoted documents.

Suốt nhiều năm qua, các nguyên tắc này đã được mô tả rất chi tiết trong vô số ấn phẩm khác nhau ¹. Các chương tiếp theo sẽ tập trung vào hệ quả kiến trúc của chúng thay vì lặp lại những cuộc thảo luận chi tiết ấy. Nếu bạn chưa quen với các nguyên tắc này thì phần sau đây không đủ để hiểu chúng một cách tường tận, và bạn nên tìm đọc những tài liệu ghi trong phần chú thích.

- 1. For example, Agile Software Development, Principles, Patterns, and Practices , Robert C. Martin,
 - 1. Ví dụ, Agile Software Development, Principles, Patterns, and Practices , Robert C. Martin,

Prentice Hall, 2002, <http://www.butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod>, and https://en.wikipedia.org/wiki/object-oriented_design) (or just google SOLID).

Prentice Hall, 2002, <http://www.butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod>, và [https://en.wikipedia.org/wiki/SOLID_\(object-oriented_design\)](https://en.wikipedia.org/wiki/SOLID_(object-oriented_design)) (hoặc chỉ cần google SOLID).

SRP: The Single Responsibility Principle — SRP: Nguyên tắc Trách nhiệm Đơn nhất

Of all **the SOLID principles**, the Single Responsibility Principle (SRP) might be the least well understood. That's likely because it has a particularly inappropriate name. It is too easy for programmers to hear the name and then assume that it means that every module should do just one thing.

Trong tất cả các nguyên tắc SOLID, SRP — Nguyên tắc Trách nhiệm Đơn nhất (Single Responsibility Principle) có lẽ là nguyên tắc bị hiểu sai nhiều nhất. Nhiều khả năng là vì nó mang một cái tên đặc biệt dễ gây hiểu lầm. Lập trình viên nghe cái tên đó rồi mặc định rằng nó có nghĩa là mỗi module chỉ nên làm đúng một việc — chuyện đó xảy ra quá dễ.

Make no mistake, there is a principle like that. A function should do one, and only one, thing. We use that principle when we are refactoring large functions into smaller functions; we use it at the lowest levels. But it is not one of the SOLID principles—it is not the SRP.

Đừng nhầm, đúng là có một nguyên tắc như vậy. Một hàm nên làm một, và chỉ một việc. Chúng ta dùng nguyên tắc đó khi tái cấu trúc các hàm lớn thành những hàm nhỏ hơn; chúng ta dùng nó ở mức thấp nhất. Nhưng đó không phải là một trong các nguyên tắc SOLID — đó không phải SRP.

Historically, the SRP has been described this way:

Về mặt lịch sử, SRP được mô tả như sau:

A module should have one, and only one, reason to change.

Một module chỉ nên có một, và chỉ một, lý do để thay đổi.

Software systems are changed to satisfy users and stakeholders; those users and stakeholders are the “reason to change” that the principle is talking about. Indeed, we can rephrase the principle to say this:

Hệ thống phần mềm bị thay đổi là để làm hài lòng người dùng và các bên liên quan (stakeholder); chính những người dùng và bên liên quan đó mới là “lý do để thay đổi” mà nguyên tắc đang nói tới. Thật vậy, ta có thể diễn đạt lại nguyên tắc như sau:

A module should be responsible to one, and only one, user or **stakeholder**.

Một module chỉ nên chịu trách nhiệm trước một, và chỉ một, người dùng hoặc bên liên quan.

Unfortunately, the words “user” and “stakeholder” aren’t really the right words to use here. There will likely be more than one user or stakeholder who wants the system changed in the same way. Instead, we’re really referring to a group—one or more people who require that change. We’ll refer to that group as an **actor**.

Tiếc thay, “người dùng” và “bên liên quan” chưa phải là những từ thật sự chuẩn xác ở đây. Rất có thể sẽ có nhiều hơn một người dùng hoặc bên liên quan cùng muốn hệ thống thay đổi theo cùng một hướng. Thứ ta thực sự muốn nói đến là một nhóm — một hoặc nhiều người cùng yêu cầu thay đổi đó. Ta sẽ gọi nhóm ấy là một actor.

Thus the final version of the SRP is:

Vậy nên phiên bản cuối cùng của SRP là:

A module should be responsible to one, and only one, actor.

Một module chỉ nên chịu trách nhiệm trước một, và chỉ một, actor.

Now, what do we mean by the word “module”? The simplest definition is just a source file. Most of the time that definition works fine. Some languages and development environments, though, don’t use source files to contain their code. In those cases a module is just a cohesive set of functions and data structures.

Bây giờ, “module” nghĩa là gì? Định nghĩa đơn giản nhất: đó là một tệp mã nguồn. Phần lớn thời gian, định nghĩa ấy dùng tốt. Tuy nhiên, một số ngôn ngữ và môi trường phát triển không đựng mã của chúng trong các tệp mã nguồn. Trong những trường hợp đó, module chỉ đơn giản là một tập hợp gắn kết gồm các hàm và cấu trúc dữ liệu.

That word “cohesive” implies the SRP. Cohesion is the force that binds together the code responsible to a single actor.

Chính chữ “gắn kết” đã hàm ý SRP. Độ gắn kết là lực gắn kết với nhau phần mã cùng chịu trách nhiệm trước một actor duy nhất.

Perhaps the best way to understand this principle is by looking at the symptoms of violating it.

Có lẽ cách hiểu nguyên tắc này tốt nhất là nhìn vào các triệu chứng khi vi phạm nó.

Symptom 1: Accidental Duplication — Triệu chứng 1: Trùng lặp tình cờ

My favorite example is the Employee class from a payroll application. It has three methods: `calculatePay()`, `reportHours()`, and `save()` (Figure 7.1).

Ví dụ tôi thích nhất là class Employee trong một ứng dụng tính lương. Nó có ba phương thức: `calculatePay()`, `reportHours()` và `save()` (Figure 7.1).

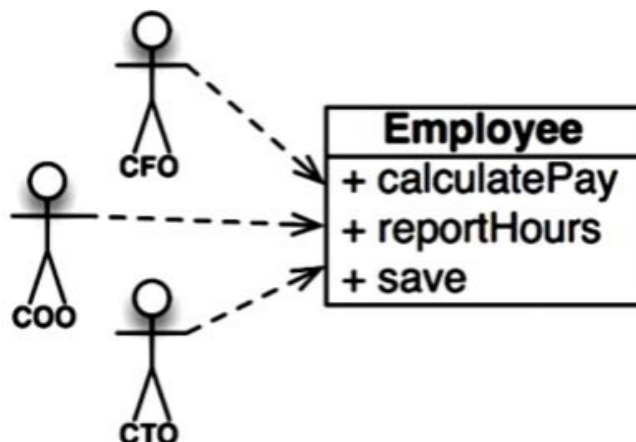


Figure 7.1 The Employee class

Figure 7.1 Class Employee

This class violates the SRP because those three methods are responsible to three very different actors.

Class này vi phạm SRP, bởi ba phương thức đó chịu trách nhiệm trước ba actor hoàn toàn khác nhau.

- The calculatePay() method is specified by the accounting department,
 - Phương thức calculatePay() do phòng kế toán đặc tả,

which reports to the CFO.

phòng này báo cáo lên CFO.

- The reportHours() method is specified and used by the **human resources**
 - Phương thức reportHours() do phòng nhân lực đặc tả và sử dụng,

department, which reports to the COO.

phòng này báo cáo lên COO.

- The save() method is specified by the database administrators (DBAs),
 - Phương thức save() do các quản trị viên cơ sở dữ liệu (DBA) đặc tả,

who report to the CTO.

họ báo cáo lên CTO.

By putting the source code for these three methods into a single Employee class, the developers have coupled each of these actors to the others. This

Bằng cách nhét mã nguồn của cả ba phương thức này vào chung một class Employee, các lập trình viên đã buộc từng actor kết dính với những actor còn lại. Sự

coupling can cause the actions of the CFO's team to affect something that the COO's team depends on.

kết dính đó có thể khiến hành động của đội CFO ảnh hưởng tới thứ mà đội COO đang phụ thuộc vào.

For example, suppose that the `calculatePay()` function and the `reportHours()` function share a common algorithm for calculating non-overtime hours. Suppose also that the developers, who are careful not to duplicate code, put that algorithm into a function named `regularHours()` (Figure 7.2).

Chẳng hạn, giả sử hàm `calculatePay()` và hàm `reportHours()` dùng chung một thuật toán để tính số giờ làm không tính tăng ca. Cũng giả sử các lập trình viên, vốn rất cẩn thận để không lặp lại mã, đã đưa thuật toán đó vào một hàm tên là `regularHours()` (Figure 7.2).

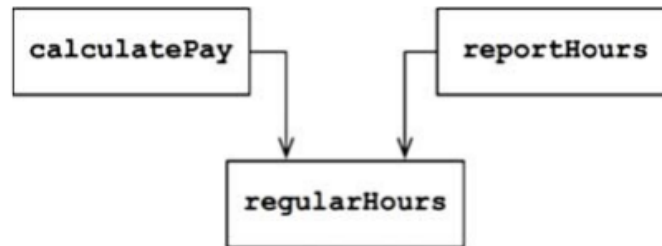


Figure 7.2 Shared algorithm

Figure 7.2 Thuật toán dùng chung

Now suppose that the CFO's team decides that the way non-overtime hours are calculated needs to be tweaked. In contrast, the COO's team in HR does not want that particular tweak because they use non-overtime hours for a different purpose.

Giờ giả sử đội của CFO quyết định rằng cách tính số giờ làm không tính tăng ca cần được chỉnh lại một chút. Ngược lại, đội của COO bên nhân sự không hề muốn chỉnh sửa đó, vì họ dùng số giờ làm không tính tăng ca cho một mục đích khác.

A developer is tasked to make the change, and sees the convenient `regularHours()` function called by the `calculatePay()` method. Unfortunately, that developer does not notice that the function is also called

Một lập trình viên được giao thực hiện thay đổi, và anh ta thấy ngay hàm `regularHours()` tiện lợi đang được phương thức `calculatePay()` gọi tới. Đáng tiếc, lập trình viên đó không để ý rằng hàm này còn được gọi ở chỗ khác nữa.

by the `reportHours()` function.

The developer makes the required change and carefully tests it. The CFO's team validates that the new function works as desired, and the system is deployed.

Lập trình viên thực hiện thay đổi được yêu cầu và kiểm thử cẩn thận. Đội của CFO xác nhận hàm mới chạy đúng như mong muốn, và hệ thống được triển khai.

Of course, the COO's team doesn't know that this is happening. The HR personnel continue to use the reports generated by the `reportHours()` function—but now they contain incorrect numbers. Eventually the problem is discovered, and the COO is livid because the bad data has cost his budget millions of dollars.

Dĩ nhiên, đội của COO không hề biết chuyện này đang diễn ra. Nhân sự bên HR vẫn tiếp tục dùng các báo cáo do hàm `reportHours()` sinh ra — nhưng giờ chúng chứa những con số sai. Rốt cuộc vấn đề cũng bị phát hiện, và COO thì nổi trận lôi đình vì dữ liệu sai đã ngốn mất hàng triệu đô la trong ngân sách của ông ta.

We've all seen things like this happen. These problems occur because we put code that different actors depend on into close proximity. The SRP says to separate the code that different actors depend on .

Tất cả chúng ta đều từng chứng kiến những chuyện như thế. Những vấn đề này xảy ra vì ta đặt sát cạnh nhau phần mã mà các actor khác nhau cùng phụ thuộc vào. SRP nói rằng: hãy tách riêng phần mã mà các actor khác nhau phụ thuộc vào.

Symptom 2: Merges — Triệu chứng 2: Merge

It's not hard to imagine that merges will be common in source files that contain many different methods. This situation is especially likely if those methods are responsible to different actors.

Không khó để hình dung rằng merge sẽ là chuyện cơm bữa với những tệp mã nguồn chứa nhiều phương thức khác nhau. Tình huống này càng dễ xảy ra khi các phương thức ấy chịu trách nhiệm trước những actor khác nhau.

For example, suppose that the CTO's team of DBAs decides that there should be a simple schema change to the Employee table of the database. Suppose also that the COO's team of HR clerks decides that they need a change in the format of the hours report.

Ví dụ, giả sử đội DBA của CTO quyết định rằng cần một thay đổi nhỏ trong schema của bảng Employee trong cơ sở dữ liệu. Cũng giả sử đội nhân viên HR của COO quyết định rằng họ cần đổi định dạng của báo cáo giờ làm.

Two different developers, possibly from two different teams, check out the Employee class and begin to make changes. Unfortunately their changes collide. The result is a merge.

Hai lập trình viên khác nhau, có thể thuộc hai đội khác nhau, cùng lấy class Employee về và bắt đầu sửa. Đáng tiếc, các thay đổi của họ đụng độ nhau. Kết quả là một cuộc merge.

I probably don't need to tell you that merges are risky affairs. Our tools are pretty good nowadays, but no tool can deal with every merge case. In the end, there is always risk.

Chắc tôi chẳng cần nói bạn cũng biết merge là chuyện đầy rủi ro. Ngày nay công cụ của chúng ta khá tốt, nhưng không công cụ nào xử lý được mọi tình huống merge. Rốt cuộc, rủi ro luôn còn đó.

In our example, the merge puts both the CTO and the COO at risk. It's not inconceivable that the CFO could be affected as well.

Trong ví dụ của chúng ta, cuộc merge đặt cả CTO lẫn COO vào vòng rủi ro. Cũng chẳng có gì khó tin nếu CFO cũng bị vạ lây.

There are many other symptoms that we could investigate, but they all involve multiple people changing the same source file for different reasons.

Còn nhiều triệu chứng khác ta có thể mổ xẻ, nhưng tất cả đều xoay quanh chuyện nhiều người cùng sửa một tệp mã nguồn vì những lý do khác nhau.

Once again, the way to avoid this problem is to separate code that supports different actors .

Một lần nữa, cách tránh vấn đề này là tách riêng phần mã phục vụ những actor khác nhau.

Solutions — Các giải pháp

There are many different solutions to this problem. Each moves the functions into different classes.

Có rất nhiều giải pháp khác nhau cho vấn đề này. Giải pháp nào cũng chuyển các hàm sang những class khác nhau.

Perhaps the most obvious way to solve the problem is to separate the data from the functions. The three classes share access to `EmployeeData`, which is a simple data **structure** with no methods (Figure 7.3). Each class holds only the source code necessary for its particular function. The three classes are not allowed to know about each other. Thus any accidental duplication is avoided.

Có lẽ cách hiển nhiên nhất để giải quyết vấn đề là tách dữ liệu ra khỏi các hàm. Ba class cùng chia sẻ quyền truy cập vào `EmployeeData`, một cấu trúc dữ liệu đơn giản không có phương thức nào (Figure 7.3). Mỗi class chỉ giữ đúng phần mã nguồn cần thiết cho chức năng riêng của nó. Ba class này không được phép biết về nhau. Nhờ vậy, mọi trùng lặp tình cờ đều bị loại trừ.

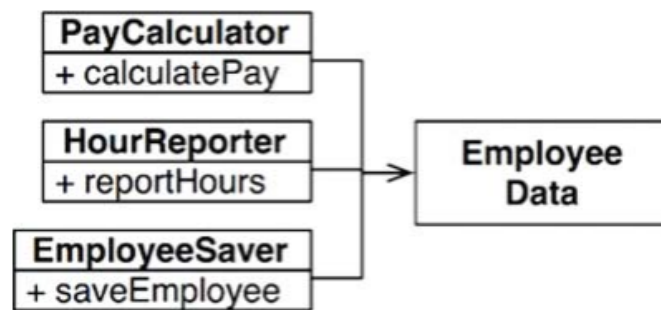


Figure 7.3 The three classes do not know about each other

Figure 7.3 Ba class không biết gì về nhau

The downside of this solution is that the developers now have three classes that they have to instantiate and track. A common solution to this dilemma is to use the Facade pattern (Figure 7.4).

Nhược điểm của giải pháp này là giờ đây các lập trình viên có tới ba class phải khởi tạo và theo dõi. Một cách xử lý phổ biến cho thể lưỡng nan này là dùng Facade pattern (Figure 7.4).

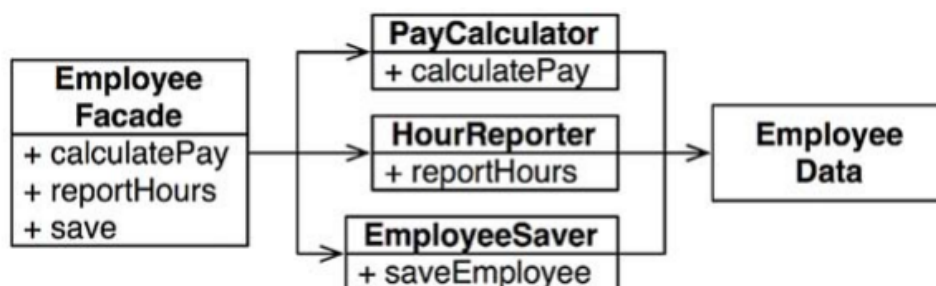


Figure 7.4 The Facade pattern

Figure 7.4 Facade pattern

The EmployeeFacade contains very little code. It is responsible for instantiating and delegating to the classes with the functions.

EmployeeFacade chứa rất ít mã. Nó chỉ chịu trách nhiệm khởi tạo và ủy thác cho các class chứa những hàm kia.

Some developers prefer to keep the most **important** business rules closer to the data. This can be done by keeping the most important method in the original Employee class and then using that class as a Facade for the lesser functions (Figure 7.5).

Một số lập trình viên thích giữ các quy tắc nghiệp vụ (business rule) quan trọng nhất ở gần dữ liệu. Có thể làm điều đó bằng cách giữ phương thức quan trọng nhất lại trong class Employee ban đầu, rồi dùng chính class đó làm Facade cho các hàm ít quan trọng hơn (Figure 7.5).

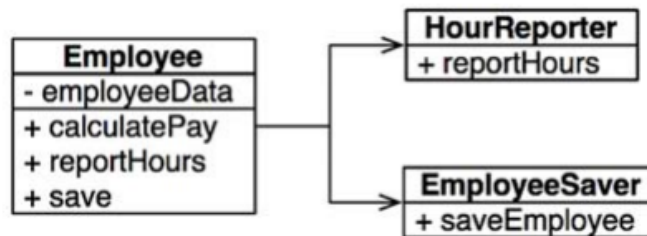


Figure 7.5 The most important method is kept in the original Employee class and used as a

Figure 7.5 Phương thức quan trọng nhất được giữ lại trong class Employee ban đầu và được dùng làm

Facade for the lesser functions

Facade cho các hàm ít quan trọng hơn

You might object to these solutions on the basis that every class would contain just one function. This is hardly the case. The number of functions required to calculate pay, generate a report, or save the data is likely to be large in each case. Each of those classes would have many private methods in them.

Bạn có thể phản đối những giải pháp này với lý do mỗi class rồi sẽ chỉ còn đúng một hàm. Hoàn toàn không phải vậy. Số lượng hàm cần để tính lương, sinh báo cáo hay lưu dữ liệu nhiều khả năng đều rất lớn trong từng trường hợp. Mỗi class trong số đó sẽ có rất nhiều phương thức private bên trong.

Each of the classes that contain such a family of methods is a scope. Outside of that scope, no one knows that the private members of the family exist.

Mỗi class chứa một “gia đình” phương thức như vậy là một phạm vi. Bên ngoài phạm vi đó, không ai biết những thành viên private của gia đình ấy tồn tại.

Conclusion — Kết luận

The Single Responsibility Principle is about functions and classes—but it reappears in a different form at two more levels. At the **level** of components, it becomes the Common Closure Principle. At

the architectural level, it becomes the Axis of Change responsible for the creation of Architectural Boundaries. We'll be studying all of these ideas in the chapters to come.

Nguyên tắc Trách nhiệm Đơn nhất nói về các hàm và class — nhưng nó còn tái xuất dưới một hình hài khác ở hai cấp cao hơn nữa. Ở cấp component, nó trở thành Common Closure Principle. Ở cấp kiến trúc, nó trở thành Trục Thay đổi (Axis of Change), thứ chịu trách nhiệm tạo ra các Ranh giới Kiến trúc (Architectural Boundaries). Chúng ta sẽ nghiên cứu tất cả những ý tưởng này trong các chương sắp tới.

OCP: The Open- Closed Principle — OCP — Nguyên tắc Đóng–Mở

The Open-Closed Principle (OCP) was coined in 1988 by Bertrand Meyer. 1 It says:

Nguyên tắc Đóng–Mở (Open-Closed Principle, OCP) được Bertrand Meyer đặt ra vào năm 1988. 1 Nguyên tắc này nói:

A software artifact should be open for extension but closed for modification.

Một artifact phần mềm nên mở cho việc mở rộng nhưng đóng với việc sửa đổi.

In other words, the **behavior** of a software artifact ought to be extendible, without having to modify that artifact.

Nói cách khác, hành vi (behavior) của một artifact phần mềm phải có thể mở rộng được mà không cần sửa chính artifact đó.

This, of course, is the most fundamental reason that we study software **architecture**. Clearly, if simple extensions to the requirements force massive changes to the software, then the architects of that software system have engaged in a spectacular failure.

Đây, tất nhiên, chính là lý do căn bản nhất khiến chúng ta nghiên cứu kiến trúc phần mềm. Rõ ràng, nếu những mở rộng đơn giản về yêu cầu lại buộc phần mềm phải thay đổi ồ ạt, thì các kiến trúc sư của hệ thống đó đã thất bại một cách ngoạn mục.

Most students of software **design** recognize the OCP as a principle that guides them in the design of classes and modules. But the principle takes on even greater significance when we consider the **level** of architectural components.

Phần lớn người học thiết kế phần mềm đều biết OCP như một nguyên tắc dẫn đường khi thiết kế class và module. Nhưng nguyên tắc này còn mang ý nghĩa lớn hơn nhiều khi ta xét tới cấp các component kiến trúc.

A thought experiment will make this clear.

Một thí nghiệm tư duy sẽ làm rõ điều đó.

A Thought Experiment — Một thí nghiệm tư duy

Imagine, for a moment, that we have a system that displays a financial summary on a web page. The data on the page is scrollable, and negative numbers are rendered in red.

Hãy hình dung trong chốc lát rằng ta có một hệ thống hiển thị bản tóm tắt tài chính trên một trang web. Dữ liệu trên trang có thể cuộn được, và các số âm được tô màu đỏ.

Now imagine that the stakeholders ask that this same information be turned into a report to be printed on a black-and-white printer. The report should be properly paginated, with appropriate page headers, page footers, and column labels. Negative numbers should be surrounded by parentheses.

Bây giờ hãy hình dung các bên liên quan (stakeholder) yêu cầu biến chính thông tin đó thành một báo cáo in ra máy in đen trắng. Báo cáo phải được phân trang đúng hàng, có header trang, footer trang và nhãn cột phù hợp. Số âm phải được đặt trong ngoặc đơn.

Clearly, some new code must be written. But how much old code will have to change?

Rõ ràng phải viết thêm một ít mã mới. Nhưng bao nhiêu mã cũ sẽ phải thay đổi?

- 1. Bertrand Meyer. Object Oriented Software Construction , Prentice Hall, 1988, p. 23.
 - 1. Bertrand Meyer. Object Oriented Software Construction , Prentice Hall, 1988, p. 23.

A good software architecture would reduce the amount of changed code to the barest minimum. Ideally, zero.

Một kiến trúc phần mềm tốt sẽ giảm lượng mã phải sửa xuống mức tối thiểu. Lý tưởng nhất là bằng không.

How? By properly separating the things that change for different reasons (the Single Responsibility Principle), and then organizing the dependencies between those things properly (the **Dependency Inversion** Principle).

Bằng cách nào? Bằng cách tách bạch đúng đắn những thứ thay đổi vì những lý do khác nhau (Single Responsibility Principle), rồi tổ chức đúng đắn các phụ thuộc giữa những thứ đó (Dependency Inversion Principle).

By applying the SRP, we might come up with the data-flow view shown in Figure 8.1. Some analysis procedure inspects the financial data and produces reportable data, which is then formatted appropriately by the two reporter processes.

Khi áp dụng SRP, ta có thể đi tới góc nhìn luồng dữ liệu như trong Figure 8.1. Một thủ tục phân tích nào đó xem xét dữ liệu tài chính và sinh ra dữ liệu có thể đưa vào báo cáo, rồi hai tiến trình lập báo cáo sẽ định dạng dữ liệu ấy cho phù hợp.



Figure 8.1 Applying the SRP

Figure 8.1 Áp dụng SRP

The essential insight here is that generating the report involves two separate responsibilities: the calculation of the reported data, and the presentation of that data into a web- and printer-friendly

form.

Điểm mấu chốt ở đây là việc sinh ra báo cáo bao gồm hai trách nhiệm tách biệt: tính toán dữ liệu cần báo cáo, và trình bày dữ liệu đó dưới dạng phù hợp cho web và cho máy in.

Having made this separation, we need to organize the source code dependencies to ensure that changes to one of those responsibilities do not cause changes in the other. Also, the new organization should ensure that the behavior can be extended without modification.

Sau khi đã tách như vậy, ta cần tổ chức các phụ thuộc mã nguồn sao cho thay đổi ở một trong hai trách nhiệm không kéo theo thay đổi ở cái còn lại. Ngoài ra, cách tổ chức mới phải bảo đảm hành vi có thể mở rộng mà không cần sửa đổi quá đáng.

We accomplish this by partitioning the processes into classes, and separating those classes into components, as shown by the double lines in the diagram in Figure 8.2. In this figure, the **component** at the upper left is the Controller . At the upper right, we have the **Interactor** . At the lower right, there is the Database . Finally, at the lower left, there are four components that represent the Presenters and the Views .

Ta làm được điều đó bằng cách chia các tiến trình thành class, rồi tách các class ấy vào những component khác nhau, như các đường kẻ đôi trong sơ đồ ở Figure 8.2. Trong hình này, component ở góc trên bên trái là Controller . Góc trên bên phải là Interactor . Góc dưới bên phải là Database . Cuối cùng, góc dưới bên trái là bốn component đại diện cho các Presenter và các View .

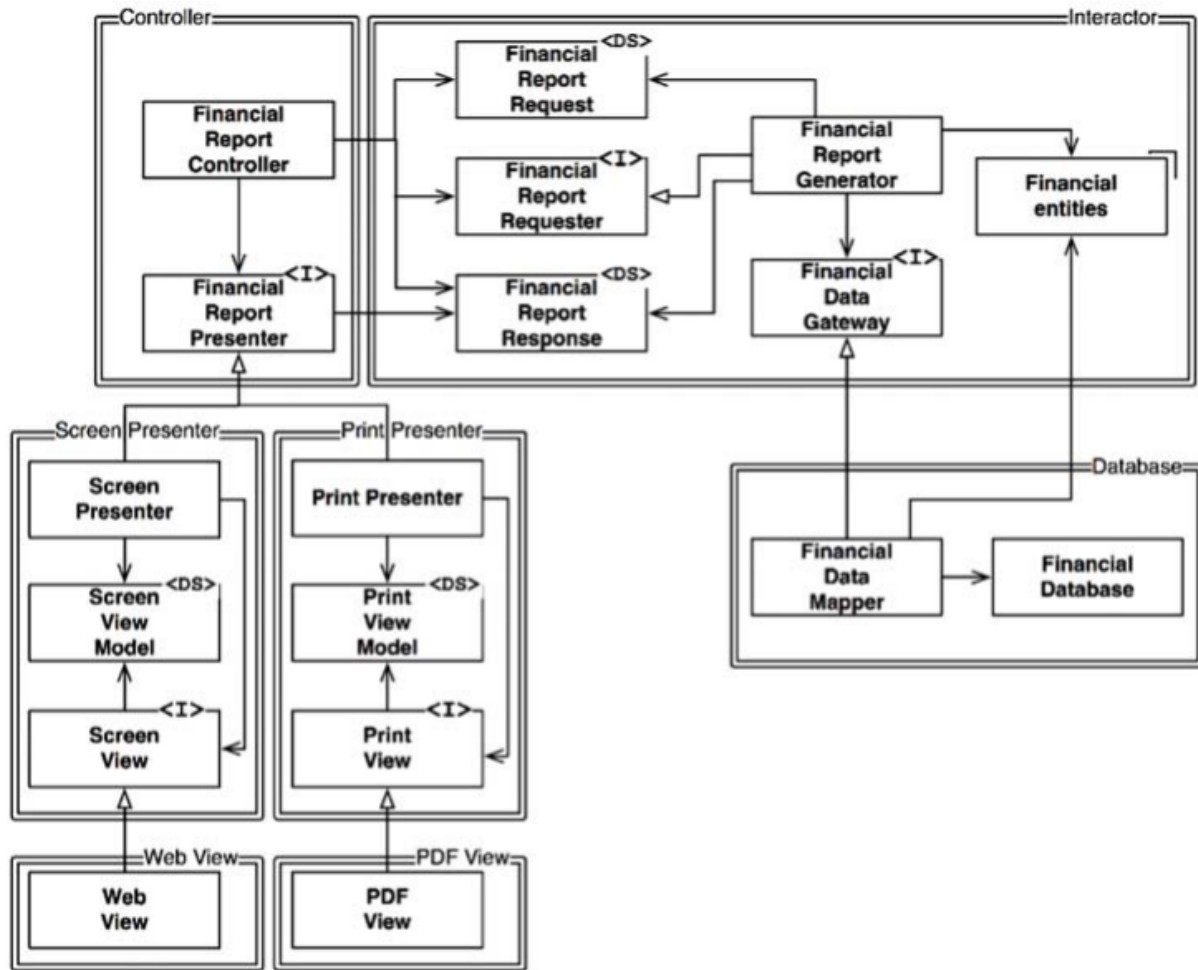


Figure 8.2 Partitioning the processes into classes and separating the classes into components

Figure 8.2 Chia các tiến trình thành class và tách các class vào những component khác nhau

Classes marked with **I** are interfaces; those marked with **DS** are data structures. Open arrowheads are using relationships. Closed arrowheads are implements or **inheritance** relationships.

Các class đánh dấu là interface; các class đánh dấu là cấu trúc dữ liệu. Mũi tên đầu hở là quan hệ sử dụng. Mũi tên đầu đặc là quan hệ hiện thực hóa hoặc kế thừa.

The first thing to notice is that all the dependencies are source code dependencies. An arrow pointing from class A to class B means that the source code of class A mentions the name of class B, but class B mentions nothing about class A. Thus, in Figure 8.2, FinancialDataMapper knows

Điều đầu tiên cần chú ý là mọi phụ thuộc ở đây đều là phụ thuộc mã nguồn. Một mũi tên đi từ class A tới class B nghĩa là mã nguồn của class A có nhắc tới tên của class B, còn class B thì không hề nhắc gì tới class A. Vì vậy, trong Figure 8.2, FinancialDataMapper biết

about FinancialDataGateway through an implements relationship, but FinancialGateway knows nothing at all about FinancialDataMapper .

FinancialGateway hoàn toàn không biết gì về FinancialDataMapper .

The next thing to notice is that each double line is crossed in one direction only . This means that all component relationships are unidirectional, as

Điều tiếp theo cần chú ý là mỗi đường kẻ đôi chỉ bị cắt qua theo một chiều duy nhất . Điều này nghĩa là mọi quan hệ giữa các component đều một chiều, như

shown in the component graph in Figure 8.3. These arrows point toward the components that we want to protect from change.

thể hiện trong đồ thị component ở Figure 8.3. Những mũi tên này chỉ về phía các component mà ta muốn bảo vệ khỏi thay đổi.

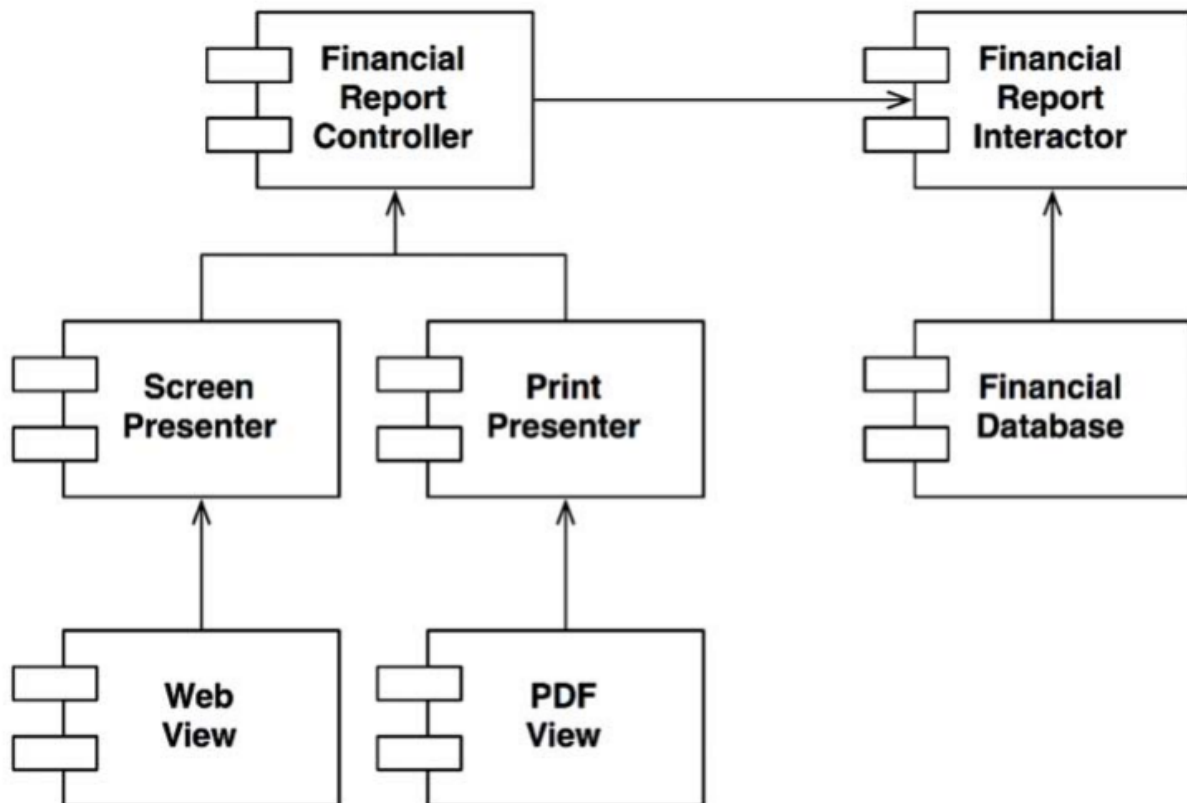


Figure 8.3 The component relationships are unidirectional

Figure 8.3 Các quan hệ giữa component đều một chiều

Let me say that again: If component A should be protected from changes in component B, then component B should depend on component A.

Xin nhắc lại: nếu component A cần được bảo vệ khỏi những thay đổi trong component B, thì component B phải phụ thuộc vào component A.

We want to protect the Controller from changes in the Presenters . We want to protect the Presenters from changes in the Views . We want to protect the Interactor from changes in—well, anything .

Ta muốn bảo vệ Controller khỏi những thay đổi trong các Presenter . Ta muốn bảo vệ các Presenter khỏi những thay đổi trong các View . Ta muốn bảo vệ Interactor khỏi những

thay đổi trong—à thì, mọi thứ .

The Interactor is in the position that best conforms to the OCP. Changes to the Database , or the Controller , or the Presenters , or the Views , will have no impact on the Interactor .

Interactor nằm ở vị trí tuân thủ OCP tốt nhất. Thay đổi ở Database , ở Controller , ở các Presenter hay các View đều không tác động gì tới Interactor .

Why should the Interactor hold such a privileged position? Because it contains the business rules. The Interactor contains the highest-level policies

Vì sao Interactor lại được hưởng vị thế đặc quyền như vậy? Vì nó chứa các quy tắc nghiệp vụ (business rule). Interactor chứa những chính sách (policy) ở mức cao nhất

of the application. All the other components are dealing with peripheral concerns. The Interactor deals with the central concern.

của ứng dụng. Tất cả các component khác đều lo những mối quan tâm ngoại vi. Interactor lo mối quan tâm trung tâm.

Even though the Controller is peripheral to the Interactor , it is nevertheless central to the Presenters and Views . And while the Presenters might be peripheral to the Controller , they are central to the Views .

Dù Controller là ngoại vi so với Interactor , nó vẫn là trung tâm so với các Presenter và các View . Và tuy các Presenter có thể là ngoại vi so với Controller , chúng lại là trung tâm so với các View .

Notice how this creates a hierarchy of protection based on the notion of “level.” Interactors are the highest-level concept, so they are the most protected. Views are among the lowest-level concepts, so they are the least protected. Presenters are higher level than Views , but lower level than the Controller or the Interactor .

Hãy để ý cách điều này tạo ra một hệ thứ bậc bảo vệ dựa trên khái niệm “cấp” (level). Interactor là khái niệm ở cấp cao nhất, nên được bảo vệ nhiều nhất. View nằm trong số những khái niệm ở cấp thấp nhất, nên được bảo vệ ít nhất. Presenter ở cấp cao hơn View , nhưng thấp hơn Controller hay Interactor .

This is how the OCP works at the architectural level. Architects separate functionality based on how, why, and when it changes, and then organize that separated functionality into a hierarchy of components. Higher-level components in that hierarchy are protected from the changes made to lower-level components.

Đó là cách OCP vận hành ở cấp kiến trúc. Kiến trúc sư tách chức năng dựa trên việc chúng thay đổi như thế nào, vì sao và khi nào, rồi tổ chức những phần chức năng đã tách ấy thành một hệ thứ bậc các component. Trong hệ thứ bậc đó, các component ở cấp cao hơn được bảo vệ khỏi những thay đổi diễn ra ở các component cấp thấp hơn.

Directional Control — Kiểm soát chiều phụ thuộc

If you recoiled in horror from the class design shown earlier, look again. Much of the complexity in that diagram was intended to make sure that the dependencies between the components pointed in the correct direction.

Nếu bạn rùng mình kinh hãi trước thiết kế class vừa trình bày, hãy nhìn lại lần nữa. Phần lớn sự phức tạp trong sơ đồ đó là để bảo đảm các phụ thuộc giữa những component trở

đúng chiều.

For example, the FinancialDataGateway **interface** between the

Chẳng hạn, interface FinancialDataGateway nằm giữa

FinancialReportGenerator and the FinancialDataMapper exists to

invert the dependency that would otherwise have pointed from the Interactor component to the Database component. The same is true of the

đảo ngược phụ thuộc mà lẽ ra sẽ trở từ component Interactor sang component Database .

Điều tương tự cũng đúng với

FinancialReportPresenter interface, and the two View interfaces.

Information Hiding — Che giấu thông tin

The FinancialReportRequester interface serves a different purpose. It is

Interface FinancialReportRequester phục vụ một mục đích khác. Nó ở đó để

there to protect the FinancialReportController from knowing too much

about the internals of the Interactor . If that interface were not there, then the Controller would have transitive dependencies on the FinancialEntities .

về nội tại của Interactor . Nếu không có interface đó, Controller sẽ có phụ thuộc bắc cầu vào FinancialEntities .

Transitive dependencies are a violation of the general principle that software entities should not depend on things they don't directly use. We'll encounter that principle again when we talk about the Interface Segregation Principle and the Common Reuse Principle.

Phụ thuộc bắc cầu vi phạm nguyên tắc chung rằng một thực thể phần mềm không nên phụ thuộc vào những thứ nó không trực tiếp dùng tới. Ta sẽ gặp lại nguyên tắc này khi bàn về Interface Segregation Principle và Common Reuse Principle.

So, even though our first priority is to protect the Interactor from changes to the Controller , we also want to protect the Controller from changes to the Interactor by hiding the internals of the Interactor .

Vậy nên, dù ưu tiên hàng đầu của ta là bảo vệ Interactor khỏi những thay đổi ở Controller , ta cũng muốn bảo vệ Controller khỏi những thay đổi ở Interactor bằng cách che giấu phần nội tại của Interactor .

Conclusion — Kết luận

The OCP is one of the driving forces behind the architecture of systems. The goal is to make the system easy to extend without incurring a high impact of change. This goal is accomplished by partitioning the system into components, and arranging those components into a dependency hierarchy that protects higher-level components from changes in lower-level components.

OCP là một trong những động lực chính đứng sau kiến trúc của các hệ thống. Mục tiêu là làm cho hệ thống dễ mở rộng mà không phải gánh chịu tác động lớn từ thay đổi. Mục tiêu này đạt được bằng cách chia hệ thống thành các component, rồi sắp xếp những component

đó thành một hệ thứ bậc phụ thuộc sao cho các component cấp cao hơn được bảo vệ khỏi những thay đổi ở các component cấp thấp hơn.

LSP: The Liskov Substitution Principle — LSP: Nguyên tắc Thay thế Liskov

In 1988, Barbara Liskov wrote the following as a way of defining subtypes.

Năm 1988, Barbara Liskov viết những dòng sau như một cách định nghĩa kiểu con (subtype).

What is wanted here is something like the following substitution property: If for each object *o1* of type *S* there is an object *o2* of type *T* such that for all programs *P* defined in terms of *T*, the **behavior** of *P* is unchanged when *o1* is substituted for *o2* then *S* is a subtype of *T*. 1

Điều chúng ta cần ở đây là một thuộc tính thay thế đại loại như sau: Nếu với mỗi đối tượng *o1* thuộc kiểu *S* đều tồn tại một đối tượng *o2* thuộc kiểu *T* sao cho với mọi chương trình *P* được định nghĩa theo *T*, hành vi (behavior) của *P* không đổi khi thay *o2* bằng *o1*, thì *S* là một kiểu con của *T*. 1

To understand this idea, which is known as the Liskov Substitution Principle (LSP), let's look at some examples.

Để hiểu ý tưởng này — thứ vẫn được gọi là LSP, tức Nguyên tắc Thay thế Liskov (Liskov Substitution Principle) — hãy cùng xem vài ví dụ.

Guiding the Use of Inheritance — Định hướng cho việc dùng kế thừa

Imagine that we have a class named `License`, as shown in Figure 9.1. This class has a method named `calcFee()`, which is called by the Billing application. There are two “subtypes” of `License`: `PersonalLicense` and `BusinessLicense`. They use different algorithms to calculate the license fee.

Hãy hình dung ta có một class tên là `License`, như trong Figure 9.1. Class này có một phương thức tên `calcFee()`, được ứng dụng Billing gọi tới. `License` có hai “kiểu con”: `PersonalLicense` và `BusinessLicense`. Chúng dùng hai thuật toán khác nhau để tính phí bản quyền.

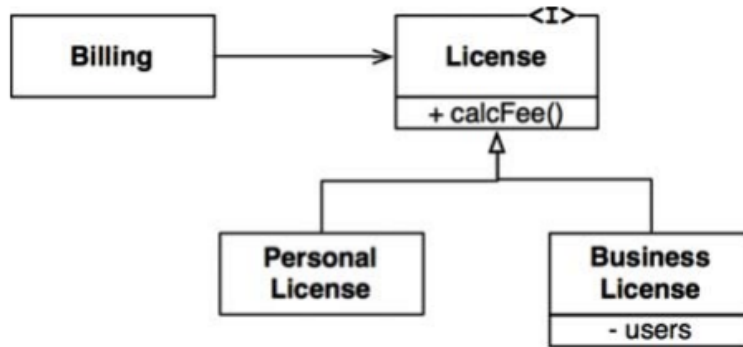


Figure 9.1 License , and its derivatives, conform to LSP

Figure 9.1 License và các lớp dẫn xuất của nó tuân thủ LSP

This **design** conforms to the LSP because the behavior of the Billing application does not depend, in any way, on which of the two subtypes it uses. Both of the subtypes are substitutable for the License type.

Thiết kế này tuân thủ LSP, bởi hành vi của ứng dụng Billing không hề phụ thuộc vào việc nó đang dùng kiểu con nào trong hai kiểu con đó. Cả hai kiểu con đều có thể thay thế cho kiểu License .

- 1. Barbara Liskov, “Data Abstraction and Hierarchy,” SIGPLAN Notices 23, 5 (May 1988).
 - 1. Barbara Liskov, “Data Abstraction and Hierarchy,” SIGPLAN Notices 23, 5 (May 1988).

The Square/Rectangle Problem — Bài toán hình vuông/hình chữ nhật

The canonical example of a violation of the LSP is the famed (or infamous, depending on your perspective) square/rectangle problem (Figure 9.2).

Ví dụ kinh điển về việc vi phạm LSP là bài toán hình vuông/hình chữ nhật lừng danh (hay khét tiếng, tùy góc nhìn của bạn) (Figure 9.2).

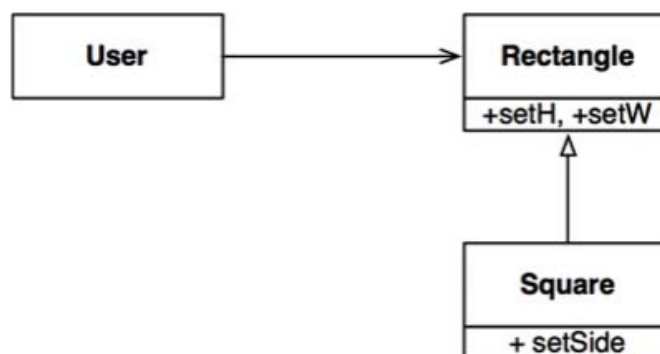


Figure 9.2 The infamous square/rectangle problem

Figure 9.2 Bài toán hình vuông/hình chữ nhật khét tiếng

In this example, Square is not a proper subtype of Rectangle because the height and width of the Rectangle are independently **mutable**; in contrast, the height and width of the Square must change together. Since the User believes it is communicating with a Rectangle, it could easily get confused. The following code shows why:

Trong ví dụ này, Square không phải là kiểu con hợp lệ của Rectangle, bởi chiều cao và chiều rộng của Rectangle có thể thay đổi độc lập với nhau; ngược lại, chiều cao và chiều rộng của Square buộc phải đổi cùng nhau. Vì User tin rằng nó đang làm việc với một Rectangle, nó rất dễ bị rối. Đoạn mã sau cho thấy vì sao:

```
Rectangle r = ...  
r.setW(5);  
r.setH(2);  
assert(r.area() == 10);
```

If the ... code produced a Square, then the assertion would fail.

Nếu đoạn mã ... tạo ra một Square, phép assert sẽ thất bại.

The only way to defend against this kind of LSP violation is to add mechanisms to the User (such as an if statement) that detects whether the Rectangle is, in fact, a Square. Since the behavior of the User depends on the types it uses, those types are not substitutable.

Cách duy nhất để phòng vệ trước kiểu vi phạm LSP này là thêm vào User một cơ chế nào đó (chẳng hạn một câu lệnh if) để phát hiện xem Rectangle thực chất có phải là một Square hay không. Mà một khi hành vi của User đã phụ thuộc vào các kiểu nó dùng, thì các kiểu đó không thể thay thế cho nhau.

LSP and Architecture — LSP và kiến trúc

In the early years of the object-oriented revolution, we thought of the LSP as a way to guide the use of **inheritance**, as shown in the previous sections. However, over the years the LSP has morphed into a broader principle of software design that pertains to interfaces and implementations.

Vào những năm đầu của cuộc cách mạng hướng đối tượng, chúng ta xem LSP như một kim chỉ nam cho việc dùng kế thừa (inheritance), đúng như các phần trên. Tuy nhiên, qua năm tháng, LSP đã lột xác thành một nguyên tắc thiết kế phần mềm rộng hơn nhiều, áp dụng cho cả interface lẫn phần cài đặt.

The interfaces in question can be of many forms. We might have a Java-style **interface**, implemented by several classes. Or we might have several Ruby classes that share the same method signatures. Or we might have a set of services that all respond to the same REST interface.

Interface ở đây có thể mang nhiều hình hài. Có thể là một interface kiểu Java, được nhiều class cài đặt. Có thể là vài class Ruby cùng chia sẻ một tập chữ ký phương thức. Cũng có thể là một nhóm dịch vụ (service) cùng đáp ứng một interface REST.

In all of these situations, and more, the LSP is applicable because there are users who depend on well-defined interfaces, and on the **substitutability** of the implementations of those interfaces.

Trong tất cả những tình huống đó — và còn nhiều nữa — LSP đều áp dụng được, bởi luôn có những bên sử dụng phụ thuộc vào các interface được định nghĩa rõ ràng, và phụ thuộc vào khả năng thay thế (substitutability) giữa các phần cài đặt của những interface ấy.

The best way to understand the LSP from an architectural viewpoint is to look at what happens to the **architecture** of a system when the principle is violated.

Cách tốt nhất để hiểu LSP dưới góc nhìn kiến trúc (architecture) là nhìn xem chuyện gì xảy ra với kiến trúc của một hệ thống khi nguyên tắc này bị vi phạm.

Example LSP Violation — Ví dụ về một vi phạm LSP

Assume that we are building an aggregator for many taxi dispatch services. Customers use our website to find the most appropriate taxi to use, regardless of taxi company. Once the customer makes a decision, our system dispatches the chosen taxi by using a restful **service**.

Giả sử ta đang xây một hệ thống tổng hợp cho nhiều dịch vụ điều xe taxi. Khách hàng vào website của ta để tìm chiếc taxi phù hợp nhất, bất kể là của hãng nào. Khi khách đã chọn xong, hệ thống của ta điều xe đã chọn tới thông qua một dịch vụ restful.

Now assume that the URI for the restful dispatch service is part of the information contained in the driver database. Once our system has chosen a driver appropriate for the customer, it gets that URI from the driver record and then uses it to dispatch the driver.

Bây giờ giả sử URI của dịch vụ điều xe restful đó là một phần thông tin nằm trong cơ sở dữ liệu tài xế. Sau khi hệ thống chọn được tài xế phù hợp với khách, nó lấy URI đó từ bản ghi tài xế rồi dùng URI này để điều tài xế đi.

Suppose Driver Bob has a dispatch URI that looks like this:

Giả sử tài xế Bob có một URI điều xe trông như thế này:

`purplecab.com/driver/Bob`

Our system will append the dispatch information onto this URI and send it with a PUT, as follows:

Hệ thống của ta sẽ nối thông tin điều xe vào URI này rồi gửi đi bằng một lệnh PUT, như sau:

```
purplecab.com/driver/Bob
/pickupAddress/24 Maple St.
/pickupTime/153
/destination/ORD
```

Clearly, this means that all the dispatch services, for all the different companies, must conform to the same REST interface. They must treat the `pickupAddress`, `pickupTime`, and `destination` fields identically.

Rõ ràng, điều này có nghĩa là mọi dịch vụ điều xe của mọi hãng đều phải tuân theo cùng một interface REST. Chúng phải xử lý các trường `pickupAddress`, `pickupTime` và `destination` y hệt nhau.

Now suppose the Acme taxi company hired some programmers who didn't read the spec very carefully. They abbreviated the `destination` field to just `dest`. Acme is the largest taxi company in our area, and Acme's CEO's ex-wife is our CEO's new wife, and ... Well, you get the picture. What would happen to the architecture of our system?

Giờ hãy giả sử hãng taxi Acme thuê phải mấy lập trình viên đọc đặc tả không kỹ. Họ viết tắt trường `destination` thành `dest`. Acme lại là hãng taxi lớn nhất khu vực, và vợ cũ của CEO bên Acme chính là vợ mới của CEO bên ta, và... Thôi, bạn hình dung được rồi đấy. Chuyện gì sẽ xảy ra với kiến trúc hệ thống của ta?

Obviously, we would need to add a special case. The dispatch **request** for any Acme driver would have to be constructed using a different set of rules from all the other drivers.

Hiển nhiên, ta sẽ phải thêm một trường hợp đặc biệt. Yêu cầu điều xe cho bất kỳ tài xế nào của Acme sẽ phải được dựng lên bằng một bộ quy tắc khác với tất cả các tài xế còn lại.

The simplest way to accomplish this goal would be to add an if statement to the module that constructed the dispatch command:

Cách đơn giản nhất để làm việc đó là thêm một câu lệnh if vào module dựng lệnh điều xe:

```
if (driver.getDispatchUri().startsWith("acme.com"))...
```

But, of course, no architect worth his or her salt would allow such a construction to exist in the system. Putting the word “acme” into the code itself creates an opportunity for all kinds of horrible and mysterious errors, not to mention security breaches.

Nhưng dĩ nhiên, không một kiến trúc sư nào ra hồn lại để một cấu trúc như vậy tồn tại trong hệ thống. Nhét chữ “acme” vào thẳng trong mã nguồn là mở đường cho đủ loại lỗi kinh dị và khó lần ra, chưa kể còn là lỗ hổng bảo mật.

For example, what if Acme became even more successful and bought the Purple Taxi company. What if the merged company maintained the separate

Chẳng hạn, sẽ ra sao nếu Acme còn ăn nên làm ra hơn nữa và thôn tóm luôn hãng Purple Taxi? Sẽ ra sao nếu công ty sau sáp nhập vẫn giữ riêng

brands and the separate websites, but unified all of the original companies’ systems? Would we have to add another if statement for “purple”?

thương hiệu và website riêng, nhưng lại hợp nhất toàn bộ hệ thống của các hãng ban đầu? Chẳng lẽ ta lại phải thêm một câu lệnh if nữa cho “purple”?

Our architect would have to insulate the system from bugs like this by creating some kind of dispatch command creation module that was driven by a configuration database keyed by the dispatch URI. The configuration data might look something like this:

Kiến trúc sư của ta sẽ phải cách ly hệ thống khỏi những con bọ kiểu này bằng cách tạo ra một module chuyên dựng lệnh điều xe, được điều khiển bởi một cơ sở dữ liệu cấu hình lấy URI điều xe làm khoá. Dữ liệu cấu hình có thể trông đại loại như sau:

URI Dispatch Format

URI Định dạng điều xe

```
Acme.com/pickupAddress/%s/pickupTime/%s/dest/%s  
*.*/pickupAddress/%s/pickupTime/%s/destination/%s
```

And so our architect has had to add a significant and complex mechanism to deal with the fact that the interfaces of the restful services are not all substitutable.

Và thế là kiến trúc sư của ta buộc phải thêm vào một cơ chế đồ sộ và phức tạp, chỉ để đối phó với thực tế rằng không phải mọi interface của những dịch vụ restful kia đều thay thế được cho nhau.

Conclusion — Kết luận

The LSP can, and should, be extended to the **level** of architecture. A simple violation of substitutability, can cause a system's architecture to be polluted with a significant amount of extra mechanisms.

LSP có thể, và nên, được mở rộng lên tới cấp kiến trúc. Chỉ một vi phạm nhỏ về khả năng thay thế cũng đủ khiến kiến trúc của cả hệ thống bị ô nhiễm bởi một đồng cơ chế phụ trợ.

ISP: The Interface Segregation Principle — ISP: Nguyên tắc Phân tách Interface

The **Interface** Segregation Principle (ISP) derives its name from the diagram shown in Figure 10.1.

Nguyên tắc Phân tách Interface (Interface Segregation Principle, ISP) lấy tên từ sơ đồ ở Figure 10.1.

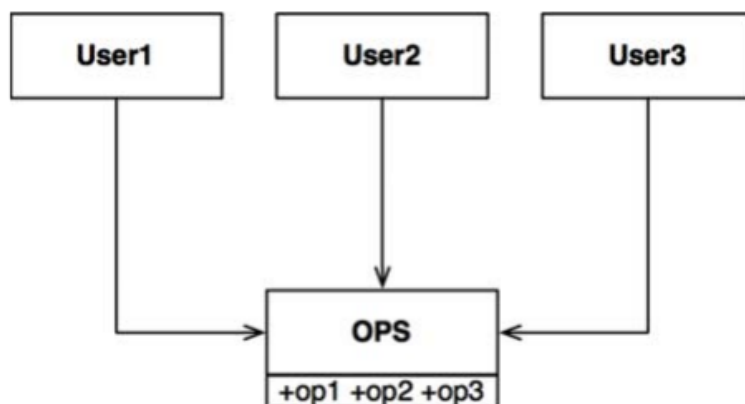


Figure 10.1 The Interface Segregation Principle

Figure 10.1 Nguyên tắc Phân tách Interface

In the situation illustrated in Figure 10.1, there are several users who use the operations of the OPS class. Let's assume that User1 uses only op1 , User2 uses only op2 , and User3 uses only op3 .

Trong tình huống được minh họa ở Figure 10.1, có vài user cùng dùng các thao tác của class OPS. Giả sử User1 chỉ dùng op1, User2 chỉ dùng op2, còn User3 chỉ dùng op3.

Now imagine that OPS is a class written in a language like Java. Clearly, in that case, the source code of User1 will inadvertently depend on op2 and op3 , even though it doesn't call them. This dependence means that a change to the source code of op2 in OPS will force User1 to be recompiled and redeployed, even though nothing that it cared about has actually changed.

Giờ hãy hình dung OPS là một class được viết bằng ngôn ngữ như Java. Rõ ràng khi đó mã nguồn của User1 sẽ vô tình phụ thuộc vào op2 và op3, dù nó chẳng hề gọi tới chúng. Sự phụ thuộc ấy có nghĩa là một thay đổi trong mã nguồn của op2 bên trong OPS sẽ buộc

User1 phải biên dịch lại và triển khai lại, dù chẳng có thứ gì nó quan tâm thực sự thay đổi cả.

This problem can be resolved by segregating the operations into interfaces as shown in Figure 10.2.

Vấn đề này có thể được hoá giải bằng cách phân tách các thao tác vào những interface riêng, như ở Figure 10.2.

Again, if we imagine that this is implemented in a statically typed language like Java, then the source code of User1 will depend on U1Ops , and op1 , but will not depend on OPS . Thus a change to OPS that User1 does not care about will not cause User1 to be recompiled and redeployed.

Một lần nữa, nếu ta hình dung điều này được cài đặt bằng một ngôn ngữ định kiểu tĩnh như Java, thì mã nguồn của User1 sẽ phụ thuộc vào U1Ops và op1, chứ không phụ thuộc vào OPS. Nhờ vậy, một thay đổi trong OPS mà User1 không hề quan tâm sẽ không khiến User1 phải biên dịch lại và triển khai lại.

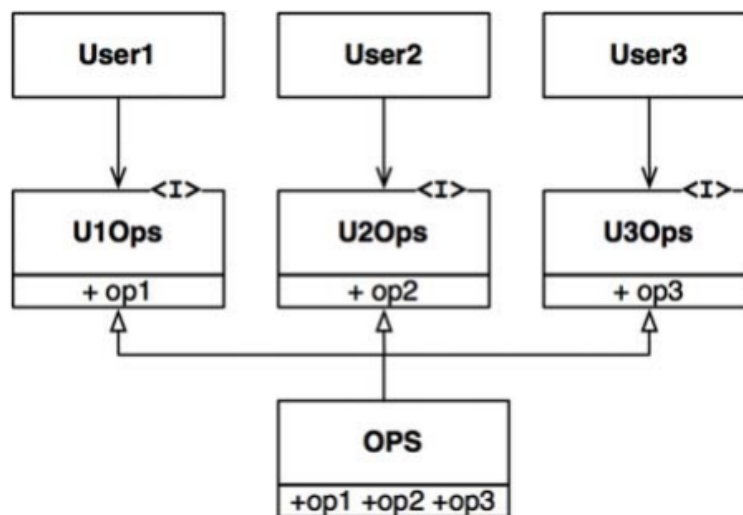


Figure 10.2 Segregated operations

Figure 10.2 Các thao tác đã được phân tách

ISP and Language — ISP và ngôn ngữ

Clearly, the previously given description depends critically on language type. Statically typed languages like Java force programmers to create declarations that users must import , or use , or otherwise include . It is these included declarations in source code that create the source code dependencies that force recompilation and redeployment.

Rõ ràng, phần mô tả vừa rồi phụ thuộc rất nhiều vào kiểu của ngôn ngữ. Những ngôn ngữ định kiểu tĩnh như Java buộc lập trình viên phải tạo ra các khai báo mà phía sử dụng phải import, hoặc use, hoặc include theo cách nào đó. Chính những khai báo được đưa vào mã nguồn đó đã sinh ra phụ thuộc mã nguồn, và phụ thuộc mã nguồn thì buộc ta phải biên dịch lại và triển khai lại.

In dynamically typed languages like Ruby and Python, such declarations don't exist in source code. Instead, they are inferred at runtime. Thus there are no source code dependencies to force

recompilation and redeployment. This is the primary reason that dynamically typed languages create systems that are more flexible and less tightly coupled than statically typed languages.

Trong các ngôn ngữ định kiểu động như Ruby và Python, những khai báo như thế không tồn tại trong mã nguồn. Thay vào đó, chúng được suy ra lúc chạy. Vì vậy chẳng có phụ thuộc mã nguồn nào buộc phải biên dịch lại và triển khai lại. Đây chính là lý do chủ yếu khiến các ngôn ngữ định kiểu động tạo ra những hệ thống linh hoạt hơn và ít kết dính chặt hơn so với ngôn ngữ định kiểu tĩnh.

This fact could lead you to conclude that the ISP is a language issue, rather than an **architecture** issue.

Thực tế này có thể dẫn bạn tới kết luận rằng ISP là chuyện của ngôn ngữ, chứ không phải chuyện của kiến trúc (architecture).

ISP and Architecture — ISP và kiến trúc

If you take a step back and look at the root motivations of the ISP, you can see a deeper concern lurking there. In general, it is harmful to depend on modules that contain more than you need. This is obviously true for source code dependencies that can force unnecessary recompilation and redeployment—but it is also true at a much higher, architectural **level**.

Nếu lùi lại một bước và nhìn vào động cơ gốc rễ của ISP, bạn sẽ thấy có một mối lo sâu xa hơn đang ẩn nấp ở đó. Nói chung, phụ thuộc vào những module chứa nhiều hơn thứ bạn cần là điều có hại. Chuyện đó hiển nhiên đúng với phụ thuộc mã nguồn—thứ có thể buộc ta biên dịch lại và triển khai lại một cách không cần thiết—nhưng nó cũng đúng ở một mức cao hơn nhiều: mức kiến trúc.

Consider, for example, an architect working on a system, S. He wants to include a certain **framework**, F, into the system. Now suppose that the authors of F have bound it to a particular database, D. So S depends on F, which depends on D (Figure 10.3).

Chẳng hạn, hãy xét một kiến trúc sư đang làm việc trên hệ thống S. Anh ta muốn đưa một framework F nào đó vào hệ thống. Giả sử các tác giả của F đã trói nó vào một cơ sở dữ liệu cụ thể, D. Thế là S phụ thuộc vào F, mà F lại phụ thuộc vào D (Figure 10.3).

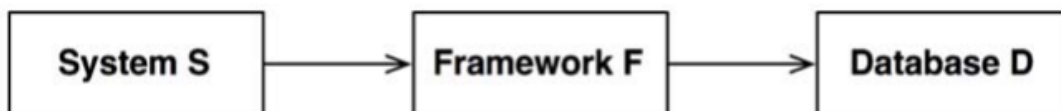


Figure 10.3 A problematic architecture

Figure 10.3 Một kiến trúc có vấn đề

Now suppose that D contains features that F does not use and, therefore, that S does not care about. Changes to those features within D may well force the redeployment of F and, therefore, the redeployment of S. Even worse, a failure of one of the features within D may cause failures in F and S.

Giờ giả sử D chứa những tính năng mà F không dùng đến, và do đó S cũng chẳng bận tâm. Những thay đổi ở các tính năng ấy bên trong D rất có thể buộc F phải triển khai lại, và kéo

theo cả S cũng phải triển khai lại. Tệ hơn nữa, một hỏng hóc ở một trong các tính năng của D có thể gây ra hỏng hóc trong F và trong S.

Conclusion — Kết luận

The lesson here is that depending on something that carries baggage that you don't need can cause you troubles that you didn't expect.

Bài học ở đây là: phụ thuộc vào một thứ mang theo đồng hành lý mà bạn không cần có thể gây ra cho bạn những rắc rối bạn chẳng ngờ tới.

We'll explore this idea in more **detail** when we discuss the Common Reuse Principle in Chapter 13, “**Component Cohesion.**”

Chúng ta sẽ đào sâu ý tưởng này khi bàn về Common Reuse Principle ở Chương 13, “Độ gắn kết của component.”

DIP: The Dependency Inversion Principle — DIP: Nguyên tắc Đảo ngược Phụ thuộc

The **Dependency Inversion** Principle (DIP) tells us that the most flexible systems are those in which source code dependencies refer only to abstractions, not to concretions.

Nguyên tắc Đảo ngược Phụ thuộc (DIP) nói với chúng ta rằng những hệ thống linh hoạt nhất là những hệ thống mà ở đó phụ thuộc mã nguồn (source code dependency) chỉ trở tới các trừu tượng, chứ không trở tới các thứ cụ thể.

In a statically typed language, like Java, this means that the use , import , and include statements should refer only to source modules containing interfaces, abstract classes, or some other kind of abstract declaration. Nothing concrete should be depended on.

Trong một ngôn ngữ định kiểu tĩnh như Java, điều này có nghĩa là các câu lệnh use, import và include chỉ nên trở tới những module nguồn chứa interface, abstract class, hoặc một dạng khai báo trừu tượng nào đó. Không được phụ thuộc vào bất cứ thứ gì cụ thể.

The same rule applies for dynamically typed languages, like Ruby and Python. Source code dependencies should not refer to concrete modules. However, in these languages it is a bit harder to define what a concrete module is. In particular, it is any module in which the functions being called are implemented.

Quy tắc đó cũng áp dụng cho các ngôn ngữ định kiểu động như Ruby và Python. Phụ thuộc mã nguồn không nên trở tới các module cụ thể. Tuy nhiên, trong những ngôn ngữ này thì việc định nghĩa thế nào là một module cụ thể có phần khó hơn: đó là bất kỳ module nào chứa phần cài đặt của các hàm đang được gọi.

Clearly, treating this idea as a rule is unrealistic, because software systems must depend on many concrete facilities. For example, the String class in Java is concrete, and it would be unrealistic to try to force it to be abstract. The **source code dependency** on the concrete java.lang.string cannot, and should not, be avoided.

Rõ ràng, coi ý tưởng này như một luật cứng là điều phi thực tế, bởi hệ thống phần mềm buộc phải phụ thuộc vào rất nhiều tiện ích cụ thể. Chẳng hạn, class String trong Java là cụ thể, và sẽ thật vô lý nếu cố ép nó thành trừu tượng. Phụ thuộc mã nguồn vào java.lang.string cụ thể là thứ không thể — và cũng không nên — né tránh.

By comparison, the String class is very **stable**. Changes to that class are very rare and tightly controlled. Programmers and architects do not have to worry about frequent and capricious changes to String .

Đổi lại, class String rất ổn định. Nó hiếm khi thay đổi, và mọi thay đổi đều được kiểm soát chặt chẽ. Lập trình viên và kiến trúc sư không phải nơm nớp lo String đổi liên tục và tùy hứng.

For these reasons, we tend to ignore the stable background of operating system and platform facilities when it comes to DIP. We tolerate those concrete dependencies because we know we can rely on them not to change.

Vì những lý do đó, khi bàn về DIP chúng ta thường bỏ qua cái nền ổn định gồm các tiện ích của hệ điều hành và của nền tảng. Chúng ta chấp nhận những phụ thuộc cụ thể ấy vì biết rằng có thể tin tưởng là chúng sẽ không thay đổi.

It is the **volatile** concrete elements of our system that we want to avoid depending on. Those are the modules that we are actively developing, and that are undergoing frequent change.

Thứ mà chúng ta muốn tránh phụ thuộc vào chính là những phần tử cụ thể và dễ biến động (volatile) của hệ thống. Đó là các module ta đang tích cực phát triển, và đang thay đổi xoành xoạch.

Stable Abstractions — Những trừu tượng ổn định

Every change to an abstract **interface** corresponds to a change to its concrete implementations. Conversely, changes to concrete implementations do not always, or even usually, require changes to the interfaces that they implement. Therefore interfaces are less volatile than implementations.

Mỗi thay đổi ở một interface trừu tượng đều kéo theo thay đổi ở các phần cài đặt cụ thể của nó. Ngược lại, thay đổi ở phần cài đặt cụ thể không phải lúc nào — thậm chí thường là không — đòi hỏi phải sửa những interface mà chúng cài đặt. Vì vậy interface ít biến động hơn phần cài đặt.

Indeed, good software designers and architects work hard to reduce the volatility of interfaces. They try to find ways to add functionality to implementations without making changes to the interfaces. This is Software **Design** 101.

Thật ra, những nhà thiết kế và kiến trúc sư phần mềm giỏi luôn nỗ lực hết sức để giảm độ biến động của interface. Họ tìm cách thêm chức năng vào phần cài đặt mà không phải đụng tới interface. Đây là kiến thức nhập môn của Thiết kế Phần mềm.

The implication, then, is that stable software architectures are those that avoid depending on volatile concretions, and that favor the use of stable abstract interfaces. This implication boils down to a set of very specific coding practices:

Hệ quả rút ra là: kiến trúc phần mềm ổn định là kiến trúc tránh phụ thuộc vào những thứ cụ thể dễ biến động, và ưu tiên dùng các interface trừu tượng ổn định. Hệ quả này quy về một tập các thực hành viết mã rất cụ thể:

- Don't refer to volatile concrete classes. Refer to abstract interfaces instead.
 - Đừng tham chiếu tới các class cụ thể (concrete class) dễ biến động. Hãy tham chiếu tới các interface trừu tượng thay cho chúng.

This rule applies in all languages, whether statically or dynamically typed. It also puts severe constraints on the creation of objects and generally enforces the use of Abstract Factories .

Quy tắc này áp dụng cho mọi ngôn ngữ, bất kể định kiểu tĩnh hay động. Nó cũng đặt ra ràng buộc rất ngặt nghèo cho việc tạo đối tượng, và nhìn chung buộc ta phải dùng Abstract

Factory.

- Don't derive from volatile concrete classes. This is a corollary to the
 - Đừng kế thừa từ các class cụ thể dễ biến động. Đây là hệ quả của

previous rule, but it bears special mention. In statically typed languages, **inheritance** is the strongest, and most rigid, of all the source code relationships; consequently, it should be used with great care. In dynamically typed languages, inheritance is less of a problem, but it is still a dependency—and caution is always the wisest choice.

quy tắc trước, nhưng nó xứng đáng được nhắc riêng. Trong các ngôn ngữ định kiểu tĩnh, kế thừa (inheritance) là quan hệ mạnh nhất và cứng nhắc nhất trong mọi quan hệ mã nguồn; do đó phải dùng hết sức thận trọng. Trong các ngôn ngữ định kiểu động, kế thừa bớt phiền hơn, nhưng nó vẫn là một phụ thuộc — và cần trọng bao giờ cũng là lựa chọn khôn ngoan nhất.

- Don't override concrete functions. Concrete functions often require source
 - Đừng override các hàm cụ thể. Các hàm cụ thể thường kéo theo phụ thuộc

code dependencies. When you override those functions, you do not eliminate those dependencies—indeed, you inherit them. To manage those dependencies, you should make the function abstract and create multiple implementations.

mã nguồn. Khi bạn override những hàm đó, bạn không hề loại bỏ được các phụ thuộc ấy — thực chất bạn thừa hưởng chúng. Muốn quản lý những phụ thuộc đó, bạn nên biến hàm thành trừu tượng rồi tạo nhiều phần cài đặt cho nó.

- Never mention the name of anything concrete and volatile. This is really
 - Đừng bao giờ nhắc tới tên của bất cứ thứ gì vừa cụ thể vừa dễ biến động. Đây thực ra

just a restatement of the principle itself.

chỉ là cách phát biểu lại chính nguyên tắc này.

Factories — Factory

To comply with these rules, the creation of volatile concrete objects requires special handling. This caution is warranted because, in virtually all languages, the creation of an object requires a source code dependency on the concrete definition of that object.

Để tuân thủ những quy tắc trên, việc tạo ra các đối tượng cụ thể dễ biến động cần được xử lý đặc biệt. Sự thận trọng này là chính đáng, bởi trong hầu như mọi ngôn ngữ, tạo một đối tượng đồng nghĩa với việc tạo một phụ thuộc mã nguồn vào định nghĩa cụ thể của đối tượng đó.

In most object-oriented languages, such as Java, we would use an Abstract Factory to manage this undesirable dependency.

Trong phần lớn các ngôn ngữ hướng đối tượng, chẳng hạn Java, ta sẽ dùng Abstract Factory để quản lý cái phụ thuộc phiền phức này.

The diagram in Figure 11.1 shows the **structure**. The Application uses the ConcreteImpl through the **Service** interface. However, the Application

Sơ đồ trong Figure 11.1 cho thấy cấu trúc đó. Application dùng ConcreteImpl thông qua interface Service. Tuy nhiên, Application

must somehow create instances of the ConcreteImpl . To achieve this without creating a source code dependency on the ConcreteImpl , the

bằng cách nào đó vẫn phải tạo ra các instance của ConcreteImpl. Để làm được điều này mà không sinh ra phụ thuộc mã nguồn vào ConcreteImpl,

Application calls the makeSvc method of the ServiceFactory interface.

This method is implemented by the ServiceFactoryImpl class, which derives from ServiceFactory . That implementation instantiates the ConcreteImpl and returns it as a Service .

Phương thức này được cài đặt bởi class ServiceFactoryImpl, class này kế thừa từ ServiceFactory. Phần cài đặt đó khởi tạo ConcreteImpl và trả về nó dưới dạng một Service.

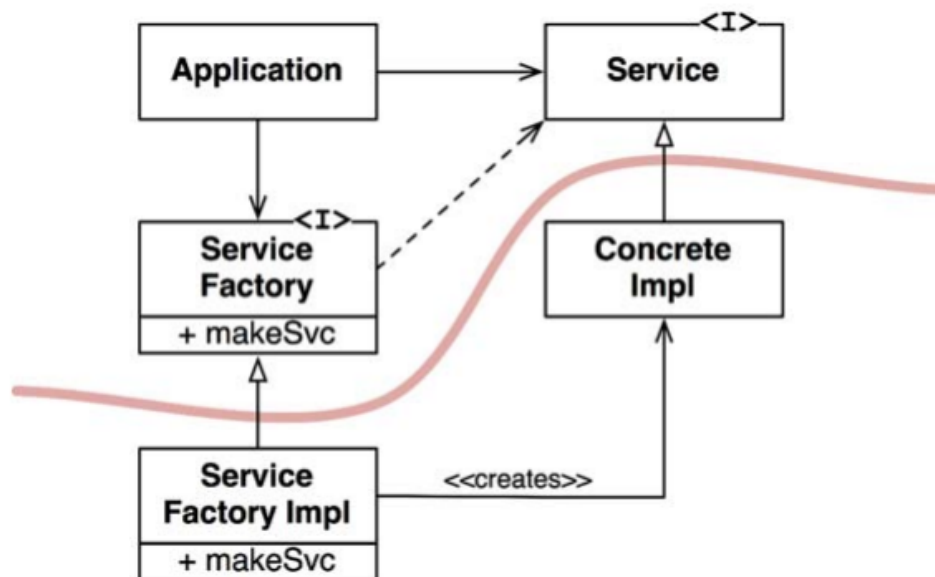


Figure 11.1 Use of the Abstract Factory pattern to manage the dependency

Figure 11.1 Dùng mẫu Abstract Factory để quản lý phụ thuộc

The curved line in Figure 11.1 is an **architectural boundary**. It separates the abstract from the concrete. All source code dependencies cross that curved line pointing in the same direction, toward the abstract side.

Đường cong trong Figure 11.1 là một ranh giới kiến trúc (architectural boundary). Nó tách phần trừu tượng khỏi phần cụ thể. Mọi phụ thuộc mã nguồn đều cắt qua đường cong ấy theo cùng một chiều, hướng về phía trừu tượng.

The curved line divides the system into two components: one abstract and the other concrete. The abstract **component** contains all the **high-level** business rules of the application. The concrete component contains all the implementation details that those business rules manipulate.

Đường cong này chia hệ thống thành hai component: một trừu tượng và một cụ thể. Component trừu tượng chứa toàn bộ quy tắc nghiệp vụ (business rule) mức cao của ứng

dụng. Component cụ thể chứa toàn bộ chi tiết cài đặt mà những quy tắc nghiệp vụ đó thao tác lên.

Note that the **flow of control** crosses the curved line in the opposite direction of the source code dependencies. The source code dependencies are inverted against the flow of control—which is why we refer to this principle as Dependency Inversion.

Để ý rằng luồng điều khiển cắt qua đường cong theo chiều ngược lại với các phụ thuộc mã nguồn. Phụ thuộc mã nguồn bị đảo ngược so với luồng điều khiển — và đó chính là lý do ta gọi nguyên tắc này là Đảo ngược Phụ thuộc (Dependency Inversion).

Concrete Components — Các component cụ thể

The concrete component in Figure 11.1 contains a single dependency, so it violates the DIP. This is typical. DIP violations cannot be entirely removed, but they can be gathered into a small number of concrete components and kept separate from the rest of the system.

Component cụ thể trong Figure 11.1 chứa đúng một phụ thuộc, nên nó vi phạm DIP. Chuyện đó là bình thường. Không thể xóa sạch mọi vi phạm DIP, nhưng có thể gom chúng lại vào một số ít component cụ thể và giữ chúng tách biệt khỏi phần còn lại của hệ thống.

Most systems will contain at least one such concrete component—often called `main` because it contains the `main` function. In the case illustrated in Figure 11.1, the `main` function would instantiate the `ServiceFactoryImpl` and place that instance in a global variable of type `ServiceFactory`. The Application would then access the factory through that global variable.

Phần lớn hệ thống sẽ có ít nhất một component cụ thể kiểu đó — thường được gọi là `main` vì nó chứa hàm `main`. Trong trường hợp minh họa ở Figure 11.1, hàm `main` sẽ khởi tạo `ServiceFactoryImpl` rồi đặt instance ấy vào một biến toàn cục kiểu `ServiceFactory`. Application sau đó truy cập factory thông qua biến toàn cục này.

Conclusion — Kết luận

As we move forward in this book and cover higher-level architectural principles, the DIP will show up again and again. It will be the most visible organizing principle in our **architecture** diagrams. The curved line in Figure 11.1 will become the architectural boundaries in later chapters. The way the dependencies cross that curved line in one direction, and toward more abstract entities, will become a new rule that we will call **the Dependency Rule**.

Càng đi xa trong cuốn sách này, càng bàn tới những nguyên tắc kiến trúc ở mức cao hơn, ta sẽ càng thấy DIP xuất hiện lặp đi lặp lại. Nó sẽ là nguyên tắc tổ chức dễ thấy nhất trong các sơ đồ kiến trúc của chúng ta. Đường cong trong Figure 11.1 sẽ trở thành những ranh giới kiến trúc ở các chương sau. Cái cách các phụ thuộc cắt qua đường cong ấy theo một chiều duy nhất, hướng về phía những thực thể trừu tượng hơn, sẽ trở thành một quy tắc mới mà ta gọi là Quy tắc Phụ thuộc (the Dependency Rule).

- 1. In other words, the function that is invoked by the operating system when the application is first started up.
 - 1. Nói cách khác, đó là hàm được hệ điều hành gọi khi ứng dụng vừa khởi động.

Component Principles — Các nguyên tắc về component

If **the SOLID principles** tell us how to arrange the bricks into walls and rooms, then the **component** principles tell us how to arrange the rooms into buildings. Large software systems, like large buildings, are built out of smaller components.

Nếu các nguyên tắc SOLID chỉ cho ta cách xếp gạch thành tường và thành phòng, thì các nguyên tắc về component chỉ cho ta cách xếp các phòng thành toà nhà. Những hệ thống phần mềm lớn, cũng như những toà nhà lớn, đều được dựng lên từ các component nhỏ hơn.

In Part IV, we will discuss what software components are, which elements should compose them, and how they should be composed together into systems.

Ở Part IV, chúng ta sẽ bàn xem component phần mềm là gì, những phần tử nào nên hợp thành chúng, và chúng nên được ghép lại với nhau thành hệ thống ra sao.

Components — Components

Components are the units of deployment. They are the smallest entities that can be deployed as part of a system. In Java, they are jar files. In Ruby, they are gem files. In .Net, they are DLLs. In compiled languages, they are aggregations of binary files. In interpreted languages, they are aggregations of source files. In all languages, they are the granule of deployment.

Component là đơn vị triển khai. Chúng là những thực thể nhỏ nhất có thể được triển khai như một phần của hệ thống. Trong Java, đó là các file jar. Trong Ruby, đó là các file gem. Trong .Net, đó là các DLL. Với ngôn ngữ biên dịch, chúng là tập hợp các file nhị phân. Với ngôn ngữ thông dịch, chúng là tập hợp các file mã nguồn. Trong mọi ngôn ngữ, chúng là hạt nhỏ nhất của việc triển khai.

Components can be linked together into a single executable. Or they can be aggregated together into a single archive, such as a .war file. Or they can be independently deployed as separate dynamically loaded plugins, such as .jar or .dll or .exe files. Regardless of how they are eventually deployed, well- designed components always retain the ability to be independently deployable and, therefore, independently developable.

Các component có thể được liên kết với nhau thành một file thực thi duy nhất. Hoặc chúng có thể được gộp lại thành một kho lưu trữ duy nhất, chẳng hạn file .war. Hoặc chúng có thể được triển khai độc lập dưới dạng những plugin nạp động riêng biệt, chẳng hạn file .jar, .dll hay .exe. Bất kể cuối cùng được triển khai theo cách nào, component được thiết kế tốt luôn giữ được khả năng triển khai độc lập, và do đó, khả năng phát triển độc lập.

A Brief History of Components — Lược sử của component

In the early years of software development, programmers controlled the memory location and layout of their programs. One of the first lines of code in a program would be the origin statement, which declared the address at which the program was to be loaded.

Những năm đầu của ngành phát triển phần mềm, lập trình viên tự kiểm soát vị trí và bố cục chương trình của mình trong bộ nhớ. Một trong những dòng mã đầu tiên của chương trình sẽ là câu lệnh origin, khai báo địa chỉ mà chương trình sẽ được nạp vào.

Consider the following simple PDP-8 program. It consists of a subroutine named GETSTR that inputs a string from the keyboard and saves it in a buffer. It also has a little unit test program to exercise GETSTR.

Hãy xem chương trình PDP-8 đơn giản sau đây. Nó gồm một subroutine tên GETSTR nhận một chuỗi từ bàn phím và lưu vào bộ đệm. Nó cũng có một chương trình unit test nho nhỏ để chạy thử GETSTR.

```

*200
START,  TLS
        CLA
        TAD BUFR
        JMS GETSTR
        CLA
        TAD BUFR
        JMS PUTSTR
        JMP START

BUFR,   3000
GETSTR, 0
        DCA PTR
NXTCH,  KSF
        JMP -1
        KRB
        DCA I PTR
        TAD I PTR
        AND K177
        ISZ PTR
        TAD MCR
        SZA
        JMP NXTCH

K177,   177
MCR,    -15

```

Note the *200 command at the start of this program. It tells the compiler to generate code that will be loaded at address 200 .

Chú ý lệnh *200 ở đầu chương trình này. Nó bảo trình biên dịch sinh ra mã sẽ được nạp vào địa chỉ 200 .

8

8

This kind of programming is a foreign concept for most programmers today. They rarely have to think about where a program is loaded in the memory of the computer. But in the early days, this was one of the first decisions a programmer needed to make. In those days, programs were not relocatable.

Kiểu lập trình này là một khái niệm xa lạ với hầu hết lập trình viên ngày nay. Họ hiếm khi phải nghĩ xem chương trình được nạp vào chỗ nào trong bộ nhớ máy tính. Nhưng thời kỳ đầu, đây là một trong những quyết định đầu tiên mà lập trình viên phải đưa ra. Thời đó, chương trình không thể tái định vị được.

How did you access a library function in those olden days? The preceding code illustrates the approach used. Programmers included the source code of the library functions with their application code, and compiled them all as a single program. 1 Libraries were kept in source, not in binary.

Vậy hồi ấy người ta gọi một hàm thư viện bằng cách nào? Đoạn mã phía trên minh họa đúng cách làm khi đó. Lập trình viên đưa mã nguồn của các hàm thư viện vào chung với mã ứng dụng của mình, rồi biên dịch tất cả thành một chương trình duy nhất. 1 Thư viện được lưu ở dạng mã nguồn, không phải dạng nhị phân.

- 1. My first employer kept several dozen decks of the subroutine library source code on a shelf. When you
 - 1. Nơi làm việc đầu tiên của tôi giữ vài chục cỗ bìa đục lỗ chứa mã nguồn thư viện subroutine trên một cái kệ. Khi bạn

wrote a new program, you simply grabbed one of those decks and slapped it onto the end of your deck.

viết một chương trình mới, bạn chỉ việc vớ lấy một cỗ bìa trong số đó và đập nó vào cuối cỗ bìa của mình.

The problem with this approach was that, during this era, devices were slow and memory was expensive and, therefore, limited. Compilers needed to make several passes over the source code, but memory was too limited to keep all the source code resident. Consequently, the compiler had to read in the source code several times using the slow devices.

Vấn đề của cách làm này là ở thời ấy thiết bị thì chậm còn bộ nhớ thì đắt, và vì thế rất hạn hẹp. Trình biên dịch cần quét mã nguồn vài lượt, nhưng bộ nhớ quá nhỏ để giữ toàn bộ mã nguồn thường trú. Hệ quả là trình biên dịch phải đọc mã nguồn nhiều lần từ những thiết bị chậm chạp kia.

This took a long time—and the larger your function library, the longer the compiler took. Compiling a large program could take hours.

Việc này mất rất nhiều thời gian — và thư viện hàm càng lớn thì trình biên dịch càng lâu. Biên dịch một chương trình lớn có thể mất hàng giờ.

To shorten the compile times, programmers separated the source code of the function library from the applications. They compiled the function library separately and loaded the binary at a known address—say, 2000 . They

Để rút ngắn thời gian biên dịch, lập trình viên tách mã nguồn của thư viện hàm ra khỏi ứng dụng. Họ biên dịch thư viện hàm riêng rồi nạp bản nhị phân vào một địa chỉ đã biết — chẳng hạn 2000 . Họ

8

8

created a symbol table for the function library and compiled that with their application code. When they wanted to run an application, they would load the binary function library, 2 and then load the application. Memory looked like the layout shown in Figure 12.1.

tạo một bảng ký hiệu cho thư viện hàm và biên dịch bảng đó cùng với mã ứng dụng. Khi muốn chạy ứng dụng, họ sẽ nạp thư viện hàm dạng nhị phân, 2 rồi nạp ứng dụng. Bộ nhớ trông giống như bố cục trong Figure 12.1.

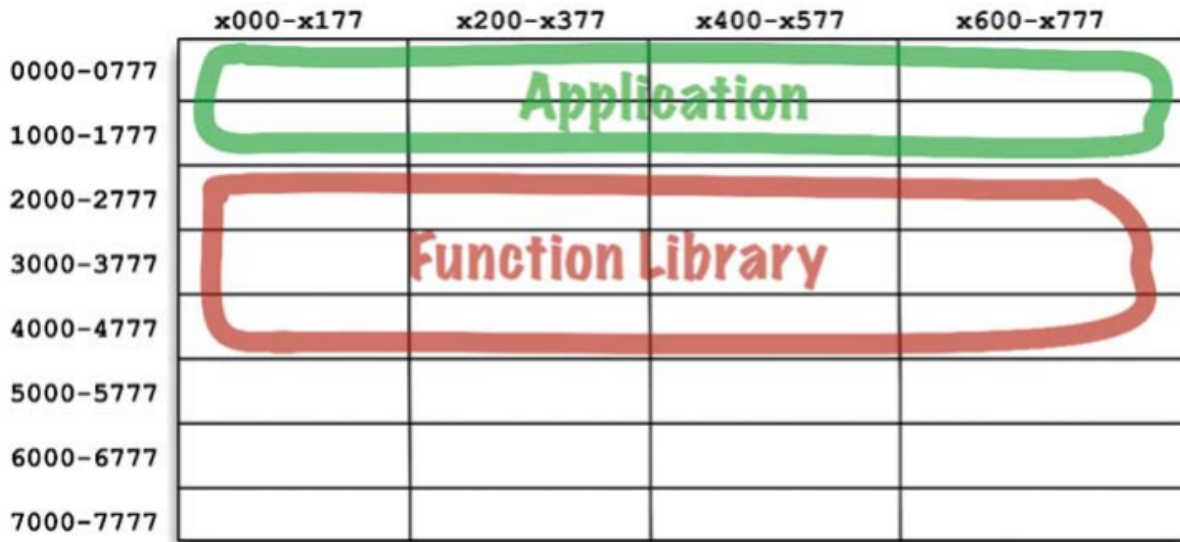


Figure 12.1 Early memory layout

Figure 12.1 Bố cục bộ nhớ thời kỳ đầu

This worked fine so long as the application could fit between addresses 0000

Cách này chạy tốt chừng nào ứng dụng còn nằm lọt giữa địa chỉ 0000

8

8

and 1777 . But soon applications grew to be larger than the space allotted

và 1777 . Nhưng chẳng bao lâu ứng dụng lớn vượt quá khoảng không gian dành

8

8

- 2. Actually, most of those old machines used core memory, which did not get erased when you powered the

- 2. Thật ra hầu hết các máy đời cũ đó dùng bộ nhớ lõi từ, thứ không bị xóa khi bạn tắt

computer down. We often left the function library loaded for days at a time.

máy tính. Chúng tôi thường để thư viện hàm nằm nguyên trong bộ nhớ suốt nhiều ngày liền.

for them. At that point, programmers had to split their applications into two address segments, jumping around the function library (Figure 12.2).

cho chúng. Đến lúc đó, lập trình viên phải cắt ứng dụng của mình thành hai đoạn địa chỉ, nhảy vòng qua thư viện hàm (Figure 12.2).

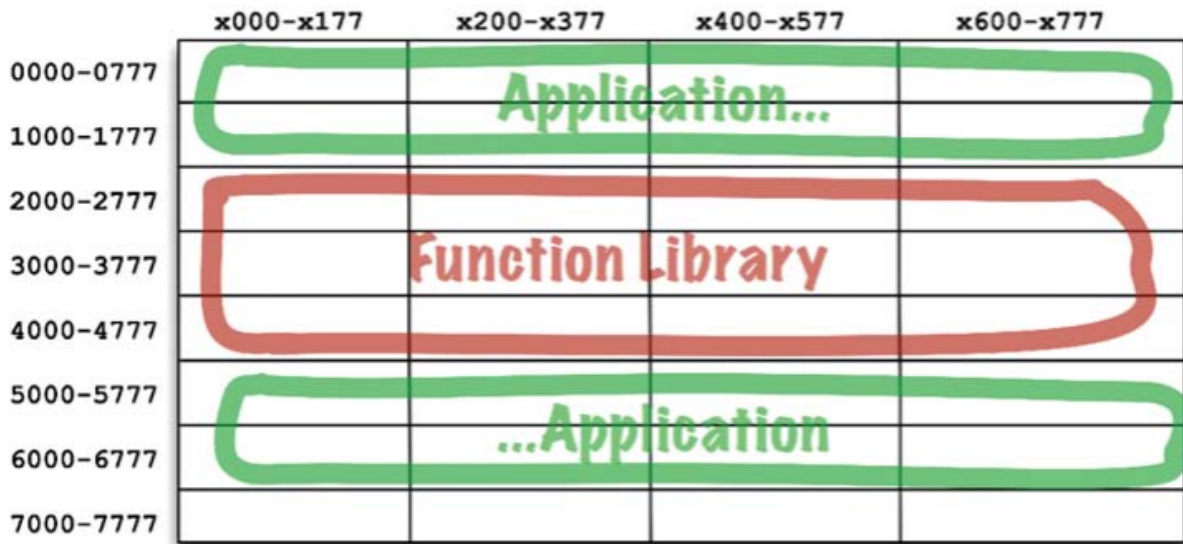


Figure 12.2 Splitting the application into two address segments

Figure 12.2 Cắt ứng dụng thành hai đoạn địa chỉ

Obviously, this was not a sustainable situation. As programmers added more functions to the function library, it exceeded its bounds, and they had to allocate more space for it (in this example, near 7000). This fragmentation of

Rõ ràng đây không phải một tình thế bền vững. Lập trình viên càng thêm hàm vào thư viện, thư viện càng tràn khỏi giới hạn của nó, và họ lại phải cấp thêm chỗ cho nó (trong ví dụ này là quanh 7000). Sự phân mảnh này của

8

8

programs and libraries necessarily continued as computer memory grew.

chương trình và thư viện tất yếu cứ tiếp diễn khi bộ nhớ máy tính lớn dần lên.

Clearly, something had to be done.

Rõ ràng phải làm gì đó.

Relocatability — Khả năng tái định vị

The solution was relocatable binaries. The idea behind them was very simple. The compiler was changed to output binary code that could be relocated in memory by a smart **loader**. The loader would be told where to load the relocatable code. The relocatable code was instrumented with flags that told the loader which parts of the loaded data had to be altered to be loaded at the selected address. Usually this just meant adding the starting address to any memory reference addresses in the binary.

Lời giải là các bản nhị phân tái định vị được. Ý tưởng đằng sau chúng rất đơn giản. Trình biên dịch được sửa để xuất ra mã nhị phân có thể được một loader thông minh dời đi trong

bộ nhớ. Loader sẽ được cho biết cần nạp mã tái định vị đó vào đâu. Mã tái định vị được gắn kèm các cờ chỉ cho loader biết phần nào của dữ liệu đã nạp cần được sửa lại để chạy được ở địa chỉ đã chọn. Thường thì việc này chỉ đơn giản là cộng địa chỉ bắt đầu vào mọi địa chỉ tham chiếu bộ nhớ trong bản nhị phân.

Now the programmer could tell the loader where to load the function library, and where to load the application. In fact, the loader would accept several binary inputs and simply load them in memory one right after the other, relocating them as it loaded them. This allowed programmers to load only those functions that they needed.

Giờ đây lập trình viên có thể bảo loader nạp thư viện hàm vào đâu, và nạp ứng dụng vào đâu. Thực tế, loader sẽ nhận nhiều đầu vào nhị phân rồi nạp chúng vào bộ nhớ nối tiếp nhau, vừa nạp vừa tái định vị. Điều này cho phép lập trình viên chỉ nạp đúng những hàm mình cần.

The compiler was also changed to emit the names of the functions as metadata in the relocatable binary. If a program called a library function, the compiler would emit that name as an external reference. If a program defined a library function, the compiler would emit that name as an external definition. Then the loader could link the external references to the external definitions once it had determined where it had loaded those definitions.

Trình biên dịch cũng được sửa để phát ra tên các hàm dưới dạng metadata trong bản nhị phân tái định vị. Nếu một chương trình gọi một hàm thư viện, trình biên dịch sẽ phát ra tên đó như một external reference. Nếu một chương trình định nghĩa một hàm thư viện, trình biên dịch sẽ phát ra tên đó như một external definition. Sau đó loader có thể liên kết các external reference với các external definition một khi nó đã xác định được mình nạp những định nghĩa ấy vào đâu.

And the linking loader was born.

Và thế là linking loader ra đời.

Linkers — Linker

The linking loader allowed programmers to divide their programs up onto separately compilable and loadable segments. This worked well when relatively small programs were being linked with relatively small libraries. However, in the late 1960s and early 1970s, programmers got more ambitious, and their programs got a lot bigger.

Linking loader cho phép lập trình viên chia chương trình của mình thành những đoạn có thể biên dịch và nạp riêng rẽ. Cách này chạy tốt khi những chương trình tương đối nhỏ được liên kết với những thư viện tương đối nhỏ. Tuy nhiên, vào cuối thập niên 1960 và đầu thập niên 1970, lập trình viên trở nên tham vọng hơn, và chương trình của họ phình to hơn nhiều.

Eventually, the linking loaders were too slow to tolerate. Function libraries were stored on slow devices such as magnetic tape. Even the disks, back then, were quite slow. Using these relatively slow devices, the linking loaders had to read dozens, if not hundreds, of binary libraries to resolve the external references. As programs grew larger and larger, and more library functions accumulated in libraries, a linking loader could take more than an hour just to load the program.

Rất cuộc, linking loader chậm tới mức không chịu nổi. Thư viện hàm được lưu trên những thiết bị chậm như băng từ. Ngay cả đĩa từ hồi đó cũng khá chậm. Với những thiết bị chậm chạp như vậy, linking loader phải đọc hàng chục, nếu không nói là hàng trăm, thư viện

nhị phân để phân giải các external reference. Khi chương trình ngày một lớn hơn, và ngày càng nhiều hàm dồn vào thư viện, một linking loader có thể mất hơn một giờ chỉ để nạp chương trình.

Eventually, the loading and the linking were separated into two phases. Programmers took the slow part—the part that did that linking—and put it into a separate application called the **linker**. The output of the linker was a linked relocatable that a relocating loader could load very quickly. This

Cuối cùng, việc nạp và việc liên kết được tách thành hai giai đoạn. Lập trình viên lấy phần chậm chạp — phần lo việc liên kết — và đưa nó vào một ứng dụng riêng gọi là linker. Đầu ra của linker là một bản tái định vị đã liên kết mà relocating loader có thể nạp rất nhanh. Cách này

allowed programmers to prepare an executable using the slow linker, but then they could load it quickly, at any time.

cho phép lập trình viên dùng linker chậm để chuẩn bị sẵn một file thực thi, rồi sau đó nạp nó thật nhanh, vào bất cứ lúc nào.

Then came the 1980s. Programmers were working in C or some other high- **level** language. As their ambitions grew, so did their programs. Programs that numbered hundreds of thousands of lines of code were not unusual.

Rồi thập niên 1980 tới. Lập trình viên làm việc bằng C hoặc một ngôn ngữ bậc cao nào đó. Tham vọng của họ lớn lên, và chương trình của họ cũng vậy. Những chương trình dài hàng trăm nghìn dòng mã không còn là chuyện hiếm.

Source modules were compiled from .c files into .o files, and then fed into the linker to create executable files that could be quickly loaded. Compiling each individual module was relatively fast, but compiling all the modules took a bit of time. The linker would then take even more time. Turnaround had again grown to an hour or more in many cases.

Các module nguồn được biên dịch từ file .c thành file .o, rồi đưa vào linker để tạo ra file thực thi có thể nạp nhanh. Biên dịch từng module riêng lẻ thì tương đối nhanh, nhưng biên dịch tất cả các module thì mất một lúc. Rồi linker còn tốn thêm nhiều thời gian nữa. Thời gian quay vòng lại lên tới một giờ hoặc hơn trong nhiều trường hợp.

It seemed as if programmers were doomed to endlessly chase their tails. Throughout the 1960s, 1970s, and 1980s, all the changes made to speed up workflow were thwarted by programmers' ambitions, and the size of the programs they wrote. They could not seem to escape from the hour-long turnaround times. Loading time remained fast, but compile-link times were the bottleneck.

Có vẻ như lập trình viên bị nguyên rủa phải mãi mãi đuổi theo cái đuôi của chính mình. Suốt các thập niên 1960, 1970 và 1980, tham vọng của lập trình viên và kích thước những chương trình họ viết ra đã vô hiệu hoá mọi thay đổi nhằm tăng tốc quy trình làm việc. Họ dường như không sao thoát khỏi cảnh thời gian quay vòng dài cả tiếng đồng hồ. Thời gian nạp vẫn nhanh, nhưng thời gian biên dịch và liên kết mới là nút thắt.

We were, of course, experiencing Murphy's law of program size:

Tất nhiên, chúng tôi đang nếm trải định luật Murphy về kích thước chương trình:

Programs will grow to fill all available compile and link time.

Chương trình sẽ phình ra cho tới khi ngốn hết mọi thời gian biên dịch và liên kết sẵn có.

But Murphy was not the only contender in town. Along came Moore, 3 and in the late 1980s, the two battled it out. Moore won that battle. Disks started to shrink and got significantly faster. Computer

memory started to get so ridiculously cheap that much of the data on disk could be cached in RAM. Computer clock rates increased from 1 MHz to 100 MHz.

Nhưng Murphy không phải kẻ duy nhất trên sân. Moore xuất hiện, 3 và vào cuối thập niên 1980, hai bên so găng. Moore thắng trận đó. Đĩa bắt đầu nhỏ lại và nhanh lên đáng kể. Bộ nhớ máy tính bắt đầu rẻ đến mức lỗ bịch, tới nỗi phần lớn dữ liệu trên đĩa có thể được cache trong RAM. Xung nhịp máy tính tăng từ 1 MHz lên 100 MHz.

By the mid-1990s, the time spent linking had begun to shrink faster than our ambitions could make programs grow. In many cases, link time decreased to a matter of seconds. For small jobs, the idea of a linking loader became feasible again.

Đến giữa thập niên 1990, thời gian liên kết bắt đầu co lại nhanh hơn tốc độ mà tham vọng của chúng ta có thể làm chương trình phình ra. Trong nhiều trường hợp, thời gian liên kết giảm xuống chỉ còn vài giây. Với những tác vụ nhỏ, ý tưởng về linking loader lại trở nên khả thi.

- 3. Moore's law: Computer speed, memory, and density double every 18 months. This law held from the
 - 3. Định luật Moore: tốc độ, bộ nhớ và mật độ của máy tính tăng gấp đôi sau mỗi 18 tháng. Định luật này đúng từ

1950s to 2000, but then, at least for clock rates, stopped cold.

thập niên 1950 tới năm 2000, nhưng sau đó, ít nhất là với xung nhịp, nó dừng phất lại.

This was the era of Active-X, shared libraries, and the beginnings of .jar files. Computers and devices had gotten so fast that we could, once again, do the linking at load time. We could link together several .jar files, or several shared libraries in a matter of seconds, and execute the resulting program. And so the **component plugin architecture** was born.

Đó là thời của Active-X, thư viện chia sẻ, và những file .jar đầu tiên. Máy tính và thiết bị đã nhanh tới mức chúng ta lại có thể thực hiện việc liên kết ngay lúc nạp. Chúng ta có thể liên kết vài file .jar, hoặc vài thư viện chia sẻ, chỉ trong vài giây, rồi chạy chương trình thu được. Và thế là kiến trúc plugin dạng component ra đời.

Today we routinely ship .jar files or DLLs or shared libraries as plugins to existing applications. If you want to create a mod to Minecraft, for example, you simply include your custom .jar files in a certain folder. If you want to plug Resharper into Visual Studio, you simply include the appropriate DLLs.

Ngày nay chúng ta thường xuyên phát hành file .jar hoặc DLL hoặc thư viện chia sẻ như những plugin cắm vào các ứng dụng sẵn có. Chẳng hạn, nếu bạn muốn tạo một bản mod cho Minecraft, bạn chỉ việc đặt file .jar tùy chỉnh của mình vào một thư mục nhất định. Nếu bạn muốn cắm Resharper vào Visual Studio, bạn chỉ việc đặt vào đó những DLL thích hợp.

Conclusion — Kết luận

These dynamically linked files, which can be plugged together at runtime, are the software components of our architectures. It has taken 50 years, but we have arrived at a place where component plugin **architecture** can be the casual default as opposed to the herculean **effort** it once was.

Những file liên kết động ấy, thứ có thể ghép lại với nhau lúc chạy, chính là các software component trong kiến trúc của chúng ta. Phải mất 50 năm, nhưng chúng ta đã đến được chỗ mà kiến trúc plugin dạng component có thể là lựa chọn mặc định nhẹ nhàng, thay vì một nỗ lực phi thường như xưa.

Component Cohesion — Độ gắn kết của component

Which classes belong in which components? This is an **important** decision, and requires guidance from good software engineering principles. Unfortunately, over the years, this decision has been made in an ad hoc manner based almost entirely on context.

Class nào thì thuộc về component nào? Đây là một quyết định quan trọng, và nó cần được dẫn dắt bởi những nguyên tắc kỹ nghệ phần mềm tử tế. Đáng tiếc là suốt nhiều năm qua, quyết định này thường được đưa ra một cách tùy hứng, gần như hoàn toàn dựa vào ngữ cảnh.

In this chapter we will discuss the three principles of **component cohesion**:

Trong chương này chúng ta sẽ bàn về ba nguyên tắc gắn kết của component:

- REP: The Reuse/**Release Equivalence Principle**
 - REP — Nguyên tắc Tương đương Tái dùng/Phát hành (The Reuse/Release Equivalence Principle)
- CCP: The Common Closure Principle
 - CCP — Nguyên tắc Đóng chung (The Common Closure Principle)
- CRP: The Common Reuse Principle
 - CRP — Nguyên tắc Tái dùng chung (The Common Reuse Principle)

The Reuse/Release Equivalence Principle — Nguyên tắc Tương đương Tái dùng/Phát hành

The granule of reuse is the granule of release.

Đơn vị tái dùng chính là đơn vị phát hành.

The last decade has seen the rise of a menagerie of module management tools, such as Maven, Leiningen, and RVM. These tools have grown in importance because, during that time, a vast number of reusable components and **component** libraries have been created. We are now living in the age of software reuse—a fulfillment of one of the oldest promises of the object-oriented model.

Thập kỷ vừa qua đã chứng kiến sự nở rộ của cả một “vườn thú” công cụ quản lý module, chẳng hạn Maven, Leiningen và RVM. Những công cụ này ngày càng quan trọng, bởi trong khoảng thời gian đó, vô số component và thư viện component có thể tái dùng đã ra đời.

Chúng ta đang sống trong thời đại tái dùng phần mềm — sự hiện thực hoá một trong những lời hứa xưa cũ nhất của mô hình hướng đối tượng.

The Reuse/Release Equivalence Principle (REP) is a principle that seems obvious, at least in hindsight. People who want to reuse software components cannot, and will not, do so unless those components are tracked through a release process and are given release numbers.

REP là một nguyên tắc trông có vẻ hiển nhiên, ít nhất là khi nhìn lại. Người muốn tái dùng các component phần mềm sẽ không thể, và cũng không chịu, làm điều đó trừ khi các component ấy được theo dõi qua một quy trình phát hành và được gán số hiệu phát hành.

This is not simply because, without release numbers, there would be no way to ensure that all the reused components are compatible with each other. Rather, it also reflects the fact that software developers need to know when new releases are coming, and which changes those new releases will bring.

Lý do không đơn thuần chỉ là: thiếu số hiệu phát hành thì chẳng có cách nào bảo đảm mọi component được tái dùng tương thích với nhau. Nó còn phản ánh một thực tế khác: lập trình viên cần biết khi nào có bản phát hành mới, và bản mới ấy mang theo những thay đổi gì.

It is not uncommon for developers to be alerted about a new release and decide, based on the changes made in that release, to continue to use the old

Chuyện thường gặp là lập trình viên được thông báo về một bản phát hành mới, rồi căn cứ vào những thay đổi trong bản đó mà quyết định tiếp tục dùng bản

release instead. Therefore the release process must produce the appropriate notifications and release documentation so that users can make informed decisions about when and whether to integrate the new release.

cũ. Vì vậy, quy trình phát hành phải sinh ra thông báo và tài liệu phát hành thích đáng, để người dùng có thể quyết định một cách sáng suốt xem có nên tích hợp bản mới hay không, và khi nào thì tích hợp.

From a software **design** and **architecture** point of view, this principle means that the classes and modules that are formed into a component must belong to a cohesive group. The component cannot simply consist of a random hodgepodge of classes and modules; instead, there must be some overarching theme or purpose that those modules all share.

Nhìn từ góc độ thiết kế và kiến trúc phần mềm, nguyên tắc này có nghĩa: các class và module được gom vào một component phải thuộc về một nhóm gắn kết. Component không thể chỉ là một mớ class và module hồ lộn ghép đại với nhau; phải có một chủ đề hay mục đích bao trùm mà tất cả các module đó cùng chia sẻ.

Of course, this should be obvious. However, there is another way to look at this issue that is perhaps not quite so obvious. Classes and modules that are grouped together into a component should be releasable together. The fact that they share the same version number and the same release tracking, and are included under the same release documentation, should make sense both to the author and to the users.

Dĩ nhiên, điều đó là hiển nhiên. Nhưng còn một cách nhìn khác về vấn đề này, có lẽ không hiển nhiên đến thế. Các class và module được gom lại thành một component thì phải phát hành được cùng nhau. Việc chúng dùng chung số hiệu phiên bản, chung cơ chế theo dõi phát hành, và cùng nằm trong một tài liệu phát hành, phải hợp lý cả với tác giả lẫn với người dùng.

This is weak advice: Saying that something should “make sense” is just a way of waving your hands in the air and trying to sound authoritative. The advice is weak because it is hard to precisely explain the glue that holds the classes and modules together into a single component. Weak though the advice may be, the principle itself is important, because violations are easy to detect— they don’t “make sense.” If you violate the REP, your users will know, and they won’t be impressed with your architectural skills.

Đây là một lời khuyên yếu: nói rằng cái gì đó phải “hợp lý” chẳng qua là khoát tay trong không khí và cố tỏ ra có thẩm quyền. Lời khuyên này yếu vì rất khó giải thích chính xác thứ keo nào gắn các class và module lại thành một component duy nhất. Dù lời khuyên có yếu, bản thân nguyên tắc vẫn quan trọng, bởi vi phạm nó thì dễ phát hiện — chúng “không hợp lý”. Nếu bạn vi phạm REP, người dùng của bạn sẽ nhận ra, và họ sẽ chẳng nề phục tay nghề kiến trúc của bạn.

The weakness of this principle is more than compensated for by the strength of the next two principles. Indeed, the CCP and the CRP strongly define this principle, but in a negative sense.

Điểm yếu của nguyên tắc này được bù đắp quá đủ bởi sức mạnh của hai nguyên tắc tiếp theo. Thật vậy, CCP và CRP định nghĩa nguyên tắc này rất chặt, nhưng theo chiều phủ định.

The Common Closure Principle — Nguyên tắc Đóng chung

Gather into components those classes that change for the same reasons and at the same times. Separate into different components those classes that change at different times and for different reasons.

Hãy gom vào cùng một component những class thay đổi vì cùng lý do và vào cùng thời điểm. Hãy tách sang các component khác nhau những class thay đổi vào những thời điểm khác nhau và vì những lý do khác nhau.

This is the Single Responsibility Principle restated for components. Just as the SRP says that a class should not contain multiple reasons to change, so the Common Closure Principle (CCP) says that a component should not have multiple reasons to change.

Đây chính là SRP được phát biểu lại cho component. Cũng như SRP nói rằng một class không nên chứa nhiều lý do để thay đổi, CCP nói rằng một component không nên có nhiều lý do để thay đổi.

For most applications, maintainability is more important than reusability. If the code in an application must change, you would rather that all of the changes occur in one component, rather than being distributed across many components. 1 If changes are confined to a single component, then we need to redeploy only the one changed component. Other components that don’t depend on the changed component do not need to be revalidated or redeployed.

Với phần lớn ứng dụng, khả năng bảo trì quan trọng hơn khả năng tái dùng. Nếu mã trong một ứng dụng buộc phải thay đổi, bạn sẽ muốn mọi thay đổi ấy dồn vào một component, thay vì rải rác khắp nhiều component. 1 Nếu thay đổi bị giới hạn trong một component duy nhất, ta chỉ cần triển khai lại đúng component đã đổi. Những component khác không phụ thuộc vào nó thì chẳng cần kiểm định lại hay triển khai lại.

The CCP prompts us to gather together in one place all the classes that are likely to change for the same reasons. If two classes are so tightly bound, either physically or conceptually, that they always change together, then they belong in the same component. This minimizes the workload related to releasing, revalidating, and redeploying the software.

CCP thúc chúng ta gom về một chỗ tất cả các class có khả năng thay đổi vì cùng lý do. Nếu hai class ràng buộc chặt với nhau đến mức — về mặt vật lý hay khái niệm — chúng luôn thay đổi cùng nhau, thì chúng thuộc về cùng một component. Điều này giảm thiểu khối lượng công việc liên quan tới phát hành, kiểm định lại và triển khai lại phần mềm.

This principle is closely associated with the Open Closed Principle (OCP). Indeed, it is “closure” in the OCP sense of the word that the CCP addresses. The OCP states that classes should be closed for modification but open for extension. Because 100% closure is not attainable, closure must be strategic. We design our classes such that they are closed to the most common kinds of changes that we expect or have experienced.

Nguyên tắc này gắn bó chặt chẽ với OCP. Thật vậy, chính chữ “đóng” (closure) theo nghĩa của OCP mới là thứ mà CCP nhắm tới. OCP phát biểu rằng class phải đóng với việc sửa đổi nhưng mở với việc mở rộng. Vì không thể đạt được mức đóng 100%, việc đóng phải mang tính chiến lược. Ta thiết kế các class sao cho chúng đóng với những kiểu thay đổi phổ biến nhất mà ta dự đoán được hoặc đã từng trải qua.

The CCP amplifies this lesson by gathering together into the same component those classes that are closed to the same types of changes. Thus, when a change in requirements comes along, that change has a good chance of being restricted to a minimal number of components.

CCP khuếch đại bài học đó bằng cách gom vào cùng một component những class cùng đóng trước cùng một loại thay đổi. Nhờ vậy, khi một thay đổi về yêu cầu ập tới, thay đổi ấy có nhiều khả năng chỉ bó hẹp trong số ít component nhất có thể.

- 1. See the section on “The **Kitty Problem**” in Chapter 27, “Services: Great and Small.”
 - 1. Xem mục “Bài toán mèo cưng” trong Chương 27, “Services: Great and Small.”

Similarity with SRP — Điểm tương đồng với SRP

As stated earlier, the CCP is the component form of the SRP. The SRP tells us to separate methods into different classes, if they change for different reasons. The CCP tells us to separate classes into different components, if they change for different reasons. Both principles can be summarized by the following sound bite:

Như đã nói ở trên, CCP là dạng component của SRP. SRP bảo ta tách các method sang những class khác nhau, nếu chúng thay đổi vì những lý do khác nhau. CCP bảo ta tách các class sang những component khác nhau, nếu chúng thay đổi vì những lý do khác nhau. Cả hai nguyên tắc đều có thể tóm gọn trong một câu:

Gather together those things that change at the same times and for the same reasons. Separate those things that change at different times or for different reasons.

Gom lại với nhau những thứ thay đổi vào cùng thời điểm và vì cùng lý do. Tách rời những thứ thay đổi vào những thời điểm khác nhau hoặc vì những lý do khác nhau.

The Common Reuse Principle — Nguyên tắc Tái dùng chung

Don’t force users of a component to depend on things they don’t need.

Đừng ép người dùng của một component phải phụ thuộc vào những thứ họ không cần.

The Common Reuse Principle (CRP) is yet another principle that helps us to decide which classes and modules should be placed into a component. It states that classes and modules that tend to be reused together belong in the same component.

CRP là thêm một nguyên tắc nữa giúp ta quyết định class và module nào nên đặt vào component nào. Nó phát biểu rằng những class và module thường được tái dùng cùng nhau thì thuộc về cùng một component.

Classes are seldom reused in isolation. More typically, reusable classes collaborate with other classes that are part of the reusable abstraction. The CRP states that these classes belong together in the same component. In such a component we would expect to see classes that have lots of dependencies on each other.

Class hiếm khi được tái dùng đơn độc. Thông thường hơn, các class tái dùng được sẽ cộng tác với những class khác nằm trong cùng một trừu tượng tái dùng được. CRP phát biểu rằng các class ấy thuộc về nhau, trong cùng một component. Trong một component như thế, ta sẽ trông đợi thấy những class có rất nhiều phụ thuộc lẫn nhau.

A simple example might be a container class and its associated iterators. These classes are reused together because they are tightly coupled to each other. Thus they ought to be in the same component.

Một ví dụ đơn giản là một class container và các iterator đi kèm nó. Những class này được tái dùng cùng nhau vì chúng kết dính chặt với nhau. Vậy nên chúng phải nằm trong cùng một component.

But the CRP tells us more than just which classes to put together into a component: It also tells us which classes not to keep together in a component. When one component uses another, a dependency is created between the components. Perhaps the using component uses only one class within the used

Nhưng CRP nói với ta nhiều hơn là chỉ những class nào nên gom chung vào một component: nó còn nói cho ta biết class nào không nên giữ chung trong một component. Khi một component dùng một component khác, một phụ thuộc được tạo ra giữa hai component. Có thể component sử dụng chỉ dùng đúng một class bên trong component

component—but that still doesn't weaken the dependency. The using component still depends on the used component.

được sử dụng — nhưng điều đó vẫn không làm phụ thuộc kia yếu đi. Component sử dụng vẫn phụ thuộc vào component được sử dụng.

Because of that dependency, every time the used component is changed, the using component will likely need corresponding changes. Even if no changes are necessary to the using component, it will likely still need to be recompiled, revalidated, and redeployed. This is true even if the using component doesn't care about the change made in the used component.

Vì phụ thuộc ấy, mỗi lần component được sử dụng thay đổi, component sử dụng nhiều khả năng cũng phải thay đổi theo. Ngay cả khi component sử dụng chẳng cần sửa gì, nó vẫn nhiều khả năng phải được biên dịch lại, kiểm định lại và triển khai lại. Điều này đúng ngay cả khi component sử dụng chẳng bận tâm gì tới thay đổi vừa diễn ra bên trong component được sử dụng.

Thus when we depend on a component, we want to make sure we depend on every class in that component. Put another way, we want to make sure that the classes that we put into a component are inseparable—that it is impossible to depend on some and not on the others. Otherwise, we will be redeploying more components than is necessary, and wasting significant **effort**.

Vì vậy, khi ta phụ thuộc vào một component, ta muốn chắc rằng mình phụ thuộc vào mọi class trong component đó. Nói cách khác, ta muốn chắc rằng các class ta đặt vào một component là không thể tách rời — rằng không thể phụ thuộc vào vài class trong đó mà không phụ thuộc vào những class còn lại. Bằng không, ta sẽ phải triển khai lại nhiều component hơn mức cần thiết, và phí phạm công sức đáng kể.

Therefore the CRP tells us more about which classes shouldn't be together than about which classes should be together. The CRP says that classes that are not tightly bound to each other should not be in the same component.

Do đó, CRP nói cho ta biết về những class không nên ở cùng nhau nhiều hơn là về những class nên ở cùng nhau. CRP nói rằng các class không ràng buộc chặt với nhau thì không nên nằm trong cùng một component.

Relation to ISP — Quan hệ với ISP

The CRP is the generic version of the ISP. The ISP advises us not to depend on classes that have methods we don't use. The CRP advises us not to depend on components that have classes we don't use.

CRP là phiên bản tổng quát của ISP. ISP khuyên ta đừng phụ thuộc vào những class có các method mà ta không dùng. CRP khuyên ta đừng phụ thuộc vào những component có các class mà ta không dùng.

All of this advice can be reduced to a single sound bite:

Tất cả những lời khuyên này có thể rút gọn thành một câu duy nhất:

Don't depend on things you don't need.

Đừng phụ thuộc vào những thứ bạn không cần.

The Tension Diagram for Component Cohesion — Sơ đồ căng kéo cho độ gắn kết của component

You may have already realized that the three cohesion principles tend to fight each other. The REP and CCP are inclusive principles: Both tend to make

Có lẽ bạn đã nhận ra rằng ba nguyên tắc gắn kết này có xu hướng chống lại nhau. REP và CCP là những nguyên tắc bao hàm: cả hai đều có xu hướng làm

components larger. The CRP is an exclusive principle, driving components to be smaller. It is the tension between these principles that good architects seek to resolve.

component phình to. CRP là nguyên tắc loại trừ, đẩy component nhỏ lại. Chính sự căng kéo giữa các nguyên tắc này là thứ mà kiến trúc sư giỏi tìm cách hoá giải.

Figure 13.1 is a tension diagram 2 that shows how the three principles of cohesion interact with each other. The edges of the diagram describe the cost of abandoning the principle on the opposite vertex.

Figure 13.1 là một sơ đồ căng kéo 2 cho thấy ba nguyên tắc gắn kết tương tác với nhau ra sao. Các cạnh của sơ đồ mô tả cái giá phải trả khi từ bỏ nguyên tắc nằm ở đỉnh đối diện.

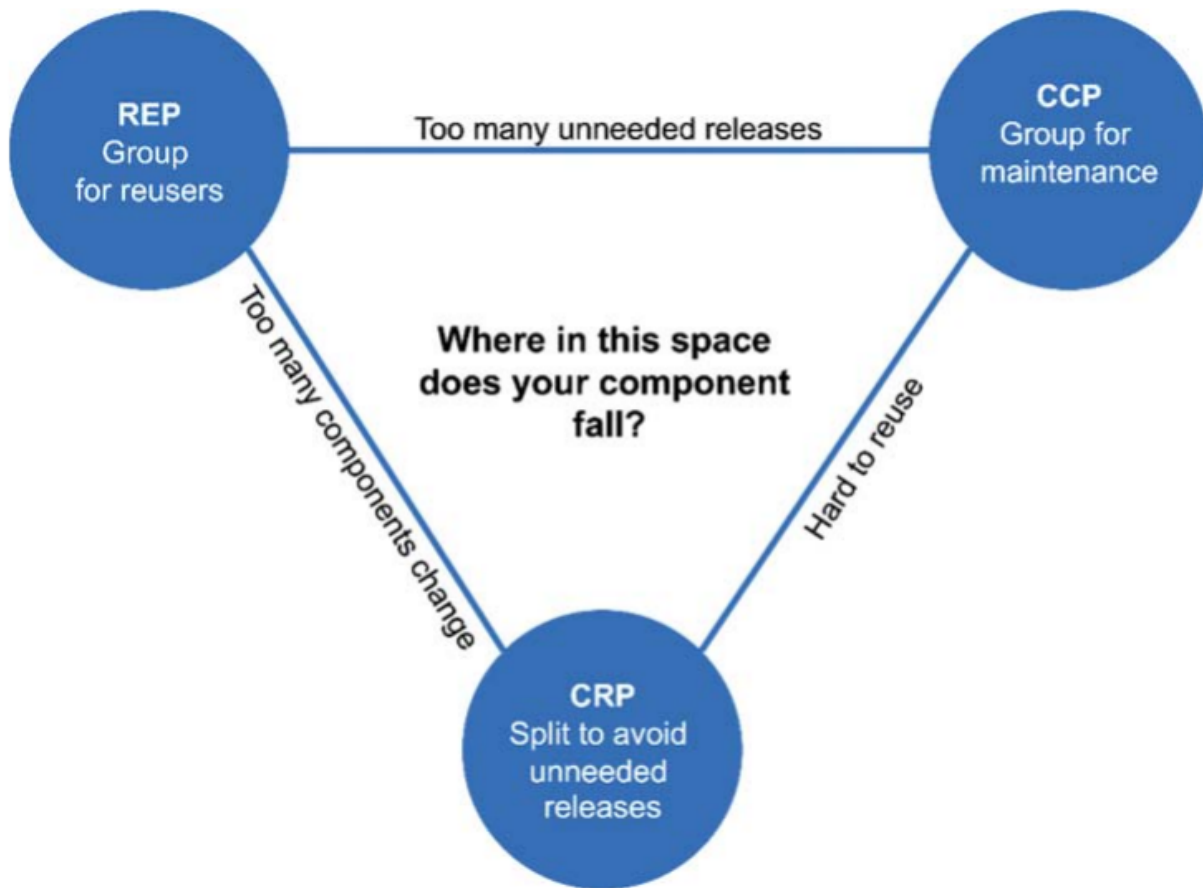


Figure 13.1 Cohesion principles tension diagram

Figure 13.1 Sơ đồ căng kéo giữa các nguyên tắc gắn kết

An architect who focuses on just the REP and CRP will find that too many components are impacted when simple changes are made. In contrast, an architect who focuses too strongly on the CCP and REP will cause too many unneeded releases to be generated.

Một kiến trúc sư chỉ chăm chăm vào REP và CRP sẽ thấy rằng có quá nhiều component bị ảnh hưởng mỗi khi thực hiện những thay đổi đơn giản. Ngược lại, kiến trúc sư quá nghiêng về CCP và REP sẽ khiến quá nhiều bản phát hành không cần thiết được sinh ra.

- 2. Thanks to Tim Ottinger for this idea.
- 2. Cảm ơn Tim Ottinger vì ý tưởng này.

A good architect finds a position in that tension triangle that meets the current concerns of the development team, but is also aware that those concerns will change over time. For example, early in the development of a project, the CCP is much more important than the REP, because developability is more important than reuse.

Một kiến trúc sư giỏi sẽ tìm được một vị trí trong tam giác căng kéo đó, vị trí đáp ứng những mối bận tâm hiện tại của đội phát triển, đồng thời ý thức được rằng những mối bận tâm ấy sẽ đổi thay theo thời gian. Chẳng hạn, ở giai đoạn đầu của một dự án, CCP quan trọng hơn REP rất nhiều, bởi khả năng phát triển quan trọng hơn khả năng tái dùng.

Generally, projects tend to start on the right hand side of the triangle, where the only sacrifice is reuse. As the project matures, and other projects begin to draw from it, the project will slide over to the left. This means that the component **structure** of a project can vary with time and maturity. It has more to do with the way that project is developed and used, than with what the project actually does.

Nhìn chung, các dự án có xu hướng khởi đầu ở cạnh bên phải của tam giác, nơi thứ duy nhất bị hy sinh là khả năng tái dùng. Khi dự án chín dần, và các dự án khác bắt đầu tái dùng mã của nó, dự án sẽ trượt dần sang trái. Điều này có nghĩa là cấu trúc component của một dự án có thể biến đổi theo thời gian và độ chín. Nó liên quan tới cách dự án được phát triển và được sử dụng nhiều hơn là tới việc dự án thực sự làm gì.

Conclusion — Kết luận

In the past, our view of cohesion was much simpler than the REP, CCP, and CRP implied. We once thought that cohesion was simply the attribute that a module performs one, and only one, function. However, the three principles of component cohesion describe a much more complex variety of cohesion. In choosing the classes to group together into components, we must consider the opposing forces involved in reusability and develop-ability. Balancing these forces with the needs of the application is nontrivial. Moreover, the balance is almost always dynamic. That is, the partitioning that is appropriate today might not be appropriate next year. As a consequence, the composition of the components will likely jitter and evolve with time as the focus of the project changes from develop-ability to reusability.

Trong quá khứ, cách chúng ta nhìn nhận độ gắn kết đơn giản hơn nhiều so với những gì REP, CCP và CRP hàm ý. Đã có thời ta nghĩ gắn kết chỉ đơn thuần là thuộc tính “một module thực hiện một, và chỉ một, chức năng”. Nhưng ba nguyên tắc gắn kết của component lại mô tả một dạng gắn kết phức tạp hơn thế nhiều. Khi chọn các class để gom vào component, ta phải cân nhắc những lực đối kháng giữa khả năng tái dùng và khả năng phát triển. Cân bằng những lực ấy với nhu cầu của ứng dụng là chuyện không hề tầm thường. Hơn nữa, thế cân bằng đó gần như luôn động. Nghĩa là cách phân hoạch phù hợp hôm nay có thể không còn phù hợp vào năm sau. Hệ quả là thành phần cấu tạo nên các component nhiều khả năng sẽ rung lắc và tiến hoá theo thời gian, khi trọng tâm của dự án chuyển dịch từ khả năng phát triển sang khả năng tái dùng.

Component Coupling — Độ kết dính giữa các component

The next three principles deal with the relationships between components. Here again we will run into the tension between develop-ability and logical **design**. The forces that impinge upon the **architecture** of a **component structure** are technical, political, and **volatile**.

Ba nguyên tắc tiếp theo bàn về mối quan hệ giữa các component. Ở đây, một lần nữa ta lại đụng phải sự giằng co giữa khả năng phát triển và thiết kế logic. Những lực tác động lên kiến trúc (architecture) của một cấu trúc component vừa mang tính kỹ thuật, vừa mang tính chính trị, lại vừa rất dễ biến động (volatile).

The Acyclic Dependencies Principle — Nguyên tắc Phụ thuộc Phi chu trình (The Acyclic Dependencies Principle)

Allow no cycles in the component dependency graph.

Không cho phép tồn tại chu trình nào trong đồ thị phụ thuộc giữa các component.

Have you ever worked all day, gotten some stuff working, and then gone home, only to arrive the next morning to find that your stuff no longer works? Why doesn't it work? Because somebody stayed later than you and changed something you depend on! I call this “**the morning after syndrome**.”

Bạn đã bao giờ làm việc cả ngày trời, cho chạy được kha khá thứ, rồi ra về, để sáng hôm sau tới nơi và phát hiện những thứ đó không còn chạy nữa chưa? Vì sao nó không chạy? Vì có ai đó ở lại muộn hơn bạn và đã sửa một thứ mà bạn phụ thuộc vào! Tôi gọi đây là hội chứng “sáng hôm sau” (the morning after syndrome).

The “morning after syndrome” occurs in development environments where many developers are modifying the same source files. In relatively small projects with just a few developers, it isn't too big a problem. But as the size of the project and the development team grow, the mornings after can get pretty nightmarish. It is not uncommon for weeks to go by without the team being able to build a **stable** version of the project. Instead, everyone keeps on changing and changing their code trying to make it work with the last changes that someone else made.

Hội chứng “sáng hôm sau” xảy ra ở những môi trường phát triển mà nhiều lập trình viên cùng sửa chung một tập tệp nguồn. Trong các dự án tương đối nhỏ với vài lập trình viên, nó chưa phải vấn đề lớn. Nhưng khi quy mô dự án và đội phát triển phình ra, những buổi sáng hôm sau có thể biến thành ác mộng. Chuyện nhiều tuần liền trôi qua mà đội không build nổi một phiên bản ổn định của dự án là chuyện không hiếm. Thay vào đó, ai nấy cứ

liên tục sửa đi sửa lại mã của mình để cố cho nó chạy được với những thay đổi mới nhất mà người khác vừa tạo ra.

Over the last several decades, two solutions to this problem have evolved, both of which came from the telecommunications industry. The first is “the **weekly build**,” and the second is the Acyclic Dependencies Principle (ADP).

Trong vài thập kỷ qua, hai giải pháp cho vấn đề này đã dần hình thành, và cả hai đều đến từ ngành viễn thông. Thứ nhất là “build hằng tuần” (the weekly build), thứ hai là Acyclic Dependencies Principle (ADP).

The Weekly Build — Build hằng tuần

The weekly build used to be common in medium-sized projects. It works like this: All the developers ignore each other for the first four days of the week. They all work on private copies of the code, and don’t worry about integrating their work on a collective basis. Then, on Friday, they integrate all their changes and build the system.

Build hằng tuần từng rất phổ biến trong các dự án cỡ vừa. Nó vận hành thế này: bốn ngày đầu tuần, tất cả lập trình viên mặc kệ nhau. Ai cũng làm việc trên bản sao mã nguồn riêng của mình, chẳng bận tâm chuyện tích hợp công việc trên bình diện chung. Rồi đến thứ Sáu, họ gộp tất cả thay đổi lại và build cả hệ thống.

This approach has the wonderful advantage of allowing the developers to live in an isolated world for four days out of five. The disadvantage, of course, is the large integration penalty that is paid on Friday.

Cách làm này có một ưu điểm tuyệt vời: nó cho phép lập trình viên sống trong một thế giới cách biệt suốt bốn trên năm ngày. Nhược điểm, dĩ nhiên, là cái giá tích hợp khổng lồ phải trả vào thứ Sáu.

Unfortunately, as the project grows, it becomes less feasible to finish integrating the project on Friday. The integration burden grows until it starts

Đáng tiếc, khi dự án lớn lên, việc tích hợp xong xuôi trong ngày thứ Sáu ngày càng bất khả thi. Gánh nặng tích hợp cứ phình ra cho tới lúc nó bắt đầu

to overflow into Saturday. A few such Saturdays are enough to convince the developers that integration should really begin on Thursday—and so the start of integration slowly creeps toward the middle of the week.

tràn sang cả thứ Bảy. Chỉ vài ngày thứ Bảy như vậy là đủ để thuyết phục các lập trình viên rằng tích hợp thật ra nên bắt đầu từ thứ Năm — và thế là thời điểm khởi động tích hợp cứ từ từ bò dần về giữa tuần.

As the duty cycle of development versus integration decreases, the efficiency of the team decreases, too. Eventually this situation becomes so frustrating that the developers, or the project managers, declare that the schedule should be changed to a biweekly build. This suffices for a time, but the integration time continues to grow with project size.

Khi tỷ lệ giữa thời gian phát triển và thời gian tích hợp giảm xuống, hiệu suất của cả đội cũng giảm theo. Rốt cuộc, tình cảnh này trở nên bức bối tới mức các lập trình viên, hoặc các quản lý dự án, tuyên bố phải đổi lịch sang build hai tuần một lần. Cách đó đủ dùng một thời gian, nhưng thời gian tích hợp vẫn tiếp tục tăng theo quy mô dự án.

Eventually, this scenario leads to a crisis. To maintain efficiency, the build schedule has to be continually lengthened—but lengthening the build schedule increases project risks. Integration and testing become increasingly harder to do, and the team loses the benefit of rapid feedback.

Cuối cùng, kịch bản này dẫn tới khủng hoảng. Muốn giữ hiệu suất thì lịch build phải kéo dài ra mãi — nhưng kéo dài lịch build lại làm tăng rủi ro dự án. Tích hợp và kiểm thử ngày một khó khăn, còn cả đội thì mất đi lợi ích của phản hồi nhanh.

Eliminating Dependency Cycles — Loại bỏ chu trình phụ thuộc

The solution to this problem is to partition the development environment into releasable components. The components become units of work that can be the responsibility of a single developer, or a team of developers. When developers get a component working, they release it for use by the other developers. They give it a release number and move it into a directory for other teams to use. They then continue to modify their component in their own private areas. Everyone else uses the released version.

Giải pháp cho vấn đề này là chia môi trường phát triển thành những component có thể phát hành được. Các component trở thành đơn vị công việc, có thể do một lập trình viên hoặc một nhóm chịu trách nhiệm. Khi lập trình viên làm cho component của mình chạy được, họ phát hành nó cho những người khác dùng. Họ gán cho nó một số phát hành và đưa nó vào một thư mục để các nhóm khác sử dụng. Sau đó họ tiếp tục sửa component của mình trong khu vực riêng của họ. Mọi người khác dùng phiên bản đã phát hành.

As new releases of a component are made available, other teams can decide whether they will immediately adopt the new release. If they decide not to, they simply continue using the old release. Once they decide that they are ready, they begin to use the new release.

Khi các bản phát hành mới của một component xuất hiện, những nhóm khác có thể tự quyết định có dùng ngay bản mới hay không. Nếu quyết định là không, họ cứ tiếp tục dùng bản cũ. Đến khi thấy đã sẵn sàng, họ mới chuyển sang bản mới.

Thus no team is at the mercy of the others. Changes made to one component do not need to have an immediate affect on other teams. Each team can decide for itself when to adapt its own components to new releases of the components. Moreover, integration happens in small increments. There is no single point in time when all developers must come together and integrate everything they are doing.

Nhờ vậy, không nhóm nào bị phó mặc cho nhóm khác định đoạt. Thay đổi ở một component không nhất thiết phải tác động tức thì lên các nhóm khác. Mỗi nhóm tự quyết định khi nào thì điều chỉnh component của mình theo các bản phát hành mới của những component kia. Hơn nữa, tích hợp diễn ra theo từng bước nhỏ. Không còn một thời điểm duy nhất nào mà mọi lập trình viên phải ngồi lại với nhau và tích hợp tất cả những gì họ đang làm.

This is a very simple and rational process, and it is widely used. To make it work successfully, however, you must manage the dependency structure of the components. There can be no cycles. If there are cycles in the dependency structure, then the “morning after syndrome” cannot be avoided.

Đây là một quy trình rất đơn giản, rất hợp lý, và được dùng rộng rãi. Tuy nhiên, để nó chạy trơn trượt, bạn phải quản lý được cấu trúc phụ thuộc giữa các component. Không được có chu trình. Nếu trong cấu trúc phụ thuộc có chu trình thì hội chứng “sáng hôm sau” là không thể tránh khỏi.

Consider the component diagram in Figure 14.1. It shows a rather typical structure of components assembled into an application. The function of this application is unimportant for the purpose of

this example. What is **important** is the dependency structure of the components. Notice that this structure is a directed graph . The components are the nodes , and the dependency relationships are the directed edges .

Hãy xem sơ đồ component ở Figure 14.1. Nó cho thấy một cấu trúc khá điển hình của các component được ráp lại thành một ứng dụng. Chức năng của ứng dụng này không quan trọng đối với ví dụ đang bàn. Điều quan trọng là cấu trúc phụ thuộc giữa các component. Hãy để ý rằng cấu trúc này là một đồ thị có hướng. Các component là các đỉnh, còn các quan hệ phụ thuộc là các cạnh có hướng.

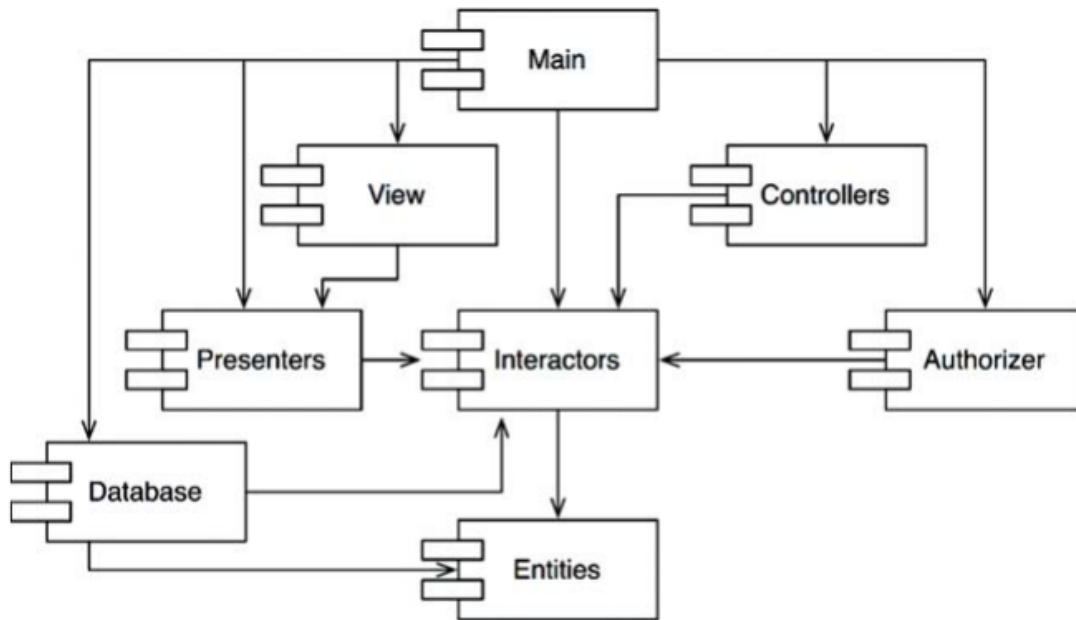


Figure 14.1 Typical component diagram

Figure 14.1 Sơ đồ component điển hình

Notice one more thing: Regardless of which component you begin at, it is impossible to follow the dependency relationships and wind up back at that component. This structure has no cycles. It is a directed acyclic graph (DAG).

Để ý thêm một điều nữa: dù bạn xuất phát từ component nào đi nữa, bạn cũng không thể đi theo các quan hệ phụ thuộc rồi quay về đúng component đó. Cấu trúc này không có chu trình. Nó là một đồ thị có hướng phi chu trình (directed acyclic graph — DAG).

Now consider what happens when the team responsible for Presenters makes a new release of their component. It is easy to find out who is affected

Giờ hãy xem điều gì xảy ra khi nhóm phụ trách Presenters phát hành một bản mới cho component của họ. Rất dễ tìm ra ai bị ảnh hưởng

by this release; you just follow the dependency arrows backward. Thus View and Main will both be affected. The developers currently working on those components will have to decide when they should integrate their work with the new release of Presenters .

bởi bản phát hành này; bạn chỉ việc lần ngược các mũi tên phụ thuộc. Như vậy, cả View lẫn Main đều bị ảnh hưởng. Các lập trình viên đang làm việc trên hai component đó sẽ phải

quyết định khi nào thì tích hợp công việc của họ với bản phát hành mới của Presenters.

Notice also that when Main is released, it has utterly no effect on any of the other components in the system. They don't know about Main, and they don't care when it changes. This is nice. It means that the impact of releasing Main is relatively small.

Cũng hãy để ý rằng khi Main được phát hành, nó chẳng gây ảnh hưởng gì tới bất kỳ component nào khác trong hệ thống. Chúng không biết gì về Main, và cũng chẳng bận tâm khi nó thay đổi. Thế thì tốt. Điều đó có nghĩa là tác động của việc phát hành Main tương đối nhỏ.

When the developers working on the Presenters component would like to run a test of that component, they just need to build their version of Presenters with the versions of the Interactors and Entities components that they are currently using. None of the other components in the system need be involved. This is nice. It means that the developers working on Presenters have relatively little work to do to set up a test, and that they have relatively few variables to consider.

Khi các lập trình viên làm việc trên component Presenters muốn chạy thử component đó, họ chỉ cần build phiên bản Presenters của mình cùng với các phiên bản Interactors và Entities mà họ đang dùng. Không component nào khác trong hệ thống phải tham gia vào. Thế thì tốt. Điều đó có nghĩa là các lập trình viên làm việc trên Presenters phải bỏ ra tương đối ít công sức để dựng một phép kiểm thử, và họ cũng có tương đối ít biến số phải cân nhắc.

When it is time to release the whole system, the process proceeds from the bottom up. First the Entities component is compiled, tested, and released. Then the same is done for Database and Interactors. These components are followed by Presenters, View, Controllers, and then Authorizer. Main goes last. This process is very clear and easy to deal with. We know how to build the system because we understand the dependencies between its parts.

Đến lúc phát hành cả hệ thống, quy trình đi từ dưới lên. Trước hết, component Entities được biên dịch, kiểm thử và phát hành. Rồi đến lượt Database và Interactors. Sau các component này là Presenters, View, Controllers, rồi tới Authorizer. Main đi sau cùng. Quy trình này rất rõ ràng và dễ xử lý. Chúng ta biết cách build hệ thống bởi vì chúng ta hiểu các phụ thuộc giữa các bộ phận của nó.

The Effect of a Cycle in the Component Dependency Graph — Hệ quả của một chu trình trong đồ thị phụ thuộc component

Suppose that a new requirement forces us to change one of the classes in Entities such that it makes use of a class in Authorizer. For example, let's say that the User class in Entities uses the Permissions class in Authorizer. This creates a **dependency cycle**, as shown in Figure 14.2.

Giả sử một yêu cầu mới buộc chúng ta phải sửa một trong các class thuộc Entities sao cho nó dùng tới một class trong Authorizer. Chẳng hạn, giả sử class User trong Entities dùng class Permissions trong Authorizer. Việc này tạo ra một chu trình phụ thuộc, như minh họa ở Figure 14.2.

This cycle creates some immediate problems. For example, the developers working on the Database component know that to release it, the component

Chu trình này lập tức gây ra vài rắc rối. Ví dụ, các lập trình viên làm việc trên component Database biết rằng để phát hành được nó, component ấy

must be compatible with Entities . However, with the cycle in place, the Database component must now also be compatible with Authorizer . But Authorizer depends on Interactors . This makes Database much more difficult to release. Entities , Authorizer , and Interactors have, in effect, become one large component—which means that all of the developers working on any of those components will experience the dreaded “morning after syndrome.” They will be stepping all over one another because they must all use exactly the same release of one another’s components.

phải tương thích với Entities. Thế nhưng, khi chu trình đã hình thành, component Database giờ còn phải tương thích cả với Authorizer. Mà Authorizer lại phụ thuộc vào Interactors. Điều này khiến Database khó phát hành hơn nhiều. Trên thực tế, Entities, Authorizer và Interactors đã trở thành một component to đùng — nghĩa là tất cả lập trình viên làm việc trên bất kỳ component nào trong số đó đều sẽ ném trái hội chứng “sáng hôm sau” đáng sợ. Họ sẽ giẫm chân lên nhau, bởi tất cả đều buộc phải dùng đúng cùng một bản phát hành của component của nhau.

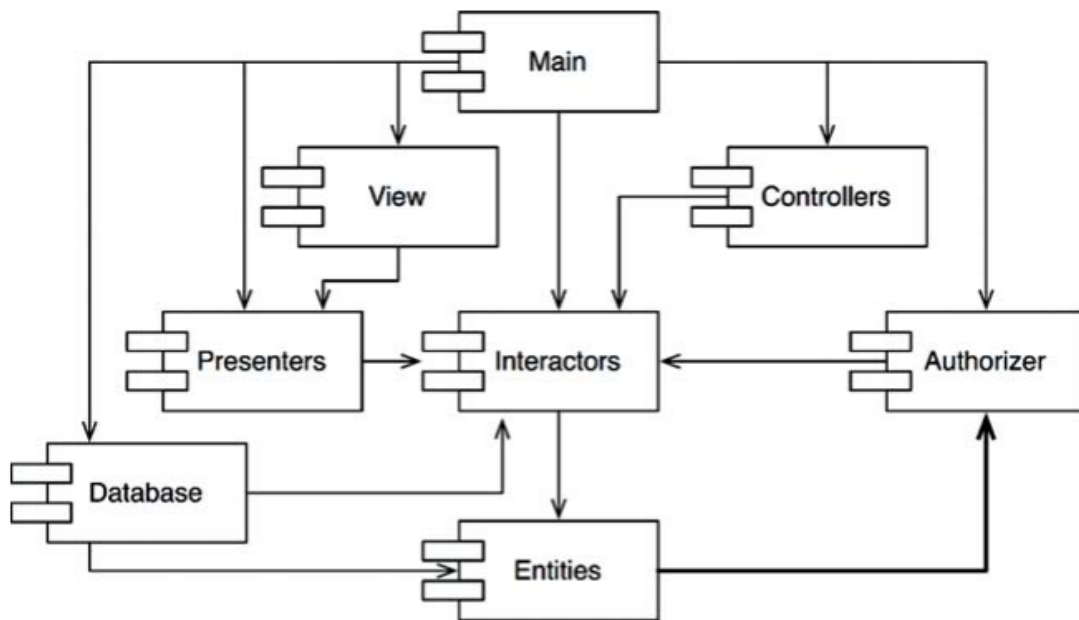


Figure 14.2 A dependency cycle

Figure 14.2 Một chu trình phụ thuộc

But this is just part of the trouble. Consider what happens when we want to test the Entities component. To our chagrin, we find that we must build and integrate with Authorizer and Interactors . This **level** of coupling between components is troubling, if not intolerable.

Nhưng đó mới chỉ là một phần của rắc rối. Hãy nghĩ xem chuyện gì xảy ra khi ta muốn kiểm thử component Entities. Ngán ngẩm thay, ta phát hiện mình phải build và tích hợp cùng với Authorizer và Interactors. Mức kết dính giữa các component như vậy là rất đáng lo, nếu không muốn nói là không thể chấp nhận nổi.

You may have wondered why you have to include so many different libraries, and so much of everybody else’s stuff, just to run a simple unit test of one of your classes. If you investigate the matter a bit, you will probably discover that there are cycles in the dependency graph. Such cycles make it very difficult to isolate components. Unit testing and releasing become very difficult and error prone. In addition, build issues grow geometrically with the number of modules.

Có lẽ bạn từng thắc mắc vì sao mình phải nhét vào bao nhiêu thư viện khác nhau, bao nhiêu thứ của thiên hạ, chỉ để chạy một unit test đơn giản cho một class của mình. Nếu đào sâu một chút, hẳn bạn sẽ phát hiện ra rằng đồ thị phụ thuộc có chu trình. Những chu trình như thế khiến việc cô lập các component trở nên cực khó. Unit test và phát hành trở nên rất khó khăn và dễ sai sót. Thêm nữa, các vấn đề khi build tăng theo cấp số nhân với số lượng module.

Moreover, when there are cycles in the dependency graph, it can be very difficult to work out the order in which you must build the components. Indeed, there probably is no correct order. This can lead to some very nasty problems in languages like Java that read their declarations from compiled binary files.

Hơn nữa, khi đồ thị phụ thuộc có chu trình, việc tìm ra thứ tự phải build các component có thể rất nhọc nhằn. Thật ra, có lẽ chẳng có thứ tự nào đúng cả. Điều này có thể dẫn tới những vấn đề rất khó chịu trong các ngôn ngữ như Java, vốn đọc phần khai báo từ các tệp nhị phân đã biên dịch.

Breaking the Cycle — Phá vỡ chu trình

It is always possible to break a cycle of components and reinstate the dependency graph as a DAG. There are two primary mechanisms for doing so:

Luôn luôn có thể phá vỡ một chu trình giữa các component và đưa đồ thị phụ thuộc trở lại dạng DAG. Có hai cơ chế chính để làm việc đó:

- 1. Apply the **Dependency Inversion** Principle (DIP). In the case in Figure 14.3,
 - 1. Áp dụng Dependency Inversion Principle (DIP). Trong trường hợp ở Figure 14.3,

we could create an **interface** that has the methods that User needs. We could then put that interface into Entities and inherit it into Authorizer . This inverts the dependency between Entities and Authorizer , thereby breaking the cycle.

chúng ta có thể tạo một interface chứa các phương thức mà User cần. Rồi đặt interface đó vào Entities và cho Authorizer kế thừa nó. Cách này đảo ngược phụ thuộc giữa Entities và Authorizer, nhờ đó phá vỡ chu trình.

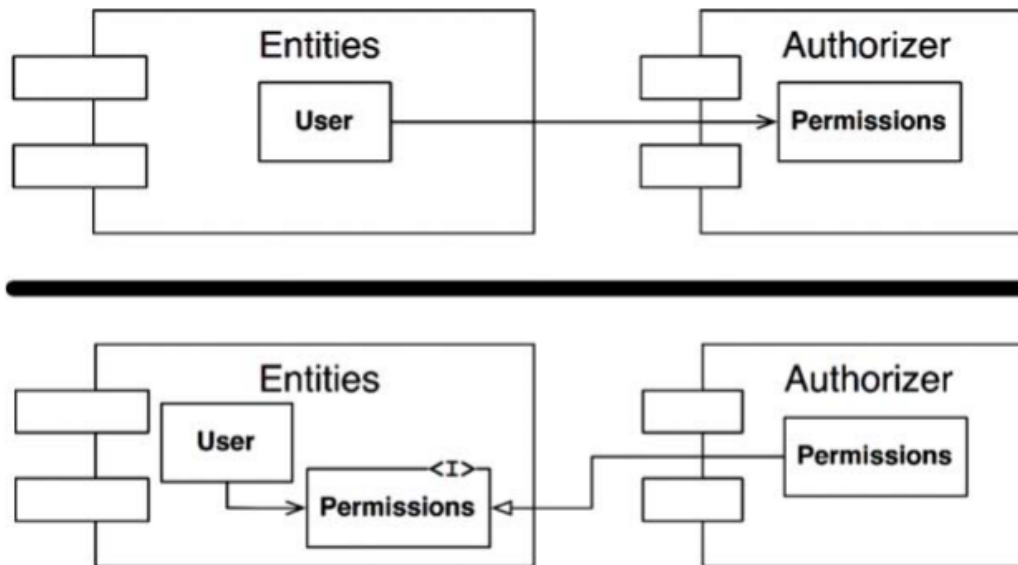


Figure 14.3 Inverting the dependency between Entities and Authorizer

Figure 14.3 Đảo ngược phụ thuộc giữa Entities và Authorizer

- 2. Create a new component that both Entities and Authorizer depend on.
 - 2. Tạo một component mới mà cả Entities lẫn Authorizer cùng phụ thuộc vào.

Move the class(es) that they both depend on into that new component (Figure 14.4).

Chuyển (các) class mà cả hai cùng phụ thuộc vào sang component mới đó (Figure 14.4).

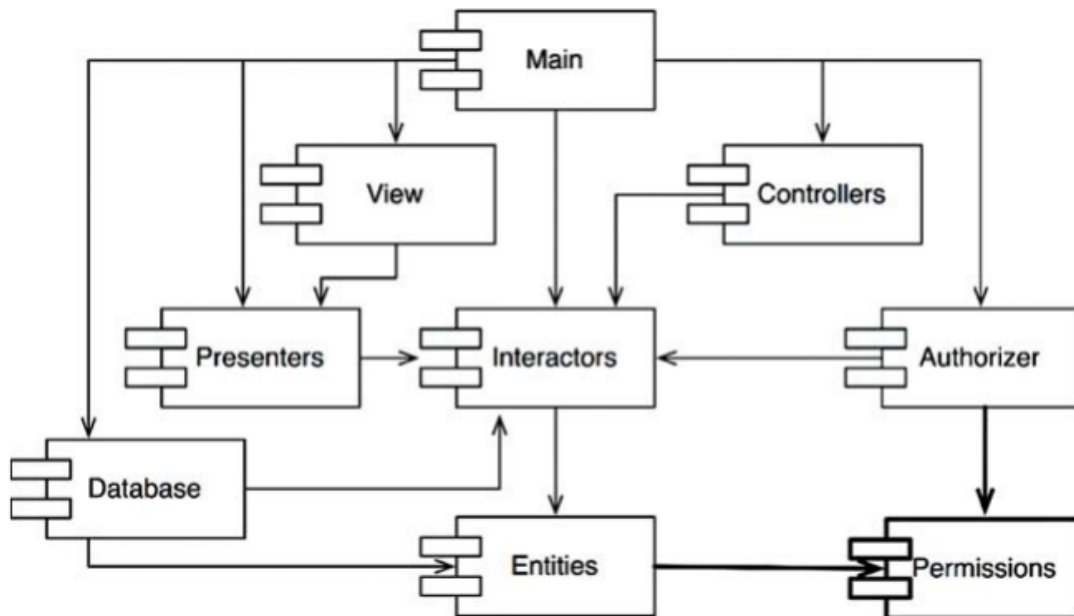


Figure 14.4 The new component that both Entities and Authorizer depend on

Figure 14.4 Component mới mà cả Entities và Authorizer cùng phụ thuộc vào

The “Jitters” — Hiện tượng “rung lắc”

The second solution implies that the component structure is volatile in the presence of changing requirements. Indeed, as the application grows, the component dependency structure jitters and grows. Thus the dependency structure must always be monitored for cycles. When cycles occur, they must be broken somehow. Sometimes this will mean creating new components, making the dependency structure grow.

Giải pháp thứ hai hàm ý rằng cấu trúc component sẽ dễ biến động mỗi khi yêu cầu thay đổi. Quả thật, khi ứng dụng lớn lên, cấu trúc phụ thuộc giữa các component vừa rung lắc vừa nở ra. Vì vậy, cấu trúc phụ thuộc phải luôn được theo dõi để phát hiện chu trình. Khi chu trình xuất hiện, bằng cách nào đó ta phải phá vỡ nó. Đôi lúc điều đó đồng nghĩa với việc tạo thêm component mới, khiến cấu trúc phụ thuộc lớn dần lên.

Top-Down Design — Thiết kế từ trên xuống

The issues we have discussed so far lead to an inescapable conclusion: The component structure cannot be designed from the top down. It is not one of the first things about the system that is designed, but rather evolves as the system grows and changes.

Những vấn đề chúng ta bàn tới này giờ dẫn tới một kết luận không thể né tránh: cấu trúc component không thể được thiết kế từ trên xuống. Nó không phải là một trong những thứ đầu tiên được thiết kế trong hệ thống, mà tiến hóa dần khi hệ thống lớn lên và thay đổi.

Some readers may find this point to be counterintuitive. We have come to expect that large-grained decompositions, like components, will also be high-level functional decompositions.

Có thể vài độc giả thấy điểm này trái với trực giác. Chúng ta vốn quen kỳ vọng rằng những phép phân rã ở mức thô, như component, cũng đồng thời là phép phân rã theo hàm ở mức cao.

When we see a large-grained grouping such as a component dependency structure, we believe that the components ought to somehow represent the functions of the system. Yet this does not seem to be an attribute of component dependency diagrams.

Khi nhìn thấy một cách gom nhóm thô như cấu trúc phụ thuộc giữa các component, ta tin rằng các component bằng cách nào đó phải thể hiện được các chức năng của hệ thống. Vậy mà đó lại không phải là thuộc tính của sơ đồ phụ thuộc component.

In fact, component dependency diagrams have very little to do with describing the function of the application. Instead, they are a map to the buildability and maintainability of the application. This is why they aren’t designed at the beginning of the project. There is no software to build or maintain, so there is no need for a build and maintenance map. But as more and more modules accumulate in the early stages of implementation and design, there is a growing need to manage the dependencies so that the project can be developed without the “morning after syndrome.” Moreover, we want to keep changes as localized as possible, so we start paying attention to the SRP and CCP and collocate classes that are likely to change together.

Thực tế, sơ đồ phụ thuộc component chẳng dính dáng bao nhiêu tới việc mô tả chức năng của ứng dụng. Thay vào đó, chúng là tấm bản đồ về khả năng build và khả năng bảo trì của ứng dụng. Đó là lý do chúng không được thiết kế ngay đầu dự án. Lúc ấy chưa có phần

mềm nào để build hay bảo trì, nên chẳng cần tấm bản đồ build và bảo trì làm gì. Nhưng khi ngày càng nhiều module tích tụ lại trong các giai đoạn đầu của cài đặt và thiết kế, nhu cầu quản lý các phụ thuộc cũng lớn dần, để dự án có thể phát triển mà không dính hội chứng “sáng hôm sau”. Hơn nữa, chúng ta muốn giữ cho thay đổi càng cục bộ càng tốt, nên bắt đầu để ý tới SRP và CCP và đặt cạnh nhau những class có khả năng cùng thay đổi.

One of the overriding concerns with this dependency structure is the isolation of volatility. We don't want components that change frequently and for capricious reasons to affect components that otherwise ought to be stable. For example, we don't want cosmetic changes to the GUI to have an impact on our business rules. We don't want the addition or modification of reports to have an impact on our highest-level policies. Consequently, the component dependency graph is created and molded by architects to protect stable high-value components from volatile components.

Một trong những mối bận tâm bao trùm đối với cấu trúc phụ thuộc này là việc cô lập những phần dễ biến động. Chúng ta không muốn những component thay đổi thường xuyên và vì những lý do đồng đánh lại tác động tới những component lẽ ra phải ổn định. Chẳng hạn, ta không muốn những thay đổi bề ngoài của GUI ảnh hưởng tới các quy tắc nghiệp vụ (business rule). Ta không muốn việc thêm hay sửa báo cáo lại ảnh hưởng tới các chính sách (policy) ở mức cao nhất. Bởi vậy, đồ thị phụ thuộc component được các kiến trúc sư tạo ra và nhào nặn để bảo vệ những component ổn định, giá trị cao khỏi những component dễ biến động.

As the application continues to grow, we start to become concerned about creating reusable elements. At this point, the CRP begins to influence the composition of the components. Finally, as cycles appear, the ADP is applied and the component dependency graph jitters and grows.

Khi ứng dụng tiếp tục lớn lên, chúng ta bắt đầu quan tâm tới việc tạo ra những phần tử tái dùng được. Đến lúc đó, CRP bắt đầu chi phối cách cấu thành các component. Cuối cùng, khi các chu trình xuất hiện, ADP được áp dụng, và đồ thị phụ thuộc component vừa rung lắc vừa lớn dần.

If we tried to design the component dependency structure before we designed any classes, we would likely fail rather badly. We would not know much about common closure, we would be unaware of any reusable elements, and we would almost certainly create components that produced dependency cycles. Thus the component dependency structure grows and evolves with the logical design of the system.

Nếu chúng ta cố thiết kế cấu trúc phụ thuộc component trước khi thiết kế bất kỳ class nào, hẳn ta sẽ thất bại thảm hại. Ta sẽ không biết mấy về chuyện đóng chung, ta sẽ chẳng hay biết gì về các phần tử tái dùng được, và gần như chắc chắn ta sẽ tạo ra những component sinh ra chu trình phụ thuộc. Vì thế, cấu trúc phụ thuộc component lớn lên và tiến hóa cùng với thiết kế logic của hệ thống.

The Stable Dependencies Principle — Nguyên tắc Phụ thuộc Ổn định (The Stable Dependencies Principle)

Depend in the direction of **stability**.

Hãy phụ thuộc theo hướng ổn định dần.

Designs cannot be completely static. Some volatility is necessary if the design is to be maintained. By conforming to the Common Closure Principle (CCP), we create components that are sensitive to

certain kinds of changes but immune to others. Some of these components are designed to be volatile. We expect them to change.

Thiết kế không thể hoàn toàn tĩnh tại. Muốn duy trì được thiết kế thì phải chấp nhận một mức biến động nào đó. Khi tuân theo Common Closure Principle (CCP), chúng ta tạo ra những component nhạy cảm với một số loại thay đổi nhưng miễn nhiễm với những loại khác. Một vài trong số các component đó được thiết kế để dễ biến động. Chúng ta trông đợi chúng sẽ thay đổi.

Any component that we expect to be volatile should not be depended on by a component that is difficult to change. Otherwise, the volatile component will also be difficult to change.

Bất kỳ component nào ta trông đợi sẽ dễ biến động đều không nên bị một component khó thay đổi phụ thuộc vào. Nếu không, chính component dễ biến động ấy cũng sẽ trở nên khó thay đổi.

It is the perversity of software that a module that you have designed to be easy to change can be made difficult to change by someone else who simply hangs a dependency on it. Not a line of source code in your module need change, yet your module will suddenly become more challenging to change. By conforming to the Stable Dependencies Principle (SDP), we ensure that modules that are intended to be easy to change are not depended on by modules that are harder to change.

Cái oái oăm của phần mềm là ở chỗ: một module bạn thiết kế cho dễ thay đổi lại có thể trở nên khó thay đổi chỉ vì người khác treo lên nó một phụ thuộc. Không một dòng mã nguồn nào trong module của bạn cần đổi, vậy mà module ấy đột nhiên trở nên khó sửa hơn hẳn. Bằng cách tuân theo Stable Dependencies Principle (SDP), chúng ta bảo đảm rằng những module vốn được định cho dễ thay đổi sẽ không bị những module khó thay đổi hơn phụ thuộc vào.

Stability — Độ ổn định

What is meant by “stability”? Stand a penny on its side. Is it stable in that position? You would likely say “no.” However, unless disturbed, it will remain in that position for a very long time. Thus stability has nothing directly to do with frequency of change. The penny is not changing, but it is difficult to think of it as stable.

“Ổn định” nghĩa là gì? Hãy dựng đứng một đồng xu trên cạnh của nó. Ở tư thế đó nó có ổn định không? Hẳn bạn sẽ nói “không”. Thế nhưng, nếu không bị ai đụng vào, nó sẽ đứng nguyên như thế rất lâu. Vậy nên độ ổn định (stability) chẳng liên quan trực tiếp gì tới tần suất thay đổi. Đồng xu không hề thay đổi, nhưng thật khó mà coi nó là ổn định.

Webster’s Dictionary says that something is stable if it is “not easily moved.” Stability is related to the amount of work required to make a change. On the one hand, the standing penny is not stable because it requires very little work to topple it. On the other hand, a table is very stable because it takes a considerable amount of **effort** to turn it over.

Từ điển Webster nói một thứ là ổn định nếu nó “không dễ bị dịch chuyển”. Độ ổn định liên quan tới lượng công sức cần bỏ ra để tạo ra một thay đổi. Một mặt, đồng xu dựng đứng không ổn định bởi chỉ cần rất ít công sức là làm nó đổ. Mặt khác, một chiếc bàn thì rất ổn định, bởi phải tốn khá nhiều công sức mới lật được nó.

How does this relate to software? Many factors may make a software component hard to change—for example, its size, complexity, and clarity, among

Chuyện này liên quan gì tới phần mềm? Nhiều yếu tố có thể khiến một component phần mềm khó thay đổi — chẳng hạn kích thước, độ phức tạp và độ rõ ràng của nó, cùng

other characteristics. We will ignore all those factors and focus on something different here. One sure way to make a software component difficult to change, is to make lots of other software components depend on it. A component with lots of incoming dependencies is very stable because it requires a great deal of work to reconcile any changes with all the dependent components.

nhiều đặc tính khác. Ở đây chúng ta sẽ bỏ qua tất cả những yếu tố đó và tập trung vào một thứ khác. Một cách chắc chắn để khiến một component phần mềm khó thay đổi là làm cho thật nhiều component phần mềm khác phụ thuộc vào nó. Một component có nhiều phụ thuộc đi vào thì rất ổn định, bởi phải tốn rất nhiều công sức mới dung hòa được mọi thay đổi với tất cả các component phụ thuộc vào nó.

The diagram in Figure 14.5 shows X , which is a stable component. Three components depend on X , so it has three good reasons not to change. We say that X is responsible to those three components. Conversely, X depends on nothing, so it has no external influence to make it change. We say it is independent .

Sơ đồ ở Figure 14.5 cho thấy X, một component ổn định. Có ba component phụ thuộc vào X, nên nó có ba lý do chính đáng để không thay đổi. Ta nói rằng X có trách nhiệm với ba component đó. Ngược lại, X không phụ thuộc vào thứ gì cả, nên chẳng có ảnh hưởng nào từ bên ngoài buộc nó phải đổi. Ta nói nó độc lập.

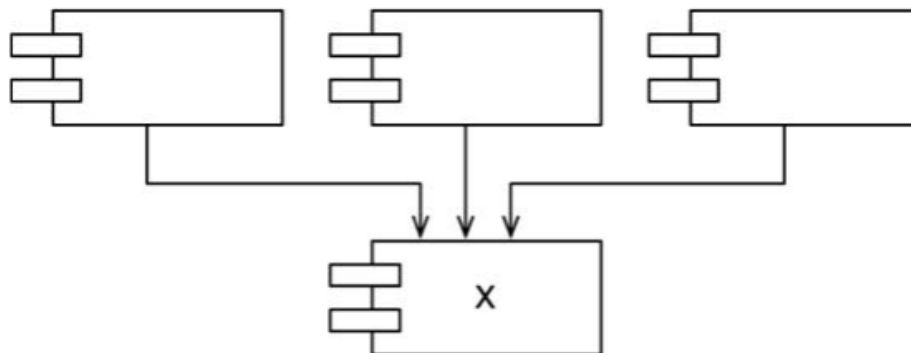


Figure 14.5 X : a stable component

Figure 14.5 X: một component ổn định

Figure 14.6 shows Y , which is a very **unstable** component. No other components depend on Y , so we say that it is irresponsible. Y also has three components that it depends on, so changes may come from three external sources. We say that Y is dependent.

Figure 14.6 cho thấy Y, một component rất bất ổn định. Không component nào khác phụ thuộc vào Y, nên ta nói nó vô trách nhiệm. Y lại phụ thuộc vào ba component, nên thay đổi có thể ập tới từ ba nguồn bên ngoài. Ta nói rằng Y là phụ thuộc.

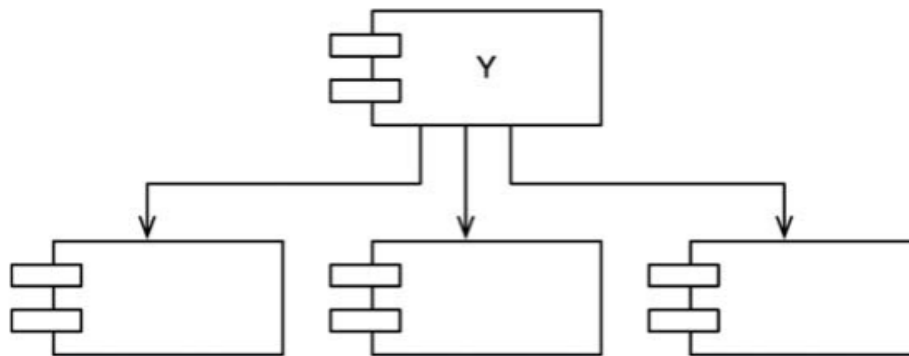


Figure 14.6 Y : a very unstable component

Figure 14.6 Y: một component rất bất ổn định

Stability Metrics — Các chỉ số đo độ ổn định

How can we measure the stability of a component? One way is to count the number of dependencies that enter and leave that component. These counts will allow us to calculate the positional stability of the component.

Làm sao đo được độ ổn định của một component? Một cách là đếm số phụ thuộc đi vào và đi ra khỏi component đó. Những con số này cho phép ta tính ra độ ổn định về mặt vị trí của component.

- **Fan-in** : Incoming dependencies. This metric identifies the number of classes

- Fan-in: các phụ thuộc đi vào. Chỉ số này cho biết số class

outside this component that depend on classes within the component.

nằm ngoài component nhưng phụ thuộc vào các class bên trong component.

- **Fan-out** : Outgoing dependencies. This metric identifies the number of classes

- Fan-out: các phụ thuộc đi ra. Chỉ số này cho biết số class

inside this component that depend on classes outside the component.

nằm bên trong component nhưng phụ thuộc vào các class ngoài component.

- **I : Instability**: $I = \text{Fan-out} / (\text{Fan-in} + \text{Fan-out})$. This metric has the range

- I: Độ bất ổn định: $I = \text{Fan-out} \div (\text{Fan-in} + \text{Fan-out})$. Chỉ số này có miền giá trị

$[0, 1]$. $I = 0$ indicates a maximally stable component. $I = 1$ indicates a maximally unstable component.

$[0, 1]$. $I = 0$ chỉ một component ổn định tối đa. $I = 1$ chỉ một component bất ổn định tối đa.

The Fan-in and Fan-out metrics are calculated by counting the number of classes outside the component in question that have dependencies with the classes inside the component in question. Consider the example in Figure 14.7.

Các chỉ số Fan-in và Fan-out được tính bằng cách đếm số class bên ngoài component đang xét mà có quan hệ phụ thuộc với các class bên trong component đó. Hãy xem ví dụ ở Figure 14.7.

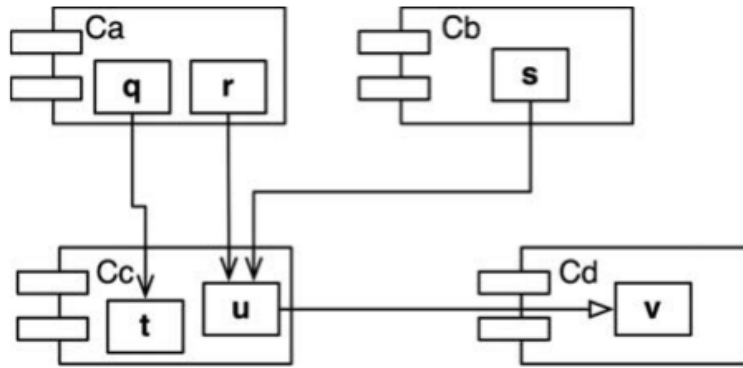


Figure 14.7 Our example

Figure 14.7 Ví dụ của chúng ta

Let's say we want to calculate the stability of the component Cc. We find that there are three classes outside Cc that depend on classes in Cc. Thus, Fan-in = 3. Moreover, there is one class outside Cc that classes in Cc depend on. Thus, Fan-out = 1 and $I = 1/4$.

Giả sử ta muốn tính độ ổn định của component Cc. Ta thấy có ba class bên ngoài Cc phụ thuộc vào các class trong Cc. Vậy Fan-in = 3. Ngoài ra, có một class bên ngoài Cc mà các class trong Cc phụ thuộc vào. Vậy Fan-out = 1 và $I = 1/4$.

- 1. In previous publications, I used the names Efferent and Afferent couplings (Ce and Ca) for Fan-out and
 - 1. Trong các ấn phẩm trước, tôi dùng tên Efferent và Afferent couplings (Ce và Ca) cho Fan-out và

Fan-in, respectively. That was just hubris on my part: I liked the metaphor of the central nervous system.

Fan-in, theo thứ tự đó. Đó chỉ là thói tự phụ của tôi thôi: tôi thích ẩn dụ về hệ thần kinh trung ương.

In C++, these dependencies are typically represented by `#include` statements. Indeed, the I metric is easiest to calculate when you have organized your source code such that there is one class in each source file. In Java, the I metric can be calculated by counting import statements and qualified names.

Trong C++, những phụ thuộc này thường được thể hiện bằng các câu lệnh `#include`. Quả thật, chỉ số I dễ tính nhất khi bạn đã tổ chức mã nguồn sao cho mỗi tệp nguồn chỉ chứa một class. Trong Java, chỉ số I có thể tính bằng cách đếm các câu lệnh import và các tên có định danh đầy đủ.

When the I metric is equal to 1, it means that no other component depends on this component (Fan-in = 0), and this component depends on other components (Fan-out > 0). This situation is as unstable as a component can get; it is irresponsible and dependent. Its lack of dependents gives the component no reason not to change, and the components that it depends on may give it ample reason to change.

Khi chỉ số I bằng 1, điều đó nghĩa là không component nào khác phụ thuộc vào component này (Fan-in = 0), còn component này thì phụ thuộc vào các component khác (Fan-out > 0). Đây là mức bất ổn định cao nhất mà một component có thể đạt tới; nó vừa vô trách nhiệm vừa phụ thuộc. Việc không có ai phụ thuộc vào nó khiến component chẳng có lý do gì để

không thay đổi, còn những component mà nó phụ thuộc vào thì lại có thể cho nó thừa lý do để phải thay đổi.

In contrast, when the I metric is equal to 0, it means that the component is depended on by other components (Fan-in $\neq 0$), but does not itself depend on any other components (Fan-out = 0). Such a component is responsible and independent . It is as stable as it can get. Its dependents make it hard to change the component, and its has no dependencies that might force it to change.

Ngược lại, khi chỉ số I bằng 0, điều đó nghĩa là component này bị các component khác phụ thuộc vào (Fan-in > 0), nhưng bản thân nó không phụ thuộc vào component nào cả (Fan-out = 0). Một component như vậy là có trách nhiệm và độc lập. Nó ổn định ở mức cao nhất có thể. Những kẻ phụ thuộc vào nó khiến việc thay đổi component trở nên khó khăn, còn bản thân nó không có phụ thuộc nào có thể buộc nó phải đổi.

The SDP says that the I metric of a component should be larger than the I metrics of the components that it depends on. That is, I metrics should decrease in the direction of dependency.

SDP nói rằng chỉ số I của một component phải lớn hơn chỉ số I của những component mà nó phụ thuộc vào. Nói cách khác, chỉ số I phải giảm dần theo chiều của phụ thuộc.

Not All Components Should Be Stable — Không phải component nào cũng nên ổn định

If all the components in a system were maximally stable, the system would be unchangeable. This is not a desirable situation. Indeed, we want to design our component structure so that some components are unstable and some are stable. The diagram in Figure 14.8 shows an ideal configuration for a system with three components.

Nếu mọi component trong hệ thống đều ổn định tối đa thì hệ thống sẽ không thể thay đổi được. Đó không phải tình cảnh đáng mong muốn. Thật ra, chúng ta muốn thiết kế cấu trúc component sao cho có component bất ổn định và có component ổn định. Sơ đồ ở Figure 14.8 cho thấy một cấu hình lý tưởng cho hệ thống gồm ba component.

The changeable components are on top and depend on the stable component at the bottom. Putting the unstable components at the top of the diagram is a useful convention because any arrow that points up is violating the SDP (and, as we shall see later, the ADP).

Những component hay thay đổi nằm ở trên và phụ thuộc vào component ổn định ở dưới. Đặt các component bất ổn định lên phía trên sơ đồ là một quy ước hữu ích, bởi bất kỳ mũi tên nào chỉ lên trên đều đang vi phạm SDP (và, như ta sẽ thấy sau, cả ADP nữa).

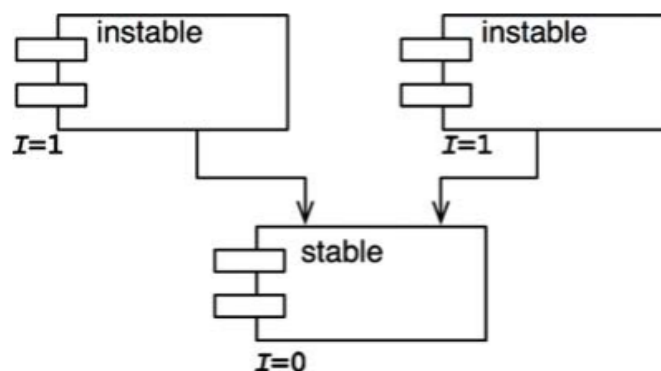


Figure 14.8 An ideal configuration for a system with three components

Figure 14.8 Một cấu hình lý tưởng cho hệ thống gồm ba component

The diagram in Figure 14.9 shows how the SDP can be violated.

Sơ đồ ở Figure 14.9 cho thấy SDP có thể bị vi phạm ra sao.

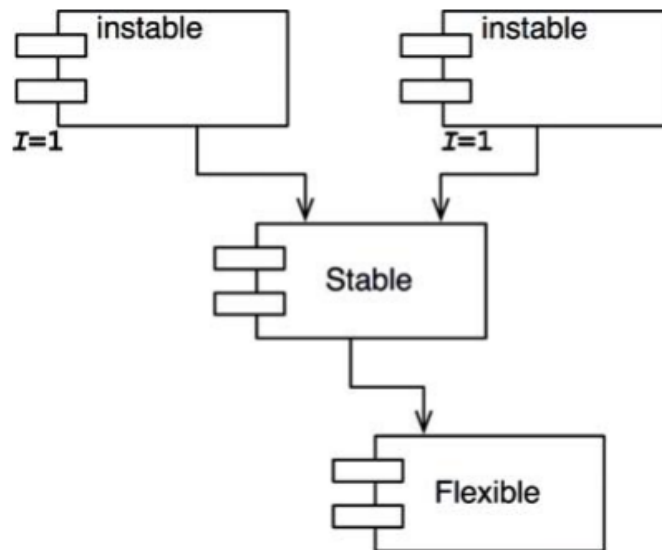


Figure 14.9 SDP violation

Figure 14.9 Vi phạm SDP

Flexible is a component that we have designed to be easy to change. We want Flexible to be unstable. However, some developer, working in the component named Stable, has hung a dependency on Flexible. This violates the SDP because the I metric for Stable is much smaller than the I metric for Flexible. As a result, Flexible will no longer be easy to change. A change to Flexible will force us to deal with Stable and all its dependents.

Flexible là một component mà chúng ta thiết kế cho dễ thay đổi. Chúng ta muốn Flexible bất ổn định. Thế nhưng, một lập trình viên nào đó, khi làm việc trong component tên là Stable, đã treo lên Flexible một phụ thuộc. Việc này vi phạm SDP, bởi chỉ số I của Stable nhỏ hơn nhiều so với chỉ số I của Flexible. Hậu quả là Flexible sẽ không còn dễ thay đổi nữa. Một thay đổi trong Flexible sẽ buộc chúng ta phải xử lý cả Stable lẫn mọi kẻ phụ thuộc vào Stable.

To fix this problem, we somehow have to break the dependence of Stable on Flexible. Why does this dependency exist? Let's assume that there is a

Để sửa vấn đề này, bằng cách nào đó ta phải cắt bỏ phụ thuộc của Stable vào Flexible. Vì sao phụ thuộc này tồn tại? Hãy giả sử rằng có một

class C within Flexible that another class U within Stable needs to use (Figure 14.10).

class C nằm trong Flexible mà một class U khác nằm trong Stable cần dùng tới (Figure 14.10).

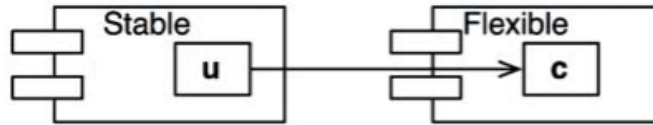


Figure 14.10 U within Stable uses C within Flexible

Figure 14.10 U trong Stable dùng C trong Flexible

We can fix this by employing the DIP. We create an interface class called US and put it in a component named UServer . We make sure that this interface declares all the methods that U needs to use. We then make C implement this interface as shown in Figure 14.11. This breaks the dependency of Stable on Flexible , and forces both components to depend on UServer . UServer is very stable ($I = 0$), and Flexible retains its necessary instability ($I = 1$). All the dependencies now flow in the direction of decreasing I .

Chúng ta có thể sửa việc này bằng cách vận dụng DIP. Ta tạo một interface class tên là US và đặt nó vào một component tên là UServer. Ta bảo đảm rằng interface này khai báo mọi phương thức mà U cần dùng. Rồi ta cho C hiện thực interface đó, như minh họa ở Figure 14.11. Cách này cắt bỏ phụ thuộc của Stable vào Flexible, và buộc cả hai component cùng phụ thuộc vào UServer. UServer rất ổn định ($I = 0$), còn Flexible vẫn giữ được mức bất ổn định cần thiết của nó ($I = 1$). Giờ đây mọi phụ thuộc đều chảy theo chiều I giảm dần.

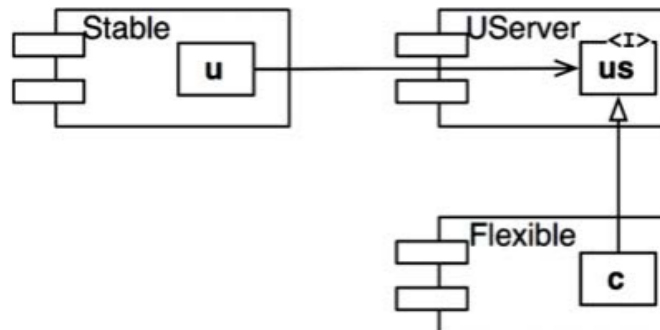


Figure 14.11 C implements the interface class US

Figure 14.11 C hiện thực interface class US

AbstractComponents — Các component trừu tượng

You may find it strange that we would create a component—in this example, `UService`—that contains nothing but an interface. Such a component contains no executable code! It turns out, however, that this is a very common, and necessary, tactic when using statically typed languages like Java and C#. These abstract components are very stable and, therefore, are ideal targets for less stable components to depend on.

Bạn có thể thấy lạ khi chúng ta tạo ra một component — trong ví dụ này là `UService` — mà bên trong không chứa gì ngoài một interface. Một component như thế không có lấy một dòng mã thực thi nào! Nhưng hoá ra đây lại là một chiến thuật rất phổ biến, và cần thiết, khi làm việc với các ngôn ngữ định kiểu tĩnh như Java và C#. Những component trừu tượng này rất ổn định, và vì vậy chúng là đích ngắm lý tưởng để các component kém ổn định hơn phụ thuộc vào.

When using dynamically typed languages like Ruby and Python, these abstract components don't exist at all, nor do the dependencies that would

Khi dùng các ngôn ngữ định kiểu động như Ruby và Python, những component trừu tượng này hoàn toàn không tồn tại, và cũng chẳng có những phụ thuộc lẽ ra sẽ

have targeted them. Dependency structures in these languages are much simpler because dependency inversion does not require either the declaration or the **inheritance** of interfaces.

trở tới chúng. Cấu trúc phụ thuộc trong các ngôn ngữ này đơn giản hơn nhiều, bởi đảo ngược phụ thuộc (dependency inversion) không đòi hỏi phải khai báo hay kế thừa interface.

The Stable Abstractions Principle — Nguyên tắc Trừu tượng Ổn định (The Stable Abstractions Principle)

A component should be as abstract as it is stable.

Một component nên trừu tượng đúng bằng mức độ ổn định của nó.

Where Do We Put the High-Level Policy? — Đặt chính sách mức cao ở đâu?

Some software in the system should not change very often. This software represents **high-level** architecture and **policy** decisions. We don't want these business and architectural decisions to be volatile. Thus the software that encapsulates the high-level policies of the system should be placed

into stable components ($I = 0$). Unstable components ($I = 1$) should contain only the software that is volatile—software that we want to be able to quickly and easily change.

Có những phần phần mềm trong hệ thống lẽ ra không nên thay đổi thường xuyên. Chúng đại diện cho các quyết định về kiến trúc (architecture) và chính sách (policy) ở mức cao. Ta không muốn những quyết định nghiệp vụ và kiến trúc đó dễ biến động (volatile). Vì vậy, phần phần mềm đóng gói các chính sách mức cao của hệ thống nên được đặt vào các component ổn định ($I = 0$). Các component bất ổn định ($I = 1$) chỉ nên chứa những phần dễ biến động — phần mà ta muốn có thể thay đổi nhanh và dễ.

However, if the high-level policies are placed into stable components, then the source code that represents those policies will be difficult to change. This could make the overall architecture inflexible. How can a component that is maximally stable ($I = 0$) be flexible enough to withstand change? The answer is found in the OCP. This principle tells us that it is possible and desirable to create classes that are flexible enough to be extended without requiring modification. Which kind of classes conform to this principle? Abstract classes.

Nhưng nếu đặt chính sách mức cao vào các component ổn định, thì chính mã nguồn thể hiện các chính sách đó lại trở nên khó thay đổi. Điều này có thể khiến toàn bộ kiến trúc trở nên cứng nhắc. Làm sao một component ổn định tối đa ($I = 0$) lại có thể đủ mềm dẻo để chịu được thay đổi? Câu trả lời nằm ở OCP. Nguyên tắc này bảo ta rằng hoàn toàn có thể — và nên — tạo ra những class đủ mềm dẻo để mở rộng mà không cần sửa đổi. Loại class nào tuân thủ được nguyên tắc đó? Các abstract class.

Introducing the Stable Abstractions Principle — Giới thiệu Nguyên tắc Trừu tượng Ổn định

The Stable Abstractions Principle (SAP) sets up a relationship between stability and **abstractness**. On the one hand, it says that a stable component should also be abstract so that its stability does not prevent it from being extended. On the other hand, it says that an unstable component should be concrete since its instability allows the concrete code within it to be easily changed.

Nguyên tắc Trừu tượng Ổn định (SAP) thiết lập một mối quan hệ giữa độ ổn định (stability) và độ trừu tượng (abstractness). Một mặt, nó nói rằng một component ổn định thì cũng nên trừu tượng, để chính sự ổn định ấy không ngăn cản nó được mở rộng. Mặt khác, nó nói rằng một component bất ổn định thì nên cụ thể, vì chính sự bất ổn định cho phép mã cụ thể bên trong nó được thay đổi dễ dàng.

Thus, if a component is to be stable, it should consist of interfaces and abstract classes so that it can be extended. Stable components that are extensible are flexible and do not overly constrain the architecture.

Vậy nên, nếu một component muốn ổn định, nó phải gồm các interface và abstract class để có thể mở rộng được. Những component ổn định mà mở rộng được thì mềm dẻo và không trói buộc kiến trúc quá mức.

The SAP and the SDP combined amount to the DIP for components. This is true because the SDP says that dependencies should run in the direction of stability, and the SAP says that stability implies abstraction. Thus dependencies run in the direction of abstraction.

SAP và SDP gộp lại chính là DIP ở cấp component. Đúng vậy: SDP nói rằng phụ thuộc phải chạy theo hướng ổn định, còn SAP nói rằng ổn định kéo theo trừu tượng. Vậy nên phụ thuộc chạy theo hướng trừu tượng.

The DIP, however, is a principle that deals with classes—and with classes there are no shades of gray. Either a class is abstract or it is not. The combination of the SDP and the SAP deals with components, and allows that a component can be partially abstract and partially stable.

Tuy nhiên, DIP là nguyên tắc bàn về class — mà với class thì không có vùng xám nào cả. Một class hoặc là abstract, hoặc không. Còn SDP kết hợp với SAP thì bàn về component, và cho phép một component vừa trừu tượng một phần vừa ổn định một phần.

Measuring Abstraction — Đo độ trừu tượng

The A metric is a measure of the abstractness of a component. Its value is simply the ratio of interfaces and abstract classes in a component to the total number of classes in the component.

Chỉ số A là thước đo độ trừu tượng của một component. Giá trị của nó đơn giản là tỉ lệ giữa số interface và abstract class trong component với tổng số class của component đó.

- Nc: The number of classes in the component.
 - Nc: Số class trong component.
- Na: The number of abstract classes and interfaces in the component.
 - Na: Số abstract class và interface trong component.
- A: Abstractness. $A = Na / Nc$.
 - A: Độ trừu tượng. $A = Na \div Nc$.

The A metric ranges from 0 to 1. A value of 0 implies that the component has no abstract classes at all. A value of 1 implies that the component contains nothing but abstract classes.

Chỉ số A chạy từ 0 đến 1. Giá trị 0 nghĩa là component không có lấy một abstract class nào. Giá trị 1 nghĩa là component không chứa gì ngoài abstract class.

The Main Sequence — Đường chính (The Main Sequence)

We are now in a position to define the relationship between stability (I) and abstractness (A). To do so, we create a graph with A on the vertical axis and I on the horizontal axis (Figure 14.12). If we plot the two “good” kinds of components on this graph, we will find the components that are maximally stable and abstract at the upper left at (0, 1). The components that are maximally unstable and concrete are at the lower right at (1, 0).

Giờ ta đã đủ cơ sở để định nghĩa mối quan hệ giữa độ ổn định (I) và độ trừu tượng (A). Để làm điều đó, ta dựng một đồ thị với A trên trục tung và I trên trục hoành (Figure 14.12). Nếu vẽ hai loại component “tốt” lên đồ thị này, ta sẽ thấy những component ổn định và trừu tượng tối đa nằm ở góc trên bên trái, tại (0, 1). Còn những component bất ổn định và cụ thể tối đa nằm ở góc dưới bên phải, tại (1, 0).

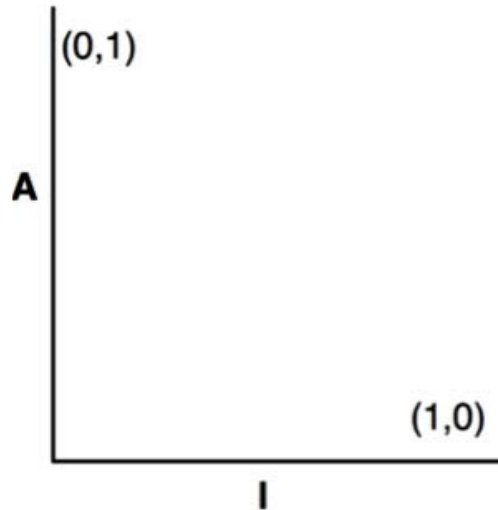


Figure 14.12 The I/A graph

Figure 14.12 Đồ thị I/A

Not all components fall into one of these two positions, because components often have degrees of abstraction and stability. For example, it is very common for one **abstract class** to derive from another abstract class. The derivative is an abstraction that has a dependency. Thus, though it is maximally abstract, it will not be maximally stable. Its dependency will decrease its stability.

Không phải component nào cũng rơi đúng vào một trong hai vị trí đó, bởi component thường có những mức độ trừu tượng và ổn định khác nhau. Chẳng hạn, chuyện một abstract class dẫn xuất từ một abstract class khác là hết sức bình thường. Lớp dẫn xuất là một trừu tượng nhưng lại có phụ thuộc. Vậy nên dù nó trừu tượng tối đa, nó sẽ không ổn định tối đa. Phụ thuộc của nó sẽ kéo độ ổn định xuống.

Since we cannot enforce a rule that all components sit at either (0, 1) or (1, 0), we must assume that there is a locus of points on the A / I graph that defines reasonable positions for components. We can infer what that locus is by finding the areas where components should not be—in other words, by determining the zones of exclusion (Figure 11.13).

Vì không thể áp đặt quy tắc rằng mọi component đều phải nằm ở (0, 1) hoặc (1, 0), ta phải giả định rằng có một quỹ tích các điểm trên đồ thị A/I xác định những vị trí hợp lý cho component. Ta có thể suy ra quỹ tích ấy bằng cách tìm ra những vùng mà component *không nên* nằm vào — nói cách khác, bằng cách xác định các vùng loại trừ (Figure 11.13).

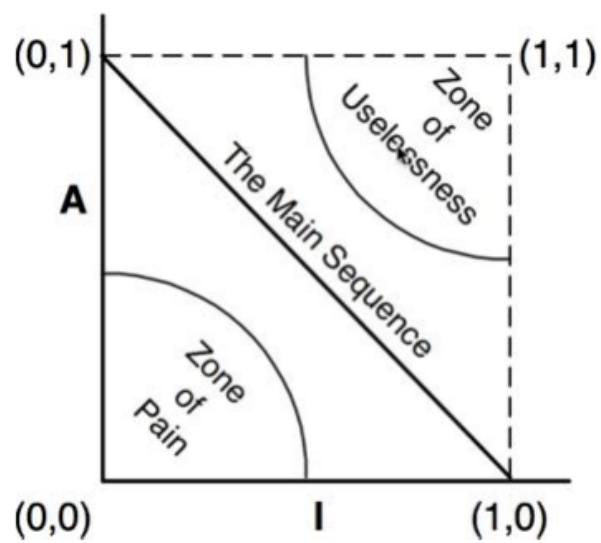


Figure 14.13 Zones of exclusion

Figure 14.13 Các vùng loại trừ

The Zone of Pain — Vùng đau đớn

Consider a component in the area of (0, 0). This is a highly stable and concrete component. Such a component is not desirable because it is rigid. It cannot be extended because it is not abstract, and it is very difficult to change because of its stability. Thus we do not normally expect to see well-designed components sitting near (0, 0). The area around (0, 0) is a zone of exclusion called the Zone of Pain .

Hãy xét một component nằm quanh vùng (0, 0). Đây là một component vừa rất ổn định vừa rất cụ thể. Một component như thế là điều không mong muốn, vì nó cứng đờ. Nó không mở rộng được vì không trừu tượng, và nó cũng rất khó thay đổi vì quá ổn định. Vậy nên bình thường ta không kỳ vọng thấy những component được thiết kế tốt nằm gần (0, 0). Khu vực quanh (0, 0) là một vùng loại trừ, gọi là *vùng đau đớn (Zone of Pain)*.

Some software entities do, in fact, fall within the **Zone of Pain**. An example would be a database schema. Database schemas are notoriously volatile, extremely concrete, and highly depended on. This is one reason why the interface between OO applications and databases is so difficult to manage, and why schema updates are generally painful.

Trên thực tế, có những thực thể phần mềm quả thật rơi vào vùng đau đớn. Một ví dụ là lược đồ cơ sở dữ liệu. Lược đồ cơ sở dữ liệu nổi tiếng là dễ biến động, cực kỳ cụ thể, lại bị rất nhiều thứ phụ thuộc vào. Đó là một lý do khiến phần giao tiếp giữa ứng dụng OO và cơ sở dữ liệu khó quản lý đến vậy, và khiến việc cập nhật lược đồ nói chung là một cực hình.

Another example of software that sits near the area of (0, 0) is a concrete utility library. Although such a library has an I metric of 1, it may actually be nonvolatile. Consider the String component, for example. Even though all the classes within it are concrete, it is so commonly used that changing it would create chaos. Therefore String is nonvolatile.

Một ví dụ khác về phần mềm nằm gần vùng (0, 0) là một thư viện tiện ích cụ thể. Dù thư viện kiểu này có chỉ số I bằng 1, trên thực tế nó có thể chẳng dễ biến động chút nào. Hãy lấy component String làm ví dụ. Dù mọi class bên trong nó đều là class cụ thể, nó được dùng phổ biến đến mức thay đổi nó sẽ gây ra hỗn loạn. Vì vậy String không dễ biến động.

Nonvolatile components are harmless in the (0, 0) zone since they are not likely to be changed. For that reason, it is only volatile software components that are problematic in the Zone of Pain. The more volatile a component in the Zone of Pain, the more “painful” it is. Indeed, we might consider volatility to be a third axis of the graph. With this understanding, Figure 14.13 shows only the most painful plane, where volatility = 1.

Những component không dễ biến động thì vô hại khi nằm trong vùng (0, 0), bởi chúng gần như không bị thay đổi. Vì lẽ đó, chỉ những component phần mềm dễ biến động mới thành vấn đề trong vùng đau đớn. Một component trong vùng đau đớn càng dễ biến động thì càng “đau”. Thật vậy, ta hoàn toàn có thể xem tính dễ biến động là trục thứ ba của đồ thị. Hiểu như vậy thì Figure 14.13 chỉ đang thể hiện mặt phẳng đau đớn nhất, nơi volatility =

1.

The Zone of Uselessness — Vùng vô dụng

Consider a component near (1, 1). This location is undesirable because it is maximally abstract, yet has no dependents. Such components are useless. Thus this area is called the Zone of Uselessness .

Hãy xét một component nằm gần (1, 1). Vị trí này cũng không mong muốn, vì component đó trừu tượng tối đa nhưng lại chẳng có ai phụ thuộc vào nó. Những component như vậy là vô dụng. Vì thế khu vực này được gọi là vùng vô dụng (*Zone of Uselessness*).

The software entities that inhabit this region are a kind of detritus. They are often leftover abstract classes that no one ever implemented. We find them in systems from time to time, sitting in the code base, unused.

Các thực thể phần mềm cư ngụ ở vùng này là một dạng rác thải. Chúng thường là những abstract class còn sót lại mà chẳng ai từng hiện thực hoá. Thỉnh thoảng ta lại bắt gặp chúng trong hệ thống, nằm im trong code base, không ai dùng đến.

A component that has a position deep within the **Zone of Uselessness** must contain a significant fraction of such entities. Clearly, the presence of such useless entities is undesirable.

Một component nằm sâu trong vùng vô dụng chắc chắn chứa một tỉ lệ đáng kể những thực thể như thế. Rõ ràng, sự hiện diện của những thực thể vô dụng ấy là điều không mong muốn.

Avoiding the Zones of Exclusion — Tránh các vùng loại trừ

It seems clear that our most volatile components should be kept as far from both zones of exclusion as possible. The locus of points that are maximally distant from each zone is the line that connects (1, 0) and (0, 1). I call this line **the Main Sequence** . 2

Có vẻ khá rõ rằng những component dễ biến động nhất của ta nên được giữ càng xa cả hai vùng loại trừ càng tốt. Quỹ tích các điểm cách xa mỗi vùng nhiều nhất chính là đường thẳng nối (1, 0) với (0, 1). Tôi gọi đường này là đường chính — *the Main Sequence* . 2

A component that sits on the Main Sequence is not “too abstract” for its stability, nor is it “too unstable” for its abstractness. It is neither useless nor particularly painful. It is depended on to the extent that it is abstract, and it depends on others to the extent that it is concrete.

Một component nằm trên đường chính thì không “quá trừu tượng” so với độ ổn định của nó, cũng chẳng “quá bất ổn định” so với độ trừu tượng của nó. Nó không vô dụng mà cũng chẳng đau đớn gì đặc biệt. Nó trừu tượng tới đâu thì kẻ khác phụ thuộc vào nó tới đó; nó cụ thể tới đâu thì nó phụ thuộc vào kẻ khác tới đó.

The most desirable position for a component is at one of the two endpoints of the Main Sequence. Good architects strive to position the majority of their components at those endpoints. However, in my experience, some small fraction of the components in a large system are neither perfectly abstract nor perfectly stable. Those components have the best characteristics if they are on, or close, to the Main Sequence.

Vị trí đáng mong muốn nhất cho một component là một trong hai đầu mút của đường chính. Kiến trúc sư giỏi luôn cố đưa phần lớn component của mình về hai đầu mút đó. Tuy nhiên, theo kinh nghiệm của tôi, trong một hệ thống lớn luôn có một phần nhỏ component chẳng hoàn toàn trừu tượng mà cũng chẳng hoàn toàn ổn định. Những component ấy sẽ có đặc tính tốt nhất khi chúng nằm trên, hoặc gần, đường chính.

Distance from the Main Sequence — Khoảng cách tới đường chính

This leads us to our last metric. If it is desirable for components to be on, or close, to the Main Sequence, then we can create a metric that measures how far away a component is from this ideal.

Điều này dẫn ta tới chỉ số cuối cùng. Nếu component nên nằm trên hoặc gần đường chính, thì ta có thể tạo ra một chỉ số đo xem một component cách lý tưởng đó bao xa.

- D 3 : Distance. $D = |A + I - 1|$. The range of this metric is [0, 1]. A value of 0

- D 3 : Khoảng cách. $D = |A + I - 1|$. Miền giá trị của chỉ số này là [0, 1]. Giá trị 0

indicates that the component is directly on the Main Sequence. A value of 1 indicates that the component is as far away as possible from the Main Sequence.

cho biết component nằm ngay trên đường chính. Giá trị 1 cho biết component cách đường chính xa nhất có thể.

- 2. The author begs the reader's indulgence for the arrogance of borrowing such an important term from
 - 2. Tác giả xin độc giả lượng thứ cho sự ngạo mạn khi mượn một thuật ngữ quan trọng đến vậy từ

astronomy.

thiên văn học.

- 3. In previous publications, I called this metric D_3 . I see no reason to continue that practice.
 - 3. Trong các ấn phẩm trước, tôi gọi chỉ số này là D_3 . Tôi thấy không còn lý do gì để tiếp tục cách gọi đó.

Given this metric, a design can be analyzed for its overall conformance to the Main Sequence. The D metric for each component can be calculated. Any component that has a D value that is not near zero can be reexamined and restructured.

Có chỉ số này rồi, ta có thể phân tích một thiết kế xem nó tuân theo đường chính đến mức nào. Chỉ số D của từng component đều tính được. Bất kỳ component nào có giá trị D không gần 0 đều đáng được xem xét lại và tái cấu trúc.

Statistical analysis of a design is also possible. We can calculate the mean and variance of all the D metrics for the components within a design. We would expect a conforming design to have a mean and variance that are close to zero. The variance can be used to establish “control limits” so as to identify components that are “exceptional” in comparison to all the others.

Ta cũng có thể phân tích thống kê một thiết kế. Ta tính được giá trị trung bình và phương sai của toàn bộ các chỉ số D của những component trong thiết kế. Với một thiết kế tuân thủ tốt, ta kỳ vọng cả trung bình lẫn phương sai đều gần 0. Phương sai có thể dùng để thiết lập các “giới hạn kiểm soát”, nhờ đó nhận diện những component “bất thường” so với phần còn lại.

In the scatterplot in Figure 14.14, we see that the bulk of the components lie along the Main Sequence, but some of them are more than one standard deviation ($Z = 1$) away from the mean. These aberrant components are worth examining more closely. For some reason, they are either very abstract with few dependents or very concrete with many dependents.

Trong biểu đồ phân tán ở Figure 14.14, ta thấy phần lớn component nằm dọc theo đường chính, nhưng một số cách trung bình hơn một độ lệch chuẩn ($Z = 1$). Những component lệch chuẩn đó đáng được soi kỹ hơn. Vì lý do nào đó, chúng hoặc rất trừu tượng mà ít kẻ phụ thuộc vào, hoặc rất cụ thể mà lại có nhiều kẻ phụ thuộc.

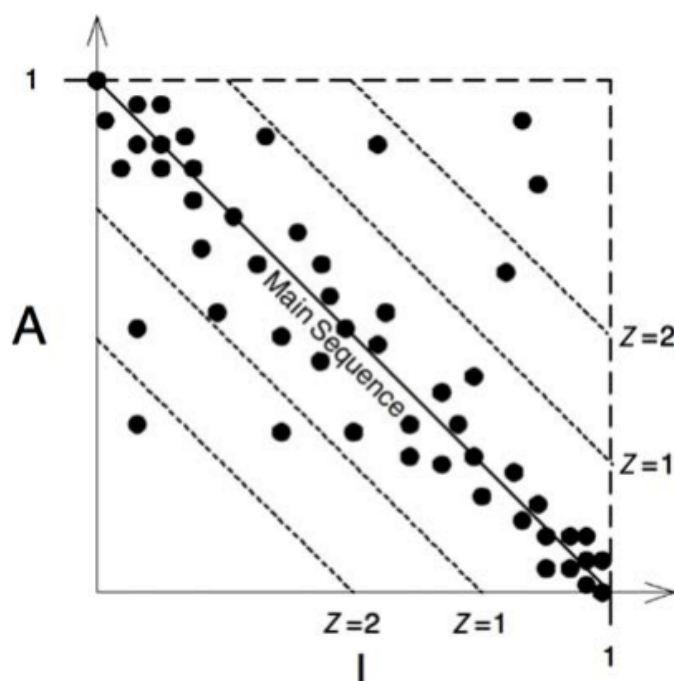


Figure 14.14 Scatterplot of the components

Figure 14.14 Biểu đồ phân tán của các component

Another way to use the metrics is to plot the D metric of each component over time. The graph in Figure 14.15 is a mock-up of such a plot. You can see

Một cách dùng khác của các chỉ số này là vẽ chỉ số D của từng component theo thời gian. Đồ thị ở Figure 14.15 là một bản dựng mẫu của kiểu vẽ đó. Bạn có thể thấy

that some strange dependencies have been creeping into the Payroll component over the last few releases. The plot shows a control threshold at $D = 0.1$. The R2.1 point has exceeded this control limit, so it would be worth our while to find out why this component is so far from the main sequence.

rằng vài phụ thuộc kỳ quặc đã len lỏi vào component Payroll qua mấy lần phát hành gần đây. Đồ thị cho thấy một ngưỡng kiểm soát tại $D = 0.1$. Điểm R2.1 đã vượt qua giới hạn

kiểm soát này, nên sẽ rất đáng công để tìm hiểu vì sao component này lại xa đường chính đến thế.

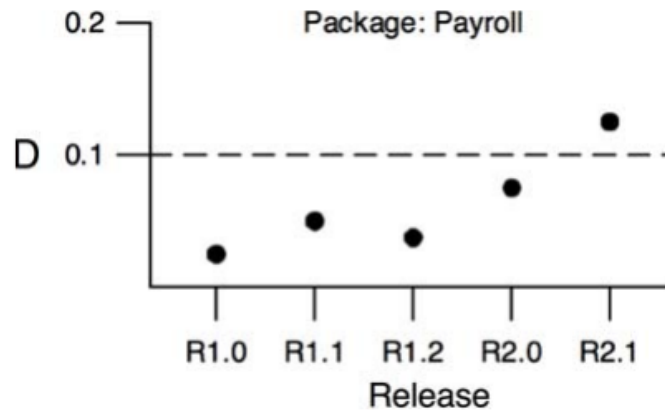


Figure 14.15 Plot of D for a single component over time

Figure 14.15 Đồ thị D của một component theo thời gian

Conclusion — Kết luận

The dependency management metrics described in this chapter measure the conformance of a design to a pattern of dependency and abstraction that I think is a “good” pattern. Experience has shown that certain dependencies are good and others are bad. This pattern reflects that experience. However, a metric is not a god; it is merely a measurement against an arbitrary standard. These metrics are imperfect, at best, but it is my hope that you find them useful.

Các chỉ số quản lý phụ thuộc mô tả trong chương này đo mức độ tuân thủ của một thiết kế đối với một khuôn mẫu phụ thuộc và trừu tượng mà tôi cho là khuôn mẫu “tốt”. Kinh nghiệm cho thấy có những phụ thuộc là tốt và có những phụ thuộc là xấu. Khuôn mẫu này phản ánh chính kinh nghiệm đó. Tuy nhiên, một chỉ số không phải là thần thánh; nó chỉ là một phép đo dựa trên một chuẩn mực do ta tự đặt ra. Các chỉ số này, giỏi lắm cũng chỉ là chưa hoàn hảo, nhưng tôi hy vọng bạn thấy chúng hữu ích.

Architecture — Kiến trúc

What Is Architecture? — Kiến trúc là gì?

The word “**architecture**” conjures visions of power and mystery. It makes us think of weighty decisions and deep technical prowess. Software architecture is at the pinnacle of technical achievement. When we think of a software architect, we think of someone who has power, and who commands respect. What young aspiring software developer has not dreamed of one day becoming a software architect?

Từ “kiến trúc” gợi lên hình ảnh của quyền lực và bí ẩn. Nó khiến ta nghĩ tới những quyết định nặng ký và trình độ kỹ thuật thâm hậu. Kiến trúc phần mềm nằm ở đỉnh cao của thành tựu kỹ thuật. Khi nghĩ về một kiến trúc sư phần mềm, ta nghĩ tới một người có quyền lực và được nể trọng. Có lập trình viên trẻ đầy khát vọng nào lại chưa từng mơ một ngày mình trở thành kiến trúc sư phần mềm?

But what is software architecture? What does a software architect do, and when does he or she do it?

Nhưng kiến trúc phần mềm là gì? Kiến trúc sư phần mềm làm gì, và làm vào lúc nào?

First of all, a software architect is a programmer; and continues to be a programmer. Never fall for the lie that suggests that software architects pull back from code to focus on higher-**level** issues. They do not! Software architects are the best programmers, and they continue to take programming tasks, while they also guide the rest of the team toward a **design** that maximizes productivity. Software architects may not write as much code as other programmers do, but they continue to engage in programming tasks. They do this because they cannot do their jobs properly if they are not experiencing the problems that they are creating for the rest of the programmers.

Trước hết, kiến trúc sư phần mềm là một lập trình viên; và vẫn tiếp tục là lập trình viên. Đừng bao giờ tin vào lời dối trá rằng kiến trúc sư phần mềm rút khỏi mã nguồn để tập trung vào những vấn đề ở mức cao hơn. Không hề! Kiến trúc sư phần mềm là những lập trình viên giỏi nhất, và họ vẫn tiếp tục nhận các nhiệm vụ lập trình, đồng thời dẫn dắt phần còn lại của đội hướng tới một thiết kế (design) giúp tối đa hoá năng suất. Kiến trúc sư phần mềm có thể không viết nhiều mã như các lập trình viên khác, nhưng họ vẫn tiếp tục dẫn thân vào công việc lập trình. Họ làm vậy vì họ không thể làm tốt công việc của mình nếu không tự mình nếm trải những vấn đề mà chính họ đang tạo ra cho phần còn lại của đội.

The architecture of a software system is the shape given to that system by those who build it. The form of that shape is in the division of that system into components, the arrangement of those components, and the ways in which those components communicate with each other.

Kiến trúc (architecture) của một hệ thống phần mềm là hình hài mà những người xây dựng

nó trao cho hệ thống ấy. Hình hài đó thể hiện qua việc chia hệ thống thành các component, cách sắp xếp các component đó, và cách chúng giao tiếp với nhau.

The purpose of that shape is to facilitate the development, deployment, operation, and maintenance of the software system contained within it.

Mục đích của hình hài ấy là tạo thuận lợi cho việc phát triển, triển khai, vận hành và bảo trì hệ thống phần mềm nằm bên trong nó.

The strategy behind that facilitation is to leave as many options open as possible, for as long as possible.

Chiến lược đằng sau sự thuận lợi ấy là để ngỏ càng nhiều lựa chọn càng tốt, trong thời gian càng lâu càng tốt.

Perhaps this statement has surprised you. Perhaps you thought that the goal of software architecture was to make the system work properly. Certainly we want the system to work properly, and certainly the architecture of the system must support that as one of its highest priorities.

Có lẽ câu này làm bạn ngạc nhiên. Có lẽ bạn từng nghĩ mục tiêu của kiến trúc phần mềm là làm cho hệ thống chạy đúng. Chắc chắn chúng ta muốn hệ thống chạy đúng, và chắc chắn kiến trúc của hệ thống phải xem đó là một trong những ưu tiên cao nhất.

However, the architecture of a system has very little bearing on whether that system works. There are many systems out there, with terrible architectures, that work just fine. Their troubles do not lie in their operation; rather, they occur in their deployment, maintenance, and ongoing development.

Tuy nhiên, kiến trúc của một hệ thống có rất ít liên quan tới chuyện hệ thống đó có chạy được hay không. Ngoài kia có vô số hệ thống với kiến trúc tồi tệ mà vẫn chạy ngon lành. Rắc rối của chúng không nằm ở khâu vận hành; mà nằm ở khâu triển khai, bảo trì và phát triển tiếp.

This is not to say that architecture plays no role in supporting the proper **behavior** of the system. It certainly does, and that role is critical. But the role is passive and cosmetic, not active or essential. There are few, if any, behavioral options that the architecture of a system can leave open.

Nói vậy không có nghĩa là kiến trúc chẳng đóng vai trò gì trong việc hỗ trợ hành vi (behavior) đúng đắn của hệ thống. Chắc chắn là có, và vai trò đó rất quan trọng. Nhưng đó là vai trò thụ động và bề ngoài, chứ không chủ động hay thiết yếu. Có rất ít — nếu có — lựa chọn về hành vi mà kiến trúc của một hệ thống có thể để ngỏ.

The primary purpose of architecture is to support the life cycle of the system. Good architecture makes the system easy to understand, easy to develop, easy to maintain, and easy to deploy. The ultimate goal is to minimize the lifetime cost of the system and to maximize programmer productivity.

Mục đích chính của kiến trúc là hỗ trợ vòng đời của hệ thống. Kiến trúc tốt làm cho hệ thống dễ hiểu, dễ phát triển, dễ bảo trì và dễ triển khai. Mục tiêu tối hậu là giảm thiểu chi phí trọn đời của hệ thống và tối đa hoá năng suất của lập trình viên.

Development — Phát triển

A software system that is hard to develop is not likely to have a long and healthy lifetime. So the architecture of a system should make that system easy to develop, for the team(s) who develop it.

Một hệ thống phần mềm khó phát triển thì khó lòng có một vòng đời dài và khoẻ mạnh. Vì thế kiến trúc của một hệ thống phải làm cho hệ thống đó dễ phát triển, đối với (những) đội đang phát triển nó.

Different team structures imply different architectural decisions. On the one hand, a small team of five developers can quite effectively work together to develop a monolithic system without well-defined components or interfaces. In fact, such a team would likely find the strictures of an architecture something of an impediment during the early days of development. This is likely the reason why so many systems lack good architecture: They were begun with none, because the team was small and did not want the impediment of a superstructure.

Cấu trúc đội khác nhau kéo theo những quyết định kiến trúc khác nhau. Một mặt, một đội nhỏ gồm năm lập trình viên hoàn toàn có thể phối hợp hiệu quả để phát triển một hệ thống nguyên khối mà không cần component hay interface được định nghĩa rõ ràng. Thật ra, một đội như thế có lẽ sẽ thấy những ràng buộc của kiến trúc là một thứ vướng chân trong những ngày đầu phát triển. Đây rất có thể là lý do vì sao quá nhiều hệ thống thiếu kiến trúc tốt: Chúng khởi đầu mà chẳng có kiến trúc nào, vì đội thì nhỏ và không muốn bị vướng bởi một bộ khung đồ sộ.

On the other hand, a system being developed by five different teams, each of which includes seven developers, cannot make progress unless the system is divided into well-defined components with reliably **stable** interfaces. If no other factors are considered, the architecture of that system will likely evolve into five components—one for each team.

Mặt khác, một hệ thống được phát triển bởi năm đội khác nhau, mỗi đội bảy lập trình viên, thì không thể tiến triển nếu hệ thống không được chia thành các component được định nghĩa rõ ràng với interface ổn định đáng tin cậy. Nếu không xét tới yếu tố nào khác, kiến trúc của hệ thống đó nhiều khả năng sẽ tiến hoá thành năm component — mỗi đội một cái.

Such a **component**-per-team architecture is not likely to be the best architecture for deployment, operation, and maintenance of the system. Nevertheless, it is the architecture that a group of teams will gravitate toward if they are driven solely by development schedule.

Kiểu kiến trúc “mỗi đội một component” như thế khó có thể là kiến trúc tốt nhất cho việc triển khai, vận hành và bảo trì hệ thống. Dẫu vậy, đó chính là kiến trúc mà một nhóm các đội sẽ trôi dạt về nếu họ chỉ bị dẫn dắt bởi lịch phát triển.

Deployment — Triển khai

To be effective, a software system must be deployable. The higher the cost of deployment, the less useful the system is. A goal of a software architecture, then, should be to make a system that can be easily deployed with a single action .

Để hữu dụng, một hệ thống phần mềm phải triển khai được. Chi phí triển khai càng cao thì hệ thống càng ít giá trị. Vậy nên một mục tiêu của kiến trúc phần mềm phải là tạo ra hệ thống có thể triển khai dễ dàng chỉ bằng một thao tác duy nhất.

Unfortunately, deployment strategy is seldom considered during initial development. This leads to architectures that may make the system easy to develop, but leave it very difficult to deploy.

Đáng tiếc, chiến lược triển khai hiếm khi được cân nhắc trong giai đoạn phát triển ban đầu. Điều này dẫn tới những kiến trúc có thể làm hệ thống dễ phát triển, nhưng lại khiến nó cực kỳ khó triển khai.

For example, in the early development of a system, the developers may decide to use a “**micro-service** architecture.” They may find that this approach makes the system very easy to develop since the component boundaries are very firm and the interfaces relatively stable. However, when it comes time to deploy the system, they may discover that the number of micro-services has become daunting; configuring the connections between them, and the timing of their initiation, may also turn out to be a huge source of errors.

Chẳng hạn, trong giai đoạn phát triển đầu của một hệ thống, các lập trình viên có thể quyết định dùng “kiến trúc micro-service”. Họ có thể thấy cách tiếp cận này làm hệ thống rất dễ phát triển vì ranh giới (boundary) giữa các component rất rạch ròi và interface tương đối ổn định. Tuy nhiên, tới lúc phải triển khai hệ thống, họ có thể phát hiện ra số lượng micro-service đã trở nên đáng sợ; việc cấu hình kết nối giữa chúng, cùng thời điểm khởi động của từng cái, cũng có thể hoá thành một nguồn lỗi khổng lồ.

Had the architects considered deployment issues early on, they might have decided on fewer services, a hybrid of services and in-process components, and a more integrated means of managing the interconnections.

Nếu các kiến trúc sư đã cân nhắc vấn đề triển khai từ sớm, có lẽ họ đã chọn ít dịch vụ (service) hơn, một dạng lai giữa dịch vụ và component chạy trong cùng tiến trình, cùng một cách quản lý các kết nối liên thông tích hợp hơn.

Operation — Vận hành

The impact of architecture on system operation tends to be less dramatic than the impact of architecture on development, deployment, and maintenance. Almost any operational difficulty can be resolved by throwing more hardware at the system without drastically impacting the software architecture.

Ảnh hưởng của kiến trúc lên việc vận hành hệ thống thường ít kịch tính hơn ảnh hưởng của nó lên phát triển, triển khai và bảo trì. Gần như mọi khó khăn vận hành đều có thể giải quyết bằng cách ném thêm phần cứng vào hệ thống mà không cần tác động mạnh tới kiến trúc phần mềm.

Indeed, we have seen this happen over and over again. Software systems that have inefficient architectures can often be made to work effectively simply by adding more storage and more servers. The fact that hardware is cheap and people are expensive means that architectures that impede operation are not as costly as architectures that impede development, deployment, and maintenance.

Quả thật, chúng ta đã chứng kiến chuyện này lặp đi lặp lại. Những hệ thống phần mềm có kiến trúc kém hiệu quả thường vẫn có thể chạy tốt chỉ bằng cách thêm dung lượng lưu trữ và thêm máy chủ. Việc phần cứng thì rẻ mà con người thì đắt có nghĩa là kiến trúc cản trở vận hành không tốn kém bằng kiến trúc cản trở phát triển, triển khai và bảo trì.

This is not to say that an architecture that is well tuned to the operation of the system is not desirable. It is! It's just that the cost equation leans more toward development, deployment, and maintenance.

Nói vậy không có nghĩa là một kiến trúc được tinh chỉnh tốt cho việc vận hành hệ thống là không đáng mong muốn. Rất đáng chú! Chỉ là phương trình chi phí nghiêng nhiều hơn về phía phát triển, triển khai và bảo trì.

Having said that, there is another role that architecture plays in the operation of the system: A good software architecture communicates the operational needs of the system.

Dù vậy, kiến trúc còn đóng thêm một vai trò nữa trong việc vận hành hệ thống: một kiến trúc phần mềm tốt nói lên được nhu cầu vận hành của hệ thống.

Perhaps a better way to say this is that the architecture of a system makes the operation of the system readily apparent to the developers. Architecture should reveal operation. The architecture of the system should elevate the use cases, the features, and the required behaviors of the system to first-class entities that are visible landmarks for the developers. This simplifies the understanding of the system and, therefore, greatly aids in development and maintenance.

Có lẽ cách nói hay hơn là: kiến trúc của một hệ thống làm cho việc vận hành hệ thống trở nên hiển hiện ngay trước mắt các lập trình viên. Kiến trúc phải phơi bày cách vận hành. Kiến trúc của hệ thống phải nâng các use case, các tính năng và các hành vi bắt buộc của hệ thống lên thành những thực thể hạng nhất, thành những cột mốc mà lập trình viên nhìn thấy được. Điều này làm cho hệ thống dễ hiểu hơn nhiều, và nhờ đó hỗ trợ rất lớn cho việc phát triển và bảo trì.

Maintenance — Bảo trì

Of all the aspects of a software system, maintenance is the most costly. The never-ending parade of new features and the inevitable trail of defects and corrections consume vast amounts of **human resources**.

Trong tất cả các khía cạnh của một hệ thống phần mềm, bảo trì là thứ tốn kém nhất. Đoàn diễu hành bất tận của các tính năng mới cùng cái đuôi không tránh khỏi gồm lỗi và sửa lỗi ngốn một lượng nhân lực khổng lồ.

The primary cost of maintenance is in spelunking and risk. Spelunking is the cost of digging through the existing software, trying to determine the best place and the best strategy to add a new feature or to repair a defect. While making such changes, the likelihood of creating inadvertent defects is always there, adding to the cost of risk.

Chi phí chính của bảo trì nằm ở việc mò mẫm và ở rủi ro. Mò mẫm là chi phí của việc đào bới xuyên qua phần mềm hiện có, cố xác định đâu là chỗ tốt nhất và chiến lược tốt nhất để thêm một tính năng mới hoặc sửa một lỗi. Trong khi thực hiện những thay đổi ấy, khả năng vô tình tạo ra lỗi mới luôn rình rập, cộng thêm vào đó chi phí của rủi ro.

A carefully thought-through architecture vastly mitigates these costs. By separating the system into components, and isolating those components through stable interfaces, it is possible to illuminate the pathways for future features and greatly reduce the risk of inadvertent breakage.

Một kiến trúc được cân nhắc kỹ lưỡng làm giảm mạnh những chi phí này. Bằng cách tách hệ thống thành các component, và cô lập các component đó qua những interface ổn định, ta có thể soi sáng lối đi cho các tính năng tương lai và giảm đáng kể rủi ro vô tình làm vỡ thứ khác.

Keeping Options Open — Để ngỏ các lựa chọn

As we described in an earlier chapter, software has two types of value: the value of its behavior and the value of its **structure**. The second of these is the greater of the two because it is this value that makes software soft .

Như đã mô tả ở một chương trước, phần mềm có hai loại giá trị: giá trị của hành vi và giá trị của cấu trúc (structure). Loại thứ hai lớn hơn loại thứ nhất, bởi chính giá trị này mới làm cho phần mềm trở nên *mềm*.

Software was invented because we needed a way to quickly and easily change the behavior of machines. But that flexibility depends critically on the shape of the system, the arrangement of its components, and the way those components are interconnected.

Phần mềm được phát minh vì chúng ta cần một cách để thay đổi hành vi của máy móc thật nhanh và thật dễ. Nhưng sự linh hoạt đó phụ thuộc sống còn vào hình hài của hệ thống, cách sắp xếp các component của nó, và cách các component đó nối kết với nhau.

The way you keep software soft is to leave as many options open as possible, for as long as possible. What are the options that we need to leave open? They are the details that don't matter.

Cách để giữ cho phần mềm luôn mềm là để ngỏ càng nhiều lựa chọn càng tốt, trong thời gian càng lâu càng tốt. Vậy những lựa chọn nào cần để ngỏ? Đó là những chi tiết (detail) không quan trọng.

All software systems can be decomposed into two major elements: **policy** and details. The policy element embodies all the business rules and procedures. The policy is where the true value of the system lives.

Mọi hệ thống phần mềm đều có thể phân rã thành hai thành phần chính: chính sách (policy) và chi tiết. Phần chính sách chứa đựng toàn bộ quy tắc nghiệp vụ (business rule) và quy trình. Chính sách là nơi giá trị thật sự của hệ thống trú ngụ.

The details are those things that are necessary to enable humans, other systems, and programmers to communicate with the policy, but that do not impact the behavior of the policy at all. They include IO devices, databases, web systems, servers, frameworks, communication protocols, and so forth.

Chi tiết là những thứ cần thiết để con người, các hệ thống khác và lập trình viên giao tiếp được với chính sách, nhưng lại hoàn toàn không tác động tới hành vi của chính sách. Chúng bao gồm thiết bị IO, cơ sở dữ liệu, hệ thống web, máy chủ, framework, giao thức truyền thông, v.v.

The goal of the architect is to create a shape for the system that recognizes policy as the most essential element of the system while making the details irrelevant to that policy. This allows decisions about those details to be delayed and deferred .

Mục tiêu của kiến trúc sư là tạo ra một hình hài cho hệ thống, trong đó chính sách được thừa nhận là thành phần thiết yếu nhất, còn các chi tiết thì chẳng còn liên quan gì tới chính sách ấy. Nhờ vậy, các quyết định về những chi tiết đó có thể được *hoãn lại và trì hoãn*.

For example:

Ví dụ:

- It is not necessary to choose a database system in the early days of
 - Không cần chọn hệ cơ sở dữ liệu ngay trong những ngày đầu

development, because the **high-level** policy should not care which kind of database will be used. Indeed, if the architect is careful, the high-level policy will not care if the database is relational, distributed, hierarchical, or just plain flat files.

phát triển, vì chính sách mức cao lẽ ra không nên bận tâm sẽ dùng loại cơ sở dữ liệu nào. Thật vậy, nếu kiến trúc sư cẩn thận, chính sách mức cao sẽ chẳng quan tâm cơ sở dữ liệu là quan hệ, phân tán, phân cấp, hay chỉ là mấy tệp phẳng trơ trọi.

- It is not necessary to choose a web server early in development, because the

- Không cần chọn web server sớm trong quá trình phát triển, vì

high-level policy should not know that it is being delivered over the web. If the high-level policy is unaware of HTML, AJAX, JSP, JSF, or any of the rest of the alphabet soup of web development, then you don't need to decide which web system to use until much later in the project. Indeed, you don't even have to decide if the system will be delivered over the web .

chính sách mức cao lẽ ra không nên biết rằng nó đang được phân phối qua web. Nếu chính sách mức cao không hề hay biết gì về HTML, AJAX, JSP, JSF, hay bất kỳ món nào khác trong nôi lấu chữ cái của phát triển web, thì bạn không cần quyết định dùng hệ thống web nào cho tới tận rất muộn trong dự án. Thật vậy, bạn thậm chí còn chưa cần quyết định liệu hệ thống có được phân phối qua web hay không.

- It is not necessary to adopt REST early in development, because the high-

- Không cần áp dụng REST sớm trong quá trình phát triển, vì chính

level policy should be agnostic about the **interface** to the outside world. Nor is it necessary to adopt a micro-services **framework**, or a SOA framework. Again, the high-level policy should not care about these things.

sách mức cao lẽ ra phải bàng quan với giao diện hướng ra thế giới bên ngoài. Cũng không cần áp dụng một framework micro-service, hay một framework SOA. Một lần nữa, chính sách mức cao không nên quan tâm tới những thứ này.

- It is not necessary to adopt a dependency injection framework early in

- Không cần áp dụng framework dependency injection sớm trong quá trình

development, because the high-level policy should not care how dependencies are resolved.

phát triển, vì chính sách mức cao không nên bận tâm các phụ thuộc được giải quyết ra sao.

I think you get the point. If you can develop the high-level policy without committing to the details that surround it, you can delay and defer decisions about those details for a long time. And the longer you wait to make those decisions, the more information you have with which to make them properly

.

Tôi nghĩ bạn đã hiểu ý. Nếu bạn có thể phát triển chính sách mức cao mà không phải cam kết với những chi tiết vây quanh nó, bạn có thể hoãn và trì hoãn các quyết định về những chi tiết đó thật lâu. Và bạn càng chờ lâu mới ra quyết định, bạn càng có nhiều thông tin để ra quyết định cho đúng.

This also leaves you the option to try different experiments. If you have a portion of the high-level policy working, and it is agnostic about the database, you could try connecting it to several different databases to check applicability and performance. The same is true with web systems, web frameworks, or even the web itself.

Điều này cũng để ngỏ cho bạn lựa chọn thử nghiệm nhiều hướng khác nhau. Nếu bạn đã có một phần chính sách mức cao chạy được, và nó bàng quan với cơ sở dữ liệu, bạn có thể thử nối nó với vài cơ sở dữ liệu khác nhau để kiểm tra mức độ phù hợp và hiệu năng. Với hệ thống web, framework web, hay thậm chí bản thân web, chuyện cũng y hệt.

The longer you leave options open, the more experiments you can run, the more things you can try, and the more information you will have when you reach the point at which those decisions can no longer be deferred.

Bạn để ngỏ các lựa chọn càng lâu thì càng chạy được nhiều thử nghiệm, càng thử được nhiều thứ, và càng có nhiều thông tin khi đến lúc không thể trì hoãn những quyết định ấy thêm nữa.

What if the decisions have already been made by someone else? What if your company has made a commitment to a certain database, or a certain web server, or a certain framework? A good architect pretends that the decision has not been made, and shapes the system such that those decisions can still be deferred or changed for as long as possible.

Còn nếu quyết định đã được người khác đưa ra rồi thì sao? Nếu công ty bạn đã cam kết với một cơ sở dữ liệu nhất định, hay một web server nhất định, hay một framework nhất định thì sao? Một kiến trúc sư giỏi sẽ *giả vờ như quyết định đó chưa được đưa ra*, và định hình hệ thống sao cho những quyết định ấy vẫn có thể trì hoãn hoặc thay đổi trong thời gian lâu nhất có thể.

A good architect maximizes the number of decisions not made.

Một kiến trúc sư giỏi tối đa hoá số lượng *quyết định chưa đưa ra*.

Device Independence — Độc lập với thiết bị

As an example of this kind of thinking, let's take a trip back to the 1960s, when computers were teenagers and most programmers were mathematicians or engineers from other disciplines (and one third or more were women).

Để làm ví dụ cho lối tư duy này, ta hãy quay ngược về thập niên 1960, khi máy tính còn đang tuổi teen và phần lớn lập trình viên là nhà toán học hoặc kỹ sư từ các ngành khác (và một phần ba trở lên là phụ nữ).

In those days we made a lot of mistakes. We didn't know they were mistakes at the time, of course. How could we?

Hồi đó chúng tôi phạm rất nhiều sai lầm. Dĩ nhiên lúc ấy chúng tôi không biết đó là sai lầm. Làm sao biết được?

One of those mistakes was to bind our code directly to the IO devices. If we needed to print something on a printer, we wrote code that used the IO instructions that would control the printer. Our code was device dependent.

Một trong những sai lầm đó là buộc chặt mã của chúng tôi trực tiếp vào các thiết bị IO. Nếu cần in cái gì đó ra máy in, chúng tôi viết mã dùng thẳng các lệnh IO điều khiển máy in. Mã của chúng tôi *phụ thuộc vào thiết bị*.

For example, when I wrote PDP-8 programs that printed on the teleprinter, I used a set of machine instructions that looked like this:

Chẳng hạn, khi tôi viết chương trình PDP-8 in ra máy teleprinter, tôi dùng một tập lệnh máy trông như thế này:

```
PRTCHR, 0
    TSF
    JMP . - 1
    TLS
    JMP I PRTCHR
```

PRTCHR is a subroutine that prints one character on the teleprinter. The beginning zero was used as the storage for the return address. (Don't ask.) The TSF instruction skipped the next instruction if the teleprinter was ready to print a character. If the teleprinter was busy, then TSF just fell through to

PRTCHR là một subroutine in một ký tự ra teleprinter. Số 0 ở đầu được dùng làm chỗ lưu địa chỉ trả về. (Đừng hỏi.) Lệnh TSF sẽ bỏ qua lệnh kế tiếp nếu teleprinter đã sẵn sàng in một ký tự. Nếu teleprinter đang bận, TSF sẽ rơi thẳng xuống

the JMP -1 instruction, which just jumped back to the TSF instruction. If the teleprinter was ready, then TSF would skip to the TLS instruction, which sent the character in the A register to the teleprinter. Then the JMP I PRTCHR instruction returned to the caller.

lệnh JMP -1, và lệnh này nhảy ngược về TSF. Nếu teleprinter đã sẵn sàng, TSF sẽ nhảy cóc tới lệnh TLS, lệnh này gửi ký tự đang nằm trong thanh ghi A tới teleprinter. Sau đó lệnh JMP I PRTCHR trả điều khiển về cho nơi gọi.

At first this strategy worked fine. If we needed to read cards from the card reader, we used code that talked directly to the card reader. If we needed to punch cards, we wrote code that directly manipulated the punch. The programs worked perfectly. How could we know this was a mistake?

Ban đầu chiến lược này chạy tốt. Nếu cần đọc thẻ từ đầu đọc thẻ, chúng tôi dùng mã nói chuyện trực tiếp với đầu đọc thẻ. Nếu cần đục thẻ, chúng tôi viết mã điều khiển trực tiếp máy đục. Chương trình chạy hoàn hảo. Làm sao chúng tôi biết đó là một sai lầm?

But big batches of punched cards are difficult to manage. They can be lost, mutilated, spindled, shuffled, or dropped. Individual cards can be lost and extra cards can be inserted. So data integrity became a significant problem.

Nhưng những xấp thẻ đục lỗ lớn thì rất khó quản lý. Chúng có thể bị mất, bị rách, bị kẹt trục, bị xáo trộn hoặc rơi vãi. Có thể mất từng lá thẻ và cũng có thể bị chèn thêm thẻ lạ. Thế là tính toàn vẹn dữ liệu trở thành một vấn đề nghiêm trọng.

Magnetic tape was the solution. We could move the card images to tape. If you drop a magnetic tape, the records don't get shuffled. You can't accidentally lose a record, or insert a blank record simply by handing the tape. The tape is much more secure. It's also faster to read and write, and it is very easy to make backup copies.

Băng từ chính là giải pháp. Chúng tôi có thể chuyển ảnh của các lá thẻ sang băng. Bạn có làm rơi cuộn băng từ thì các bản ghi cũng chẳng bị xáo tung. Bạn không thể vô tình đánh mất một bản ghi, hay chèn vào một bản ghi trống chỉ vì cầm nhầm cuộn băng. Băng an toàn hơn nhiều. Nó cũng đọc ghi nhanh hơn, và rất dễ tạo bản sao lưu.

Unfortunately, all our software was written to manipulate card readers and card punches. Those programs had to be rewritten to use magnetic tape. That was a big job.

Khổ nỗi, toàn bộ phần mềm của chúng tôi lại được viết để điều khiển đầu đọc thẻ và máy đục thẻ. Những chương trình đó phải viết lại để dùng băng từ. Đó là một khối lượng công việc khổng lồ.

By the late 1960s, we had learned our lesson—and we invented device independence. The operating systems of the day abstracted the IO devices into software functions that handled unit records that looked like cards. The programs would invoke operating system services that dealt with abstract unit-record devices. Operators could tell the operating system whether those abstract services should be connected to card readers, magnetic tape, or any other unit-record device.

Tới cuối thập niên 1960, chúng tôi đã học được bài học của mình — và chúng tôi phát minh ra *tính độc lập với thiết bị*. Các hệ điều hành thời đó trừu tượng hoá thiết bị IO thành những

hàm phần mềm xử lý các bản ghi đơn vị trông giống như lá thẻ. Chương trình sẽ gọi các dịch vụ của hệ điều hành làm việc với những thiết bị bản-ghi-đơn-vị trừu tượng. Người vận hành có thể bảo hệ điều hành rằng những dịch vụ trừu tượng ấy nên được nối vào đầu đọc thẻ, băng từ, hay bất kỳ thiết bị bản-ghi-đơn-vị nào khác.

Now the same program could read and write cards, or read and write tape, without any change. The Open-Closed Principle was born (but not yet named).

Giờ đây cùng một chương trình có thể đọc và ghi thẻ, hoặc đọc và ghi băng, *mà không cần sửa gì cả*. Open-Closed Principle đã ra đời (dù chưa được đặt tên).

Junk Mail — Thư rác quảng cáo

In the late 1960s, I worked for a company that printed junk mail for clients. The clients would send us magnetic tapes with unit records containing the names and addresses of their customers, and we would write programs that printed nice personalized advertisements.

Cuối thập niên 1960, tôi làm cho một công ty in thư rác quảng cáo cho khách hàng. Khách hàng gửi cho chúng tôi băng từ chứa các bản ghi đơn vị với tên và địa chỉ khách hàng của họ, còn chúng tôi viết chương trình in ra những mẫu quảng cáo cá nhân hoá rất bắt mắt.

You know the kind:

Kiểu như thế này:

Hello Mr. Martin, Congratulations!

Xin chào ông Martin, chúc mừng ông!

We chose YOU from everyone else who lives on Witchwood Lane to participate in our new fantastic one-time-only offering...

Chúng tôi đã chọn CHÍNH ÔNG trong số tất cả cư dân sống trên phố Witchwood Lane để tham gia chương trình ưu đãi mới tuyệt vời chỉ có một lần duy nhất...

The clients would send us huge rolls of form letters with all the words except the name and address, and any other element they wanted us to print. We wrote programs that extracted the names, addresses, and other elements from the magnetic tape, and printed those elements exactly where they needed to appear on the forms.

Khách hàng gửi cho chúng tôi những cuộn thư mẫu khổng lồ đã in sẵn mọi thứ trừ tên và địa chỉ, cùng bất kỳ phần nào khác mà họ muốn chúng tôi in. Chúng tôi viết chương trình trích tên, địa chỉ và các phần khác từ băng từ, rồi in chúng vào đúng vị trí cần xuất hiện trên các mẫu thư.

These rolls of form letters weighed 500 pounds and contained thousands of letters. Clients would send us hundreds of these rolls. We would print each one individually.

Những cuộn thư mẫu này nặng 500 pound và chứa hàng nghìn lá thư. Khách hàng gửi cho chúng tôi hàng trăm cuộn như vậy. Chúng tôi in từng cuộn một.

At first, we had an IBM 360 doing the printing on its sole line printer. We could print a few thousand letters per shift. Unfortunately, this tied up a very expensive machine for a very long time. In those days, IBM 360s rented for tens of thousands of dollars per month.

Ban đầu, chúng tôi dùng một chiếc IBM 360 để in qua chiếc máy in dòng duy nhất của nó. Mỗi ca chúng tôi in được vài nghìn lá thư. Đáng tiếc, việc này chiếm dụng một cỗ máy cực

đắt trong thời gian cực dài. Hồi đó, tiền thuê một chiếc IBM 360 lên tới hàng chục nghìn đô la mỗi tháng.

So we told the operating system to use magnetic tape instead of the line printer. Our programs didn't care, because they had been written to use the IO abstractions of the operating system.

Vậy nên chúng tôi bảo hệ điều hành dùng băng từ thay cho máy in dòng. Chương trình của chúng tôi chẳng bận tâm, bởi chúng đã được viết để dùng các trừu tượng IO của hệ điều hành.

The 360 could pump out a full tape in 10 minutes or so—enough to print several rolls of form letters. The tapes were taken outside of the computer

Chiếc 360 có thể bơm đầy một cuộn băng trong khoảng 10 phút — đủ để in vài cuộn thư mẫu. Các cuộn băng được mang ra khỏi phòng

room and mounted on tape drives connected to offline printers. We had five of them, and we ran those five printers 24 hours per day, seven days per week, printing hundreds of thousands of pieces of junk mail every week.

máy tính và lắp lên những ổ băng nối với các máy in offline. Chúng tôi có năm máy như vậy, và cho năm chiếc máy in đó chạy 24 giờ mỗi ngày, bảy ngày mỗi tuần, in ra hàng trăm nghìn mẫu thư rác quảng cáo mỗi tuần.

The value of device independence was enormous! We could write our programs without knowing or caring which device would be used. We could test those programs using the local line printer connected to the computer. Then we could tell the operating system to “print” to magnetic tape and run off hundreds of thousands of forms.

Giá trị của tính độc lập với thiết bị là khổng lồ! Chúng tôi có thể viết chương trình mà không cần biết hay bận tâm thiết bị nào sẽ được dùng. Chúng tôi có thể kiểm thử những chương trình đó bằng chiếc máy in dòng nối trực tiếp với máy tính. Rồi chúng tôi có thể bảo hệ điều hành “in” ra băng từ và cho chạy ra hàng trăm nghìn mẫu thư.

Our programs had a shape. That shape disconnected policy from **detail**. The policy was the formatting of the name and address records. The detail was the device. We deferred the decision about which device we would use.

Chương trình của chúng tôi có một hình hài. Hình hài đó ngắt kết nối chính sách khỏi chi tiết. Chính sách là việc định dạng các bản ghi tên và địa chỉ. Chi tiết là thiết bị. Chúng tôi đã trì hoãn quyết định về việc sẽ dùng thiết bị nào.

Physical Addressing — Đánh địa chỉ vật lý

In the early 1970s, I worked on a large accounting system for a local truckers union. We had a 25MB disk drive on which we stored records for Agents, Employers, and Members. The different records had different sizes, so we formatted the first few cylinders of the disk so that each sector was just the size of an Agent record. The next few cylinders were formatted to have sectors that fit the Employer records. The last few cylinders were formatted to fit the Member records.

Đầu thập niên 1970, tôi làm việc trên một hệ thống kế toán lớn cho một nghiệp đoàn tài xế xe tải ở địa phương. Chúng tôi có một ổ đĩa 25MB để lưu các bản ghi *Agents*, *Employers* và *Members*. Các loại bản ghi có kích thước khác nhau, nên chúng tôi định dạng vài cylinder đầu của đĩa sao cho mỗi sector vừa khít kích thước một bản ghi Agent. Vài cylinder tiếp

theo được định dạng để có các sector vừa với bản ghi Employer. Vài cylinder cuối được định dạng vừa với bản ghi Member.

We wrote our software to know the detailed structure of the disk. It knew that the disk had 200 cylinders and 10 heads, and that each cylinder had several dozen sectors per head. It knew which cylinders held the Agents, Employers, and Members. All this was hard-wired into the code.

Chúng tôi viết phần mềm của mình sao cho nó biết cấu trúc chi tiết của đĩa. Nó biết đĩa có 200 cylinder và 10 đầu đọc, và mỗi cylinder có vài chục sector trên mỗi đầu đọc. Nó biết cylinder nào chứa Agents, Employers và Members. Tất cả những thứ này được đấu cứng vào mã nguồn.

We kept an index on the disk that allowed us to look up each of the Agents, Employers, and Members. This index was in yet another specially formatted set of cylinders on the disk. The Agent index was composed of records that contained the ID of an agent, and the cylinder number, head number, and sector number of that Agent record. Employers and Members had similar indices. Members were also kept in a doubly linked list on the disk. Each

Chúng tôi giữ trên đĩa một chỉ mục cho phép tra cứu từng Agent, Employer và Member. Chỉ mục này lại nằm trong một tập cylinder khác được định dạng đặc biệt trên đĩa. Chỉ mục Agent gồm các bản ghi chứa ID của một agent, cùng số cylinder, số đầu đọc và số sector của bản ghi Agent đó. Employers và Members cũng có những chỉ mục tương tự. Members còn được lưu trên đĩa dưới dạng danh sách liên kết đôi. Mỗi

Member record held the cylinder, head, and sector number of the next Member record, and of the previous Member record.

bản ghi Member giữ số cylinder, số đầu đọc và số sector của bản ghi Member kế tiếp, và của bản ghi Member liền trước.

What would happen if we needed to upgrade to a new disk drive—one with more heads, or one with more cylinders, or one with more sectors per cylinder? We had to write a special program to read in the old data from the old disk, and then write it out to the new disk, translating all of the cylinder/head/sector numbers. We also had to change all the hard-wiring in our code—and that hard-wiring was everywhere! All the business rules knew the cylinder/head/sector scheme in detail.

Chuyện gì sẽ xảy ra nếu chúng tôi cần nâng cấp lên một ổ đĩa mới — loại có nhiều đầu đọc hơn, hoặc nhiều cylinder hơn, hoặc nhiều sector trên mỗi cylinder hơn? Chúng tôi phải viết một chương trình riêng để đọc dữ liệu cũ từ đĩa cũ, rồi ghi ra đĩa mới, chuyển đổi toàn bộ số cylinder/đầu đọc/sector. Chúng tôi còn phải sửa hết mọi chỗ đấu cứng trong mã — và những chỗ đấu cứng ấy nằm *khắp nơi*! Mọi quy tắc nghiệp vụ đều nắm rõ chi tiết sơ đồ cylinder/đầu đọc/sector.

One day a more experienced programmer joined our ranks. When he saw what we had done, the blood drained from his face, and he stared aghast at us, as if we were aliens of some kind. Then he gently advised us to change our addressing scheme to use relative addresses.

Một ngày nọ, một lập trình viên kỳ cựu hơn gia nhập hàng ngũ của chúng tôi. Khi thấy những gì chúng tôi đã làm, mặt anh ta tái mét, và anh ta trợn mắt kinh hoàng nhìn chúng tôi như thể chúng tôi là người ngoài hành tinh. Rồi anh ta nhẹ nhàng khuyên chúng tôi đổi sơ đồ đánh địa chỉ sang dùng địa chỉ tương đối.

Our wiser colleague suggested that we consider the disk to be one huge linear array of sectors, each addressable by a sequential integer. Then we could write a little conversion routine that knew the physical structure of the disk, and could translate the relative address to a cylinder/head/sector number on the fly.

Người đồng nghiệp khôn ngoan hơn ấy gợi ý rằng chúng tôi hãy coi đĩa như một mảng tuyến tính khổng lồ gồm các sector, mỗi sector được đánh địa chỉ bằng một số nguyên tuần tự. Rồi chúng tôi có thể viết một thủ tục chuyển đổi nhỏ, nó biết cấu trúc vật lý của đĩa và có thể dịch địa chỉ tương đối thành số cylinder/đầu đọc/sector ngay tại chỗ.

Fortunately for us, we took his advice. We changed the high-level policy of the system to be agnostic about the physical structure of the disk. That allowed us to decouple the decision about disk drive structure from the application.

May cho chúng tôi, chúng tôi đã nghe lời anh ấy. Chúng tôi sửa chính sách mức cao của hệ thống để nó bàng quan với cấu trúc vật lý của đĩa. Điều đó cho phép chúng tôi tách rời quyết định về cấu trúc ổ đĩa khỏi ứng dụng.

Conclusion — Kết luận

The two stories in this chapter are examples, in the small, of a principle that architects employ in the large. Good architects carefully separate details from policy, and then decouple the policy from the details so thoroughly that the policy has no knowledge of the details and does not depend on the details in any way. Good architects design the policy so that decisions about the details can be delayed and deferred for as long as possible.

Hai câu chuyện trong chương này là ví dụ ở quy mô nhỏ cho một nguyên tắc mà các kiến trúc sư áp dụng ở quy mô lớn. Kiến trúc sư giỏi tách bạch cẩn thận chi tiết khỏi chính sách, rồi tách rời chính sách khỏi chi tiết triệt để đến mức chính sách chẳng hay biết gì về chi tiết và không phụ thuộc vào chi tiết theo bất kỳ cách nào. Kiến trúc sư giỏi thiết kế chính sách sao cho các quyết định về chi tiết có thể được hoãn lại và trì hoãn lâu nhất có thể.

Independence — Tính độc lập

As we previously stated, a good **architecture** must support:

Như đã nói ở phần trước, một kiến trúc (architecture) tốt phải hỗ trợ:

- The use cases and operation of the system.
 - Các use case và hoạt động vận hành của hệ thống.
- The maintenance of the system.
 - Việc bảo trì hệ thống.
- The development of the system.
 - Việc phát triển hệ thống.
- The deployment of the system.
 - Việc triển khai hệ thống.

Use Cases — Use case

The first bullet—use cases—means that the architecture of the system must support the intent of the system. If the system is a shopping cart application, then the architecture must support shopping cart use cases. Indeed, this is the first concern of the architect, and the first priority of the architecture. The architecture must support the use cases.

Gạch đầu dòng thứ nhất — use case — nghĩa là kiến trúc của hệ thống phải hỗ trợ cho ý đồ của hệ thống. Nếu hệ thống là một ứng dụng giỏ hàng, thì kiến trúc phải hỗ trợ các use case của giỏ hàng. Quả thật, đây là mối bận tâm đầu tiên của kiến trúc sư, và là ưu tiên số một của kiến trúc. Kiến trúc phải hỗ trợ các use case.

However, as we discussed previously, architecture does not wield much influence over the **behavior** of the system. There are very few behavioral options that the architecture can leave open. But influence isn't everything. The most **important** thing a good architecture can do to support behavior is to clarify and expose that behavior so that the intent of the system is visible at the architectural **level**.

Tuy nhiên, như đã bàn trước đó, kiến trúc không có mấy ảnh hưởng lên hành vi (behavior) của hệ thống. Có rất ít lựa chọn về hành vi mà kiến trúc có thể để ngỏ. Nhưng ảnh hưởng không phải là tất cả. Điều quan trọng nhất mà một kiến trúc tốt có thể làm để hỗ trợ hành vi là làm rõ và phơi bày hành vi đó, sao cho ý đồ của hệ thống hiện ra ngay ở mức kiến trúc.

A shopping cart application with a good architecture will look like a shopping cart application. The use cases of that system will be plainly visible within the **structure** of that system. Developers will not have to hunt for behaviors, because those behaviors will be first-class elements visible at the top level of the system. Those elements will be classes or functions or modules that have prominent positions within the architecture, and they will have names that clearly describe their function.

Một ứng dụng giỏ hàng có kiến trúc tốt sẽ trông giống một ứng dụng giỏ hàng. Các use case của hệ thống đó sẽ hiện rõ mồn một trong cấu trúc (structure) của hệ thống. Lập trình viên sẽ không phải đi lùng sục các hành vi, bởi những hành vi ấy là các phần tử hạng nhất, nhìn thấy được ngay ở mức cao nhất của hệ thống. Chúng sẽ là những class, hàm hoặc module chiếm vị trí nổi bật trong kiến trúc, và mang những cái tên mô tả rõ ràng chức năng của mình.

Chapter 21, “**Screaming Architecture**,” will make this point much clearer.

Chương 21, “Kiến trúc biết gào lên” (Screaming Architecture), sẽ làm rõ điểm này hơn nhiều.

Operation — Vận hành

Architecture plays a more substantial, and less cosmetic, role in supporting the operation of the system. If the system must handle 100,000 customers per second, the architecture must support that kind of throughput and response time for each **use case** that demands it. If the system must query big data cubes in milliseconds, then the architecture must be structured to allow this kind of operation.

Kiến trúc đóng một vai trò thực chất hơn, và ít mang tính hình thức hơn, trong việc hỗ trợ vận hành hệ thống. Nếu hệ thống phải xử lý 100.000 khách hàng mỗi giây, thì kiến trúc phải hỗ trợ được mức thông lượng và thời gian đáp ứng đó cho từng use case đòi hỏi điều ấy. Nếu hệ thống phải truy vấn những khối dữ liệu lớn trong vài mili-giây, thì kiến trúc phải được cấu trúc để cho phép kiểu vận hành này.

For some systems, this will mean arranging the processing elements of the system into an array of little services can be run in parallel on many different servers. For other systems, it will mean a plethora of little lightweight threads sharing the address space of a single process within a single processor. Still other systems will need just a few processes running in isolated address spaces. And some systems can even survive as simple monolithic programs running in a single process.

Với một số hệ thống, điều đó nghĩa là sắp xếp các phần tử xử lý của hệ thống thành một dàn các dịch vụ (service) nhỏ có thể chạy song song trên nhiều máy chủ khác nhau. Với hệ thống khác, nó lại nghĩa là hàng loạt luồng nhẹ dùng chung không gian địa chỉ của một tiến trình duy nhất trên một bộ xử lý duy nhất. Lại có những hệ thống chỉ cần vài tiến trình chạy trong các không gian địa chỉ tách biệt. Và một số hệ thống thậm chí vẫn sống khỏe dưới dạng chương trình monolithic đơn giản chạy trong một tiến trình.

As strange as it may seem, this decision is one of the options that a good architect leaves open. A system that is written as a monolith, and that depends on that monolithic structure, cannot easily be upgraded to multiple processes, multiple threads, or micro-services should the need arise. By comparison, an architecture that maintains the proper isolation of its components, and does not assume the means of communication between those components, will be much easier to transition through the spectrum of threads, processes, and services as the operational needs of the system change over time.

Nghe có vẻ lạ, nhưng quyết định này lại là một trong những lựa chọn mà kiến trúc sư giải để ngỏ. Một hệ thống được viết như một khối monolith, và phụ thuộc vào chính cấu trúc

monolithic ấy, sẽ không dễ nâng cấp thành nhiều tiến trình, nhiều luồng hay micro-service khi nhu cầu phát sinh. Ngược lại, một kiến trúc duy trì được sự cô lập đúng mực giữa các component của nó, và không giả định trước cách các component đó giao tiếp với nhau, sẽ dễ dàng chuyển đổi hơn nhiều dọc theo phổ luồng — tiến trình — dịch vụ, khi nhu cầu vận hành của hệ thống thay đổi theo thời gian.

Development — Phát triển

Architecture plays a significant role in supporting the development environment. This is where Conway's law comes into play. Conway's law says:

Kiến trúc đóng vai trò đáng kể trong việc hỗ trợ môi trường phát triển. Đây là lúc định luật Conway lên tiếng. Định luật Conway nói rằng:

Any organization that designs a system will produce a **design** whose structure is a copy of the organization's communication structure.

Bất kỳ tổ chức nào thiết kế một hệ thống cũng sẽ tạo ra một thiết kế mà cấu trúc của nó là bản sao cấu trúc giao tiếp của chính tổ chức đó.

A system that must be developed by an organization with many teams and many concerns must have an architecture that facilitates independent actions by those teams, so that the teams do not interfere with each other during development. This is accomplished by properly partitioning the system into well-isolated, independently developable components. Those components can then be allocated to teams that can work independently of each other.

Một hệ thống phải được phát triển bởi một tổ chức có nhiều đội và nhiều mối bận tâm thì phải có kiến trúc tạo điều kiện cho các đội hành động độc lập, sao cho các đội không giẫm chân nhau trong quá trình phát triển. Điều này đạt được bằng cách phân hoạch hệ thống một cách đúng đắn thành các component được cô lập tốt và có khả năng phát triển độc lập. Những component ấy sau đó có thể được phân cho các đội làm việc độc lập với nhau.

Deployment — Triển khai

The architecture also plays a huge role in determining the ease with which the system is deployed. The goal is “immediate deployment.” A good architecture does not rely on dozens of little configuration scripts and property file tweaks. It does not require manual creation of directories or files that must be arranged just so. A good architecture helps the system to be immediately deployable after build.

Kiến trúc cũng đóng vai trò to lớn trong việc quyết định hệ thống dễ triển khai đến đâu. Mục tiêu là “triển khai tức thì”. Một kiến trúc tốt không dựa vào hàng tá script cấu hình vụn vặt và những lần chỉnh tay file property. Nó không đòi hỏi phải tạo thủ công các thư mục hay tập tin rồi xếp đặt sao cho vừa vặn. Một kiến trúc tốt giúp hệ thống triển khai được ngay sau khi build.

Again, this is achieved through the proper partitioning and isolation of the components of the system, including those master components that tie the whole system together and ensure that each **component** is properly started, integrated, and supervised.

Một lần nữa, điều này đạt được nhờ phân hoạch và cô lập đúng đắn các component của hệ thống, kể cả những component chủ chốt gắn kết cả hệ thống lại với nhau và bảo đảm mỗi component được khởi động, tích hợp và giám sát đúng cách.

Leaving Options Open — Để ngỏ các lựa chọn

A good architecture balances all of these concerns with a component structure that mutually satisfies them all. Sounds easy, right? Well, it's easy for me to write that.

Một kiến trúc tốt cân bằng tất cả những mối bận tâm này bằng một cấu trúc component thỏa mãn đồng thời tất cả. Nghe dễ nhỉ? Ừ thì, viết ra câu đó thì dễ với tôi thật.

The reality is that achieving this balance is pretty hard. The problem is that most of the time we don't know what all the use cases are, nor do we know the operational constraints, the team structure, or the deployment requirements. Worse, even if we did know them, they will inevitably change as the system moves through its life cycle. In short, the goals we must meet are indistinct and inconstant. Welcome to the real world.

Thực tế là đạt được thể cân bằng ấy khá khó. Vấn đề nằm ở chỗ phần lớn thời gian ta không biết hết các use case là gì, cũng chẳng biết các ràng buộc vận hành, cơ cấu đội ngũ hay yêu cầu triển khai. Tệ hơn, cho dù có biết đi nữa, chúng chắc chắn vẫn sẽ đổi khi hệ thống đi qua vòng đời của nó. Nói ngắn gọn, những mục tiêu ta phải đạt vừa mờ mịt vừa hay đổi. Chào mừng đến với thế giới thực.

But all is not lost: Some principles of architecture are relatively inexpensive to implement and can help balance those concerns, even when you don't have a clear picture of the targets you have to hit. Those principles help us partition our systems into well-isolated components that allow us to leave as many options open as possible, for as long as possible.

Nhưng chưa mất tất cả: có một số nguyên tắc kiến trúc tương đối rẻ để áp dụng mà vẫn giúp cân bằng những mối bận tâm ấy, ngay cả khi bạn chưa hình dung rõ những cái đích phải nhắm tới. Những nguyên tắc đó giúp ta phân hoạch hệ thống thành các component được cô lập tốt, cho phép ta để ngỏ càng nhiều lựa chọn càng tốt, và giữ chúng ngỏ càng lâu càng tốt.

A good architecture makes the system easy to change, in all the ways that it must change, by leaving options open.

Một kiến trúc tốt làm cho hệ thống dễ thay đổi, theo mọi hướng mà nó buộc phải thay đổi, bằng cách để ngỏ các lựa chọn.

Decoupling Layers — Tách rời các tầng

Consider the use cases. The architect wants the structure of the system to support all the necessary use cases, but does not know what all those use cases are. However, the architect does know the basic intent of the system. It's a shopping cart system, or it's a bill of materials system, or it's an order processing system. So the architect can employ the Single Responsibility Principle and the Common Closure Principle to separate those things that change for different reasons, and to collect those things that change for the same reasons—given the context of the intent of the system.

Hãy xét các use case. Kiến trúc sư muốn cấu trúc hệ thống hỗ trợ mọi use case cần thiết, nhưng lại không biết hết những use case đó là gì. Tuy vậy, kiến trúc sư biết ý đồ cơ bản của hệ thống. Đó là hệ thống giỏ hàng, hoặc hệ thống danh mục vật tư, hoặc hệ thống xử lý đơn hàng. Vậy nên kiến trúc sư có thể vận dụng Single Responsibility Principle và Common Closure Principle để tách những thứ thay đổi vì những lý do khác nhau, và gom những thứ thay đổi vì cùng một lý do — trong bối cảnh ý đồ của hệ thống.

What changes for different reasons? There are some obvious things. User interfaces change for reasons that have nothing to do with business rules. Use cases have elements of both. Clearly, then, a good architect will want to separate the UI portions of a use case from the **business rule** portions in such a way that they can be changed independently of each other, while keeping those use cases visible and clear.

Cái gì thay đổi vì những lý do khác nhau? Có vài thứ khá hiển nhiên. Giao diện người dùng thay đổi vì những lý do chẳng liên quan gì tới quy tắc nghiệp vụ (business rule). Use case thì có cả hai thành phần. Rõ ràng, một kiến trúc sư giỏi sẽ muốn tách phần UI của một use case khỏi phần quy tắc nghiệp vụ, sao cho chúng có thể thay đổi độc lập với nhau, mà vẫn giữ các use case hiện rõ và mạch lạc.

Business rules themselves may be closely tied to the application, or they may be more general. For example, the validation of input fields is a business rule that is closely tied to the application itself. In contrast, the calculation of interest on an account and the counting of inventory are business rules that are more closely associated with the domain. These two different kinds of rules will change at different rates, and for different reasons—so they should be separated so that they can be independently changed.

Bản thân các quy tắc nghiệp vụ có thể gắn chặt với ứng dụng, hoặc có thể mang tính tổng quát hơn. Chẳng hạn, việc kiểm tra tính hợp lệ của các trường nhập liệu là một quy tắc nghiệp vụ gắn chặt với chính ứng dụng. Ngược lại, việc tính lãi cho một tài khoản hay việc đếm tồn kho là những quy tắc nghiệp vụ gắn bó mật thiết hơn với miền nghiệp vụ. Hai loại quy tắc khác nhau này sẽ thay đổi với nhịp độ khác nhau và vì những lý do khác nhau—nên chúng cần được tách ra để có thể thay đổi độc lập.

The database, the query language, and even the schema are technical details that have nothing to do with the business rules or the UI. They will change at rates, and for reasons, that are independent of other aspects of the system. Consequently, the architecture should separate them from the rest of the system so that they can be independently changed.

Cơ sở dữ liệu, ngôn ngữ truy vấn, và thậm chí cả schema đều là những chi tiết (detail) kỹ thuật chẳng liên quan gì đến quy tắc nghiệp vụ hay UI. Chúng sẽ thay đổi với nhịp độ và vì những lý do độc lập với các khía cạnh khác của hệ thống. Do đó, kiến trúc nên tách chúng khỏi phần còn lại của hệ thống để chúng có thể thay đổi độc lập.

Thus we find the system divided into decoupled horizontal layers—the UI, application-specific business rules, application-independent business rules, and the database, just to mention a few.

Thế là ta thấy hệ thống được chia thành các tầng ngang tách rời (decoupling) — UI, quy tắc nghiệp vụ riêng của ứng dụng, quy tắc nghiệp vụ độc lập với ứng dụng, và cơ sở dữ liệu, ấy là mới kể vài cái.

Decoupling Use Cases — Tách rời các use case

What else changes for different reasons? The use cases themselves! The use case for adding an order to an order entry system almost certainly will change at a different rate, and for different reasons, than the use case that deletes an order from the system. Use cases are a very natural way to divide the system.

Còn cái gì thay đổi vì những lý do khác nhau nữa? Chính bản thân các use case! Use case thêm một đơn hàng vào hệ thống nhập đơn gần như chắc chắn sẽ thay đổi với nhịp độ khác, và vì lý do khác, so với use case xóa một đơn hàng khỏi hệ thống. Use case là một cách rất tự nhiên để chia hệ thống.

At the same time, use cases are narrow vertical slices that cut through the horizontal layers of the system. Each use case uses some UI, some application-specific business rules, some application-independent business rules, and some database functionality. Thus, as we are dividing the system into horizontal layers, we are also dividing the system into thin vertical use cases that cut through those layers.

Đồng thời, use case là những lát dọc hẹp cắt xuyên qua các tầng ngang của hệ thống. Mỗi use case dùng một chút UI, một chút quy tắc nghiệp vụ riêng của ứng dụng, một chút quy tắc nghiệp vụ độc lập với ứng dụng, và một chút chức năng cơ sở dữ liệu. Vậy nên, trong khi chia hệ thống thành các tầng ngang, ta cũng đồng thời chia hệ thống thành những lát dọc mỏng là các use case cắt xuyên qua các tầng ấy.

To achieve this **decoupling**, we separate the UI of the add-order use case from the UI of the delete-order use case. We do the same with the business rules, and with the database. We keep the use cases separate down the vertical height of the system.

Để đạt được sự tách rời này, ta tách UI của use case thêm-đơn-hàng khỏi UI của use case xóa-đơn-hàng. Ta cũng làm y như vậy với các quy tắc nghiệp vụ và với cơ sở dữ liệu. Ta giữ các use case tách biệt suốt chiều dọc của hệ thống.

You can see the pattern here. If you decouple the elements of the system that change for different reasons, then you can continue to add new use cases without interfering with old ones. If you also group the UI and database in support of those use cases, so that each use case uses a different aspect of the UI and database, then adding new use cases will be unlikely to affect older ones.

Bạn thấy được khuôn mẫu ở đây rồi đấy. Nếu bạn tách rời những phần tử của hệ thống thay đổi vì những lý do khác nhau, thì bạn có thể liên tục thêm use case mới mà không đụng chạm tới use case cũ. Nếu bạn còn nhóm cả UI và cơ sở dữ liệu theo các use case đó, sao cho mỗi use case dùng một mảng khác nhau của UI và cơ sở dữ liệu, thì việc thêm use case mới sẽ khó lòng ảnh hưởng tới những use case cũ.

Decoupling Mode — Chế độ tách rời

Now think of what all that decoupling means for the second bullet: operations. If the different aspects of the use cases are separated, then those that must run at a high throughput are likely already separated from those that must run at a low throughput. If the UI and the database have been separated from the business rules, then they can run in different servers. Those that require higher bandwidth can be replicated in many servers.

Giờ hãy nghĩ xem toàn bộ sự tách rời ấy có ý nghĩa gì với gạch đầu dòng thứ hai: vận hành. Nếu các khía cạnh khác nhau của use case đã được tách ra, thì những phần phải chạy với thông lượng cao nhiều khả năng đã tách sẵn khỏi những phần chỉ chạy với thông lượng thấp. Nếu UI và cơ sở dữ liệu đã được tách khỏi quy tắc nghiệp vụ, thì chúng có thể chạy trên các máy chủ khác nhau. Những phần cần băng thông lớn hơn có thể được nhân bản ra nhiều máy chủ.

In short, the decoupling that we did for the sake of the use cases also helps with operations. However, to take advantage of the operational benefit, the decoupling must have the appropriate mode. To run in separate servers, the separated components cannot depend on being together in the same address space of a processor. They must be independent services, which communicate over a network of some kind.

Ngắn gọn, sự tách rời mà ta làm vì các use case cũng giúp ích cho vận hành. Tuy nhiên, muốn hưởng lợi ích vận hành ấy, sự tách rời phải ở đúng chế độ. Để chạy trên các máy

chủ riêng, các component đã tách không thể phụ thuộc vào việc nằm chung trong cùng một không gian địa chỉ của một bộ xử lý. Chúng phải là những dịch vụ độc lập, giao tiếp qua một mạng nào đó.

Many architects call such components “services” or “micro-services,” depending upon some vague notion of line count. Indeed, an architecture based on services is often called a **service-oriented** architecture.

Nhiều kiến trúc sư gọi những component như vậy là “service” hay “micro-service”, tùy theo một khái niệm mơ hồ nào đó về số dòng mã. Quả thật, một kiến trúc dựa trên dịch vụ thường được gọi là service-oriented architecture.

If that nomenclature set off some alarm bells in your mind, don’t worry. I’m not going to tell you that SoA is the best possible architecture, or that micro- services are the wave of the future. The point being made here is that sometimes we have to separate our components all the way to the service level.

Nếu cách gọi tên đó vừa giống lên vài hồi chuông báo động trong đầu bạn, đừng lo. Tôi không định bảo bạn rằng SoA là kiến trúc tốt nhất có thể, hay micro-service là làn sóng của tương lai. Điều tôi muốn nói ở đây là đôi khi ta phải tách các component của mình ra tới tận mức dịch vụ.

Remember, a good architecture leaves options open. The **decoupling mode** is one of those options.

Nhớ nhé, một kiến trúc tốt để ngỏ các lựa chọn. Chế độ tách rời là một trong những lựa chọn đó.

Before we explore that topic further, let’s look to the other two bullets.

Trước khi đào sâu chủ đề ấy, hãy nhìn sang hai gạch đầu dòng còn lại.

Independent Develop-ability — Khả năng phát triển độc lập

The third bullet was development. Clearly when components are strongly decoupled, the interference between teams is mitigated. If the business rules

Gạch đầu dòng thứ ba là phát triển. Rõ ràng khi các component được tách rời mạnh mẽ, sự giẫm chân nhau giữa các đội sẽ giảm bớt. Nếu các quy tắc nghiệp vụ

don’t know about the UI, then a team that focuses on the UI cannot much affect a team that focuses on the business rules. If the use cases themselves are decoupled from one another, then a team that focuses on the addOrder use case is not likely to interfere with a team that focuses on the deleteOrder use case.

không biết gì về UI, thì một đội tập trung vào UI sẽ không mấy ảnh hưởng tới đội tập trung vào quy tắc nghiệp vụ. Nếu bản thân các use case cũng tách rời nhau, thì đội tập trung vào use case addOrder sẽ khó mà giẫm chân đội tập trung vào use case deleteOrder.

So long as the layers and use cases are decoupled, the architecture of the system will support the organization of the teams, irrespective of whether they are organized as feature teams, component teams, layer teams, or some other variation.

Chừng nào các tầng và các use case còn tách rời, kiến trúc của hệ thống sẽ hỗ trợ được cách tổ chức đội ngũ, bất kể họ được tổ chức thành đội theo tính năng, đội theo component, đội theo tầng, hay một biến thể nào khác.

Independent Deployability — Khả năng triển khai độc lập

The decoupling of the use cases and layers also affords a high degree of flexibility in deployment. Indeed, if the decoupling is done well, then it should be possible to hot-swap layers and use cases in running systems. Adding a new use case could be as simple as adding a few new jar files or services to the system while leaving the rest alone.

Việc tách rời các use case và các tầng cũng đem lại độ linh hoạt cao trong triển khai. Thật vậy, nếu tách rời tốt, ta hoàn toàn có thể thay nóng các tầng và use case ngay trên hệ thống đang chạy. Thêm một use case mới có thể đơn giản chỉ là thêm vài file jar hoặc vài dịch vụ mới vào hệ thống, còn phần còn lại giữ nguyên.

Duplication — Trùng lặp

Architects often fall into a trap—a trap that hinges on their fear of duplication.

Kiến trúc sư thường sa vào một cái bẫy — cái bẫy bắt nguồn từ nỗi sợ trùng lặp.

Duplication is generally a bad thing in software. We don't like duplicated code. When code is truly duplicated, we are honor-bound as professionals to reduce and eliminate it.

Trùng lặp nhìn chung là điều tệ hại trong phần mềm. Chúng ta không thích mã trùng lặp. Khi mã thực sự trùng lặp, chúng ta có bốn phận nghề nghiệp phải giảm bớt và loại bỏ nó.

But there are different kinds of duplication. There is true duplication, in which every change to one instance necessitates the same change to every duplicate of that instance. Then there is false or accidental duplication. If two apparently duplicated sections of code evolve along different paths—if they change at different rates, and for different reasons—then they are not

Nhưng có nhiều kiểu trùng lặp khác nhau. Có trùng lặp thật sự, khi mọi thay đổi ở một bản đều bắt buộc phải làm y hệt ở mọi bản sao của nó. Rồi có trùng lặp giả, hay trùng lặp tình cờ. Nếu hai đoạn mã trông có vẻ trùng nhau lại tiến hóa theo những con đường khác nhau — nếu chúng thay đổi với nhịp độ khác nhau và vì những lý do khác nhau — thì chúng không phải là

true duplicates. Return to them in a few years, and you'll find that they are very different from each other.

trùng lặp thật sự. Vài năm sau quay lại nhìn, bạn sẽ thấy chúng khác nhau một trời một vực.

Now imagine two use cases that have very similar screen structures. The architects will likely be strongly tempted to share the code for that structure. But should they? Is that true duplication? Or it is accidental?

Giờ hãy hình dung hai use case có cấu trúc màn hình rất giống nhau. Kiến trúc sư nhiều khả năng sẽ bị cám dỗ mạnh mẽ muốn dùng chung mã cho cấu trúc đó. Nhưng có nên không? Đó có phải trùng lặp thật sự không? Hay chỉ là tình cờ?

Most likely it is accidental. As time goes by, the odds are that those two screens will diverge and eventually look very different. For this reason, care must be taken to avoid unifying them. Otherwise, separating them later will be a challenge.

Nhiều khả năng nhất là tình cờ. Thời gian trôi đi, khả năng cao là hai màn hình ấy sẽ phân kỳ và cuối cùng trông rất khác nhau. Vì lẽ đó, phải cẩn thận đừng hợp nhất chúng. Nếu không, tách chúng ra về sau sẽ là một thử thách.

When you are vertically separating use cases from one another, you will run into this issue, and your temptation will be to couple the use cases because they have similar screen structures, or similar algorithms, or similar database queries and/or schemas. Be careful. Resist the temptation to commit the sin of knee-jerk elimination of duplication. Make sure the duplication is real.

Khi bạn tách các use case khỏi nhau theo chiều dọc, bạn sẽ đụng phải chuyện này, và bạn sẽ bị cám dỗ muốn gắn kết các use case lại vì chúng có cấu trúc màn hình giống nhau, hay thuật toán giống nhau, hay truy vấn và/hoặc schema cơ sở dữ liệu giống nhau. Cần thận đấy. Hãy cưỡng lại cám dỗ phạm phải cái tội xóa trùng lặp theo phản xạ. Hãy chắc chắn rằng trùng lặp đó là thật.

By the same token, when you are separating layers horizontally, you might notice that the data structure of a particular database record is very similar to the data structure of a particular screen view. You may be tempted to simply pass the database record up to the UI, rather than to create a **view model** that looks the same and copy the elements across. Be careful: This duplication is almost certainly accidental. Creating the separate view model is not a lot of **effort**, and it will help you keep the layers properly decoupled.

Cũng vậy, khi tách các tầng theo chiều ngang, bạn có thể nhận thấy cấu trúc dữ liệu của một bản ghi cơ sở dữ liệu nào đó rất giống cấu trúc dữ liệu của một khung nhìn màn hình nào đó. Bạn có thể bị cám dỗ chỉ việc đẩy thẳng bản ghi cơ sở dữ liệu lên UI, thay vì tạo một view model trông y hệt rồi chép từng phần tử sang. Cần thận: sự trùng lặp này gần như chắc chắn là tình cờ. Tạo một view model riêng chẳng tốn bao nhiêu công sức, mà lại giúp bạn giữ cho các tầng tách rời đúng cách.

Decoupling Modes (Again) — Chế độ tách rời (lần nữa)

Back to modes. There are many ways to decouple layers and use cases. They can be decoupled at the source code level, at the binary code (deployment) level, and at the execution unit (service) level.

Quay lại chuyện chế độ. Có nhiều cách để tách rời các tầng và các use case. Chúng có thể được tách ở mức mã nguồn, ở mức mã nhị phân (triển khai), và ở mức đơn vị thực thi (dịch vụ).

- Source level. We can control the dependencies between source code
 - Mức mã nguồn. Ta có thể kiểm soát các phụ thuộc giữa những module mã

modules so that changes to one module do not force changes or recompilation of others (e.g., Ruby Gems).

nguồn sao cho thay đổi ở một module không buộc các module khác phải thay đổi hay biên dịch lại (ví dụ: Ruby Gems).

In this decoupling mode the components all execute in the same address space, and communicate with each other using simple function calls. There is a single executable loaded into computer memory. People often call this a monolithic structure.

Ở chế độ tách rời này, tất cả các component đều chạy trong cùng một không gian địa chỉ và giao tiếp với nhau bằng những lời gọi hàm đơn giản. Chỉ có một tệp thực thi duy nhất được nạp vào bộ nhớ máy tính. Người ta thường gọi đây là cấu trúc monolithic.

- Deployment level. We can control the dependencies between deployable
 - Mức triển khai. Ta có thể kiểm soát các phụ thuộc giữa những đơn vị

units such as jar files, DLLs, or shared libraries, so that changes to the source code in one module do not force others to be rebuilt and redeployed. Many of the components may still live in the same address space, and communicate through function calls. Other components may live in other processes in the same processor, and communicate through interprocess communications, sockets, or shared memory. The important thing here is that the decoupled components are partitioned into independently deployable units such as jar files, Gem files, or DLLs.

triển khai như file jar, DLL hay shared library, sao cho thay đổi mã nguồn ở một module không buộc các module khác phải build lại và triển khai lại. Nhiều component vẫn có thể sống chung trong cùng một không gian địa chỉ và giao tiếp qua lời gọi hàm. Các component khác có thể sống trong những tiến trình khác trên cùng bộ xử lý và giao tiếp qua cơ chế liên tiến trình, socket hoặc bộ nhớ chia sẻ. Điều quan trọng ở đây là các component đã tách rời được phân hoạch thành những đơn vị triển khai độc lập như file jar, file Gem hay DLL.

- Service level . We can reduce the dependencies down to the level of data
 - Mức dịch vụ. Ta có thể giảm các phụ thuộc xuống tận mức cấu trúc dữ

structures, and communicate solely through network packets such that every execution unit is entirely independent of source and binary changes to others (e.g., services or micro-services).

liệu, và giao tiếp thuần túy qua các gói tin mạng, sao cho mọi đơn vị thực thi đều hoàn toàn độc lập với những thay đổi mã nguồn và mã nhị phân của các đơn vị khác (ví dụ: service hay micro-service).

What is the best mode to use?

Chế độ nào là tốt nhất để dùng?

The answer is that it's hard to know which mode is best during the early phases of a project. Indeed, as the project matures, the optimal mode may change.

Câu trả lời là rất khó biết chế độ nào tốt nhất trong những giai đoạn đầu của một dự án. Thật vậy, khi dự án trưởng thành, chế độ tối ưu có thể đổi khác.

For example, it's not difficult to imagine that a system that runs comfortably on one server right now might grow to the point where some of its components ought to run on separate servers. While the system runs on a single server, the source-level decoupling might be sufficient. Later, however, it might require decoupling into deployable units, or even services.

Chẳng hạn, không khó hình dung một hệ thống hiện đang chạy thông dong trên một máy chủ có thể lớn tới mức vài component của nó phải chạy trên những máy chủ riêng. Chừng nào hệ thống còn chạy trên một máy chủ duy nhất, tách rời ở mức mã nguồn có lẽ là đủ. Nhưng về sau, có thể nó sẽ đòi hỏi tách thành các đơn vị triển khai, hoặc thậm chí thành dịch vụ.

One solution (which seems to be popular at the moment) is to simply decouple at the service level by default. A problem with this approach is that it is expensive and encourages coarse-grained decoupling. No matter how “micro” the micro-services get, the decoupling is not likely to be fine-grained enough.

Một giải pháp (có vẻ đang thịnh hành lúc này) là cứ mặc định tách rời ở mức dịch vụ. Vấn đề của cách tiếp cận ấy là nó đắt đỏ và cổ vũ lối tách rời thô. Dù các micro-service có “micro” đến mấy, sự tách rời cũng khó mà đủ mịn.

Another problem with service-level decoupling is that it is expensive, both in development time and in system resources. Dealing with service boundaries where none are needed is a waste of effort,

memory, and cycles. And, yes, I know that the last two are cheap—but the first is not.

Một vấn đề nữa của tách rời ở mức dịch vụ là nó tốn kém, cả về thời gian phát triển lẫn tài nguyên hệ thống. Loay hoay với những ranh giới (boundary) dịch vụ ở chỗ chẳng cần đến chúng là phí phạm công sức, bộ nhớ và chu kỳ CPU. Và vâng, tôi biết hai thứ sau thì rẻ — nhưng thứ đầu tiên thì không.

My preference is to push the decoupling to the point where a service could be formed. should it become necessary; but then to leave the components in the same address space as long as possible. This leaves the option for a service open.

Tôi thích đẩy sự tách rời tới điểm mà một dịch vụ có thể hình thành được, nếu điều đó trở nên cần thiết; nhưng rồi vẫn để các component nằm chung một không gian địa chỉ càng lâu càng tốt. Làm vậy là để ngỏ lựa chọn dịch vụ.

With this approach, initially the components are separated at the source code level. That may be good enough for the duration of the project's lifetime. If, however, deployment or development issues arise, driving some of the decoupling to a deployment level may be sufficient—at least for a while.

Với cách tiếp cận này, ban đầu các component được tách ở mức mã nguồn. Chừng đó có thể là đủ tốt cho suốt vòng đời dự án. Tuy nhiên, nếu nảy sinh vấn đề về triển khai hay phát triển, thì đẩy một phần sự tách rời lên mức triển khai có lẽ đã đủ — ít nhất là trong một thời gian.

As the development, deployment, and operational issues increase, I carefully choose which deployable units to turn into services, and gradually shift the system in that direction.

Khi các vấn đề về phát triển, triển khai và vận hành ngày một nhiều lên, tôi cẩn thận chọn xem đơn vị triển khai nào sẽ biến thành dịch vụ, rồi dịch chuyển hệ thống dần dần theo hướng đó.

Over time, the operational needs of the system may decline. What once required decoupling at the service level may now require only deployment- level or even source-level decoupling.

Theo thời gian, nhu cầu vận hành của hệ thống có thể giảm xuống. Cái từng đòi hỏi tách rời ở mức dịch vụ giờ có thể chỉ cần tách ở mức triển khai, hoặc thậm chí ở mức mã nguồn.

A good architecture will allow a system to be born as a monolith, deployed in a single file, but then to grow into a set of independently deployable units, and then all the way to independent services and/or micro-services. Later, as things change, it should allow for reversing that progression and sliding all the way back down into a monolith.

Một kiến trúc tốt sẽ cho phép một hệ thống sinh ra như một khối monolith, triển khai trong một tệp duy nhất, rồi lớn lên thành một tập các đơn vị triển khai độc lập, rồi đi hết đường thành các dịch vụ độc lập và/hoặc micro-service. Về sau, khi mọi thứ đổi thay, nó phải cho phép đảo ngược tiến trình ấy và trượt ngược trở lại thành một khối monolith.

A good architecture protects the majority of the source code from those changes. It leaves the decoupling mode open as an option so that large deployments can use one mode, whereas small deployments can use another.

Một kiến trúc tốt bảo vệ phần lớn mã nguồn khỏi những thay đổi đó. Nó để ngỏ chế độ tách rời như một lựa chọn, sao cho những bản triển khai lớn có thể dùng chế độ này, còn những bản triển khai nhỏ dùng chế độ khác.

Conclusion — Kết luận

Yes, this is tricky. And I'm not saying that the change of decoupling modes should be a trivial configuration option (though sometimes that is appropriate). What I'm saying is that the decoupling mode of a system is one of those things that is likely to change with time, and a good architect foresees and appropriately facilitates those changes.

Đúng, chuyện này rắc rối. Và tôi không nói rằng việc đổi chế độ tách rời phải là một tùy chọn cấu hình tầm thường (dù đôi khi như vậy là thích hợp). Điều tôi muốn nói là chế độ tách rời của một hệ thống là một trong những thứ nhiều khả năng sẽ thay đổi theo thời gian, và một kiến trúc sư giỏi phải lường trước và tạo điều kiện thích đáng cho những thay đổi ấy.

Boundaries: Drawing Lines — Ranh giới: Vạch những đường kẻ

Software **architecture** is the art of drawing lines that I call boundaries . Those boundaries separate software elements from one another, and restrict those on one side from knowing about those on the other. Some of those lines are drawn very early in a project's life—even before any code is written. Others are drawn much later. Those that are drawn early are drawn for the purposes of deferring decisions for as long as possible, and of keeping those decisions from polluting the core business logic.

Kiến trúc phần mềm là nghệ thuật vạch ra những đường kẻ mà tôi gọi là ranh giới (boundary). Những ranh giới đó tách các thành phần phần mềm ra khỏi nhau, và ngăn không cho bên này biết gì về bên kia. Một số đường kẻ được vạch rất sớm trong đời một dự án — thậm chí trước khi viết dòng mã nào. Số khác được vạch muộn hơn nhiều. Những đường vạch sớm là để trì hoãn các quyết định càng lâu càng tốt, và để giữ cho những quyết định ấy không làm ô nhiễm phần logic nghiệp vụ cốt lõi.

Recall that the goal of an architect is to minimize the **human resources** required to build and maintain the required system. What it is that saps this kind of people-power? Coupling —and especially coupling to premature decisions.

Hãy nhớ lại rằng mục tiêu của một kiến trúc sư là giảm thiểu nhân lực (human resources) cần thiết để xây dựng và bảo trì hệ thống được yêu cầu. Vậy cái gì bòn rút thứ sức người ấy? Sự kết dính — và đặc biệt là kết dính vào những quyết định vội vàng.

Which kinds of decisions are premature? Decisions that have nothing to do with the business requirements—the use cases—of the system. These include decisions about frameworks, databases, web servers, utility libraries, dependency injection, and the like. A good system architecture is one in which decisions like these are rendered ancillary and deferrable. A good system architecture does not depend on those decisions. A good system architecture allows those decisions to be made at the latest possible moment, without significant impact.

Những quyết định nào là vội vàng? Là những quyết định chẳng liên quan gì đến yêu cầu nghiệp vụ — tức các use case — của hệ thống. Chúng bao gồm quyết định về framework, cơ sở dữ liệu, web server, thư viện tiện ích, dependency injection, và những thứ tương tự. Một kiến trúc hệ thống tốt là kiến trúc biến những quyết định như vậy thành thứ phụ trợ và có thể hoãn lại. Một kiến trúc hệ thống tốt không phụ thuộc vào những quyết định đó. Một kiến trúc hệ thống tốt cho phép đưa ra những quyết định đó ở thời điểm muộn nhất có thể, mà không gây tác động đáng kể.

A Couple of Sad Stories — Vài câu chuyện buồn

Here's the sad story of company P, which serves as a warning about making premature decisions. In the 1980s the founders of P wrote a simple monolithic desktop application. They enjoyed a great deal of success and grew the product through the 1990s into a popular and successful desktop GUI application.

Đây là câu chuyện buồn của công ty P, một lời cảnh báo về việc ra quyết định vội vàng. Thập niên 1980, những người sáng lập P viết một ứng dụng desktop nguyên khối đơn giản. Họ thành công lớn và nuôi lớn sản phẩm ấy suốt thập niên 1990 thành một ứng dụng GUI desktop phổ biến và ăn khách.

But then, in the late 1990s, the web emerged as a force. Suddenly everybody had to have a web solution, and P was no exception. P's customers clamored for a version of the product on the web. To meet this demand, the company hired a bunch of hotshot twenty-something Java programmers and embarked upon a project to webify their product.

Nhưng rồi, cuối thập niên 1990, web nổi lên thành một thế lực. Đột nhiên ai cũng phải có giải pháp web, và P không ngoại lệ. Khách hàng của P nằng nặc đòi một phiên bản sản phẩm chạy trên web. Để đáp ứng nhu cầu đó, công ty tuyển một đám lập trình viên Java tuổi đôi mươi đầy tự tin và khởi động dự án đưa sản phẩm lên web.

The Java guys had dreams of server farms dancing in their heads, so they adopted a rich three-tiered “architecture” 1 that they could distribute through such farms. There would be servers for the GUI, servers for the middleware, and servers for the database. Of course.

Mấy anh chàng Java mơ màng những trang trại server nhảy múa trong đầu, nên họ chọn một “kiến trúc” ba tầng đồ sộ 1 có thể phân tán qua các trang trại ấy. Sẽ có server cho GUI, server cho middleware, và server cho cơ sở dữ liệu. Tất nhiên rồi.

The programmers decided, very early on, that all domain objects would have three instantiations: one in the GUI tier, one in the middleware tier, and one in the database tier. Since these instantiations lived on different machines, a rich system of interprocessor and inter-tier communications was set up. Method invocations between tiers were converted to objects, serialized, and marshaled across the wire.

Ngay từ rất sớm, các lập trình viên quyết định rằng mọi đối tượng nghiệp vụ sẽ có ba lần khởi tạo: một ở tầng GUI, một ở tầng middleware, và một ở tầng cơ sở dữ liệu. Vì các thể hiện này nằm trên những máy khác nhau, người ta dựng lên cả một hệ thống đồ sộ để liên lạc giữa các bộ xử lý và giữa các tầng. Lời gọi phương thức giữa các tầng được chuyển thành đối tượng, tuần tự hóa, rồi marshal qua đường truyền.

Now imagine what it took to implement a simple feature like adding a new field to an existing record. That field had to be added to the classes in all three tiers, and to several of the inter-tier messages. Since data traveled in both directions, four message protocols needed to be designed. Each protocol had a sending and receiving side, so eight protocol handlers were required. Three executables had to be built, each with three updated business objects, four new messages, and eight new handlers.

Giờ hãy tưởng tượng phải làm gì để hiện thực một tính năng đơn giản như thêm một trường mới vào một bản ghi có sẵn. Trường đó phải được thêm vào các class ở cả ba tầng, và vào vài thông điệp liên tầng. Vì dữ liệu đi theo cả hai chiều, phải thiết kế bốn giao thức thông điệp. Mỗi giao thức có một phía gửi và một phía nhận, nên cần tám bộ xử lý giao thức. Phải build ba chương trình chạy được, mỗi cái với ba đối tượng nghiệp vụ đã cập nhật, bốn thông điệp mới, và tám handler mới.

And think of what those executables had to do to implement the simplest of features. Think of all the object instantiations, all the serializations, all the marshaling and de-marshaling, all the building and parsing of messages, all the socket communications, timeout managers, retry scenarios, and all the other extra stuff that you have to do just to get one simple thing done.

Và hãy nghĩ xem những chương trình ấy phải làm gì để hiện thực tính năng đơn giản nhất. Nghĩ tới tất cả những lần khởi tạo đối tượng, tất cả những lần tuần tự hóa, tất cả những lần marshal và de-marshal, tất cả việc dựng và phân tích thông điệp, tất cả việc giao tiếp qua socket, các bộ quản lý timeout, các kịch bản thử lại, và tất cả những thứ dôi ra khác mà bạn phải làm chỉ để hoàn thành một việc đơn giản.

Of course, during development the programmers did not have a server farm. Indeed, they simply ran all three executables in three different processes on a single machine. They developed this way for several years. But they were convinced that their architecture was right. And so, even though they were executing in a single machine, they continued all the object instantiations, all the serializations, all the marshaling and de-marshaling, all the building and parsing of messages, all the socket communications, and all the extra stuff in a single machine.

Tất nhiên, trong lúc phát triển thì các lập trình viên chẳng có trang trại server nào. Thực tế, họ chỉ chạy cả ba chương trình trong ba tiến trình khác nhau trên một máy duy nhất. Họ phát triển kiểu đó suốt vài năm. Nhưng họ tin chắc kiến trúc của mình là đúng. Thế nên, dù chạy trên một máy duy nhất, họ vẫn tiếp tục tất cả những lần khởi tạo đối tượng, tất cả những lần tuần tự hóa, tất cả những lần marshal và de-marshal, tất cả việc dựng và phân tích thông điệp, tất cả việc giao tiếp qua socket, và tất cả những thứ dôi ra — trên một máy duy nhất.

- 1. The word “architecture” appears in quotes here because three-tier is not an architecture; it’s a topology.
 - 1. Từ “kiến trúc” được để trong ngoặc kép ở đây vì ba tầng không phải là một kiến trúc; nó là một topology.

It’s exactly the kind of decision that a good architecture strives to defer.

Đó chính xác là loại quyết định mà một kiến trúc tốt cố gắng trì hoãn.

The irony is that company P never sold a system that required a server farm. Every system they ever deployed was a single server. And in that single server all three executables continued all the object instantiations, all the serializations, all the marshaling and de-marshaling, all the building and parsing of messages, all the socket communications, and all the extra stuff, in anticipation of a server farm that never existed, and never would.

Trớ trêu thay, công ty P chưa bao giờ bán được một hệ thống nào cần đến trang trại server. Mọi hệ thống họ từng triển khai đều nằm trên một server duy nhất. Và trong cái server duy nhất ấy, cả ba chương trình vẫn tiếp tục tất cả những lần khởi tạo đối tượng, tất cả những lần tuần tự hóa, tất cả những lần marshal và de-marshal, tất cả việc dựng và phân tích thông điệp, tất cả việc giao tiếp qua socket, và tất cả những thứ dôi ra, để chờ đợi một trang trại server chưa từng tồn tại, và sẽ không bao giờ tồn tại.

The tragedy is that the architects, by making a premature decision, multiplied the development **effort** enormously.

Bi kịch là ở chỗ các kiến trúc sư, chỉ bằng một quyết định vội vàng, đã nhân công sức phát triển lên gấp bội.

The story of P is not isolated. I’ve seen it many times and in many places. Indeed, P is a superposition of all those places.

Câu chuyện của P không phải cá biệt. Tôi đã thấy nó nhiều lần, ở nhiều nơi. Thật ra, P là hình ảnh chồng ghép của tất cả những nơi ấy.

But there are worse fates than P.

Nhưng vẫn có những số phận còn tệ hơn P.

Consider W, a local business that manages fleets of company cars. They recently hired an “Architect” to get their rag-tag software effort under control. And, let me tell you, control was this guy’s middle name. He quickly realized that what this little operation needed was a full-blown, enterprise-scale, **service**- oriented “ARCHITECTURE.” He created a huge domain model of all the different “objects” in the business, designed a suite of services to manage these domain objects, and put all the developers on a path to Hell . As a simple example, suppose you wanted to add the name, address, and phone number of a contact person to a sales record. You had to go to the ServiceRegistry and ask for the service ID of the ContactService . Then you had to send a CreateContact message to the ContactService . Of course, this message had dozens of fields that all had to have valid data in them—data to which the programmer had no access, since all the programmer had was a name, address, and phone number. After faking the data, the programmer had to jam the ID of the newly created contact into the sales record and send the UpdateContact

Hãy xét W, một doanh nghiệp địa phương quản lý đội xe công ty. Gần đây họ thuê một “Kiến trúc sư” để chấn chỉnh mớ phần mềm lôm côm của mình. Và để tôi kể bạn nghe, “kiểm soát” là tên đệm của gã này. Gã nhanh chóng nhận ra rằng cái cơ sở kinh doanh bé tí này cần một “KIẾN TRÚC” hướng dịch vụ, quy mô doanh nghiệp, đầy đủ mọi thứ. Gã tạo ra một mô hình nghiệp vụ khổng lồ gồm tất cả các “đối tượng” khác nhau trong doanh nghiệp, thiết kế một bộ dịch vụ để quản lý các đối tượng nghiệp vụ ấy, và đẩy toàn bộ lập trình viên xuống Địa ngục. Lấy một ví dụ đơn giản: giả sử bạn muốn thêm tên, địa chỉ và số điện thoại của một người liên hệ vào một bản ghi bán hàng. Bạn phải đến ServiceRegistry và hỏi service ID của ContactService . Rồi bạn phải gửi thông điệp CreateContact tới ContactService . Tất nhiên, thông điệp này có hàng tá trường mà tất cả đều phải chứa dữ liệu hợp lệ — thứ dữ liệu mà lập trình viên chẳng có cách nào lấy được, vì tất cả những gì anh ta có chỉ là một cái tên, một địa chỉ và một số điện thoại. Sau khi chế ra dữ liệu giả, lập trình viên phải nhồi ID của contact vừa tạo vào bản ghi bán hàng rồi gửi UpdateContact

message to the SaleRecordService.

Of course, to test anything you had to fire up all the necessary services, one by one, and fire up the message bus, and the BPel server, and ... And then, there were the propagation delays as these messages bounced from service to service, and waited in queue after queue.

Tất nhiên, để kiểm thử bất cứ thứ gì, bạn phải khởi động tất cả các dịch vụ cần thiết, từng cái một, rồi khởi động message bus, rồi BPel server, rồi... Và rồi còn độ trễ lan truyền khi các thông điệp nảy từ dịch vụ này sang dịch vụ khác, và xếp hàng chờ hết hàng đợi này đến hàng đợi khác.

And then if you wanted to add a new feature—well, you can imagine the coupling between all those services, and the sheer volume of WSDLs that needed changing, and all the redeployments those changes necessitated ...

Và rồi nếu bạn muốn thêm một tính năng mới — chà, bạn có thể hình dung mức kết dính giữa tất cả các dịch vụ ấy, và cả núi WSDL phải sửa, cùng tất cả những lần triển khai lại mà các thay đổi đó kéo theo...

Hell starts to seem like a nice place by comparison.

So ra thì Địa ngục bắt đầu trông có vẻ là một nơi dễ chịu.

There's nothing intrinsically wrong with a software system that is structured around services. The error at W was the premature adoption and enforcement of a suite of tools that promised SoA—that is, the premature adoption of a massive suite of domain object services. The cost of those errors was sheer person-hours—person-hours in droves—flushed down the SoA vortex.

Bản thân việc dựng một hệ thống phần mềm quanh các dịch vụ chẳng có gì sai. Sai lầm ở W là việc áp dụng và cưỡng ép quá sớm một bộ công cụ hứa hẹn SoA — nghĩa là việc áp dụng quá sớm một bộ dịch vụ đối tượng nghiệp vụ khổng lồ. Cái giá của những sai lầm đó là thuần túy giờ công — giờ công hàng đồng — đổ tuột xuống cái xoáy nước SoA.

I could go on describing one architectural failure after another. But let's talk about an architectural success instead.

Tôi có thể kể mãi hết thất bại kiến trúc này đến thất bại kiến trúc khác. Nhưng thay vào đó, hãy nói về một thành công kiến trúc.

FitNesse — FitNesse

My Son, Micah, and I started work on FitNesse in 2001. The idea was to create a simple wiki that wrapped Ward Cunningham's FIT tool for writing acceptance tests.

Con trai tôi, Micah, và tôi bắt đầu làm FitNesse năm 2001. Ý tưởng là tạo một wiki đơn giản bọc quanh công cụ FIT của Ward Cunningham để viết acceptance test.

This was back in the days before Maven “solved” the jar file problem. I was adamant that anything we produced should not require people to download more than one jar file. I called this rule, “Download and Go.” This rule drove many of our decisions.

Đó là thời trước khi Maven “giải quyết” bài toán file jar. Tôi kiên quyết rằng bất cứ thứ gì chúng tôi làm ra cũng không được bắt người ta tải về nhiều hơn một file jar. Tôi gọi quy tắc này là “Download and Go.” Quy tắc ấy chi phối rất nhiều quyết định của chúng tôi.

One of the first decisions was to write our own web server, specific to the needs of FitNesse . This might sound absurd. Even in 2001 there were plenty of open source web servers that we could have used. Yet writing our own turned out to be a really good decision because a bare-bones web server is a very simple piece of software to write and it allowed us to postpone any web **framework** decision until much later. 2

Một trong những quyết định đầu tiên là tự viết web server riêng, đúng theo nhu cầu của FitNesse . Nghe có vẻ vô lý. Ngay cả năm 2001 cũng đã có đầy web server mã nguồn mở để dùng. Vậy mà tự viết lại hóa ra là một quyết định rất tốt, bởi một web server trần trụi là một mẫu phần mềm cực kỳ đơn giản để viết, và nó cho phép chúng tôi hoãn mọi quyết định về web framework tới tận rất lâu sau. 2

- 2. Many years later we were able to slip the Velocity framework into FitNesse .
 - 2. Nhiều năm sau, chúng tôi đã có thể luồn framework Velocity vào FitNesse .

Another early decision was to avoid thinking about a database. We had MySQL in the back of our minds, but we purposely delayed that decision by employing a **design** that made the decision irrelevant. That design was simply to put an **interface** between all data accesses and the data repository itself.

Một quyết định sớm khác là tránh nghĩ tới cơ sở dữ liệu. Chúng tôi có để MySQL trong đầu, nhưng cố ý trì hoãn quyết định đó bằng cách dùng một thiết kế khiến quyết định ấy trở

nên vô nghĩa. Thiết kế đó đơn giản là đặt một interface giữa mọi truy cập dữ liệu và bản thân kho dữ liệu.

We put the data access methods into an interface named WikiPage . Those methods provided all the functionality we needed to find, fetch, and save pages. Of course, we didn't implement those methods at first; we simply stubbed them out while we worked on features that didn't involve fetching and saving the data.

Chúng tôi đặt các phương thức truy cập dữ liệu vào một interface tên là WikiPage . Các phương thức đó cung cấp toàn bộ chức năng chúng tôi cần để tìm, lấy và lưu các trang. Tất nhiên, ban đầu chúng tôi không hiện thực chúng; chúng tôi chỉ để chúng rỗng trong khi làm những tính năng không dính đến việc lấy và lưu dữ liệu.

Indeed, for three months we simply worked on translating wiki text into HTML. This didn't require any kind of data storage, so we created a class named MockWikiPage that simply left the data access methods stubbed.

Quả thật, suốt ba tháng chúng tôi chỉ làm mỗi việc chuyển văn bản wiki thành HTML. Việc này không đòi hỏi bất kỳ dạng lưu trữ dữ liệu nào, nên chúng tôi tạo một class tên MockWikiPage chỉ đơn giản để trống các phương thức truy cập dữ liệu.

Eventually, those stubs became insufficient for the features we wanted to write. We needed real data access, not stubs. So we created a new derivative of WikiPage named InMemoryPage . This derivative implemented the data access method to manage a hash table of wiki pages, which we kept in RAM.

Cuối cùng, những phương thức rỗng (stub) ấy không còn đủ cho các tính năng chúng tôi muốn viết. Chúng tôi cần truy cập dữ liệu thật, chứ không phải stub. Thế là chúng tôi tạo một dẫn xuất mới của WikiPage tên là InMemoryPage . Dẫn xuất này hiện thực phương thức truy cập dữ liệu để quản lý một bảng băm các trang wiki, giữ trong RAM.

This allowed us to write feature after feature for a full year. In fact, we got the whole first version of the FitNesse program working this way. We could create pages, link to other pages, do all the fancy wiki formatting, and even run tests with FIT. What we couldn't do was save any of our work.

Nhờ vậy chúng tôi viết được hết tính năng này đến tính năng khác trong trọn một năm. Thực tế, chúng tôi đã cho cả phiên bản đầu tiên của chương trình FitNesse chạy được theo cách đó. Chúng tôi có thể tạo trang, liên kết tới trang khác, làm mọi kiểu định dạng wiki hoa mỹ, thậm chí chạy test bằng FIT. Thứ duy nhất chúng tôi không làm được là lưu lại bất cứ thành quả nào.

When it came time to implement persistence, we thought again about MySQL, but decided that wasn't necessary in the short term, because it would be really easy to write the hash tables out to flat files. So we implemented FileSystemWikiPage , which just moved the functionality out to flat files, and then we continued developing more features.

Đến lúc phải hiện thực khả năng lưu trữ bền vững (persistence), chúng tôi lại nghĩ tới MySQL, nhưng kết luận rằng trước mắt chưa cần, vì ghi các bảng băm ra file phẳng thì quá dễ. Vậy là chúng tôi hiện thực FileSystemWikiPage , thứ chỉ chuyển chức năng ấy ra file phẳng, rồi tiếp tục phát triển thêm tính năng.

Three months later, we reached the conclusion that the flat file solution was good enough; we decided to abandon the idea of MySQL altogether. We deferred that decision into nonexistence and never looked back.

Ba tháng sau, chúng tôi đi tới kết luận rằng giải pháp file phẳng là đủ tốt; chúng tôi quyết định dẹp hẳn ý tưởng MySQL. Chúng tôi đã trì hoãn quyết định ấy cho tới khi nó không còn tồn tại, và không bao giờ ngoái lại.

That would be the end of the story if it weren't for one of our customers who decided that he needed to put the wiki into MySQL for his own purposes. We

Câu chuyện hẳn đã kết thúc ở đó nếu không có một khách hàng của chúng tôi quyết định rằng anh ta cần đưa wiki vào MySQL cho mục đích riêng. Chúng tôi

showed him the architecture of WikiPages that had allowed us to defer the decision. He came back a day later with the whole system working in MySQL. He simply wrote a MySqlWikiPage derivative and got it working.

đã cho anh ta xem kiến trúc WikiPages vốn cho phép chúng tôi trì hoãn quyết định. Một ngày sau anh ta quay lại với cả hệ thống chạy trên MySQL. Anh ta chỉ đơn giản viết một dẫn xuất MySqlWikiPage và cho nó chạy được.

We used to bundle that option with FitNesse , but nobody else ever used it, so eventually we dropped it. Even the customer who wrote the derivative eventually dropped it.

Chúng tôi từng đóng gói kèm tùy chọn đó vào FitNesse , nhưng chẳng ai khác dùng, nên rốt cuộc chúng tôi bỏ đi. Ngay cả vị khách hàng đã viết dẫn xuất ấy cuối cùng cũng bỏ nó.

Early in the development of FitNesse , we drew a **boundary** line between business rules and databases. That line prevented the business rules from knowing anything at all about the database, other than the simple data access methods. That decision allowed us to defer the choice and implementation of the database for well over a year. It allowed us to try the file system option, and it allowed us to change direction when we saw a better solution. Yet it did not prevent, or even impede, moving in the original direction (MySQL) when someone wanted it.

Ngay từ đầu quá trình phát triển FitNesse , chúng tôi đã vạch một đường ranh giới giữa quy tắc nghiệp vụ (business rule) và cơ sở dữ liệu. Đường kẻ ấy ngăn các quy tắc nghiệp vụ biết bất cứ điều gì về cơ sở dữ liệu, ngoài mấy phương thức truy cập dữ liệu đơn giản. Quyết định đó cho phép chúng tôi trì hoãn việc chọn và hiện thực cơ sở dữ liệu suốt hơn một năm trời. Nó cho phép chúng tôi thử phương án file system, và cho phép chúng tôi đổi hướng khi thấy một giải pháp tốt hơn. Vậy mà nó không hề ngăn cản, thậm chí không hề cản trở, việc đi theo hướng ban đầu (MySQL) khi có người muốn thế.

The fact that we did not have a database running for 18 months of development meant that, for 18 months, we did not have schema issues, query issues, database server issues, password issues, connection time issues, and all the other nasty issues that raise their ugly heads when you fire up a database. It also meant that all our tests ran fast, because there was no database to slow them down.

Việc suốt 18 tháng phát triển mà không có cơ sở dữ liệu nào chạy có nghĩa là, suốt 18 tháng ấy, chúng tôi không gặp vấn đề về schema, vấn đề về truy vấn, vấn đề về database server, vấn đề về mật khẩu, vấn đề về thời gian kết nối, và tất cả những rắc rối khó ưa khác vẩn ngóc cái đầu xấu xí lên mỗi khi bạn khởi động một cơ sở dữ liệu. Nó còn có nghĩa là mọi test của chúng tôi đều chạy nhanh, vì chẳng có cơ sở dữ liệu nào làm chúng chậm lại.

In short, drawing the boundary lines helped us delay and defer decisions, and it ultimately saved us an enormous amount of time and headaches. And that's what a good architecture should do.

Nói ngắn gọn, việc vạch các đường ranh giới đã giúp chúng tôi hoãn và trì hoãn các quyết định, và rốt cuộc tiết kiệm cho chúng tôi vô số thời gian lẫn đau đầu. Và đó chính là điều một kiến trúc tốt phải làm được.

Which Lines Do You Draw, and When Do You Draw Them? — Bạn vạch những đường nào, và vạch khi nào?

You draw lines between things that matter and things that don't. The GUI doesn't matter to the business rules, so there should be a line between them. The database doesn't matter to the GUI, so there should be a line between

Bạn vạch các đường kẻ giữa những thứ có ý nghĩa và những thứ không. GUI chẳng có ý nghĩa gì với các quy tắc nghiệp vụ, nên phải có một đường kẻ giữa chúng. Cơ sở dữ liệu chẳng có ý nghĩa gì với GUI, nên phải có một đường kẻ giữa

them. The database doesn't matter to the business rules, so there should be a line between them.

chúng. Cơ sở dữ liệu chẳng có ý nghĩa gì với các quy tắc nghiệp vụ, nên phải có một đường kẻ giữa chúng.

Some of you may have rejected one or more of those statements, especially the part about the business rules not caring about the database. Many of us have been taught to believe that the database is inextricably connected to the business rules. Some of us have even been convinced that the database is the embodiment of the business rules.

Một số bạn có thể đã bác bỏ một hoặc vài phát biểu đó, nhất là đoạn nói rằng quy tắc nghiệp vụ chẳng quan tâm tới cơ sở dữ liệu. Nhiều người trong chúng ta được dạy để tin rằng cơ sở dữ liệu gắn chặt không thể tách rời với quy tắc nghiệp vụ. Thậm chí có người còn bị thuyết phục rằng cơ sở dữ liệu chính là hiện thân của quy tắc nghiệp vụ.

But, as we shall see in another chapter, this idea is misguided. The database is a tool that the business rules can use indirectly. The business rules don't need to know about the schema, or the query language, or any of the other details about the database. All the business rules need to know is that there is a set of functions that can be used to fetch or save data. This allows us to put the database behind an interface.

Nhưng, như ta sẽ thấy ở một chương khác, ý tưởng này là lầm lạc. Cơ sở dữ liệu là một công cụ mà quy tắc nghiệp vụ có thể dùng một cách gián tiếp. Quy tắc nghiệp vụ không cần biết về schema, hay ngôn ngữ truy vấn, hay bất kỳ chi tiết nào khác của cơ sở dữ liệu. Tất cả những gì quy tắc nghiệp vụ cần biết là có một tập hàm dùng để lấy hoặc lưu dữ liệu. Điều này cho phép ta đặt cơ sở dữ liệu ra sau một interface.

You can see this clearly in Figure 17.1. The BusinessRules use the DatabaseInterface to load and save data. The DatabaseAccess implements the interface and directs the operation of the actual Database

Bạn có thể thấy rõ điều này trong Figure 17.1. BusinessRules dùng DatabaseInterface để nạp và lưu dữ liệu. DatabaseAccess hiện thực interface đó và điều khiển hoạt động của Database thật.

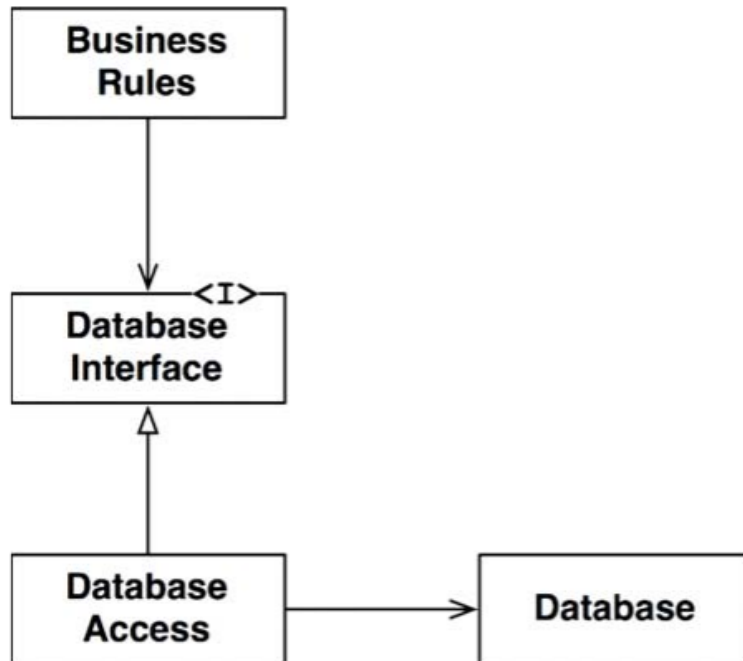


Figure 17.1 The database behind an interface

Figure 17.1 Cơ sở dữ liệu nằm sau một interface

The classes and interfaces in this diagram are symbolic. In a real application, there would be many **business rule** classes, many database interface classes, and many database access implementations. All of them, though, would follow roughly the same pattern.

Các class và interface trong sơ đồ này mang tính tượng trưng. Trong một ứng dụng thật, sẽ có nhiều class quy tắc nghiệp vụ, nhiều class interface cơ sở dữ liệu, và nhiều hiện thực truy cập cơ sở dữ liệu. Tuy vậy, tất cả chúng đều theo đại thể cùng một khuôn mẫu.

Where is the boundary line? The boundary is drawn across the **inheritance** relationship, just below the DatabaseInterface (Figure 17.2).

Đường ranh giới nằm ở đâu? Ranh giới được vạch cắt ngang quan hệ kế thừa, ngay bên dưới DatabaseInterface (Figure 17.2).

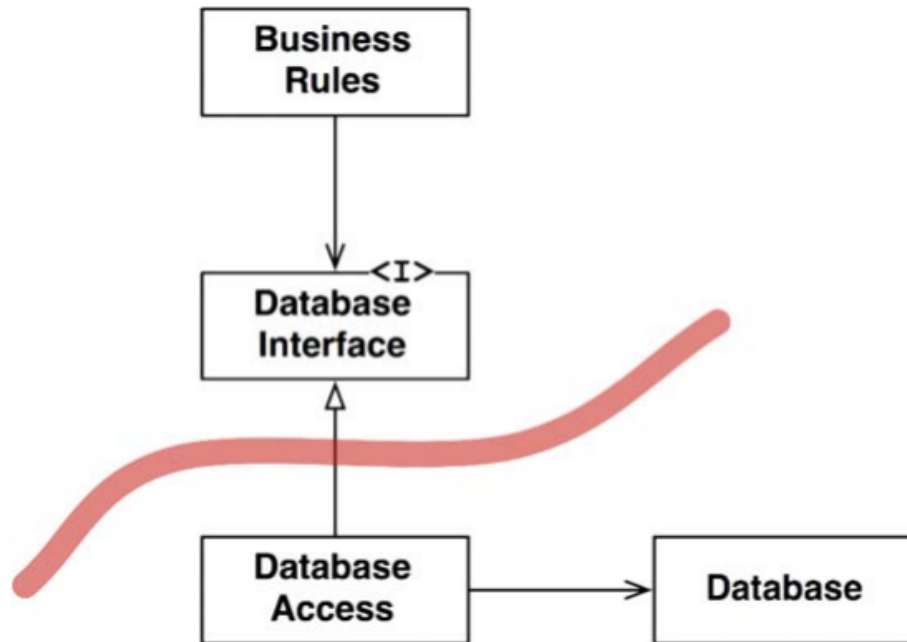


Figure 17.2 The boundary line

Figure 17.2 Đường ranh giới

Note the two arrows leaving the DatabaseAccess class. Those two arrows point away from the DatabaseAccess class. That means that none of these classes knows that the DatabaseAccess class exists.

Chú ý hai mũi tên rời khỏi class DatabaseAccess . Hai mũi tên đó chỉ ra xa khỏi class DatabaseAccess . Điều đó có nghĩa là không class nào trong số này biết rằng class DatabaseAccess tồn tại.

Now let's pull back a bit. We'll look at the **component** that contains many business rules, and the component that contains the database and all its access classes (Figure 17.3).

Giờ hãy lùi ra một chút. Ta sẽ nhìn vào component chứa nhiều quy tắc nghiệp vụ, và component chứa cơ sở dữ liệu cùng toàn bộ các class truy cập của nó (Figure 17.3).

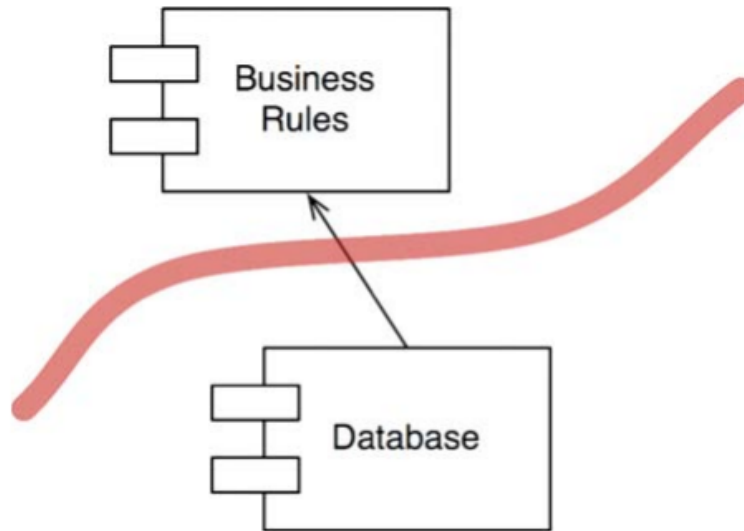


Figure 17.3 The business rules and database components

Figure 17.3 Các component quy tắc nghiệp vụ và cơ sở dữ liệu

Note the direction of the arrow. The Database knows about the BusinessRules . The BusinessRules do not know about the Database . This implies that the DatabaseInterface classes live in the BusinessRules component, while the DatabaseAccess classes live in the Database component.

Chú ý chiều của mũi tên. Database biết về BusinessRules . BusinessRules không biết về Database . Điều này hàm ý rằng các class DatabaseInterface sống trong component BusinessRules , còn các class DatabaseAccess sống trong component Database .

The direction of this line is **important**. It shows that the Database does not matter to the BusinessRules , but the Database cannot exist without the

Chiều của đường kẻ này rất quan trọng. Nó cho thấy Database chẳng có ý nghĩa gì với BusinessRules , nhưng Database không thể tồn tại nếu không có

BusinessRules .

If that seems strange to you, just remember this point: The Database component contains the code that translates the calls made by the BusinessRules into the query language of the database. It is that translation code that knows about the BusinessRules .

Nếu điều đó nghe lạ tai với bạn, hãy nhớ điểm này: component Database chứa đoạn mã dịch các lời gọi do BusinessRules thực hiện sang ngôn ngữ truy vấn của cơ sở dữ liệu. Chính đoạn mã dịch ấy mới là thứ biết về BusinessRules .

Having drawn this boundary line between the two components, and having set the direction of the arrow toward the BusinessRules , we can now see that the BusinessRules could use any kind of database. The Database component could be replaced with many different implementations—the

Sau khi đã vạch đường ranh giới này giữa hai component, và đã đặt chiều mũi tên hướng về phía BusinessRules , giờ ta có thể thấy rằng BusinessRules có thể dùng bất kỳ loại cơ sở dữ liệu nào. Component Database có thể được thay bằng nhiều hiện thực khác nhau — thứ

BusinessRules don't care.

The database could be implemented with Oracle, or MySQL, or Couch, or Datomic, or even flat files. The business rules don't care at all. And that means that the database decision can be deferred and you can focus on getting the business rules written and tested before you have to make the database decision.

Cơ sở dữ liệu có thể được hiện thực bằng Oracle, hoặc MySQL, hoặc Couch, hoặc Datomic, hoặc thậm chí file phẳng. Quy tắc nghiệp vụ chẳng bận tâm chút nào. Và điều đó có nghĩa là quyết định về cơ sở dữ liệu có thể được trì hoãn, còn bạn thì tập trung viết và kiểm thử xong các quy tắc nghiệp vụ trước khi buộc phải chọn cơ sở dữ liệu.

What About Input and Output? — Còn đầu vào và đầu ra thì sao?

Developers and customers often get confused about what the system is. They see the GUI, and think that the GUI is the system. They define a system in terms of the GUI, so they believe that they should see the GUI start working immediately. They fail to realize a critically important principle: The IO is irrelevant.

Lập trình viên lẫn khách hàng thường lẫn lộn về việc hệ thống là cái gì. Họ nhìn thấy GUI, và nghĩ rằng GUI chính là hệ thống. Họ định nghĩa hệ thống theo GUI, nên họ tin rằng họ phải thấy GUI chạy được ngay lập tức. Họ không nhận ra một nguyên tắc cực kỳ quan trọng: IO là chuyện không liên quan.

This may be hard to grasp at first. We often think about the **behavior** of the system in terms of the behavior of the IO. Consider a video game, for example. Your experience is dominated by the interface: the screen, the mouse, the buttons, and the sounds. You forget that behind that interface there is a model—a sophisticated set of data structures and functions—driving it. More importantly, that model does not need the interface. It would happily execute its duties, modeling all the events in the game, without the game ever being displayed on the screen. The interface does not matter to the model—the business rules.

Bạn đầu điều này có thể khó nắm bắt. Ta thường nghĩ về hành vi của hệ thống theo hành vi của IO. Ví dụ, hãy xét một trò chơi điện tử. Trải nghiệm của bạn bị chi phối bởi giao diện: màn hình, chuột, các nút bấm, và âm thanh. Bạn quên mất rằng đằng sau giao diện đó có một mô hình — một tập cấu trúc dữ liệu và hàm tính vi — đang vận hành nó. Quan trọng hơn, mô hình ấy không cần tới giao diện. Nó vẫn vui vẻ làm tròn phận sự, mô hình hóa mọi sự kiện trong trò chơi, mà chẳng cần trò chơi được hiển thị lên màn hình bao giờ. Giao diện chẳng có ý nghĩa gì với mô hình — tức các quy tắc nghiệp vụ.

And so, once again, we see the GUI and BusinessRules components separated by a boundary line (Figure 17.4). Once again, we see that the less relevant component depends on the more relevant component. The arrows show which component knows about the other and, therefore, which component cares about the other. The GUI cares about the

Và thế là, một lần nữa, ta thấy component GUI và BusinessRules bị ngăn cách bởi một đường ranh giới (Figure 17.4). Một lần nữa, ta thấy component ít quan trọng hơn phụ thuộc vào component quan trọng hơn. Các mũi tên cho thấy component nào biết về component kia, và do đó, component nào bận tâm tới component kia. GUI bận tâm tới

BusinessRules.

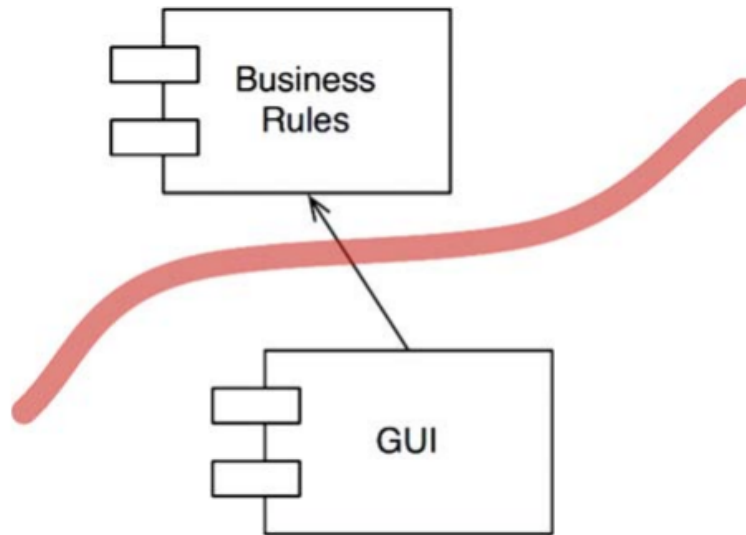


Figure 17.4 The boundary between GUI and BusinessRules components

Figure 17.4 Ranh giới giữa các component GUI và BusinessRules

Having drawn this boundary and this arrow, we can now see that the GUI could be replaced with any other kind of interface—and the BusinessRules would not care.

Sau khi đã vạch ranh giới này cùng mũi tên này, giờ ta có thể thấy rằng GUI có thể được thay bằng bất kỳ loại giao diện nào khác — và BusinessRules sẽ chẳng bận tâm.

Plugin Architecture — Kiến trúc plugin

Taken together, these two decisions about the database and the GUI create a kind of pattern for the addition of other components. That pattern is the same pattern that is used by systems that allow third-party plugins.

Gộp lại, hai quyết định về cơ sở dữ liệu và GUI tạo ra một dạng khuôn mẫu cho việc thêm các component khác. Khuôn mẫu ấy chính là khuôn mẫu mà các hệ thống cho phép plugin của bên thứ ba vẫn dùng.

Indeed, the history of software development technology is the story of how to conveniently create plugins to establish a scalable and maintainable system architecture. The core business rules are kept separate from, and independent of, those components that are either optional or that can be implemented in many different forms (Figure 17.5).

Thật vậy, lịch sử công nghệ phát triển phần mềm là câu chuyện về việc làm sao tạo plugin một cách thuận tiện để dựng nên một kiến trúc hệ thống có khả năng mở rộng và bảo trì được. Các quy tắc nghiệp vụ cốt lõi được giữ tách biệt và độc lập với những component vốn hoặc là tùy chọn, hoặc là có thể được hiện thực theo nhiều dạng khác nhau (Figure 17.5).

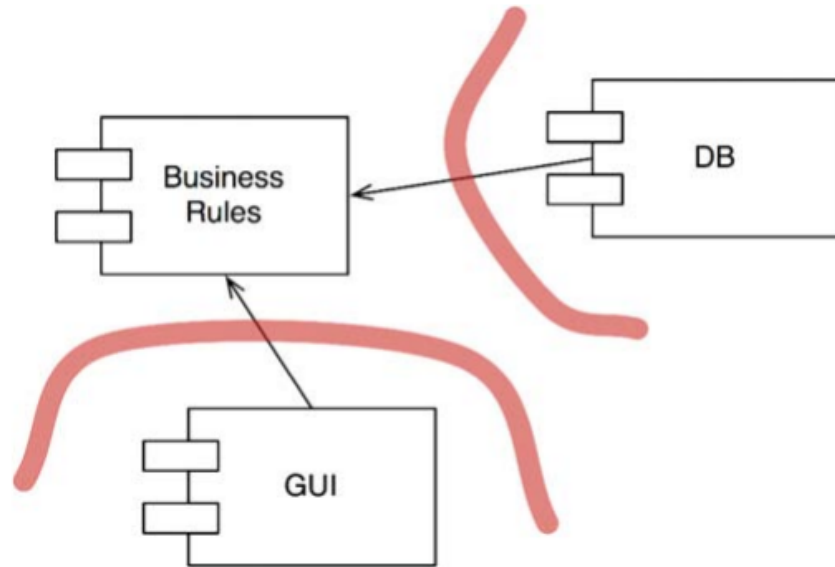


Figure 17.5 Plugging in to the business rules

Figure 17.5 Cắm vào các quy tắc nghiệp vụ

Because the user interface in this design is considered to be a plugin, we have made it possible to plug in many different kinds of user interfaces. They could be web based, client/server based, SOA based, Console based, or based on any other kind of user interface technology.

Vì giao diện người dùng trong thiết kế này được coi là một plugin, ta đã khiến cho việc cắm vào nhiều loại giao diện người dùng khác nhau trở nên khả thi. Chúng có thể dựa trên web, dựa trên client/server, dựa trên SOA, dựa trên Console, hoặc dựa trên bất kỳ loại công nghệ giao diện người dùng nào khác.

The same is true of the database. Since we have chosen to treat it as a plugin, we can replace it with any of the various SQL databases, or a NOSQL database, or a file system-based database, or any other kind of database technology we might deem necessary in the future.

Cơ sở dữ liệu cũng vậy. Vì ta đã chọn đối xử với nó như một plugin, ta có thể thay nó bằng bất kỳ cơ sở dữ liệu SQL nào, hoặc một cơ sở dữ liệu NOSQL, hoặc một cơ sở dữ liệu dựa trên file system, hoặc bất kỳ loại công nghệ cơ sở dữ liệu nào khác mà ta thấy cần trong tương lai.

These replacements might not be trivial. If the initial deployment of our system was web-based, then writing the plugin for a client-server UI could be challenging. It is likely that some of the communications between the business rules and the new UI would have to be reworked. Even so, by starting with the presumption of a plugin **structure**, we have at very least made such a change practical.

Những phép thay thế này có thể không hề tầm thường. Nếu bản triển khai ban đầu của hệ thống là nền web, thì viết plugin cho một UI client-server có thể là thử thách. Nhiều khả năng một phần giao tiếp giữa quy tắc nghiệp vụ và UI mới sẽ phải làm lại. Dù vậy, bằng cách khởi đầu với giả định về một cấu trúc plugin, ít nhất ta đã khiến một thay đổi như thế trở nên khả thi.

The Plugin Argument — Lập luận về plugin

Consider the relationship between ReSharper and Visual Studio. These components are produced by completely different development teams in completely different companies. Indeed, JetBrains, the maker of ReSharper, lives in Russia. Microsoft, of course, resides in Redmond, Washington. It's hard to imagine two development teams that are more separate.

Hãy xét quan hệ giữa ReSharper và Visual Studio. Hai component này do hai đội phát triển hoàn toàn khác nhau, ở hai công ty hoàn toàn khác nhau tạo ra. Thật vậy, JetBrains, hãng làm ra ReSharper, ở Nga. Microsoft, tất nhiên, đóng tại Redmond, Washington. Khó mà hình dung nổi hai đội phát triển nào tách biệt nhau hơn thế.

Which team can damage the other? Which team is immune to the other? The dependency structure tells the story (Figure 17.6). The source code of ReSharper depends on the source code of Visual Studio. Thus there is nothing that the ReSharper team can do to disturb the Visual Studio team. But the Visual Studio team could completely disable the ReSharper team if they so desired.

Đội nào có thể làm hại đội kia? Đội nào miễn nhiễm với đội kia? Cấu trúc phụ thuộc kể cho ta câu chuyện đó (Figure 17.6). Mã nguồn của ReSharper phụ thuộc vào mã nguồn của Visual Studio. Do đó, đội ReSharper chẳng làm gì được để quấy nhiễu đội Visual Studio. Nhưng đội Visual Studio thì có thể vô hiệu hóa hoàn toàn đội ReSharper nếu họ muốn.

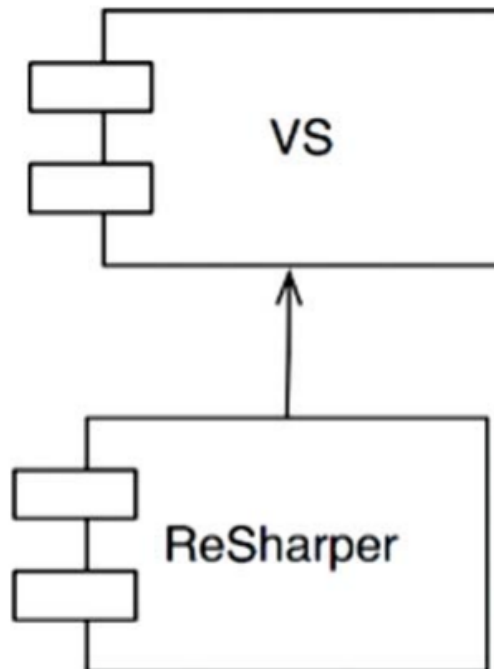


Figure 17.6 ReSharper depends on Visual Studio

Figure 17.6 ReSharper phụ thuộc vào Visual Studio

That's a deeply asymmetric relationship, and it is one that we desire to have in our own systems. We want certain modules to be immune to others. For example, we don't want the business rules to break when someone changes the format of a web page, or changes the schema of the database. We don't

Đó là một quan hệ bất đối xứng sâu sắc, và là thứ ta mong muốn có trong chính hệ thống của mình. Ta muốn một số module miễn nhiễm với những module khác. Chẳng hạn, ta không muốn quy tắc nghiệp vụ vỡ khi có ai đó đổi định dạng một trang web, hay đổi schema của cơ sở dữ liệu. Ta không

want changes in one part of the system to cause other unrelated parts of the system to break. We don't want our systems to exhibit that kind of fragility.

muốn những thay đổi ở một phần của hệ thống làm vỡ các phần khác chẳng liên quan. Ta không muốn hệ thống của mình bệch lộ kiểu mong manh đó.

Arranging our systems into a **plugin architecture** creates firewalls across which changes cannot propagate. If the GUI plugs in to the business rules, then changes in the GUI cannot affect those business rules.

Sắp xếp hệ thống theo kiến trúc plugin tạo ra những bức tường lửa mà thay đổi không thể lan qua. Nếu GUI cắm vào các quy tắc nghiệp vụ, thì thay đổi trong GUI không thể ảnh hưởng tới những quy tắc nghiệp vụ ấy.

Boundaries are drawn where there is an axis of change. The components on one side of the boundary change at different rates, and for different reasons, than the components on the other side of the boundary.

Ranh giới được vạch ở nơi có một trục thay đổi. Các component ở một bên ranh giới thay đổi với nhịp độ khác, và vì lý do khác, so với các component ở bên kia ranh giới.

GUIs change at different times and at different rates than business rules, so there should be a boundary between them. Business rules change at different times and for different reasons than dependency injection frameworks, so there should be a boundary between them.

GUI thay đổi vào những thời điểm khác và với nhịp độ khác so với quy tắc nghiệp vụ, nên phải có một ranh giới giữa chúng. Quy tắc nghiệp vụ thay đổi vào những thời điểm khác và vì những lý do khác so với các framework dependency injection, nên phải có một ranh giới giữa chúng.

This is simply the Single Responsibility Principle again. The SRP tells us where to draw our boundaries.

Đây chỉ đơn giản lại là SRP — Nguyên tắc Trách nhiệm Đơn nhất (Single Responsibility Principle) mà thôi. SRP cho ta biết nên vạch ranh giới ở đâu.

Conclusion — Kết luận

To draw boundary lines in a software architecture, you first partition the system into components. Some of those components are core business rules; others are plugins that contain necessary functions that are not directly related to the core business. Then you arrange the code in those components such that the arrows between them point in one direction—toward the core business.

Để vạch các đường ranh giới trong một kiến trúc phần mềm, trước hết bạn phân hoạch hệ thống thành các component. Một số component là quy tắc nghiệp vụ cốt lõi; số khác là plugin chứa những chức năng cần thiết nhưng không liên quan trực tiếp tới nghiệp vụ cốt lõi. Rồi bạn sắp xếp mã trong các component đó sao cho mũi tên giữa chúng đều chỉ về một hướng — hướng về phía nghiệp vụ cốt lõi.

You should recognize this as an application of the **Dependency Inversion** Principle and the **Stable** Abstractions Principle. Dependency arrows are arranged to point from lower-**level** details to higher-level abstractions.

Bạn hẳn nhận ra đây là một ứng dụng của DIP — Nguyên tắc Đảo ngược Phụ thuộc (Dependency Inversion Principle) và SAP — Nguyên tắc Trừu tượng Ổn định (Stable Abstractions Principle). Các mũi tên phụ thuộc được sắp xếp sao cho chúng hướng từ các chi tiết mức thấp lên các phần trừu tượng mức cao.

Boundary Anatomy — Giải phẫu ranh giới

The **architecture** of a system is defined by a set of software components and the boundaries that separate them. Those boundaries come in many different forms. In this chapter we'll look at some of the most common.

Kiến trúc (architecture) của một hệ thống được định nghĩa bởi một tập các component phần mềm và các ranh giới (boundary) chia tách chúng. Những ranh giới đó có rất nhiều hình dạng khác nhau. Trong chương này, chúng ta sẽ xem xét vài dạng phổ biến nhất.

Boundary Crossing — Vượt ranh giới

At runtime, a **boundary crossing** is nothing more than a function on one side of the **boundary** calling a function on the other side and passing along some data. The trick to creating an appropriate boundary crossing is to manage the source code dependencies.

Ở thời điểm chạy, vượt ranh giới (boundary crossing) chẳng qua chỉ là một hàm ở phía bên này ranh giới gọi một hàm ở phía bên kia và truyền kèm ít dữ liệu. Bí quyết để tạo ra một điểm vượt ranh giới hợp lý nằm ở chỗ quản lý các phụ thuộc mã nguồn.

Why source code? Because when one source code module changes, other source code modules may have to be changed or recompiled, and then redeployed. Managing and building firewalls against this change is what boundaries are all about.

Tại sao lại là mã nguồn? Bởi vì khi một module mã nguồn thay đổi, các module mã nguồn khác có thể phải sửa theo, phải biên dịch lại, rồi triển khai lại. Quản lý và dựng tường lửa chống lại kiểu thay đổi lan truyền đó chính là toàn bộ ý nghĩa của ranh giới.

The Dreaded Monolith — Con quái vật monolith đáng sợ

The simplest and most common of the architectural boundaries has no strict physical representation. It is simply a disciplined segregation of functions and data within a single processor and a single address space. In a previous chapter, I called this the **source-level decoupling mode**.

Dạng ranh giới kiến trúc đơn giản nhất và phổ biến nhất lại không có biểu hiện vật lý rõ rệt nào. Nó đơn thuần là sự phân tách có kỷ luật giữa các hàm và dữ liệu bên trong cùng một bộ xử lý và cùng một không gian địa chỉ. Ở một chương trước, tôi đã gọi đây là chế độ tách rời ở mức mã nguồn.

From a deployment point of view, this amounts to nothing more than a single executable file—the so-called monolith. This file might be a statically linked C or C++ project, a set of Java class files bound together into an executable jar file, a set of .NET binaries bound into a single .EXE file, and so on.

Nhìn từ góc độ triển khai, thứ này chẳng qua chỉ là một file thực thi duy nhất — cái gọi là monolith. File đó có thể là một dự án C hay C++ liên kết tĩnh, một tập các file class Java gói chung vào một file jar thực thi được, một tập binary .NET gói vào một file .EXE duy nhất, v.v.

The fact that the boundaries are not visible during the deployment of a monolith does not mean that they are not present and meaningful. Even when statically linked into a single executable, the ability to independently develop and marshal the various components for final assembly is immensely valuable.

Việc các ranh giới không lộ ra khi triển khai một monolith không có nghĩa là chúng không tồn tại hay không có ý nghĩa. Ngay cả khi mọi thứ đã được liên kết tĩnh vào một file thực thi duy nhất, khả năng phát triển độc lập rồi tập kết các component khác nhau lại để lắp ráp lần cuối vẫn cực kỳ giá trị.

Such architectures almost always depend on some kind of dynamic **polymorphism** 1 to manage their internal dependencies. This is one of the reasons that object-oriented development has become such an **important paradigm** in recent decades. Without OO, or an equivalent form of polymorphism, architects must fall back on the dangerous practice of using pointers to functions to achieve the appropriate **decoupling**. Most architects find prolific use of pointers to functions to be too risky, so they are forced to abandon any kind of **component** partitioning.

Những kiến trúc như vậy gần như luôn dựa vào một dạng đa hình (polymorphism) động 1 nào đó để quản lý các phụ thuộc bên trong. Đây là một trong những lý do khiến lập trình hướng đối tượng trở thành mô thức (paradigm) quan trọng đến thế trong vài thập niên gần đây. Không có OO, hoặc một dạng đa hình tương đương, kiến trúc sư buộc phải quay về với trò nguy hiểm là dùng con trỏ hàm để đạt được mức tách rời (decoupling) mong muốn. Phần lớn kiến trúc sư thấy việc dùng con trỏ hàm tràn lan là quá rủi ro, nên họ đành từ bỏ mọi ý định phân chia component.

The simplest possible boundary crossing is a function call from a **low-level** client to a higher-level **service**. Both the runtime dependency and the compile-time dependency point in the same direction, toward the higher-level component.

Kiểu vượt ranh giới đơn giản nhất có thể có là một lời gọi hàm từ client mức thấp tới service mức cao hơn. Cả phụ thuộc lúc chạy lẫn phụ thuộc lúc biên dịch đều chỉ về cùng một hướng: hướng về component ở mức cao hơn.

In Figure 18.1, the **flow of control** crosses the boundary from left to right. The Client calls function `f()` on the Service. It passes along an instance of `Data`. The marker simply indicates a data **structure**. The `Data` may be passed as a function argument or by some other more elaborate means. Note that the definition of the `Data` is on the called side of the boundary.

Trong Figure 18.1, luồng điều khiển vượt ranh giới từ trái sang phải. Client gọi hàm `f()` trên Service. Nó truyền kèm một instance của `Data`. Ký hiệu đơn giản là để chỉ một cấu trúc dữ liệu. `Data` có thể được truyền như một đối số hàm, hoặc bằng cách nào đó cầu kỳ hơn. Hãy để ý rằng định nghĩa của `Data` nằm ở phía bị gọi của ranh giới.

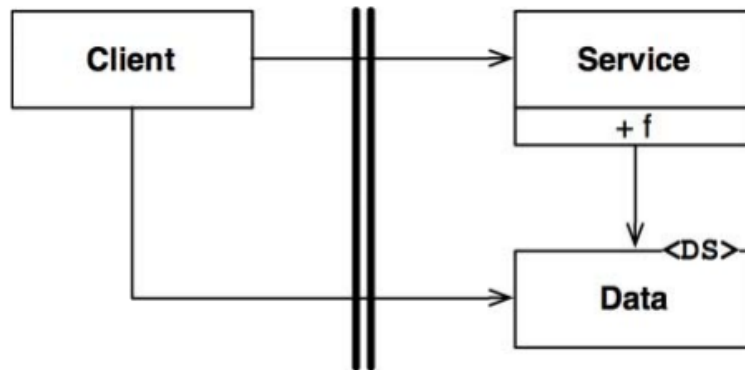


Figure 18.1 Flow of control crosses the boundary from a lower level to a higher level

Figure 18.1 Luồng điều khiển vượt ranh giới từ mức thấp lên mức cao

When a **high-level** client needs to invoke a lower-level service, dynamic polymorphism is used to invert the dependency against the flow of control. The runtime dependency opposes the compile-time dependency.

Khi một client mức cao cần gọi một service mức thấp hơn, ta dùng đa hình động để đảo ngược phụ thuộc so với luồng điều khiển. Phụ thuộc lúc chạy đi ngược với phụ thuộc lúc biên dịch.

- 1. Static polymorphism (e.g., generics or templates) can sometimes be a viable means of dependency manage-
 - 1. Đa hình tĩnh (chẳng hạn generic hay template) đôi khi cũng là một cách khả dĩ để quản lý phụ thuộc

ment in monolithic systems, especially in languages like C++. However, the decoupling afforded by generics cannot protect you from the need for recompilation and redeployment the way dynamic polymorphism can.

trong các hệ thống monolith, đặc biệt là với những ngôn ngữ như C++. Tuy nhiên, mức tách rời mà generic mang lại không thể bảo vệ bạn khỏi nhu cầu biên dịch lại và triển khai lại theo cách mà đa hình động làm được.

In Figure 18.2, the flow of control crosses the boundary from left to right as before. The high-level Client calls the f() function of the lower-level ServiceImpl through the Service **interface**. Note, however, that all dependencies cross the boundary from right to left toward the higher-level component . Note, also, that the definition of the data structure is on the calling side of the boundary.

Trong Figure 18.2, luồng điều khiển vẫn vượt ranh giới từ trái sang phải như trước. Client mức cao gọi hàm f() của ServiceImpl mức thấp hơn thông qua interface Service . Nhưng hãy để ý rằng mọi phụ thuộc đều vượt ranh giới từ phải sang trái, hướng về component ở mức cao hơn . Cũng để ý rằng định nghĩa của cấu trúc dữ liệu lần này nằm ở phía gọi của ranh giới.

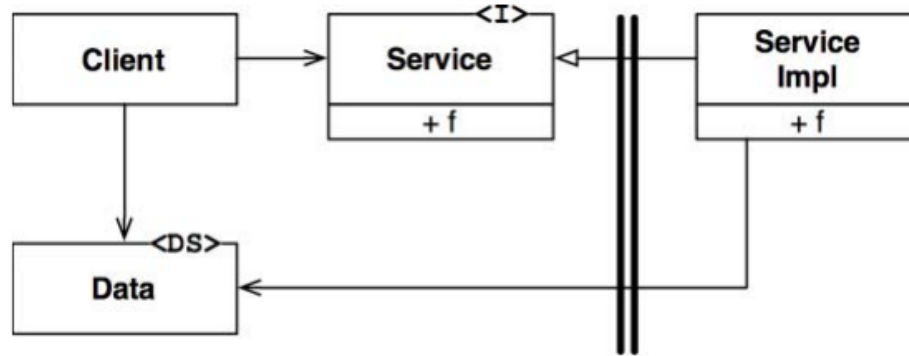


Figure 18.2 Crossing the boundary against the flow of control

Figure 18.2 Vượt ranh giới ngược với luồng điều khiển

Even in a monolithic, statically linked executable, this kind of disciplined partitioning can greatly aid the job of developing, testing, and deploying the project. Teams can work independently of each other on their own components without treading on each other's toes. High-level components remain independent of lower-level details.

Ngay cả trong một file thực thi monolith liên kết tĩnh, kiểu phân chia có kỷ luật này vẫn hỗ trợ rất nhiều cho việc phát triển, kiểm thử và triển khai dự án. Các nhóm có thể làm việc độc lập với nhau trên component của mình mà không giẫm chân nhau. Các component mức cao vẫn độc lập với những chi tiết ở mức thấp hơn.

Communications between components in a monolith are very fast and inexpensive. They are typically just function calls. Consequently, communications across source-level decoupled boundaries can be very chatty.

Giao tiếp giữa các component trong một monolith rất nhanh và rất rẻ. Chúng thường chỉ là những lời gọi hàm. Hệ quả là giao tiếp qua các ranh giới tách rời ở mức mã nguồn có thể rất “lắm lời”.

Since the deployment of monoliths usually requires compilation and static linking, components in these systems are typically delivered as source code.

Vì việc triển khai monolith thường đòi hỏi biên dịch và liên kết tĩnh, các component trong những hệ thống này thường được giao dưới dạng mã nguồn.

Deployment Components — Component triển khai

The simplest physical representation of an **architectural boundary** is a dynamically linked library like a .Net DLL, a Java jar file, a Ruby Gem, or a UNIX shared library. Deployment does not involve compilation. Instead, the

Biểu hiện vật lý đơn giản nhất của một ranh giới kiến trúc là một thư viện liên kết động: DLL của .Net, file jar của Java, Gem của Ruby, hay shared library của UNIX. Triển khai không kèm biên dịch. Thay vào đó,

components are delivered in binary, or some equivalent deployable form. This is the deployment-level decoupling mode. The act of deployment is simply the gathering of these deployable units together in some convenient form, such as a WAR file, or even just a directory.

các component được giao dưới dạng binary, hoặc một dạng triển khai được tương đương. Đây là chế độ tách rời ở mức triển khai. Hành động triển khai đơn giản là gom các đơn vị triển khai này lại với nhau dưới một hình thức thuận tiện nào đó, chẳng hạn một file WAR, hoặc thậm chí chỉ là một thư mục.

With that one exception, deployment-level components are the same as monoliths. The functions generally all exist in the same processor and address space. The strategies for segregating the components and managing their dependencies are the same. 2

Ngoài khác biệt duy nhất đó, component ở mức triển khai chẳng khác gì monolith. Nhìn chung mọi hàm vẫn nằm trong cùng một bộ xử lý và cùng một không gian địa chỉ. Chiến lược phân tách component và quản lý phụ thuộc giữa chúng cũng y hệt. 2

As with monoliths, communications across deployment component boundaries are just function calls and, therefore, are very inexpensive. There may be a one-time hit for dynamic linking or runtime loading, but communications across these boundaries can still be very chatty.

Giống như với monolith, giao tiếp qua ranh giới của component triển khai chỉ là những lời gọi hàm, nên rất rẻ. Có thể có một khoản chi phí một lần cho việc liên kết động hoặc nạp lúc chạy, nhưng giao tiếp qua những ranh giới này vẫn có thể rất “lắm lời”.

Threads — Thread

Both monoliths and deployment components can make use of threads. Threads are not architectural boundaries or units of deployment, but rather a way to organize the schedule and order of execution. They may be wholly contained within a component, or spread across many components.

Cả monolith lẫn component triển khai đều có thể dùng thread. Thread không phải là ranh giới kiến trúc, cũng không phải đơn vị triển khai, mà là một cách tổ chức lịch trình và thứ tự thực thi. Chúng có thể nằm trọn trong một component, hoặc trải ra nhiều component.

Local Processes — Tiến trình cục bộ

A much stronger physical architectural boundary is the local process. A local process is typically created from the command line or an equivalent system call. Local processes run in the same processor, or in the same set of processors within a multicore, but run in separate address spaces. Memory protection generally prevents such processes from sharing memory, although shared memory partitions are often used.

Một ranh giới kiến trúc vật lý mạnh hơn nhiều là tiến trình cục bộ (local process). Tiến trình cục bộ thường được tạo ra từ dòng lệnh hoặc một lời gọi hệ thống tương đương. Các tiến trình cục bộ chạy trên cùng một bộ xử lý, hoặc trên cùng một cụm lõi của bộ xử lý đa nhân, nhưng chạy trong những không gian địa chỉ riêng biệt. Cơ chế bảo vệ bộ nhớ nhìn chung ngăn các tiến trình này chia sẻ bộ nhớ với nhau, dù người ta vẫn hay dùng các vùng bộ nhớ chia sẻ.

- 2. Although static polymorphism is not an option in this case.
 - 2. Dù trong trường hợp này thì đa hình tĩnh không phải một lựa chọn.

Most often, local processes communicate with each other using sockets, or some other kind of operating system communications facility such as mailboxes or message queues.

Thường thì các tiến trình cục bộ giao tiếp với nhau qua socket, hoặc một tiện ích giao tiếp nào đó của hệ điều hành như mailbox hay message queue.

Each local process may be a statically linked monolith, or it may be composed of dynamically linked deployment components. In the former case, several monolithic processes may have the same components compiled and linked into them. In the latter, they may share the same dynamically linked deployment components.

Mỗi tiến trình cục bộ có thể là một monolith liên kết tĩnh, hoặc có thể được ghép từ các component triển khai liên kết động. Ở trường hợp thứ nhất, nhiều tiến trình monolith có thể cùng biên dịch và liên kết những component giống nhau vào bên trong mình. Ở trường hợp thứ hai, chúng có thể dùng chung những component triển khai liên kết động giống nhau.

Think of a local process as a kind of uber-component: The process consists of lower-level components that manage their dependencies through dynamic polymorphism.

Hãy hình dung tiến trình cục bộ như một thứ siêu component: tiến trình đó gồm các component mức thấp hơn, và những component này quản lý phụ thuộc của chúng bằng đa hình động.

The segregation strategy between local processes is the same as for monoliths and binary components. Source code dependencies point in the same direction across the boundary, and always toward the higher-level component.

Chiến lược phân tách giữa các tiến trình cục bộ cũng giống như với monolith và component nhị phân. Các phụ thuộc mã nguồn vượt ranh giới đều chỉ về cùng một hướng, và luôn hướng về component ở mức cao hơn.

For local processes, this means that the source code of the higher-level processes must not contain the names, or physical addresses, or registry lookup keys of lower-level processes. Remember that the architectural goal is for lower-level processes to be plugins to higher-level processes.

Với tiến trình cục bộ, điều này có nghĩa là mã nguồn của tiến trình mức cao hơn không được chứa tên, địa chỉ vật lý, hay khóa tra cứu trong registry của các tiến trình mức thấp hơn. Hãy nhớ rằng mục tiêu kiến trúc là để tiến trình mức thấp trở thành plugin của tiến trình mức cao.

Communication across local process boundaries involve operating system calls, data marshaling and decoding, and interprocess context switches, which are moderately expensive. Chattiness should be carefully limited.

Giao tiếp qua ranh giới tiến trình cục bộ kéo theo lời gọi hệ điều hành, marshal và giải mã dữ liệu, cùng những lần chuyển ngữ cảnh giữa các tiến trình — tất cả đều tốn kém ở mức vừa phải. Cần hạn chế “lảm lòi” một cách cẩn thận.

Services — Dịch vụ

The strongest boundary is a service. A service is a process, generally started from the command line or through an equivalent system call. Services do not depend on their physical location. Two communicating services may, or may not, operate in the same physical processor or multicore. The services assume that all communications take place over the network.

Ranh giới mạnh nhất là dịch vụ (service). Một dịch vụ là một tiến trình, thường được khởi động từ dòng lệnh hoặc qua một lời gọi hệ thống tương đương. Dịch vụ không phụ thuộc

vào vị trí vật lý của nó. Hai dịch vụ đang giao tiếp với nhau có thể chạy trên cùng một bộ xử lý vật lý hay cùng một chip đa nhân, mà cũng có thể không. Các dịch vụ mặc định rằng mọi giao tiếp đều diễn ra qua mạng.

Communications across service boundaries are very slow compared to function calls. Turnaround times can range from tens of milliseconds to seconds. Care must be taken to avoid chatting where possible. Communications at this level must deal with high levels of latency.

Giao tiếp qua ranh giới dịch vụ chậm hơn rất nhiều so với lời gọi hàm. Thời gian phản hồi có thể dao động từ vài chục mili-giây đến vài giây. Phải hết sức tránh “lầm lờ” ở những chỗ có thể. Giao tiếp ở cấp này buộc phải sống chung với độ trễ cao.

Otherwise, the same rules apply to services as apply to local processes. Lower-level services should “plug in” to higher-level services. The source code of higher-level services must not contain any specific physical knowledge (e.g., a URI) of any lower-level service.

Ngoài ra, những quy tắc áp dụng cho dịch vụ cũng chính là những quy tắc áp dụng cho tiến trình cục bộ. Dịch vụ mức thấp nên “cắm vào” dịch vụ mức cao như một plugin. Mã nguồn của dịch vụ mức cao không được chứa bất kỳ hiểu biết vật lý cụ thể nào (chẳng hạn một URI) về bất kỳ dịch vụ mức thấp nào.

Conclusion — Kết luận

Most systems, other than monoliths, use more than one boundary strategy. A system that makes use of service boundaries may also have some local process boundaries. Indeed, a service is often just a facade for a set of interacting local processes. A service, or a local process, will almost certainly be either a monolith composed of source code components or a set of dynamically linked deployment components.

Phần lớn hệ thống, trừ monolith, đều dùng nhiều hơn một chiến lược ranh giới. Một hệ thống có ranh giới dịch vụ cũng có thể có vài ranh giới tiến trình cục bộ. Thật vậy, một dịch vụ thường chỉ là bộ mặt tiền của một tập các tiến trình cục bộ đang tương tác với nhau. Còn một dịch vụ, hay một tiến trình cục bộ, gần như chắc chắn hoặc là một monolith ghép từ các component mã nguồn, hoặc là một tập các component triển khai liên kết động.

This means that the boundaries in a system will often be a mixture of local chatty boundaries and boundaries that are more concerned with latency.

Điều này có nghĩa là các ranh giới trong một hệ thống thường sẽ là hỗn hợp giữa những ranh giới cục bộ “lầm lờ” và những ranh giới bận tâm nhiều hơn tới độ trễ.

Policy and Level — Chính sách và cấp

Software systems are statements of **policy**. Indeed, at its core, that's all a computer program actually is. A computer program is a detailed description of the policy by which inputs are transformed into outputs.

Hệ thống phần mềm là những phát biểu về chính sách (policy). Thật ra, ở cốt lõi, một chương trình máy tính chỉ có vậy mà thôi. Một chương trình máy tính là bản mô tả chi tiết cái chính sách dùng để biến đầu vào thành đầu ra.

In most nontrivial systems, that policy can be broken down into many different smaller statements of policy. Some of those statements will describe how particular business rules are to be calculated. Others will describe how certain reports are to be formatted. Still others will describe how input data are to be validated.

Trong hầu hết các hệ thống không tầm thường, chính sách đó có thể được chẻ nhỏ thành rất nhiều phát biểu chính sách con khác nhau. Một số phát biểu sẽ mô tả cách tính toán những quy tắc nghiệp vụ (business rule) cụ thể. Số khác mô tả cách định dạng một số báo cáo nhất định. Số khác nữa lại mô tả cách kiểm tra tính hợp lệ của dữ liệu đầu vào.

Part of the art of developing a software **architecture** is carefully separating those policies from one another, and regrouping them based on the ways that they change. Policies that change for the same reasons, and at the same times, are at the same **level** and belong together in the same **component**. Policies that change for different reasons, or at different times, are at different levels and should be separated into different components.

Một phần của nghệ thuật phát triển kiến trúc phần mềm là tách bạch cẩn thận các chính sách đó khỏi nhau, rồi gom nhóm lại theo cách chúng thay đổi. Những chính sách thay đổi vì cùng một lý do, và vào cùng một thời điểm, thì nằm cùng một cấp (level) và thuộc về cùng một component. Những chính sách thay đổi vì lý do khác nhau, hoặc vào những thời điểm khác nhau, thì nằm ở các cấp khác nhau và nên được tách ra thành các component khác nhau.

The art of architecture often involves forming the regrouped components into a directed acyclic graph. The nodes of the graph are the components that contain policies at the same level. The directed edges are the dependencies between those components. They connect components that are at different levels.

Nghệ thuật kiến trúc thường bao gồm việc sắp các component đã gom nhóm ấy thành một đồ thị có hướng không chu trình. Các nút của đồ thị là những component chứa các chính sách cùng cấp. Các cạnh có hướng là những phụ thuộc giữa các component đó. Chúng nối các component nằm ở những cấp khác nhau.

Those dependencies are source code, compile-time dependencies. In Java, they are import statements. In C#, they are using statements. In Ruby, they are require statements. They are the dependencies that are necessary for the compiler to function.

Những phụ thuộc ấy là phụ thuộc mã nguồn, phụ thuộc lúc biên dịch. Trong Java, chúng là các câu lệnh import. Trong C#, chúng là các câu lệnh using. Trong Ruby, chúng là các câu lệnh require. Đó là những phụ thuộc mà trình biên dịch cần có để làm việc.

In a good architecture, the direction of those dependencies is based on the level of the components that they connect. In every case, **low-level** components are designed so that they depend on **high-level** components.

Trong một kiến trúc tốt, chiều của những phụ thuộc đó dựa trên cấp của các component mà chúng nối lại. Trong mọi trường hợp, các component mức thấp được thiết kế sao cho chúng phụ thuộc vào các component mức cao.

Level — Cấp

A strict definition of “level” is “the distance from the inputs and outputs.” The farther a policy is from both the inputs and the outputs of the system, the higher its level. The policies that manage input and output are the lowest-level policies in the system.

Định nghĩa chặt chẽ của “cấp” là “khoảng cách tới đầu vào và đầu ra”. Một chính sách càng xa cả đầu vào lẫn đầu ra của hệ thống thì cấp của nó càng cao. Những chính sách lo việc đầu vào và đầu ra là các chính sách thấp cấp nhất trong hệ thống.

The data flow diagram in Figure 19.1 depicts a simple encryption program that reads characters from an input device, translates the characters using a

Sơ đồ luồng dữ liệu ở Figure 19.1 mô tả một chương trình mã hoá đơn giản: nó đọc các ký tự từ một thiết bị đầu vào, chuyển đổi các ký tự đó bằng một

table, and then writes the translated characters to an output device. The data flows are shown as curved solid arrows. The properly designed source code dependencies are shown as straight dashed lines.

bảng tra, rồi ghi các ký tự đã chuyển đổi ra một thiết bị đầu ra. Các luồng dữ liệu được vẽ bằng mũi tên cong nét liền. Các phụ thuộc mã nguồn được thiết kế đúng đắn được vẽ bằng đường thẳng nét đứt.

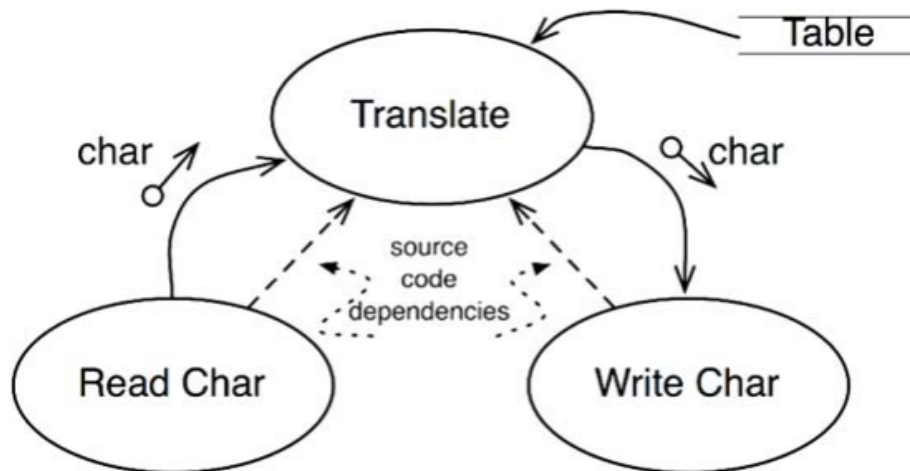


Figure 19.1 A simple encryption program

Figure 19.1 Một chương trình mã hoá đơn giản

The Translate component is the highest-level component in this system because it is the component that is farthest from the inputs and outputs. 1

Component Translate là component cao cấp nhất trong hệ thống này, bởi nó là component nằm xa đầu vào và đầu ra nhất. 1

Note that the data flows and the source code dependencies do not always point in the same direction. This, again, is part of the art of software architecture. We want source code dependencies to be decoupled from data flow and coupled to level .

Hãy để ý rằng luồng dữ liệu và phụ thuộc mã nguồn không phải lúc nào cũng chỉ về cùng một hướng. Đây, một lần nữa, chính là một phần của nghệ thuật kiến trúc phần mềm. Chúng ta muốn các phụ thuộc mã nguồn tách rời khỏi luồng dữ liệu và gắn với cấp .

It would be easy to create an incorrect architecture by writing the encryption program like this:

Sẽ rất dễ tạo ra một kiến trúc sai lầm nếu viết chương trình mã hoá như thế này:

```

function encrypt() {
  while(true)
    writeChar(translate(readChar()));
}
  
```

- 1. Meilir Page-Jones called this component the “Central Transform” in his book The Practical Guide to
 - 1. Meilir Page-Jones gọi component này là “Central Transform” trong cuốn The Practical Guide to

Structured Systems **Design** , 2nd ed. (Yourdon Press, 1988).

Structured Systems Design , ấn bản 2 (Yourdon Press, 1988).

This is incorrect architecture because the high-level encrypt function depends on the lower-level readChar and writeChar functions.

Đây là kiến trúc sai, bởi hàm encrypt ở mức cao lại phụ thuộc vào các hàm readChar và writeChar ở mức thấp hơn.

A better architecture for this system is shown in the class diagram in Figure 19.2. Note the dashed border surrounding the Encrypt class, and the CharWriter and CharReader interfaces. All dependencies crossing that border point inward. This unit is the highest-level element in the system.

Một kiến trúc tốt hơn cho hệ thống này được thể hiện trong sơ đồ lớp ở Figure 19.2. Hãy để ý đường viền nét đứt bao quanh class Encrypt cùng các interface CharWriter và CharReader. Mọi phụ thuộc vượt qua đường viền đó đều chỉ vào trong. Đơn vị này là phần tử cao cấp nhất trong hệ thống.

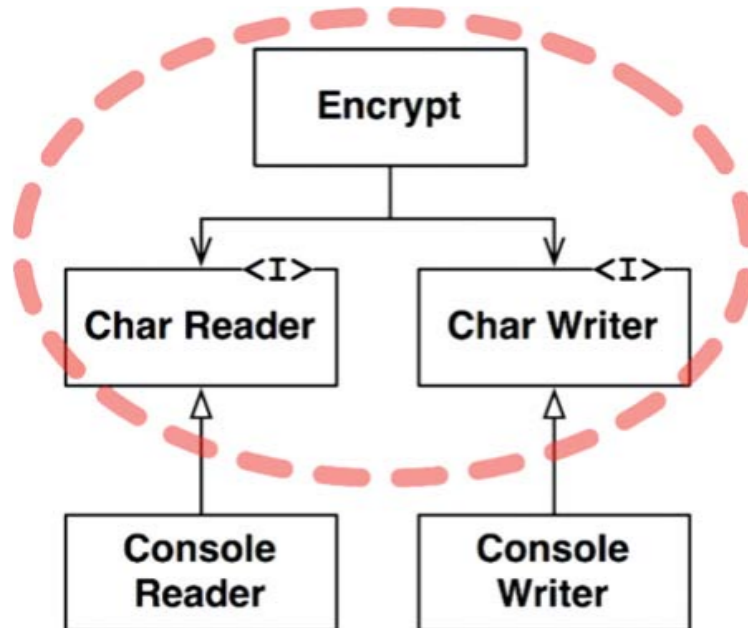


Figure 19.2 Class diagram showing a better architecture for the system

Figure 19.2 Sơ đồ lớp cho thấy một kiến trúc tốt hơn cho hệ thống

ConsoleReader and ConsoleWriter are shown here as classes. They are low level because they are close to the inputs and outputs.

ConsoleReader và ConsoleWriter ở đây được vẽ dưới dạng các class. Chúng ở mức thấp vì chúng gần với đầu vào và đầu ra.

Note how this **structure** decouples the high-level encryption policy from the lower-level input/output policies. This makes the encryption policy usable in a wide range of contexts. When changes are made to the input and output policies, they are not likely to affect the encryption policy.

Hãy để ý cấu trúc này tách rời chính sách mã hoá mức cao khỏi các chính sách vào/ra mức thấp hơn như thế nào. Nhờ vậy, chính sách mã hoá có thể dùng được trong rất nhiều ngữ cảnh khác nhau. Khi các chính sách vào và ra thay đổi, chúng khó lòng ảnh hưởng tới chính sách mã hoá.

Recall that policies are grouped into components based on the way that they change. Policies that change for the same reasons and at the same times are grouped together by the SRP and CCP. Higher-

level policies—those that are farthest from the inputs and outputs—tend to change less frequently, and for

Hãy nhớ lại rằng các chính sách được gom vào component theo cách chúng thay đổi. Những chính sách thay đổi vì cùng lý do và vào cùng thời điểm thì được gom lại với nhau theo SRP và CCP. Các chính sách cao cấp hơn — những chính sách nằm xa đầu vào và đầu ra nhất — thường ít thay đổi hơn, và khi thay đổi thì vì

more **important** reasons, than lower-level policies. Lower-level policies—those that are closest to the inputs and outputs—tend to change frequently, and with more urgency, but for less important reasons.

những lý do quan trọng hơn, so với các chính sách thấp cấp hơn. Các chính sách thấp cấp hơn — những chính sách gần đầu vào và đầu ra nhất — thường xuyên thay đổi, và thay đổi gấp gáp hơn, nhưng vì những lý do kém quan trọng hơn.

For example, even in the trivial example of the encryption program, it is far more likely that the IO devices will change than that the encryption algorithm will change. If the encryption algorithm does change, it will likely be for a more substantive reason than a change to one of the IO devices.

Chẳng hạn, ngay cả trong ví dụ tầm thường là chương trình mã hoá kia, khả năng các thiết bị IO thay đổi vẫn cao hơn nhiều so với khả năng thuật toán mã hoá thay đổi. Còn nếu thuật toán mã hoá có thay đổi thật, hẳn là vì một lý do thực chất hơn nhiều so với việc đổi một trong các thiết bị IO.

Keeping these policies separate, with all source code dependencies pointing in the direction of the higher-level policies, reduces the impact of change. Trivial but **urgent** changes at the lowest levels of the system have little or no impact on the higher, more important, levels.

Giữ những chính sách này tách bạch, với mọi phụ thuộc mã nguồn đều chỉ về phía các chính sách cao cấp hơn, sẽ làm giảm tác động của thay đổi. Những thay đổi tầm thường nhưng gấp gáp ở các cấp thấp nhất của hệ thống hầu như không ảnh hưởng, hoặc chẳng ảnh hưởng gì, tới các cấp cao hơn và quan trọng hơn.

Another way to look at this issue is to note that lower-level components should be plugins to the higher-level components. The component diagram in Figure 19.3 shows this arrangement. The Encryption component knows nothing of the IODevices component; the IODevices component depends

Một cách nhìn khác về vấn đề này là: các component mức thấp nên là plugin cắm vào các component mức cao. Sơ đồ component ở Figure 19.3 cho thấy cách sắp xếp đó. Component Encryption chẳng biết gì về component IODevices; chính component IODevices mới phụ thuộc

on the Encryption component.

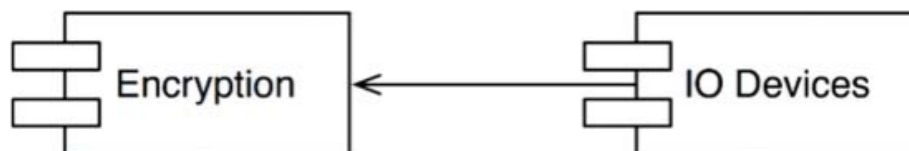


Figure 19.3 Lower-level components should plug in to higher-level components

Figure 19.3 Các component mức thấp nên cắm vào các component mức cao

Conclusion — Kết luận

At this point, this discussion of policies has involved a mixture of the Single Responsibility Principle, the Open-Closed Principle, the Common Closure Principle, the **Dependency Inversion** Principle, the **Stable** Dependencies Principle, and the Stable Abstractions Principle. Look back and see if you can identify where each principle was used, and why.

Đến đây, cuộc bàn luận về chính sách đã pha trộn Single Responsibility Principle, Open-Closed Principle, Common Closure Principle, Dependency Inversion Principle, Stable Dependencies Principle và Stable Abstractions Principle. Hãy nhìn lại và thử xem bạn có nhận ra mỗi nguyên tắc đã được dùng ở đâu, và vì sao không.

Business Rules — Quy tắc nghiệp vụ

If we are going to divide our application into business rules and plugins, we'd better get a good grasp on just what business rules actually are. It turns out there are several different kinds.

Nếu định chia ứng dụng thành quy tắc nghiệp vụ (business rule) và plugin, tốt hơn hết chúng ta nên nắm cho thật chắc quy tắc nghiệp vụ thực sự là gì. Hoá ra chúng có mấy loại khác nhau.

Strictly speaking, business rules are rules or procedures that make or save the business money. Very strictly speaking, these rules would make or save the business money, irrespective of whether they were implemented on a computer. They would make or save money even if they were executed manually.

Nói cho chặt chẽ, quy tắc nghiệp vụ là những quy tắc hoặc thủ tục làm ra tiền hoặc tiết kiệm tiền cho doanh nghiệp. Nói cho thật chặt chẽ, những quy tắc này làm ra tiền hoặc tiết kiệm tiền cho doanh nghiệp bất kể chúng có được cài đặt trên máy tính hay không. Chúng vẫn làm ra tiền hoặc tiết kiệm tiền ngay cả khi được thực thi thủ công.

The fact that a bank charges $N\%$ interest for a loan is a **business rule** that makes the bank money. It doesn't matter if a computer program calculates the interest, or if a clerk with an abacus calculates the interest.

Chuyện một ngân hàng tính lãi $N\%$ cho một khoản vay là một quy tắc nghiệp vụ làm ra tiền cho ngân hàng. Chẳng quan trọng việc tiền lãi được tính bởi một chương trình máy tính hay bởi một nhân viên với cái bàn tính.

We shall call these rules Critical Business Rules, because they are critical to the business itself, and would exist even if there were no system to automate them.

Ta sẽ gọi những quy tắc này là quy tắc nghiệp vụ cốt lõi (Critical Business Rules), bởi chúng cốt lõi với chính bản thân doanh nghiệp, và sẽ tồn tại ngay cả khi không có hệ thống nào tự động hoá chúng.

Critical Business Rules usually require some data to work with. For example, our loan requires a loan balance, an interest rate, and a payment schedule.

Quy tắc nghiệp vụ cốt lõi thường cần dữ liệu để làm việc. Chẳng hạn, khoản vay của ta cần số dư khoản vay, lãi suất và lịch trả nợ.

We shall call this data **Critical Business Data**. This is the data that would exist even if the system were not automated.

Ta sẽ gọi dữ liệu này là dữ liệu nghiệp vụ cốt lõi (Critical Business Data). Đây là dữ liệu vẫn tồn tại ngay cả khi hệ thống không được tự động hoá.

The critical rules and critical data are inextricably bound, so they are a good candidate for an object. We'll call this kind of object an **Entity** . 1

Quy tắc cốt lõi và dữ liệu cốt lõi ràng buộc với nhau đến mức không tách rời được, nên chúng là ứng viên tuyệt vời cho một đối tượng. Ta sẽ gọi loại đối tượng này là Entity. 1

Entities — Entity

An Entity is an object within our computer system that embodies a small set of critical business rules operating on Critical Business Data. The Entity object either contains the Critical Business Data or has very easy access to that data. The **interface** of the Entity consists of the functions that implement the Critical Business Rules that operate on that data.

Một Entity là một đối tượng trong hệ thống máy tính của ta, hiện thân cho một nhóm nhỏ các quy tắc nghiệp vụ cốt lõi thao tác trên dữ liệu nghiệp vụ cốt lõi. Đối tượng Entity hoặc chứa luôn dữ liệu nghiệp vụ cốt lõi, hoặc truy cập được dữ liệu đó rất dễ dàng. Interface của Entity gồm các hàm cài đặt những quy tắc nghiệp vụ cốt lõi thao tác trên dữ liệu ấy.

- 1. This is Ivar Jacobson's name for this concept (I. Jacobson et al., Object Oriented Software Engineering ,
 - 1. Đây là tên gọi mà Ivar Jacobson đặt cho khái niệm này (I. Jacobson và cộng sự, Object Oriented Software Engineering ,

Addison-Wesley, 1992).

Addison-Wesley, 1992).

For example, Figure 20.1 shows what our Loan entity might look like as a class in UML. It has three pieces of Critical Business Data, and presents three related Critical Business Rules at its interface.

Ví dụ, Figure 20.1 cho thấy Entity Loan của ta có thể trông thế nào khi là một class trong UML. Nó có ba mẫu dữ liệu nghiệp vụ cốt lõi, và phơi ra ba quy tắc nghiệp vụ cốt lõi tương ứng ở interface.

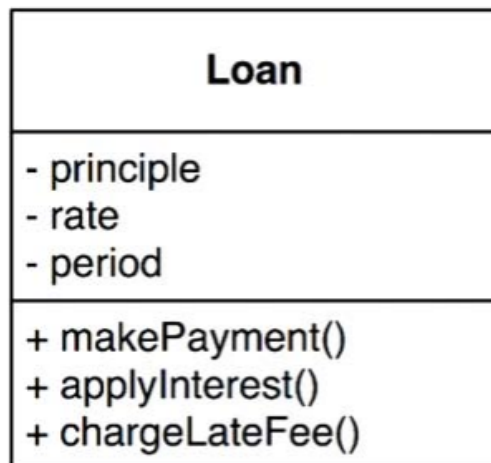


Figure 20.1 Loan entity as a class in UML

Figure 20.1 Entity Loan dưới dạng một class trong UML

When we create this kind of class, we are gathering together the software that implements a concept that is critical to the business, and separating it from every other concern in the automated system we are building. This class stands alone as a representative of the business. It is unsullied with concerns about databases, user interfaces, or third-party frameworks. It could serve the business in any system, irrespective of how that system was presented, or how the data was stored, or how the computers in that system were arranged. The Entity is pure business and nothing else .

Khi tạo ra một class kiểu này, ta đang gom lại phần phần mềm cài đặt một khái niệm cốt lõi với doanh nghiệp, rồi tách nó khỏi mọi mối bận tâm khác trong hệ thống tự động hoá mà ta đang xây. Class này đứng một mình như đại diện của nghiệp vụ. Nó không bị vấy bẩn bởi những chuyện cơ sở dữ liệu, giao diện người dùng hay framework của bên thứ ba. Nó có thể phục vụ doanh nghiệp trong bất kỳ hệ thống nào, bất kể hệ thống đó được trình bày ra sao, dữ liệu được lưu thế nào, hay máy tính trong hệ thống đó được bố trí kiểu gì. Entity là nghiệp vụ thuần khiết, không gì khác.

Some of you may be concerned that I called it a class. Don't be. You don't need to use an object-oriented language to create an Entity. All that is required is that you bind the Critical Business Data and the Critical Business Rules together in a single and separate software module.

Có thể vài người trong các bạn thấy gợn khi tôi gọi nó là một class. Đừng lo. Bạn không cần dùng ngôn ngữ hướng đối tượng mới tạo được Entity. Tất cả những gì cần là buộc dữ liệu nghiệp vụ cốt lõi và quy tắc nghiệp vụ cốt lõi lại với nhau trong một module phần mềm duy nhất và tách biệt.

Use Cases — Use case

Not all business rules are as pure as Entities. Some business rules make or save money for the business by defining and constraining the way that an automated system operates. These rules would not be used in a manual environment, because they make sense only as part of an automated system.

Không phải quy tắc nghiệp vụ nào cũng thuần khiết như Entity. Một số quy tắc nghiệp vụ làm ra tiền hoặc tiết kiệm tiền cho doanh nghiệp bằng cách định nghĩa và ràng buộc cách vận hành của một hệ thống tự động hoá. Những quy tắc này sẽ chẳng dùng được trong môi trường thủ công, vì chúng chỉ có nghĩa khi là một phần của hệ thống tự động hoá.

For example, imagine an application that is used by bank officers to create a new loan. The bank may decide that it does not want the loan officers to offer loan payment estimates until they have first gathered, and validated, contact information and ensured that the candidate's credit score is 500 or higher. For this reason, the bank may specify that the system will not proceed to the payment estimation screen until the contact information screen has been filled out and verified, and the credit score has been confirmed to be greater than the cutoff.

Chẳng hạn, hãy hình dung một ứng dụng được các cán bộ ngân hàng dùng để tạo một khoản vay mới. Ngân hàng có thể quyết định rằng họ không muốn cán bộ tín dụng đưa ra ước tính khoản trả nợ trước khi đã thu thập và xác thực thông tin liên hệ, đồng thời bảo đảm điểm tín dụng của ứng viên từ 500 trở lên. Vì vậy, ngân hàng có thể quy định rằng hệ thống sẽ không chuyển sang màn hình ước tính khoản trả nợ cho tới khi màn hình thông tin liên hệ đã được điền và kiểm chứng, và điểm tín dụng đã được xác nhận là lớn hơn ngưỡng.

This is a **use case** . 2 A use case is a description of the way that an automated system is used. It specifies the input to be provided by the user, the output to be returned to the user, and the processing

steps involved in producing that output. A use case describes application-specific business rules as opposed to the Critical Business Rules within the Entities.

Đây là một use case. 2 Use case là mô tả về cách một hệ thống tự động hoá được sử dụng. Nó đặc tả đầu vào mà người dùng phải cung cấp, đầu ra sẽ trả về cho người dùng, và các bước xử lý để tạo ra đầu ra đó. Use case mô tả các quy tắc nghiệp vụ đặc thù cho ứng dụng, khác với quy tắc nghiệp vụ cốt lõi nằm trong các Entity.

Figure 20.2 shows an example of a use case. Notice that in the last line it mentions the Customer. This is a reference to the Customer entity, which contains the Critical Business Rules that govern the relationship between the bank and its customers.

Figure 20.2 cho thấy một ví dụ về use case. Để ý rằng ở dòng cuối nó nhắc tới Customer. Đó là tham chiếu tới Entity Customer, nơi chứa các quy tắc nghiệp vụ cốt lõi chi phối quan hệ giữa ngân hàng và khách hàng.



Figure 20.2 Example use case

Figure 20.2 Ví dụ về use case

- 2. Ibid.
- 2. Sách đã dẫn.

Use cases contain the rules that specify how and when the Critical Business Rules within the Entities are invoked. Use cases control the dance of the Entities.

Use case chứa các quy tắc đặc tả rằng quy tắc nghiệp vụ cốt lõi bên trong Entity được gọi ra như thế nào và khi nào. Use case điều khiển vũ điệu của các Entity.

Notice also that the use case does not describe the user interface other than to informally specify the data coming in from that interface, and the data going back out through that interface. From the use case, it is impossible to tell whether the application is delivered on the web, or on a thick client, or on a console, or is a pure **service**.

Cũng để ý rằng use case không mô tả giao diện người dùng, ngoài việc đặc tả một cách phi hình thức dữ liệu đi vào từ giao diện đó và dữ liệu đi ngược trở ra qua giao diện đó.

Nhìn vào use case, ta không thể biết được ứng dụng sẽ chạy trên web, trên thick client, trên console, hay là một dịch vụ thuần túy.

This is very **important**. Use cases do not describe how the system appears to the user. Instead, they describe the application-specific rules that govern the interaction between the users and the Entities. How the data gets in and out of the system is irrelevant to the use cases.

Điều này rất quan trọng. Use case không mô tả hệ thống hiện ra trước mắt người dùng thế nào. Thay vào đó, chúng mô tả các quy tắc đặc thù cho ứng dụng, những quy tắc chi phối tương tác giữa người dùng và các Entity. Dữ liệu vào ra hệ thống bằng cách nào là chuyện chẳng liên quan tới use case.

A use case is an object. It has one or more functions that implement the application-specific business rules. It also has data elements that include the input data, the output data, and the references to the appropriate Entities with which it interacts.

Use case là một đối tượng. Nó có một hoặc nhiều hàm cài đặt các quy tắc nghiệp vụ đặc thù cho ứng dụng. Nó cũng có các phần tử dữ liệu, gồm dữ liệu đầu vào, dữ liệu đầu ra, và tham chiếu tới những Entity thích hợp mà nó tương tác cùng.

Entities have no knowledge of the use cases that control them. This is another example of the direction of the dependencies following the **Dependency Inversion** Principle. **High-level** concepts, such as Entities, know nothing of lower-level concepts, such as use cases. Instead, the lower-level use cases know about the higher-level Entities.

Entity không hề hay biết gì về những use case đang điều khiển chúng. Đây lại là một ví dụ nữa về hướng của các phụ thuộc tuân theo Dependency Inversion Principle. Những khái niệm mức cao, như Entity, chẳng biết gì về những khái niệm mức thấp hơn, như use case. Ngược lại, use case ở mức thấp hơn thì biết về Entity ở mức cao hơn.

Why are Entities high level and use cases lower level? Because use cases are specific to a single application and, therefore, are closer to the inputs and outputs of that system. Entities are generalizations that can be used in many different applications, so they are farther from the inputs and outputs of the system. Use cases depend on Entities; Entities do not depend on use cases.

Tại sao Entity lại ở mức cao còn use case ở mức thấp hơn? Bởi use case đặc thù cho một ứng dụng duy nhất, và do đó gần với đầu vào và đầu ra của hệ thống đó hơn. Entity là những khái quát hoá có thể dùng trong nhiều ứng dụng khác nhau, nên chúng xa đầu vào và đầu ra của hệ thống hơn. Use case phụ thuộc vào Entity; Entity không phụ thuộc vào use case.

Request and Response Models — Mô hình request và response

Use cases expect input data, and they produce output data. However, a well- formed use case object should have no inkling about the way that data is

Use case mong đợi dữ liệu đầu vào, và chúng tạo ra dữ liệu đầu ra. Tuy nhiên, một đối tượng use case được dựng đúng cách thì không được có mảy may ý niệm nào về cách dữ liệu đó được

communicated to the user, or to any other **component**. We certainly don't want the code within the use case class to know about HTML or SQL!

truyền tới người dùng, hay tới bất kỳ component nào khác. Chắc chắn ta không muốn mã bên trong class use case biết gì về HTML hay SQL!

The use case class accepts simple **request** data structures for its input, and returns simple response data structures as its output. These data structures are not dependent on anything. They do not derive from standard **framework** interfaces such as `HttpRequest` and `HttpResponse`. They know nothing of the web, nor do they share any of the trappings of whatever user interface might be in place.

Class use case nhận vào những cấu trúc dữ liệu request đơn giản, và trả về những cấu trúc dữ liệu response đơn giản. Các cấu trúc dữ liệu này không phụ thuộc vào bất cứ thứ gì. Chúng không dẫn xuất từ các interface chuẩn của framework như `HttpRequest` hay `HttpResponse`. Chúng chẳng biết gì về web, cũng không mang theo bất kỳ thứ trang sức nào của giao diện người dùng đang dùng.

This lack of dependencies is critical. If the request and response models are not independent, then the use cases that depend on them will be indirectly bound to whatever dependencies the models carry with them.

Sự vắng mặt của các phụ thuộc này là điều cốt yếu. Nếu mô hình request và response không độc lập, thì những use case phụ thuộc vào chúng sẽ bị buộc gián tiếp vào mọi phụ thuộc mà các mô hình ấy mang theo.

You might be tempted to have these data structures contain references to Entity objects. You might think this makes sense because the Entities and the request/response models share so much data. Avoid this temptation! The purpose of these two objects is very different. Over time they will change for very different reasons, so tying them together in any way violates the Common Closure and Single Responsibility Principles. The result would be lots of tramp data, and lots of conditionals in your code.

Bạn có thể bị cám dỗ cho những cấu trúc dữ liệu này chứa tham chiếu tới các đối tượng Entity. Bạn có thể nghĩ thế là hợp lý vì Entity và mô hình request/response dùng chung quá nhiều dữ liệu. Hãy tránh cám dỗ đó! Mục đích của hai loại đối tượng này rất khác nhau. Theo thời gian, chúng sẽ thay đổi vì những lý do rất khác nhau, nên trói chúng vào nhau theo bất kỳ cách nào cũng vi phạm Common Closure Principle và Single Responsibility Principle. Kết quả sẽ là hàng đống dữ liệu bị chuyển tay vô ích, và hàng đống câu điều kiện trong mã của bạn.

Conclusion — Kết luận

Business rules are the reason a software system exists. They are the core functionality. They carry the code that makes, or saves, money. They are the family jewels.

Quy tắc nghiệp vụ là lý do tồn tại của một hệ thống phần mềm. Chúng là chức năng cốt lõi. Chúng mang trong mình đoạn mã làm ra tiền, hoặc tiết kiệm tiền. Chúng là báu vật gia truyền.

The business rules should remain pristine, unsullied by baser concerns such as the user interface or database used. Ideally, the code that represents the business rules should be the heart of the system, with lesser concerns being plugged in to them. The business rules should be the most independent and reusable code in the system.

Quy tắc nghiệp vụ phải luôn tinh khôi, không bị vấy bẩn bởi những mối bận tâm tầm thường hơn như giao diện người dùng hay cơ sở dữ liệu đang dùng. Lý tưởng nhất, đoạn mã biểu diễn quy tắc nghiệp vụ phải là trái tim của hệ thống, còn những mối bận tâm nhỏ hơn thì được cắm vào nó. Quy tắc nghiệp vụ phải là phần mã độc lập nhất và tái dùng được nhiều nhất trong hệ thống.

Screaming Architecture — Kiến trúc biết “gào lên” (Screaming Architecture)

Imagine that you are looking at the blueprints of a building. This document, prepared by an architect, provides the plans for the building. What do these plans tell you?

Hãy tưởng tượng bạn đang nhìn vào bản vẽ thiết kế của một toà nhà. Tài liệu này do một kiến trúc sư lập ra, nó trình bày phương án cho toà nhà đó. Những bản vẽ ấy nói với bạn điều gì?

If the plans you are viewing are for a single-family residence, then you’ll likely see a front entrance, a foyer leading to a living room, and perhaps a dining room. There will likely be a kitchen a short distance away, close to the dining room. Perhaps there is a dinette area next to the kitchen, and probably a family room close to that. When you looked at those plans, there would be no question that you were looking at a single family home. The **architecture** would scream: “HOME.”

Nếu bản vẽ bạn đang xem là của một căn nhà ở dành cho một hộ gia đình, hẳn bạn sẽ thấy một lối vào chính, một tiền sảnh dẫn vào phòng khách, và có lẽ cả một phòng ăn. Nhiều khả năng sẽ có một gian bếp ngay gần đó, sát bên phòng ăn. Có thể có một góc ăn sáng cạnh bếp, và chắc cũng có một phòng sinh hoạt chung ở gần đấy. Khi nhìn vào những bản vẽ đó, bạn chẳng cần phải hỏi: rõ ràng đây là nhà ở cho một hộ gia đình. Kiến trúc ấy gào lên: “NHÀ Ở.”

Now suppose you were looking at the architecture of a library. You would likely see a grand entrance, an area for check-in/out clerks, reading areas, small conference rooms, and gallery after gallery capable of holding bookshelves for all the books in the library. That architecture would scream: “LIBRARY.”

Bây giờ giả sử bạn đang xem kiến trúc (architecture) của một thư viện. Hẳn bạn sẽ thấy một lối vào bề thế, một khu vực cho nhân viên làm thủ tục mượn/trả, các khu đọc sách, những phòng họp nhỏ, rồi hết gian này tới gian khác đủ sức chứa giá sách cho toàn bộ sách của thư viện. Kiến trúc ấy gào lên: “THƯ VIỆN.”

So what does the architecture of your application scream? When you look at the top-**level** directory **structure**, and the source files in the highest-level package, do they scream “Health Care System,” or “Accounting System,” or “Inventory Management System”? Or do they scream “Rails,” or “Spring/Hibernate,” or “ASP”?

Vậy kiến trúc của ứng dụng của bạn gào lên điều gì? Khi bạn nhìn vào cấu trúc thư mục ở mức cao nhất, và các tệp mã nguồn trong package cấp cao nhất, chúng có gào lên “Hệ thống

Chăm sóc Sức khỏe,” hay “Hệ thống Kế toán,” hay “Hệ thống Quản lý Kho” hay không? Hay chúng gào lên “Rails,” hoặc “Spring/Hibernate,” hoặc “ASP”?

The Theme of an Architecture — Chủ đề của một kiến trúc

Go back and read Ivar Jacobson’s seminal work on software architecture: *Object Oriented Software Engineering*. Notice the subtitle of the book: *A Use Case Driven Approach*. In this book Jacobson makes the point that software architectures are structures that support the use cases of the system. Just as the plans for a house or a library scream about the use cases of those buildings, so should the architecture of a software application scream about the use cases of the application.

Hãy tìm đọc lại công trình kinh điển của Ivar Jacobson về kiến trúc phần mềm: *Object Oriented Software Engineering*. Để ý phụ đề của cuốn sách: *A Use Case Driven Approach*. Trong sách đó, Jacobson nhấn mạnh rằng kiến trúc phần mềm là những cấu trúc (structure) nâng đỡ các use case của hệ thống. Cũng như bản vẽ của một căn nhà hay một thư viện gào lên các use case của toà nhà ấy, kiến trúc của một ứng dụng phần mềm cũng phải gào lên các use case của ứng dụng.

Architectures are not (or should not be) about frameworks. Architectures should not be supplied by frameworks. Frameworks are tools to be used, not architectures to be conformed to. If your architecture is based on frameworks, then it cannot be based on your use cases.

Kiến trúc không phải là (hoặc không nên là) chuyện framework. Kiến trúc không nên do framework cung cấp. Framework là công cụ để dùng, không phải kiến trúc để tuân theo. Nếu kiến trúc của bạn dựa trên framework, thì nó không thể dựa trên các use case của bạn được.

The Purpose of an Architecture — Mục đích của một kiến trúc

Good architectures are centered on use cases so that architects can safely describe the structures that support those use cases without committing to frameworks, tools, and environments. Again, consider the plans for a house. The first concern of the architect is to make sure that the house is usable— not to ensure that the house is made of bricks. Indeed, the architect takes pains to ensure that the homeowner can make decisions about the exterior material (bricks, stone, or cedar) later, after the plans ensure that the use cases are met.

Kiến trúc tốt lấy use case làm trung tâm, nhờ đó kiến trúc sư có thể yên tâm mô tả những cấu trúc nâng đỡ các use case ấy mà không phải cam kết với framework, công cụ hay môi trường nào. Một lần nữa, hãy nghĩ về bản vẽ của một căn nhà. Mối quan tâm đầu tiên của kiến trúc sư là bảo đảm căn nhà dùng được — chứ không phải bảo đảm căn nhà được xây bằng gạch. Thật vậy, kiến trúc sư cất công lo sao cho chủ nhà có thể quyết định về vật liệu mặt ngoài (gạch, đá, hay gỗ tuyết tùng) muộn hơn, sau khi bản vẽ đã bảo đảm các use case được đáp ứng.

A good software architecture allows decisions about frameworks, databases, web servers, and other environmental issues and tools to be deferred and delayed. Frameworks are options to be left open. A good architecture makes it unnecessary to decide on Rails, or Spring, or Hibernate, or Tomcat, or MySQL, until much later in the project. A good architecture makes it easy to change your mind about those decisions, too. A good architecture emphasizes the use cases and decouples them from peripheral concerns.

Một kiến trúc phần mềm tốt cho phép hoãn lại và trì hoãn các quyết định về framework, cơ sở dữ liệu, web server, cùng những vấn đề môi trường và công cụ khác. Framework là những lựa chọn cần để ngỏ. Một kiến trúc tốt khiến bạn không cần phải chốt Rails, hay Spring, hay Hibernate, hay Tomcat, hay MySQL, cho tới tận rất muộn trong dự án. Một kiến trúc tốt cũng khiến việc đổi ý về những quyết định đó trở nên dễ dàng. Một kiến trúc tốt làm nổi bật các use case và tách rời (decoupling) chúng khỏi những mối bận tâm ngoại vi.

But What About the Web? — Thế còn web thì sao?

Is the web an architecture? Does the fact that your system is delivered on the web dictate the architecture of your system? Of course not! The web is a delivery mechanism—an IO device—and your application architecture should treat it as such. The fact that your application is delivered over the web is a **detail** and should not dominate your system structure. Indeed, the decision that your application will be delivered over the web is one that you should defer. Your system architecture should be as ignorant as possible about how it

Web có phải là một kiến trúc không? Việc hệ thống của bạn được phân phối qua web có quyết định kiến trúc của hệ thống không? Dĩ nhiên là không! Web là một cơ chế phân phối — một thiết bị IO — và kiến trúc ứng dụng của bạn phải đối xử với nó đúng như vậy. Chuyện ứng dụng của bạn được phân phối qua web là một chi tiết (detail) và không được phép chi phối cấu trúc hệ thống. Thật ra, quyết định rằng ứng dụng sẽ được phân phối qua web là một quyết định bạn nên hoãn lại. Kiến trúc hệ thống của bạn nên càng ít biết càng tốt về chuyện nó

will be delivered. You should be able to deliver it as a console app, or a web app, or a thick client app, or even a web **service** app, without undue complication or change to the fundamental architecture.

sẽ được phân phối ra sao. Bạn phải có khả năng phân phối nó dưới dạng ứng dụng console, hay ứng dụng web, hay ứng dụng thick client, hay thậm chí ứng dụng web service, mà không gặp phiền phức quá đáng nào và không phải thay đổi kiến trúc nền tảng.

Frameworks Are Tools, Not Ways of Life — Framework là công cụ, không phải lối sống

Frameworks can be very powerful and very useful. **Framework** authors often believe very deeply in their frameworks. The examples they write for how to use their frameworks are told from the point of view of a true believer. Other authors who write about the framework also tend to be disciples of the true belief. They show you the way to use the framework. Often they assume an all-encompassing, all-pervading, let-the-framework-do-everything position.

Framework có thể rất mạnh và rất hữu ích. Tác giả của framework thường tin vào framework của mình một cách hết sức sâu sắc. Những ví dụ họ viết ra để hướng dẫn dùng framework được kể từ điểm nhìn của một tín đồ thực thụ. Các tác giả khác viết về framework ấy cũng hay là môn đồ của cùng một đức tin. Họ chỉ cho bạn con đường dùng framework. Và họ thường mặc định một lập trường bao trùm tất cả, thấm vào mọi thứ, cứ-để-framework-làm-hết.

This is not the position you want to take.

Đó không phải là lập trường bạn nên chọn.

Look at each framework with a jaded eye. View it skeptically. Yes, it might help, but at what cost? Ask yourself how you should use it, and how you should protect yourself from it. Think about how you can preserve the use- case emphasis of your architecture. Develop a strategy that prevents the framework from taking over that architecture.

Hãy nhìn mọi framework bằng con mắt từng trải. Hãy hoài nghi nó. Đúng, nó có thể giúp ích, nhưng cái giá là gì? Hãy tự hỏi bạn nên dùng nó thế nào, và nên tự bảo vệ mình khỏi nó ra sao. Hãy nghĩ xem làm cách nào giữ được trọng tâm use case cho kiến trúc của mình. Hãy xây dựng một chiến lược ngăn framework chiếm quyền kiểm soát kiến trúc ấy.

Testable Architectures — Kiến trúc kiểm thử được

If your system architecture is all about the use cases, and if you have kept your frameworks at arm's length, then you should be able to unit-test all those use cases without any of the frameworks in place. You shouldn't need the web server running to run your tests. You shouldn't need the database connected to run your tests. Your **Entity** objects should be plain old objects that have no dependencies on frameworks or databases or other complications. Your use case objects should coordinate your Entity objects. Finally, all of them together should be testable in situ, without any of the complications of frameworks.

Nếu kiến trúc hệ thống của bạn xoay quanh các use case, và nếu bạn đã giữ framework ở một khoảng cách an toàn, thì bạn phải có khả năng unit-test toàn bộ các use case đó mà không cần framework nào có mặt. Bạn không cần web server phải đang chạy mới chạy được test. Bạn không cần cơ sở dữ liệu phải kết nối mới chạy được test. Các đối tượng Entity của bạn phải là những đối tượng thuần túy, không phụ thuộc vào framework, cơ sở dữ liệu hay bất kỳ thứ rắc rối nào khác. Các đối tượng use case của bạn phải điều phối các đối tượng Entity. Và cuối cùng, tất cả chúng gộp lại phải kiểm thử được tại chỗ, không cần tới bất kỳ mớ rắc rối nào của framework.

Conclusion — Kết luận

Your architecture should tell readers about the system, not about the frameworks you used in your system. If you are building a health care system, then when new programmers look at the source repository, their first impression should be, “Oh, this is a health care system.” Those new programmers should be able to learn all the use cases of the system, yet still not know how the system is delivered. They may come to you and say:

Kiến trúc của bạn phải kể cho người đọc nghe về hệ thống, chứ không phải về những framework bạn đã dùng trong hệ thống. Nếu bạn đang xây một hệ thống chăm sóc sức khỏe, thì khi các lập trình viên mới nhìn vào kho mã nguồn, ấn tượng đầu tiên của họ phải là: “À, đây là một hệ thống chăm sóc sức khỏe.” Những lập trình viên mới đó phải học được toàn bộ use case của hệ thống, mà vẫn chưa biết hệ thống được phân phối bằng cách nào. Có thể họ sẽ tới gặp bạn và hỏi:

“We see some things that look like models—but where are the views and controllers?”

“Bọn em thấy có vài thứ trông giống model — nhưng view với controller đâu rồi ạ?”

And you should respond:

Và bạn nên đáp:

“Oh, those are details that needn't concern us at the moment. We'll decide about them later.”

“Ồ, mấy cái đó là chi tiết, giờ chưa cần bận tâm. Lát nữa ta sẽ quyết định về chúng.”

The Clean Architecture — The Clean Architecture

Over the last several decades we've seen a whole range of ideas regarding the **architecture** of systems. These include:

Suốt mấy thập kỷ qua, chúng ta đã chứng kiến hàng loạt ý tưởng về kiến trúc (architecture) của hệ thống. Trong đó có:

- Hexagonal Architecture (also known as **Ports and Adapters**), developed by
 - Hexagonal Architecture (còn được gọi là Ports and Adapters), do

Alistair Cockburn, and adopted by Steve Freeman and Nat Pryce in their wonderful book Growing Object Oriented Software with Tests

Alistair Cockburn phát triển, và được Steve Freeman cùng Nat Pryce áp dụng trong cuốn sách tuyệt vời Growing Object Oriented Software with Tests của họ

- DCI from James Coplien and Trygve Reenskaug
 - DCI của James Coplien và Trygve Reenskaug
- BCE, introduced by Ivar Jacobson from his book Object Oriented Software
 - BCE, do Ivar Jacobson giới thiệu trong cuốn Object Oriented Software

Engineering: A Use-Case Driven Approach

Engineering: A Use-Case Driven Approach

Although these architectures all vary somewhat in their details, they are very similar. They all have the same objective, which is the separation of concerns. They all achieve this separation by dividing the software into layers. Each has at least one layer for business rules, and another layer for user and system interfaces.

Tuy các kiến trúc này có khác nhau đôi chút ở chi tiết, chúng rất giống nhau. Tất cả đều nhắm tới cùng một mục tiêu: phân tách các mối quan tâm. Tất cả đều đạt được sự phân tách đó bằng cách chia phần mềm thành các tầng. Mỗi kiến trúc có ít nhất một tầng dành cho các quy tắc nghiệp vụ (business rule), và một tầng khác dành cho giao diện người dùng và giao diện hệ thống.

Each of these architectures produces systems that have the following characteristics:

Mỗi kiến trúc kể trên đều tạo ra những hệ thống có các đặc điểm sau:

- Independent of frameworks. The architecture does not depend on the

- Độc lập với framework. Kiến trúc không phụ thuộc vào sự

existence of some library of feature-laden software. This allows you to use such frameworks as tools, rather than forcing you to cram your system into their limited constraints.

tồn tại của một thư viện phần mềm nặng tính năng nào đó. Điều này cho phép bạn dùng các framework như những công cụ, thay vì buộc bạn phải nhồi nhét hệ thống của mình vào các ràng buộc hạn hẹp của chúng.

- Testable. The business rules can be tested without the UI, database, web
 - Kiểm thử được. Các quy tắc nghiệp vụ có thể được kiểm thử mà không cần UI, database, web

server, or any other external element.

server, hay bất kỳ thành phần bên ngoài nào khác.

- Independent of the UI. The UI can change easily, without changing the rest
 - Độc lập với UI. UI có thể thay đổi dễ dàng mà không phải đụng tới phần còn lại

of the system. A web UI could be replaced with a console UI, for example, without changing the business rules.

của hệ thống. Chẳng hạn, một UI web có thể được thay bằng một UI dòng lệnh mà không cần sửa các quy tắc nghiệp vụ.

- Independent of the database. You can swap out Oracle or SQL Server for
 - Độc lập với database. Bạn có thể thay Oracle hay SQL Server bằng

Mongo, BigTable, CouchDB, or something else. Your business rules are not bound to the database.

Mongo, BigTable, CouchDB, hoặc thứ gì khác. Các quy tắc nghiệp vụ của bạn không bị trói vào database.

- Independent of any external agency. In fact, your business rules don't
 - Độc lập với mọi tác nhân bên ngoài. Thật ra, các quy tắc nghiệp vụ của bạn chẳng

know anything at all about the interfaces to the outside world.

hề biết bất cứ điều gì về những interface hướng ra thế giới bên ngoài.

The diagram in Figure 22.1 is an attempt at integrating all these architectures into a single actionable idea.

Sơ đồ trong Figure 22.1 là một nỗ lực hợp nhất tất cả các kiến trúc đó thành một ý tưởng duy nhất, đem ra dùng được ngay.

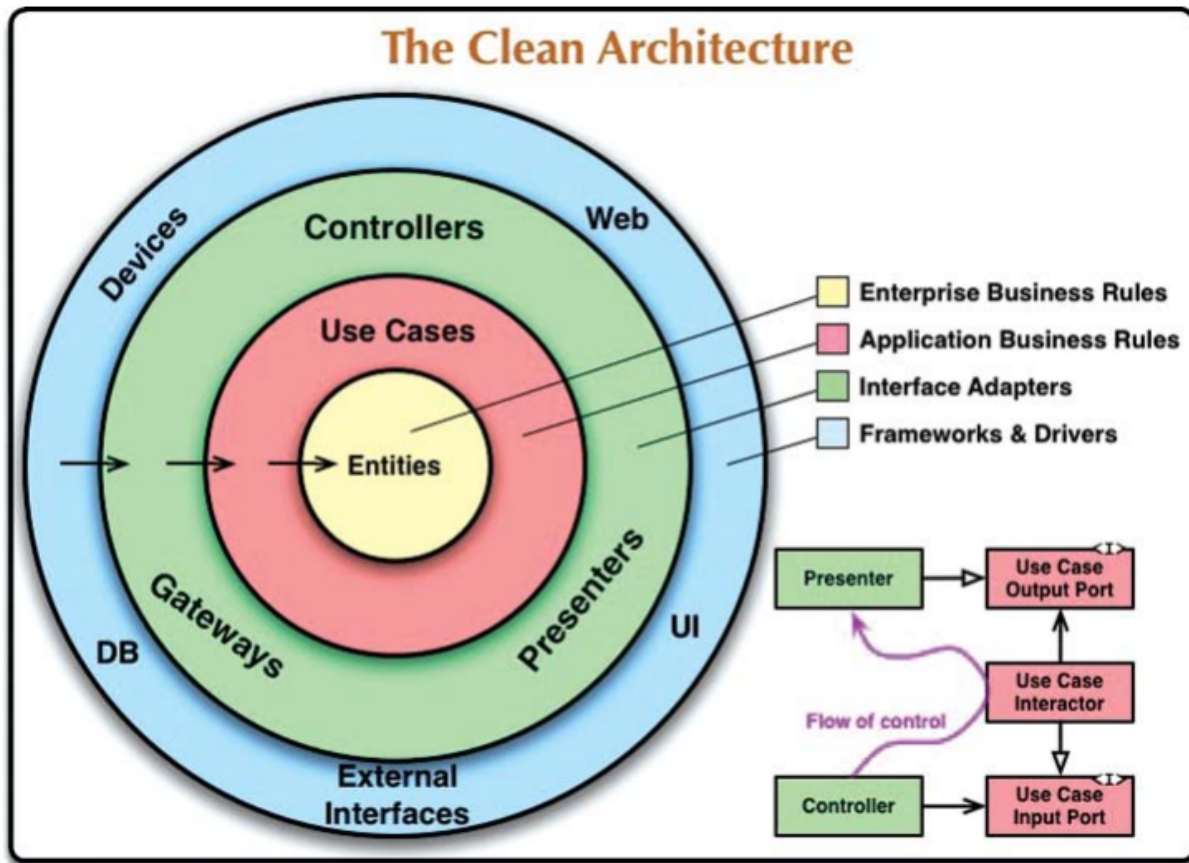


Figure 22.1 The **clean architecture**

Figure 22.1 The Clean Architecture

The Dependency Rule — Quy tắc Phụ thuộc (the Dependency Rule)

The concentric circles in Figure 22.1 represent different areas of software. In general, the further in you go, the higher **level** the software becomes. The outer circles are mechanisms. The inner circles are policies.

Các vòng tròn đồng tâm trong Figure 22.1 biểu diễn những vùng khác nhau của phần mềm. Nói chung, càng đi vào trong, phần mềm càng ở mức cao. Các vòng ngoài là cơ chế. Các vòng trong là chính sách (policy).

The overriding rule that makes this architecture work is **the Dependency Rule** :

Quy tắc bao trùm khiến kiến trúc này chạy được chính là Quy tắc Phụ thuộc:

Source code dependencies must point only inward, toward higher-level policies.

Phụ thuộc mã nguồn chỉ được hướng vào trong, về phía các chính sách ở mức cao hơn.

Nothing in an inner circle can know anything at all about something in an outer circle. In particular,

the name of something declared in an outer circle must not be mentioned by the code in an inner circle. That includes functions, classes, variables, or any other named software **entity**.

Không thứ gì ở vòng trong được biết bất cứ điều gì về thứ nằm ở vòng ngoài. Cụ thể, tên của một thứ được khai báo ở vòng ngoài không được phép xuất hiện trong mã ở vòng trong. Điều đó áp dụng cho hàm, class, biến, hay bất kỳ thực thể phần mềm có tên nào khác.

By the same token, data formats declared in an outer circle should not be used by an inner circle, especially if those formats are generated by a **framework** in an outer circle. We don't want anything in an outer circle to impact the inner circles.

Cũng theo lẽ đó, các định dạng dữ liệu được khai báo ở vòng ngoài không nên bị vòng trong sử dụng, nhất là khi những định dạng ấy do một framework ở vòng ngoài sinh ra. Chúng ta không muốn bất cứ thứ gì ở vòng ngoài tác động tới các vòng trong.

Entities — Entities

Entities encapsulate enterprise-wide Critical Business Rules. An entity can be an object with methods, or it can be a set of data structures and functions. It doesn't matter so long as the entities can be used by many different applications in the enterprise.

Entity đóng gói (encapsulation) các quy tắc nghiệp vụ cốt lõi ở phạm vi toàn doanh nghiệp. Một entity có thể là một object có các method, hoặc có thể là một tập cấu trúc dữ liệu và hàm. Điều đó không quan trọng, miễn là các entity có thể được nhiều ứng dụng khác nhau trong doanh nghiệp sử dụng.

If you don't have an enterprise and are writing just a single application, then these entities are the business objects of the application. They encapsulate the most general and **high-level** rules. They are the least likely to change when something external changes. For example, you would not expect these objects to be affected by a change to page navigation or security. No operational change to any particular application should affect the entity layer.

Nếu bạn không có cả một doanh nghiệp mà chỉ đang viết một ứng dụng đơn lẻ, thì các entity này chính là những đối tượng nghiệp vụ của ứng dụng. Chúng đóng gói những quy tắc tổng quát nhất và ở mức cao nhất. Chúng là thứ ít có khả năng thay đổi nhất khi có gì đó bên ngoài thay đổi. Chẳng hạn, bạn sẽ không mong đợi những object này bị ảnh hưởng bởi một thay đổi về điều hướng trang hay về bảo mật. Không một thay đổi vận hành nào của bất kỳ ứng dụng cụ thể nào được phép ảnh hưởng tới tầng entity.

Use Cases — Use Cases

The software in the use cases layer contains application-specific business rules. It encapsulates and implements all of the use cases of the system. These use cases orchestrate the flow of data to and from the entities, and direct those entities to use their Critical Business Rules to achieve the goals of the **use case**.

Phần mềm ở tầng use case chứa các quy tắc nghiệp vụ đặc thù của ứng dụng. Nó đóng gói và hiện thực toàn bộ use case của hệ thống. Những use case này điều phối dòng dữ liệu đi vào và đi ra khỏi các entity, đồng thời chỉ đạo các entity dùng quy tắc nghiệp vụ cốt lõi của chúng để đạt được mục tiêu của use case.

We do not expect changes in this layer to affect the entities. We also do not expect this layer to be affected by changes to externalities such as the database, the UI, or any of the common frameworks. The use cases layer is isolated from such concerns.

Chúng ta không mong đợi các thay đổi ở tầng này ảnh hưởng tới entity. Chúng ta cũng không mong đợi tầng này bị ảnh hưởng bởi những thay đổi ở bên ngoài như database, UI, hay bất kỳ framework phổ biến nào. Tầng use case được cách ly khỏi những mối bận tâm ấy.

We do, however, expect that changes to the operation of the application will affect the use cases and, therefore, the software in this layer. If the details of a use case change, then some code in this layer will certainly be affected.

Tuy nhiên, chúng ta có mong đợi rằng thay đổi trong cách vận hành của ứng dụng sẽ ảnh hưởng tới các use case, và do đó ảnh hưởng tới phần mềm ở tầng này. Nếu chi tiết của một use case thay đổi, chắc chắn sẽ có mã ở tầng này bị ảnh hưởng.

Interface Adapters — Interface Adapters

The software in the **interface** adapters layer is a set of adapters that convert data from the format most convenient for the use cases and entities, to the format most convenient for some external agency such as the database or the web. It is this layer, for example, that will wholly contain the MVC architecture of a GUI. The presenters, views, and controllers all belong in the interface adapters layer. The models are likely just data structures that are passed from the controllers to the use cases, and then back from the use cases to the presenters and views.

Phần mềm ở tầng interface adapter là một tập các adapter chuyển đổi dữ liệu từ định dạng tiện nhất cho use case và entity sang định dạng tiện nhất cho một tác nhân bên ngoài nào đó, chẳng hạn database hay web. Chẳng hạn, chính tầng này sẽ chứa trọn vẹn kiến trúc MVC của một GUI. Presenter, view và controller đều thuộc về tầng interface adapter. Còn model thì nhiều khả năng chỉ là những cấu trúc dữ liệu được truyền từ controller vào use case, rồi từ use case quay ngược ra presenter và view.

Similarly, data is converted, in this layer, from the form most convenient for entities and use cases, to the form most convenient for whatever persistence framework is being used (i.e., the database). No code inward of this circle should know anything at all about the database. If the database is a SQL database, then all SQL should be restricted to this layer—and in particular to the parts of this layer that have to do with the database.

Tương tự, ở tầng này dữ liệu cũng được chuyển từ dạng tiện nhất cho entity và use case sang dạng tiện nhất cho bất cứ framework lưu trữ nào đang được dùng (tức là database). Không đoạn mã nào nằm phía trong vòng tròn này được biết bất cứ điều gì về database. Nếu database là một SQL database, thì toàn bộ SQL phải bị giới hạn trong tầng này — và cụ thể hơn nữa là trong những phần của tầng này có liên quan tới database.

Also in this layer is any other adapter necessary to convert data from some external form, such as an external **service**, to the internal form used by the use cases and entities.

Cũng nằm ở tầng này là mọi adapter khác cần thiết để chuyển dữ liệu từ một dạng bên ngoài nào đó, chẳng hạn từ một dịch vụ (service) bên ngoài, sang dạng nội bộ mà use case và entity sử dụng.

Frameworks and Drivers — Frameworks and Drivers

The outermost layer of the model in Figure 22.1 is generally composed of frameworks and tools such as the database and the web framework. Generally you don't write much code in this layer, other than glue code that communicates to the next circle inward.

Tầng ngoài cùng của mô hình trong Figure 22.1 nói chung gồm các framework và công cụ như database và web framework. Thường thì bạn không viết nhiều mã ở tầng này, ngoài phần mã keo (glue code) để giao tiếp với vòng tròn kế tiếp bên trong.

The frameworks and drivers layer is where all the details go. The web is a **detail**. The database is a detail. We keep these things on the outside where they can do little harm.

Tầng framework và driver là nơi mọi chi tiết (detail) được đẩy tới. Web là một chi tiết. Database là một chi tiết. Chúng ta giữ những thứ đó ở bên ngoài, nơi chúng khó gây hại.

Only Four Circles? — Chỉ bốn vòng tròn thôi sao?

The circles in Figure 22.1 are intended to be schematic: You may find that you need more than just these four. There's no rule that says you must always have

Các vòng tròn trong Figure 22.1 chỉ mang tính minh họa: bạn có thể thấy mình cần nhiều hơn bốn vòng này. Không có luật nào bảo rằng bạn luôn phải có

just these four. However, the Dependency Rule always applies. Source code dependencies always point inward. As you move inward, the level of abstraction and **policy** increases. The outermost circle consists of **low-level** concrete details. As you move inward, the software grows more abstract and encapsulates higher-level policies. The innermost circle is the most general and highest level.

đúng bốn vòng đó. Tuy nhiên, Quy tắc Phụ thuộc thì luôn có hiệu lực. Phụ thuộc mã nguồn luôn hướng vào trong. Càng đi vào trong, mức trừu tượng và mức chính sách càng tăng. Vòng ngoài cùng gồm những chi tiết cụ thể ở mức thấp. Càng vào trong, phần mềm càng trừu tượng hơn và đóng gói những chính sách ở mức cao hơn. Vòng trong cùng là tổng quát nhất và ở mức cao nhất.

Crossing Boundaries — Vượt ranh giới

At the lower right of the diagram in Figure 22.1 is an example of how we cross the circle boundaries. It shows the controllers and presenters communicating with the use cases in the next layer. Note the **flow of control**: It begins in the controller, moves through the use case, and then winds up executing in the **presenter**. Note also the source code dependencies: Each points inward toward the use cases.

Ở góc dưới bên phải sơ đồ trong Figure 22.1 là một ví dụ về cách chúng ta vượt ranh giới (boundary) giữa các vòng tròn. Nó cho thấy controller và presenter giao tiếp với use case ở tầng kế tiếp. Hãy để ý luồng điều khiển: nó bắt đầu ở controller, đi xuyên qua use case, rồi kết thúc bằng việc thực thi trong presenter. Cũng hãy để ý các phụ thuộc mã nguồn: mỗi phụ thuộc đều hướng vào trong, về phía use case.

We usually resolve this apparent contradiction by using the **Dependency Inversion** Principle. In a language like Java, for example, we would arrange interfaces and **inheritance** relationships such that the source code dependencies oppose the flow of control at just the right points across the **boundary**.

Chúng ta thường hoá giải mâu thuẫn bề ngoài này bằng Dependency Inversion Principle. Chẳng hạn, trong một ngôn ngữ như Java, ta sẽ bố trí các interface và quan hệ kế thừa (inheritance) sao cho phụ thuộc mã nguồn đi ngược lại luồng điều khiển đúng tại những điểm cần thiết trên ranh giới.

For example, suppose the use case needs to call the presenter. This call must not be direct because that would violate the Dependency Rule: No name in an outer circle can be mentioned by an inner

circle. So we have the use case call an interface (shown in Figure 22.1 as “use case output port”) in the inner circle, and have the presenter in the outer circle implement it.

Ví dụ, giả sử use case cần gọi presenter. Lời gọi này không được phép trực tiếp, vì như vậy là vi phạm Quy tắc Phụ thuộc: không cái tên nào ở vòng ngoài được vòng trong nhắc tới. Thế nên ta cho use case gọi một interface (trong Figure 22.1 ghi là “use case output port”) nằm ở vòng trong, và cho presenter ở vòng ngoài hiện thực interface đó.

The same technique is used to cross all the boundaries in the architectures. We take advantage of dynamic **polymorphism** to create source code dependencies that oppose the flow of control so that we can conform to the Dependency Rule, no matter which direction the flow of control travels.

Cùng kỹ thuật ấy được dùng để vượt mọi ranh giới trong các kiến trúc này. Chúng ta tận dụng đa hình (polymorphism) động để tạo ra những phụ thuộc mã nguồn đi ngược luồng điều khiển, nhờ đó tuân thủ được Quy tắc Phụ thuộc, bất kể luồng điều khiển chạy theo hướng nào.

Which Data Crosses the Boundaries — Dữ liệu nào vượt ranh giới

Typically the data that crosses the boundaries consists of simple data structures. You can use basic structs or simple data transfer objects if you like. Or the data can simply be arguments in function calls. Or you can pack it into a hashmap, or construct it into an object. The **important** thing is that isolated, simple data structures are passed across the boundaries. We don’t want to cheat and pass Entity objects or database rows. We don’t want the data structures to have any kind of dependency that violates the Dependency Rule.

Thông thường, dữ liệu vượt ranh giới chỉ gồm những cấu trúc dữ liệu đơn giản. Bạn có thể dùng struct cơ bản hoặc các data transfer object đơn giản nếu thích. Hoặc dữ liệu có thể chỉ là các tham số trong lời gọi hàm. Hoặc bạn có thể nhồi nó vào một hashmap, hay dựng nó thành một object. Điều quan trọng là thứ được truyền qua ranh giới phải là những cấu trúc dữ liệu biệt lập, đơn giản. Chúng ta không muốn gian lận bằng cách truyền các object Entity hay các hàng dữ liệu của database. Chúng ta không muốn các cấu trúc dữ liệu mang bất kỳ phụ thuộc nào vi phạm Quy tắc Phụ thuộc.

For example, many database frameworks return a convenient data format in response to a query. We might call this a “row **structure**.” We don’t want to pass that row structure inward across a boundary. Doing so would violate the Dependency Rule because it would force an inner circle to know something about an outer circle.

Ví dụ, nhiều database framework trả về một định dạng dữ liệu tiện lợi để đáp lại một truy vấn. Ta có thể gọi đó là một “cấu trúc hàng” (row structure). Chúng ta không muốn truyền cấu trúc hàng đó vào trong, vượt qua ranh giới. Làm vậy là vi phạm Quy tắc Phụ thuộc, vì nó buộc một vòng trong phải biết điều gì đó về vòng ngoài.

Thus, when we pass data across a boundary, it is always in the form that is most convenient for the inner circle.

Vì thế, khi truyền dữ liệu qua một ranh giới, dữ liệu luôn ở dạng tiện nhất cho vòng trong.

A Typical Scenario — Một kịch bản điển hình

The diagram in Figure 22.2 shows a typical scenario for a web-based Java system using a database. The web server gathers input data from the user and hands it to the Controller on the upper left. The Controller packages that data into a plain old Java object and passes this object through the

Sơ đồ trong Figure 22.2 mô tả một kịch bản điển hình của một hệ thống Java trên nền web có dùng database. Web server thu thập dữ liệu nhập từ người dùng rồi giao cho Controller ở phía trên bên trái. Controller đóng gói dữ liệu đó thành một plain old Java object và truyền object này qua

InputBoundary to the UseCaseInteractor. The UseCaseInteractor

interprets that data and uses it to control the dance of the Entities. It also uses the DataAccessInterface to bring the data used by those Entities into memory from the Database. Upon completion, the UseCaseInteractor gathers data from the Entities and constructs the OutputData as another plain old Java object. The OutputData is then passed through the OutputBoundary interface to the Presenter.

diễn giải dữ liệu đó và dùng nó để điều khiển vũ điệu của các Entity. Nó cũng dùng DataAccessInterface để đưa dữ liệu mà các Entity cần từ Database vào bộ nhớ. Khi hoàn tất, UseCaseInteractor thu thập dữ liệu từ các Entity và dựng nên OutputData như một plain old Java object nữa. OutputData sau đó được truyền qua interface OutputBoundary tới Presenter.

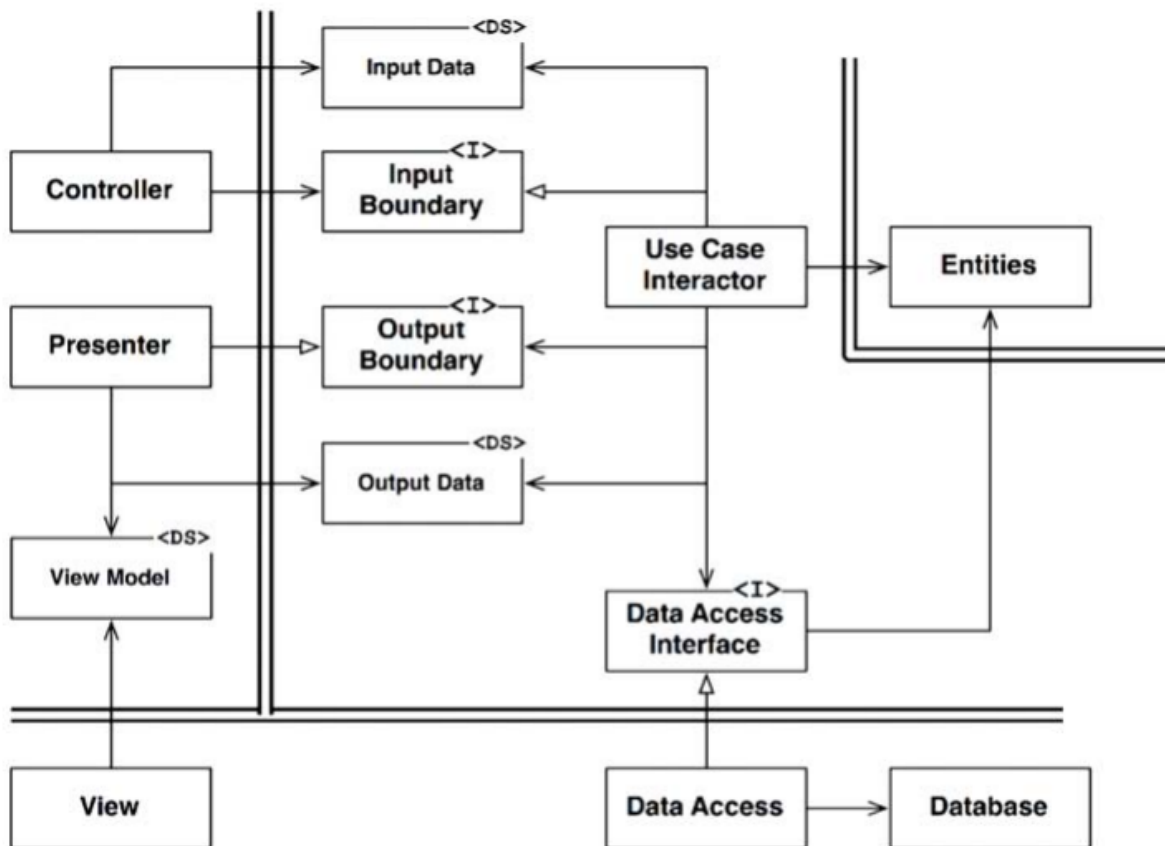


Figure 22.2 A typical scenario for a web-based Java system utilizing a database

Figure 22.2 Một kịch bản điển hình của hệ thống Java trên nền web có sử dụng database

The job of the Presenter is to repack the OutputData into viewable form as the ViewModel, which is yet another plain old Java object. The ViewModel contains mostly Strings and flags that the View uses to display the data. Whereas the OutputData may contain Date objects, the Presenter will load the ViewModel with corresponding Strings already formatted properly for the user. The same is true

of Currency objects or any other business-related data. Button and MenuItem names are placed in the ViewModel, as are flags that tell the View whether those Buttons and MenuItems should be gray.

Nhiệm vụ của Presenter là đóng gói lại OutputData thành dạng hiển thị được, tức ViewModel, lại là một plain old Java object nữa. ViewModel chủ yếu chứa các String và các cờ mà View dùng để hiển thị dữ liệu. Trong khi OutputData có thể chứa các object Date, thì Presenter sẽ nạp vào ViewModel những String tương ứng đã được định dạng sẵn cho người dùng. Với các object Currency hay bất kỳ dữ liệu liên quan tới nghiệp vụ nào khác cũng vậy. Tên của các Button và MenuItem được đặt trong ViewModel, cùng với những cờ cho View biết các Button và MenuItem đó có nên bị làm mờ hay không.

This leaves the View with almost nothing to do other than to move the data

Nhờ vậy, View gần như chẳng còn việc gì để làm ngoài chuyện chuyển dữ liệu from the ViewModel into the HTML page.

Note the directions of the dependencies. All dependencies cross the boundary lines pointing inward, following the Dependency Rule.

Hãy để ý hướng của các phụ thuộc. Mọi phụ thuộc đều cắt qua các đường ranh giới theo hướng vào trong, đúng theo Quy tắc Phụ thuộc.

Conclusion — Kết luận

Conforming to these simple rules is not difficult, and it will save you a lot of headaches going forward. By separating the software into layers and conforming to the Dependency Rule, you will create a system that is intrinsically testable, with all the benefits that implies. When any of the external parts of the system become obsolete, such as the database, or the web framework, you can replace those obsolete elements with a minimum of fuss.

Tuân thủ mấy quy tắc đơn giản này không hề khó, và về sau nó sẽ giúp bạn tránh được vô số cơn đau đầu. Bằng cách tách phần mềm thành các tầng và tuân thủ Quy tắc Phụ thuộc, bạn sẽ tạo ra một hệ thống tự bản chất đã kiểm thử được, kèm theo mọi lợi ích đi cùng điều đó. Khi bất kỳ phần bên ngoài nào của hệ thống trở nên lỗi thời, chẳng hạn database hay web framework, bạn có thể thay chúng mà chẳng phải phiền hà gì mấy.

Presenters and Humble Objects — Presenter và Humble Object

In Chapter 22, we introduced the notion of presenters. Presenters are a form of the **Humble Object pattern**, which helps us identify and protect architectural boundaries. Actually, the **Clean Architecture** in the last chapter was full of Humble Object implementations.

Ở Chương 22, chúng ta đã giới thiệu khái niệm presenter. Presenter là một dạng của Humble Object pattern, thứ giúp ta nhận diện và bảo vệ các ranh giới kiến trúc. Thật ra, Clean Architecture ở chương trước đây rầy những hiện thực của Humble Object.

The Humble Object Pattern — Humble Object pattern

The Humble Object pattern 1 is a **design** pattern that was originally identified as a way to help unit testers to separate behaviors that are hard to test from behaviors that are easy to test. The idea is very simple: Split the behaviors into two modules or classes. One of those modules is humble; it contains all the hard-to-test behaviors stripped down to their barest essence. The other module contains all the testable behaviors that were stripped out of the humble object.

Humble Object pattern 1 là một mẫu thiết kế ban đầu được nhận diện như một cách giúp người viết unit test tách những hành vi (behavior) khó kiểm thử ra khỏi những hành vi dễ kiểm thử. Ý tưởng rất đơn giản: chia các hành vi thành hai module hoặc hai class. Một trong hai module đó là “humble” — khiêm nhường; nó chứa toàn bộ những hành vi khó kiểm thử, đã được lược bỏ tới mức tinh gọn nhất. Module còn lại chứa tất cả những hành vi có thể kiểm thử đã được tách ra khỏi humble object.

For example, GUIs are hard to unit test because it is very difficult to write tests that can see the screen and check that the appropriate elements are displayed there. However, most of the **behavior** of a GUI is, in fact, easy to test. Using the Humble Object pattern, we can separate these two kinds of behaviors into two different classes called the **Presenter** and the View.

Ví dụ, GUI rất khó unit test, bởi viết được test có thể “nhìn” vào màn hình và kiểm tra xem các phần tử thích hợp có hiện ra ở đó hay không là chuyện cực khó. Nhưng phần lớn hành vi của một GUI thì lại rất dễ kiểm thử. Dùng Humble Object pattern, ta có thể tách hai loại hành vi ấy vào hai class khác nhau, gọi là Presenter và View.

Presenters and Views — Presenter và View

The View is the humble object that is hard to test. The code in this object is kept as simple as possible. It moves data into the GUI but does not process that data.

View là humble object — phần khó kiểm thử. Mã trong đối tượng này được giữ đơn giản hết mức có thể. Nó chuyển dữ liệu vào GUI chứ không xử lý dữ liệu đó.

The Presenter is the testable object. Its job is to accept data from the application and format it for presentation so that the View can simply move it to the screen. For example, if the application wants a date displayed in a field, it will hand the Presenter a Date object. The Presenter will then format that

Presenter là đối tượng có thể kiểm thử. Việc của nó là nhận dữ liệu từ ứng dụng và định dạng dữ liệu ấy để trình bày, sao cho View chỉ việc đưa lên màn hình. Chẳng hạn, nếu ứng dụng muốn hiển thị một ngày trong một trường nào đó, nó sẽ đưa cho Presenter một đối tượng Date. Presenter khi đó sẽ định dạng

- 1. xUnit Patterns , Meszaros, Addison-Wesley, 2007, p. 695.
- 1. xUnit Patterns , Meszaros, Addison-Wesley, 2007, p. 695.

data into an appropriate string and place it in a simple data **structure** called the **View Model**, where the View can find it.

dữ liệu ấy thành một chuỗi thích hợp và đặt vào một cấu trúc dữ liệu đơn giản gọi là View Model, nơi View có thể tìm thấy nó.

If the application wants to display money on the screen, it might pass a Currency object to the Presenter. The Presenter will format that object with the appropriate decimal places and currency markers, creating a string that it can place in the View Model. If that currency value should be turned red if it is negative, then a simple boolean flag in the View model will be set appropriately.

Nếu ứng dụng muốn hiển thị số tiền lên màn hình, nó có thể truyền một đối tượng Currency cho Presenter. Presenter sẽ định dạng đối tượng đó với số chữ số thập phân và ký hiệu tiền tệ thích hợp, tạo ra một chuỗi để đặt vào View Model. Nếu giá trị tiền tệ ấy cần hiển thị màu đỏ khi âm, thì một cờ boolean đơn giản trong View Model sẽ được đặt cho đúng.

Every button on the screen will have a name. That name will be a string in the View Model, placed there by the presenter. If those buttons should be grayed out, the Presenter will set an appropriate boolean flag in the View model. Every menu item name is a string in the View model, loaded by the Presenter. The names for every radio button, check box, and text field are loaded, by the Presenter, into appropriate strings and booleans in the View model. Tables of numbers that should be displayed on the screen are loaded, by the Presenter, into tables of properly formatted strings in the View model.

Mỗi nút bấm trên màn hình đều có một cái tên. Cái tên đó là một chuỗi trong View Model, do presenter đặt vào. Nếu những nút ấy cần bị làm mờ, Presenter sẽ đặt một cờ boolean thích hợp trong View Model. Tên của mỗi mục menu là một chuỗi trong View Model, do Presenter nạp vào. Tên của mỗi radio button, check box và text field đều được Presenter nạp vào các chuỗi và boolean tương ứng trong View Model. Những bảng số cần hiển thị trên màn hình cũng được Presenter nạp vào những bảng chuỗi đã định dạng sẵn trong View Model.

Anything and everything that appears on the screen, and that the application has some kind of control over, is represented in the View Model as a string, or a boolean, or an enum. Nothing is left for the View to do other than to load the data from the View Model into the screen. Thus the View is humble.

Bất cứ thứ gì xuất hiện trên màn hình mà ứng dụng có quyền kiểm soát ở mức nào đó đều được biểu diễn trong View Model dưới dạng một chuỗi, một boolean, hoặc một enum.

Chẳng còn việc gì cho View làm ngoài việc nạp dữ liệu từ View Model lên màn hình. Vì thế View mới “humble”.

Testing and Architecture — Kiểm thử và kiến trúc

It has long been known that testability is an attribute of good architectures. The Humble Object pattern is a good example, because the separation of the behaviors into testable and non-testable parts often defines an **architectural boundary**. The Presenter/View **boundary** is one of these boundaries, but there are many others.

Từ lâu người ta đã biết khả năng kiểm thử là một thuộc tính của kiến trúc (architecture) tốt. Humble Object pattern là một ví dụ hay, bởi việc tách các hành vi thành phần kiểm thử được và phần không kiểm thử được thường vạch ra chính một ranh giới kiến trúc. Ranh giới Presenter/View là một trong những ranh giới đó, nhưng còn nhiều ranh giới khác nữa.

Database Gateways — Database gateway

Between the **use case** interactors and the database are the database gateways. 2 These gateways are polymorphic interfaces that contain methods for every create, read, update, or delete operation that can be performed by the application on the database. For example, if the application needs to know the last names of all the users who logged in yesterday, then the UserGateway **interface** will have a

Giữa các use case interactor và cơ sở dữ liệu là các database gateway. 2 Những gateway này là các interface đa hình (polymorphism) chứa phương thức cho mọi thao tác create, read, update hay delete mà ứng dụng có thể thực hiện trên cơ sở dữ liệu. Ví dụ, nếu ứng dụng cần biết họ của tất cả người dùng đã đăng nhập hôm qua, thì interface UserGateway sẽ có một

method named `getLastNamesOfUsersWhoLoggedInAfter` that takes a `Date` as its argument and returns a list of last names.

làm tham số và trả về một danh sách các họ.

Recall that we do not allow SQL in the use cases layer; instead, we use gateway interfaces that have appropriate methods. Those gateways are implemented by classes in the database layer. That implementation is the humble object. It simply uses SQL, or whatever the interface to the database is, to access the data required by each of the methods. The interactors, in contrast, are not humble because they encapsulate application-specific business rules. Although they are not humble, those interactors are testable, because the gateways can be replaced with appropriate stubs and test-doubles.

Hãy nhớ rằng chúng ta không cho phép SQL xuất hiện ở tầng use case; thay vào đó, ta dùng các interface gateway với những phương thức thích hợp. Các gateway ấy được hiện thực bởi những class nằm ở tầng cơ sở dữ liệu. Chính phần hiện thực đó là humble object. Nó chỉ đơn thuần dùng SQL, hay bất cứ giao diện nào của cơ sở dữ liệu, để truy xuất dữ liệu mà mỗi phương thức cần. Ngược lại, các interactor thì không “humble”, vì chúng đóng gói (encapsulation) các quy tắc nghiệp vụ (business rule) riêng của ứng dụng. Dù không “humble”, những interactor ấy vẫn *kiểm thử được*, bởi ta có thể thay các gateway bằng những stub và test-double thích hợp.

Data Mappers — Data mapper

Going back to the topic of databases, in which layer do you think ORMs like Hibernate belong?

Trở lại chuyện cơ sở dữ liệu: bạn nghĩ những ORM như Hibernate thuộc về tầng nào?

First, let's get something straight: There is no such thing as an object relational mapper (ORM). The reason is simple: Objects are not data structures. At least, they are not data structures from their users' point of view. The users of an object cannot see the data, since it is all private. Those users see only the public methods of that object. So, from the user's point of view, an object is simply a set of operations.

Trước hết, hãy nói cho rõ: làm gì có cái gọi là object relational mapper (ORM). Lý do rất đơn giản: đối tượng không phải là cấu trúc dữ liệu. Ít nhất thì chúng không phải cấu trúc dữ liệu dưới góc nhìn của người dùng chúng. Người dùng một đối tượng không thể nhìn thấy dữ liệu, vì tất cả đều là private. Họ chỉ thấy các phương thức public của đối tượng đó. Vậy nên, dưới góc nhìn của người dùng, một đối tượng chỉ đơn giản là một tập các thao tác.

- 2. Patterns of Enterprise Application **Architecture** , Martin Fowler, et. al., Addison-Wesley, 2003, p. 466.
- 2. Patterns of Enterprise Application Architecture , Martin Fowler, et. al., Addison-Wesley, 2003, p. 466.

A data structure, in contrast, is a set of public data variables that have no implied behavior. ORMs would be better named “data mappers,” because they load data into data structures from relational database tables.

Ngược lại, một cấu trúc dữ liệu là một tập các biến dữ liệu public không hàm ý hành vi nào cả. Gọi ORM là “data mapper” thì đúng hơn, bởi chúng nạp dữ liệu từ các bảng của cơ sở dữ liệu quan hệ vào các cấu trúc dữ liệu.

Where should such ORM systems reside? In the database layer of course. Indeed, ORMs form another kind of Humble Object boundary between the gateway interfaces and the database.

Vậy những hệ thống ORM ấy nên nằm ở đâu? Dĩ nhiên là ở tầng cơ sở dữ liệu. Thật vậy, ORM tạo nên một dạng ranh giới Humble Object nữa, nằm giữa các interface gateway và cơ sở dữ liệu.

Service Listeners — Service listener

What about services? If your application must communicate with other services, or if your application provides a set of services, will we find the Humble Object pattern creating a **service** boundary?

Còn các dịch vụ (service) thì sao? Nếu ứng dụng của bạn phải giao tiếp với các dịch vụ khác, hoặc nếu ứng dụng của bạn cung cấp một tập dịch vụ, liệu ta có thấy Humble Object pattern tạo ra một ranh giới dịch vụ hay không?

Of course! The application will load data into simple data structures and then pass those structures across the boundary to modules that properly format the data and send it to external services. On the input side, the service listeners will receive data from the service interface and format it into a simple data structure that can be used by the application. That data structure is then passed across the service boundary.

Tất nhiên là có! Ứng dụng sẽ nạp dữ liệu vào các cấu trúc dữ liệu đơn giản, rồi chuyển những cấu trúc ấy qua ranh giới tới các module lo việc định dạng dữ liệu cho đúng và gửi đi tới các dịch vụ bên ngoài. Ở chiều đầu vào, các service listener sẽ nhận dữ liệu từ interface của dịch vụ và định dạng nó thành một cấu trúc dữ liệu đơn giản mà ứng dụng dùng được. Cấu trúc dữ liệu đó sau đó được chuyển qua ranh giới dịch vụ.

Conclusion — Kết luận

At each architectural boundary, we are likely to find the Humble Object pattern lurking somewhere nearby. The communication across that boundary will almost always involve some kind of simple data structure, and the boundary will frequently divide something that is hard to test from something that is easy to test. The use of this pattern at architectural boundaries vastly increases the testability of the entire system.

Ở mỗi ranh giới kiến trúc, nhiều khả năng ta sẽ bắt gặp Humble Object pattern đang ẩn nấp đâu đó quanh đấy. Việc giao tiếp qua ranh giới ấy hầu như luôn dính tới một dạng cấu trúc dữ liệu đơn giản nào đó, và ranh giới ấy thường chia tách phần khó kiểm thử khỏi phần dễ kiểm thử. Dùng mẫu này ở các ranh giới kiến trúc làm tăng vọt khả năng kiểm thử của toàn hệ thống.

Partial Boundaries — Ranh giới bán phần (partial boundary)

Full-fledged architectural boundaries are expensive. They require reciprocal polymorphic **Boundary** interfaces, Input and Output data structures, and all of the dependency management necessary to isolate the two sides into independently compilable and deployable components. That takes a lot of work. It's also a lot of work to maintain.

Ranh giới kiến trúc đầy đủ thì tốn kém. Chúng đòi hỏi các Boundary interface đa hình (polymorphism) theo cả hai chiều, các cấu trúc dữ liệu Input và Output, cùng toàn bộ việc quản lý phụ thuộc cần thiết để cô lập hai phía thành những component biên dịch và triển khai được độc lập. Chừng đó là rất nhiều việc. Và duy trì nó cũng là rất nhiều việc.

In many situations, a good architect might judge that the expense of such a boundary is too high—but might still want to hold a place for such a boundary in case it is needed later.

Trong nhiều tình huống, một kiến trúc sư giỏi có thể nhận định rằng cái giá của một ranh giới như vậy là quá cao — nhưng vẫn muốn giữ sẵn một chỗ cho ranh giới đó, phòng khi về sau cần đến.

This kind of anticipatory **design** is often frowned upon by many in the Agile community as a violation of YAGNI: “You Aren’t Going to Need It.” Architects, however, sometimes look at the problem and think, “Yeah, but I might.” In that case, they may implement a **partial boundary**.

Kiểu thiết kế đón đầu này thường bị nhiều người trong cộng đồng Agile cau mày, vì cho rằng nó vi phạm YAGNI: “You Aren’t Going to Need It.” Tuy vậy, đôi khi kiến trúc sư nhìn vào bài toán và nghĩ: “Ừ, nhưng biết đâu tôi lại cần.” Trong trường hợp đó, họ có thể hiện thực một ranh giới bán phần (partial boundary).

Skip the Last Step — Bỏ qua bước cuối

One way to construct a partial boundary is to do all the work necessary to create independently compilable and deployable components, and then simply keep them together in the same **component**. The reciprocal interfaces are there, the input/output data structures are there, and everything is all set up—but we compile and deploy all of them as a single component.

Một cách dựng ranh giới bán phần là làm hết mọi việc cần thiết để tạo ra những component biên dịch và triển khai được độc lập, rồi đơn giản là giữ tất cả trong cùng một component. Các interface đối ứng đều có đó, các cấu trúc dữ liệu input/output đều có đó, mọi thứ đã sẵn sàng — chỉ có điều ta biên dịch và triển khai tất cả như một component duy nhất.

Obviously, this kind of partial boundary requires the same amount of code and preparatory design work as a full boundary. However, it does not require the administration of multiple components. There's no version number tracking or release management burden. That difference should not be taken lightly.

Rõ ràng, kiểu ranh giới bán phần này đòi hỏi đúng lượng mã và đúng lượng công sức thiết kế chuẩn bị như một ranh giới đầy đủ. Tuy nhiên, nó không đòi hỏi phải quản trị nhiều component. Không có gánh nặng theo dõi số hiệu phiên bản hay quản lý phát hành. Khác biệt đó không nên bị xem nhẹ.

This was the early strategy behind FitNesse . The web server component of FitNesse was designed to be separable from the wiki and testing part of FitNesse . The idea was that we might want to create other web-based applications by using that web component. At the same, we did not want users to have to download two components. Recall that one of our design goals was “ download and go .” It was our intent that users would download

Đây chính là chiến lược thuở ban đầu của FitNesse . Component web server của FitNesse được thiết kế để có thể tách khỏi phần wiki và phần kiểm thử của FitNesse . Ý tưởng là chúng tôi có thể muốn tạo ra những ứng dụng web khác dựa trên component web đó. Đồng thời, chúng tôi lại không muốn người dùng phải tải về hai component. Hãy nhớ rằng một trong các mục tiêu thiết kế của chúng tôi là ” tải về là chạy .” Chúng tôi muốn người dùng chỉ tải về

one jar file and execute it without having to hunt for other jar files, work out version compatibilities, and so on.

một file jar rồi chạy nó, khỏi phải đi lùng những file jar khác, khỏi phải mày mò xem các phiên bản có tương thích với nhau hay không, v.v.

The story of FitNesse also points out one of the dangers of this approach. Over time, as it became clear that there would never be a need for a separate web component, the separation between the web component and the wiki component began to weaken. Dependencies started to cross the line in the wrong direction. Nowadays, it would be something of a chore to re-separate them.

Câu chuyện FitNesse cũng cho thấy một trong những hiểm họa của cách làm này. Theo thời gian, khi đã rõ là sẽ chẳng bao giờ có nhu cầu về một component web riêng biệt, sự phân tách giữa component web và component wiki bắt đầu yếu đi. Các phụ thuộc bắt đầu vượt qua lằn ranh theo chiều sai. Đến giờ, tách chúng ra lại sẽ là một việc khá cực nhọc.

One-Dimensional Boundaries — Ranh giới một chiều

The full-fledged **architectural boundary** uses reciprocal boundary interfaces to maintain isolation in both directions. Maintaining separation in both directions is expensive both in initial setup and in ongoing maintenance.

Ranh giới kiến trúc đầy đủ dùng các boundary interface đối ứng để duy trì sự cô lập theo cả hai chiều. Giữ được sự phân tách theo cả hai chiều là tốn kém, cả ở khâu thiết lập ban đầu lẫn ở khâu bảo trì về sau.

A simpler **structure** that serves to hold the place for later extension to a full- fledged boundary is shown in Figure 24.1. It exemplifies the traditional Strategy pattern. A ServiceBoundary **interface** is used by clients and implemented by ServiceImpl classes.

Một cấu trúc đơn giản hơn, đủ để giữ chỗ cho việc mở rộng thành ranh giới đầy đủ sau này, được minh họa trong Figure 24.1. Nó là một ví dụ điển hình của Strategy pattern

truyền thống. Một interface ServiceBoundary được các client sử dụng và được các class ServiceImpl hiện thực.

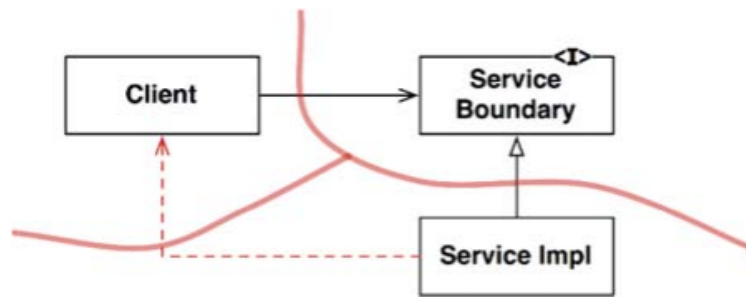


Figure 24.1 The Strategy pattern

Figure 24.1 Strategy pattern

It should be clear that this sets the stage for a future architectural boundary. The necessary **dependency inversion** is in place in an attempt to isolate the Client from the ServiceImpl. It should also be clear that the separation can degrade pretty rapidly, as shown by the nasty dotted arrow in the diagram. Without reciprocal interfaces, nothing prevents this kind of backchannel other than the diligence and discipline of the developers and architects.

Hẳn bạn đã thấy rõ rằng cách này dọn sẵn sân khấu cho một ranh giới kiến trúc trong tương lai. Phép đảo ngược phụ thuộc (dependency inversion) cần thiết đã nằm sẵn ở đó để cô lập Client khỏi ServiceImpl. Cũng phải thấy rõ rằng sự phân tách này có thể xuống cấp khá nhanh, như mũi tên nét đứt khó ưa trong sơ đồ đã chỉ ra. Không có các interface đối ứng, chẳng gì ngăn được kiểu “đi cửa sau” đó, ngoài sự cẩn trọng và kỷ luật của lập trình viên và kiến trúc sư.

Facades — Facade

An even simpler boundary is the Facade pattern, illustrated in Figure 24.2. In this case, even the dependency inversion is sacrificed. The boundary is simply defined by the Facade class, which lists all the services as methods, and deploys the **service** calls to classes that the client is not supposed to access.

Một ranh giới còn đơn giản hơn nữa là Facade pattern, minh họa trong Figure 24.2. Ở đây, ngay cả phép đảo ngược phụ thuộc cũng bị hy sinh. Ranh giới chỉ đơn thuần được định nghĩa bởi class Facade, nơi liệt kê tất cả các dịch vụ (service) dưới dạng method rồi chuyển các lời gọi dịch vụ tới những class mà client lẽ ra không được phép chạm tới.

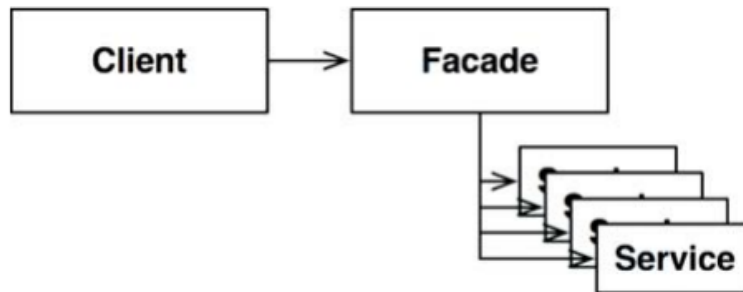


Figure 24.2 The Facade pattern

Figure 24.2 Facade pattern

Note, however, that the Client has a transitive dependency on all those service classes. In static languages, a change to the source code in one of the Service classes will force the Client to recompile. Also, you can imagine how easy backchannels are to create with this structure.

Tuy nhiên, hãy để ý rằng Client có phụ thuộc bắc cầu vào toàn bộ những class dịch vụ đó. Trong các ngôn ngữ tĩnh, chỉ cần đổi mã nguồn của một class Service là Client buộc phải biên dịch lại. Ngoài ra, bạn có thể hình dung việc tạo ra các lối “đi cửa sau” trong cấu trúc này dễ đến mức nào.

Conclusion — Kết luận

We’ve seen three simple ways to partially implement an architectural boundary. There are, of course, many others. These three strategies are simply offered as examples.

Chúng ta vừa xem qua ba cách đơn giản để hiện thực một phần ranh giới kiến trúc. Dĩ nhiên còn nhiều cách khác nữa. Ba chiến lược này chỉ được đưa ra làm ví dụ.

Each of these approaches has its own set of costs and benefits. Each is appropriate, in certain contexts, as a placeholder for an eventual full-fledged boundary. Each can also be degraded if that boundary never materializes.

Mỗi cách đều có tập chi phí và lợi ích riêng. Mỗi cách đều thích hợp, trong những bối cảnh nhất định, để giữ chỗ cho một ranh giới đầy đủ sau này. Và mỗi cách cũng đều có thể xuống cấp nếu ranh giới đó chẳng bao giờ thành hình.

It is one of the functions of an architect to decide where an architectural boundary might one day exist, and whether to fully or partially implement that boundary.

Một trong những chức năng của kiến trúc sư là quyết định xem một ranh giới kiến trúc có thể sẽ tồn tại ở đâu, và có nên hiện thực ranh giới đó một cách đầy đủ hay chỉ một phần.

Layers and Boundaries — Tầng và ranh giới

It is easy to think of systems as being composed of three components: UI, business rules, and database. For some simple systems, this is sufficient. For most systems, though, the number of components is larger than that.

Rất dễ hình dung hệ thống chỉ gồm ba component: UI, quy tắc nghiệp vụ (business rule) và cơ sở dữ liệu. Với vài hệ thống đơn giản, chừng đó là đủ. Nhưng với hầu hết hệ thống, số component nhiều hơn thế.

Consider, for example, a simple computer game. It is easy to imagine the three components. The UI handles all messages from the player to the game rules. The game rules store the state of the game in some kind of persistent data **structure**. But is that all there is?

Hãy lấy ví dụ một trò chơi máy tính đơn giản. Ba component kia hiện ra ngay trong đầu. UI xử lý mọi thông điệp từ người chơi tới các quy tắc của trò chơi. Các quy tắc của trò chơi lưu trạng thái ván chơi vào một cấu trúc dữ liệu bền vững nào đó. Nhưng chỉ có vậy thôi sao?

Hunt the Wumpus — Hunt the Wumpus

Let's put some flesh on these bones. Let's assume that the game is the venerable Hunt the Wumpus adventure game from 1972. This text-based game uses very simple commands like GO EAST and SHOOT WEST. The player enters a command, and the computer responds with what the player sees, smells, hears, and experiences. The player is hunting for a Wumpus in a system of caverns, and must avoid traps, pits, and other dangers lying in wait. If you are interested, the rules of the game are easy to find on the web.

Hãy đắp chút da thịt lên bộ xương này. Giả sử trò chơi là Hunt the Wumpus, trò phiêu lưu kỳ cựu ra đời năm 1972. Trò này chạy bằng văn bản, dùng những lệnh rất đơn giản như GO EAST và SHOOT WEST. Người chơi gõ một lệnh, máy tính đáp lại bằng những gì người chơi thấy, ngửi, nghe và trải qua. Người chơi đang săn con Wumpus trong một hệ thống hang động, phải né bẫy, né hố và những hiểm nguy khác đang rình rập. Nếu bạn quan tâm, luật chơi rất dễ tìm trên mạng.

Let's assume that we'll keep the text-based UI, but decouple it from the game rules so that our version can use different languages in different markets. The game rules will communicate with the UI **component** using a language-independent API, and the UI will translate the API into the appropriate human language.

Giả sử ta vẫn giữ UI dạng văn bản, nhưng tách nó khỏi các quy tắc của trò chơi để phiên bản của ta có thể dùng những ngôn ngữ khác nhau ở những thị trường khác nhau. Các quy tắc của trò chơi sẽ trao đổi với component UI qua một API độc lập với ngôn ngữ, còn UI sẽ dịch API đó sang ngôn ngữ tự nhiên tương ứng.

If the source code dependencies are properly managed, as shown in Figure 25.1, then any number of UI components can reuse the same game rules. The game rules do not know, nor do they care, which human language is being used.

Nếu các phụ thuộc mã nguồn được quản lý đúng đắn, như trong Figure 25.1, thì bao nhiêu component UI cũng có thể tái dùng chung một bộ quy tắc trò chơi. Các quy tắc trò chơi không biết, mà cũng chẳng cần bận tâm, ngôn ngữ nào đang được dùng.

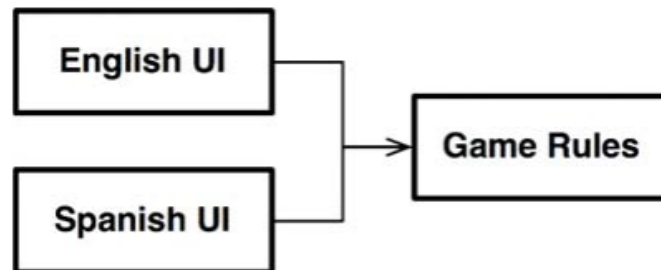


Figure 25.1 Any number of UI components can reuse the game rules

Figure 25.1 Bao nhiêu component UI cũng có thể tái dùng chung bộ quy tắc trò chơi

Let's also assume that the state of the game is maintained on some persistent store—perhaps in flash, or perhaps in the cloud, or maybe just in RAM. In any of those cases, we don't want the game rules to know the details. So, again, we'll create an API that the game rules can use to communicate with the data storage component.

Ta cũng giả sử trạng thái ván chơi được lưu trên một kho bền vững nào đó — có thể là flash, có thể là trên cloud, hoặc chỉ đơn giản là trong RAM. Trong mọi trường hợp, ta không muốn các quy tắc trò chơi biết những chi tiết đó. Vậy nên, một lần nữa, ta sẽ tạo một API để các quy tắc trò chơi dùng khi trao đổi với component lưu trữ dữ liệu.

We don't want the game rules to know anything about the different kinds of data storage, so the dependencies have to be properly directed following **the Dependency Rule**, as shown in Figure 25.2.

Ta không muốn các quy tắc trò chơi biết bất cứ điều gì về những kiểu lưu trữ dữ liệu khác nhau, nên các phụ thuộc phải được hướng đúng theo Quy tắc Phụ thuộc (the Dependency Rule), như trong Figure 25.2.

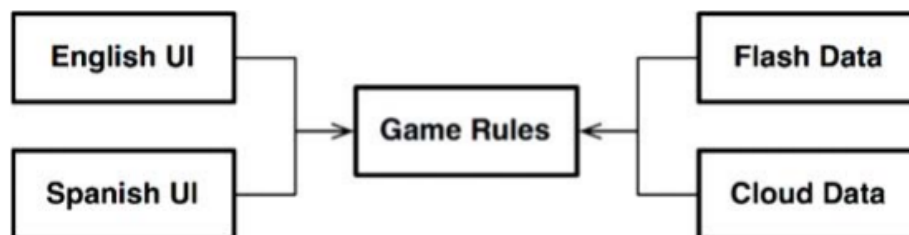


Figure 25.2 Following the Dependency Rule

Figure 25.2 Tuân theo Quy tắc Phụ thuộc

Clean Architecture? — Clean Architecture?

It should be clear that we could easily apply the **clean architecture** approach in this context, 1 with all the use cases, boundaries, entities, and corresponding data structures. But have we really found all the significant architectural boundaries?

Hẳn đã rõ rằng ta có thể dễ dàng áp dụng lối tiếp cận Clean Architecture vào bối cảnh này, 1 với đầy đủ use case, ranh giới (boundary), entity và các cấu trúc dữ liệu tương ứng. Nhưng liệu ta đã thực sự tìm ra hết những ranh giới kiến trúc đáng kể chưa?

For example, language is not the only axis of change for the UI. We also might want to vary the mechanism by which we communicate the text. For example, we might want to use a normal shell window, or text messages, or a chat application. There are many different possibilities.

Chẳng hạn, ngôn ngữ không phải là trục thay đổi duy nhất của UI. Ta cũng có thể muốn thay đổi cơ chế truyền tải văn bản. Ví dụ, ta có thể muốn dùng một cửa sổ shell thông thường, hoặc tin nhắn, hoặc một ứng dụng chat. Có rất nhiều khả năng khác nhau.

That means that there is a potential **architectural boundary** defined by this axis of change. Perhaps we should construct an API that crosses that **boundary** and isolates the language from the communications mechanism; that idea is illustrated in Figure 25.3.

Điều đó có nghĩa là trục thay đổi này định ra một ranh giới kiến trúc tiềm năng. Có lẽ ta nên dựng một API cắt ngang ranh giới ấy để cách ly ngôn ngữ khỏi cơ chế truyền tin; ý tưởng đó được minh họa trong Figure 25.3.

- 1. It should be just as clear that we would not apply the clean **architecture** approach to something as trivial as
 - 1. Cũng phải rõ không kém rằng ta sẽ không áp dụng lối tiếp cận Clean Architecture cho một thứ tầm thường như

this game. After all, the entire program can probably be written in 200 lines of code or less. In this case, we're using a simple program as a proxy for a much larger system with significant architectural boundaries.

trò chơi này. Suy cho cùng, cả chương trình chắc viết chưa tới 200 dòng code. Ở đây, ta dùng một chương trình đơn giản làm đại diện cho một hệ thống lớn hơn nhiều với những ranh giới kiến trúc đáng kể.

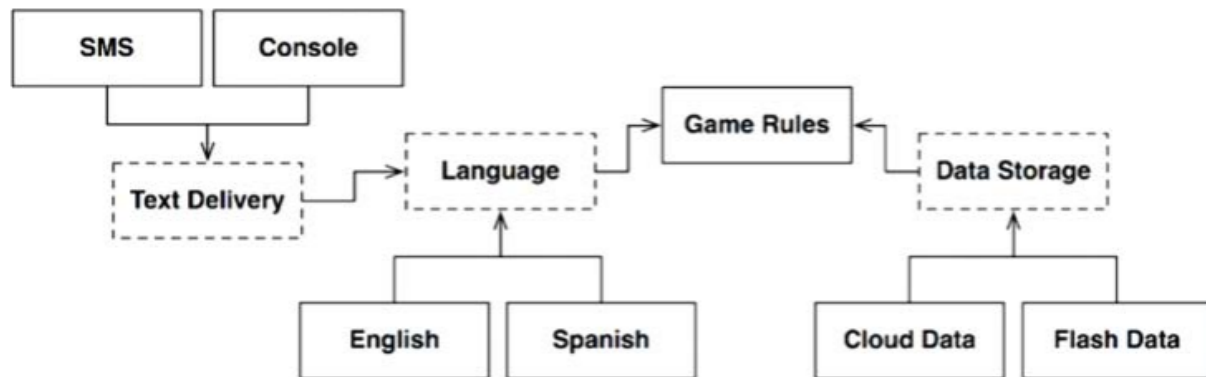


Figure 25.3 The revised diagram

Figure 25.3 Sơ đồ đã sửa lại

The diagram in Figure 25.3 has gotten a little complicated, but should contain no surprises. The dashed outlines indicate abstract components that define an API that is implemented by the components above or below them. For example, the Language API is implemented by English and Spanish .

Sơ đồ trong Figure 25.3 có hơi rối một chút, nhưng chẳng có gì bất ngờ. Các đường viền nét đứt chỉ những component trừu tượng định nghĩa một API được các component nằm trên hoặc dưới chúng hiện thực. Ví dụ, Language API được English và Spanish hiện thực.

GameRules communicates with Language through an API that GameRules defines and Language implements. Language communicates with TextDelivery using an API that Language defines but TextDelivery implements. The API is defined and owned by the user, rather than by the implementer.

GameRules trao đổi với Language qua một API do GameRules định nghĩa còn Language hiện thực. Language trao đổi với TextDelivery qua một API do Language định nghĩa nhưng TextDelivery hiện thực. API do bên sử dụng định nghĩa và sở hữu, chứ không phải bên hiện thực.

If we were to look inside GameRules , we would find polymorphic Boundary interfaces used by the code inside GameRules and implemented by the code inside the Language component. We would also find polymorphic Boundary interfaces used by Language and implemented by code inside

Nếu nhìn vào bên trong GameRules , ta sẽ thấy những interface Boundary đa hình được code bên trong GameRules sử dụng và được code bên trong component Language hiện thực. Ta cũng sẽ thấy những interface Boundary đa hình được Language sử dụng và được hiện thực bởi code bên trong

GameRules .

If we were to look inside of Language , we would find the same thing: Polymorphic Boundary interfaces implemented by the code inside TextDelivery , and polymorphic Boundary interfaces used by TextDelivery and implemented by Language .

Nếu nhìn vào bên trong Language , ta sẽ thấy y hệt như vậy: những interface Boundary đa hình được code bên trong TextDelivery hiện thực, và những interface Boundary đa hình được TextDelivery sử dụng còn Language hiện thực.

In each case, the API defined by those Boundary interfaces is owned by the upstream component.

Trong mọi trường hợp, API do các interface Boundary đó định nghĩa đều thuộc sở hữu của component ở thượng nguồn (upstream).

The variations, such as English , SMS , and CloudData , are provided by polymorphic interfaces defined in the abstract API component, and implemented by the concrete components that serve them. For example, we would expect polymorphic interfaces defined in Language to be implemented

Các biến thể, như English , SMS và CloudData , được cung cấp thông qua những interface đa hình định nghĩa trong component API trừu tượng, và được các component cụ thể phục vụ chúng hiện thực. Ví dụ, ta trông đợi những interface đa hình định nghĩa trong Language sẽ được hiện thực

by English and Spanish.

We can simplify this diagram by eliminating all the variations and focusing on just the API components. Figure 25.4 shows this diagram.

Ta có thể làm sơ đồ này gọn lại bằng cách bỏ hết các biến thể và chỉ tập trung vào các component API. Figure 25.4 cho thấy sơ đồ ấy.

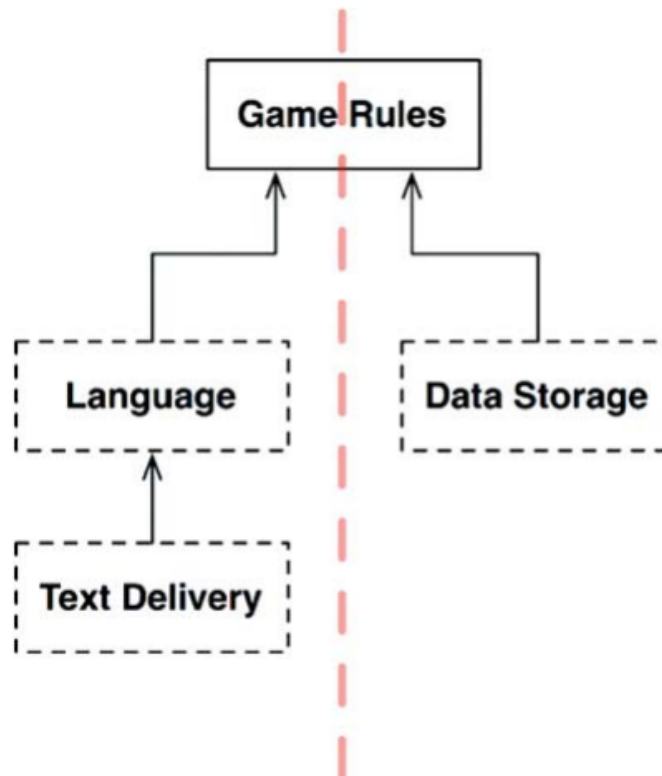


Figure 25.4 Simplified diagram

Figure 25.4 Sơ đồ đã rút gọn

Notice that the diagram is oriented in Figure 25.4 so that all the arrows point up. This puts GameRules at the top. This orientation makes sense because GameRules is the component that contains the highest-level policies.

Để ý rằng sơ đồ trong Figure 25.4 được xoay sao cho mọi mũi tên đều hướng lên. Điều này đặt GameRules lên trên cùng. Cách bố trí đó hợp lý, bởi GameRules là component chứa

những chính sách (policy) ở mức cao nhất.

Consider the direction of information flow. All input comes from the user through the TextDelivery component at the bottom left. That information rises through the Language component, getting translated into commands to GameRules. GameRules processes the user input and sends appropriate data down to DataStorage at the lower right.

Hãy xét chiều luồng thông tin. Mọi đầu vào đều đến từ người dùng, đi qua component TextDelivery ở góc dưới bên trái. Thông tin ấy dâng lên qua component Language, được dịch thành các lệnh gửi tới GameRules. GameRules xử lý đầu vào của người dùng rồi gửi dữ liệu tương ứng xuống DataStorage ở góc dưới bên phải.

GameRules then sends output back down to Language, which translates the API back to the appropriate language and then delivers that language to the user through TextDelivery.

Sau đó GameRules gửi kết quả ngược xuống Language, nơi dịch API trở lại ngôn ngữ phù hợp rồi chuyển ngôn ngữ đó tới người dùng thông qua TextDelivery.

This organization effectively divides the flow of data into two streams. 2 The stream on the left is concerned with communicating with the user, and the stream on the right is concerned with data persistence. Both streams meet at the top 3 at GameRules, which is the ultimate processor of the data that goes through both streams.

Cách tổ chức này trên thực tế chia luồng dữ liệu thành hai dòng. 2 Dòng bên trái lo việc giao tiếp với người dùng, còn dòng bên phải lo việc lưu trữ dữ liệu. Cả hai dòng gặp nhau ở trên đỉnh 3 tại GameRules, thứ rốt cuộc là bộ xử lý cho dữ liệu chảy qua cả hai dòng.

Crossing the Streams — Bắt chéo các dòng

Are there always two data streams as in this example? No, not at all. Imagine that we would like to play Hunt the Wumpus on the net with multiple players. In this case, we would need a network component, like that shown in Figure 25.5. This organization divides the data flow into three streams, all controlled by the

Có phải lúc nào cũng có hai dòng dữ liệu như trong ví dụ này không? Không hề. Hãy tưởng tượng ta muốn chơi Hunt the Wumpus trên mạng với nhiều người chơi. Khi đó ta sẽ cần một component mạng, như trong Figure 25.5. Cách tổ chức này chia luồng dữ liệu thành ba dòng, tất cả đều do

GameRules.

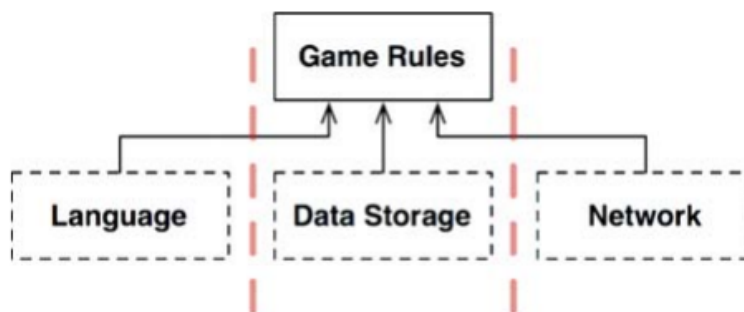


Figure 25.5 Adding a network component

Figure 25.5 Thêm một component mạng

So, as systems become more complex, the component structure may split into many such streams.

Vậy đó, khi hệ thống ngày càng phức tạp, cấu trúc component có thể tách ra thành nhiều dòng như thế.

- 2. If you are confused by the direction of the arrows, remember that they point in the direction of source
 - 2. Nếu bạn thấy khó hiểu về chiều của các mũi tên, hãy nhớ rằng chúng chỉ theo chiều của phụ thuộc

code dependencies, not in the direction of data flow.

mã nguồn, chứ không phải chiều của luồng dữ liệu.

- 3. In days long past, we would have called that top component the Central Transform. See Practical Guide
 - 3. Thuở xa xưa, ta hẳn đã gọi component trên đỉnh đó là Central Transform. Xem Practical Guide

to Structured Systems **Design**, 2nd ed., Meilir Page-Jones, 1988.

to Structured Systems Design, ấn bản thứ 2, Meilir Page-Jones, 1988.

Splitting the Streams — Tách các dòng

At this point you may be thinking that all the streams eventually meet at the top in a single component. If only life were so simple! The reality, of course, is much more complex.

Đến đây, có lẽ bạn đang nghĩ rằng rốt cuộc mọi dòng đều gặp nhau ở trên đỉnh trong một component duy nhất. Giá mà đời đơn giản đến thế! Dĩ nhiên, thực tế phức tạp hơn nhiều.

Consider the GameRules component for Hunt the Wumpus. Part of the game rules deal with the mechanics of the map. They know how the caverns are connected, and which objects are located in each cavern. They know how to move the player from cavern to cavern, and how to determine the events that the player must deal with.

Hãy xét component GameRules của Hunt the Wumpus. Một phần các quy tắc trò chơi lo phần cơ chế vận hành của tấm bản đồ. Chúng biết các hang động nối với nhau ra sao, và vật thể nào nằm trong hang nào. Chúng biết cách di chuyển người chơi từ hang này sang hang khác, và cách xác định những sự kiện mà người chơi phải đối mặt.

But there is another set of policies at an even higher level—policies that know the health of the player, and the cost or benefit of a particular event. These policies could cause the player to gradually lose health, or to gain health by discovering food. The lower-level mechanics **policy** would declare events to this higher-level policy, such as FoundFood or FellInPit. The higher-level policy would then manage the state of the player (as shown in Figure 25.6). Eventually that policy would decide whether the player wins or loses.

Nhưng còn một tập chính sách khác ở mức thậm chí còn cao hơn — những chính sách biết sức khỏe của người chơi, và cái giá hay lợi ích của một sự kiện cụ thể. Những chính sách này có thể khiến người chơi hao dần sức khỏe, hoặc hồi phục sức khỏe khi tìm được thức ăn. Chính sách cơ chế vận hành ở mức thấp hơn sẽ khai báo các sự kiện lên chính sách mức cao hơn này, chẳng hạn FoundFood hay FellInPit. Chính sách mức cao hơn sau

đó quản lý trạng thái của người chơi (như trong Figure 25.6). Rốt cuộc, chính sách đó sẽ quyết định người chơi thắng hay thua.

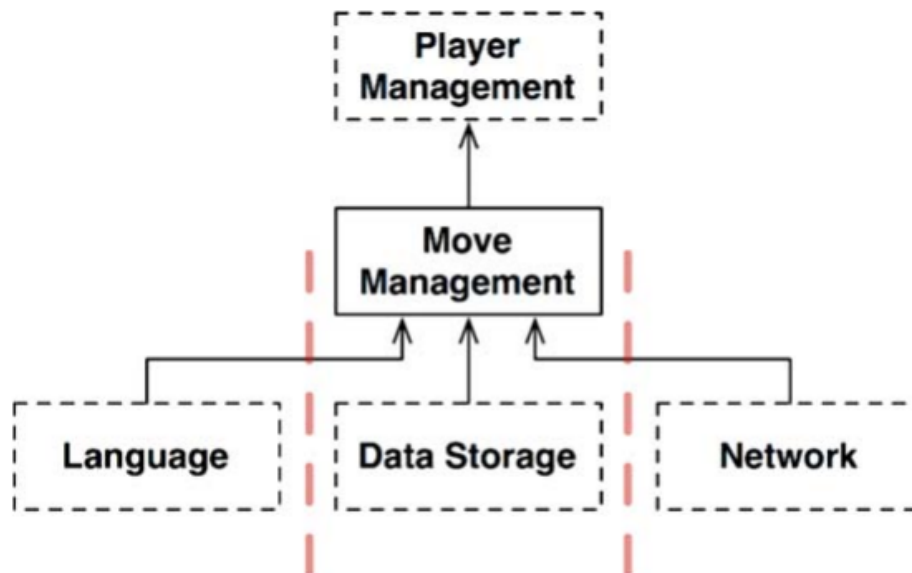


Figure 25.6 The higher-level policy manages the player

Figure 25.6 Chính sách mức cao hơn quản lý người chơi

Is this an architectural boundary? Do we need an API that separates MoveManagement from PlayerManagement? Well, let's make this a bit more interesting and add micro-services.

Đây có phải một ranh giới kiến trúc không? Ta có cần một API tách MoveManagement khỏi PlayerManagement không? Thôi thì, hãy làm cho chuyện này thú vị hơn một chút bằng cách thêm micro-service vào.

Let's assume that we've got a massive multiplayer version of Hunt the Wumpus. MoveManagement is handled locally within the player's computer, but PlayerManagement is handled by a server. PlayerManagement offers a **micro-service** API to all the connected MoveManagement components.

Giả sử ta có một phiên bản Hunt the Wumpus nhiều người chơi quy mô lớn. MoveManagement được xử lý cục bộ ngay trên máy của người chơi, còn PlayerManagement do một server đảm nhiệm. PlayerManagement cung cấp một API micro-service cho tất cả các component MoveManagement đang kết nối.

The diagram in Figure 25.7 depicts this scenario in a somewhat abbreviated fashion. The Network elements are a bit more complex than depicted—but you can probably still get the idea. A full-fledged architectural boundary

Sơ đồ trong Figure 25.7 mô tả kịch bản này theo lối hơi rút gọn. Các thành phần Network phức tạp hơn những gì được vẽ ra một chút — nhưng chắc bạn vẫn nắm được ý. Một ranh giới kiến trúc đầy đủ

exists between MoveManagement and PlayerManagement in this case.

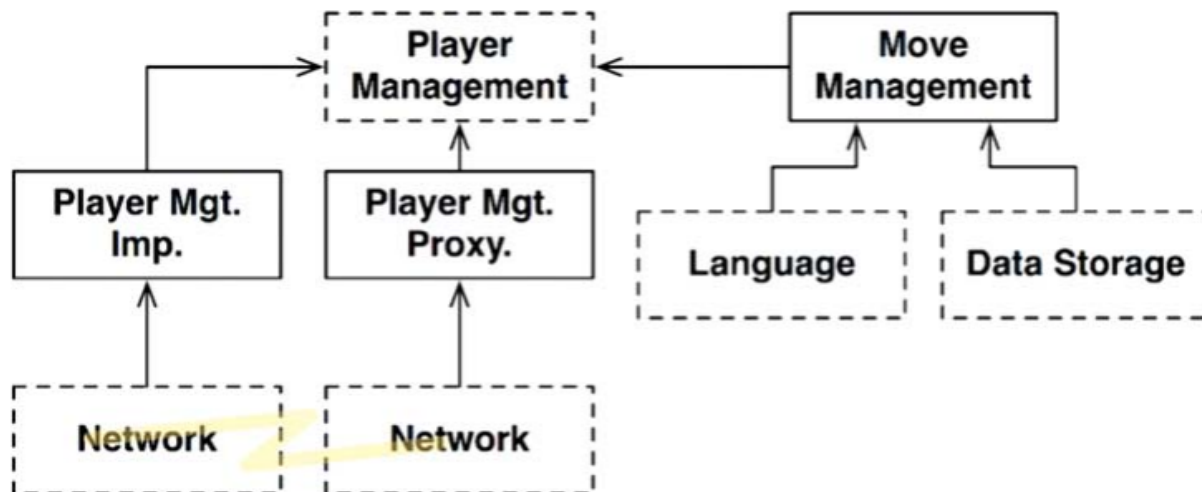


Figure 25.7 Adding a micro-**service** API

Figure 25.7 Thêm một API micro-service

Conclusion — Kết luận

What does all this mean? Why have I taken this absurdly simple program, which could be implemented in 200 lines of Kornshell, and extrapolated it out with all these crazy architectural boundaries?

Tất cả những chuyện này có ý nghĩa gì? Tại sao tôi lại lấy một chương trình đơn giản đến mức lố bịch, thứ có thể viết gọn trong 200 dòng Kornshell, rồi suy diễn nó ra thành đủ thứ ranh giới kiến trúc điên rồ như vậy?

This example is intended to show that architectural boundaries exist everywhere. We, as architects, must be careful to recognize when they are needed. We also have to be aware that such boundaries, when fully implemented, are expensive.

Ví dụ này nhằm cho thấy rằng ranh giới kiến trúc tồn tại ở khắp nơi. Chúng ta, với tư cách kiến trúc sư, phải cẩn thận nhận ra khi nào chúng thực sự cần thiết. Ta cũng phải ý thức rằng những ranh giới như thế, khi được hiện thực đầy đủ, là rất đắt.

At the same time, we have to recognize that when such boundaries are ignored, they are very expensive to add in later—even in the presence of comprehensive test-suites and refactoring discipline.

Đồng thời, ta phải nhận ra rằng khi những ranh giới ấy bị bỏ qua, cái giá để thêm chúng vào sau này là rất đắt — kể cả khi đã có bộ test đầy đủ và kỷ luật refactoring.

So what do we do, we architects? The answer is dissatisfying. On the one hand, some very smart people have told us, over the years, that we should not anticipate the need for abstraction. This is the philosophy of YAGNI: “You aren’t going to need it.” There is wisdom in this message, since over-engineering is often much worse than under-engineering. On the other hand, when you discover that you truly do need an architectural boundary where none exists, the costs and risks can be very high to add such a boundary.

Vậy chúng ta, những kiến trúc sư, phải làm gì? Câu trả lời không mấy dễ chịu. Một mặt, suốt bao năm qua, nhiều người rất thông minh đã bảo ta rằng đừng đoán trước nhu cầu trừu tượng hoá. Đó là triết lý YAGNI: “You aren’t going to need it.” (Bạn sẽ không cần tới nó đâu.) Thông điệp này có cái lý của nó, bởi thiết kế quá đà thường tệ hơn nhiều so với thiết kế thiếu. Mặt khác, khi bạn phát hiện mình thực sự cần một ranh giới kiến trúc ở nơi chưa hề có, chi phí và rủi ro để thêm ranh giới ấy vào có thể rất cao.

So there you have it. O Software Architect, you must see the future. You must guess—intelligently. You must weigh the costs and determine where the architectural boundaries lie, and which should be fully implemented, and which should be partially implemented, and which should be ignored.

Vậy đây, mọi chuyện là thế. Hỡi Kiến trúc sư Phần mềm, người phải thấy được tương lai. Người phải đoán — nhưng đoán một cách thông minh. Người phải cân nhắc cái giá và xác định xem những ranh giới kiến trúc nằm ở đâu, cái nào nên hiện thực đầy đủ, cái nào chỉ hiện thực một phần, và cái nào nên bỏ qua.

But this is not a one-time decision. You don’t simply decide at the start of a project which boundaries to implement and which to ignore. Rather, you watch . You pay attention as the system evolves. You note where boundaries may be required, and then carefully watch for the first inkling of friction because those boundaries don’t exist.

Nhưng đây không phải quyết định một lần rồi thôi. Bạn không đơn giản chỉ ngồi quyết ngay từ đầu dự án xem ranh giới nào sẽ làm và ranh giới nào sẽ bỏ. Thay vào đó, bạn quan sát . Bạn để mắt theo hệ thống khi nó tiến hoá. Bạn ghi nhận những nơi có thể sẽ cần ranh giới, rồi canh chừng thật kỹ dấu hiệu cọ xát đầu tiên nảy sinh vì những ranh giới ấy chưa có.

At that point, you weigh the costs of implementing those boundaries versus the cost of ignoring them—and you review that decision frequently. Your goal is to implement the boundaries right at the inflection point where the cost of implementing becomes less than the cost of ignoring.

Tới lúc đó, bạn cân đo chi phí hiện thực những ranh giới ấy so với chi phí của việc bỏ qua chúng — và bạn xem lại quyết định đó thường xuyên. Mục tiêu của bạn là hiện thực ranh giới đúng ngay tại điểm uốn, nơi chi phí hiện thực trở nên thấp hơn chi phí bỏ qua.

It takes a watchful eye.

Việc đó đòi hỏi một con mắt canh chừng.

The Main Component — Component Main

In every system, there is at least one **component** that creates, coordinates, and oversees the others. I call this component Main .

Trong mọi hệ thống, luôn có ít nhất một component tạo ra, điều phối và giám sát những component còn lại. Tôi gọi component đó là Main .

The Ultimate Detail — Chi tiết tận cùng

The **Main component** is the ultimate **detail**—the lowest-level **policy**. It is the initial entry point of the system. Nothing, other than the operating system, depends on it. Its job is to create all the Factories, Strategies, and other global facilities, and then hand control over to the **high-level** abstract portions of the system.

Component Main là chi tiết (detail) tận cùng — chính sách (policy) ở mức thấp nhất. Nó là điểm vào đầu tiên của hệ thống. Ngoài hệ điều hành ra, không có gì phụ thuộc vào nó. Việc của nó là tạo ra toàn bộ các Factory, Strategy và những tiện ích toàn cục khác, rồi trao quyền điều khiển cho các phần trừu tượng mức cao của hệ thống.

It is in this Main component that dependencies should be injected by a Dependency Injection **framework**. Once they are injected into Main , Main should distribute those dependencies normally, without using the framework.

Chính tại component Main này, các phụ thuộc nên được tiêm vào bởi một framework Dependency Injection. Một khi chúng đã được tiêm vào Main , Main nên phân phát các phụ thuộc đó theo cách thông thường, không dùng đến framework nữa.

Think of Main as the dirtiest of all the dirty components.

Hãy xem Main là component bẩn nhất trong tất cả những component bẩn.

Consider the following Main component from a recent version of Hunt the Wumpus. Notice how it loads up all the strings that we don't want the main body of the code to know about.

Hãy xem component Main sau đây, lấy từ một phiên bản gần đây của Hunt the Wumpus. Để ý cách nó nạp toàn bộ các chuỗi ký tự mà chúng ta không muốn phần thân chính của mã nguồn phải biết đến.

```
public class Main implements HtwMessageReceiver {  
    private static HuntTheWumpus game;  
    private static int hitPoints = 10;  
}
```

```

    private static final List<String> caverns = new
ArrayList<>();
    private static final String[] environments = new String[]{
        "bright",
        "humid",
        "dry",
        "creepy",
        "ugly",
        "foggy",
        "hot",

        "cold",
        "drafty",
        "dreadful"
    };
    private static final String[] shapes = new String[] {
        "round",
        "square",
        "oval",
        "irregular",
        "long",
        "craggy",
        "rough",
        "tall",
        "narrow"
    };
    private static final String[] cavernTypes = new String[] {
        "cavern",
        "room",
        "chamber",
        "catacomb",
        "crevasse",
        "cell",
        "tunnel",
        "passageway",
        "hall",
        "expanse"
    };
    private static final String[] adornments = new String[] {
        "smelling of sulfur",
        "with engravings on the walls",
        "with a bumpy floor",
        "",
        "littered with garbage",
        "spattered with guano",
        "with piles of Wumpus droppings",
        "with bones scattered around",
        "with a corpse on the floor",
        "that seems to vibrate",
        "that feels stuffy",
    };

```

```

    "that fills you with dread"
};

```

Now here's the main function. Notice how it uses the HtwFactory to create the game. It passes in the name of the class, htw.game.HuntTheWumpusFacade, because that class is even dirtier than Main. This prevents changes in that class from causing Main to recompile/redeploy.

Còn đây là hàm main. Để ý cách nó dùng HtwFactory để tạo ra trò chơi. Nó truyền vào tên của class, htw.game.HuntTheWumpusFacade, bởi vì class đó còn bẩn hơn cả Main. Nhờ vậy, thay đổi trong class đó không khiến Main phải biên dịch lại/triển khai lại.

```

public static void main(String[] args) throws IOException {
    game = HtwFactory.makeGame("htw.game.HuntTheWumpusFacade",
                                new Main());

    createMap();
    BufferedReader br =
        new BufferedReader(new InputStreamReader(System.in));
    game.makeRestCommand().execute();
    while (true) {
        System.out.println(game.getPlayerCavern());
        System.out.println("Health: " + hitPoints + " arrows: " +
                            game.getQuiver());
        HuntTheWumpus.Command c = game.makeRestCommand();

        System.out.println(">");
        String command = br.readLine();
        if (command.equalsIgnoreCase("e"))
            c = game.makeMoveCommand(EAST);
        else if (command.equalsIgnoreCase("w"))
            c = game.makeMoveCommand(WEST);
        else if (command.equalsIgnoreCase("n"))
            c = game.makeMoveCommand(NORTH);
        else if (command.equalsIgnoreCase("s"))
            c = game.makeMoveCommand(SOUTH);
        else if (command.equalsIgnoreCase("r"))
            c = game.makeRestCommand();
        else if (command.equalsIgnoreCase("sw"))
            c = game.makeShootCommand(WEST);
        else if (command.equalsIgnoreCase("se"))
            c = game.makeShootCommand(EAST);
        else if (command.equalsIgnoreCase("sn"))
            c = game.makeShootCommand(NORTH);
        else if (command.equalsIgnoreCase("ss"))
            c = game.makeShootCommand(SOUTH);
        else if (command.equalsIgnoreCase("q"))
            return;
        c.execute();
    }
}

```

Notice also that main creates the input stream and contains the main loop of the game, interpreting the simple input commands, but then defers all processing to other, higher-level components.

Cũng hãy để ý rằng main tạo ra luồng nhập liệu và chứa vòng lặp chính của trò chơi, diễn giải các lệnh nhập đơn giản, nhưng rồi giao lại toàn bộ phần xử lý cho những component khác ở mức cao hơn.

Finally, notice that main creates the map.

Cuối cùng, hãy để ý rằng main tạo ra bản đồ.

```
private static void createMap() {
    int nCaverns = (int) (Math.random() * 30.0 + 10.0);
    while (nCaverns-- > 0)
        caverns.add(makeName());
    for (String cavern : caverns) {
        maybeConnectCavern(cavern, NORTH);
        maybeConnectCavern(cavern, SOUTH);
        maybeConnectCavern(cavern, EAST);
        maybeConnectCavern(cavern, WEST);
    }
    String playerCavern = anyCavern();
    game.setPlayerCavern(playerCavern);
    game.setWumpusCavern(anyOther(playerCavern));
    game.addBatCavern(anyOther(playerCavern));
    game.addBatCavern(anyOther(playerCavern));
    game.addBatCavern(anyOther(playerCavern));
    game.addPitCavern(anyOther(playerCavern));
    game.addPitCavern(anyOther(playerCavern));
    game.addPitCavern(anyOther(playerCavern));
    game.setQuiver(5);
}
// much code removed...
}
```

The point is that Main is a dirty **low-level** module in the outermost circle of the **clean architecture**. It loads everything up for the high level system, and then hands control over to it.

Điểm mấu chốt là: Main là một module bẩn, mức thấp, nằm ở vòng ngoài cùng của Clean Architecture. Nó nạp mọi thứ lên cho hệ thống mức cao, rồi trao quyền điều khiển cho hệ thống ấy.

Conclusion — Kết luận

Think of Main as a plugin to the application—a plugin that sets up the initial conditions and configurations, gathers all the outside resources, and then hands control over to the high-level policy of the application. Since it is a plugin, it is possible to have many Main components, one for each configuration of your application.

Hãy xem Main như một plugin của ứng dụng — một plugin thiết lập các điều kiện và cấu hình ban đầu, thu gom mọi tài nguyên bên ngoài, rồi trao quyền điều khiển cho chính sách mức cao của ứng dụng. Vì nó là plugin, ta hoàn toàn có thể có nhiều component Main, mỗi cái ứng với một cấu hình của ứng dụng.

For example, you could have a Main plugin for Dev, another for Test, and yet another for Production. You could also have a Main plugin for each country you deploy to, or each jurisdiction, or each

customer.

Chẳng hạn, bạn có thể có một plugin Main cho Dev , một cái khác cho Test , và một cái nữa cho Production . Bạn cũng có thể có một plugin Main cho mỗi quốc gia mà bạn triển khai tới, hoặc mỗi khu vực pháp lý, hoặc mỗi khách hàng.

When you think about Main as a plugin component, sitting behind an **architectural boundary**, the problem of configuration becomes a lot easier to solve.

Khi bạn nghĩ về Main như một component plugin nằm sau một ranh giới kiến trúc, bài toán cấu hình bỗng trở nên dễ giải hơn rất nhiều.

Services: Great and Small — Dịch vụ: Lớn và nhỏ

Service-oriented “architectures” and **micro-service** “architectures” have become very popular of late. The reasons for their current popularity include the following:

Các “kiến trúc” hướng dịch vụ (service-oriented) và “kiến trúc” micro-service gần đây rất được ưa chuộng. Những lý do cho sự ưa chuộng hiện nay của chúng gồm:

- Services seem to be strongly decoupled from each other. As we shall see,
 - Các dịch vụ (service) có vẻ được tách rời (decoupling) mạnh mẽ khỏi nhau. Nhưng như ta sẽ thấy,

this is only partially true.

điều này chỉ đúng một phần.

- Services appear to support independence of development and deployment.
 - Các dịch vụ có vẻ hỗ trợ khả năng phát triển và triển khai độc lập.

Again, as we shall see, this is only partially true.

Một lần nữa, như ta sẽ thấy, điều này cũng chỉ đúng một phần.

Service Architecture? — Kiến trúc dịch vụ?

First, let’s consider the notion that using services, by their nature, is an **architecture**. This is patently untrue. The architecture of a system is defined by boundaries that separate **high-level policy** from **low-level detail** and follow **the Dependency Rule**. Services that simply separate application behaviors are little more than expensive function calls, and are not necessarily architecturally significant.

Trước hết, hãy xét cái ý niệm rằng dùng dịch vụ, tự bản chất của nó, đã là một kiến trúc (architecture). Điều đó rõ ràng là sai. Kiến trúc của một hệ thống được định nghĩa bởi những ranh giới (boundary) tách chính sách mức cao khỏi chi tiết mức thấp và tuân theo Quy tắc Phụ thuộc (the Dependency Rule). Những dịch vụ chỉ đơn thuần tách các hành vi của ứng dụng ra thì chẳng khác gì mấy so với những lời gọi hàm đắt đỏ, và không nhất thiết có ý nghĩa về mặt kiến trúc.

This is not to say that all services should be architecturally significant. There are often substantial benefits to creating services that separate functionality across processes and platforms—whether

they obey the Dependency Rule or not. It's just that services, in and of themselves, do not define an architecture.

Nói vậy không có nghĩa là mọi dịch vụ đều phải có ý nghĩa kiến trúc. Việc tạo ra các dịch vụ để tách chức năng qua nhiều tiến trình và nền tảng thường mang lại lợi ích đáng kể — dù chúng có tuân theo Quy tắc Phụ thuộc hay không. Chỉ là bản thân các dịch vụ, tự chúng, không định nghĩa nên một kiến trúc.

A helpful analogy is the organization of functions. The architecture of a monolithic or **component**-based system is defined by certain function calls that cross architectural boundaries and follow the Dependency Rule. Many other functions in those systems, however, simply separate one **behavior** from another and are not architecturally significant.

Một phép so sánh hữu ích là cách tổ chức các hàm. Kiến trúc của một hệ thống nguyên khối hay dựa trên component được định nghĩa bởi một số lời gọi hàm nhất định vượt qua các ranh giới kiến trúc và tuân theo Quy tắc Phụ thuộc. Tuy nhiên, nhiều hàm khác trong những hệ thống đó chỉ đơn giản tách hành vi này khỏi hành vi kia, và không có ý nghĩa kiến trúc.

So it is with services. Services are, after all, just function calls across process and/or platform boundaries. Some of those services are architecturally significant, and some aren't. Our interest, in this chapter, is with the former.

Với dịch vụ thì cũng thế. Suy cho cùng, dịch vụ chỉ là những lời gọi hàm vượt qua ranh giới tiến trình và/hoặc nền tảng. Một số dịch vụ có ý nghĩa kiến trúc, một số thì không. Trong chương này, ta quan tâm tới loại thứ nhất.

Service Benefits? — Lợi ích của dịch vụ?

The question mark in the preceding heading indicates that this section is going to challenge the current popular orthodoxy of service architecture. Let's tackle the benefits one at a time.

Dấu hỏi trong tiêu đề trên cho thấy phần này sắp thách thức thứ giáo lý chính thống đang thịnh hành về kiến trúc dịch vụ. Hãy xử lý từng lợi ích một.

The Decoupling Fallacy — Nguy hiểm về tách rời

One of the big supposed benefits of breaking a system up into services is that services are strongly decoupled from each other. After all, each service runs in a different process, or even a different processor; therefore those services do not have access to each other's variables. What's more, the **interface** of each service must be well defined.

Một trong những lợi ích lớn được cho là có khi chẻ hệ thống thành các dịch vụ là các dịch vụ được tách rời mạnh mẽ khỏi nhau. Rốt cuộc thì mỗi dịch vụ chạy trong một tiến trình khác nhau, thậm chí trên một bộ xử lý khác nhau; do đó chúng không truy cập được biến của nhau. Hơn nữa, interface của mỗi dịch vụ buộc phải được định nghĩa rõ ràng.

There is certainly some truth to this—but not very much truth. Yes, services are decoupled at the **level** of individual variables. However, they can still be coupled by shared resources within a processor, or on the network. What's more, they are strongly coupled by the data they share.

Điều này chắc chắn có phần đúng — nhưng đúng không nhiều lắm. Vâng, các dịch vụ được tách rời ở mức từng biến riêng lẻ. Tuy nhiên, chúng vẫn có thể bị kết dính với nhau qua

các tài nguyên dùng chung bên trong một bộ xử lý, hoặc trên mạng. Hơn thế nữa, chúng bị kết dính rất mạnh qua dữ liệu mà chúng chia sẻ.

For example, if a new field is added to a data record that is passed between services, then every service that operates on the new field must be changed. The services must also strongly agree about the interpretation of the data in that field. Thus those services are strongly coupled to the data record and, therefore, indirectly coupled to each other.

Chẳng hạn, nếu ta thêm một trường mới vào một bản ghi dữ liệu được truyền qua lại giữa các dịch vụ thì mọi dịch vụ có thao tác trên trường mới đó đều phải thay đổi. Các dịch vụ ấy cũng phải thống nhất chặt chẽ với nhau về cách diễn giải dữ liệu trong trường đó. Vậy nên chúng bị kết dính mạnh với bản ghi dữ liệu và, qua đó, gián tiếp kết dính với nhau.

As for interfaces being well defined, that's certainly true—but it is no less true for functions. Service interfaces are no more formal, no more rigorous, and no better defined than function interfaces. Clearly, then, this benefit is something of an illusion.

Còn chuyện interface được định nghĩa rõ ràng, điều đó chắc chắn đúng — nhưng với hàm thì cũng đúng y như vậy. Interface của dịch vụ không hề hình thức hơn, không hề chặt chẽ hơn, và cũng chẳng được định nghĩa tốt hơn interface của hàm. Rõ ràng, lợi ích này ít nhiều chỉ là ảo tưởng.

The Fallacy of Independent Development and Deployment — Ngụy biện về phát triển và triển khai độc lập

Another of the supposed benefits of services is that they can be owned and operated by a dedicated team. That team can be responsible for writing, maintaining, and operating the service as part of a dev-ops strategy. This independence of development and deployment is presumed to be scalable. It is believed that large enterprise systems can be created from dozens, hundreds, or even thousands of independently developable and deployable services. Development, maintenance, and operation of the system can be partitioned between a similar number of independent teams.

Một lợi ích được cho là có nữa của dịch vụ là chúng có thể do một đội chuyên trách sở hữu và vận hành. Đội đó chịu trách nhiệm viết, bảo trì và vận hành dịch vụ như một phần của chiến lược dev-ops. Khả năng phát triển và triển khai độc lập này được cho là *có thể mở rộng quy mô*. Người ta tin rằng các hệ thống doanh nghiệp lớn có thể được tạo nên từ hàng chục, hàng trăm, thậm chí hàng nghìn dịch vụ phát triển và triển khai độc lập được. Việc phát triển, bảo trì và vận hành hệ thống có thể chia cho một số lượng đội độc lập tương đương.

There is some truth to this belief—but only some. First, history has shown that large enterprise systems can be built from monoliths and component-based systems as well as service-based systems. Thus services are not the only option for building scalable systems.

Niềm tin này có phần đúng — nhưng chỉ một phần. Thứ nhất, lịch sử đã cho thấy các hệ thống doanh nghiệp lớn có thể được xây từ hệ nguyên khối và hệ dựa trên component, chứ không chỉ từ hệ dựa trên dịch vụ. Vậy nên dịch vụ không phải lựa chọn duy nhất để xây hệ thống mở rộng được.

Second, the **decoupling** fallacy means that services cannot always be independently developed, deployed, and operated. To the extent that they are coupled by data or behavior, the development, deployment, and operation must be coordinated.

Thứ hai, ngụy biện về tách rời có nghĩa là các dịch vụ không phải lúc nào cũng có thể được phát triển, triển khai và vận hành độc lập. Chừng nào chúng còn bị kết dính qua dữ liệu

hay hành vi, thì việc phát triển, triển khai và vận hành vẫn phải được phối hợp với nhau.

The Kitty Problem — Bài toán “mèo cưng”

As an example of these two fallacies, let’s look at our taxi aggregator system again. Remember, this system knows about many taxi providers in a given city, and allows customers to order rides. Let’s assume that the customers select taxis based on a number of criteria, such as pickup time, cost, luxury, and driver experience.

Để minh họa hai nguy hiểm này, hãy nhìn lại hệ thống tổng hợp dịch vụ taxi của chúng ta. Nhớ rằng hệ thống này biết về nhiều nhà cung cấp taxi trong một thành phố, và cho phép khách hàng đặt chuyến. Giả sử khách hàng chọn taxi dựa trên một số tiêu chí, chẳng hạn thời gian đón, chi phí, mức sang trọng và kinh nghiệm tài xế.

We wanted our system to be scalable, so we chose to build it out of lots of little micro-services. We subdivided our development staff into many small teams, each of which is responsible for developing, maintaining, and operating a correspondingly 1 small number of services.

Ta muốn hệ thống của mình mở rộng được, nên đã chọn xây nó từ rất nhiều micro-service nhỏ. Ta chia đội ngũ phát triển thành nhiều nhóm nhỏ, mỗi nhóm chịu trách nhiệm phát triển, bảo trì và vận hành một số lượng dịch vụ nhỏ tương ứng 1.

The diagram in Figure 27.1 shows how our fictitious architects arranged services to implement this application. The TaxiUI service deals with the customers, who use mobile devices to order taxis. The TaxiFinder service examines the inventories of the various TaxiSuppliers and determines which taxis are possible candidates for the user. It deposits these into a short-term data record attached to that user. The TaxiSelector service takes the user’s criteria of cost, time, luxury, and so forth, and chooses an appropriate taxi from among the candidates. It hands that taxi off to the TaxiDispatcher service, which orders the appropriate taxi.

Sơ đồ trong Figure 27.1 cho thấy các kiến trúc sư tưởng tượng của ta đã sắp xếp các dịch vụ ra sao để hiện thực ứng dụng này. Dịch vụ TaxiUI giao tiếp với khách hàng, những người dùng thiết bị di động để gọi taxi. Dịch vụ TaxiFinder xem xét kho xe của các TaxiSupplier khác nhau và xác định những chiếc taxi nào là ứng viên khả dĩ cho người dùng. Nó ghi các ứng viên đó vào một bản ghi dữ liệu ngắn hạn gắn với người dùng ấy. Dịch vụ TaxiSelector lấy các tiêu chí của người dùng về chi phí, thời gian, mức sang trọng, v.v., rồi chọn ra chiếc taxi phù hợp trong số các ứng viên. Nó chuyển chiếc taxi đó cho dịch vụ TaxiDispatcher, và dịch vụ này đặt đúng chiếc taxi ấy.

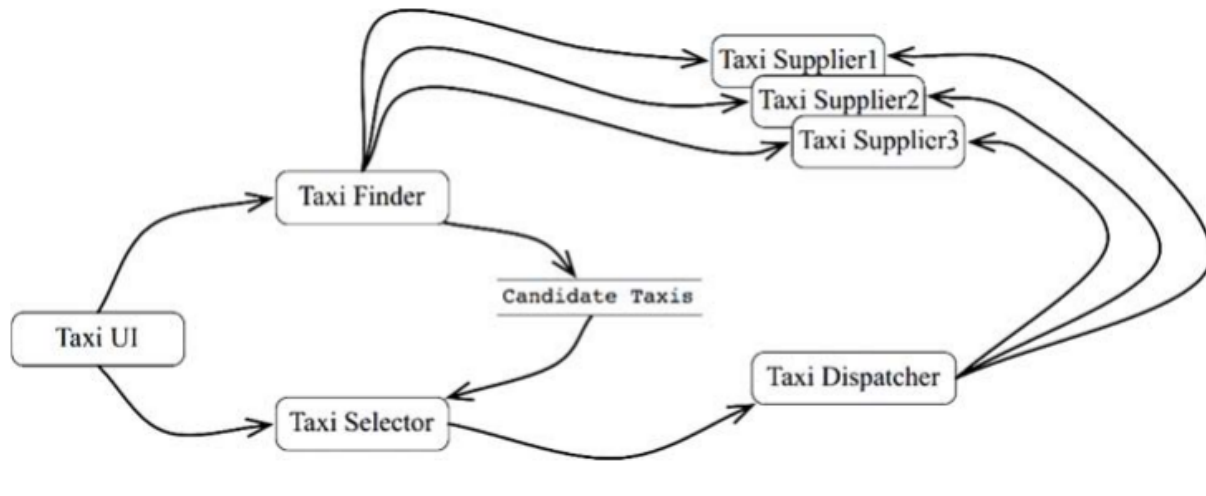


Figure 27.1 Services arranged to implement the taxi aggregator system

Figure 27.1 Các dịch vụ được sắp xếp để hiện thực hệ thống tổng hợp taxi

- 1. Therefore the number of micro-services will be roughly equal to the number of programmers.
- 1. Vì vậy số lượng micro-service sẽ xấp xỉ bằng số lượng lập trình viên.

Now let us suppose that this system has been in operation for more than a year. Our staff of developers have been happily developing new features while maintaining and operating all these services.

Bây giờ hãy giả sử hệ thống này đã vận hành hơn một năm. Đội ngũ lập trình viên của ta vẫn vui vẻ phát triển tính năng mới trong khi bảo trì và vận hành tất cả những dịch vụ đó.

One bright and cheerful day, the marketing department holds a meeting with the development team. In this meeting, they announce their plans to offer a kitten delivery service to the city. Users can order kittens to be delivered to their homes or to their places of business.

Một ngày tươi sáng và vui vẻ nọ, phòng marketing họp với đội phát triển. Trong cuộc họp, họ công bố kế hoạch cung cấp dịch vụ giao mèo con trong thành phố. Người dùng có thể đặt mèo con giao tới nhà hoặc tới nơi làm việc.

The company will set up several kitten collection points across the city. When a kitten order is placed, a nearby taxi will be selected to collect a kitten from one of those collection points, and then deliver it to the appropriate address.

Công ty sẽ lập ra vài điểm tập kết mèo con khắp thành phố. Khi có đơn đặt mèo con, một chiếc taxi ở gần sẽ được chọn để tới lấy mèo tại một trong các điểm tập kết ấy, rồi giao tới đúng địa chỉ.

One of the taxi suppliers has agreed to participate in this program. Others are likely to follow. Still others may decline.

Một nhà cung cấp taxi đã đồng ý tham gia chương trình này. Các bên khác nhiều khả năng sẽ theo sau. Cũng có những bên có thể sẽ từ chối.

Of course, some drivers may be allergic to cats, so those drivers should never be selected for this service. Also, some customers will undoubtedly have similar allergies, so a vehicle that has been

used to deliver kittens within the last 3 days should not be selected for customers who declare such allergies.

Dĩ nhiên, một số tài xế có thể dị ứng mèo, nên tuyệt đối không được chọn họ cho dịch vụ này. Ngoài ra, chắc chắn cũng có khách hàng bị dị ứng tương tự, nên một chiếc xe từng chở mèo con trong vòng 3 ngày gần nhất không được chọn cho những khách hàng đã khai báo dị ứng.

Look at that diagram of services. How many of those services will have to change to implement this feature? All of them . Clearly, the development and deployment of the kitty feature will have to be very carefully coordinated.

Hãy nhìn lại sơ đồ các dịch vụ đó. Bao nhiêu dịch vụ sẽ phải thay đổi để hiện thực tính năng này? *Tất cả*. Rõ ràng, việc phát triển và triển khai tính năng “mèo cưng” sẽ phải được phối hợp hết sức cẩn thận.

In other words, the services are all coupled, and cannot be independently developed, deployed, and maintained.

Nói cách khác, các dịch vụ đều bị kết dính với nhau, và không thể được phát triển, triển khai, bảo trì một cách độc lập.

This is the problem with cross-cutting concerns. Every software system must face this problem, whether service oriented or not. Functional decompositions, of the kind depicted in the service diagram in Figure 27.1, are very vulnerable to new features that cut across all those functional behaviors.

Đây chính là vấn đề của các mối quan tâm xuyên suốt (cross-cutting concern). Mọi hệ thống phần mềm đều phải đối mặt với vấn đề này, dù hướng dịch vụ hay không. Cách phân rã theo hàm, kiểu như sơ đồ dịch vụ trong Figure 27.1, rất dễ tổn thương trước những tính năng mới cắt ngang qua toàn bộ các hành vi chức năng ấy.

Objects to the Rescue — Đối tượng ra tay cứu nguy

How would we have solved this problem in a component-based architecture? Careful consideration of the SOLID **design** principles would have prompted us to create a set of classes that could be polymorphically extended to handle new features.

Ta sẽ giải quyết vấn đề này ra sao trong một kiến trúc dựa trên component? Cân nhắc kỹ các nguyên tắc thiết kế SOLID sẽ dẫn ta tới việc tạo ra một tập các class có thể mở rộng theo kiểu đa hình (polymorphism) để xử lý các tính năng mới.

The diagram in Figure 27.2 shows the strategy. The classes in this diagram roughly correspond to the services shown in Figure 27.1. However, note the boundaries. Note also that the dependencies follow the Dependency Rule.

Sơ đồ trong Figure 27.2 cho thấy chiến lược đó. Các class trong sơ đồ này tương ứng đại khái với các dịch vụ trong Figure 27.1. Tuy nhiên, hãy để ý các ranh giới. Cũng hãy để ý rằng các phụ thuộc đều tuân theo Quy tắc Phụ thuộc.

Much of the logic of the original services is preserved within the base classes of the object model. However, that portion of the logic that was specific to rides has been extracted into a Rides component. The new feature for kittens has been placed into a Kittens component. These two components override the abstract base classes in the original components using a pattern such as Template Method or Strategy .

Phần lớn logic của các dịch vụ ban đầu được giữ lại trong các base class của mô hình đối tượng. Tuy nhiên, phần logic dành riêng cho các chuyến xe đã được tách ra thành component Rides. Tính năng mới cho mèo con được đặt vào component Kittens. Hai component này override các abstract base class trong các component gốc bằng một mẫu như Template Method hay *Strategy*.

Note again that the two new components, Rides and Kittens , follow the Dependency Rule. Note also that the classes that implement those features are created by factories under the control of the UI.

Xin nhắc lại, hãy để ý rằng hai component mới, Rides và Kittens, đều tuân theo Quy tắc Phụ thuộc. Cũng hãy để ý rằng các class hiện thực những tính năng đó được tạo bởi các factory nằm dưới quyền điều khiển của UI.

Clearly, in this scheme, when the Kitty feature is implemented, the TaxiUI must change. But nothing else needs to be changed. Rather, a new jar file, or Gem, or DLL is added to the system and dynamically loaded at runtime.

Rõ ràng, trong cách bố trí này, khi tính năng “mèo cưng” được hiện thực thì TaxiUI buộc phải thay đổi. Nhưng không có gì khác phải đổi cả. Thay vào đó, ta chỉ thêm một file jar mới, hoặc một Gem, hoặc một DLL vào hệ thống và nạp động lúc chạy.

Thus the Kitty feature is decoupled, and independently developable and deployable.

Nhờ vậy, tính năng “mèo cưng” được tách rời, phát triển và triển khai độc lập được.

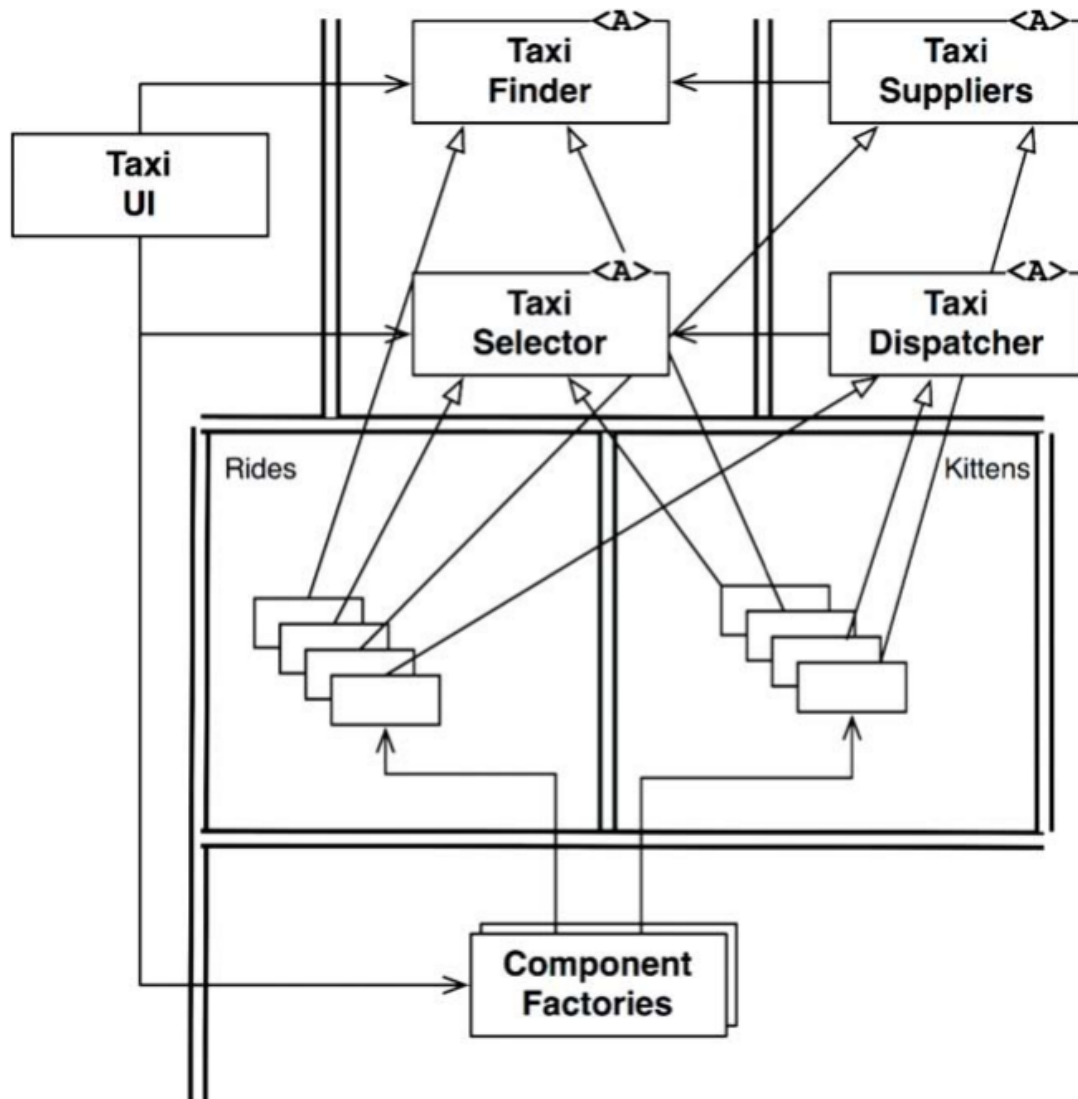


Figure 27.2 Using an object-oriented approach to deal with cross-cutting concerns

Figure 27.2 Dùng cách tiếp cận hướng đối tượng để xử lý các mối quan tâm xuyên suốt

Component-Based Services — Dịch vụ dựa trên component

The obvious question is: Can we do that for services? And the answer is, of course: Yes! Services do not need to be little monoliths. Services can, instead, be designed using **the SOLID principles**, and given a component **structure** so that new components can be added to them without changing the existing components within the service.

Câu hỏi hiển nhiên là: Ta có làm được như vậy với dịch vụ không? Và câu trả lời, dĩ nhiên, là: Có! Dịch vụ không nhất thiết phải là những khối nguyên khối tí hon. Thay vào đó, dịch vụ có thể được thiết kế theo các nguyên tắc SOLID, và được cho một cấu trúc component sao cho ta thêm được component mới vào chúng mà không phải sửa các component sẵn có bên trong dịch vụ.

Think of a service in Java as a set of abstract classes in one or more jar files. Think of each new feature or feature extension as another jar file that contains classes that extend the abstract classes in the first jar files. Deploying a new feature then becomes not a matter of redeploying the services, but rather a matter of simply adding the new jar files to the load paths of those services. In other words, adding new features conforms to the Open-Closed Principle.

Hãy hình dung một dịch vụ trong Java là một tập các abstract class nằm trong một hoặc vài file jar. Hãy hình dung mỗi tính năng mới hay phần mở rộng tính năng là một file jar khác, chứa những class kế thừa các abstract class trong các file jar đầu tiên. Triển khai một tính năng mới khi đó không còn là chuyện triển khai lại các dịch vụ, mà chỉ đơn giản là thêm các file jar mới vào load path của những dịch vụ ấy. Nói cách khác, việc thêm tính năng mới tuân thủ Open-Closed Principle.

The service diagram in Figure 27.3 shows the structure. The services still exist as before, but each has its own internal component design, allowing new features to be added as new derivative classes. Those derivative classes live within their own components.

Sơ đồ dịch vụ trong Figure 27.3 cho thấy cấu trúc này. Các dịch vụ vẫn tồn tại như trước, nhưng mỗi dịch vụ có thiết kế component nội bộ của riêng mình, cho phép thêm tính năng mới dưới dạng các class dẫn xuất mới. Những class dẫn xuất ấy nằm trong component của riêng chúng.

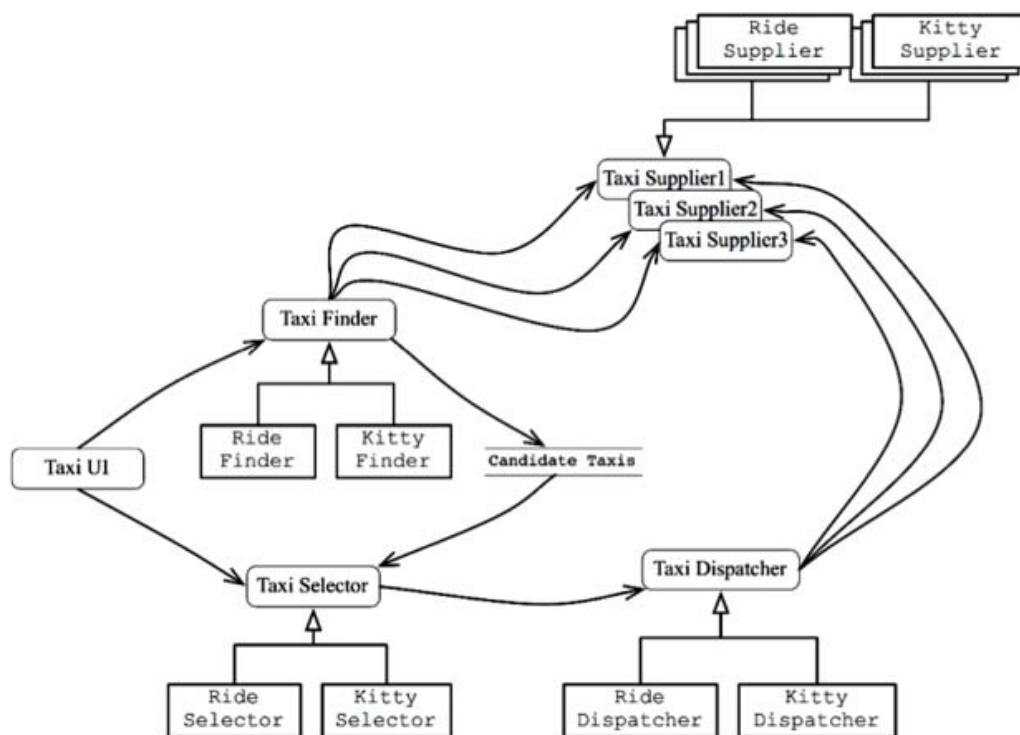


Figure 27.3 Each service has its own internal component design, enabling new features to be

Figure 27.3 Mỗi dịch vụ có thiết kế component nội bộ của riêng mình, cho phép thêm tính năng mới

added as new derivative classes

dưới dạng các class dẫn xuất mới

Cross-Cutting Concerns — Mỗi quan tâm xuyên suốt

What we have learned is that architectural boundaries do not fall between services. Rather, those boundaries run through the services, dividing them into components.

Điều ta rút ra được là: các ranh giới kiến trúc không nằm giữa các dịch vụ. Đúng hơn, những ranh giới đó chạy xuyên qua các dịch vụ, chia chúng thành các component.

To deal with the cross-cutting concerns that all significant systems face, services must be designed with internal component architectures that follow the Dependency Rule, as shown in the diagram in Figure 27.4. Those services do not define the architectural boundaries of the system; instead, the components within the services do.

Để xử lý các mối quan tâm xuyên suốt mà mọi hệ thống đáng kể đều phải đối mặt, dịch vụ phải được thiết kế với kiến trúc component nội bộ tuân theo Quy tắc Phụ thuộc, như trong sơ đồ ở Figure 27.4. Không phải các dịch vụ đó định nghĩa ranh giới kiến trúc của hệ thống; mà chính các component bên trong dịch vụ mới làm việc ấy.

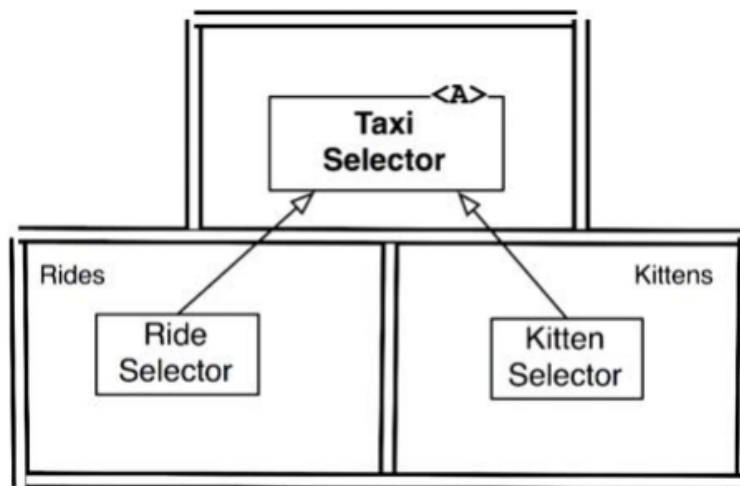


Figure 27.4 Services must be designed with internal component architectures that follow

Figure 27.4 Dịch vụ phải được thiết kế với kiến trúc component nội bộ tuân theo the Dependency Rule

Quy tắc Phụ thuộc

Conclusion — Kết luận

As useful as services are to the scalability and develop-ability of a system, they are not, in and of themselves, architecturally significant elements. The architecture of a system is defined by the boundaries drawn within that system, and by the dependencies that cross those boundaries. That architecture is not defined by the physical mechanisms by which elements communicate and execute.

Dù dịch vụ hữu ích tới đâu cho khả năng mở rộng và khả năng phát triển của một hệ thống, tự thân chúng vẫn không phải là những thành tố có ý nghĩa kiến trúc. Kiến trúc của một

hệ thống được định nghĩa bởi các ranh giới vạch ra bên trong hệ thống đó, và bởi các phụ thuộc vượt qua những ranh giới ấy. Kiến trúc ấy không được định nghĩa bởi cơ chế vật lý mà nhờ đó các thành tố giao tiếp và thực thi.

A service might be a single component, completely surrounded by an **architectural boundary**. Alternatively, a service might be composed of several components separated by architectural boundaries. In rare 2 cases, clients and services may be so coupled as to have no architectural significance whatever.

Một dịch vụ có thể là một component duy nhất, được bao trọn bởi một ranh giới kiến trúc. Hoặc ngược lại, một dịch vụ có thể gồm nhiều component được ngăn cách bởi các ranh giới kiến trúc. Trong những trường hợp hiếm hoi 2, client và dịch vụ có thể kết dính với nhau tới mức chẳng còn chút ý nghĩa kiến trúc nào.

- 2. We hope they are rare. Unfortunately, experience suggests otherwise.
 - 2. Chúng tôi hy vọng đó là chuyện hiếm. Đáng tiếc, kinh nghiệm lại cho thấy điều ngược lại.

The Test Boundary — Ranh giới kiểm thử (test boundary)

Yes, that's right: The tests are part of the system, and they participate in the **architecture** just like every other part of the system does. In some ways, that participation is pretty normal. In other ways, it can be pretty unique.

Đúng vậy: test là *một phần của hệ thống*, và chúng tham gia vào kiến trúc (architecture) y như mọi phần khác của hệ thống. Ở một vài khía cạnh, sự tham gia đó khá bình thường. Ở vài khía cạnh khác, nó lại rất đặc biệt.

Tests as System Components — Test như những component của hệ thống

There is a great deal of confusion about tests. Are they part of the system? Are they separate from the system? Which kinds of tests are there? Are unit tests and integration tests different things? What about acceptance tests, functional tests, Cucumber tests, TDD tests, BDD tests, **component** tests, and so on?

Có vô số nhầm lẫn quanh chuyện test. Chúng có thuộc về hệ thống không? Hay chúng tách rời khỏi hệ thống? Có những loại test nào? Unit test và integration test có phải hai thứ khác nhau? Còn acceptance test, functional test, Cucumber test, TDD test, BDD test, component test, và vân vân thì sao?

It is not the role of this book to get embroiled in that particular debate, and fortunately it isn't necessary. From an architectural point of view, all tests are the same. Whether they are the tiny little tests created by TDD, or large FitNesse, Cucumber, SpecFlow, or JBehave tests, they are architecturally equivalent.

Cuốn sách này không có nhiệm vụ lao vào cuộc tranh cãi đó, và may thay, cũng chẳng cần phải thế. Từ góc nhìn kiến trúc, mọi test đều như nhau. Dù chúng là những test bé tí xíu sinh ra từ TDD, hay là những test đồ sộ kiểu FitNesse, Cucumber, SpecFlow hay JBehave, về mặt kiến trúc chúng tương đương nhau.

Tests, by their very nature, follow **the Dependency Rule**; they are very detailed and concrete; and they always depend inward toward the code being tested. In fact, you can think of the tests as the outermost circle in the architecture. Nothing within the system depends on the tests, and the tests always depend inward on the components of the system.

Tự bản chất, test tuân theo Quy tắc Phụ thuộc (the Dependency Rule); chúng rất chi tiết và cụ thể; và chúng luôn phụ thuộc hướng vào trong, về phía đoạn mã đang được kiểm

thử. Thực tế, bạn có thể coi test là vòng tròn ngoài cùng trong kiến trúc. Không có gì bên trong hệ thống phụ thuộc vào test cả, còn test thì luôn phụ thuộc hướng vào trong, vào các component của hệ thống.

Tests are also independently deployable. In fact, most of the time they are deployed in test systems, rather than in production systems. So, even in systems where independent deployment is not otherwise necessary, the tests will still be independently deployed.

Test cũng có khả năng triển khai độc lập. Thật ra, phần lớn thời gian chúng được triển khai trong các hệ thống test chứ không phải hệ thống production. Vậy nên, ngay cả trong những hệ thống mà việc triển khai độc lập vốn không cần thiết, test vẫn cứ được triển khai độc lập.

Tests are the most isolated system component. They are not necessary for system operation. No user depends on them. Their role is to support development, not operation. And yet, they are no less a system component than any other. In fact, in many ways they represent the model that all other system components should follow.

Test là component bị cô lập nhất trong hệ thống. Chúng không cần thiết cho việc vận hành hệ thống. Không người dùng nào phụ thuộc vào chúng. Vai trò của chúng là hỗ trợ phát triển, không phải vận hành. Vậy mà chúng vẫn là component của hệ thống chẳng kém gì những component khác. Thật ra, ở nhiều khía cạnh chúng chính là hình mẫu mà mọi component khác trong hệ thống nên noi theo.

Design for Testability — Thiết kế để kiểm thử được

The extreme isolation of the tests, combined with the fact that they are not usually deployed, often causes developers to think that tests fall outside of the **design** of the system. This is a catastrophic point of view. Tests that are not well integrated into the design of the system tend to be fragile, and they make the system rigid and difficult to change.

Sự cô lập cực độ của test, cộng thêm việc chúng thường không được triển khai, khiến các lập trình viên hay nghĩ rằng test nằm ngoài thiết kế (design) của hệ thống. Đây là một quan điểm tai hại. Những test không được tích hợp tốt vào thiết kế của hệ thống thường dễ vỡ, và chúng làm hệ thống trở nên cứng nhắc, khó thay đổi.

The issue, of course, is coupling. Tests that are strongly coupled to the system must change along with the system. Even the most trivial change to a system component can cause many coupled tests to break or require changes.

Vấn đề, dĩ nhiên, là sự kết dính. Test bị kết dính chặt với hệ thống thì phải thay đổi theo hệ thống. Ngay cả một thay đổi tầm thường nhất trong một component của hệ thống cũng có thể làm hàng loạt test kết dính với nó vỡ tan hoặc buộc phải sửa.

This situation can become acute. Changes to common system components can cause hundreds, or even thousands, of tests to break. This is known as the Fragile Tests Problem .

Tình trạng này có thể trở nên nghiêm trọng. Thay đổi ở những component dùng chung có thể làm hàng trăm, thậm chí hàng nghìn test vỡ. Đây chính là cái người ta gọi là *vấn đề test dễ vỡ* (Fragile Tests Problem).

It is not hard to see how this can happen. Imagine, for example, a suite of tests that use the GUI to verify business rules. Such tests may start on the login screen and then navigate through the page **structure** until they can check particular business rules. Any change to the login page, or the navigation structure, can cause an enormous number of tests to break.

Không khó để hình dung chuyện đó xảy ra thế nào. Chẳng hạn, hãy tưởng tượng một bộ test dùng GUI để kiểm chứng các quy tắc nghiệp vụ (business rule). Những test kiểu đó có thể khởi đầu ở màn hình đăng nhập rồi đi qua cấu trúc trang cho tới khi kiểm tra được một quy tắc nghiệp vụ cụ thể. Bất kỳ thay đổi nào ở trang đăng nhập, hay ở cấu trúc điều hướng, đều có thể làm một số lượng khổng lồ test vỡ.

Fragile tests often have the perverse effect of making the system rigid. When developers realize that simple changes to the system can cause massive test failures, they may resist making those changes. For example, imagine the conversation between the development team and a marketing team that requests a simple change to the page navigation structure that will cause 1000 tests to break.

Test dễ vỡ thường gây ra một hệ quả trớ trêu: chúng làm hệ thống cứng nhắc. Khi lập trình viên nhận ra rằng những thay đổi đơn giản trong hệ thống có thể làm test hỏng hàng loạt, họ sẽ ngại thực hiện những thay đổi đó. Ví dụ, hãy tưởng tượng cuộc trao đổi giữa đội phát triển và đội marketing, khi đội marketing đề nghị một thay đổi đơn giản ở cấu trúc điều hướng trang mà thay đổi ấy sẽ làm 1000 test vỡ.

The solution is to design for testability. The first rule of software design— whether for testability or for any other reason—is always the same: Don’t depend on **volatile** things . GUIs are volatile. Test suites that operate the system through the GUI must be fragile . Therefore design the system, and the tests, so that business rules can be tested without using the GUI.

Lời giải là hãy thiết kế để kiểm thử được. Quy tắc đầu tiên của thiết kế phần mềm — dù là để kiểm thử được hay vì bất kỳ lý do nào khác — luôn luôn là một: *Đừng phụ thuộc vào những thứ dễ biến động*. GUI thì dễ biến động (volatile). Những bộ test vận hành hệ thống thông qua GUI *chắc chắn* sẽ dễ vỡ. Vì vậy, hãy thiết kế hệ thống, và cả các test, sao cho quy tắc nghiệp vụ có thể được kiểm thử mà không cần đụng tới GUI.

The Testing API — API dành cho kiểm thử

The way to accomplish this goal is to create a specific API that the tests can use to verify all the business rules. This API should have superpowers that allow the tests to avoid security constraints, bypass expensive resources (such as databases), and force the system into particular testable states. This API will be a superset of the suite of interactors and **interface** adapters that are used by the user interface.

Cách để đạt mục tiêu này là tạo ra một API riêng để test dùng nhằm kiểm chứng mọi quy tắc nghiệp vụ. API này nên có những “siêu năng lực” cho phép test né các ràng buộc bảo mật, đi vòng qua những tài nguyên đắt đỏ (như cơ sở dữ liệu), và ép hệ thống rơi vào những trạng thái cụ thể có thể kiểm thử được. API này sẽ là một tập cha của bộ interactor và interface adapter mà giao diện người dùng đang dùng.

The purpose of the **testing API** is to decouple the tests from the application. This **decoupling** encompasses more than just detaching the tests from the UI: The goal is to decouple the structure of the tests from the structure of the application.

Mục đích của API dành cho kiểm thử là tách rời (decoupling) test khỏi ứng dụng. Sự tách rời này còn đi xa hơn việc chỉ gỡ test ra khỏi UI: mục tiêu là tách rời *cấu trúc* của test khỏi *cấu trúc* của ứng dụng.

Structural Coupling — Kết dính về cấu trúc

Structural coupling is one of the strongest, and most insidious, forms of test coupling. Imagine a test suite that has a test class for every production class, and a set of test methods for every production method. Such a test suite is deeply coupled to the structure of the application.

Kết dính về cấu trúc là một trong những dạng kết dính test mạnh nhất và cũng âm thầm nguy hiểm nhất. Hãy tưởng tượng một bộ test có một test class cho mỗi class production, và một nhóm test method cho mỗi method production. Bộ test như vậy bị kết dính sâu sắc với cấu trúc của ứng dụng.

When one of those production methods or classes changes, a large number of tests must change as well. Consequently, the tests are fragile, and they make the production code rigid.

Khi một trong những method hay class production đó thay đổi, một lượng lớn test cũng phải thay đổi theo. Hệ quả là test trở nên dễ vỡ, và chúng làm mã production trở nên cứng nhắc.

The role of the testing API is to hide the structure of the application from the tests. This allows the production code to be refactored and evolved in ways that don't affect the tests. It also allows the tests to be refactored and evolved in ways that don't affect the production code.

Vai trò của API dành cho kiểm thử là che giấu cấu trúc của ứng dụng khỏi test. Nhờ đó mã production có thể được refactor và tiến hóa theo những cách không ảnh hưởng tới test. Và ngược lại, test cũng có thể được refactor và tiến hóa theo những cách không ảnh hưởng tới mã production.

This separation of evolution is necessary because as time passes, the tests tend to become increasingly more concrete and specific. In contrast, the production code tends to become increasingly more abstract and general. Strong structural coupling prevents—or at least impedes—this necessary evolution, and prevents the production code from being as general, and flexible, as it could be.

Sự tách biệt trong tiến hóa này là cần thiết, bởi theo thời gian, test có xu hướng ngày càng cụ thể và chi tiết hơn. Ngược lại, mã production lại có xu hướng ngày càng trừu tượng và tổng quát hơn. Kết dính cấu trúc chặt chẽ sẽ ngăn cản — hoặc chí ít là cản trở — quá trình tiến hóa cần thiết đó, và ngăn mã production đạt tới mức tổng quát và linh hoạt mà đáng ra nó có thể đạt được.

Security — Bảo mật

The superpowers of the testing API could be dangerous if they were deployed in production systems. If this is a concern, then the testing API, and the dangerous parts of its implementation, should be kept in a separate, independently deployable component.

Những “siêu năng lực” của API dành cho kiểm thử có thể trở nên nguy hiểm nếu chúng được triển khai lên hệ thống production. Nếu đó là điều đáng lo, thì API dành cho kiểm thử, cùng những phần cài đặt nguy hiểm của nó, nên được giữ trong một component riêng, có khả năng triển khai độc lập.

Conclusion — Kết luận

Tests are not outside the system; rather, they are parts of the system that must be well designed if they are to provide the desired benefits of **stability** and regression. Tests that are not designed as part of

the system tend to be fragile and difficult to maintain. Such tests often wind up on the maintenance room floor—discarded because they are too difficult to maintain.

Test không nằm ngoài hệ thống; ngược lại, chúng là những phần của hệ thống và phải được thiết kế tử tế nếu ta muốn có được những lợi ích mong đợi là sự ổn định và khả năng chống hồi quy. Những test không được thiết kế như một phần của hệ thống thường dễ vỡ và khó bảo trì. Kết cục, những test như thế thường bị vứt lại trên sàn phòng bảo trì — bị loại bỏ vì quá khó bảo trì.

Clean Embedded Architecture — Kiến trúc nhúng sạch (Clean Embedded Architecture)

By James Grenning

Tác giả: James Grenning

A while ago I read an article entitled “The Growing Importance of Sustaining Software for the DoD”¹ on Doug Schmidt’s blog. Doug made the following claim:

Cách đây ít lâu tôi đọc một bài viết tên là “The Growing Importance of Sustaining Software for the DoD”¹ trên blog của Doug Schmidt. Doug đưa ra nhận định sau:

“Although software does not wear out, **firmware** and hardware become obsolete, thereby requiring software modifications.”

“Tuy phần mềm không hao mòn, firmware và phần cứng thì lỗi thời, và điều đó buộc phần mềm phải sửa đổi.”

It was a clarifying moment for me. Doug mentioned two terms that I would have thought to be obvious—but maybe not. Software is this thing that can have a long useful life, but firmware will become obsolete as hardware evolves. If you have spent any time in **embedded** systems development, you know the hardware is continually evolving and being improved. At the same time, features are added to the new “software,” and it continually grows in complexity.

Với tôi đó là một khoảnh khắc sáng tỏ. Doug nhắc tới hai khái niệm mà đáng lẽ tôi phải thấy là hiển nhiên — nhưng có lẽ không hiển nhiên chút nào. Phần mềm là thứ có thể sống hữu ích rất lâu, còn firmware sẽ lỗi thời khi phần cứng tiến hoá. Nếu bạn từng làm phát triển hệ thống nhúng (embedded), bạn biết phần cứng liên tục tiến hoá và được cải tiến. Cùng lúc đó, người ta thêm tính năng vào “phần mềm” mới, và nó không ngừng phình ra về độ phức tạp.

I’d like to add to Doug’s statement:

Tôi muốn bổ sung vào câu của Doug:

Although software does not wear out, it can be destroyed from within by unman- aged dependencies on firmware and hardware.

Tuy phần mềm không hao mòn, nó có thể bị huỷ hoại từ bên trong bởi những phụ thuộc vào firmware và phần cứng mà không ai quản lý.

It is not uncommon for embedded software to be denied a potentially long life due to being infected with dependencies on hardware.

Chuyện phần mềm nhúng bị tước mất một vòng đời dài tiềm năng chỉ vì nhiễm đầy phụ thuộc vào phần cứng là chuyện không hiếm.

I like Doug's definition of firmware, but let's see which other definitions are out there. I found these alternatives:

Tôi thích định nghĩa firmware của Doug, nhưng hãy thử xem ngoài kia còn những định nghĩa nào khác. Tôi tìm được mấy phương án sau:

- “Firmware is held in non-**volatile** memory devices such as ROM, EPROM,
 - “Firmware nằm trong các thiết bị nhớ non-volatile như ROM, EPROM, or flash memory.” (<https://en.wikipedia.org/wiki/Firmware>) hoặc bộ nhớ flash.” (<https://en.wikipedia.org/wiki/Firmware>)
- “Firmware is a software program or set of instructions programmed on a
 - “Firmware là một chương trình phần mềm hoặc một tập chỉ thị được nạp vào một hardware device.” (<https://techterms.com/definition/firmware>) thiết bị phần cứng.” (<https://techterms.com/definition/firmware>)
- “Firmware is software that is embedded in a piece of hardware.” (<https://www.lifewire.com/what-is-firmware-2625881>)
 - “Firmware là phần mềm được nhúng bên trong một khối phần cứng.” (<https://www.lifewire.com/what-is-firmware-2625881>)
- Firmware is “Software (programs or data) that has been written onto read-only memory (ROM).” (<http://www.webopedia.com/TERM/F/firmware.html>)
 - Firmware là “Phần mềm (chương trình hoặc dữ liệu) đã được ghi vào bộ nhớ chỉ đọc (ROM).” (<http://www.webopedia.com/TERM/F/firmware.html>)
- 1. [https://insights.sei.cmu.edu/sei_blog/2011/08/the-growing-importance-of-sustaining-software-for-the-](https://insights.sei.cmu.edu/sei_blog/2011/08/the-growing-importance-of-sustaining-software-for-the-dod.html)
 - 1. [https://insights.sei.cmu.edu/sei_blog/2011/08/the-growing-importance-of-sustaining-software-for-the-](https://insights.sei.cmu.edu/sei_blog/2011/08/the-growing-importance-of-sustaining-software-for-the-dod.html)

dod.html

dod.html

Doug's statement makes me realize that these accepted definitions of firmware are wrong, or at least obsolete. Firmware does not mean code lives in ROM. It's not firmware because of where it is stored; rather, it is firmware because of what it depends on and how hard it is to change as hardware evolves. Hardware does evolve (pause and look at your phone for evidence), so we should **structure** our embedded code with that reality in mind.

Câu nói của Doug khiến tôi nhận ra rằng những định nghĩa firmware được chấp nhận rộng rãi kia là sai, hoặc ít nhất đã lỗi thời. Firmware không có nghĩa là mã nằm trong ROM. Nó là firmware không phải vì nó được lưu ở đâu; nó là firmware vì nó phụ thuộc vào cái gì và vì nó khó sửa tới mức nào khi phần cứng tiến hoá. Phần cứng thì có tiến hoá

(hãy dừng lại và nhìn cái điện thoại của bạn để làm bằng chứng), nên chúng ta cần cấu trúc mã nhúng của mình với thực tế đó trong đầu.

I have nothing against firmware, or firmware engineers (I've been known to write some firmware myself). But what we really need is less firmware and more software. Actually, I am disappointed that firmware engineers write so much firmware!

Tôi chẳng có gì chống lại firmware, hay các kỹ sư firmware (bản thân tôi thỉnh thoảng cũng có viết firmware đấy chứ). Nhưng thứ chúng ta thực sự cần là ít firmware hơn và nhiều phần mềm hơn. Thật ra, tôi thấy thất vọng vì các kỹ sư firmware viết quá nhiều firmware!

Non-embedded engineers also write firmware! You non-embedded developers essentially write firmware whenever you bury SQL in your code or when you spread platform dependencies throughout your code. Android app developers write firmware when they don't separate their business logic from the Android API.

Kỹ sư không làm nhúng cũng viết firmware đấy! Các bạn lập trình viên không làm nhúng, về bản chất là đang viết firmware mỗi khi các bạn chôn SQL vào trong mã của mình, hoặc khi các bạn rải phụ thuộc vào nền tảng khắp mã nguồn. Lập trình viên ứng dụng Android viết firmware khi họ không tách business logic của mình ra khỏi Android API.

I've been involved in a lot of efforts where the line between the product code (the software) and the code that interacts with the product's hardware (the firmware) is fuzzy to the point of nonexistence. For example, in the late 1990s I had the fun of helping redesign a communications subsystem that was transitioning from time-division multiplexing (TDM) to voice over IP (VOIP). VOIP is how things are done now, but TDM was considered the state of the art from the 1950s and 1960s, and was widely deployed in the 1980s and 1990s.

Tôi từng tham gia rất nhiều dự án mà ranh giới giữa mã sản phẩm (phần mềm) và mã tương tác với phần cứng của sản phẩm (firmware) mờ nhạt tới mức gần như không tồn tại. Chẳng hạn, cuối thập niên 1990 tôi có cái thú được góp tay thiết kế lại một hệ thống con truyền thông đang chuyển từ time-division multiplexing (TDM) sang voice over IP (VOIP). VOIP là cách người ta làm bây giờ, còn TDM từng được coi là đỉnh cao công nghệ của thập niên 1950 và 1960, và được triển khai rộng rãi trong thập niên 1980 và 1990.

Whenever we had a question for the systems engineer about how a call should react to a given situation, he would disappear and a little later emerge with a very detailed answer. "Where did he get that answer?" we asked. "From the current product's code," he'd answer. The tangled legacy code was the spec for the new product! The existing implementation had no separation between TDM and the business logic of making calls. The whole product was hardware/technology dependent from top to bottom and could not be untangled. The whole product had essentially become firmware.

Mỗi khi chúng tôi có câu hỏi cho kỹ sư hệ thống về việc một cuộc gọi phải phản ứng thế nào trong một tình huống nào đó, anh ta biến mất một lúc rồi lát sau quay ra với một câu trả lời cực kỳ chi tiết. "Anh lấy câu trả lời đó ở đâu vậy?" chúng tôi hỏi. "Từ mã của sản phẩm hiện tại," anh ta đáp. Cái mớ mã cũ rồi bởi chính là bản đặc tả cho sản phẩm mới! Bản cài đặt hiện có không hề tách bạch TDM với business logic của việc thực hiện cuộc gọi. Cả sản phẩm phụ thuộc vào phần cứng/công nghệ từ trên xuống dưới và không thể gỡ rối nổi. Về bản chất, cả sản phẩm đã trở thành firmware.

Consider another example: Command messages arrive to this system via serial port. Unsurprisingly, there is a message processor/dispatcher. The message

Hãy xét một ví dụ khác: Các thông điệp lệnh đi vào hệ thống này qua cổng serial. Chẳng

có gì bất ngờ, có một bộ xử lý/điều phối thông điệp. Bộ xử lý

processor knows the format of messages, is able to parse them, and can then dispatch the message to the code that can handle the **request**. None of this is surprising, except that the message processor/dispatcher resides in the same file as code that interacts with a UART 2 hardware. The message processor is polluted with UART details. The message processor could have been software with a potentially long useful life, but instead it is firmware. The message processor is denied the opportunity to become software—and that is just not right!

thông điệp biết định dạng của thông điệp, có khả năng phân tích chúng, rồi có thể điều phối thông điệp tới đoạn mã xử lý được yêu cầu đó. Chẳng có gì trong số này là bất ngờ cả, ngoại trừ việc bộ xử lý/điều phối thông điệp nằm chung một file với đoạn mã tương tác với phần cứng UART 2. Bộ xử lý thông điệp bị ô nhiễm bởi các chi tiết của UART. Bộ xử lý thông điệp lẽ ra đã có thể là phần mềm với một vòng đời hữu ích rất dài, nhưng thay vào đó nó lại là firmware. Bộ xử lý thông điệp bị tước mất cơ hội trở thành phần mềm — và như thế thì thật không phải!

I’ve known and understood the need for separating software from hardware for a long time, but Doug’s words clarified how to use the terms software and firmware in relationship to each other.

Tôi đã biết và hiểu nhu cầu tách phần mềm khỏi phần cứng từ lâu rồi, nhưng lời của Doug làm sáng tỏ cách dùng hai từ phần mềm và firmware trong quan hệ với nhau.

For engineers and programmers, the message is clear: Stop writing so much firmware and give your code a chance at a long useful life. Of course, demanding it won’t make it so. Let’s look at how we can keep embedded software **architecture** clean to give the software a fighting chance of having a long and useful life.

Với các kỹ sư và lập trình viên, thông điệp rất rõ: Hãy thôi viết quá nhiều firmware và cho mã của bạn một cơ hội sống lâu và hữu ích. Tất nhiên, đòi hỏi suông thì chẳng làm điều đó thành sự thật. Hãy cùng xem làm thế nào để giữ cho kiến trúc phần mềm nhúng luôn sạch, để phần mềm có cơ hội chiến đấu cho một vòng đời dài và hữu ích.

App-titude Test — Bài kiểm tra App-titude

Why does so much potential embedded software become firmware? It seems that most of the emphasis is on getting the embedded code to work, and not so much emphasis is placed on structuring it for a long useful life. Kent Beck describes three activities in building software (the quoted text is Kent’s words and the italics are my commentary):

Vì sao lại có nhiều phần mềm nhúng tiềm năng đến thế bị biến thành firmware? Có vẻ như phần lớn sự chú tâm dồn vào việc làm cho mã nhúng chạy được, chứ không mấy ai đặt trọng tâm vào việc cấu trúc nó cho một vòng đời dài. Kent Beck mô tả ba hoạt động trong việc xây dựng phần mềm (phần trong ngoặc kép là lời Kent, phần *in nghiêng* là bình luận của tôi):

- 1. “First make it work.” You are out of business if it doesn’t work .
 - 1. “Trước hết hãy làm cho nó chạy được.” *Nếu nó không chạy được thì bạn hết đường làm ăn.*
- 2. “Then make it right.” Refactor the code so that you and others can
 - 2. “Rồi làm cho nó đúng đắn.” *Refactor mã sao cho bạn và người khác có thể*

understand it and evolve it as needs change or are better understood .

hiểu được nó và tiến hoá nó khi nhu cầu thay đổi hoặc được hiểu rõ hơn.

- 3. “Then make it fast.” Refactor the code for “needed” performance.
 - 3. “Rồi làm cho nó nhanh.” *Refactor mã để đạt hiệu năng “cần thiết”.*

Much of the embedded systems software that I see in the wild seems to have been written with “Make it work” in mind—and perhaps also with an

Phần lớn phần mềm hệ thống nhúng mà tôi gặp ngoài đời thực có vẻ được viết với “Làm cho nó chạy được” trong đầu — và có lẽ kèm theo một

- 2. The hardware device that controls the serial port.
 - 2. Thiết bị phần cứng điều khiển cổng serial.

obsession for the “Make it fast” goal, achieved by adding micro-optimizations at every opportunity. In *The Mythical Man-Month*, Fred Brooks suggests we “plan to throw one away.” Kent and Fred are giving virtually the same advice: Learn what works, then make a better solution.

nỗi ám ảnh với mục tiêu “Làm cho nó nhanh”, đạt được bằng cách nhét các tối ưu vi mô (micro-optimization) vào mọi cơ hội có thể. Trong *The Mythical Man-Month*, Fred Brooks gợi ý rằng chúng ta nên “lên kế hoạch vứt bỏ một bản”. Kent và Fred đang đưa ra gần như cùng một lời khuyên: Hãy học xem cái gì chạy được, rồi làm ra một lời giải tốt hơn.

Embedded software is not special when it comes to these problems. Most non-embedded apps are built just to work, with little regard to making the code right for a long useful life.

Phần mềm nhúng chẳng có gì đặc biệt khi nói tới những vấn đề này. Hầu hết ứng dụng không phải nhúng cũng chỉ được xây dựng cho chạy được, ít ai bận tâm làm cho mã đúng đắn để sống lâu và hữu ích.

Getting an app to work is what I call the App-titude test for a programmer. Programmers, embedded or not, who just concern themselves with getting their app to work are doing their products and employers a disservice. There is much more to programming than just getting an app to work.

Việc làm cho một ứng dụng chạy được là thứ tôi gọi là bài kiểm tra App-titude dành cho lập trình viên. Lập trình viên, dù nhúng hay không, mà chỉ lo làm cho ứng dụng của mình chạy được thì đang làm hại chính sản phẩm và người thuê mình. Lập trình còn nhiều thứ hơn là chỉ làm cho ứng dụng chạy được.

As an example of code produced while passing the App-titude test, check out these functions located in one file of a small embedded system:

Làm ví dụ cho loại mã sinh ra khi người ta vượt qua bài kiểm tra App-titude, hãy xem những hàm sau đây nằm trong một file của một hệ thống nhúng nhỏ:

```
ISR(TIMER1_vect) { ... }
ISR(INT2_vect) { ... }
void btn_Handler(void) { ... }
float calc_RPM(void) { ... }
static char Read_RawData(void) { ... }
void Do_Average(void) { ... }
void Get_Next_Measurement(void) { ... }
void Zero_Sensor_1(void) { ... }
void Zero_Sensor_2(void) { ... }
void Dev_Control(char Activation) { ... }
char Load_FLASH_Setup(void) { ... }
```



```

void Save_FLASH_Setup(void) { ... }
void Store_DataSet(void) { ... }
float bytes2float(char bytes[4]) { ... }
void Recall_DataSet(void) { ... }
void Sensor_init(void) { ... }
void uC_Sleep(void) { ... }

```

That list of functions is in the order I found them in the source file. Now I'll separate them and group them by concern:

Danh sách hàm đó được liệt kê theo đúng thứ tự tôi tìm thấy chúng trong file nguồn. Giờ tôi sẽ tách chúng ra và nhóm lại theo mỗi quan tâm:

- Functions that have domain logic
 - Các hàm chứa logic nghiệp vụ

```

float calc_RPM(void) { ... }
void Do_Average(void) { ... }
void Get_Next_Measurement(void) { ... }
void Zero_Sensor_1(void) { ... }
void Zero_Sensor_2(void) { ... }

```

- Functions that set up the hardware platform
 - Các hàm thiết lập nền tảng phần cứng

```

ISR(TIMER1_vect) { ... }*
ISR(INT2_vect) { ... }
void uC_Sleep(void) { ... }
Functions that react to the on off button press
void btn_Handler(void) { ... }
void Dev_Control(char Activation) { ... }
A Function that can get A/D input readings from the hardware
static char Read_RawData(void) { ... }

```

- Functions that store values to the persistent storage
 - Các hàm lưu giá trị xuống bộ nhớ lưu trữ lâu dài

```

char Load_FLASH_Setup(void) { ... }
void Save_FLASH_Setup(void) { ... }
void Store_DataSet(void) { ... }
float bytes2float(char bytes[4]) { ... }
void Recall_DataSet(void) { ... }

```

- Function that does not do what its name implies
 - Hàm không làm đúng cái mà tên nó ngụ ý

```

void Sensor_init(void) { ... }

```

Looking at some of the other files in this application, I found many impediments to understanding the code. I also found a file structure that implied that the only way to test any of this code is in the embedded target. Virtually every bit of this code knows it is in a special microprocessor architecture, using “extended” C constructs 3 that tie the code to a particular tool chain and microprocessor. There

is no way for this code to have a long useful life unless the product never needs to be moved to a different hardware environment.

Nhìn sang vài file khác trong ứng dụng này, tôi thấy vô số chương ngại cho việc hiểu mã. Tôi cũng thấy một cấu trúc file ngụ ý rằng cách duy nhất để kiểm thử bất kỳ đoạn nào của mã này là chạy trên chính phần cứng đích. Gần như từng mẫu mã ở đây đều biết rằng nó đang nằm trong một kiến trúc vi xử lý đặc thù, dùng các cấu trúc C “mở rộng” 3 gắn chặt mã vào một tool chain và một vi xử lý cụ thể. Không đời nào mã này có được vòng đời dài lâu, trừ phi sản phẩm không bao giờ cần chuyển sang một môi trường phần cứng khác.

This application works: The engineer passed the App-titude test. But the application can't be said to have a clean embedded architecture.

Ứng dụng này chạy được: Vị kỹ sư đã vượt qua bài kiểm tra App-titude. Nhưng không thể nói ứng dụng này có một kiến trúc nhúng sạch.

The Target-Hardware Bottleneck — Nút thắt phần cứng đích

There are many special concerns that embedded developers have to deal with that non-embedded developers do not—for example, limited memory space, real-time constraints and deadlines, limited IO, unconventional user interfaces, and sensors and connections to the real world. Most of the time the hardware is concurrently developed with the software and firmware. As an engineer developing code for this kind of system, you may have no place to run the code. If that's not bad enough, once you get the hardware, it is likely that the hardware will have its own defects, making software development progress even slower than usual.

Có rất nhiều mối bận tâm đặc thù mà lập trình viên nhúng phải đối mặt còn lập trình viên không nhúng thì không — chẳng hạn không gian bộ nhớ hạn chế, ràng buộc và hạn chót thời gian thực, IO hạn chế, giao diện người dùng phi truyền thống, rồi cảm biến và các kết nối với thế giới thực. Phần lớn thời gian, phần cứng được phát triển song song với phần mềm và firmware. Là kỹ sư viết mã cho loại hệ thống này, có khi bạn chẳng có chỗ nào để chạy mã cả. Như thế còn chưa đủ tệ, một khi có phần cứng trong tay, nhiều khả năng chính phần cứng lại có lỗi của riêng nó, khiến tiến độ phát triển phần mềm còn chậm hơn thường lệ.

Yes, embedded is special. Embedded engineers are special. But embedded development is not so special that the principles in this book are not applicable to embedded systems.

Đúng, nhúng là đặc biệt. Kỹ sư nhúng là những người đặc biệt. Nhưng phát triển nhúng không đặc biệt tới mức các nguyên tắc trong cuốn sách này không áp dụng được cho hệ thống nhúng.

One of the special embedded problems is the **target-hardware bottleneck**. When embedded code is structured without applying **clean architecture** principles and practices, you will often face the scenario in which you can test your code only on the target. If the target is the only place where testing is possible, the target-hardware bottleneck will slow you down.

Một trong những vấn đề đặc thù của nhúng là *nút thắt phần cứng đích*. Khi mã nhúng được cấu trúc mà không áp dụng các nguyên tắc và thực hành của clean architecture, bạn sẽ thường xuyên rơi vào cảnh chỉ có thể kiểm thử mã của mình trên phần cứng đích. Nếu phần cứng đích là nơi duy nhất kiểm thử được, nút thắt phần cứng đích sẽ làm bạn chậm lại.

- 3. Some silicon providers add keywords to the C language to make accessing the registers and IO ports
 - 3. Một số nhà cung cấp chip thêm từ khoá vào ngôn ngữ C để việc truy cập các thanh ghi và cổng IO

simple from C. Unfortunately, once that is done, the code is no longer C.

trở nên đơn giản từ C. Đáng tiếc, một khi làm vậy thì mã đó không còn là C nữa.

A Clean Embedded Architecture Is a Testable Embedded Architecture — Kiến trúc nhúng sạch là kiến trúc nhúng kiểm thử được

Let's see how to apply some of the architectural principles to embedded software and firmware to help you eliminate the target-hardware bottleneck.

Hãy xem cách áp dụng một vài nguyên tắc kiến trúc vào phần mềm nhúng và firmware để giúp bạn loại bỏ nút thắt phần cứng đích.

Layers — Các tầng

Layering comes in many flavors. Let's start with three layers, as shown in Figure 29.1. At the bottom, there is the hardware. As Doug warns us, due to technology advances and Moore's law, the hardware will change. Parts become obsolete, and new parts use less power or provide better performance or are cheaper. Whatever the reason, as an embedded engineer, I don't want to have a bigger job than is necessary when the inevitable hardware change finally happens.

Phân tầng có nhiều kiểu. Hãy bắt đầu với ba tầng, như minh hoạ trong Figure 29.1. Dưới cùng là phần cứng. Như Doug cảnh báo, do tiến bộ công nghệ và định luật Moore, phần cứng sẽ thay đổi. Linh kiện trở nên lỗi thời, và linh kiện mới thì tốn ít điện hơn, cho hiệu năng tốt hơn, hoặc rẻ hơn. Lý do là gì đi nữa, với tư cách một kỹ sư nhúng, tôi không muốn phải làm nhiều việc hơn mức cần thiết khi cái thay đổi phần cứng không thể tránh khỏi kia cuối cùng cũng xảy đến.

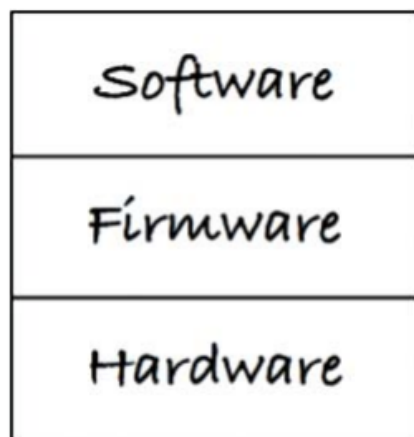


Figure 29.1 Three layers

Figure 29.1 Ba tầng

The separation between hardware and the rest of the system is a given—at least once the hardware is defined (Figure 29.2). Here is where the problems often begin when you are trying to pass the App-titude test. There is nothing that keeps hardware knowledge from polluting all the code. If you are not careful about where you put things and what one module is allowed to know about another module, the code will be very hard to change. I'm not just talking about when the hardware changes, but when the user asks for a change, or when a bug needs to be fixed.

Việc tách bạch giữa phần cứng và phần còn lại của hệ thống là điều mặc nhiên — ít nhất là một khi phần cứng đã được định nghĩa (Figure 29.2). Đây chính là chỗ rắc rối thường

bắt đầu khi bạn đang cố vượt qua bài kiểm tra App-titude. Chẳng có gì ngăn kiến thức về phần cứng lan ra làm ô nhiễm toàn bộ mã. Nếu bạn không cẩn thận về chuyện đặt thứ gì ở đâu và module nào được phép biết gì về module khác, mã sẽ rất khó sửa. Tôi không chỉ nói tới lúc phần cứng thay đổi, mà cả lúc người dùng yêu cầu một thay đổi, hay lúc cần sửa một lỗi.

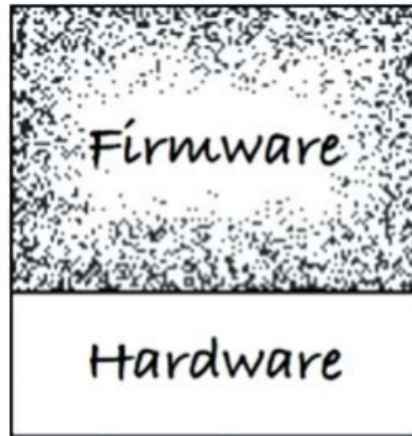


Figure 29.2 Hardware must be separated from the rest of the system

Figure 29.2 Phần cứng phải được tách khỏi phần còn lại của hệ thống

Software and firmware intermingling is an anti-pattern. Code exhibiting this anti-pattern will resist changes. In addition, changes will be dangerous, often leading to unintended consequences. Full regression tests of the whole system will be needed for minor changes. If you have not created externally instrumented tests, expect to get bored with manual tests—and then you can expect new bug reports.

Trộn lẫn phần mềm với firmware là một anti-pattern. Mã mang anti-pattern này sẽ chống lại mọi thay đổi. Hơn nữa, các thay đổi sẽ nguy hiểm, thường dẫn tới những hệ quả ngoài ý muốn. Chỉ một thay đổi nhỏ cũng đòi hỏi test hồi quy đầy đủ trên toàn hệ thống. Nếu bạn chưa tạo được các test đo đạc từ bên ngoài, hãy chuẩn bị tinh thần chán ngán với test thủ công — và rồi hãy chuẩn bị đón thêm các báo cáo lỗi mới.

TheHardwareIsaDetail — Phần cứng là một chi tiết

The line between software and firmware is typically not so well defined as the line between code and hardware, as shown in Figure 29.3.

Ranh giới (boundary) giữa phần mềm và firmware thường không rõ ràng bằng ranh giới giữa mã và phần cứng, như minh họa trong Figure 29.3.



Figure 29.3 The line between software and firmware is a bit fuzzier than the line between

Figure 29.3 Ranh giới giữa phần mềm và firmware có phần mờ nhạt hơn ranh giới giữa code and hardware
mã và phần cứng

One of your jobs as an embedded software developer is to firm up that line. The name of the **boundary** between the software and the firmware is the **hardware abstraction layer** (HAL) (Figure 29.4). This is not a new idea: It has been in PCs since the days before Windows.

Một trong những việc của bạn với tư cách lập trình viên phần mềm nhúng là làm cho cái ranh giới đó cứng cáp lên. Tên gọi của ranh giới giữa phần mềm và firmware là tầng trừu tượng phần cứng (HAL) (Figure 29.4). Đây không phải ý tưởng mới: Nó đã có trong PC từ trước thời Windows.

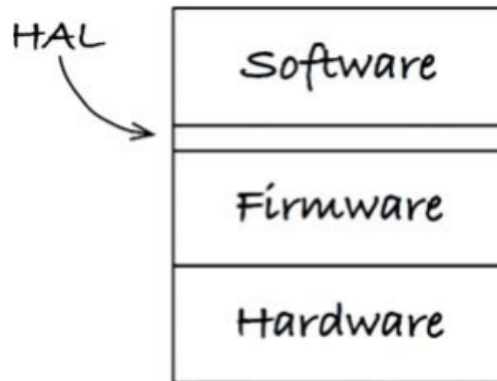


Figure 29.4 The hardware abstraction layer

Figure 29.4 Tầng trừu tượng phần cứng

The HAL exists for the software that sits on top of it, and its API should be tailored to that software's needs. As an example, the firmware can store bytes and arrays of bytes into flash memory. In contrast, the application needs to store and read name/value pairs to some persistence mechanism. The software should not be concerned that the name/value pairs are stored in flash memory, a spinning disk, the cloud, or core memory. The HAL provides a **service**, and it does not reveal to the software how it does it. The flash implementation is a **detail** that should be hidden from software.

HAL tồn tại là vì phần mềm nằm bên trên nó, và API của nó phải được may đo theo nhu cầu của phần mềm đó. Chẳng hạn, firmware có thể lưu byte và mảng byte vào bộ nhớ flash. Ngược lại, ứng dụng lại cần lưu và đọc các cặp name/value tới một cơ chế lưu trữ lâu dài nào đó. Phần mềm không nên bận tâm rằng các cặp name/value được cất trong bộ nhớ flash, trên một cái đĩa quay, trên đám mây, hay trong core memory. HAL cung cấp một dịch vụ, và nó không tiết lộ cho phần mềm biết nó làm điều đó bằng cách nào. Bản cài đặt trên flash là một chi tiết (detail) cần được giấu đi khỏi phần mềm.

As another example, an LED is tied to a GPIO bit. The firmware could provide access to the GPIO bits, where a HAL might provide `Led_TurnOn(5)`. That is a pretty **low-level** hardware abstraction layer. Let's consider raising the **level** of abstraction from a hardware perspective to the software/product perspective. What is the LED indicating? Suppose that it indicated low battery power. At some level, the firmware (or a board support package) could provide `Led_TurnOn(5)`, while the HAL provides `Indicate_LowBattery()`. You can see the HAL expressing services needed by the application. You can also see that layers may contain layers. It is more of a repeating fractal pattern than a limited set of predefined layers. The GPIO assignments are details that should be hidden from the software.

Một ví dụ khác: một đèn LED được nối vào một bit GPIO. Firmware có thể cung cấp quyền truy cập tới các bit GPIO, còn HAL thì có thể cung cấp `Led_TurnOn(5)`. Đó là một tầng trừu tượng phần cứng khá thấp. Hãy thử nâng mức trừu tượng từ góc nhìn phần cứng lên góc nhìn phần mềm/sản phẩm. Cái đèn LED đó đang báo hiệu điều gì? Giả sử nó báo pin yếu. Ở một mức nào đó, firmware (hoặc một board support package) có thể cung cấp `Led_TurnOn(5)`, trong khi HAL cung cấp `Indicate_LowBattery()`. Bạn thấy đấy, HAL diễn đạt những dịch vụ mà ứng dụng cần. Bạn cũng thấy rằng tầng có thể chứa tầng. Nó giống một mẫu fractal lặp lại hơn là một tập hữu hạn các tầng định sẵn. Việc gán chân GPIO là những chi tiết cần được giấu đi khỏi phần mềm.

Don't Reveal Hardware Details to the User of the HAL — Đừng để lộ chi tiết phần cứng cho người dùng HAL

A clean embedded architecture's software is testable off the target hardware. A successful HAL provides that seam or set of substitution points that facilitate off-target testing.

Phần mềm của một kiến trúc nhúng sạch là thứ kiểm thử được ngoài phần cứng đích. Một HAL thành công cung cấp cái đường nối (seam), hay tập điểm thay thế, giúp cho việc kiểm thử ngoài phần cứng đích trở nên khả thi.

TheProcessorIsaDetail — Bộ xử lý là một chi tiết

When your embedded application uses a specialized tool chain, it will often provide header files to help you . 4 These compilers often take liberties with the C language, adding new keywords to access their processor features. The code will look like C, but it is no longer C.

Khi ứng dụng nhúng của bạn dùng một tool chain chuyên biệt, tool chain đó thường cung cấp các header file để giúp bạn . 4 Những trình biên dịch này thường tự tiện thêm bớt với ngôn ngữ C, bổ sung từ khoá mới để truy cập các tính năng của bộ xử lý. Mã trông vẫn giống C, nhưng nó không còn là C nữa.

Sometimes vendor-supplied C compilers provide what look like global variables to give access directly to processor registers, IO ports, clock timers, IO bits, interrupt controllers, and other processor functions. It is helpful to get access to these things easily, but realize that any of your code that uses these helpful facilities is no longer C. It won't compile for another processor, or maybe even with a different compiler for the same processor.

Đôi khi trình biên dịch C do nhà cung cấp phát hành đưa ra những thứ trông như biến toàn cục để cho phép truy cập trực tiếp tới thanh ghi bộ xử lý, cổng IO, bộ đếm thời gian, các bit IO, bộ điều khiển ngắt và những chức năng khác của bộ xử lý. Truy cập được những thứ này một cách dễ dàng thì rất tiện, nhưng hãy nhận ra rằng bất kỳ đoạn mã nào của bạn dùng tới các tiện ích hữu ích đó đều không còn là C nữa. Nó sẽ không biên dịch được cho một bộ xử lý khác, thậm chí có khi cũng không biên dịch được bằng một trình biên dịch khác cho cùng bộ xử lý ấy.

I would hate to think that the silicon and tool provider is being cynical, tying your product to the compiler. Let's give the provider the benefit of a doubt by assuming that it is truly trying to help. But now it's up to you to use that help in a way that does not hurt in the future. You will have to limit which files are allowed to know about the C extensions.

Tôi không muốn nghĩ rằng nhà cung cấp chip và tool chain đang toan tính vụ lợi, cột chặt sản phẩm của bạn vào trình biên dịch của họ. Hãy tin vào thiện chí của họ, cứ giả định rằng họ thật lòng muốn giúp. Nhưng giờ thì việc dùng sự giúp đỡ đó theo cách không gây đau đớn về sau là phần của bạn. Bạn sẽ phải giới hạn xem những file nào được phép biết về các phần mở rộng của C.

Let's look at this header file designed for the ACME family of DSPs—you know, the ones used by Wile E. Coyote:

Hãy nhìn header file này, thiết kế cho dòng DSP của ACME — bạn biết đấy, đúng loại mà Wile E. Coyote hay dùng:

```
#ifndef _ACME_STD_TYPES
#define _ACME_STD_TYPES
```

- 4. This statement intentionally uses HTML.
- 4. Câu này cố ý dùng HTML.

```
#if defined(_ACME_X42)
    typedef unsigned int      Uint_32;
    typedef unsigned short    Uint_16;
    typedef unsigned char     Uint_8;
    typedef int               Int_32;
    typedef short             Int_16;
    typedef char              Int_8;
#elif defined(_ACME_A42)
    typedef unsigned long     Uint_32;
    typedef unsigned int      Uint_16;
    typedef unsigned char     Uint_8;
    typedef long              Int_32;
    typedef int               Int_16;
    typedef char              Int_8;
#else
    #error <acmetypes.h> is not supported for this environment
#endif
#endif
```

The acmetypes.h header file should not be used directly. If you do, your code gets tied to one of the ACME DSPs. You are using an ACME DSP, you say, so what is the harm? You can't compile your code unless you include this header. If you use the header and define *ACME_X42* or *ACME_A42*, your integers will be the wrong size if you try to test your code off-target. If that is not bad enough, one day you'll want to port your application to another processor, and you will have made that task much more difficult by not choosing portability and by not limiting what files know about ACME.

Không nên dùng trực tiếp header file `acmetypes.h`. Nếu bạn dùng, mã của bạn sẽ bị cột vào một trong các DSP của ACME. Bạn bảo là bạn đang dùng DSP của ACME, vậy thì hại gì? Bạn không biên dịch được mã của mình nếu không include cái header đó. Nếu bạn dùng header đó và định nghĩa `_ACME_X42` hay `_ACME_A42`, các số nguyên của bạn sẽ có kích thước sai khi bạn thử kiểm thử mã ngoài phần cứng đích. Như thế còn chưa đủ tệ, một ngày nào đó bạn sẽ muốn chuyển ứng dụng của mình sang một bộ xử lý khác, và bạn đã khiến việc ấy khó hơn rất nhiều vì đã không chọn tính khả chuyển và không giới hạn xem file nào biết về ACME.

Instead of using `acmetypes.h`, you should try to follow a more standardized path and use `stdint.h`. But what if the target compiler does not provide `stdint.h`? You can write this header file. The `stdint.h` you write for target builds uses the `acmetypes.h` for target compiles like this:

Thay vì dùng `acmetypes.h`, bạn nên cố đi theo con đường chuẩn hoá hơn và dùng `stdint.h`. Nhưng nếu trình biên dịch cho phần cứng đích không có `stdint.h` thì sao? Bạn có thể tự viết header file này. Cái `stdint.h` bạn viết cho các bản build trên phần cứng đích sẽ dùng `acmetypes.h` khi biên dịch cho đích, đại loại như sau:

```
#ifndef _STDINT_H_
#define _STDINT_H_
#include <acmetypes.h>
```

```

typedef Uint_32 uint32_t;
typedef Uint_16 uint16_t;
typedef Uint_8  uint8_t;
typedef Int_32  int32_t;
typedef Int_16  int16_t;
typedef Int_8   int8_t;
#endif

```

Having your embedded software and firmware use `stdint.h` helps keep your code clean and portable. Certainly, all of the software should be processor independent, but not all of the firmware can be. This next code snippet takes advantage of special extensions to C that gives your code access to the peripherals in the micro-controller. It's likely your product uses this micro-controller so that you can use its integrated peripherals. This function outputs a line that says "hi" to the serial output port. (This example is based on real code from the wild.)

Việc để phần mềm nhúng và firmware của bạn dùng `stdint.h` giúp giữ cho mã sạch và khả chuyển. Chắc chắn là toàn bộ phần mềm phải độc lập với bộ xử lý, nhưng không phải toàn bộ firmware đều làm được vậy. Đoạn mã tiếp theo đây tận dụng các phần mở rộng đặc biệt của C để cho mã của bạn truy cập được các ngoại vi trong vi điều khiển. Nhiều khả năng sản phẩm của bạn dùng vi điều khiển này chính là để dùng được các ngoại vi tích hợp của nó. Hàm này xuất ra một dòng chữ "hi" tới cổng serial. (Ví dụ này dựa trên mã thật ngoài đời.)

```

void say_hi()
{
    IE = 0b11000000;
    SBUF0 = (0x68);
    while(TI_0 == 0);
    TI_0 = 0;
    SBUF0 = (0x69);
    while(TI_0 == 0);
    TI_0 = 0;
    SBUF0 = (0x0a);
    while(TI_0 == 0);
    TI_0 = 0;
    SBUF0 = (0x0d);
    while(TI_0 == 0);
    TI_0 = 0;
    IE = 0b11010000;
}

```

There are lots of problems with this small function. One thing that might jump out at you is the presence of `0b11000000`. This binary notation is cool; can C do that? Unfortunately, no. A few other problems relate to this code directly using the custom C extensions:

Cái hàm nhỏ xíu này có rất nhiều vấn đề. Một thứ có thể đập ngay vào mắt bạn là sự hiện diện của `0b11000000`. Ký pháp nhị phân này trông ngẫu đấy; C làm được vậy sao? Đáng tiếc là không. Vài vấn đề khác liên quan tới việc mã này dùng trực tiếp các phần mở rộng C tùy biến:

IE : Interrupt enable bits.

IE: Các bit cho phép ngắt.

SBUF0 : Serial output buffer.

SBUF0: Bộ đệm xuất serial.

TI_0 : Serial transmit buffer empty interrupt. Reading a 1 indicates the buffer is empty.

TI_0: Ngắt báo bộ đệm truyền serial rỗng. Đọc ra 1 nghĩa là bộ đệm đang rỗng.

The uppercase variables actually access micro-controller built-in peripherals. If you want to control interrupts and output characters, you must use these peripherals. Yes, this is convenient—but it's not C.

Các biến viết hoa thực chất truy cập vào các ngoại vi tích hợp sẵn của vi điều khiển. Nếu bạn muốn điều khiển ngắt và xuất ký tự, bạn buộc phải dùng những ngoại vi này. Đúng, nó tiện — nhưng nó không phải C.

A clean embedded architecture would use these device access registers directly in very few places and confine them totally to the firmware . Anything that knows about these registers becomes firmware and is consequently bound to the silicon. Tying code to the processor will hurt you when you want to get code working before you have **stable** hardware. It will also hurt you when you move your embedded application to a new processor.

Một kiến trúc nhúng sạch sẽ chỉ dùng trực tiếp các thanh ghi truy cập thiết bị này ở rất ít chỗ, và nhốt chúng hoàn toàn trong *firmware*. Bất cứ thứ gì biết về những thanh ghi này đều trở thành firmware và do đó bị trói vào con chip. Việc cột mã vào bộ xử lý sẽ làm bạn đau khi bạn muốn mã chạy được trước lúc có phần cứng ổn định. Nó cũng làm bạn đau khi bạn chuyển ứng dụng nhúng của mình sang một bộ xử lý mới.

If you use a micro-controller like this, your firmware could isolate these low- level functions with some form of a processor abstraction layer (PAL). Firmware above the PAL could be tested off-target, making it a little less firm.

Nếu bạn dùng một vi điều khiển kiểu này, firmware của bạn có thể cô lập các hàm mức thấp ấy bằng một dạng tầng trừu tượng bộ xử lý (PAL). Firmware nằm trên PAL có thể kiểm thử được ngoài phần cứng đích, khiến nó bớt “cứng” đi một chút.

TheOperatingSystemIsaDetail — Hệ điều hành là một chi tiết

A HAL is necessary, but is it sufficient? In bare-metal embedded systems, a HAL may be all you need to keep your code from getting too addicted to the operating environment. But what about embedded systems that use a real-time operating system (RTOS) or some embedded version of Linux or Windows?

HAL là cần thiết, nhưng đã đủ chưa? Trong các hệ thống nhúng chạy trực tiếp trên phần cứng trần (bare-metal), một HAL có thể là tất cả những gì bạn cần để mã nguồn không quá nghiện môi trường vận hành. Nhưng còn những hệ thống nhúng dùng hệ điều hành thời gian thực (RTOS), hay một phiên bản Linux hoặc Windows dành cho nhúng thì sao?

To give your embedded code a good chance at a long life, you have to treat the operating system as a detail and protect against OS dependencies.

Muốn mã nhúng của bạn có cơ hội sống thọ, bạn phải coi hệ điều hành là một chi tiết (detail) và tự bảo vệ mình trước các phụ thuộc vào OS.

The software accesses the services of the operating environment through the OS. The OS is a layer separating the software from firmware (Figure 29.5). Using an OS directly can cause problems. For example, what if your RTOS supplier is bought by another company and the royalties go up, or the quality goes down? What if your needs change and your RTOS does not have the capabilities that you now require? You'll have to change lots of code. These won't just be simple syntactical changes due to the new OS's API, but will likely have to adapt semantically to the new OS's different capabilities and primitives.

Phần mềm truy cập các dịch vụ của môi trường vận hành thông qua OS. OS là một tầng ngăn cách phần mềm với firmware (Figure 29.5). Dùng OS trực tiếp có thể gây rắc rối. Chẳng hạn, sẽ ra sao nếu nhà cung cấp RTOS của bạn bị một công ty khác mua lại rồi phí bản quyền tăng vọt, hoặc chất lượng đi xuống? Sẽ ra sao nếu nhu cầu của bạn thay đổi và RTOS đó không có những khả năng mà bây giờ bạn cần? Bạn sẽ phải sửa rất nhiều mã. Và đó không chỉ là những thay đổi cú pháp đơn giản do API của OS mới, mà nhiều khả năng còn phải thích nghi về mặt ngữ nghĩa với những khả năng và primitive khác biệt của OS mới.



Figure 29.5 Adding in an operating system

Figure 29.5 Bổ sung thêm một hệ điều hành

A clean embedded architecture isolates software from the operating system, through an **operating system abstraction layer** (OSAL) (Figure 29.6). In some cases, implementing this layer might be as simple as changing the name of a function. In other cases, it might involve wrapping several functions together.

Một kiến trúc nhúng sạch sẽ cách ly phần mềm khỏi hệ điều hành, thông qua một tầng trừu tượng hệ điều hành (OSAL) (Figure 29.6). Trong một số trường hợp, hiện thực tầng này có thể đơn giản như đổi tên một hàm. Trong những trường hợp khác, nó có thể là gói chung vài hàm lại với nhau.

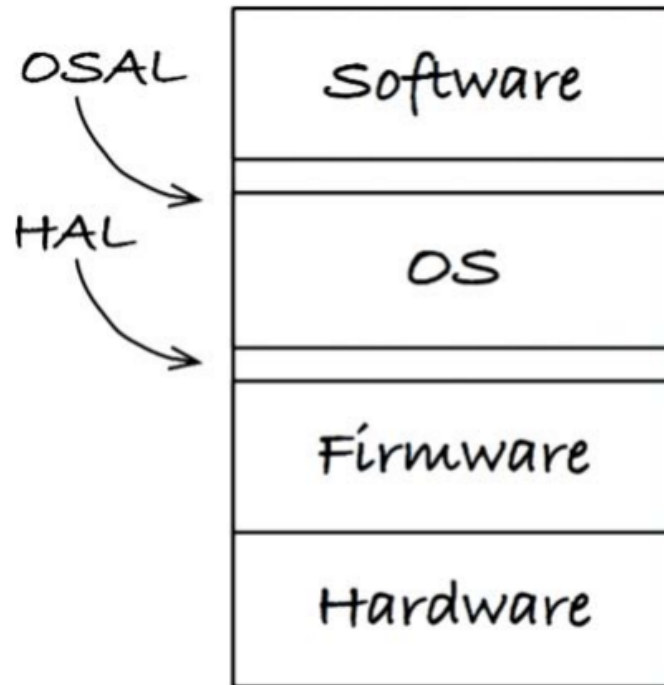


Figure 29.6 The operating system abstraction layer

Figure 29.6 Tầng trừu tượng hệ điều hành

If you have ever moved your software from one RTOS to another, you know it is painful. If your software depended on an OSAL instead of the OS directly, you would largely be writing a new OSAL that is compatible with the old OSAL. Which would you rather do: modify a bunch of complex existing code, or write new code to a defined **interface** and **behavior**? This is not a trick question. I choose the latter.

Nếu từng chuyển phần mềm của mình từ RTOS này sang RTOS khác, bạn biết việc đó đau đớn thế nào. Nếu phần mềm của bạn phụ thuộc vào một OSAL thay vì phụ thuộc trực tiếp vào OS, thì phần lớn công việc chỉ là viết một OSAL mới tương thích với OSAL cũ. Bạn thích làm việc nào hơn: sửa một đống mã nguồn phức tạp đang có, hay viết mã mới theo một interface và hành vi đã được định nghĩa rõ? Đây không phải câu hỏi gài bẫy. Tôi chọn về sau.

You might start worrying about code bloat about now. Really, though, the layer becomes the place where much of the duplication around using an OS is isolated. This duplication does not have to impose a big overhead. If you define an OSAL, you can also encourage your applications to have a common structure. You might provide message passing mechanisms, rather than having every thread handcraft its concurrency model.

Đến đây có lẽ bạn bắt đầu lo mã sẽ phình ra. Nhưng thật ra, tầng này chính là nơi phần lớn sự trùng lặp quanh việc dùng OS được cô lập lại. Sự trùng lặp đó không nhất thiết phải kéo theo chi phí lớn. Nếu định nghĩa một OSAL, bạn còn có thể khuyến khích các ứng dụng của mình dùng chung một cấu trúc. Bạn có thể cung cấp sẵn cơ chế truyền thông điệp, thay vì để mỗi thread tự chế ra mô hình đồng thời của riêng nó.

The OSAL can help provide test points so that the valuable application code in the software layer can be tested off-target and off-OS. A clean embedded architecture's software is testable off the target

operating system. A successful OSAL provides that seam or set of substitution points that facilitate off-target testing.

OSAL còn giúp tạo ra các điểm kiểm thử, nhờ đó phần mã ứng dụng giá trị nằm ở tầng phần mềm có thể được kiểm thử bên ngoài phần cứng đích và bên ngoài OS. Phần mềm trong một kiến trúc nhúng sạch sẽ kiểm thử được mà không cần hệ điều hành đích. Một OSAL thành công sẽ cung cấp đường nối (seam) hay tập điểm thay thế giúp việc kiểm thử ngoài phần cứng đích trở nên dễ dàng.

Programming to Interfaces and Substitutability — Lập trình theo interface và khả năng thay thế

In addition to adding a HAL and potentially an OSAL inside each of the major layers (software, OS, firmware, and hardware), you can—and should— apply the principles described throughout this book. These principles encourage separation of concerns, programming to interfaces, and **substitutability**.

Ngoài việc thêm một HAL và có thể cả một OSAL vào bên trong mỗi tầng chính (phần mềm, OS, firmware và phần cứng), bạn có thể — và nên — áp dụng những nguyên tắc được trình bày xuyên suốt cuốn sách này. Chúng khuyến khích tách bạch các mối quan tâm, lập trình theo interface, và khả năng thay thế (substitutability).

The idea of a layered architecture is built on the idea of programming to interfaces. When one module interacts with another through an interface, you can substitute one service provider for another. Many readers will have written their own small version of `printf` for deployment in the target. As long as the interface to your `printf` is the same as the standard version of `printf`, you can override the service one for the other.

Ý tưởng về kiến trúc phân tầng được xây trên ý tưởng lập trình theo interface. Khi một module tương tác với module khác qua một interface, bạn có thể thay nhà cung cấp dịch vụ này bằng nhà cung cấp dịch vụ khác. Nhiều bạn đọc hẳn đã từng tự viết một phiên bản `printf` nhỏ gọn của riêng mình để triển khai lên thiết bị đích. Chỉ cần interface của `printf` do bạn viết giống hệt phiên bản `printf` chuẩn, bạn có thể ghi đè cái này bằng cái kia.

One basic rule of thumb is to use header files as interface definitions. When you do so, however, you have to be careful about what goes in the header file. Limit header file contents to function declarations as well as the constants and struct names that are needed by the function.

Một quy tắc cơ bản là dùng header file làm nơi định nghĩa interface. Nhưng khi làm vậy, bạn phải cẩn thận với những gì được đưa vào header file. Hãy giới hạn nội dung header file ở phần khai báo hàm, cùng các hằng số và tên struct mà hàm đó cần đến.

Don't clutter the interface header files with data structures, constants, and typedefs that are needed by only the implementation. It's not just a matter of clutter: That clutter will lead to unwanted dependencies. Limit the visibility of the implementation details. Expect the implementation details to change. The fewer places where code knows the details, the fewer places where code will have to be tracked down and modified.

Đừng nhồi vào các header file interface những cấu trúc dữ liệu, hằng số và typedef mà chỉ phần hiện thực mới cần. Đây không chỉ là chuyện bừa bộn: chính sự bừa bộn ấy sẽ dẫn tới những phụ thuộc không mong muốn. Hãy giới hạn phạm vi nhìn thấy của các chi tiết hiện thực. Hãy chờ đợi rằng chi tiết hiện thực sẽ thay đổi. Càng ít chỗ trong mã biết về các chi tiết ấy, thì càng ít chỗ phải lần ra và sửa lại.

A clean embedded architecture is testable within the layers because modules interact through interfaces. Each interface provides that seam or substitution point that facilitates off-target testing.

Một kiến trúc nhúng sạch có thể kiểm thử được ngay bên trong từng tầng, bởi các module tương tác với nhau qua interface. Mỗi interface đều cung cấp một đường nối hay điểm thay thế, giúp việc kiểm thử ngoài phần cứng đích trở nên dễ dàng.

DRY Conditional Compilation Directives — Chỉ thị biên dịch có điều kiện theo tinh thần DRY

One use of substitutability that is often overlooked relates to how embedded C and C++ programs handle different targets or operating systems. There is a tendency to use conditional compilation to turn on and off segments of code. I recall one especially problematic case where the statement `#ifdef BOARD_V2` was mentioned several thousand times in a telecom application.

Có một cách dùng khả năng thay thế thường bị bỏ qua, liên quan đến chuyện các chương trình C và C++ nhúng xử lý nhiều loại thiết bị đích hoặc nhiều hệ điều hành khác nhau ra sao. Người ta có xu hướng dùng biên dịch có điều kiện để bật tắt từng đoạn mã. Tôi nhớ một trường hợp đặc biệt tai hại: câu lệnh `#ifdef BOARD_V2` xuất hiện tới vài nghìn lần trong một ứng dụng viễn thông.

This repetition of code violates the Don't Repeat Yourself (DRY) principle. 5 If I see `#ifdef BOARD_V2` once, it's not really a problem. Six thousand times is an extreme problem. Conditional compilation identifying the target-hardware's type is often repeated in embedded systems. But what else can we do?

Kiểu lặp lại mã này vi phạm nguyên tắc Don't Repeat Yourself (DRY). 5 Nếu tôi thấy `#ifdef BOARD_V2` một lần, chuyện đó chẳng có gì đáng nói. Sáu nghìn lần thì là một vấn đề cực kỳ nghiêm trọng. Việc biên dịch có điều kiện để phân biệt loại phần cứng đích thường bị lặp đi lặp lại trong các hệ thống nhúng. Nhưng ta còn có thể làm gì khác?

What if there is a hardware abstraction layer? The hardware type would become a detail hidden under the HAL. If the HAL provides a set of interfaces, instead of using conditional compilation, we could use the **linker** or some form of runtime binding to connect the software to the hardware.

Nếu có một tầng trừu tượng phần cứng thì sao? Loại phần cứng sẽ trở thành một chi tiết được giấu bên dưới HAL. Nếu HAL cung cấp một tập interface, thì thay vì dùng biên dịch có điều kiện, ta có thể dùng linker hoặc một dạng ràng buộc lúc chạy nào đó để nối phần mềm với phần cứng.

- 5. Hunt and Thomas, The Pragmatic Programmer .
- 5. Hunt và Thomas, The Pragmatic Programmer .

Conclusion — Kết luận

People who are developing embedded software have a lot to learn from experiences outside of embedded software. If you are an embedded developer who has picked up this book, you will find a wealth of software development wisdom in the words and ideas.

Những người đang phát triển phần mềm nhúng có rất nhiều điều để học từ kinh nghiệm bên ngoài thế giới nhúng. Nếu bạn là một lập trình viên nhúng đã cầm cuốn sách này lên, bạn sẽ tìm thấy trong những con chữ và ý tưởng ở đây cả một kho minh triết về phát triển phần mềm.

Letting all code become firmware is not good for your product's long-term health. Being able to test only in the target hardware is not good for your product's long-term health. A clean embedded architecture is good for your product's long-term health.

Để mặc cho toàn bộ mã trở thành firmware là điều không tốt cho sức khỏe lâu dài của sản phẩm. Chỉ kiểm thử được trên chính phần cứng đích cũng không tốt cho sức khỏe lâu dài của sản phẩm. Một kiến trúc nhúng sạch thì tốt cho sức khỏe lâu dài của sản phẩm.

Details — Chi tiết

The Database Is a Detail — Database là một chi tiết

From an architectural point of view, the database is a non-**entity**—it is a **detail** that does not rise to the **level** of an architectural element. Its relationship to the **architecture** of a software system is rather like the relationship of a doorknob to the architecture of your home.

Dưới góc nhìn kiến trúc (architecture), database chẳng là gì cả — nó là một chi tiết (detail) không đủ tầm để được xem là một phần tử kiến trúc. Quan hệ giữa nó với kiến trúc của một hệ thống phần mềm cũng na ná quan hệ giữa cái nắm cửa với kiến trúc ngôi nhà bạn.

I realize that these are fighting words. Believe me, I've had the fight. So let me be clear: I am not talking about the data model. The **structure** you give to the data within your application is highly significant to the architecture of your system. But the database is not the data model. The database is piece of software. The database is a utility that provides access to the data. From the architecture's point of view, that utility is irrelevant because it's a **low-level** detail—a mechanism. And a good architect does not allow low-level mechanisms to pollute the system architecture.

Tôi biết đây là những lời gây hấn. Tin tôi đi, tôi đã cãi nhau vì chuyện này rồi. Nên hãy để tôi nói cho rõ: tôi không nói về mô hình dữ liệu. Cấu trúc (structure) mà bạn áp cho dữ liệu bên trong ứng dụng là cực kỳ quan trọng với kiến trúc hệ thống. Nhưng database không phải là mô hình dữ liệu. Database là một phần mềm. Database là một tiện ích cung cấp đường truy cập đến dữ liệu. Dưới góc nhìn của kiến trúc, cái tiện ích đó không liên quan, vì nó là một chi tiết mức thấp — một cơ chế. Và một kiến trúc sư giỏi không để những cơ chế mức thấp làm ô nhiễm kiến trúc hệ thống.

Relational Databases — Database quan hệ

Edgar Codd defined the principles of relational databases in 1970. By the mid-1980s, the relational model had grown to become the dominant form of data storage. There was a good reason for this popularity: The relational model is elegant, disciplined, and robust. It is an excellent data storage and access technology.

Edgar Codd định nghĩa các nguyên tắc của database quan hệ vào năm 1970. Đến giữa thập niên 1980, mô hình quan hệ đã lớn mạnh thành dạng lưu trữ dữ liệu thống trị. Sự phổ biến ấy có lý do chính đáng: mô hình quan hệ thanh lịch, kỷ luật và vững chãi. Nó là một công nghệ lưu trữ và truy cập dữ liệu tuyệt vời.

But no matter how brilliant, useful, and mathematically sound a technology it is, it is still just a technology. And that means it's a detail.

Nhưng dù có xuất sắc, hữu dụng và chặt chẽ về mặt toán học đến đâu, nó vẫn chỉ là một công nghệ. Và điều đó có nghĩa nó là một chi tiết.

While relational tables may be convenient for certain forms of data access, there is nothing architecturally significant about arranging data into rows within tables. The use cases of your application should neither know nor care about such matters. Indeed, knowledge of the tabular structure of the data should be restricted to the lowest-level utility functions in the outer circles of the architecture.

Bảng quan hệ có thể tiện cho một số dạng truy cập dữ liệu, nhưng việc sắp dữ liệu thành các dòng trong bảng chẳng có ý nghĩa gì về mặt kiến trúc. Các use case của ứng dụng không nên biết mà cũng chẳng nên bận tâm tới những chuyện đó. Thật ra, hiểu biết về cấu trúc bảng của dữ liệu nên bị giới hạn trong những hàm tiện ích ở mức thấp nhất, nằm tại các vòng tròn ngoài cùng của kiến trúc.

Many data access frameworks allow database rows and tables to be passed around the system as objects. Allowing this is an architectural error. It couples the use cases, business rules, and in some cases even the UI to the relational structure of the data.

Nhiều framework truy cập dữ liệu cho phép truyền các dòng và bảng của database đi khắp hệ thống dưới dạng object. Cho phép điều đó là một sai lầm kiến trúc. Nó ràng buộc các use case, các quy tắc nghiệp vụ (business rule), và trong vài trường hợp là cả UI, vào cấu trúc quan hệ của dữ liệu.

Why Are Database Systems So Prevalent? — Vì sao các hệ database lại phổ biến đến thế?

Why are software systems and software enterprises dominated by database systems? What accounts for the preeminence of Oracle, MySQL, and SQL Server? In a word: disks.

Vì sao các hệ thống phần mềm và các doanh nghiệp phần mềm lại bị các hệ database thống trị? Điều gì tạo nên vị thế áp đảo của Oracle, MySQL và SQL Server? Chỉ một từ: đĩa.

The rotating magnetic disk was the mainstay of data storage for five decades. Several generations of programmers have known no other form of data storage. Disk technology has grown from huge stacks of massive platters 48 inches in diameter that weighed thousands of pounds and held 20 megabytes, to single thin circles, 3 inches in diameter, that weigh just a few grams and hold a terabyte or more. It's been a wild ride. And throughout that ride programmers have been plagued by one fatal trait of disk technology: Disks are slow .

Đĩa từ quay tròn đã là trụ cột của việc lưu trữ dữ liệu suốt năm thập kỷ. Vài thế hệ lập trình viên chưa từng biết đến một dạng lưu trữ nào khác. Công nghệ đĩa đã đi từ những chồng đĩa khổng lồ đường kính 48 inch, nặng hàng nghìn pound và chứa được 20 megabyte, tới những vòng tròn mỏng đơn lẻ đường kính 3 inch, nặng vài gram và chứa cả terabyte hoặc hơn. Một hành trình điên rồ. Và suốt hành trình ấy, lập trình viên luôn bị ám bởi một khuyết tật chí mạng của công nghệ đĩa: đĩa thì chậm.

On a disk, data is stored within circular tracks. Those tracks are divided into sectors that hold a convenient number of bytes, often 4K. Each platter may have hundreds of tracks, and there can be a dozen or so platters. If you want to read a particular byte off the disk, you have to move the head to the proper track, wait for the disk to rotate to the proper sector, read all 4K of that sector into RAM, and then index into that RAM buffer to get the byte you want. And all that takes time—milliseconds of times.

Trên đĩa, dữ liệu được lưu trong các rãnh tròn. Những rãnh đó được chia thành các sector chứa một lượng byte thuận tiện, thường là 4K. Mỗi phiến đĩa có thể có hàng trăm rãnh, và có thể có tới cả chục phiến. Nếu bạn muốn đọc một byte cụ thể trên đĩa, bạn phải đưa đầu đọc tới đúng rãnh, chờ đĩa quay tới đúng sector, đọc trọn 4K của sector đó vào RAM, rồi đánh chỉ số vào bộ đệm RAM ấy để lấy ra byte mình cần. Và tất cả những việc đó tốn thời gian — hàng mili giây.

Milliseconds might not seem like a lot, but a millisecond is a million times longer than the cycle time of most processors. If that data was not on a disk, it could be accessed in nanoseconds, instead of milliseconds.

Vài mili giây nghe có vẻ chẳng đáng bao nhiêu, nhưng một mili giây dài gấp một triệu lần chu kỳ của hầu hết bộ vi xử lý. Nếu dữ liệu ấy không nằm trên đĩa, ta có thể truy cập nó trong vài nano giây thay vì vài mili giây.

To mitigate the time delay imposed by disks, you need indexes, caches, and optimized query schemes; and you need some kind of regular means of representing the data so that these indexes, caches, and query schemes know what they are working with. In short, you need a data access and management system. Over the years these systems have split into two distinct kinds: file systems and relational database management systems (RDBMS).

Để giảm bớt độ trễ mà đĩa áp đặt, bạn cần index, cache và các lược đồ truy vấn được tối ưu; và bạn cần một cách biểu diễn dữ liệu có quy củ để những index, cache và lược đồ truy vấn đó biết chúng đang làm việc với cái gì. Nói ngắn gọn, bạn cần một hệ thống truy cập và quản lý dữ liệu. Qua nhiều năm, những hệ thống này đã tách thành hai loại rõ rệt: hệ thống file và hệ quản trị database quan hệ (RDBMS).

File systems are document based. They provide a natural and convenient way to store whole documents. They work well when you need to save and retrieve

Hệ thống file dựa trên tài liệu. Chúng cho ta một cách tự nhiên và tiện lợi để lưu trọn vẹn các tài liệu. Chúng hoạt động tốt khi bạn cần lưu lại và lấy ra

a set of documents by name, but they don't offer a lot of help when you're searching the content of those documents. It's easy to find a file named login.c, but it's hard, and slow, to find every .c file that has a variable named x in it.

một tập tài liệu theo tên, nhưng chúng không giúp được nhiều khi bạn muốn tìm kiếm trong nội dung của những tài liệu đó. Tìm một file tên login.c thì dễ, nhưng tìm mọi file .c có chứa biến tên x thì khó và chậm.

Database systems are content based. They provide a natural and convenient way to find records based on their content. They are very good at associating multiple records based on some bit of content that they all share. Unfortunately, they are rather poor at storing and retrieving opaque documents.

Hệ database dựa trên nội dung. Chúng cho ta một cách tự nhiên và tiện lợi để tìm các bản ghi dựa trên nội dung của chúng. Chúng rất giỏi trong việc liên kết nhiều bản ghi dựa trên một mẫu nội dung mà tất cả cùng chia sẻ. Đáng tiếc, chúng khá tệ trong việc lưu và lấy ra những tài liệu “đục” (không nhìn thấu được nội dung).

Both of these systems organize the data on disk so that it can be stored and retrieved in as efficient a way as possible, given their particular access needs. Each has their own scheme for indexing and arranging the data. In addition, each eventually brings the relevant data into RAM, where it can be quickly manipulated.

Cả hai loại hệ thống này đều tổ chức dữ liệu trên đĩa sao cho việc lưu và lấy ra hiệu quả nhất có thể, tương ứng với nhu cầu truy cập riêng của chúng. Mỗi loại có lược đồ riêng để đánh chỉ mục và sắp xếp dữ liệu. Ngoài ra, rốt cuộc mỗi loại đều đưa dữ liệu liên quan vào RAM, nơi có thể thao tác thật nhanh.

What If There Were No Disk? — Nếu không còn đĩa thì sao?

As prevalent as disks once were, they are now a dying breed. Soon they will have gone the way of tape drives, floppy drives, and CDs. They are being replaced by RAM.

Đĩa từng phổ biến bao nhiêu thì giờ đây chúng cũng đang là một giống loài sắp tuyệt chủng bấy nhiêu. Chẳng bao lâu nữa chúng sẽ đi theo con đường của ổ băng từ, ổ đĩa mềm và CD. Chúng đang bị RAM thay thế.

Ask yourself this question: When all the disks are gone, and all your data is stored in RAM, how will you organize that data? Will you organize it into tables and access it with SQL? Will you organize it into files and access it through a directory?

Hãy tự hỏi mình câu này: khi mọi cái đĩa đã biến mất và toàn bộ dữ liệu của bạn nằm trong RAM, bạn sẽ tổ chức dữ liệu đó ra sao? Bạn sẽ sắp nó thành bảng rồi truy cập bằng SQL chẳng? Bạn sẽ sắp nó thành file rồi truy cập qua thư mục chẳng?

Of course not. You'll organize it into linked lists, trees, hash tables, stacks, queues, or any of the other myriad data structures, and you'll access it using pointers or references—because that's what programmers do .

Tất nhiên là không. Bạn sẽ tổ chức nó thành linked list, cây, hash table, stack, queue, hay bất kỳ cấu trúc dữ liệu nào trong vô vàn lựa chọn khác, và bạn sẽ truy cập nó bằng pointer hoặc reference — vì đó là việc lập trình viên vẫn làm.

In fact, if you think carefully about this issue, you'll realize that this is what you already do. Even though the data is kept in a database or a file system, you read it into RAM and then you reorganize it, for your own convenience,

Thật ra, nếu ngẫm kỹ chuyện này, bạn sẽ nhận ra đó chính là điều bạn vẫn đang làm. Dù dữ liệu được giữ trong database hay trong hệ thống file, bạn vẫn đọc nó vào RAM rồi tổ chức lại cho tiện với mình,

into lists, sets, stacks, queues, trees, or whatever data structure meets your fancy. It is very unlikely that you leave the data in the form of files or tables.

thành list, set, stack, queue, cây, hay bất cứ cấu trúc dữ liệu nào bạn thấy hợp ý. Rất hiếm khi bạn để dữ liệu nguyên dạng file hay bảng.

Details — Chi tiết

This reality is why I say that the database is a detail. It's just a mechanism we use to move the data back and forth between the surface of the disk and RAM. The database is really nothing more than a big bucket of bits where we store our data on a long-term basis. But we seldom use the data in that form.

Chính thực tế đó khiến tôi nói rằng database là một chi tiết. Nó chỉ là một cơ chế ta dùng để chuyển dữ liệu qua lại giữa bề mặt đĩa và RAM. Database thực chất chẳng qua là một cái thùng bit to dùng, nơi ta cất dữ liệu lâu dài. Nhưng ta hiếm khi dùng dữ liệu ở dạng đó.

Thus, from an architectural viewpoint, we should not care about the form that the data takes while it is on the surface of a rotating magnetic disk. Indeed, we should not acknowledge that the disk exists at all.

Vì vậy, dưới góc nhìn kiến trúc, ta không nên bận tâm dữ liệu mang hình hài gì khi nó nằm trên bề mặt một cái đĩa từ đang quay. Thật ra, ta thậm chí không nên thừa nhận là cái đĩa đó tồn tại.

But What about Performance? — Nhưng còn hiệu năng thì sao?

Isn't performance an architectural concern? Of course it is—but when it comes to data storage, it's a concern that can be entirely encapsulated and separated from the business rules. Yes, we need to get the data in and out of the data store quickly, but that's a low-level concern. We can address that concern with low-level data access mechanisms. It has nothing whatsoever to do with the overall architecture of our systems.

Chẳng phải hiệu năng là một mối quan tâm kiến trúc sao? Tất nhiên là có — nhưng khi nói tới lưu trữ dữ liệu, đó là mối quan tâm có thể đóng gói (encapsulation) trọn vẹn và tách khỏi các quy tắc nghiệp vụ. Đúng, ta cần đưa dữ liệu vào và lấy dữ liệu ra khỏi kho lưu trữ thật nhanh, nhưng đó là mối quan tâm mức thấp. Ta có thể xử lý nó bằng các cơ chế truy cập dữ liệu mức thấp. Nó chẳng liên quan gì tới kiến trúc tổng thể của hệ thống.

Anecdote — Giai thoại

In the late 1980s, I led a team of software engineers at a startup company that was trying to build and market a network management system that measured the communications integrity of T1 telecommunication lines. The system retrieved data from the devices at the endpoints of those lines, and then ran a series of predictive algorithms to detect and report problems.

Cuối thập niên 1980, tôi dẫn dắt một nhóm kỹ sư phần mềm tại một công ty khởi nghiệp đang cố xây dựng và bán ra thị trường một hệ thống quản trị mạng, dùng để đo tính toàn vẹn truyền thông của các đường T1. Hệ thống lấy dữ liệu từ những thiết bị ở hai đầu các đường truyền đó, rồi chạy một loạt thuật toán dự báo để phát hiện và báo cáo sự cố.

We were using UNIX platforms, and we stored our data in simple random access files. We had no need of a relational database because our data had few content-based relationships. It was better kept in trees and linked lists in

Chúng tôi dùng nền tảng UNIX và lưu dữ liệu trong các file truy cập ngẫu nhiên đơn giản. Chúng tôi không cần database quan hệ vì dữ liệu của chúng tôi có rất ít quan hệ dựa trên nội dung. Nó được giữ tốt hơn dưới dạng cây và linked list bên trong

those random access files. In short, we kept the data in a form that was most convenient to load into RAM where it could be manipulated.

chính những file truy cập ngẫu nhiên ấy. Nói ngắn gọn, chúng tôi giữ dữ liệu ở dạng tiện nhất cho việc nạp vào RAM, nơi có thể thao tác với nó.

We hired a marketing manager for this startup—a nice and knowledgeable guy. But he immediately told me that we had to have a relational database in the system. It wasn't an option and it wasn't an engineering issue—it was a marketing issue.

Công ty khởi nghiệp này tuyển một giám đốc marketing — một người tử tế và am hiểu. Nhưng ngay lập tức anh ta bảo tôi rằng hệ thống buộc phải có một database quan hệ. Đó không phải là một lựa chọn, và cũng không phải chuyện kỹ thuật — đó là chuyện marketing.

This made no sense to me. Why in the world would I want to rearrange my linked lists and trees into a bunch of rows and tables accessed through SQL? Why would I introduce all the overhead and expense of a massive RDBMS when a simple random access file system was more than sufficient? So I fought him, tooth and nail.

Với tôi thì điều đó vô lý. Đòi nào tôi lại muốn xáo tung mấy cái linked list và cây của mình thành một mớ dòng và bảng truy cập qua SQL? Đòi nào tôi lại rước về toàn bộ chi phí và phần phụ trội của một RDBMS đồ sộ, trong khi một hệ thống file truy cập ngẫu nhiên đơn giản đã quá đủ dùng? Thế là tôi đấu với anh ta, quyết liệt đến cùng.

We had a hardware engineer at this company who took up the RDBMS chant. He became convinced that our software system needed an RDBMS for technical reasons. He held meetings behind my back with the executives of the company, drawing stick figures on the whiteboard of a house balancing on a pole, and he would ask the executives, “Would you build a house on a pole?” His implied message was that an RDBMS that keeps its tables in random access files was somehow more reliable than the random access files that we were using.

Công ty có một kỹ sư phần cứng cũng hô theo bài ca RDBMS. Anh ta đinh ninh rằng hệ thống phần mềm của chúng tôi cần một RDBMS vì lý do kỹ thuật. Anh ta họp sau lưng tôi với các sếp trong công ty, vẽ lên bảng trắng hình que một ngôi nhà thăng bằng trên một cái cột, rồi hỏi các sếp: “Các anh có xây nhà trên một cái cột không?” Thông điệp ngầm của anh ta là: một RDBMS lưu bảng của nó trong các file truy cập ngẫu nhiên thì bằng cách nào đó lại đáng tin cậy hơn chính những file truy cập ngẫu nhiên mà chúng tôi đang dùng.

I fought him. I fought the marketing guy. I stuck to my engineering principles in the face of incredible ignorance. I fought, and fought, and fought.

Tôi đấu với anh ta. Tôi đấu với gã marketing. Tôi bám chặt các nguyên tắc kỹ thuật của mình trước một sự thiếu hiểu biết khó tin. Tôi đấu, đấu, và đấu.

In the end, the hardware developer was promoted over my head to become the software manager. In the end, they put a RDBMS into that poor system. And, in the end, they were absolutely right and I was wrong.

Rốt cuộc, anh kỹ sư phần cứng được thăng chức vượt qua đầu tôi để trở thành quản lý phần mềm. Rốt cuộc, họ nhét một RDBMS vào cái hệ thống tội nghiệp đó. Và rốt cuộc, họ hoàn toàn đúng còn tôi thì sai.

Not for engineering reasons, mind you: I was right about that. I was right to fight against putting an RDBMS into the architectural core of the system. The reason I was wrong was because our customers expected us to have a relational database. They didn’t know what they would do with it. They didn’t have any realistic way of using the relational data in our system. But it didn’t matter: Our customers fully expected an RDBMS. It had become a

Xin nhấn mạnh: không phải sai vì lý do kỹ thuật — về mặt đó thì tôi đúng. Tôi đã đúng khi chống lại việc đưa RDBMS vào lõi kiến trúc của hệ thống. Lý do tôi sai là vì khách hàng của chúng tôi mong đợi chúng tôi phải có một database quan hệ. Họ chẳng biết sẽ làm gì với nó. Họ không có cách nào thực tế để dùng dữ liệu quan hệ trong hệ thống của chúng tôi. Nhưng chuyện đó không quan trọng: khách hàng của chúng tôi hoàn toàn mong đợi một RDBMS. Nó đã trở thành một

check box item that all the software purchasers had on their list. There was no engineering rationale—rationality had nothing to do with it. It was an irrational, external, and entirely baseless need, but it was no less real.

ô đánh dấu (check box) mà mọi người mua phần mềm đều có trong danh sách của họ. Chẳng có lý lẽ kỹ thuật nào cả — lý trí chẳng liên quan gì tới chuyện này. Đó là một nhu cầu phi lý, đến từ bên ngoài và hoàn toàn không có căn cứ, nhưng nó vẫn có thật.

Where did that need come from? It originated from the highly effective marketing campaigns employed by the database vendors at the time. They had managed to convince **high-level** executives that their corporate “data assets” needed protection, and that the database systems they offered were the ideal means of providing that protection.

Nhu cầu ấy từ đâu ra? Nó bắt nguồn từ những chiến dịch marketing cực kỳ hiệu quả mà các nhà cung cấp database triển khai thời đó. Họ đã thuyết phục được các sếp cấp cao rằng “tài sản dữ liệu” của doanh nghiệp cần được bảo vệ, và rằng những hệ database họ chào bán chính là phương tiện lý tưởng để bảo vệ nó.

We see the same kind of marketing campaigns today. The word “enterprise” and the notion of “**Service**-Oriented Architecture” have much more to do with marketing than with reality.

Ngày nay ta vẫn thấy đúng kiểu chiến dịch marketing như vậy. Chữ “enterprise” và khái niệm “Service-Oriented Architecture” liên quan tới marketing nhiều hơn là tới thực tế.

What should I have done in that long-ago scenario? I should have bolted an RDBMS on the side of the system and provided some narrow and safe data access channel to it, while maintaining the random access files in the core of the system. What did I do? I quit and became a consultant.

Đáng lẽ tôi nên làm gì trong tình huống xa xưa ấy? Đáng lẽ tôi nên bắt vít một RDBMS vào bên hông hệ thống, cấp cho nó một kênh truy cập dữ liệu hẹp và an toàn, đồng thời vẫn giữ các file truy cập ngẫu nhiên ở lõi hệ thống. Còn tôi đã làm gì? Tôi nghỉ việc và trở thành tư vấn viên.

Conclusion — Kết luận

The organizational structure of data, the data model, is architecturally significant. The technologies and systems that move data on and off a rotating magnetic surface are not. Relational database systems that force the data to be organized into tables and accessed with SQL have much more to do with the latter than with the former. The data is significant. The database is a detail.

Cấu trúc tổ chức của dữ liệu, tức mô hình dữ liệu, là thứ có ý nghĩa về mặt kiến trúc. Còn các công nghệ và hệ thống chuyển dữ liệu vào ra một bề mặt từ đang quay thì không. Những hệ database quan hệ ép dữ liệu phải được tổ chức thành bảng và truy cập bằng SQL thuộc về vế sau nhiều hơn là vế trước. Dữ liệu thì quan trọng. Database chỉ là một chi tiết.

The Web Is a Detail — Web là một chi tiết

Were you a developer in the 1990s? Do you remember how the web changed everything? Do you remember how we looked at our old client–server architectures with disdain in the face of the shiny new technology of The Web?

Bạn có phải là lập trình viên hồi thập niên 1990 không? Bạn có nhớ web đã thay đổi mọi thứ như thế nào không? Bạn có nhớ chúng ta đã nhìn những kiến trúc client–server cũ kỹ của mình bằng ánh mắt khinh thường ra sao trước công nghệ mới bóng bẩy mang tên The Web không?

Actually the web didn’t change anything. Or, at least, it shouldn’t have. The web is just the latest in a series of oscillations that our industry has gone through since the 1960s. These oscillations move back and forth between putting all the computer power in central servers and putting all computer power out at the terminals.

Thật ra web chẳng thay đổi điều gì cả. Hoặc ít nhất, lẽ ra nó không nên thay đổi. Web chỉ là dao động mới nhất trong một chuỗi dao động mà ngành của chúng ta đã trải qua kể từ thập niên 1960. Những dao động ấy cứ đi qua đi lại giữa hai thái cực: dồn toàn bộ sức mạnh tính toán vào các máy chủ trung tâm, và đẩy toàn bộ sức mạnh tính toán ra các thiết bị đầu cuối.

We’ve seen several of these oscillations just in the last decade or so since the web became prominent. At first we thought all the computer power would be in server farms, and the browsers would be stupid. Then we started putting applets in the browsers. But we didn’t like that, so we moved dynamic content back to the servers. But then we didn’t like that, so we invented Web 2.0 and moved lots of processing back into the browser with Ajax and JavaScript. We went so far as to create whole huge applications written to execute in the browsers. And now we’re all excited about pulling that JavaScript back into the server with Node.

Chỉ trong khoảng một thập kỷ kể từ khi web lên ngôi, chúng ta đã chứng kiến vài lần dao động như thế. Ban đầu chúng ta tưởng mọi sức mạnh tính toán sẽ nằm trong các server farm, còn trình duyệt thì ngu ngơ. Rồi chúng ta bắt đầu nhét applet vào trình duyệt. Nhưng chúng ta không thích thế, nên lại chuyển nội dung động về phía server. Rồi chúng ta cũng không thích thế nốt, nên chúng ta phát minh ra Web 2.0 và dời cả đồng xử lý ngược về trình duyệt bằng Ajax và JavaScript. Chúng ta đi xa tới mức viết hẳn những ứng dụng khổng lồ chạy hoàn toàn trong trình duyệt. Và giờ đây chúng ta lại đang hào hứng kéo chính JavaScript đó ngược về server với Node.

(Sigh.)

(Thở dài.)

The Endless Pendulum — Con lắc bất tận

Of course, it would be incorrect to think that those oscillations started with the web. Before the web, there was client-server **architecture**. Before that, there were central minicomputers with arrays of dumb terminals. Before that, there were mainframes with smart green-screen terminals (that were very much analogous to modern-day browsers). Before that, there were computer rooms and punched cards ...

Dĩ nhiên, sẽ là sai nếu nghĩ rằng những dao động ấy bắt đầu cùng với web. Trước web đã có kiến trúc client-server. Trước đó nữa là những chiếc minicomputer trung tâm với cả dàn terminal câm. Trước đó nữa là mainframe với các terminal màn hình xanh thông minh (rất giống trình duyệt ngày nay). Trước đó nữa là những phòng máy tính và những tấm bìa đục lỗ...

And so the story goes. We can't seem to figure out where we want the computer power. We go back and forth between centralizing it and distributing it. And, I imagine, those oscillations will continue for some time to come.

Và câu chuyện cứ thế tiếp diễn. Xem ra chúng ta chẳng bao giờ quyết được mình muốn đặt sức mạnh tính toán ở đâu. Chúng ta cứ đi tới đi lui giữa tập trung hóa và phân tán. Và tôi đồ rằng những dao động ấy sẽ còn tiếp tục dài dài.

When you look at it in the overall scope of IT history, the web didn't change anything at all. The web was simply one of many oscillations in a struggle that began before most of us were born and will continue well after most of us have retired.

Nếu nhìn trên toàn cảnh lịch sử ngành IT, web chẳng thay đổi gì hết. Web chỉ đơn giản là một trong vô số dao động của một cuộc giằng co đã bắt đầu từ trước khi phần lớn chúng ta ra đời, và sẽ còn tiếp diễn rất lâu sau khi phần lớn chúng ta đã về hưu.

As architects, though, we have to look at the long term. Those oscillations are just short-term issues that we want to push away from the central core of our business rules.

Tuy nhiên, với tư cách kiến trúc sư, chúng ta phải nhìn dài hạn. Những dao động ấy chỉ là chuyện ngắn hạn, và ta muốn đẩy chúng ra xa khỏi phần lõi trung tâm chứa các quy tắc nghiệp vụ (business rule) của mình.

Let me tell you the story of company Q. Company Q built a very popular personal finance system. It was a desktop app with a very useful GUI. I loved using it.

Để tôi kể bạn nghe chuyện của công ty Q. Công ty Q làm ra một hệ thống quản lý tài chính cá nhân rất được ưa chuộng. Đó là một ứng dụng desktop với giao diện GUI cực kỳ hữu dụng. Tôi mê dùng nó.

Then came the web. In its next release, company Q changed the GUI to look, and behave, like a browser. I was thunderstruck! What marketing genius decided that personal finance software, running on a desktop, should have the look and feel of a web browser?

Rồi web ập tới. Ở bản phát hành kế tiếp, công ty Q đổi GUI sao cho trông và hoạt động y hệt một trình duyệt. Tôi choáng váng! Thiên tài marketing nào đã quyết rằng một phần mềm tài chính cá nhân chạy trên desktop lại phải mang dáng dấp và cảm giác của trình duyệt web?

Of course, I hated the new **interface**. Apparently everyone else did, too— because after a few releases, company Q gradually removed the browser-like feel and turned its personal finance system back into a regular desktop GUI.

Đĩ nhiên là tôi ghét cay ghét đắng giao diện mới đó. Và hình như ai cũng ghét — bởi sau vài bản phát hành, công ty Q dần dần gỡ bỏ cái cảm giác giống trình duyệt và trả hệ thống tài chính cá nhân của mình về đúng một GUI desktop bình thường.

Now imagine you were a software architect at Q. Imagine that some marketing genius convinces upper management that the whole UI has to change to look more like the web. What do you do? Or, rather, what should you have done before this point to protect your application from that marketing genius?

Giờ hãy hình dung bạn là kiến trúc sư phần mềm ở Q. Hình dung có một thiên tài marketing nào đó thuyết phục được ban lãnh đạo cấp cao rằng toàn bộ UI phải đổi cho giống web hơn. Bạn sẽ làm gì? Hay đúng hơn: lẽ ra bạn phải làm gì từ trước đó để bảo vệ ứng dụng của mình khỏi vị thiên tài marketing kia?

You should have decoupled your business rules from your UI. I don't know whether the Q architects had done that. One day I'd love to hear their story. Had I been there at the time, I certainly would have lobbied very hard to isolate the business rules from the GUI, because you never know what the marketing geniuses will do next.

Lẽ ra bạn phải tách rời (decoupling) các quy tắc nghiệp vụ khỏi UI. Tôi không biết các kiến trúc sư ở Q có làm vậy hay không. Một ngày nào đó tôi rất muốn nghe câu chuyện của họ. Nếu hồi ấy tôi có mặt ở đó, chắc chắn tôi đã vận động rất quyết liệt để cô lập các quy tắc nghiệp vụ khỏi GUI, bởi bạn chẳng bao giờ đoán được mấy vị thiên tài marketing sẽ dở trò gì tiếp theo.

Now consider company A, which makes a lovely smartphone. Recently it released an upgraded version of its “operating system” (it's so strange that we

Giờ hãy xét công ty A, hãng làm ra một chiếc smartphone rất đẹp. Gần đây họ phát hành bản nâng cấp cho “hệ điều hành” của mình (thật lạ là chúng ta

can talk about the operating system inside a phone). Among other things, that “operating system” upgrade completely changed the look and feel of all the applications. Why? Some marketing genius said so, I suppose.

có thể nói về hệ điều hành bên trong một chiếc điện thoại). Trong số nhiều thứ khác, bản nâng cấp “hệ điều hành” ấy đã thay đổi hoàn toàn dáng vẻ và cảm giác của mọi ứng dụng. Tại sao? Chắc lại có một thiên tài marketing nào đó phán vậy, tôi đoán thế.

I'm not an expert on the software within that device, so I don't know if that change caused any significant difficulties for the programmers of the apps that run in company A's phone. I do hope the architects at A, and the architects of the apps, keep their UI and business rules isolated from each other, because there are always marketing geniuses out there just waiting to pounce on the next little bit of coupling you create.

Tôi không phải chuyên gia về phần mềm bên trong thiết bị đó, nên tôi không biết thay đổi ấy có gây khó khăn đáng kể nào cho lập trình viên của các app chạy trên điện thoại của công ty A hay không. Tôi chỉ mong các kiến trúc sư ở A, cùng các kiến trúc sư của những app kia, đã giữ cho UI và quy tắc nghiệp vụ của họ cô lập khỏi nhau, bởi ngoài kia lúc nào cũng có sẵn những thiên tài marketing chực chờ vồ lấy mẩu kết dính nhỏ nhất tiếp theo mà bạn tạo ra.

The Upshot — Rốt cuộc là

The upshot is simply this: The GUI is a **detail**. The web is a GUI. So the web is a detail. And, as an architect, you want to put details like that behind boundaries that keep them separate from your core business logic.

Rốt cuộc chỉ có thể này: GUI là một chi tiết (detail). Web là một GUI. Vậy web là một chi tiết. Và với tư cách kiến trúc sư, bạn muốn đặt những chi tiết kiểu đó ra sau các ranh giới (boundary) giữ chúng tách biệt khỏi logic nghiệp vụ cốt lõi của mình.

Think about it this way: The WEB is an IO device . In the 1960s, we learned the value of writing applications that were device independent. The motivation for that independence has not changed. The web is not an exception to that rule.

Hãy nghĩ theo cách này: WEB là một thiết bị IO. Từ thập niên 1960, chúng ta đã học được giá trị của việc viết ứng dụng độc lập với thiết bị. Động lực cho sự độc lập đó đến nay vẫn không đổi. Web không phải ngoại lệ của quy tắc ấy.

Or is it? The argument can be made that a GUI, like the web, is so unique and rich that it is absurd to pursue a device-independent architecture. When you think about the intricacies of JavaScript validation or drag-and-drop AJAX calls, or any of the plethora of other widgets and gadgets you can put on a web page, it's easy to argue that device independence is impractical.

Hay là có? Người ta có thể lập luận rằng một GUI, như web chẳng hạn, độc đáo và phong phú đến mức theo đuổi một kiến trúc độc lập với thiết bị là điều phi lý. Khi bạn nghĩ tới những rắc rối tinh vi của việc validate bằng JavaScript, hay những lời gọi AJAX kéo-thả, hay bất kỳ thứ nào trong vô vàn widget và tiện ích mà bạn có thể nhét lên một trang web, thì thật dễ để lập luận rằng độc lập thiết bị là chuyện bất khả thi.

To some extent, this is true. The interaction between the application and the GUI is “chatty” in ways that are quite specific to the kind of GUI you have. The dance between a browser and a web application is different from the dance between a desktop GUI and its application. Trying to abstract out that dance, the way devices are abstracted out of UNIX, seems unlikely to be possible.

Ở một mức độ nào đó, điều này đúng. Tương tác giữa ứng dụng và GUI rất “lắm lời”, theo những cách đặc thù cho đúng loại GUI mà bạn đang dùng. Điệu nhảy giữa trình duyệt và một ứng dụng web khác hẳn điệu nhảy giữa một GUI desktop và ứng dụng của nó. Cố trừu tượng hóa điệu nhảy ấy, theo kiểu UNIX trừu tượng hóa các thiết bị, xem ra khó lòng khả thi.

But another **boundary** between the UI and the application can be abstracted. The business logic can be thought of as a suite of use cases, each of which performs some function on behalf of a user. Each **use case** can be described based on the input data, the processing preformed, and the output data.

Nhưng có một ranh giới khác giữa UI và ứng dụng thì lại có thể trừu tượng hóa được. Logic nghiệp vụ có thể được xem như một tập các use case, mỗi use case thực hiện một chức năng nào đó thay mặt người dùng. Mỗi use case có thể được mô tả dựa trên dữ liệu đầu vào, phần xử lý được thực hiện, và dữ liệu đầu ra.

At some point in the dance between the UI and the application, the input data can be said to be complete, allowing the use case to be executed. Upon completion, the resultant data can be fed back into the dance between the UI and the application.

Tại một thời điểm nào đó trong điệu nhảy giữa UI và ứng dụng, ta có thể nói rằng dữ liệu đầu vào đã đầy đủ, cho phép use case được thực thi. Khi hoàn tất, dữ liệu kết quả có thể được đưa ngược trở lại vào điệu nhảy giữa UI và ứng dụng.

The complete input data and the resultant output data can be placed into data structures and used as the input values and output values for a process that executes the use case. With this approach, we can consider each use case to be operating the IO device of the UI in a device-independent manner.

Toàn bộ dữ liệu đầu vào và dữ liệu đầu ra kết quả có thể được đặt vào các cấu trúc dữ liệu và dùng làm giá trị vào, giá trị ra cho một tiến trình thực thi use case đó. Với cách tiếp cận này, ta có thể coi mỗi use case như đang vận hành thiết bị IO là UI theo một cách độc lập với thiết bị.

Conclusion — Kết luận

This kind of abstraction is not easy, and it will likely take several iterations to get just right. But it is possible. And since the world is full of marketing geniuses, it's not hard to make the case that it's often very necessary.

Kiểu trừu tượng hóa này không hề dễ, và có lẽ phải mất vài vòng lặp mới làm cho thật chuẩn được. Nhưng nó khả thi. Và vì thế giới này đầy rẫy những thiên tài marketing, chẳng khó gì để lập luận rằng nó thường là rất cần thiết.

Frameworks Are Details — Framework là chi tiết

Frameworks have become quite popular. Generally speaking, this is a good thing. There are many frameworks out there that are free, powerful, and useful.

Framework đã trở nên khá phổ biến. Nói chung, đó là chuyện tốt. Ngoài kia có rất nhiều framework miễn phí, mạnh mẽ và hữu ích.

However, frameworks are not architectures—though some try to be.

Tuy nhiên, framework không phải là kiến trúc (architecture) — dù có vài cái cố tỏ ra như vậy.

Framework Authors — Tác giả của framework

Most **framework** authors offer their work for free because they want to be helpful to the community. They want to give back. This is laudable. However, regardless of their high-minded motives, those authors do not have your best interests at heart. They can't, because they don't know you, and they don't know your problems.

Phần lớn tác giả framework cho không công sức của mình vì họ muốn giúp ích cho cộng đồng. Họ muốn đền đáp lại. Điều đó đáng khen. Nhưng bất kể động cơ cao đẹp đến đâu, những tác giả ấy không hề đặt lợi ích của bạn lên hàng đầu. Họ không thể, bởi họ không biết bạn là ai, và họ không biết vấn đề của bạn là gì.

Framework authors know their own problems, and the problems of their coworkers and friends. And they write their frameworks to solve those problems—not yours.

Tác giả framework biết vấn đề của chính họ, và vấn đề của đồng nghiệp cùng bạn bè họ. Và họ viết framework để giải quyết những vấn đề đó — chứ không phải vấn đề của bạn.

Of course, your problems will likely overlap with those other problems quite a bit. If this were not the case, frameworks would not be so popular. To the extent that such overlap exists, frameworks can be very useful indeed.

Tất nhiên, vấn đề của bạn nhiều khả năng sẽ trùng lặp khá khá với những vấn đề kia. Nếu không thì framework đã chẳng phổ biến đến thế. Trong chừng mực có sự trùng lặp ấy, framework quả thực có thể rất hữu ích.

Asymmetric Marriage — Cuộc hôn nhân bất đối xứng

The relationship between you and the framework author is extraordinarily asymmetric. You must make a huge commitment to the framework, but the framework author makes no commitment to you whatsoever.

Quan hệ giữa bạn và tác giả framework bất đối xứng đến kỳ lạ. Bạn phải cam kết cực lớn với framework, còn tác giả framework thì chẳng cam kết gì với bạn cả.

Think about this point carefully. When you use a framework, you read through the documentation that the author of that framework provides. In that documentation, the author, and other users of that framework, advise you on how to integrate your software with the framework. Typically, this means wrapping your **architecture** around that framework. The author recommends that you derive from the framework's base classes, and

Hãy nghĩ kỹ điều này. Khi dùng một framework, bạn đọc hết tài liệu mà tác giả framework đó cung cấp. Trong tài liệu ấy, tác giả — và những người dùng khác của framework — khuyên bạn cách tích hợp phần mềm của mình với framework. Thông thường, điều đó có nghĩa là bọc kiến trúc của bạn quanh framework đó. Tác giả khuyên bạn nên kế thừa từ các base class của framework, và

import the framework's facilities into your business objects. The author urges you to couple your application to the framework as tightly as possible.

import các tiện ích của framework vào các đối tượng nghiệp vụ (business objects) của bạn. Tác giả hối thúc bạn kết dính ứng dụng của mình vào framework càng chặt càng tốt.

For the framework author, coupling to his or her own framework is not a risk. The author wants to couple to that framework, because the author has absolute control over that framework.

Với tác giả framework, kết dính vào chính framework của mình chẳng có rủi ro gì. Tác giả muốn kết dính vào framework đó, bởi tác giả nắm quyền kiểm soát tuyệt đối với nó.

What's more, the author wants you to couple to the framework, because once coupled in this way, it is very hard to break away. Nothing feels more validating to a framework author than a bunch of users willing to inextricably derive from the author's base classes.

Hơn nữa, tác giả muốn bạn kết dính vào framework, vì một khi đã dính kiểu đó thì rất khó dứt ra. Không gì làm một tác giả framework thấy được công nhận hơn là một đám người dùng sẵn lòng kế thừa các base class của mình đến mức không gỡ ra nổi.

In effect, the author is asking you to marry the framework—to make a huge, long-term commitment to that framework. And yet, under no circumstances will the author make a corresponding commitment to you. It's a one- directional marriage. You take on all the risk and burden; the framework author takes on nothing at all.

Trên thực tế, tác giả đang bảo bạn hãy cưới framework — hãy đưa ra một cam kết khổng lồ, dài hạn với framework đó. Vậy mà trong mọi hoàn cảnh, tác giả sẽ chẳng bao giờ đưa ra một cam kết tương ứng với bạn. Đó là một cuộc hôn nhân một chiều. Bạn gánh toàn bộ rủi ro và gánh nặng; tác giả framework chẳng gánh gì hết.

The Risks — Những rủi ro

What are the risks? Here are just a few for you to consider.

Rủi ro là gì? Đây là vài điều để bạn cân nhắc.

- The architecture of the framework is often not very clean. Frameworks tend
 - Kiến trúc của bản thân framework thường không sạch cho lắm. Framework có xu hướng

to violate the Dependency Rule. They ask you to inherit their code into your business objects—your Entities! They want their framework coupled into that innermost circle. Once in, that framework isn't coming back out. The wedding ring is on your finger; and it's going to stay there.

vi phạm Quy tắc Phụ thuộc (the Dependency Rule). Chúng bắt bạn kế thừa mã của chúng vào các đối tượng nghiệp vụ của bạn — vào chính các Entity của bạn! Chúng muốn framework của mình được kết dính vào tận vòng trong cùng. Một khi đã vào rồi, framework đó sẽ không chịu ra nữa. Chiếc nhẫn cưới đã nằm trên ngón tay bạn; và nó sẽ nằm nguyên ở đó.

- The framework may help you with some early features of your application.
 - Framework có thể giúp bạn làm được vài tính năng đầu tiên của ứng dụng.

However, as your product matures, it may outgrow the facilities of the framework. If you've put on that wedding ring, you'll find the framework fighting you more and more as time passes.

Tuy nhiên, khi sản phẩm trưởng thành, nó có thể vượt quá những gì framework cung cấp. Nếu đã đeo chiếc nhẫn cưới ấy vào, bạn sẽ thấy framework càng lúc càng chống lại bạn theo thời gian.

- The framework may evolve in a direction that you don't find helpful. You
 - Framework có thể tiến hoá theo hướng mà bạn thấy chẳng ích gì. Bạn

may be stuck upgrading to new versions that don't help you. You may even find old features, which you made use of, disappearing or changing in ways that are difficult for you to keep up with.

có thể mắc kẹt trong việc nâng cấp lên những phiên bản mới chẳng giúp được gì cho mình. Bạn thậm chí có thể thấy những tính năng cũ mà mình đang dùng bỗng biến mất, hoặc thay đổi theo cách bạn khó lòng theo kịp.

- A new and better framework may come along that you wish you could
 - Một framework mới và tốt hơn có thể xuất hiện, và bạn ước gì mình có thể

switch to.

chuyển sang dùng nó.

The Solution — Giải pháp

What is the solution?

Giải pháp là gì?

Don't marry the framework!

Đừng cưới framework!

Oh, you can use the framework—just don't couple to it. Keep it at arm's length. Treat the framework as a **detail** that belongs in one of the outer circles of the architecture. Don't let it into the inner circles.

Ồ, bạn cứ dùng framework — chỉ đừng kết dính vào nó. Hãy giữ nó ở khoảng cách một sải tay. Hãy xem framework như một chi tiết (detail) thuộc về một trong những vòng ngoài của kiến trúc. Đừng để nó lọt vào các vòng trong.

If the framework wants you to derive your business objects from its base classes, say no! Derive proxies instead, and keep those proxies in components that are plugins to your business rules.

Nếu framework muốn bạn kế thừa các đối tượng nghiệp vụ của mình từ base class của nó, hãy nói không! Thay vào đó, hãy tạo ra các proxy, và đặt những proxy ấy trong các component đóng vai trò plugin cho các quy tắc nghiệp vụ (business rule) của bạn.

Don't let frameworks into your core code. Instead, integrate them into components that plug in to your core code, following **the Dependency Rule**.

Đừng để framework lọt vào phần mã lõi của bạn. Thay vào đó, hãy tích hợp chúng vào những component cắm vào mã lõi, theo đúng Quy tắc Phụ thuộc.

For example, maybe you like Spring. Spring is a good dependency injection framework. Maybe you use Spring to auto-wire your dependencies. That's fine, but you should not sprinkle @autowired annotations all throughout your business objects. Your business objects should not know about Spring.

Ví dụ, có thể bạn thích Spring. Spring là một framework dependency injection tốt. Có thể bạn dùng Spring để tự động nối dây các phụ thuộc. Vậy cũng được thôi, nhưng bạn không nên rắc annotation @autowired khắp nơi trong các đối tượng nghiệp vụ. Các đối tượng nghiệp vụ của bạn không nên biết gì về Spring.

Instead, you can use Spring to inject dependencies into your **Main component**. It's OK for Main to know about Spring since Main is the dirtiest, lowest-level **component** in the architecture.

Thay vào đó, bạn có thể dùng Spring để tiêm phụ thuộc vào component Main. Main biết về Spring thì không sao, vì Main là component bẩn nhất, ở mức thấp nhất trong kiến trúc.

I Now Pronounce You ... — Giờ tôi tuyên bố hai người là...

There are some frameworks that you simply must marry. If you are using C++, for example, you will likely have to marry STL—it's hard to avoid. If you are using Java, you will almost certainly have to marry the standard library.

Có những framework mà bạn đơn giản là buộc phải cưới. Chẳng hạn, nếu dùng C++, nhiều khả năng bạn sẽ phải cưới STL — khó mà tránh được. Nếu dùng Java, gần như chắc chắn bạn sẽ phải cưới thư viện chuẩn.

That's normal—but it should still be a decision . You must understand that when you marry a framework to your application, you will be stuck with that framework for the rest of the life cycle of that application. For better or for worse, in sickness and in health, for richer, for poorer, forsaking all others, you will be using that framework. This is not a commitment to be entered into lightly.

Chuyện đó là bình thường — nhưng nó vẫn phải là một *quyết định*. Bạn phải hiểu rằng khi cưới một framework về cho ứng dụng của mình, bạn sẽ dính với framework đó suốt phần đời còn lại của ứng dụng. Dù tốt hay xấu, dù ốm đau hay khỏe mạnh, dù giàu hay nghèo, khước từ mọi lựa chọn khác, bạn vẫn sẽ phải dùng framework đó. Đây không phải là cam kết để đưa ra một cách nhẹ dạ.

Conclusion — Kết luận

When faced with a framework, try not to marry it right away. See if there aren't ways to date it for a while before you take the plunge. Keep the framework behind an **architectural boundary** if at all possible, for as long as possible. Perhaps you can find a way to get the milk without buying the cow.

Khi đối diện với một framework, đừng vội cưới nó ngay. Hãy xem có cách nào hẹn hò với nó một thời gian trước khi quyết định dẫn thân hay không. Hãy giữ framework nằm sau một ranh giới kiến trúc nếu có thể, và giữ càng lâu càng tốt. Biết đâu bạn tìm được cách có sữa uống mà không phải mua cả con bò.

Case Study: Video Sales — Nghiên cứu tình huống: Bán video

Now it's time to put these rules and thoughts about **architecture** together into a case study. This case study will be short and simple, yet will depict both the process a good architect uses and the decisions that such an architect makes.

Đã đến lúc gom mọi quy tắc và suy nghĩ về kiến trúc (architecture) lại thành một nghiên cứu tình huống. Nghiên cứu này ngắn và đơn giản, nhưng vẫn phác họa được cả quy trình mà một kiến trúc sư giỏi áp dụng lẫn những quyết định mà người đó đưa ra.

The Product — Sản phẩm

For this case study, I've chosen a product with which I am rather intimately familiar: the software for a website that sells videos. Of course, it is reminiscent of cleancoders.com , the site where I sell my software tutorial videos.

Cho nghiên cứu tình huống này, tôi chọn một sản phẩm mà tôi khá thân thuộc: phần mềm cho một website bán video. Dĩ nhiên nó gợi nhớ đến cleancoders.com , trang mà tôi bán các video hướng dẫn của mình.

The basic idea is trivial. We have a batch of videos we want to sell. We sell them, on the web, to both individuals and businesses. Individuals can pay one price to stream the videos, and another, higher price to download those videos and own them permanently. Business licenses are streaming only, and are purchased in batches that allow quantity discounts.

Ý tưởng cơ bản thì tầm thường. Chúng ta có một lô video muốn bán. Ta bán chúng trên web, cho cả cá nhân lẫn doanh nghiệp. Cá nhân có thể trả một mức giá để xem trực tuyến, và một mức giá khác cao hơn để tải video về và sở hữu vĩnh viễn. Giấy phép doanh nghiệp chỉ cho xem trực tuyến, và được mua theo lô để hưởng chiết khấu theo số lượng.

Individuals typically act as both the viewers and the purchasers. Businesses, in contrast, often have people who buy the videos that other people will watch.

Cá nhân thường vừa là người xem vừa là người mua. Ngược lại, ở doanh nghiệp thường có người mua video cho những người khác xem.

Video authors need to supply their video files, written descriptions, and ancillary files with exams, problems, solutions, source code, and other materials.

Tác giả video cần cung cấp tệp video, phần mô tả bằng văn bản, và các tệp phụ trợ gồm đề kiểm tra, bài tập, lời giải, mã nguồn cùng các tài liệu khác.

Administrators need to add new video series, add and delete videos to and from the series, and establish prices for various licenses.

Quản trị viên cần thêm loạt video mới, thêm và xóa video khỏi loạt, và thiết lập giá cho từng loại giấy phép.

Our first step in determining the initial architecture of the system is to identify the actors and use cases.

Bước đầu tiên để xác định kiến trúc ban đầu của hệ thống là nhận diện các actor và các use case.

Use Case Analysis — Phân tích use case

Figure 33.1 shows a typical use-case analysis.

Figure 33.1 cho thấy một bản phân tích use case điển hình.

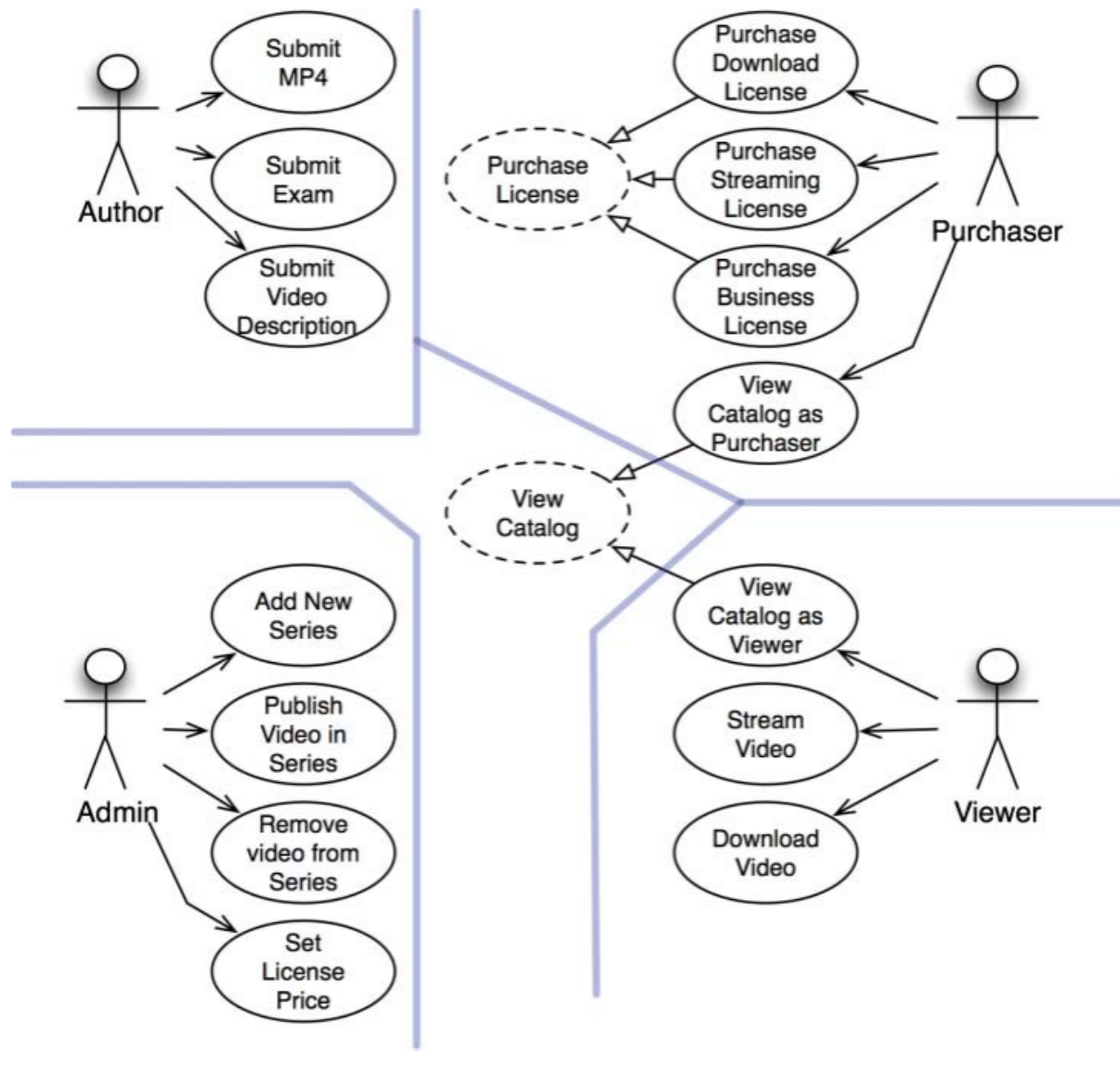


Figure 33.1 A typical use-case analysis

Figure 33.1 Một bản phân tích use case điển hình

The four main actors are evident. According to the Single Responsibility Principle, these four actors will be the four primary sources of change for the system. Every time some new feature is added, or some existing feature is changed, that step will be taken to serve one of these actors. Therefore we want to partition the system such that a change to one **actor** does not affect any of the other actors.

Bốn actor chính hiện ra rất rõ. Theo Single Responsibility Principle, bốn actor này sẽ là bốn nguồn thay đổi chủ yếu của hệ thống. Mỗi lần một tính năng mới được thêm vào, hay một tính năng sẵn có bị đổi, việc đó đều nhằm phục vụ một trong các actor này. Vì vậy ta muốn phân hoạch hệ thống sao cho một thay đổi ở actor này không ảnh hưởng tới bất kỳ actor nào khác.

The use cases shown in Figure 33.1 are not a complete list. For example, you won't find log-in or log-out use cases. The reason for this omission is simply to manage the size of the problem in this book.

If I were to include all the different use cases, then this chapter would have to turn into a book in its own right.

Các use case trong Figure 33.1 không phải là danh sách đầy đủ. Chẳng hạn, bạn sẽ không thấy use case đăng nhập hay đăng xuất. Lý do bỏ sót đơn giản là để kiểm soát quy mô bài toán trong cuốn sách này. Nếu đưa hết mọi use case vào, chương này sẽ phải phình thành một cuốn sách riêng.

Note the dashed use cases in the center of Figure 33.1. They are abstract 1 use cases. An abstract **use case** is one that sets a general **policy** that another use case will flesh out. As you can see, the View Catalog as Viewer and View Catalog as Purchaser use cases both inherit from the View Catalog abstract use case.

Hãy để ý các use case nét đứt ở giữa Figure 33.1. Đó là những use case trừu tượng 1. Một use case trừu tượng là use case đặt ra chính sách (policy) tổng quát để một use case khác cụ thể hóa. Như bạn thấy, hai use case View Catalog as Viewer và View Catalog as Purchaser đều kế thừa (inheritance) từ use case trừu tượng View Catalog.

On the one hand, it was not strictly necessary for me to create that abstraction. I could have left the abstract use case out of the diagram without compromising any of the features of the overall product. On the other hand, these two use cases are so similar that I thought it wise to recognize the similarity and find a way to unify it early in the analysis.

Một mặt, tôi không nhất thiết phải tạo ra sự trừu tượng đó. Tôi có thể bỏ hẳn use case trừu tượng khỏi sơ đồ mà không làm hỏng bất kỳ tính năng nào của sản phẩm tổng thể. Mặt khác, hai use case này giống nhau đến mức tôi thấy nên nhận ra sự tương đồng ấy và tìm cách hợp nhất nó ngay từ sớm trong quá trình phân tích.

Component Architecture — Kiến trúc component

Now that we know the actors and use cases, we can create a preliminary **component** architecture (Figure 33.2).

Giờ đã biết các actor và use case, ta có thể dựng một kiến trúc component sơ bộ (Figure 33.2).

The double lines in the drawing represent architectural boundaries as usual. You can see the typical partitioning of views, presenters, interactors, and controllers. You can also see that I've broken each of those categories up by their corresponding actors.

Các đường đôi trong hình biểu thị ranh giới kiến trúc như thường lệ. Bạn thấy cách phân hoạch quen thuộc thành view, presenter, interactor và controller. Bạn cũng thấy tôi đã tách từng nhóm đó ra theo actor tương ứng.

Each of the components in Figure 33.2 represents a potential .jar file or .dll file. Each of those components will contain the views, presenters, interactors, and controllers that have been allocated to it.

Mỗi component trong Figure 33.2 tượng trưng cho một tệp .jar hoặc .dll tiềm năng. Mỗi component sẽ chứa những view, presenter, interactor và controller đã được phân bổ cho nó.

Note the special components for the Catalog View and the Catalog **Presenter**. This is how I dealt with the abstract View Catalog use case. I assume that those views and presenters will be coded into

abstract classes within those components, and that the inheriting components will contain view and presenter classes that will inherit from those abstract classes.

Hãy chú ý hai component đặc biệt Catalog View và Catalog Presenter . Đó là cách tôi xử lý use case trừu tượng View Catalog. Tôi giả định các view và presenter đó sẽ được viết thành abstract class bên trong những component này, còn các component kế thừa sẽ chứa những class view và presenter kế thừa từ các abstract class ấy.

- 1. This is my own notation for “abstract” use cases. It would have been more standard to use a UML
 - 1. Đây là ký hiệu của riêng tôi cho use case “trừu tượng”. Đúng chuẩn hơn thì nên dùng một UML

stereotype such as <>, but I don’t find adhering to such standards very useful nowadays.

stereotype kiểu như <>, nhưng ngày nay tôi thấy việc bám theo những chuẩn như vậy chẳng mấy hữu ích.

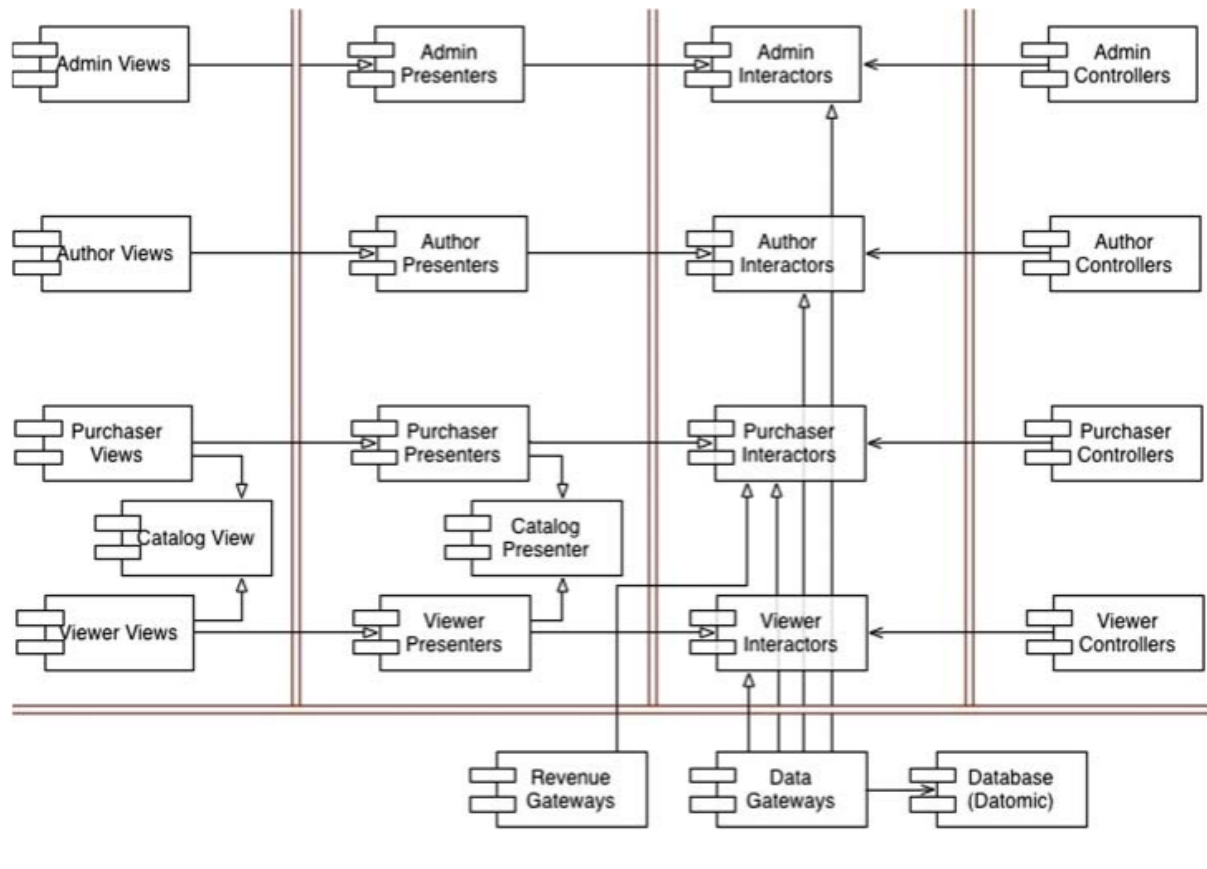


Figure 33.2 A preliminary component architecture

Figure 33.2 Kiến trúc component sơ bộ

Would I really break the system up into all these components, and deliver them as .jar or .dll files? Yes and no. I would certainly break the compile and build environment up this way, so that I could build independent deliverables like that. I would also reserve the right to combine all those deliverables into a smaller number of deliverables if necessary. For example, given the partitioning in Figure 33.2, it would be easy to combine them into five .jar files—one for views, presenters, interactors,

controllers, and utilities, respectively. I could then independently deploy the components that are most likely to change independently of each other.

Liệu tôi có thực sự chẻ hệ thống ra thành ngàn ấy component, rồi giao chúng dưới dạng các tệp .jar hay .dll không? Vừa có vừa không. Tôi chắc chắn sẽ tách môi trường biên dịch và build theo cách này, để có thể build ra những sản phẩm giao độc lập như vậy. Nhưng tôi cũng giữ quyền gộp tất cả những sản phẩm giao đó lại thành một số ít hơn nếu cần. Chẳng hạn, với cách phân hoạch trong Figure 33.2, sẽ rất dễ gộp chúng thành năm tệp .jar — lần lượt cho view, presenter, interactor, controller và tiện ích. Khi đó tôi có thể triển khai độc lập những component nào có khả năng thay đổi độc lập với nhau nhất.

Another possible grouping would be to put the views and presenters together into the same .jar file, and put the interactors, controllers, and utilities in their own .jar file. Still another, even more primitive, grouping would be to create two .jar files, with views and presenters in one file, and everything else in the other.

Một cách nhóm khả dĩ khác là đặt view và presenter chung một tệp .jar, còn interactor, controller và tiện ích vào tệp .jar riêng. Còn một cách nhóm nữa, thô sơ hơn nhiều, là tạo hai tệp .jar: view và presenter trong một tệp, mọi thứ còn lại trong tệp kia.

Keeping these options open will allow us to adapt the way we deploy the system based on how the system changes over time.

Giữ mở các lựa chọn này cho phép ta điều chỉnh cách triển khai hệ thống theo đúng cách hệ thống thay đổi theo thời gian.

Dependency Management — Quản lý phụ thuộc

The **flow of control** in Figure 33.2 proceeds from right to left. Input occurs at the controllers, and that input is processed into a result by the interactors. The presenters then format the results, and the views display those presentations.

Luồng điều khiển trong Figure 33.2 đi từ phải sang trái. Đầu vào xuất hiện tại các controller, rồi được các interactor xử lý thành kết quả. Kế đó presenter định dạng kết quả, và view hiển thị những gì đã được trình bày.

Notice that the arrows do not all flow from the right to the left. In fact, most of them point from left to right. This is because the architecture is following **the Dependency Rule**. All dependencies cross the **boundary** lines in one direction, and they always point toward the components containing the higher-level policy.

Hãy để ý rằng không phải mũi tên nào cũng chạy từ phải sang trái. Thực tế phần lớn chúng chỉ từ trái sang phải. Đó là vì kiến trúc đang tuân theo Quy tắc Phụ thuộc (the Dependency Rule). Mọi phụ thuộc đều cắt qua các đường ranh giới theo một chiều duy nhất, và luôn chỉ về phía những component chứa chính sách ở mức cao hơn.

Also notice that the using relationships (open arrows) point with the flow of control, and that the **inheritance** relationships (closed arrows) point against the flow of control. This depicts our use of the Open–Closed Principle to make sure that the dependencies flow in the right direction, and that changes to **low-level** details do not ripple upward to affect **high-level** policies.

Cũng hãy để ý rằng các quan hệ sử dụng (mũi tên mở) chỉ cùng chiều với luồng điều khiển, còn các quan hệ kế thừa (mũi tên đặc) chỉ ngược chiều luồng điều khiển. Điều này phác họa cách ta dùng Open–Closed Principle để bảo đảm phụ thuộc chảy đúng hướng, và

để những thay đổi ở chi tiết mức thấp không lan ngược lên làm ảnh hưởng tới chính sách mức cao.

Conclusion — Kết luận

The architecture diagram in Figure 33.2 includes two dimensions of separation. The first is the separation of actors based on the Single Responsibility Principle; the second is the Dependency Rule. The goal of both is to separate components that change for different reasons, and at different rates. The different reasons correspond to the actors; the different rates correspond to the different levels of policy.

Sơ đồ kiến trúc trong Figure 33.2 chứa hai chiều phân tách. Chiều thứ nhất là phân tách theo actor dựa trên Single Responsibility Principle; chiều thứ hai là Quy tắc Phụ thuộc. Mục tiêu của cả hai là tách những component thay đổi vì lý do khác nhau, và với nhịp độ khác nhau. Các lý do khác nhau ứng với các actor; các nhịp độ khác nhau ứng với các cấp chính sách khác nhau.

Once you have structured the code this way, you can mix and match how you want to actually deploy the system. You can group the components into deployable deliverables in any way that makes sense, and easily change that grouping when conditions change.

Một khi đã cấu trúc mã theo cách này, bạn có thể tùy ý pha trộn và ghép nối cách triển khai hệ thống trên thực tế. Bạn có thể gom các component thành những sản phẩm giao có thể triển khai theo bất kỳ cách nào hợp lý, và dễ dàng đổi cách gom đó khi hoàn cảnh thay đổi.

The Missing Chapter — Chương còn thiếu

By Simon Brown

Tác giả: Simon Brown

All of the advice you’ve read so far will certainly help you **design** better software, composed of classes and components with well-defined boundaries, clear responsibilities, and controlled dependencies. But it turns out that the devil is in the implementation details, and it’s really easy to fall at the last hurdle if you don’t give that some thought, too.

Tất cả những lời khuyên bạn đã đọc từ đầu tới giờ chắc chắn sẽ giúp bạn thiết kế phần mềm tốt hơn, gồm những class và component có ranh giới (boundary) rõ ràng, trách nhiệm minh bạch và các phụ thuộc được kiểm soát. Nhưng hoá ra ma quỷ nằm ở chi tiết cài đặt, và bạn rất dễ ngã ở rào chắn cuối cùng nếu không dành chút suy nghĩ cho cả chuyện đó nữa.

Let’s imagine that we’re building an online book store, and one of the use cases we’ve been asked to implement is about customers being able to view the status of their orders. Although this is a Java example, the principles apply equally to other programming languages. Let’s put the **Clean Architecture** to one side for a moment and look at a number of approaches to design and code organization.

Hãy tưởng tượng chúng ta đang xây một hiệu sách trực tuyến, và một trong những use case ta được yêu cầu cài đặt là cho phép khách hàng xem trạng thái đơn hàng của họ. Dù đây là ví dụ bằng Java, các nguyên tắc vẫn áp dụng y hệt cho những ngôn ngữ lập trình khác. Hãy tạm gác Clean Architecture sang một bên và nhìn vào một loạt cách tiếp cận khác nhau đối với việc thiết kế và tổ chức mã nguồn.

Package by Layer — Package by Layer

The first, and perhaps simplest, design approach is the traditional horizontal layered **architecture**, where we separate our code based on what it does from a technical perspective. This is often called “**package by layer**.” Figure 34.1 shows what this might look like as a UML class diagram.

Cách tiếp cận thiết kế đầu tiên, và có lẽ cũng đơn giản nhất, là kiến trúc phân tầng theo chiều ngang truyền thống, trong đó ta tách mã nguồn dựa trên việc nó làm gì xét từ góc độ kỹ thuật. Cách này thường được gọi là “package by layer” (đóng gói theo tầng). Figure 34.1 cho thấy nó trông như thế nào dưới dạng một UML class diagram.

In this typical layered architecture, we have one layer for the web code, one layer for our “business logic,” and one layer for persistence. In other words, code is sliced horizontally into layers, which are used as a way to group similar types of things. In a “strict layered architecture,” layers should depend only on the next adjacent lower layer. In Java, layers are typically implemented as packages. As you can see in Figure 34.1, all of the dependencies between layers (packages) point downward. In this example, we have the following Java types:

Trong kiến trúc phân tầng điển hình này, ta có một tầng cho mã web, một tầng cho “business logic”, và một tầng cho việc lưu trữ (persistence). Nói cách khác, mã nguồn được cắt ngang thành các tầng, và các tầng được dùng như một cách nhóm những thứ cùng loại lại với nhau. Trong một “kiến trúc phân tầng nghiêm ngặt”, mỗi tầng chỉ nên phụ thuộc vào tầng liền kề bên dưới. Trong Java, các tầng thường được cài đặt dưới dạng package. Như bạn thấy ở Figure 34.1, mọi phụ thuộc giữa các tầng (package) đều chỉ xuống dưới. Trong ví dụ này, ta có các kiểu Java sau:

- OrdersController : A web controller, something like a Spring MVC
 - OrdersController : Một web controller, kiểu như một Spring MVC

controller, that handles requests from the web.

controller, chịu trách nhiệm xử lý các request đến từ web.

- OrdersService : An **interface** that defines the “business logic” related to
 - OrdersService : Một interface định nghĩa phần “business logic” liên quan tới

orders.

orders.

- OrdersServiceImpl : The implementation of the orders **service**. 1
 - OrdersServiceImpl : Phần cài đặt của orders service. 1
- 1. This is arguably a horrible way to name a class, but as we’ll see later, perhaps it doesn’t really matter.
 - 1. Có thể nói đây là một cách đặt tên class kinh khủng, nhưng như ta sẽ thấy sau này, có lẽ điều đó cũng chẳng thực sự quan trọng.
- OrdersRepository : An interface that defines how we get access to
 - OrdersRepository : Một interface định nghĩa cách ta truy cập tới

persistent order information.

thông tin đơn hàng được lưu trữ lâu dài.

- JdbcOrdersRepository : An implementation of the repository interface.
 - JdbcOrdersRepository : Một cài đặt của interface repository.

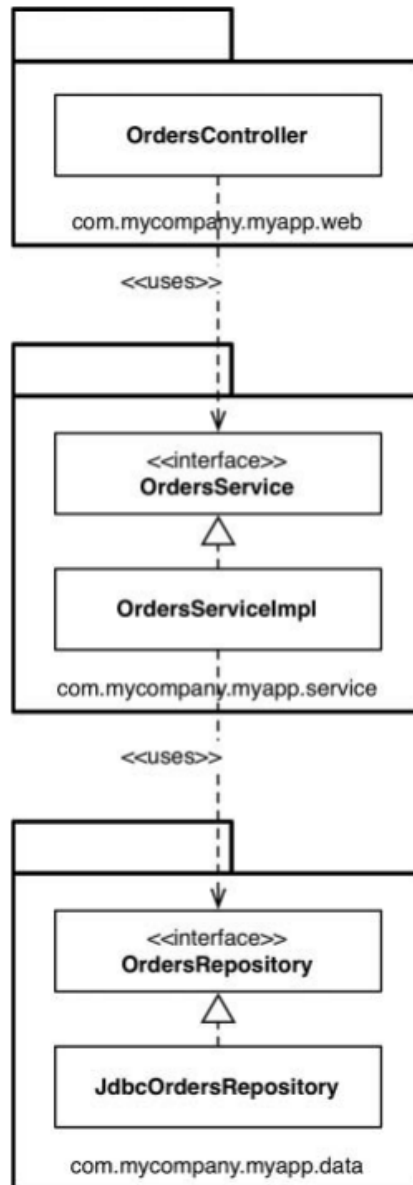


Figure 34.1 Package by layer

Figure 34.1 Package by layer

In “Presentation Domain Data Layering,” 2 Martin Fowler says that adopting such a layered architecture is a good way to get started. He’s not alone. Many of the books, tutorials, training courses, and sample code you’ll find will also

Trong bài “Presentation Domain Data Layering,” 2 Martin Fowler nói rằng áp dụng một kiến trúc phân tầng như vậy là cách khởi đầu tốt. Ông không đơn độc. Rất nhiều sách, bài hướng dẫn, khoá đào tạo và mã ví dụ mà bạn tìm được cũng sẽ

- 2. <https://martinfowler.com/bliki/PresentationDomainDataLayering.html>.
- 2. <https://martinfowler.com/bliki/PresentationDomainDataLayering.html>.

point you down the path of creating a layered architecture. It's a very quick way to get something up and running without a huge amount of complexity. The problem, as Martin points out, is that once your software grows in scale and complexity, you will quickly find that having three large buckets of code isn't sufficient, and you will need to think about modularizing further.

dẫn bạn đi theo con đường tạo ra một kiến trúc phân tầng. Đó là cách rất nhanh để dựng được thứ gì đó chạy được mà không phải gánh quá nhiều độ phức tạp. Vấn đề, như Martin chỉ ra, là một khi phần mềm của bạn lớn lên về quy mô và độ phức tạp, bạn sẽ nhanh chóng nhận ra ba cái xô mã nguồn to đùng là không đủ, và bạn sẽ phải nghĩ tới việc chia module sâu hơn nữa.

Another problem is that, as Uncle Bob has already said, a layered architecture doesn't scream anything about the business domain. Put the code for two layered architectures, from two very different business domains, side by side and they will likely look eerily similar: web, services, and repositories. There's also another huge problem with layered architectures, but we'll get to that later.

Một vấn đề khác là, như Uncle Bob đã nói, kiến trúc phân tầng chẳng gào lên điều gì về miền nghiệp vụ cả. Đặt mã của hai kiến trúc phân tầng thuộc hai miền nghiệp vụ hoàn toàn khác nhau cạnh nhau, chúng sẽ giống nhau đến rợn người: web, services và repositories. Kiến trúc phân tầng còn một vấn đề khổng lồ nữa, nhưng ta sẽ nói tới nó sau.

Package by Feature — Package by Feature

Another option for organizing your code is to adopt a “package **by feature**” style. This is a vertical slicing, based on related features, domain concepts, or aggregate roots (to use domain-driven design terminology). In the typical implementations that I've seen, all of the types are placed into a single Java package, which is named to reflect the concept that is being grouped.

Một lựa chọn khác để tổ chức mã nguồn là áp dụng phong cách “package by feature” (đóng gói theo tính năng). Đây là kiểu cắt dọc, dựa trên các tính năng liên quan, các khái niệm miền, hoặc các aggregate root (dùng thuật ngữ của domain-driven design). Trong những cài đặt điển hình mà tôi từng thấy, tất cả các kiểu đều được đặt vào một package Java duy nhất, được đặt tên phản ánh khái niệm đang được gom nhóm.

With this approach, as shown in Figure 34.2, we have the same interfaces and classes as before, but they are all placed into a single Java package rather than being split among three packages. This is a very simple refactoring from the “package by layer” style, but the top-level organization of the code now screams something about the business domain. We can now see that this code base has something to do with orders rather than the web, services, and repositories. Another benefit is that it's potentially easier to find all of the code that you need to modify in the event that the “view orders” **use case** changes. It's all sitting in a single Java package rather than being spread out. 3

Với cách tiếp cận này, như minh họa ở Figure 34.2, ta vẫn có đúng những interface và class như trước, nhưng tất cả đều nằm trong một package Java duy nhất thay vì bị chia ra ba package. Đây là một refactoring rất đơn giản so với phong cách “package by layer”, nhưng giờ đây cách tổ chức ở tầng cao nhất của mã nguồn đã gào lên điều gì đó về miền nghiệp vụ. Bây giờ ta thấy được rằng code base này có liên quan tới orders, chứ không phải tới web, services và repositories. Một lợi ích khác là có thể sẽ dễ tìm ra toàn bộ đoạn mã cần sửa hơn khi use case “view orders” thay đổi. Tất cả nằm gọn trong một package Java thay vì rải rác khắp nơi. 3

I often see software development teams realize that they have problems with horizontal layering (“package by layer”) and switch to vertical layering

Tôi thường thấy các nhóm phát triển phần mềm nhận ra họ đang gặp vấn đề với việc phân tầng theo chiều ngang (“package by layer”) rồi chuyển sang phân tầng theo chiều dọc

- 3. This benefit is much less relevant with the navigation facilities of modern IDEs, but it seems there has
 - 3. Lợi ích này ít liên quan hơn nhiều với khả năng điều hướng của các IDE hiện đại, nhưng có vẻ như đang có

been a renaissance moving back to lightweight text editors, for reasons I am clearly too old to understand.

một cuộc phục hưng quay về với các trình soạn thảo văn bản nhẹ, vì những lý do mà rõ ràng tôi đã quá già để hiểu.

(“package by feature”) instead. In my opinion, both are suboptimal. If you’ve read this book so far, you might be thinking that we can do much better— and you’re right.

(“package by feature”). Theo ý tôi, cả hai đều chưa tối ưu. Nếu bạn đã đọc cuốn sách này tới đây, có lẽ bạn đang nghĩ rằng ta có thể làm tốt hơn nhiều — và bạn nghĩ đúng.

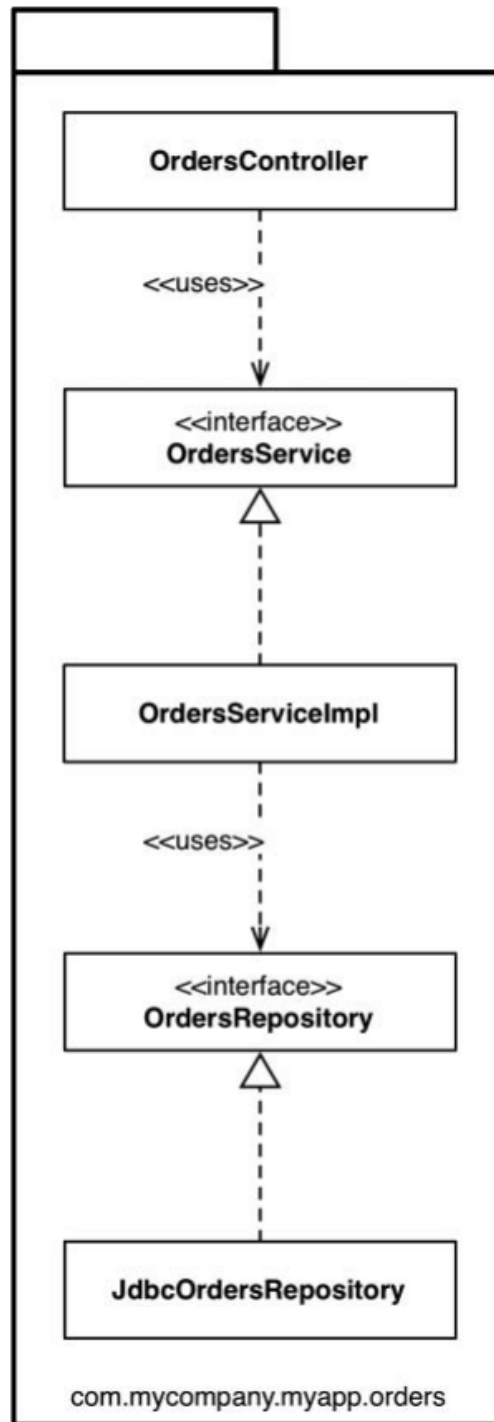


Figure 34.2 Package by feature

Figure 34.2 Package by feature

Ports and Adapters — Ports and Adapters

As Uncle Bob has said, approaches such as “**ports and adapters**,” the “hexagonal architecture,” “boundaries, controllers, entities,” and so on aim to create architectures where business/domain-focused code is independent and separate from the technical implementation details such as frameworks and databases. To summarize, you often see such code bases being composed of an “inside” (domain) and an “outside” (infrastructure), as suggested in Figure 34.3.

Như Uncle Bob đã nói, những cách tiếp cận kiểu “ports and adapters”, “hexagonal architecture”, “boundaries, controllers, entities”, v.v. đều nhằm tới việc tạo ra các kiến trúc trong đó mã tập trung vào nghiệp vụ/miền là độc lập và tách biệt khỏi các chi tiết cài đặt kỹ thuật như framework và database. Tóm lại, bạn sẽ thường thấy những code base như vậy được cấu thành từ một phần “bên trong” (miền) và một phần “bên ngoài” (hạ tầng), như gợi ý ở Figure 34.3.

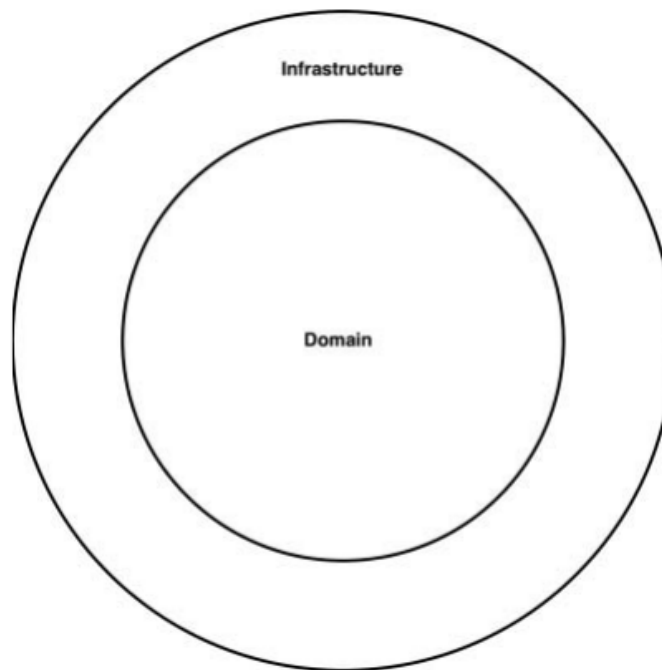


Figure 34.3 A code base with an inside and an outside

Figure 34.3 Một code base có phần bên trong và phần bên ngoài

The “inside” region contains all of the domain concepts, whereas the “outside” region contains the interactions with the outside world (e.g., UIs, databases, third-party integrations). The major rule here is that the “outside” depends on the “inside”—never the other way around. Figure 34.4 shows a version of how the “view orders” use case might be implemented.

Vùng “bên trong” chứa toàn bộ các khái niệm miền, còn vùng “bên ngoài” chứa các tương tác với thế giới bên ngoài (ví dụ: UI, database, tích hợp với bên thứ ba). Quy tắc quan trọng nhất ở đây là “bên ngoài” phụ thuộc vào “bên trong” — không bao giờ ngược lại. Figure 34.4 cho thấy một phiên bản của cách cài đặt use case “view orders”.

The `com.mycompany.myapp.domain` package here is the “inside,” and the other packages are the “outside.” Notice how the dependencies flow toward the “inside.” The keen-eyed reader will notice

that the OrdersRepository from previous diagrams has been renamed to simply be Orders . This comes

Ở đây package `com.mycompany.myapp.domain` chính là phần “bên trong”, còn các package khác là phần “bên ngoài”. Hãy chú ý cách các phụ thuộc chảy về phía “bên trong”. Người đọc tinh mắt sẽ nhận ra rằng OrdersRepository trong các sơ đồ trước đã được đổi tên thành đơn giản là Orders . Điều này đến

from the world of domain-driven design, where the advice is that the naming of everything on the “inside” should be stated in terms of the “ubiquitous domain language.” To put that another way, we talk about “orders” when we’re having a discussion about the domain, not the “orders repository.”

từ thế giới của domain-driven design, nơi lời khuyên là mọi thứ ở “bên trong” nên được đặt tên theo “ngôn ngữ miền phổ quát” (ubiquitous domain language). Nói cách khác, ta nói về “orders” khi bàn luận về miền nghiệp vụ, chứ không nói “orders repository”.

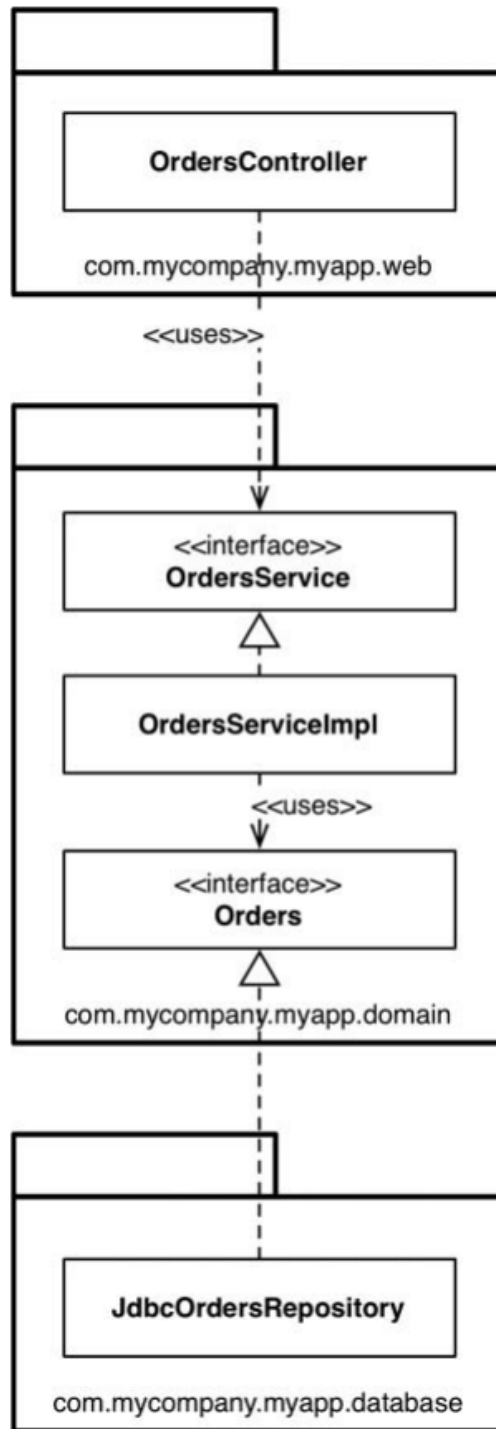


Figure 34.4 View orders use case

Figure 34.4 Use case view orders

It's also worth pointing out that this is a simplified version of what the UML class diagram might look like, because it's missing things like interactors and objects to marshal the data across the dependency boundaries.

Cũng cần chỉ rõ rằng đây là phiên bản giản lược của UML class diagram, bởi nó thiếu những thứ như interactor và các object dùng để chuyển dữ liệu qua các ranh giới phụ thuộc.

Package by Component — Package by Component

Although I agree wholeheartedly with the discussions about SOLID, REP, CCP, and CRP and most of the advice in this book, I come to a slightly different conclusion about how to organize code. So I'm going to present another option here, which I call **"package by component."** To give you some background, I've spent most of my career building enterprise software, primarily in Java, across a number of different business domains. Those software systems have varied immensely, too. A large number have been web-based, but others have been client-server, distributed, message-based, or something else. Although the technologies differed, the common theme was that the architecture for most of these software systems was based on a traditional layered architecture.

Dù tôi hoàn toàn đồng tình với những thảo luận về SOLID, REP, CCP, CRP cùng phần lớn lời khuyên trong cuốn sách này, tôi lại đi tới một kết luận hơi khác về cách tổ chức mã nguồn. Vì vậy tôi sẽ trình bày ở đây thêm một lựa chọn nữa, mà tôi gọi là "package by component" (đóng gói theo component). Kể thêm chút bối cảnh: tôi đã dành phần lớn sự nghiệp để xây dựng phần mềm doanh nghiệp, chủ yếu bằng Java, trải khắp nhiều miền nghiệp vụ khác nhau. Những hệ thống phần mềm ấy cũng khác nhau vô cùng. Rất nhiều trong số đó dựa trên nền web, nhưng số khác lại là client-server, phân tán, dựa trên thông điệp, hoặc một kiểu nào đó khác. Dù công nghệ khác nhau, điểm chung là kiến trúc của hầu hết những hệ thống phần mềm này đều dựa trên kiến trúc phân tầng truyền thống.

I've already mentioned a couple of reasons why layered architectures should be considered bad, but that's not the whole story. The purpose of a layered architecture is to separate code that has the same sort of function. Web stuff is separated from business logic, which is in turn separated from data access. As we saw from the UML class diagram, from an implementation perspective, a layer typically equates to a Java package. From a code accessibility perspective, for the OrdersController to be able to have a dependency on the OrdersService interface, the OrdersService interface needs to be marked as public, because they are in different packages. Likewise, the OrdersRepository interface needs to be marked as public so that it can be seen outside of the repository package, by the OrdersServiceImpl class.

Tôi đã nhắc tới vài lý do vì sao nên coi kiến trúc phân tầng là dở, nhưng đó chưa phải toàn bộ câu chuyện. Mục đích của kiến trúc phân tầng là gom riêng những đoạn mã có cùng loại chức năng vào một chỗ. Phần web được tách khỏi business logic, rồi business logic lại được tách khỏi phần truy cập dữ liệu. Như ta đã thấy từ UML class diagram, xét từ góc độ cài đặt, một tầng thường tương đương với một package Java. Xét từ góc độ khả năng truy cập mã, để OrdersController có thể phụ thuộc vào interface OrdersService, interface OrdersService phải được đánh dấu là public, bởi chúng nằm ở hai package khác nhau. Tương tự, interface OrdersRepository cũng phải được đánh dấu public để class OrdersServiceImpl có thể nhìn thấy nó từ bên ngoài package repository.

- 4. My first job after graduating from university in 1996 was building client-server desktop applications
 - 4. Công việc đầu tiên của tôi sau khi tốt nghiệp đại học năm 1996 là xây dựng ứng dụng desktop client-server

with a technology called PowerBuilder, a super-productive 4GL that excelled at building database-

driven applications. A couple of years later, I was building client–server applications with Java, where we had to build our own database connectivity (this was pre-JDBC) and our own GUI toolkits on top of AWT. That’s “progress” for you!

bằng một công nghệ tên là PowerBuilder, một 4GL siêu năng suất, cực mạnh trong việc dựng các ứng dụng chạy trên database. Vài năm sau, tôi xây dựng ứng dụng client–server bằng Java, nơi chúng tôi phải tự viết lấy phần kết nối database (thời đó chưa có JDBC) và tự viết bộ công cụ GUI của riêng mình trên nền AWT. “Tiến bộ” là như thế đấy!

In a strict layered architecture, the dependency arrows should always point downward, with layers depending only on the next adjacent lower layer. This comes back to creating a nice, clean, acyclic dependency graph, which is achieved by introducing some rules about how elements in a code base should depend on each other. The big problem here is that we can cheat by introducing some undesirable dependencies, yet still create a nice, acyclic dependency graph.

Trong một kiến trúc phân tầng nghiêm ngặt, các mũi tên phụ thuộc phải luôn chỉ xuống dưới, mỗi tầng chỉ phụ thuộc vào tầng liền kề bên dưới. Điều này quay lại chuyện tạo ra một đồ thị phụ thuộc đẹp, sạch, phi chu trình, đạt được bằng cách đặt ra vài quy tắc về việc các thành phần trong code base được phép phụ thuộc vào nhau như thế nào. Vấn đề lớn ở đây là ta vẫn có thể gian lận bằng cách đưa vào những phụ thuộc không mong muốn, mà đồ thị phụ thuộc thu được vẫn đẹp và vẫn phi chu trình.

Suppose that you hire someone new who joins your team, and you give the newcomer another orders-related use case to implement. Since the person is new, he wants to make a big impression and get this use case implemented as quickly as possible. After sitting down with a cup of coffee for a few minutes, the newcomer discovers an existing `OrdersController` class, so he decides that’s where the code for the new orders-related web page should go. But it needs some orders data from the database. The newcomer has an epiphany: “Oh, there’s an `OrdersRepository` interface already built, too. I can simply dependency-inject the implementation into my controller. Perfect!” After a few more minutes of hacking, the web page is working. But the resulting UML diagram looks like Figure 34.5.

Giả sử bạn tuyển một người mới vào nhóm, và giao cho người mới ấy một use case khác liên quan tới orders để cài đặt. Vì là người mới, anh ta muốn tạo ấn tượng mạnh và hoàn thành use case này càng nhanh càng tốt. Sau vài phút ngồi nhâm nhi cà phê, người mới phát hiện ra đã có sẵn một class `OrdersController`, thế là anh ta quyết định đó chính là chỗ để đặt mã cho trang web mới liên quan tới orders. Nhưng trang đó cần lấy dữ liệu orders từ database. Người mới bỗng loé lên: “Ồ, đã có sẵn cả interface `OrdersRepository` nữa. Mình chỉ cần dependency-inject phần cài đặt vào controller là xong. Tuyệt vời!” Sau vài phút gõ lạch cạch nữa, trang web đã chạy. Nhưng sơ đồ UML thu được thì trông như Figure 34.5.

The dependency arrows still point downward, but the `OrdersController` is now additionally bypassing the `OrdersService` for some use cases. This organization is often called a relaxed layered architecture, as layers are allowed to skip around their adjacent neighbor(s). In some situations, this is the intended outcome—if you’re trying to follow the CQRS 5 pattern, for example. In many other cases, bypassing the business logic layer is undesirable, especially if that business logic is responsible for ensuring authorized access to individual records, for example.

Các mũi tên phụ thuộc vẫn chỉ xuống dưới, nhưng giờ `OrdersController` còn đi vòng qua `OrdersService` ở một số use case. Cách tổ chức này thường được gọi là kiến trúc phân tầng nới lỏng (relaxed layered architecture), vì các tầng được phép nhảy cóc qua tầng kề bên. Trong một số tình huống, đó lại đúng là kết quả mong muốn — chẳng hạn nếu bạn đang cố theo mẫu CQRS 5. Nhưng trong nhiều trường hợp khác, đi vòng qua tầng business logic là điều không nên, nhất là khi tầng business logic ấy chịu trách nhiệm đảm bảo quyền truy

cập hợp lệ vào từng bản ghi.

While the new use case works, it's perhaps not implemented in the way that we were expecting. I see this happen a lot with teams that I visit as a consultant, and it's usually revealed when teams start to visualize what their code base really looks like, often for the first time.

Use case mới thì chạy được đấy, nhưng có lẽ nó không được cài đặt theo cách ta mong đợi. Tôi thấy chuyện này xảy ra rất nhiều ở các nhóm tôi ghé thăm với tư cách tư vấn, và nó thường lộ ra khi nhóm bắt đầu hình dung xem code base của mình thực sự trông ra sao — thường là lần đầu tiên.

- 5. In the Command Query Responsibility Segregation pattern, you have separate patterns for
 - 5. Trong mẫu Command Query Responsibility Segregation, bạn có các mẫu riêng biệt cho việc

updating and reading data.

cập nhật và cho việc đọc dữ liệu.

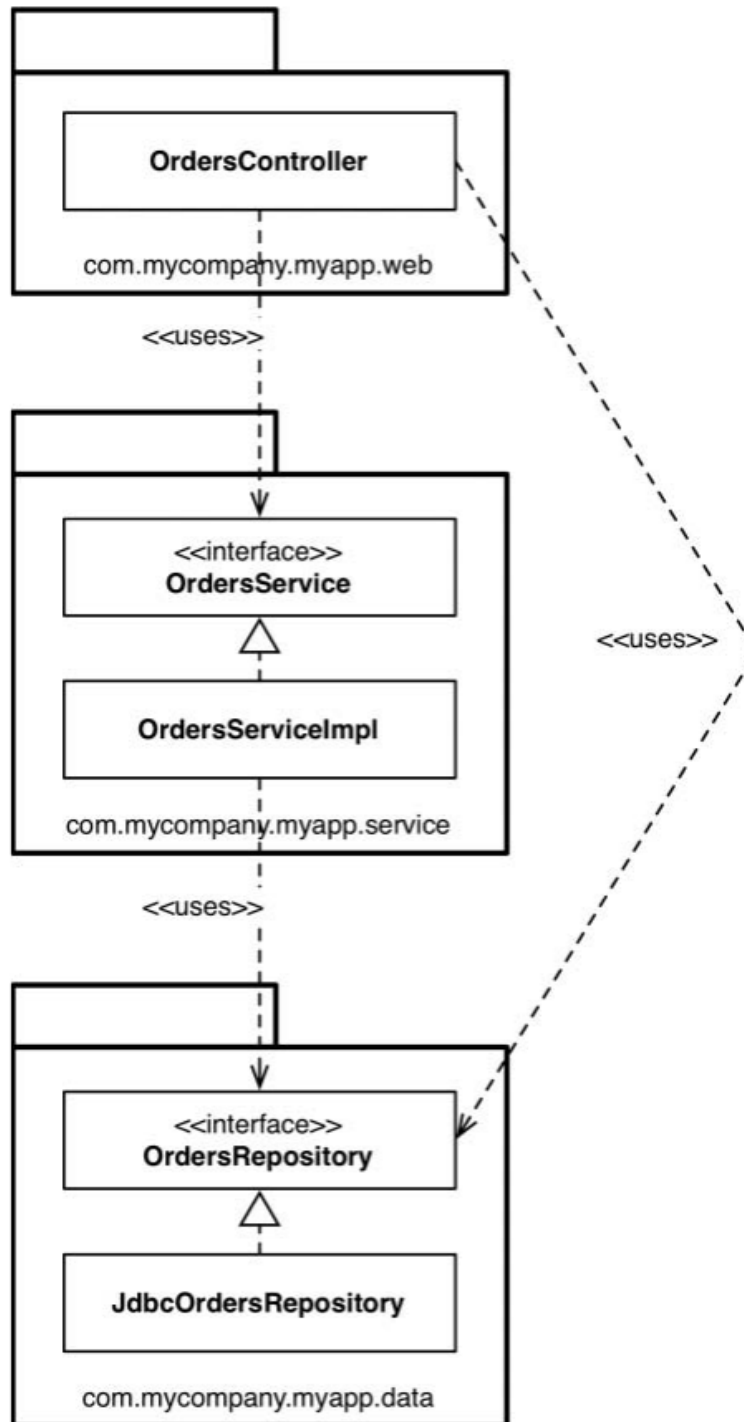


Figure 34.5 Relaxed layered architecture

Figure 34.5 Kiến trúc phân tầng nói lỏng

What we need here is a guideline—an architectural principle—that says something like, “Web controllers should never access repositories directly.” The question, of course, is enforcement. Many teams I’ve met simply say, “We enforce this principle through good discipline and code reviews,

because we

Cái ta cần ở đây là một chỉ dẫn — một nguyên tắc kiến trúc — đại loại như: “Web controller không bao giờ được truy cập repository trực tiếp.” Câu hỏi, tất nhiên, là làm sao thực thi được điều đó. Nhiều nhóm tôi từng gặp chỉ nói đơn giản: “Chúng tôi thực thi nguyên tắc này bằng kỷ luật tốt và code review, vì chúng tôi

trust our developers.” This confidence is great to hear, but we all know what happens when budgets and deadlines start looming ever closer.

tin tưởng lập trình viên của mình.” Nghe sự tự tin ấy thì thật tuyệt, nhưng ai cũng biết chuyện gì xảy ra khi ngân sách cạn dần và deadline mỗi lúc một sát.

A far smaller number of teams tell me that they use static analysis tools (e.g., NDepend, Structure101, Checkstyle) to check and automatically enforce architecture violations at build time. You may have seen such rules yourself; they usually manifest themselves as regular expressions or wildcard strings that state “types in package **/web should not access types in** /data”; and they are executed after the compilation step.

Một số ít hơn nhiều các nhóm nói với tôi rằng họ dùng công cụ phân tích tĩnh (ví dụ NDepend, Structure101, Checkstyle) để kiểm tra và tự động chặn các vi phạm kiến trúc ngay lúc build. Có thể chính bạn cũng từng thấy những luật kiểu đó; chúng thường hiện ra dưới dạng biểu thức chính quy hoặc chuỗi wildcard, phát biểu rằng “các kiểu trong package **/web không được truy cập các kiểu trong** /data”; và chúng được chạy sau bước biên dịch.

This approach is a little crude, but it can do the trick, reporting violations of the architecture principles that you’ve defined as a team and (you hope) failing the build. The problem with both approaches is that they are fallible, and the feedback loop is longer than it should be. If left unchecked, this practice can turn a code base into a “big ball of mud.”⁶ I’d personally like to use the compiler to enforce my architecture if at all possible.

Cách này hơi thô, nhưng có thể ăn thua, vì nó báo cáo những vi phạm đối với các nguyên tắc kiến trúc mà cả nhóm đã thống nhất và (bạn hy vọng thế) làm hỏng bản build. Vấn đề của cả hai cách tiếp cận trên là chúng đều có thể sai sót, và vòng phản hồi dài hơn mức đáng có. Nếu không được kiểm soát, kiểu làm việc này có thể biến một code base thành một “big ball of mud”.⁶ Cá nhân tôi thì muốn dùng chính trình biên dịch để thực thi kiến trúc của mình, nếu điều đó khả thi.

This brings us to the “package by **component**” option. It’s a hybrid approach to everything we’ve seen so far, with the goal of bundling all of the responsibilities related to a single coarse-grained component into a single Java package. It’s about taking a service-centric view of a software system, which is something we’re seeing with **micro-service** architectures as well. In the same way that ports and adapters treat the web as just another delivery mechanism, “package by component” keeps the user interface separate from these coarse-grained components. Figure 34.6 shows what the “view orders” use case might look like.

Điều đó đưa ta tới lựa chọn “package by component”. Đây là cách tiếp cận lai giữa mọi thứ ta đã thấy từ đầu tới giờ, với mục tiêu gom toàn bộ các trách nhiệm liên quan tới một component thô (coarse-grained) duy nhất vào một package Java duy nhất. Đó là cách nhìn hệ thống phần mềm theo hướng lấy dịch vụ làm trung tâm, điều mà ta cũng đang thấy ở các kiến trúc micro-service. Giống như cách ports and adapters coi web chỉ là một cơ chế phân phối nữa mà thôi, “package by component” giữ giao diện người dùng tách khỏi những component thô ấy. Figure 34.6 cho thấy use case “view orders” khi đó sẽ trông như thế nào.

In essence, this approach bundles up the “business logic” and persistence code into a single thing, which I’m calling a “component.” Uncle Bob presented his definition of “component” earlier in the book, saying:

Về bản chất, cách tiếp cận này gộp phần “business logic” và phần mã lưu trữ lại thành một khối duy nhất, thứ mà tôi gọi là “component”. Uncle Bob đã trình bày định nghĩa của ông về “component” ở phần trước của cuốn sách:

Components are the units of deployment. They are the smallest entities that can be deployed as part of a system. In Java, they are jar files.

Component là các đơn vị triển khai. Chúng là những thực thể nhỏ nhất có thể được triển khai như một phần của hệ thống. Trong Java, chúng là các file jar.

- 6. <http://www.laputan.org/mud/>
 - 6. <http://www.laputan.org/mud/>

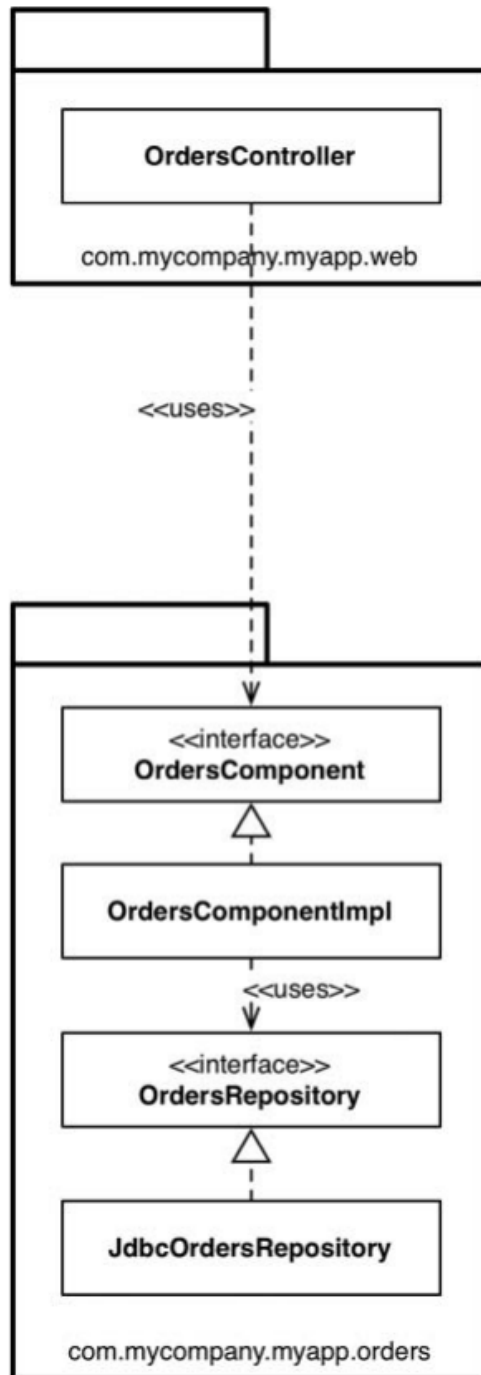


Figure 34.6 View orders use case

Figure 34.6 Use case view orders

My definition of a component is slightly different: “A grouping of related functionality behind a nice clean interface, which resides inside an execution environment like an application.” This definition comes from my “C4 software

Định nghĩa của tôi về component thì hơi khác: “Một nhóm các chức năng liên quan nằm

sau một interface đẹp và sạch, cư trú bên trong một môi trường thực thi như một ứng dụng.” Định nghĩa này đến từ “mô hình kiến trúc phần mềm C4”

architecture model,” 7 which is a simple hierarchical way to think about the static structures of a software system in terms of containers, components, and classes (or code). It says that a software system is made up of one or more containers (e.g., web applications, mobile apps, stand-alone applications, databases, file systems), each of which contains one or more components, which in turn are implemented by one or more classes (or code). Whether each component resides in a separate jar file is an orthogonal concern.

của tôi, 7 một cách phân cấp đơn giản để suy nghĩ về cấu trúc tĩnh của một hệ thống phần mềm theo các khái niệm container, component và class (hay mã nguồn). Nó nói rằng một hệ thống phần mềm được tạo thành từ một hoặc nhiều container (ví dụ: ứng dụng web, ứng dụng di động, ứng dụng độc lập, database, hệ thống tệp), mỗi container chứa một hoặc nhiều component, và mỗi component lại được cài đặt bởi một hoặc nhiều class (hay mã nguồn). Còn việc mỗi component có nằm trong một file jar riêng hay không lại là một mối bận tâm trực giao (orthogonal), hoàn toàn độc lập với chuyện này.

A key benefit of the “package by component” approach is that if you’re writing code that needs to do something with orders, there’s just one place to go—the OrdersComponent. Inside the component, the separation of concerns is still maintained, so the business logic is separate from data persistence, but that’s a component implementation **detail** that consumers don’t need to know about. This is akin to what you might end up with if you adopted a micro-services or Service-Oriented Architecture—a separate OrdersService that encapsulates everything related to handling orders. The key difference is the **decoupling mode**. You can think of well-defined components in a monolithic application as being a stepping stone to a micro-services architecture.

Một lợi ích then chốt của cách tiếp cận “package by component” là nếu bạn đang viết mã cần làm gì đó với orders, thì chỉ có đúng một chỗ để tìm tới — OrdersComponent. Bên trong component, việc tách bạch các mối quan tâm vẫn được duy trì, nên business logic vẫn tách khỏi phần lưu trữ dữ liệu, nhưng đó là chi tiết cài đặt bên trong component mà bên sử dụng không cần biết. Điều này gần giống với thứ bạn sẽ có nếu áp dụng micro-services hay Service-Oriented Architecture — một OrdersService riêng biệt đóng gói mọi thứ liên quan tới việc xử lý orders. Khác biệt mấu chốt nằm ở chế độ tách rời (decoupling mode). Bạn có thể coi những component được định nghĩa rõ ràng trong một ứng dụng nguyên khối như bậc đá lót đường tiến tới kiến trúc micro-service.

The Devil Is in the Implementation Details — Ma quỷ nằm ở chi tiết cài đặt

On the face of it, the four approaches do all look like different ways to organize code and, therefore, could be considered different architectural styles. This perception starts to unravel very quickly if you get the implementation details wrong, though.

Thoạt nhìn, cả bốn cách tiếp cận đều có vẻ chỉ là những cách khác nhau để tổ chức mã nguồn, và vì thế có thể xem là những phong cách kiến trúc khác nhau. Nhưng cảm nhận ấy sẽ nhanh chóng sụp đổ nếu bạn làm sai các chi tiết cài đặt.

Something I see on a regular basis is an overly liberal use of the public access modifier in languages such as Java. It’s almost as if we, as developers, instinctively use the public keyword without thinking. It’s in our muscle memory. If you don’t believe me, take a look at the code samples for books, tutorials, and open source frameworks on GitHub. This tendency is apparent, regardless of

which architectural style a code base is aiming to adopt— horizontal layers, vertical layers, ports and adapters, or something else.

Thứ tôi thấy đều đặn là việc lạm dụng quá mức bố từ truy cập public trong những ngôn ngữ như Java. Cứ như thể chúng ta, những lập trình viên, theo bản năng gõ từ khóa public mà chẳng cần nghĩ. Nó nằm trong trí nhớ cơ bắp của ta rồi. Nếu bạn không tin, hãy thử nhìn vào mã ví dụ của các cuốn sách, các bài hướng dẫn và các framework mã nguồn mở trên GitHub. Xu hướng này lộ rõ, bất kể code base ấy đang nhắm tới phong cách kiến trúc nào — phân tầng ngang, phân tầng dọc, ports and adapters, hay thứ gì khác.

- 7. See <https://www.structurizr.com/help/c4> for more information.
- 7. Xem <https://www.structurizr.com/help/c4> để biết thêm chi tiết.

Marking all of your types as public means you're not taking advantage of the facilities that your programming language provides with regard to **encapsulation**. In some cases, there's literally nothing preventing somebody from writing some code to instantiate a concrete implementation class directly, violating the intended architecture style.

Đánh dấu mọi kiểu của bạn là public nghĩa là bạn đang không tận dụng những phương tiện mà ngôn ngữ lập trình cung cấp cho việc đóng gói (encapsulation). Trong một số trường hợp, đúng nghĩa đen là chẳng có gì ngăn được ai đó viết đoạn mã khởi tạo trực tiếp một class cài đặt cụ thể, vi phạm phong cách kiến trúc đã định.

Organization versus Encapsulation — Tổ chức so với đóng gói

Looking at this issue another way, if you make all types in your Java application public, the packages are simply an organization mechanism (a grouping, like folders), rather than being used for encapsulation. Since public types can be used from anywhere in a code base, you can effectively ignore the packages because they provide very little real value. The net result is that if you ignore the packages (because they don't provide any means of encapsulation and hiding), it doesn't really matter which architectural style you're aspiring to create. If we look back at the example UML diagrams, the Java packages become an irrelevant detail if all of the types are marked as public. In essence, all four architectural approaches presented earlier in this chapter are exactly the same when we overuse this designation (Figure 34.7).

Nhìn vấn đề theo một góc khác: nếu bạn đánh dấu mọi kiểu trong ứng dụng Java của mình là public, thì package chỉ còn là cơ chế tổ chức (một cách gom nhóm, như thư mục), chứ không còn được dùng để đóng gói nữa. Vì kiểu public có thể được dùng từ bất cứ đâu trong code base, bạn hoàn toàn có thể phớt lờ các package bởi chúng mang lại rất ít giá trị thực. Kết quả cuối cùng là nếu bạn phớt lờ các package (vì chúng chẳng cung cấp phương tiện đóng gói và che giấu nào), thì việc bạn khát khao tạo ra phong cách kiến trúc nào cũng chẳng còn mấy ý nghĩa. Nhìn lại các sơ đồ UML ví dụ, các package Java trở thành một chi tiết vô nghĩa nếu tất cả các kiểu đều được đánh dấu public. Về bản chất, cả bốn cách tiếp cận kiến trúc trình bày ở đầu chương này là y hệt nhau khi ta lạm dụng bố từ ấy (Figure 34.7).

Take a close look at the arrows between each of the types in Figure 34.7: They're all identical regardless of which architectural approach you're trying to adopt. Conceptually the approaches are very different, but syntactically they are identical. Furthermore, you could argue that when you make all of the types public, what you really have are just four ways to describe a traditional horizontally layered architecture. This is a neat trick, and of course nobody would ever make all of their Java types public. Except when they do. And I've seen it.

Hãy nhìn kỹ các mũi tên giữa từng kiểu trong Figure 34.7: chúng giống hệt nhau, bất kể bạn đang cố áp dụng cách tiếp cận kiến trúc nào. Về mặt khái niệm, các cách tiếp cận rất khác nhau, nhưng về mặt cú pháp thì chúng y hệt nhau. Hơn nữa, bạn có thể lập luận rằng khi bạn để mọi kiểu là public, thứ bạn thực sự có chỉ là bốn cách mô tả một kiến trúc phân tầng ngang truyền thống. Đó là một mẹo hay, và tất nhiên chẳng ai lại đi để tất cả các kiểu Java của mình là public cả. Trừ khi họ làm thế thật. Và tôi đã tận mắt thấy.

The access modifiers in Java are not perfect, but ignoring them is just asking for trouble. The way Java types are placed into packages can actually make a huge difference to how accessible (or inaccessible) those types can be when

Các bộ từ truy cập trong Java không hoàn hảo, nhưng phớt lờ chúng thì đúng là chuốc lấy rắc rối. Cách các kiểu Java được đặt vào package thực ra tạo ra khác biệt khổng lồ về mức độ dễ (hay khó) truy cập tới những kiểu đó khi

- 8. In Java, for example, although we tend to think of packages as being hierarchical, it's not possible to
 - 8. Chẳng hạn trong Java, dù ta có xu hướng nghĩ về package như thứ phân cấp, nhưng không thể

create access restrictions based on a package and subpackage relationship. Any hierarchy that you create is in the name of those packages, and the directory **structure** on disk, only.

tạo ra các hạn chế truy cập dựa trên quan hệ package và subpackage. Mọi phân cấp mà bạn tạo ra chỉ nằm ở tên của các package đó và ở cấu trúc thư mục trên đĩa, chỉ vậy thôi.

Java's access modifiers are applied appropriately. If I bring the packages back and mark (by graphically fading) those types where the access modifier can be made more restrictive, the picture becomes pretty interesting (Figure 34.8).

các bộ từ truy cập của Java được áp dụng đúng cách. Nếu tôi đưa các package trở lại và đánh dấu (bằng cách làm mờ trên hình) những kiểu mà bộ từ truy cập có thể siết chặt hơn, bức tranh trở nên khá thú vị (Figure 34.8).

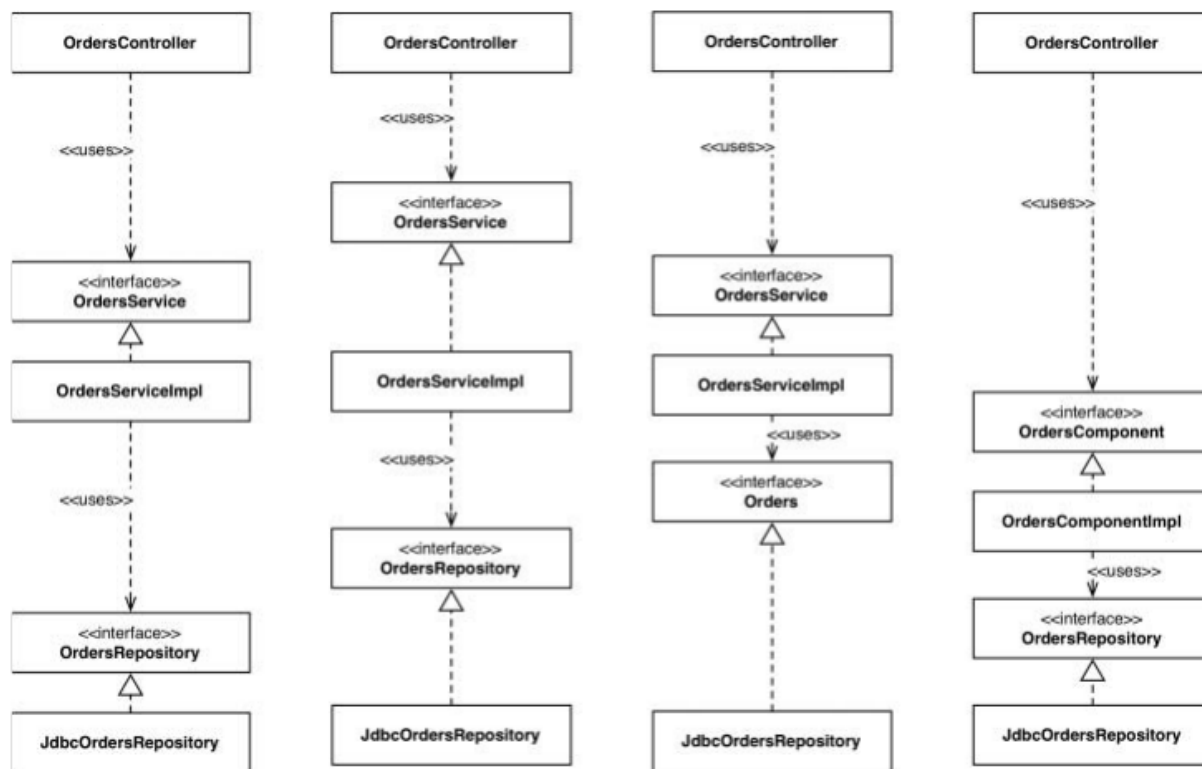


Figure 34.7 All four architectural approaches are the same

Figure 34.7 Cả bốn cách tiếp cận kiến trúc đều như nhau

Moving from left to right, in the “package by layer” approach, the OrdersService and OrdersRepository interfaces need to be public, because they have inbound dependencies from classes outside of their defining package. In contrast, the implementation classes (OrdersServiceImpl and JdbcOrdersRepository) can be made more restrictive (package protected). Nobody needs to know about them; they are an implementation detail.

Đi từ trái sang phải: trong cách tiếp cận “package by layer”, các interface OrdersService và OrdersRepository buộc phải là public, bởi chúng có những phụ thuộc đi vào từ các class nằm ngoài package định nghĩa chúng. Ngược lại, các class cài đặt (OrdersServiceImpl và JdbcOrdersRepository) có thể siết chặt hơn (package protected). Chẳng ai cần biết tới chúng cả; chúng là chi tiết cài đặt.

In the “package by feature” approach, the OrdersController provides the sole entry point into the package, so everything else can be made package protected. The big caveat here is that nothing else in the code base, outside of this package, can access information related to orders unless they go through the controller. This may or may not be desirable.

Trong cách tiếp cận “package by feature”, OrdersController là lối vào duy nhất của package, nên mọi thứ còn lại có thể để package protected. Điều cần lưu ý lớn ở đây là không thứ gì khác trong code base, nằm ngoài package này, có thể truy cập thông tin liên quan tới orders trừ khi đi qua controller. Điều đó có thể là mong muốn, mà cũng có thể không.

In the ports and adapters approach, the OrdersService and Orders interfaces have inbound

dependencies from other packages, so they need to be made public . Again, the implementation classes can be made package protected and dependency injected at runtime.

Trong cách tiếp cận ports and adapters, các interface OrdersService và Orders có phụ thuộc đi vào từ những package khác, nên chúng phải là public . Một lần nữa, các class cài đặt có thể để package protected và được dependency inject lúc chạy.

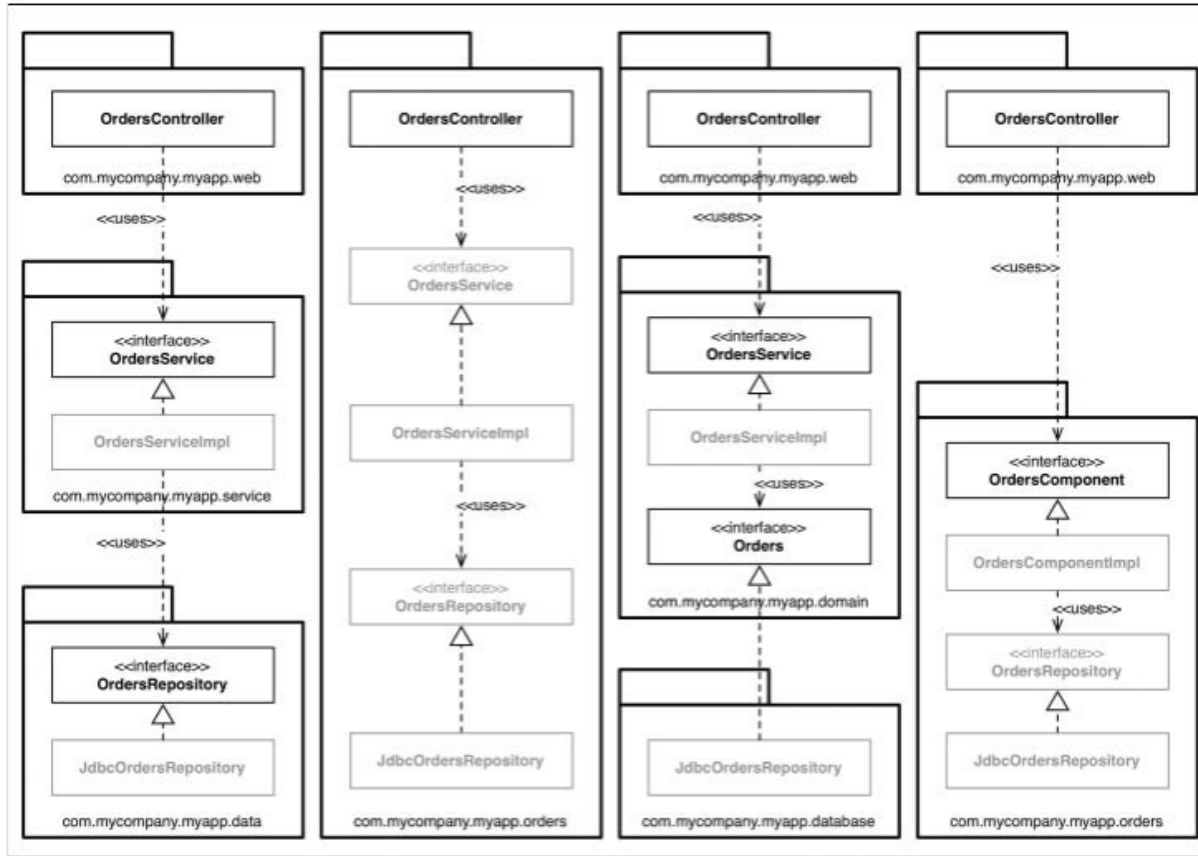


Figure 34.8 Grayed-out types are where the access modifier can be made more restrictive

Figure 34.8 Những kiểu bị làm xám là nơi bổ từ truy cập có thể siết chặt hơn

Finally, in the “package by component” approach, the OrdersComponent interface has an inbound dependency from the controller, but everything else can be made package protected. The fewer public types you have, the smaller the number of potential dependencies. There’s now no way that code outside this package can use the OrdersRepository interface or implementation directly, so we can rely on the compiler to enforce this architectural principle. You can do the same thing in .NET with the internal keyword, although you would need to create a separate assembly for every component.

Cuối cùng, trong cách tiếp cận “package by component”, interface OrdersComponent có một phụ thuộc đi vào từ controller, còn mọi thứ khác đều có thể để package protected. Bạn càng có ít kiểu public thì số phụ thuộc tiềm tàng càng nhỏ. Giờ đây không còn cách nào để mã bên ngoài package này dùng trực tiếp interface OrdersRepository hay phần cài đặt của nó, nên ta có thể dựa vào trình biên dịch để thực thi nguyên tắc kiến trúc ấy. Bạn có thể làm điều tương tự trong .NET với từ khóa internal , dù khi đó bạn sẽ phải tạo một

assembly riêng cho mỗi component.

- 9. Unless you cheat and use Java's reflection mechanism, but please don't do that!
 - 9. Trừ khi bạn gian lận và dùng cơ chế reflection của Java, nhưng làm ơn đừng làm thế!

Just to be absolutely clear, what I've described here relates to a monolithic application, where all of the code resides in a single source code tree. If you are building such an application (and many people are), I would certainly encourage you to lean on the compiler to enforce your architectural principles, rather than relying on self-discipline and post-compilation tooling.

Nói cho thật rõ ràng: những gì tôi mô tả ở đây liên quan tới một ứng dụng nguyên khối, nơi toàn bộ mã nguồn nằm trong một cây mã nguồn duy nhất. Nếu bạn đang xây một ứng dụng như vậy (và rất nhiều người đang làm thế), tôi chắc chắn khuyến khích bạn dựa vào trình biên dịch để thực thi các nguyên tắc kiến trúc của mình, thay vì trông cậy vào tính tự giác và các công cụ chạy sau biên dịch.

Other Decoupling Modes — Các chế độ tách rời khác

In addition to the programming language you're using, there are often other ways that you can decouple your source code dependencies. With Java, you have module frameworks like OSGi and the new Java 9 module system. With module systems, when used properly, you can make a distinction between types that are public and types that are published . For example, you could create an Orders module where all of the types are marked as public , but publish only a small subset of those types for external consumption. It's been a long time coming, but I'm enthusiastic that the Java 9 module system will give us another tool to build better software, and spark people's interest in design thinking once again.

Ngoài chính ngôn ngữ lập trình bạn dùng, thường vẫn còn những cách khác để tách rời các phụ thuộc mã nguồn. Với Java, bạn có các framework module như OSGi và hệ thống module mới của Java 9. Với hệ thống module, khi dùng đúng cách, bạn có thể phân biệt giữa các kiểu là public và các kiểu được published . Chẳng hạn, bạn có thể tạo một module Orders trong đó mọi kiểu đều được đánh dấu public , nhưng chỉ publish một tập con nhỏ trong số đó cho bên ngoài dùng. Phải chờ khá lâu, nhưng tôi rất hào hứng khi nghĩ tới việc hệ thống module của Java 9 sẽ cho chúng ta thêm một công cụ nữa để xây phần mềm tốt hơn, và một lần nữa khơi lại hứng thú của mọi người với tư duy thiết kế.

Another option is to decouple your dependencies at the source code level, by splitting code across different source code trees . If we take the ports and adapters example, we could have three source code trees:

Một lựa chọn khác là tách rời các phụ thuộc ở mức mã nguồn, bằng cách chia mã ra thành nhiều cây mã nguồn khác nhau. Nếu lấy ví dụ ports and adapters, ta có thể có ba cây mã nguồn:

- The source code for the business and domain (i.e., everything that is
 - Mã nguồn cho phần nghiệp vụ và miền (tức là mọi thứ

independent of technology and **framework** choices): OrdersService ,

độc lập với lựa chọn công nghệ và framework): OrdersService ,

OrdersServiceImpl, and Orders

- The source code for the web: `OrdersController`
 - Mã nguồn cho phần web: `OrdersController`
- The source code for the data persistence: `JdbcOrdersRepository`
 - Mã nguồn cho phần lưu trữ dữ liệu: `JdbcOrdersRepository`

The latter two source code trees have a compile-time dependency on the business and domain code, which itself doesn't know anything about the web or the data persistence code. From an implementation perspective, you can do this by configuring separate modules or projects in your build tool (e.g., Maven, Gradle, MSBuild). Ideally you would repeat this pattern, having a separate source code tree for each and every component in your application.

Hai cây mã nguồn sau có phụ thuộc lúc biên dịch vào phần mã nghiệp vụ và miền, còn bản thân phần mã đó chẳng biết gì về mã web hay mã lưu trữ dữ liệu. Xét từ góc độ cài đặt, bạn có thể làm điều này bằng cách cấu hình các module hoặc project riêng biệt trong công cụ build của mình (ví dụ Maven, Gradle, MSBuild). Lý tưởng nhất là bạn lặp lại mẫu này, để mỗi component trong ứng dụng đều có một cây mã nguồn riêng.

This is very much an idealistic solution, though, because there are real-world performance, complexity, and maintenance issues associated with breaking up your source code in this way.

Tuy nhiên đây là một giải pháp mang tính lý tưởng hoá, bởi việc xé nhỏ mã nguồn theo kiểu này kéo theo những vấn đề rất thật về hiệu năng, độ phức tạp và bảo trì.

A simpler approach that some people follow for their ports and adapters code is to have just two source code trees:

Một cách đơn giản hơn mà một số người theo cho phần mã ports and adapters của họ là chỉ dùng hai cây mã nguồn:

- Domain code (the “inside”)
 - Mã miền (phần “bên trong”)
- Infrastructure code (the “outside”)
 - Mã hạ tầng (phần “bên ngoài”)

This maps on nicely to the diagram (Figure 34.9) that many people use to summarize the ports and adapters architecture, and there is a compile-time dependency from the infrastructure to the domain.

Cách này ăn khớp rất đẹp với sơ đồ (Figure 34.9) mà nhiều người dùng để tóm tắt kiến trúc ports and adapters, và có một phụ thuộc lúc biên dịch từ hạ tầng vào miền.

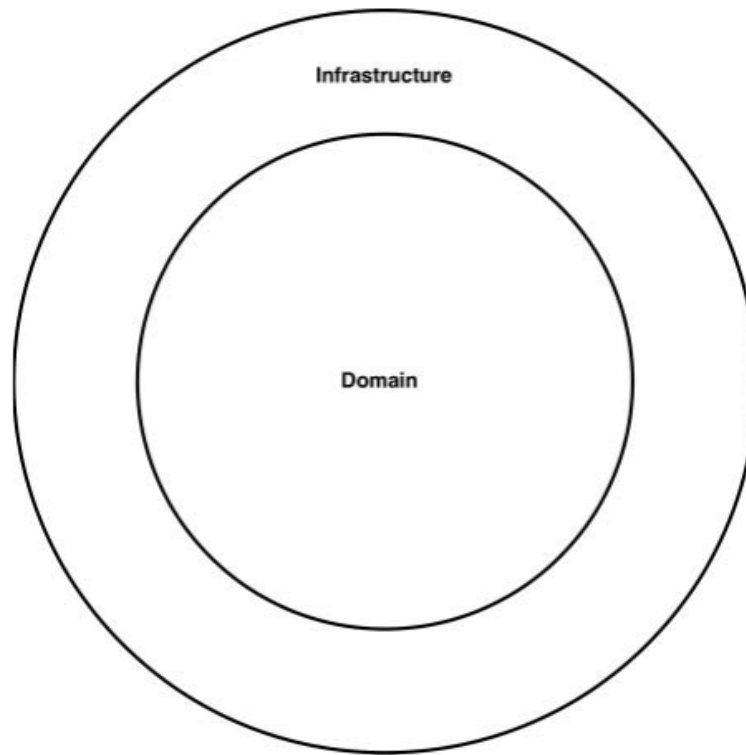


Figure 34.9 Domain and infrastructure code

Figure 34.9 Mã miền và mã hạ tầng

This approach to organizing source code will also work, but be aware of the potential trade-off. It's what I call the “Périphérique anti-pattern of ports and adapters.” The city of Paris, France, has a ring road called the Boulevard Périphérique, which allows you to circumnavigate Paris without entering the

Cách tổ chức mã nguồn này cũng chạy được, nhưng hãy cảnh giác với cái giá phải đánh đổi. Đó là thứ tôi gọi là “anti-pattern Périphérique của ports and adapters”. Thành phố Paris, Pháp, có một đường vành đai tên là Boulevard Périphérique, cho phép bạn đi vòng quanh Paris mà không phải lao vào

complexities of the city. Having all of your infrastructure code in a single source code tree means that it's potentially possible for infrastructure code in one area of your application (e.g., a web controller) to directly call code in another area of your application (e.g., a database repository), without navigating through the domain. This is especially true if you've forgotten to apply appropriate access modifiers to that code.

mớ phức tạp của nội đô. Việc dồn toàn bộ mã hạ tầng vào một cây mã nguồn duy nhất nghĩa là hoàn toàn có khả năng mã hạ tầng ở một vùng của ứng dụng (ví dụ một web controller) gọi thẳng tới mã ở một vùng khác (ví dụ một database repository), mà chẳng cần đi qua miền nghiệp vụ. Điều này càng đúng nếu bạn quên áp dụng bổ từ truy cập thích hợp cho đoạn mã đó.

Conclusion: The Missing Advice — Kết luận: Lời khuyên còn thiếu

The whole point of this chapter is to highlight that your best design intentions can be destroyed in a flash if you don't consider the intricacies of the implementation strategy. Think about how to map your desired design on to code structures, how to organize that code, and which **decoupling** modes to apply during runtime and compile-time. Leave options open where applicable, but be pragmatic, and take into consideration the size of your team, their skill level, and the complexity of the solution in conjunction with your time and budgetary constraints. Also think about using your compiler to help you enforce your chosen architectural style, and watch out for coupling in other areas, such as data models. The devil is in the implementation details.

Toàn bộ ý của chương này là để nhấn mạnh rằng những ý đồ thiết kế tốt đẹp nhất của bạn có thể tan tành trong chớp mắt nếu bạn không cân nhắc tới sự tinh vi của chiến lược cài đặt. Hãy nghĩ xem làm sao ánh xạ thiết kế mong muốn của bạn xuống các cấu trúc mã, tổ chức mã đó ra sao, và áp dụng chế độ tách rời nào lúc chạy và lúc biên dịch. Hãy để ngỏ các lựa chọn ở những chỗ phù hợp, nhưng hãy thực dụng, và cân nhắc quy mô của nhóm, trình độ của họ, độ phức tạp của giải pháp, đặt cạnh những ràng buộc về thời gian và ngân sách. Cũng hãy nghĩ tới việc dùng trình biên dịch để giúp bạn thực thi phong cách kiến trúc đã chọn, và để mắt tới sự kết dính ở những vùng khác nữa, chẳng hạn như mô hình dữ liệu. Ma quỷ nằm ở chi tiết cài đặt.

Appendix — Phụ lục

Architecture Archaeology — Khảo cổ học kiến trúc

To unearth the principles of good **architecture**, let's take a 45-year journey through some of the projects I have worked on since 1970. Some of these projects are interesting from an architectural point of view. Others are interesting because of the lessons learned and because of how they fed into subsequent projects.

Để đào lên những nguyên tắc của một kiến trúc (architecture) tốt, hãy cùng tôi làm một chuyến đi 45 năm qua vài dự án mà tôi đã tham gia kể từ năm 1970. Có dự án thú vị ở góc nhìn kiến trúc. Có dự án thú vị vì những bài học rút ra được, và vì cách chúng nuôi dưỡng những dự án về sau.

This appendix is somewhat autobiographical. I've tried to keep the discussion relevant to the topic of architecture; but, as in anything autobiographical, other factors sometimes intrude. ;-)

Phụ lục này ít nhiều mang tính tự truyện. Tôi đã cố giữ cho phần bàn luận bám sát chủ đề kiến trúc; nhưng, như mọi thứ mang tính tự truyện, đôi khi vẫn có những yếu tố khác chen vào. ;-)

Union Accounting System — Hệ thống kế toán công đoàn

In the late 1960s, a company by the name of ASC Tabulating signed a contract with Local 705 of the Teamsters Union to provide an accounting system. The computer ASC chose to implement this system on was a GE Datanet 30, as shown in Figure A.1.

Cuối những năm 1960, một công ty tên là ASC Tabulating ký hợp đồng với Local 705 của Teamsters Union để cung cấp một hệ thống kế toán. Chiếc máy tính mà ASC chọn để triển khai hệ thống này là một GE Datanet 30, như trong Figure A.1.

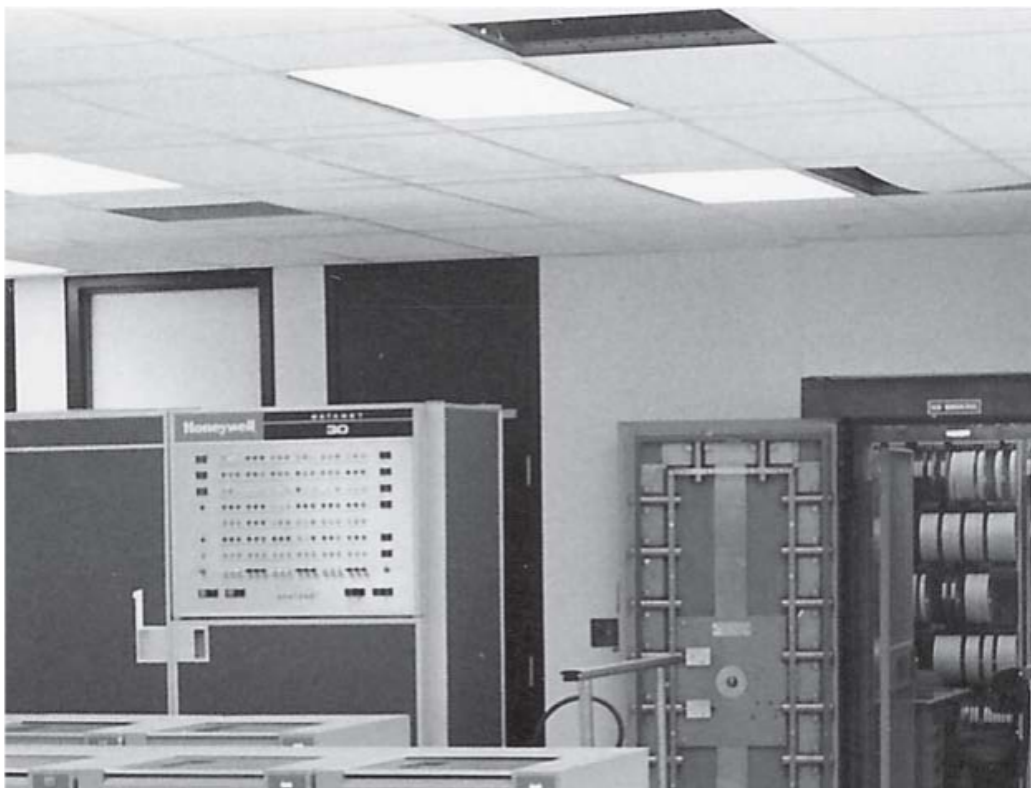


Figure A.1 GE Datanet 30

Figure A.1 GE Datanet 30

Courtesy Ed Thelen, ed-thelen.org

Ảnh của Ed Thelen, ed-thelen.org

As you can see from the picture, this was a huge 1 machine. It filled a room, and the room needed strict environmental controls.

Như bạn thấy trong ảnh, đây là một cỗ máy khổng lồ 1. Nó chiếm trọn một căn phòng, và căn phòng ấy cần được kiểm soát môi trường nghiêm ngặt.

This computer was built in the days before integrated circuits. It was built out of discrete transistors. There were even some vacuum tubes in it (albeit only in the sense amplifiers of the tape drives).

Chiếc máy này được chế tạo vào thời chưa có mạch tích hợp. Nó được dựng lên từ các transistor rời. Trong máy thậm chí còn có cả vài bóng chân không (dù chỉ nằm trong các bộ khuếch đại tín hiệu của ổ băng từ).

By today's standards the machine was huge, slow, small, and primitive. It had 16K * 18 bits of core, with a cycle time of about 7 microseconds. 2 It filled a big, environmentally controlled room. It had 7 track magnetic tape drives and a disk drive with a capacity of 20 megabytes or so.

Theo chuẩn ngày nay, cỗ máy ấy vừa to, vừa chậm, vừa nhỏ dung lượng, lại vừa thô sơ. Nó có bộ nhớ lõi 16K * 18 bit, với chu kỳ khoảng 7 micro giây. 2 Nó chiếm trọn một căn phòng lớn có kiểm soát môi trường. Nó có các ổ băng từ 7 rãnh và một ổ đĩa dung lượng cỡ 20 megabyte.

That disk was a monster. You can see it in the picture in Figure A.2—but that doesn't quite give you the scale of the beast. The top of that cabinet was over my head. The platters were 36 inches in diameter, and 3/8 of an inch thick. One of the platters is pictured in Figure A.3.

Cái ổ đĩa đó là một con quái vật. Bạn có thể thấy nó trong ảnh ở Figure A.2 — nhưng ảnh không diễn tả hết được kích cỡ của con quái này. Nóc chiếc tủ ấy cao quá đầu tôi. Các đĩa từ có đường kính 36 inch và dày 3/8 inch. Một trong các đĩa từ ấy được chụp trong Figure A.3.

Now count the platters in that first picture. There are more than a dozen. Each one had its own individual seek arm that was driven by pneumatic actuators. You could watch those seek heads move across the platters. The seek time was probably about half a second to a second.

Giờ hãy đếm số đĩa trong tấm ảnh đầu tiên. Hơn một tá. Mỗi đĩa có cần định vị (seek arm) riêng, được dẫn động bằng cơ cấu khí nén. Bạn có thể nhìn thấy những đầu định vị ấy chạy ngang qua mặt đĩa. Thời gian truy tìm (seek time) có lẽ vào khoảng nửa giây đến một giây.

When this beast was turned on, it sounded like a jet engine. The floor would rumble and shake until it got up to speed. 3

Khi con quái vật này được bật lên, nó kêu như một động cơ phản lực. Sàn nhà rung ầm ầm cho tới khi nó đạt tốc độ. 3

- 1. One of the stories we heard about the particular machine at ASC was that it was shipped in a large
 - 1. Một trong những câu chuyện chúng tôi nghe được về chính cỗ máy ở ASC là nó được chở đi trên một chiếc

semi-trailer truck along with a household of furniture. On the way, the truck hit a bridge at high speed. The computer was fine, but it slid forward and crushed the furniture into splinters.

xe đầu kéo cỡ lớn, đi kèm cả một bộ đồ nội thất gia đình. Dọc đường, chiếc xe đâm vào một cây cầu ở tốc độ cao. Máy tính thì không sao, nhưng nó trượt tới trước và nghiền nát đồng đồ nội thất thành dăm vụn.

- 2. Today we would say that it had a clock rate of 142 kHz.
 - 2. Ngày nay ta sẽ nói rằng nó có xung nhịp 142 kHz.
- 3. Imagine the mass of that disk. Imagine the kinetic energy! One day we came in and saw little metal
 - 3. Hãy hình dung khối lượng của cái đĩa đó. Hãy hình dung động năng của nó! Một hôm chúng tôi đến và thấy những mảnh kim loại nhỏ

shavings dropping out from the button of the cabinet. We called the maintenance man. He advised us to shut the unit down. When he came to repair it, he said that one of the bearings had worn out. Then he told us stories about how these disks, if not repaired, could tear loose from their moorings, plow through concrete block walls, and embed themselves into cars in the parking lot.

rơi ra từ đáy tủ máy. Chúng tôi gọi thợ bảo trì. Ông ấy khuyên chúng tôi tắt máy đi. Khi đến sửa, ông bảo một trong các vòng bi đã mòn. Rồi ông kể cho chúng tôi nghe những câu chuyện về việc mấy cái đĩa này, nếu không được sửa, có thể bứt khỏi bệ, xuyên thủng tường bê tông và cắm thẳng vào những chiếc xe ngoài bãi đỗ.



Figure A.2 The data storage unit with its platters

Figure A.2 Khối lưu trữ dữ liệu cùng các đĩa từ của nó

Courtesy Ed Thelen, ed-thelen.org

Ảnh của Ed Thelen, ed-thelen.org

The great claim to fame of the Datanet 30 was its capability to drive a large number of asynchronous terminals at relatively high speed. That's exactly what ASC needed.

Điểm làm nên danh tiếng của Datanet 30 là khả năng điều khiển một số lượng lớn terminal bất đồng bộ ở tốc độ tương đối cao. Đó đúng là thứ ASC cần.

ASC was based in Lake Bluff, Illinois, 30 miles north of Chicago. The Local 705 office was in downtown Chicago. The union wanted a dozen or so of their data entry clerks to use CRT 4 terminals (Figure A.4) to enter data into the system. They would print reports on ASR35 teletypes (Figure A.5).

ASC đặt trụ sở ở Lake Bluff, Illinois, cách Chicago 30 dặm về phía bắc. Văn phòng của Local 705 nằm ở trung tâm Chicago. Công đoàn muốn khoảng một tá nhân viên nhập liệu của họ dùng terminal CRT 4 (Figure A.4) để nhập dữ liệu vào hệ thống. Họ sẽ in báo cáo trên máy teletype ASR35 (Figure A.5).

- 4. Cathode ray tube: monochrome, green-screen, ASCII displays.

- 4. Cathode ray tube: màn hình ASCII đơn sắc, nền xanh lá.



Figure A.3 One platter of that disk: 3/8 inch thick, 36 inches in diameter

Figure A.3 Một đĩa từ của ổ đĩa đó: dày 3/8 inch, đường kính 36 inch

Courtesy, Ed Thelen, ed-thelen.org

Ảnh của Ed Thelen, ed-thelen.org

The CRT terminals ran at 30 characters per second. This was a pretty good rate for the late 1960s because modems were relatively unsophisticated in those days.

Các terminal CRT chạy ở tốc độ 30 ký tự mỗi giây. Đó là tốc độ khá tốt cho cuối thập niên 1960, bởi modem thời ấy còn tương đối thô sơ.

ASC leased a dozen or so dedicated phone lines and twice that number of 300 baud modems from the phone company to connect the Datanet 30 to these terminals.

ASC thuê của công ty điện thoại khoảng một tá đường dây riêng và số modem 300 baud gấp đôi con số đó để nối Datanet 30 với các terminal này.

These computers did not come with operating systems. They didn't even come with file systems. What you got was an assembler.

Những máy tính này không đi kèm hệ điều hành. Chúng thậm chí chẳng có cả hệ thống tập tin. Thứ bạn nhận được là một trình assembler.

If you needed to store data on the disk, you stored data on the disk. Not in a file. Not in a directory. You figured out which track, platter, and sector to put the data into, and then you operated the disk

to put the data there. Yes, that means we wrote our own disk driver.

Nếu bạn cần lưu dữ liệu lên đĩa, bạn lưu dữ liệu lên đĩa. Không phải vào một tập tin. Không phải vào một thư mục. Bạn tự tính xem sẽ đặt dữ liệu vào track nào, đĩa nào, sector nào, rồi tự điều khiển ổ đĩa để đặt dữ liệu vào đó. Đúng vậy, điều đó có nghĩa là chúng tôi tự viết driver đĩa của mình.



Figure A.4 Datapoint CRT terminal

Figure A.4 Terminal CRT Datapoint

Courtesy of Bill Degnan, vintagecomputer.net

Ảnh của Bill Degnan, vintagecomputer.net

The Union Accounting system had three kinds of records: Agents, Employers, and Members. The system was a CRUD system for these records, but also included operations for posting dues, computing changes in the general ledger, and so on.

Hệ thống kế toán công đoàn có ba loại bản ghi: Agents, Employers và Members. Hệ thống là một hệ CRUD trên các bản ghi này, nhưng cũng bao gồm các nghiệp vụ như ghi nhận

tiền hội phí, tính toán biến động trong sổ cái tổng hợp, v.v.

The original system was written in assembler by a consultant who somehow managed to cram the whole thing into 16K.

Hệ thống ban đầu do một chuyên gia tư vấn viết bằng assembler; bằng cách nào đó ông ta nhét gọn được toàn bộ thứ ấy vào 16K.

As you might imagine, that big Datanet 30 was an expensive machine to operate and maintain. The software consultant who kept the software

Như bạn có thể đoán, cỗ Datanet 30 to đùng ấy là một cỗ máy tốn kém để vận hành và bảo trì. Ông tư vấn phần mềm giữ cho phần mềm

running was expensive, too. What's more, minicomputers were becoming popular, and were much cheaper.

tiếp tục chạy cũng tốn kém không kém. Hơn nữa, máy tính mini đang trở nên phổ biến, mà lại rẻ hơn nhiều.



Figure A.5 ASR35 teletype

Figure A.5 Máy teletype ASR35

Joe Mabel, with permission

Joe Mabel, đã được cho phép sử dụng

In 1971, when I was 18 years old, ASC hired me and two of my geeky friends to replace the whole union accounting system with one that was based on a Varian 620/f minicomputer (Figure A.6). The

computer was cheap. We were cheap. So it seemed like a good deal for ASC.

Năm 1971, khi tôi 18 tuổi, ASC thuê tôi cùng hai người bạn một máy tính của tôi để thay toàn bộ hệ thống kế toán công đoàn bằng một hệ thống dựa trên máy tính mini Varian 620/f (Figure A.6). Máy tính thì rẻ. Chúng tôi cũng rẻ. Nên với ASC, đó có vẻ là một vụ hời.

The Varian machine had a 16-bit bus and 32K * 16 core memory. It had a cycle time of about 1 microsecond. It was much more powerful than the Datanet 30. It used IBM's wildly successful 2314 disk technology, allowing us to store 30 megabytes on platters that were only 14 inches in diameter and could not explode through concrete block walls!

Chiếc Varian có bus 16 bit và bộ nhớ lõi 32K * 16. Chu kỳ của nó khoảng 1 micro giây. Nó mạnh hơn Datanet 30 rất nhiều. Nó dùng công nghệ đĩa 2314 cực kỳ thành công của IBM, cho phép chúng tôi lưu 30 megabyte trên những đĩa từ chỉ có đường kính 14 inch và không thể xuyên thủng tường bê tông!

Of course, we still had no operating system. No file system. No **high-level** language. All we had was an assembler. But we made do.

Dĩ nhiên, chúng tôi vẫn không có hệ điều hành. Không hệ thống tập tin. Không ngôn ngữ mức cao. Tất cả những gì chúng tôi có là một trình assembler. Nhưng chúng tôi xoay sở được.

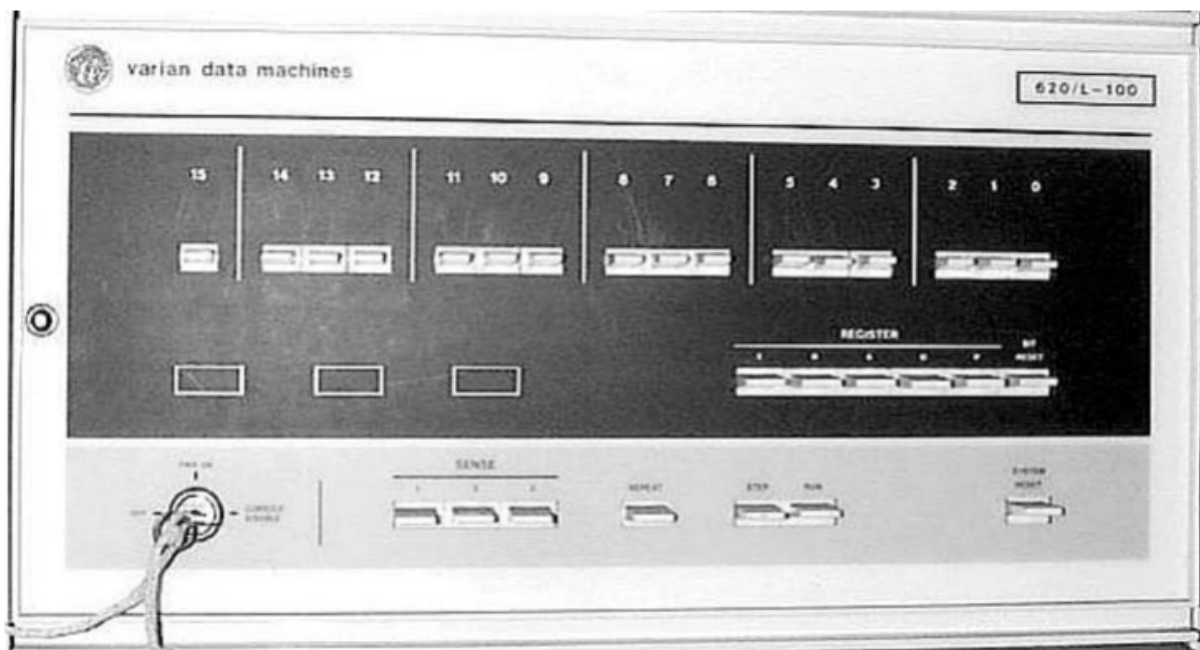


Figure A.6 Varian 620/f minicomputer

Figure A.6 Máy tính mini Varian 620/f

The Minicomputer Orphanage

Trại mồ côi máy tính mini

Instead of trying to cram the whole system into 32K, we created an overlay system. Applications would be loaded from disk into a block of memory dedicated to overlays. They would be executed

in that memory, and preemptively swapped back out onto disk, with their local RAM, to allow other programs to execute.

Thay vì cố nhét cả hệ thống vào 32K, chúng tôi tạo ra một hệ thống overlay. Các ứng dụng sẽ được nạp từ đĩa vào một vùng nhớ dành riêng cho overlay. Chúng được thực thi trong vùng nhớ ấy, rồi bị hoán đổi ra đĩa theo cơ chế trưng dụng, kèm theo cả vùng RAM cục bộ của mình, để nhường chỗ cho chương trình khác chạy.

Programs would get swapped into the overlay area, execute enough to fill the output buffers, and then get swapped out so that another program could be swapped in.

Các chương trình sẽ được hoán vào vùng overlay, chạy vừa đủ để lấp đầy các bộ đệm đầu ra, rồi bị hoán ra để một chương trình khác được hoán vào.

Of course, when your UI runs at 30 characters per second, your programs spend a lot of time waiting. We had plenty of time to swap the programs in and off the disk to keep all of the terminals running as fast as they could go. Nobody ever complained of response time issues.

Dĩ nhiên, khi giao diện người dùng của bạn chạy ở tốc độ 30 ký tự mỗi giây thì các chương trình dành phần lớn thời gian để chờ. Chúng tôi có thừa thời gian để hoán các chương trình ra vào đĩa nhằm giữ cho mọi terminal chạy nhanh hết mức có thể. Chẳng ai từng phàn nàn về thời gian phản hồi.

We wrote a preemptive supervisor that managed the interrupts and IO. We wrote the applications; we wrote the disk drivers, the terminal drivers, the tape drivers, and everything else in that system. There was not a single bit in that

Chúng tôi viết một bộ giám sát trưng dụng để quản lý ngắt và IO. Chúng tôi viết các ứng dụng; chúng tôi viết driver đĩa, driver terminal, driver băng từ, và mọi thứ khác trong hệ thống ấy. Không có lấy một bit nào trong

system that we did not write. Though it was a struggle involving far too many 80-hour weeks, we got the beast up and running in a matter of 8 or 9 months.

hệ thống đó mà chúng tôi không tự viết. Dù đó là một cuộc vật lộn với quá nhiều tuần làm việc 80 giờ, chúng tôi vẫn đưa được con quái vật ấy lên chạy trong vòng 8 hay 9 tháng.

The architecture of the system was simple (Figure A.7). When an application was started, it would generate output until its particular terminal buffer was full. Then the supervisor would swap the application out, and swap a new application in. The supervisor would continue to dribble out the contents of the terminal buffer at 30 cps until it was nearly empty. Then it would swap the application back in to fill the buffer again.

Kiến trúc của hệ thống rất đơn giản (Figure A.7). Khi một ứng dụng khởi chạy, nó sinh ra đầu ra cho tới khi bộ đệm terminal riêng của nó đầy. Rồi bộ giám sát sẽ hoán ứng dụng đó ra và hoán một ứng dụng mới vào. Bộ giám sát tiếp tục nhỏ giọt nội dung bộ đệm terminal ra ở tốc độ 30 cps cho tới khi bộ đệm gần cạn. Rồi nó hoán ứng dụng kia trở lại để lấp đầy bộ đệm lần nữa.

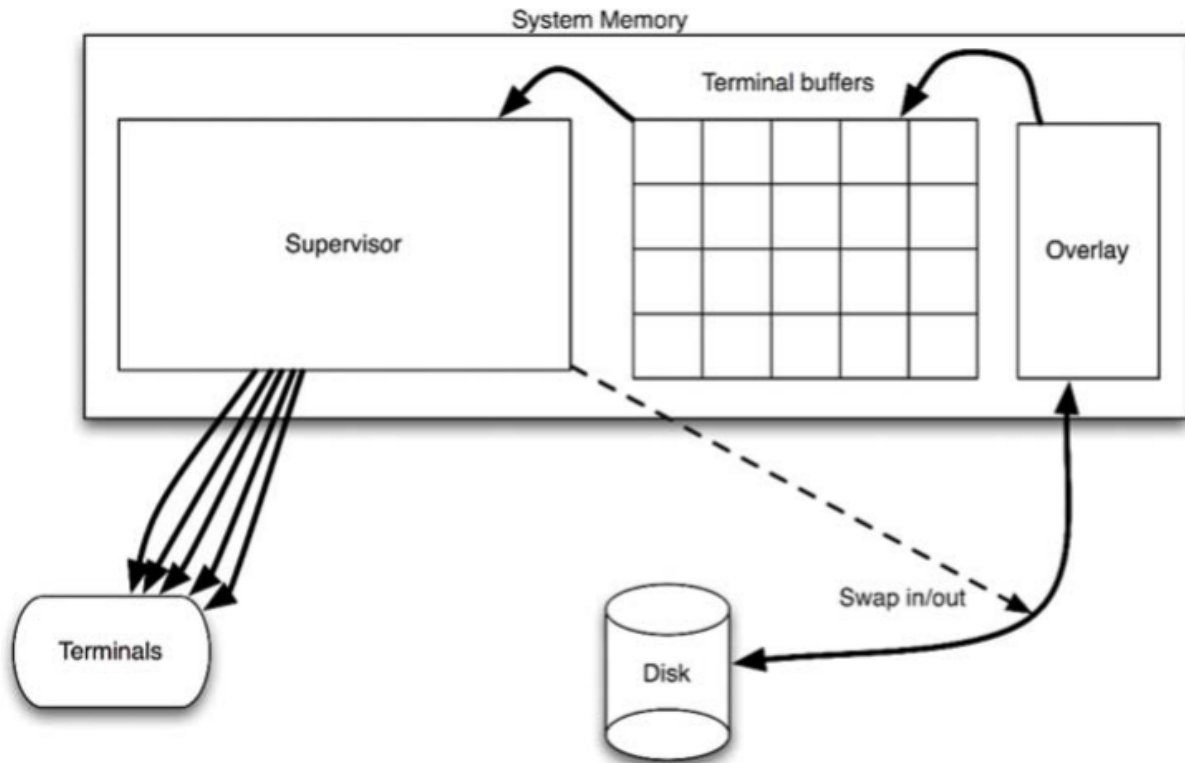


Figure A.7 The system architecture

Figure A.7 Kiến trúc của hệ thống

There are two boundaries in this system. The first is the character output **boundary**. The applications had no idea that their output was going to a 30-cps terminal. Indeed, the character output was entirely abstract from the applications' point of view. The applications simply passed strings to the supervisor, and the supervisor took care of loading the buffers, sending the characters to the terminals, and swapping the applications in and out of memory.

Hệ thống này có hai ranh giới (boundary). Ranh giới thứ nhất là ranh giới xuất ký tự. Các ứng dụng hoàn toàn không biết rằng đầu ra của chúng đang đi tới một terminal 30 cps. Thật vậy, việc xuất ký tự là hoàn toàn trừu tượng dưới góc nhìn của ứng dụng. Ứng dụng chỉ đơn giản truyền các chuỗi ký tự cho bộ giám sát, và bộ giám sát lo phần nạp bộ đệm, gửi ký tự tới terminal, cùng việc hoán ứng dụng ra vào bộ nhớ.

This boundary was dependency normal—that is, dependencies pointed with the **flow of control**. The applications had compile-time dependencies on the supervisor, and the flow of control passed from the applications to the supervisor. The boundary prevented the applications from knowing which kind of device the output was going to.

Ranh giới này có hướng phụ thuộc thuận — nghĩa là các phụ thuộc chỉ cùng chiều với luồng điều khiển. Các ứng dụng có phụ thuộc lúc biên dịch vào bộ giám sát, và luồng điều khiển đi từ ứng dụng sang bộ giám sát. Ranh giới ấy ngăn ứng dụng biết đầu ra của nó đang đi tới loại thiết bị nào.

The second boundary was dependency inverted. The supervisor could start the applications, but had no compile-time dependencies upon them. The flow of control passed from the supervisor to

the applications. The polymorphic **interface** that inverted the dependency was simply this: Every application was started by jumping to the exact same memory address within the overlay area. The boundary prevented the supervisor from knowing anything about the applications other than the starting point.

Ranh giới thứ hai thì bị đảo ngược phụ thuộc. Bộ giám sát có thể khởi chạy các ứng dụng, nhưng lại không có phụ thuộc lúc biên dịch vào chúng. Luồng điều khiển đi từ bộ giám sát sang ứng dụng. Interface đa hình (polymorphism) đã đảo ngược phụ thuộc ấy đơn giản là thế này: mọi ứng dụng đều được khởi chạy bằng cách nhảy tới đúng cùng một địa chỉ bộ nhớ bên trong vùng overlay. Ranh giới ấy ngăn bộ giám sát biết bất cứ điều gì về các ứng dụng ngoài điểm khởi đầu của chúng.

Laser Trim — Laser Trim

In 1973, I joined a company in Chicago called Teradyne Applied Systems (TAS). This was a division of Teradyne Inc., which was headquartered in Boston. Our product was a system that used relatively high-powered lasers to trim electronic components to very fine tolerances.

Năm 1973, tôi gia nhập một công ty ở Chicago tên là Teradyne Applied Systems (TAS). Đây là một bộ phận của Teradyne Inc., công ty có trụ sở chính ở Boston. Sản phẩm của chúng tôi là một hệ thống dùng laser công suất tương đối cao để tỉa các linh kiện điện tử về dung sai cực kỳ nhỏ.

Back in those days, manufacturers would silk-screen electronic components onto ceramic substrates. Those substrates were on the order of 1 inch square. The components were typically resistors—devices that resist the flow of current.

Thời ấy, các nhà sản xuất in lụa linh kiện điện tử lên đế gốm. Những tấm đế đó cỡ chừng 1 inch vuông. Linh kiện thường là điện trở — thiết bị cản trở dòng điện.

The resistance of a resistor depends on a number of factors, including its composition and its geometry. The wider the resistor, the less resistance it has.

Giá trị điện trở của một điện trở phụ thuộc vào nhiều yếu tố, trong đó có thành phần cấu tạo và hình dạng của nó. Điện trở càng rộng bản thì giá trị điện trở càng nhỏ.

Our system would position the ceramic substrate in a harness that had probes that made contact with the resistors. The system would measure the resistance of the resistors, and then use a laser to burn off parts of the resistor, making it thinner and thinner until it reached the desired resistance value within a tenth of a percent or so.

Hệ thống của chúng tôi sẽ định vị tấm đế gốm trong một bộ gá có các đầu dò tiếp xúc với các điện trở. Hệ thống đo giá trị điện trở của các điện trở, rồi dùng laser đốt bớt từng phần của điện trở, làm nó mảnh dần đi cho tới khi đạt giá trị điện trở mong muốn với sai số cỡ một phần mười phần trăm.

We sold these systems to manufacturers. We also used some in-house systems to trim relatively small batches for small manufacturers.

Chúng tôi bán những hệ thống này cho các nhà sản xuất. Chúng tôi cũng dùng vài hệ thống nội bộ để tỉa những lô tương đối nhỏ cho các nhà sản xuất nhỏ.

The computer was an M365. This was in the days when many companies built their own computers: Teradyne built the M365 and supplied it to all its divisions. The M365 was an enhanced version of a PDP-8—a popular minicomputer of the day.

Máy tính là một chiếc M365. Đó là thời mà nhiều công ty tự chế tạo máy tính riêng: Teradyne chế ra M365 và cung cấp nó cho tất cả các bộ phận của mình. M365 là một phiên bản nâng cấp của PDP-8 — chiếc máy tính mini phổ biến thời đó.

The M365 controlled the positioning table, which moved the ceramic substrates under the probes. It controlled the measurement system and the laser. The laser was positioned using X - Y mirrors that could rotate under program control. The computer could also control the power setting of the laser.

M365 điều khiển bàn định vị, thứ dịch chuyển các tấm đế gốm bên dưới đầu dò. Nó điều khiển hệ thống đo và tia laser. Laser được định vị bằng các gương X - Y có thể xoay dưới sự điều khiển của chương trình. Máy tính cũng có thể điều khiển mức công suất của laser.

The development environment of the M365 was relatively primitive. There was no disk. Mass storage was on tape cartridges that looked like old 8-track audio tape cassettes. The tapes and drives were made by Tri-Data.

Môi trường phát triển của M365 khá thô sơ. Không có đĩa. Bộ nhớ khối lượng lớn nằm trên các băng cassette trông giống mấy cuộn băng cassette âm thanh 8-track thời xưa. Băng và ổ đọc do Tri-Data sản xuất.

Like the 8-track audio cassettes of the day, the tape was oriented in a loop. The drive moved the tape in only one direction—there was no rewind! If you wanted to position the tape at the beginning, you had to send it forward to its “load point.”

Giống như băng cassette 8-track thời đó, dải băng được xếp thành một vòng khép kín. Ổ băng chỉ chạy băng theo một chiều duy nhất — không có tua ngược! Nếu bạn muốn đưa băng về đầu, bạn phải cho nó chạy tới cho tới khi gặp “load point”.

The tape moved at a speed of approximately 1 foot per second. Thus, if the tape loop was 25 feet long, it could take as long as 25 seconds to send it to the load point. For this reason Tridata made cartridges in several lengths, ranging from 10 feet to 100 feet.

Băng chạy ở tốc độ chừng 1 foot mỗi giây. Vậy nên, nếu vòng băng dài 25 foot, có thể mất tới 25 giây để đưa nó về load point. Vì lý do đó, Tridata làm ra các cuộn băng nhiều độ dài khác nhau, từ 10 foot tới 100 foot.

The M365 had a button on the front that would load memory with a little bootstrap program and execute it. This program would read the first block of data from the tape, and execute that. Typically this block held a **loader** that loaded the operating system that lived on the rest of the tape.

M365 có một nút bấm ở mặt trước, nạp vào bộ nhớ một chương trình bootstrap nhỏ và chạy nó. Chương trình này sẽ đọc khối dữ liệu đầu tiên từ băng và thực thi khối đó. Thông thường khối này chứa một loader, và loader nạp hệ điều hành nằm ở phần còn lại của cuộn băng.

The operating system would prompt the user for the name of a program to run. Those programs were stored on the tape, just after the operating system. We would type in the name of the program—for example, the ED-402

Hệ điều hành sẽ hỏi người dùng tên chương trình cần chạy. Những chương trình ấy được lưu trên băng, ngay sau hệ điều hành. Chúng tôi gõ vào tên chương trình — chẳng hạn ED-402

Editor—and the operating system would search the tape for that program, load it, and execute it.

Editor — rồi hệ điều hành sẽ dò tìm chương trình đó trên băng, nạp nó và thực thi.

The console was an ASCII CRT with green phosphors, 72 characters wide 5 by 24 lines. The characters were all uppercase.

Console là một màn hình CRT ASCII lân quang xanh lá, rộng 72 ký tự 5 và cao 24 dòng. Ký tự toàn bộ là chữ hoa.

To edit a program, you would load the ED-402 Editor, and then insert the tape that held your source code. You would read one tape block of that source code into memory, and it would be displayed on the screen. The tape block might hold 50 lines of code. You would make your edits by moving the cursor around on the screen and typing in a manner similar to vi. When you were done, you would write that block onto a different tape, and read the next block from the source tape. You kept on doing this until you were done.

Để sửa một chương trình, bạn nạp ED-402 Editor, rồi lắp cuộn băng chứa mã nguồn của mình vào. Bạn đọc một khối băng của mã nguồn ấy vào bộ nhớ, và nó sẽ hiện lên màn hình. Khối băng đó có thể chứa 50 dòng mã. Bạn thực hiện các chỉnh sửa bằng cách di con trỏ quanh màn hình và gõ theo kiểu tương tự như vi. Xong xuôi, bạn ghi khối đó lên một cuộn băng khác, rồi đọc khối tiếp theo từ cuộn băng nguồn. Bạn cứ làm vậy cho tới khi hết.

There was no scrolling back to previous blocks. You edited your program in a straight line, from beginning to end. Going back to the beginning forced you to finish copying the source code onto the output tape and then start a new editing session on that tape. Perhaps not surprisingly, given these constraints, we printed our programs out on paper, marked all the edits by hand in red ink, and then edited our programs block by block by consulting our markups on the listing.

Không có chuyện cuộn ngược lại các khối trước đó. Bạn sửa chương trình theo một đường thẳng, từ đầu tới cuối. Muốn quay lại đầu thì bạn buộc phải chép nốt phần mã nguồn còn lại sang cuộn băng đầu ra, rồi mở một phiên soạn thảo mới trên cuộn băng đó. Với những ràng buộc như thế thì cũng chẳng có gì lạ: chúng tôi in chương trình ra giấy, đánh dấu mọi chỗ cần sửa bằng bút mực đỏ, rồi sửa chương trình theo từng khối bằng cách dò lại các ghi chú trên bản in.

Once the program was edited, we returned to the OS and invoked the assembler. The assembler read the source code tape, and wrote a binary tape, while also producing a listing on our data products line printer.

Khi chương trình đã được sửa xong, chúng tôi quay về hệ điều hành và gọi trình assembler. Trình assembler đọc cuộn băng mã nguồn và ghi ra một cuộn băng nhị phân, đồng thời in ra một bản liệt kê trên máy in dòng Data Products của chúng tôi.

The tapes weren't 100% reliable, so we always wrote two tapes at the same time. That way, at least one of them had a high probability of being free of errors.

Băng từ không đáng tin cậy 100%, nên chúng tôi luôn ghi ra hai cuộn cùng lúc. Như vậy, ít nhất một trong hai cuộn có xác suất cao là không lỗi.

Our program was approximately 20,000 lines of code, and took nearly 30 minutes to compile. The odds that we would get a tape read error during that time were roughly 1 in 10. If the assembler got a tape error, it would ring the

Chương trình của chúng tôi dài khoảng 20.000 dòng mã, và mất gần 30 phút để biên dịch. Xác suất gặp lỗi đọc băng trong khoảng thời gian đó vào cỡ 1 trên 10. Nếu trình assembler gặp lỗi băng, nó sẽ rung

- 5. The magic number 72 came from Hollerith punched cards, which held 80 characters each. The last 8

- 5. Con số kỳ diệu 72 đến từ thẻ đục lỗ Hollerith, mỗi thẻ chứa 80 ký tự. 8 ký tự cuối characters were “reserved” for sequence numbers in case you dropped the deck.

được “để dành” cho số thứ tự, phòng khi bạn làm rơi cả cỗ thẻ.

bell on the console and then start printing a stream of errors on the printer. You could hear this maddening bell all across the lab. You could also hear the cursing of the poor programmer who just learned that the 30-minute compile needed to start over.

chuông trên console rồi bắt đầu in ra một tràng lỗi trên máy in. Bạn có thể nghe thấy tiếng chuông phát điên ấy khắp phòng lab. Bạn cũng có thể nghe thấy tiếng chửi thề của gã lập trình viên khốn khổ vừa mới nhận ra rằng ca biên dịch 30 phút phải làm lại từ đầu.

The architecture of the program was typical for those days. There was a Master Operating Program, appropriately called “the MOP.” Its job was to manage basic IO functions and provide the rudiments of a console “shell.” Many of the divisions of Teradyne shared the MOP source code, but each had forked it for its own uses. Consequently, we would send source code updates around to each other in the form of marked-up listings that we would then integrate manually (and very carefully).

Kiến trúc của chương trình rất điển hình cho thời đó. Có một Master Operating Program, được gọi rất đúng tên là “MOP”. Việc của nó là quản lý các chức năng IO cơ bản và cung cấp những thứ sơ đẳng của một “shell” trên console. Nhiều bộ phận của Teradyne cùng dùng chung mã nguồn MOP, nhưng mỗi nơi đã rẽ nhánh nó cho mục đích riêng. Hệ quả là chúng tôi gửi các bản cập nhật mã nguồn cho nhau dưới dạng bản liệt kê có ghi chú, rồi tự tay tích hợp lại (và rất cẩn thận).

A special-purpose utility layer controlled the measurement hardware, the positioning tables, and the laser. The boundary between this layer and the MOP was muddled at best. While the utility layer called the MOP, the MOP had been specifically modified for that layer, and often called back into it. Indeed, we didn’t really think of these two as separate layers. To us, it was just some code that we added to the MOP in a highly coupled way.

Một tầng tiện ích chuyên dụng điều khiển phần cứng đo đạc, các bàn định vị và tia laser. Ranh giới giữa tầng này và MOP, nói nhẹ nhất, là mù mờ. Tầng tiện ích gọi vào MOP, nhưng MOP lại được sửa riêng cho tầng ấy và thường gọi ngược trở lại nó. Thật ra, chúng tôi chẳng thực sự nghĩ về hai thứ này như hai tầng tách biệt. Với chúng tôi, đó chỉ là ít mã mà chúng tôi thêm vào MOP theo cách kết dính chặt chẽ.

Next came the isolation layer. This layer provided a virtual machine interface for the application programs, which were written in a completely different domain-specific data-driven language (DSL). The language had operations for moving the laser, moving the table, making cuts, making measurements, and so on. Our customers would write their laser trimming application programs in this language, and the isolation layer would execute them.

Kế đến là tầng cách ly. Tầng này cung cấp một interface máy ảo cho các chương trình ứng dụng, vốn được viết bằng một ngôn ngữ chuyên biệt theo miền, hướng dữ liệu, hoàn toàn khác biệt (DSL). Ngôn ngữ này có các lệnh để dịch chuyển laser, dịch chuyển bàn, cắt, đo đạc, v.v. Khách hàng của chúng tôi sẽ viết chương trình ứng dụng tia laser của họ bằng ngôn ngữ này, và tầng cách ly sẽ thực thi chúng.

This approach was not intended to create a machine-independent laser trim language. Indeed, the language had many idiosyncrasies that were deeply coupled to the layers below. Rather, this approach gave the application programmers a “simpler” language than M356 assembler in which to program their trim jobs.

Cách làm này không nhằm tạo ra một ngôn ngữ tia laser độc lập với máy. Thật ra, ngôn ngữ ấy có vô số nét đặc thù kết dính rất sâu với các tầng bên dưới. Đúng hơn, cách làm này cho lập trình viên ứng dụng một ngôn ngữ “đơn giản hơn” so với assembler M356 để viết các tác vụ tia của họ.

Trim jobs could be loaded from tape and executed by the system. Essentially, our system was an operating system for trim applications.

Các tác vụ tia có thể được nạp từ băng và thực thi bởi hệ thống. Về bản chất, hệ thống của chúng tôi là một hệ điều hành cho các ứng dụng tia.

The system was written in M365 assembler and compiled in a single compilation unit that produced absolute binary code.

Hệ thống được viết bằng assembler M365 và biên dịch trong một đơn vị biên dịch duy nhất, sinh ra mã nhị phân tuyệt đối.

The boundaries in this application were soft at best. Even the boundary between the system code and the applications written in the DSL was not well enforced. There were couplings everywhere.

Các ranh giới trong ứng dụng này, nói nhẹ nhất, là mềm oặt. Ngay cả ranh giới giữa mã hệ thống và các ứng dụng viết bằng DSL cũng không được thực thi cho ra hồn. Kết dính có ở khắp nơi.

But that was typical of software in the early 1970s.

Nhưng đó là chuyện điển hình của phần mềm đầu thập niên 1970.

Aluminum Die-Cast Monitoring — Giám sát đúc khuôn nhôm

In the middle of the 1970s, while OPEC was placing an embargo on oil, and gasoline shortages were causing angry drivers to get into fights at gas stations, I began working at Outboard Marine Corporation (OMC). This is the parent company of Johnson Motors and Lawnboy lawnmowers.

Giữa thập niên 1970, khi OPEC đang cấm vận dầu mỏ và tình trạng thiếu xăng khiến những tài xế nổi giận đánh nhau ở cây xăng, tôi bắt đầu làm việc tại Outboard Marine Corporation (OMC). Đây là công ty mẹ của Johnson Motors và máy cắt cỏ Lawnboy.

OMC maintained a huge facility in Waukegan, Illinois, for creating die-cast aluminum parts for all of the company's motors and products. Aluminum was melted down in huge furnaces, and then carried in large buckets to dozens upon dozens of individually operated aluminum die-cast machines. Each machine had a human operator responsible for setting the molds, cycling the machine, and extracting the newly cast parts. These operators were paid based on how many parts they produced.

OMC vận hành một cơ sở khổng lồ ở Waukegan, Illinois, để đúc khuôn các chi tiết nhôm cho toàn bộ động cơ và sản phẩm của công ty. Nhôm được nấu chảy trong những lò khổng lồ, rồi được múc bằng những gầu lớn mang tới hàng mấy chục cỗ máy đúc khuôn nhôm, mỗi cỗ do một người vận hành riêng. Mỗi máy có một thợ vận hành chịu trách nhiệm lắp khuôn, cho máy chạy hết chu kỳ và lấy chi tiết vừa đúc ra. Những người thợ này được trả lương theo số chi tiết họ làm ra.

I was hired to work on a shop-floor automation project. OMC had purchased an IBM System/7—which was IBM's answer to the minicomputer. They tied this computer to all the die-cast machines on the floor, so that we could count, and time, the cycles of each machine. Our role was to gather all that information and present it on 3270 green-screen displays.

Tôi được thuê để làm một dự án tự động hóa xưởng sản xuất. OMC đã mua một chiếc IBM System/7 — câu trả lời của IBM cho dòng máy tính mini. Họ nối chiếc máy tính này với tất cả các máy đúc khuôn trong xưởng, để chúng tôi có thể đếm và bấm giờ chu kỳ của từng máy. Vai trò của chúng tôi là thu thập toàn bộ thông tin đó và trình bày nó trên các màn hình 3270 nền xanh lá.

The language was assembler. And, again, every bit of code that executed in this computer was code that we wrote. There was no operating system, no subroutine libraries, and no **framework**. It was just raw code.

Ngôn ngữ là assembler. Và, một lần nữa, mọi bit mã chạy trên chiếc máy tính này đều là mã do chúng tôi viết. Không hệ điều hành, không thư viện chương trình con, không framework. Chỉ có mã thô.

It was also interrupt-driven real-time code. Every time a die-cast machine cycled, we had to update a batch of statistics, and send messages to a great IBM 370 in-the-sky, running a CICS-COBOL program that presented those statistics on the green screens.

Đó cũng là mã thời gian thực, chạy theo ngắt. Cứ mỗi lần một máy đúc khuôn hoàn tất chu kỳ, chúng tôi phải cập nhật một loạt số liệu thống kê và gửi thông điệp lên một cỗ IBM 370 trên chín tầng mây, nơi chạy một chương trình CICS-COBOL trình bày các số liệu ấy trên những màn hình xanh lá.

I hated this job. Oh, boy, did I. Oh, the work was fun ! But the culture ... Suffice it to say that I was required to wear a tie.

Tôi ghét công việc này. Ôi trời, ghét thật sự. Ồ, bản thân công việc thì vui ! Nhưng cái văn hóa ở đó... Chỉ cần nói thế này là đủ: tôi bị bắt buộc phải đeo cà vạt.

Oh, I tried. I really did. But I was clearly unhappy working there, and my colleagues knew it. They knew it because I couldn't remember critical dates or manage to get up early enough to attend **important** meetings. This was the only programming job I was ever fired from—and I deserved it.

Ồ, tôi đã cố. Cố thật đấy. Nhưng rõ ràng là tôi không hạnh phúc khi làm việc ở đó, và đồng nghiệp của tôi biết điều đó. Họ biết vì tôi không nhớ nổi những mốc thời gian quan trọng, cũng không dậy nổi đủ sớm để dự các cuộc họp quan trọng. Đây là công việc lập trình duy nhất mà tôi từng bị đuổi — và tôi đáng bị như vậy.

From an architectural point of view, there's not a lot to learn here except for one thing. The System/7 had a very interesting instruction called set program interrupt (SPI). This allowed you to trigger an interrupt of the processor, allowing it to handle any other queued lower-priority interrupts. Nowadays, in Java we call this Thread.yield() .

Ở góc nhìn kiến trúc, chẳng có mấy điều để học ở đây ngoài một thứ. System/7 có một lệnh rất thú vị tên là set program interrupt (SPI). Lệnh này cho phép bạn kích hoạt một ngắt của bộ xử lý, để nó xử lý bất kỳ ngắt nào có độ ưu tiên thấp hơn đang xếp hàng. Ngày nay, trong Java, chúng ta gọi thứ đó là Thread.yield() .

4-TEL — 4-TEL

In October 1976, having been fired from OMC, I returned to a different division of Teradyne—a division I would stay with for 12 years. The product I worked on was named 4-TEL. Its purpose was to test every telephone line in a telephone **service** area, every night, and produce a report of all lines requiring repair. It also allowed telephone test personnel to test specific telephone lines in **detail**.

Tháng 10 năm 1976, sau khi bị sa thải khỏi OMC, tôi quay lại Teradyne nhưng ở một bộ phận khác — bộ phận mà tôi sẽ gắn bó suốt 12 năm. Sản phẩm tôi làm có tên 4-TEL. Mục đích của nó là kiểm tra mọi đường dây điện thoại trong một vùng dịch vụ điện thoại, mỗi đêm, rồi xuất ra báo cáo liệt kê tất cả các đường dây cần sửa chữa. Nó cũng cho phép nhân viên kiểm tra đường dây điện thoại soi kỹ từng đường dây cụ thể.

This system started its life with the same kind of architecture as the Laser Trim system. It was a monolithic application written in assembly language without any significant boundaries. But at the time I joined the company, that was about to change.

Hệ thống này khởi đầu với kiểu kiến trúc (architecture) y hệt hệ thống Laser Trim. Đó là một ứng dụng nguyên khối viết bằng hợp ngữ, không có ranh giới (boundary) đáng kể nào. Nhưng đúng vào lúc tôi gia nhập công ty, điều đó sắp thay đổi.

The system was used by testers located in a service center (SC). A service center covered many central offices (CO), each of which could handle as

Hệ thống được các kỹ thuật viên kiểm tra sử dụng, họ ngồi tại một service center (SC). Mỗi service center phụ trách nhiều central office (CO), và mỗi CO có thể xử lý tới

many as 10,000 phone lines. The dialing and measurement hardware had to be located inside the CO. So that's where the M365 computers were put. We called those computers the central office line testers (COLTs). Another M365 was placed at the SC; it was called the service area computer (SAC). The SAC had several modems that it could use to dial up the COLTs and communicate at 300 baud (30 cps).

10.000 đường dây điện thoại. Phần cứng quay số và đo đặc bắt buộc phải đặt bên trong CO. Nên đó chính là nơi các máy tính M365 được đặt. Chúng tôi gọi những máy tính đó là central office line tester (COLT). Một chiếc M365 khác được đặt tại SC; nó được gọi là service area computer (SAC). SAC có vài modem để quay số tới các COLT và trao đổi dữ liệu ở tốc độ 300 baud (30 ký tự/giây).

At first, the COLT computers did everything, including all the console communication, menus, and reports. The SAC was just a simple multiplexor that took the output from the COLTs and put it on a screen.

Ban đầu, các máy COLT làm mọi thứ, bao gồm toàn bộ phần giao tiếp với console, menu và báo cáo. SAC chỉ là một bộ ghép kênh đơn giản, lấy đầu ra từ các COLT rồi đẩy lên màn hình.

The problem with this setup was that 30 cps is really slow. The testers didn't like watching the characters trickle across the screen, especially since they were only interested in a few key bits of data. Also, in those days, the core memory in the M365 was expensive, and the program was big.

Vấn đề của cách bố trí này là 30 ký tự/giây thì chậm khủng khiếp. Kỹ thuật viên chẳng thích thú gì khi phải ngồi nhìn từng ký tự nhỏ giọt bò ngang màn hình, nhất là khi họ chỉ quan tâm tới vài mẫu dữ liệu then chốt. Ngoài ra, thời đó bộ nhớ lõi (core memory) trên M365 rất đắt, mà chương trình thì lớn.

The solution was to separate the part of the software that dialed and measured lines from the part that analyzed the results and printed the reports. The latter would be moved into the SAC, and the former would remain behind in the COLTs. This would allow the COLT to be a smaller machine, with much less memory, and would greatly speed up the response at the terminal, since the reports would be generated in the SAC.

Giải pháp là tách phần phần mềm lo việc quay số và đo đường dây ra khỏi phần lo việc phân tích kết quả và in báo cáo. Phần sau sẽ được chuyển vào SAC, phần đầu vẫn ở lại

trong các COLT. Nhờ vậy COLT có thể là một cỗ máy nhỏ hơn, ít bộ nhớ hơn nhiều, đồng thời tốc độ phản hồi tại terminal tăng vọt vì báo cáo được sinh ra ngay tại SAC.

The result was remarkably successful. Screen updates were very fast (once the appropriate COLT had been dialed), and the memory footprint of the COLTs shrank a lot.

Kết quả thành công đến mức đáng nể. Màn hình cập nhật rất nhanh (một khi đã quay số tới đúng COLT), và dung lượng bộ nhớ mà COLT chiếm dụng giảm đi rất nhiều.

The boundary was very clean and highly decoupled. Very short packets of data were exchanged between the SAC and COLT. These packets were a very simple form of DSL, representing primitive commands like “DIAL XXXX” or “MEASURE.”

Ranh giới đó rất sạch sẽ và tách rời (decoupling) ở mức cao. Chỉ những gói dữ liệu cực ngắn được trao đổi giữa SAC và COLT. Các gói này là một dạng DSL hết sức đơn giản, biểu diễn những lệnh nguyên thủy như “DIAL XXXX” hay “MEASURE.”

The M365 was loaded from tape. Those tape drives were expensive and weren't very reliable—especially in the industrial environment of a telephone central office. Also, the M365 was an expensive machine relative to the rest of

M365 nạp chương trình từ băng từ. Những ổ băng đó vừa đắt vừa không đáng tin cậy lắm—nhất là trong môi trường công nghiệp của một tổng đài điện thoại. Hơn nữa, M365 là cỗ máy đắt đỏ so với phần còn lại của

the electronics within the COLT. So we embarked upon a project to replace the M365 with a microcomputer based on an 8085 μ processor.

đám linh kiện điện tử bên trong COLT. Thế là chúng tôi bắt tay vào một dự án thay M365 bằng một máy vi tính dựa trên vi xử lý 8085.

The new computer was composed of a processor board that held the 8085, a RAM board that held 32K of RAM, and three ROM boards that held 12K of read-only memory apiece. All these boards fit into the same chassis as the measurement hardware, thereby eliminating the bulky extra chassis that had housed the M365.

Cỗ máy mới gồm một bo mạch xử lý chứa con 8085, một bo RAM chứa 32K RAM, và ba bo ROM, mỗi bo chứa 12K bộ nhớ chỉ đọc. Tất cả những bo mạch này vừa khít vào cùng khung máy với phần cứng đo đạc, nhờ đó loại bỏ được cái khung phụ cồng kềnh vốn dùng để chứa M365.

The ROM boards held 12 Intel 2708 EPROM (Erasable Programmable Read- Only Memory) chips. 6 Figure A.8 shows an example of such a chip. We loaded those chips with software by inserting them into special devices called PROM burners that were driven by our development environment. The chips could be erased by exposing them to high-intensity ultraviolet light. 7

Mỗi bo ROM mang 12 con chip Intel 2708 EPROM (Erasable Programmable Read-Only Memory). 6 Figure A.8 cho thấy một ví dụ về loại chip này. Chúng tôi nạp phần mềm vào những con chip đó bằng cách cắm chúng vào các thiết bị đặc biệt gọi là PROM burner, do môi trường phát triển của chúng tôi điều khiển. Có thể xóa chip bằng cách phơi nó dưới ánh sáng cực tím cường độ cao. 7

My buddy CK and I translated the M365 assembly language program for the COLT into 8085 assembly language. This translation was done by hand and took us about 6 months. The end result was approximately 30K of 8085 code.

Tôi và ông bạn CK đã dịch chương trình hợp ngữ M365 của COLT sang hợp ngữ 8085. Việc chuyển đổi này làm bằng tay và ngón của chúng tôi khoảng 6 tháng. Kết quả cuối cùng là

chừng 30K mã 8085.

Our development environment had 64K of RAM and no ROM, so we could quickly download our compiled binaries into RAM and test them.

Môi trường phát triển của chúng tôi có 64K RAM và không có ROM, nên chúng tôi có thể nhanh chóng nạp các file nhị phân đã biên dịch vào RAM để chạy thử.

Once we got the program working, we switched to using the EPROMs. We burned 30 chips and inserted them into just the right slots in the three ROM boards. Each chip was labeled so we could tell which chip went into which slot.

Khi chương trình đã chạy được, chúng tôi chuyển sang dùng EPROM. Chúng tôi nạp 30 con chip rồi cắm chúng vào đúng khe của ba bo ROM. Mỗi chip đều được dán nhãn để biết chip nào thuộc khe nào.

The 30K program was a single binary, 30K long. To burn the chips, we simply divided that binary image into 30 different 1K segments, and burned each segment onto the appropriately labeled chip.

Chương trình 30K là một file nhị phân duy nhất, dài 30K. Để nạp chip, chúng tôi chỉ việc chia ảnh nhị phân đó thành 30 đoạn 1K khác nhau, rồi nạp từng đoạn vào con chip mang nhãn tương ứng.

- 6. Yes, I understand that's an oxymoron.
 - 6. Vâng, tôi hiểu rằng đó là một cách nói tự mâu thuẫn.
- 7. They had a little clear plastic window that allowed you to see the silicon chip inside, and allowed the UV
 - 7. Chúng có một ô cửa sổ nhựa trong nhỏ xíu cho phép bạn nhìn thấy con chip silicon bên trong, và cho tia UV lọt vào

to erase the data.

để xóa dữ liệu.



Figure A.8 EPROM chip

Figure A.8 Chip EPROM

This worked very well, and we began to mass-produce the hardware and deploy the system into the field.

Cách này chạy rất tốt, và chúng tôi bắt đầu sản xuất hàng loạt phần cứng rồi triển khai hệ thống ra thực địa.

But software is soft. 8 Features needed to be added. Bugs needed to be fixed. And as the installed base grew, the logistics of updating the software by burning 30 chips per installation, and having field service people replace all 30 chips at each site became a nightmare.

Nhưng phần mềm thì mềm. 8 Có tính năng cần thêm. Có lỗi cần sửa. Và khi số lượng máy đã lắp đặt tăng lên, việc hậu cần để cập nhật phần mềm — nạp 30 con chip cho mỗi bản cài, rồi cử người dịch vụ hiện trường đi thay cả 30 con chip ở từng địa điểm — trở thành một cơn ác mộng.

There were all kinds of problems. Sometimes chips would be mislabeled, or the labels would fall off. Sometimes the field service engineer would mistakenly replace the wrong chip. Sometimes the field service engineer would

Đủ mọi kiểu trục trặc xảy ra. Có lúc chip bị dán nhầm nhãn, hoặc nhãn bong ra. Có lúc kỹ sư dịch vụ hiện trường thay nhầm chip. Có lúc kỹ sư dịch vụ hiện trường lại

- 8. Yes, I know that when software is burned into ROM, it's called **firmware**—but even firmware is
 - 8. Vâng, tôi biết khi phần mềm được nạp vào ROM thì người ta gọi nó là firmware — nhưng ngay cả firmware cũng

really still soft.

vẫn thực sự mềm.

inadvertently break a pin off one of the new chips. Consequently, the field engineers had to carry extras of all 30 chips with them.

vô ý làm gãy mất một chân của con chip mới. Hệ quả là các kỹ sư hiện trường phải mang theo chip dự phòng cho đủ cả 30 vị trí.

Why did we have to change all 30 chips? Every time we added or removed code from our 30K executable, it changed the addresses in which each instruction was loaded. It also changed the addresses of the subroutines and functions that we called. So every chip was affected, no matter how trivial the change.

Tại sao chúng tôi phải thay cả 30 con chip? Bởi mỗi lần thêm hay bớt code trong file thực thi 30K, địa chỉ nạp của từng lệnh đều đổi. Địa chỉ của các subroutine và hàm mà chúng tôi gọi cũng đổi theo. Thế nên mọi con chip đều bị ảnh hưởng, dù thay đổi có nhỏ nhất đến đâu.

One day, my boss came to me and asked me to solve that problem. He said we needed a way to make a change to the firmware without replacing all 30 chips every time. We brainstormed this issue for a while, and then embarked upon the “Vectorization” project. It took me three months.

Một hôm, sếp tôi tới gặp và bảo tôi hãy giải quyết vấn đề đó. Ông nói chúng ta cần một cách để sửa firmware mà không phải thay cả 30 con chip mỗi lần. Chúng tôi động não một hồi, rồi khởi động dự án “Vectorization”. Nó ngốn của tôi ba tháng.

The idea was beautifully simple. We divided the 30K program into 32 independently compilable source files, each less than 1K. At the beginning of each source file, we told the compiler in which address to load the resulting program (e.g., ORG C400 for the chip that was to be inserted into the C4 position).

Ý tưởng đơn giản một cách đẹp đẽ. Chúng tôi chia chương trình 30K thành 32 file mã nguồn biên dịch được độc lập, mỗi file dưới 1K. Ở đầu mỗi file mã nguồn, chúng tôi báo cho trình biên dịch biết địa chỉ nạp chương trình kết quả (ví dụ ORG C400 cho con chip sẽ cắm vào vị trí C4).

Also at the beginning of each source file, we created a simple, fixed-size data **structure** that contained all the addresses of all the subroutines on that chip. This data structure was 40 bytes long, so it could hold no more than 20 addresses. This meant that no chip could have more than 20 subroutines.

Cũng ở đầu mỗi file mã nguồn, chúng tôi tạo một cấu trúc dữ liệu đơn giản, kích thước cố định, chứa toàn bộ địa chỉ của tất cả subroutine nằm trên con chip đó. Cấu trúc dữ liệu này dài 40 byte, nên chứa được nhiều nhất 20 địa chỉ. Điều đó có nghĩa là không con chip nào được phép có quá 20 subroutine.

Next, we created a special area in RAM known as the vectors. It contained 32 tables of 40 bytes—exactly enough RAM to hold the pointers at the start of each chip.

Tiếp theo, chúng tôi dành ra một vùng đặc biệt trong RAM gọi là các vector. Nó chứa 32 bảng, mỗi bảng 40 byte — vừa đủ RAM để giữ các con trỏ đặt ở đầu từng con chip.

Finally, we changed every call to every subroutine on every chip into an indirect call through the appropriate RAM vector.

Cuối cùng, chúng tôi đổi mọi lời gọi tới mọi subroutine trên mọi con chip thành lời gọi gián tiếp thông qua vector RAM tương ứng.

When our processor booted, it would scan each chip and load the vector table at the start of each chip into the RAM vectors. Then it would jump into the main program.

Khi bộ xử lý khởi động, nó quét từng con chip và nạp bảng vector ở đầu mỗi chip vào các vector trong RAM. Rồi nó nhảy vào chương trình chính.

This worked very well. Now, when we fixed a bug, or added a feature, we could simply recompile one or two chips, and send just those chips to the field service engineers.

Cách này chạy rất tốt. Giờ đây, khi sửa một lỗi hay thêm một tính năng, chúng tôi chỉ cần biên dịch lại một hoặc hai con chip, rồi gửi đúng những con chip đó cho các kỹ sư dịch vụ hiện trường.

We had made the chips independently deployable . We had invented polymorphic dispatch. We had invented objects.

Chúng tôi đã làm cho các con chip có khả năng triển khai độc lập. Chúng tôi đã phát minh ra polymorphic dispatch. Chúng tôi đã phát minh ra object.

This was a **plugin architecture**, quite literally. We plugged those chips in. We eventually engineered it so that a feature could be installed into our products by plugging the chip with that feature into one of the open chip sockets. The menu control would automatically appear, and the binding into the main application would happen automatically.

Đây là kiến trúc plugin, theo đúng nghĩa đen. Chúng tôi cắm những con chip đó vào. Về sau chúng tôi còn thiết kế sao cho một tính năng có thể được cài vào sản phẩm chỉ bằng cách cắm con chip mang tính năng ấy vào một trong các ổ chip còn trống. Mục điều khiển trên menu sẽ tự động hiện ra, và việc gắn kết vào ứng dụng chính diễn ra tự động.

Of course, we didn't know about object-oriented principles at the time, and we knew nothing about separating user interface from business rules. But the rudiments were there, and they were very powerful.

Dĩ nhiên thời đó chúng tôi chưa biết gì về các nguyên tắc hướng đối tượng, và cũng chẳng biết gì về việc tách giao diện người dùng khỏi quy tắc nghiệp vụ (business rule). Nhưng những mầm mống đã có ở đó, và chúng rất mạnh mẽ.

One unexpected side benefit of the approach was that we could patch the firmware over a dial-up connection. If we found a bug in the firmware, we could dial up our devices and use the on-board monitor program to alter the RAM vector for the faulty subroutine to point to a bit of empty RAM. Then we'd enter the repaired subroutine into that RAM area, by typing it in machine code, in hexadecimal.

Một lợi ích phụ ngoài dự tính của cách làm này là chúng tôi có thể vá firmware qua đường quay số. Nếu phát hiện lỗi trong firmware, chúng tôi có thể quay số tới thiết bị và dùng chương trình monitor nằm sẵn trên bo để sửa vector RAM của subroutine hỏng, cho nó

trở tới một mẫu RAM trống. Rồi chúng tôi gỡ subroutine đã sửa vào vùng RAM đó, bằng mã máy, ở dạng thập lục phân.

This was a great boon to our field service operation, and to our customers. If they had a problem, they didn't need us to ship new chips and schedule an **urgent** field service call. The system could be patched, and a new chip could be installed at the next regularly scheduled maintenance visit.

Đây là một món quà trời cho đối với bộ phận dịch vụ hiện trường của chúng tôi, và với cả khách hàng. Nếu gặp sự cố, họ không cần chúng tôi gửi chip mới rồi xếp lịch một chuyến đi khẩn cấp. Hệ thống có thể được vá ngay, còn con chip mới thì cài trong lần bảo trì định kỳ kế tiếp.

The Service Area Computer — Service Area Computer

The 4-TEL service area computer (SAC) was based on an M365 minicomputer. This system communicated with all the COLTs out in the field, through either dedicated or dial-up modems. It would command those COLTs to measure telephone lines, would receive back the raw results, and would then perform a complex analysis of those results to identify and locate any faults.

Service area computer (SAC) của 4-TEL chạy trên máy tính mini M365. Hệ thống này liên lạc với tất cả các COLT ngoài thực địa, qua modem thuê bao riêng hoặc modem quay số. Nó ra lệnh cho các COLT đo đường dây điện thoại, nhận về kết quả thô, rồi thực hiện một phân tích phức tạp trên các kết quả đó để nhận diện và định vị mọi lỗi.

Dispatch Determination — Quyết định điều thợ

One of the economic foundations for this system was based on the correct allocation of repair craftsmen. Repair craft were separated, by union rules, into three categories: central office, cable, and drop. CO craftsmen fixed problems inside the central office. Cable craftsmen fixed problems in the cable plant that connected the CO to the customer. Drop craftsmen fixed problems inside the customer's premises, and in the lines connecting the external cable to that premises (the "drop").

Một trong những nền tảng kinh tế của hệ thống này nằm ở việc phân bổ đúng thợ sửa chữa. Theo quy định công đoàn, thợ sửa chữa được chia thành ba nhóm: central office, cable và drop. Thợ CO sửa sự cố bên trong tổng đài. Thợ cable sửa sự cố trong hệ thống cáp nối tổng đài tới khách hàng. Thợ drop sửa sự cố bên trong nhà khách hàng, và trên đoạn dây nối cáp ngoài trời vào nhà (đoạn "drop").

When a customer complained about a problem, our system could diagnose that problem and determine which kind of **craftsman** to dispatch. This saved the phone companies lots of money because incorrect dispatches meant delays for the customer and wasted trips for the craftsmen.

Khi khách hàng phàn nàn về một sự cố, hệ thống của chúng tôi có thể chẩn đoán và xác định cần điều loại thợ nào. Điều này giúp các công ty điện thoại tiết kiệm rất nhiều tiền, vì điều nhằm thợ đồng nghĩa với việc khách hàng phải chờ đợi còn thợ thì đi công toi.

The code that made this dispatch determination was designed and written by someone who was very bright, but a terrible communicator. The process of writing the code has been described as "Three weeks of staring at the ceiling and two days of code pouring out of every orifice of his body—after which he quit."

Đoạn code đưa ra quyết định điều thợ này được thiết kế và viết bởi một người cực kỳ thông minh, nhưng giao tiếp thì tệ hại. Quá trình viết đoạn code ấy từng được mô tả thế này: "Ba

tuần nhìn chăm chăm lên trần nhà và hai ngày code tuôn ra từ mọi lỗ chân lông trên cơ thể anh ta — sau đó anh ta nghỉ việc.”

Nobody understood this code. Every time we tried to add a feature or fix a defect, we broke it in some way. And since it was upon this code that one of the primary economic benefits our system rested, every new defect was deeply embarrassing to the company.

Chẳng ai hiểu nổi đoạn code này. Cứ mỗi lần chúng tôi thử thêm tính năng hay sửa lỗi là lại làm hỏng nó theo một kiểu nào đó. Và vì một trong những lợi ích kinh tế chính yếu của hệ thống lại đặt trên chính đoạn code này, nên mỗi lỗi mới đều là một nỗi xấu hổ sâu sắc cho công ty.

In the end, our management simply told us to lock that code down and never modify it. That code became officially rigid .

Cuối cùng, ban quản lý chỉ đơn giản bảo chúng tôi hãy khoá đoạn code đó lại và đừng bao giờ sửa. Đoạn code ấy chính thức trở nên cứng đờ.

This experience impressed upon me the value of good, clean code.

Trải nghiệm này khắc sâu vào tôi giá trị của code tốt và sạch.

Architecture — Kiến trúc

The system was written in 1976 in M365 assembler. It was a single, monolithic program of roughly 60,000 lines. The operating system was a home-grown, nonpreemptive, task-switcher based on polling. We called it MPS for multiprocessing system. The M365 computer had no built-in stack,

Hệ thống được viết năm 1976 bằng hợp ngữ M365. Nó là một chương trình nguyên khối duy nhất, chừng 60.000 dòng. Hệ điều hành là một bộ chuyển tác vụ tự chế, không trưng dụng (nonpreemptive), dựa trên polling. Chúng tôi gọi nó là MPS, viết tắt của multiprocessing system. Máy M365 không có stack tích hợp,

so task-specific variables were kept in a special area of memory and swapped out at every context switch. Shared variables were managed with locks and semaphores. Reentrancy issues and race conditions were constant problems.

nên các biến riêng của từng tác vụ được giữ trong một vùng nhớ đặc biệt và bị hoán đổi ở mỗi lần chuyển ngữ cảnh. Biến dùng chung được quản lý bằng lock và semaphore. Vấn đề reentrancy và race condition là chuyện thường ngày.

There was no isolation of device control logic, or UI logic, from the business rules of the system. For example, modem control code could be found smeared throughout the bulk of the business rules and UI code. There was no attempt to gather it into a module or abstract the interface. The modems were controlled, at the bit **level**, by code that was scattered everywhere around the system.

Không hề có sự cô lập giữa logic điều khiển thiết bị, hay logic UI, với các quy tắc nghiệp vụ của hệ thống. Chẳng hạn, code điều khiển modem có thể tìm thấy rải rác khắp phần lớn khối code quy tắc nghiệp vụ và code UI. Không ai buồn gom nó vào một module hay trừu tượng hoá interface. Modem được điều khiển ở mức bit, bởi những đoạn code vung vãi khắp nơi trong hệ thống.

The same was true for the terminal UI. Messages and formatting control code were not isolated. They ranged far and wide throughout the 60,000-line code base.

Với UI trên terminal thì cũng vậy. Các thông điệp và code định dạng chẳng được cô lập gì cả. Chúng trải dài khắp hàng cùng ngõ hẻm của mã nguồn 60.000 dòng.

The modem modules we were using were designed to be mounted on PC boards. We bought those units from a third party, and integrated them with other circuitry onto a board that fit into our custom backplane. These units were expensive. So, after a few years, we decided to **design** our own modems. We, in the software group, begged the hardware designer to use the same bit formats for controlling the new modem. We explained that the modem control code was smeared everywhere, and that our system would have to deal with both kinds of modems in the future. So, we begged and cajoled, “Please make the new modem look just like the old modem from a software control point of view.”

Các module modem chúng tôi dùng được thiết kế để gắn lên bo mạch. Chúng tôi mua những khối modem đó từ một bên thứ ba, rồi tích hợp chúng cùng mạch điện khác lên một bo vừa với backplane tự chế của mình. Những khối này rất đắt. Nên sau vài năm, chúng tôi quyết định tự thiết kế modem riêng. Bên nhóm phần mềm chúng tôi đã năn nỉ anh kỹ sư thiết kế phần cứng hãy dùng đúng định dạng bit cũ để điều khiển modem mới. Chúng tôi giải thích rằng code điều khiển modem đang vung vãi khắp nơi, và rằng sau này hệ thống sẽ phải xoay sở với cả hai loại modem. Vậy nên chúng tôi vừa van nài vừa ngon ngọt: “Làm ơn hãy khiến modem mới trông y hệt modem cũ dưới góc nhìn điều khiển bằng phần mềm.”

But when we got the new modem, the control structured was entirely different. It was not just a little different. It was entirely, and completely, different.

Nhưng khi nhận được modem mới, cấu trúc điều khiển hoàn toàn khác. Không phải khác một chút. Mà khác hoàn toàn, khác triệt để.

Thanks, hardware engineer.

Cảm ơn nhé, anh kỹ sư phần cứng.

What were we to do? We were not simply replacing all the old modems with new modems. Instead, we were mixing old and new modems in our systems. The software needed to be able to handle both kinds of modems at the same time. Were we doomed to surround every place in the code that manipulated the modems with flags and special cases? There were hundreds of such places!

Chúng tôi phải làm gì đây? Đâu phải chúng tôi chỉ việc thay hết modem cũ bằng modem mới. Không, chúng tôi trộn cả modem cũ lẫn modem mới trong cùng hệ thống. Phần mềm phải xử lý được đồng thời cả hai loại. Chẳng lẽ chúng tôi bị kết án phải bao quanh mọi chỗ trong code có đụng tới modem bằng cờ hiệu và các trường hợp đặc biệt? Mà có tới hàng trăm chỗ như thế!

In the end, we opted for an even worse solution.

Cuối cùng, chúng tôi chọn một giải pháp còn tệ hơn.

One particular subroutine wrote data to the serial communication bus that was used to control all our devices, including our modems. We modified that subroutine to recognize the bit patterns that were specific to the old modem, and translate them into the bit patterns needed by the new modem.

Có một subroutine chuyên ghi dữ liệu ra bus truyền thông nối tiếp, vốn được dùng để điều khiển mọi thiết bị của chúng tôi, kể cả modem. Chúng tôi sửa subroutine đó để nó nhận diện các mẫu bit đặc trưng của modem cũ, rồi dịch chúng sang mẫu bit mà modem mới cần.

This was not straightforward. Commands to the modem consisted of sequences of writes to different IO addresses on the serial bus. Our hack had to interpret these commands, in sequence, and translate them into a different sequence using different IO addresses, timings, and bit positions.

Chuyện này chẳng hề đơn giản. Lệnh gửi tới modem là một chuỗi các thao tác ghi vào những địa chỉ IO khác nhau trên bus nối tiếp. Bản hack của chúng tôi phải diễn giải các lệnh này theo đúng thứ tự, rồi dịch chúng thành một chuỗi khác với địa chỉ IO khác, thời điểm khác và vị trí bit khác.

We got it to work, but it was the worst hack imaginable. It was because of this fiasco that I learned the value of isolating hardware from business rules, and of abstracting interfaces.

Chúng tôi làm cho nó chạy được, nhưng đó là bản hack tệ hại nhất có thể tưởng tượng. Chính nhờ vụ thảm hoạ này mà tôi học được giá trị của việc cô lập phần cứng khỏi quy tắc nghiệp vụ, và của việc trừu tượng hoá các interface.

The Grand Redesign in the Sky — Cuộc tái thiết kế vĩ đại trên chín tầng mây

By the time the 1980s rolled around, the idea of producing your own minicomputer and your own computer architecture was beginning to fall out of fashion. There were many microcomputers on the market, and getting them to work was cheaper and more standard than continuing to rely on proprietary computer architectures from the late 1960s. That, plus the horrible architecture of the SAC software, induced our technical management to start a complete re-architecture of the SAC system.

Đến khi thập niên 1980 ập tới, ý tưởng tự sản xuất máy tính mini và kiến trúc máy tính của riêng mình bắt đầu lỗi mốt. Trên thị trường đã có rất nhiều máy vi tính, và việc bắt chúng chạy vừa rẻ hơn vừa chuẩn hoá hơn so với việc tiếp tục bám vào những kiến trúc máy tính độc quyền từ cuối thập niên 1960. Điều đó, cộng với kiến trúc kinh khủng của phần mềm SAC, đã thúc ban quản lý kỹ thuật khởi động một cuộc tái kiến trúc toàn diện cho hệ thống SAC.

The new system was to be written in C using a UNIX O/S on disk, running on an Intel 8086 microcomputer. Our hardware guys started working on the new computer hardware, and a select group of software developers, “The Tiger Team,” was commissioned with the rewrite.

Hệ thống mới sẽ được viết bằng C, dùng hệ điều hành UNIX trên đĩa, chạy trên máy vi tính Intel 8086. Đám phần cứng bắt tay vào làm phần cứng máy tính mới, còn một nhóm lập trình viên được tuyển chọn — “Tiger Team” — được giao nhiệm vụ viết lại.

I won’t bore you with the details of the initial fiasco. Suffice it to say that the first Tiger Team failed entirely after burning two or three man-years on a software project that never delivered anything.

Tôi sẽ không làm bạn chán với chi tiết của thảm hoạ đầu tiên. Chỉ cần nói rằng Tiger Team đầu tiên thất bại hoàn toàn sau khi đốt hai ba người-năm cho một dự án phần mềm chẳng bao giờ giao được thứ gì.

A year or two later, probably 1982, the process was started again. The goal was the total and complete redesign of the SAC in C and UNIX on our own,

Một hai năm sau, có lẽ là 1982, quy trình lại được khởi động lần nữa. Mục tiêu là thiết kế lại SAC toàn bộ và triệt để bằng C và UNIX, trên phần cứng 80286 do chính chúng tôi

newly designed, impressively powerful 80286 hardware. We called that computer “Deep Thought.”

thiết kế, mới toanh và mạnh đến ấn tượng. Chúng tôi gọi cỗ máy đó là “Deep Thought.”

It took years, then more years, and then even more years. I don’t know when the first UNIX-based SAC was finally deployed; I believe I had left the company by then (1988). Indeed, I’m not at all sure it ever was deployed.

Nó ngắn nhiều năm, rồi thêm nhiều năm nữa, rồi lại thêm nhiều năm nữa. Tôi không biết chiếc SAC chạy UNIX đầu tiên rốt cuộc được triển khai khi nào; tôi tin là lúc đó mình đã rời công ty rồi (1988). Thật ra, tôi hoàn toàn không chắc nó có từng được triển khai hay không.

Why the delay? In short, it is very difficult for a redesign team to catch up with a large staff of programmers who are actively maintaining the old system. Here's just one example of the difficulties they encountered.

Vì sao lại chậm trễ như vậy? Nói ngắn gọn: một đội tái thiết kế rất khó đuổi kịp một dàn lập trình viên đông đảo đang tích cực bảo trì hệ thống cũ. Đây chỉ là một ví dụ trong số những khó khăn mà họ gặp phải.

Europe — Châu Âu

At about the same time that the SAC was being redesigned in C, the company started to expand sales into Europe. They could not wait for the redesigned software to be finished, so of course, they deployed the old M365 systems into Europe.

Vào khoảng thời gian SAC đang được thiết kế lại bằng C, công ty bắt đầu mở rộng bán hàng sang châu Âu. Họ không thể chờ phần mềm thiết kế lại hoàn tất, nên tất nhiên họ triển khai các hệ thống M365 cũ sang châu Âu.

The problem was that the phone systems in Europe were very different from the phone systems in the United States. The organization of the craft and of the bureaucracies were different as well. So one of our best programmers was sent to the United Kingdom to lead a team of U.K. developers to modify the SAC software to deal with all these European issues.

Vấn đề là hệ thống điện thoại ở châu Âu rất khác hệ thống điện thoại ở Mỹ. Cách tổ chức đội ngũ thợ và bộ máy hành chính cũng khác. Thế là một trong những lập trình viên giỏi nhất của chúng tôi được cử sang Anh để dẫn dắt một đội lập trình viên người Anh, sửa phần mềm SAC cho hợp với mọi đặc thù châu Âu ấy.

Of course, no serious attempt was made to integrate these changes into the U.S.-based software. This was long before networks made it feasible to transmit large code bases across the ocean. These U.K. developers simply forked the U.S.-based code and modified it as needed.

Dĩ nhiên, chẳng có nỗ lực nghiêm túc nào nhằm tích hợp những thay đổi này ngược vào phần mềm bản Mỹ. Chuyện này diễn ra rất lâu trước khi mạng máy tính khiến việc truyền những mã nguồn lớn vượt đại dương trở nên khả thi. Đám lập trình viên Anh chỉ đơn giản fork mã nguồn bản Mỹ ra rồi sửa theo nhu cầu.

This, of course, caused difficulties. Bugs were found on both sides of the Atlantic that needed repair on the other side. But the modules had changed significantly, so it was very difficult to determine whether the fix made in the United States would work in the United Kingdom.

Tất nhiên, điều đó gây rắc rối. Lỗi được phát hiện ở cả hai bờ Đại Tây Dương và cần được sửa ở bờ bên kia. Nhưng các module đã thay đổi đáng kể, nên rất khó xác định bản sửa làm ở Mỹ có chạy được ở Anh hay không.

After a few years of heartburn, and the installation of a high-throughput line connecting the U.S. and U.K. offices, a serious attempt was made to integrate

Sau vài năm đau đầu vì chuyện này, và sau khi lắp một đường truyền băng thông cao nối văn phòng Mỹ với văn phòng Anh, một nỗ lực nghiêm túc đã được tiến hành nhằm hợp nhất

these two forks back together again, making the differences a matter of configuration. This **effort** failed the first, second, and third times it was tried. The two code bases, though remarkably similar, were still too different to reintegrate—especially in the rapidly changing market environment that existed at that time.

hai nhánh fork này lại với nhau, biến khác biệt thành chuyện cấu hình. Nỗ lực đó thất bại ở lần thử thứ nhất, thứ hai và thứ ba. Hai mã nguồn, dù giống nhau đến mức đáng kinh ngạc, vẫn khác nhau quá nhiều để tích hợp lại — nhất là trong môi trường thị trường biến động chóng mặt thời đó.

Meanwhile, the “Tiger Team,” trying to rewrite everything in C and UNIX, realized that it also had to deal with this European/US dichotomy. And, of course, that did nothing to accelerate their progress.

Trong khi đó, “Tiger Team”, vốn đang cố viết lại mọi thứ bằng C và UNIX, nhận ra rằng họ cũng phải xử lý luôn cái thể lưỡng phân Âu/Mỹ này. Và tất nhiên, điều đó chẳng giúp tiến độ của họ nhanh lên chút nào.

SAC Conclusion — Kết luận về SAC

There are many other stories I could tell you about this system, but it’s just too depressing for me to continue. Suffice it to say that many of the hard lessons of my software life were learned while immersed in the horrible assembler code of the SAC.

Còn nhiều chuyện khác tôi có thể kể về hệ thống này, nhưng kể tiếp thì tôi buồn quá. Chỉ cần nói rằng nhiều bài học xương máu trong đời làm phần mềm của tôi đã được rút ra khi ngập lặn trong đống code hợp ngữ kinh hoàng của SAC.

C Language — Ngôn ngữ C

The 8085 computer hardware that we used in the 4-Tel Micro project gave us a relatively low-cost computing platform for many different projects that could be **embedded** into industrial environments. We could load it up with 32K of RAM and another 32K of ROM, and we had an extremely flexible and powerful scheme for controlling peripherals. What we did not have was a flexible and convenient language with which to program the machine. The 8085 assembler was simply not fun to write code in.

Phần cứng máy tính 8085 mà chúng tôi dùng trong dự án 4-Tel Micro đã cho chúng tôi một nền tảng tính toán tương đối rẻ, dùng được cho nhiều dự án khác nhau có thể nhúng vào môi trường công nghiệp. Chúng tôi có thể nhồi vào đó 32K RAM cùng 32K ROM nữa, và chúng tôi có một cơ chế điều khiển thiết bị ngoại vi cực kỳ linh hoạt và mạnh mẽ. Thứ chúng tôi không có là một ngôn ngữ linh hoạt và tiện lợi để lập trình cỗ máy ấy. Viết code bằng hợp ngữ 8085 đơn giản là chẳng vui vẻ gì.

On top of that, the assembler we were using was written by our own programmers. It ran on our M365 computers, using the cartridge tape operating system described in the “Laser Trim” section.

Thêm vào đó, trình hợp dịch chúng tôi đang dùng lại do chính các lập trình viên trong công ty viết ra. Nó chạy trên máy M365, dùng hệ điều hành băng cassette đã mô tả ở phần “Laser Trim”.

As fate would have it, our lead hardware engineer convinced our CEO that we needed a real computer. He didn’t actually know what he would do with it, but he had a lot of political clout. So we purchased a PDP-11/60.

Duyên số thế nào, anh kỹ sư phần cứng trưởng thuyết phục được CEO rằng chúng tôi cần một cỗ máy tính thực thụ. Thật ra anh ta chẳng biết sẽ làm gì với nó, nhưng anh ta có rất nhiều sức nặng chính trị. Thế là chúng tôi tậu một chiếc PDP-11/60.

I, a lowly software developer at the time, was ecstatic. I knew precisely what I wanted to do with that computer. I was determined that this was going to be my machine.

Tôi, một lập trình viên quèn thời đó, sướng phát điên. Tôi biết chính xác mình muốn làm gì với cỗ máy ấy. Tôi nhất quyết rằng đây sẽ là cỗ máy của tôi.

When the manuals arrived, many months before the delivery of the machine, I took them home and devoured them. By the time the computer was delivered, I knew how to operate both the hardware and the software at an intimate level—at least, as intimate as home study can make it.

Khi bộ tài liệu hướng dẫn về tới, nhiều tháng trước khi máy được giao, tôi mang chúng về nhà và ngấu nghiến. Đến lúc máy được giao, tôi đã biết vận hành cả phần cứng lẫn phần mềm ở mức thân thuộc — ít nhất là thân thuộc trong giới hạn mà việc tự học ở nhà có thể mang lại.

I helped to write the purchase order. In particular, I specified the disk storage that the new computer would have. I decided we should buy two disk drives that could take removable disk packs that held 25 megabytes each. 9

Tôi tham gia viết đơn đặt hàng. Cụ thể, tôi là người đặc tả phần lưu trữ đĩa cho cỗ máy mới. Tôi quyết định chúng tôi nên mua hai ổ đĩa nhận được các gói đĩa tháo rời, mỗi gói chứa 25 megabyte. 9

Fifty megabytes! The number seemed infinite! I remember walking through the halls of the office, late at night, cackling like the Wicked Witch of the West: “Fifty megabytes! Hahahahahahahahahah!”

Năm mươi megabyte! Con số ấy nghe như vô tận! Tôi vẫn nhớ cảnh mình đi dọc hành lang văn phòng lúc đêm khuya, cười khùng khục như mụ Phù thủy Độc ác miền Tây: “Năm mươi megabyte! Hahahahahahahahahah!”

I had the facilities manager build a little room that would house six VT100 terminals. I decorated it with pictures from space. Our software developers would use this room to write and compile code.

Tôi nhờ anh quản lý cơ sở vật chất dựng một căn phòng nhỏ chứa được sáu terminal VT100. Tôi trang trí nó bằng những bức ảnh vũ trụ. Đội lập trình viên của chúng tôi sẽ dùng căn phòng này để viết và biên dịch code.

When the machine arrived, I spent several days setting it up, wiring all the terminals, and getting everything to work. It was a joy—a labor of love.

Khi máy về tới, tôi dành mấy ngày trời lắp đặt, đi dây cho tất cả terminal, và làm cho mọi thứ chạy được. Đó là một niềm vui — một công việc làm vì yêu.

We purchased standard assemblers for the 8085 from Boston Systems Office, and we translated the 4-Tel Micro code into that syntax. We built a cross- compilation system that allowed us to download compiled binaries from the PDP-11 to our 8085 development environments, and ROM burners. And—Bob’s your Uncle—it all worked like a champ.

Chúng tôi mua các trình hợp dịch chuẩn cho 8085 từ Boston Systems Office, rồi chuyển code 4-Tel Micro sang cú pháp đó. Chúng tôi dựng một hệ thống biên dịch chéo cho phép tải các file nhị phân đã biên dịch từ PDP-11 xuống môi trường phát triển 8085 và xuống các máy nạp ROM. Và — thế là xong, Bob’s your Uncle — mọi thứ chạy ngon lành.

C — C

But that left us with the problem of still using 8085 assembler. That was not a situation that I was happy with. I had heard that there was this “new”

Nhưng chuyện đó vẫn để lại cho chúng tôi vấn đề là vẫn đang dùng hợp ngữ 8085. Đó không phải tình cảnh khiến tôi hài lòng. Tôi từng nghe nói có một ngôn ngữ “mới”

- 9. RKO7.
- 9. RKO7.

language that was heavily used at Bell Labs. They called it “C.” So I purchased a copy of The C Programming Language by Kernighan and Ritchie. Like the PDP-11 manuals a few months before, I inhaled this book.

được dùng rất nhiều ở Bell Labs. Người ta gọi nó là “C.” Thế là tôi mua một cuốn The C Programming Language của Kernighan và Ritchie. Giống như mấy quyển hướng dẫn PDP-11 vài tháng trước đó, tôi ngấu nghiến hết cuốn sách này.

I was astounded by the simple elegance of this language. It sacrificed none of the power of assembly language, and provided access to that power with a much more convenient syntax. I was sold.

Tôi kinh ngạc trước vẻ thanh lịch giản dị của ngôn ngữ này. Nó chẳng hy sinh chút sức mạnh nào của hợp ngữ, mà lại cho phép tiếp cận sức mạnh ấy bằng một cú pháp tiện lợi hơn hẳn. Tôi bị thuyết phục hoàn toàn.

I purchased a C compiler from Whitesmiths, and got it running on the PDP-11. The output of the compiler was assembler syntax that was compatible with the Boston Systems Office 8085 compiler. So we had a pathway to go from C to the 8085 hardware! We were in business.

Tôi mua một trình biên dịch C từ Whitesmiths và cho nó chạy trên PDP-11. Đầu ra của trình biên dịch là cú pháp hợp ngữ tương thích với trình biên dịch 8085 của Boston Systems Office. Vậy là chúng tôi đã có một con đường đi từ C xuống phần cứng 8085! Chúng tôi vào cuộc.

Now the only problem was convincing a group of embedded assembly language programmers that they should be using C. But that’s a nightmare tale for another time ...

Giờ vấn đề duy nhất còn lại là thuyết phục một nhóm lập trình viên hợp ngữ nhúng rằng họ nên dùng C. Nhưng đó lại là một câu chuyện ác mộng để dành cho dịp khác...

BOSS — BOSS

Our 8085 platform had no operating system. My experience with the MPS system of the M365, and the primitive interrupt mechanisms of the IBM System 7, convinced me that we needed a simple task switcher for the 8085. So I conceived of BOSS: Basic Operating System and Scheduler. 10

Nền tảng 8085 của chúng tôi không có hệ điều hành. Kinh nghiệm với hệ MPS trên M365, cùng cơ chế ngắt thô sơ của IBM System 7, thuyết phục tôi rằng chúng tôi cần một bộ chuyển tác vụ đơn giản cho 8085. Thế là tôi thai nghén BOSS: Basic Operating System and Scheduler. 10

The vast majority of BOSS was written in C. It provided the ability to create concurrent tasks. Those tasks were not preemptive—task switching did not take place based on interrupts. Instead, and just like with the MPS system on the M365, tasks were switched based on a simple polling mechanism. The polling happened whenever a task blocked for an event.

Tuyệt đại đa số BOSS được viết bằng C. Nó cung cấp khả năng tạo các tác vụ chạy đồng thời. Những tác vụ đó không trưng dụng — việc chuyển tác vụ không diễn ra dựa trên ngắt. Thay vào đó, y như hệ MPS trên M365, tác vụ được chuyển dựa trên một cơ chế polling đơn giản. Việc polling diễn ra mỗi khi một tác vụ bị chặn để chờ một sự kiện.

The BOSS call to block a task looked like this:

Lời gọi của BOSS để chặn một tác vụ trông như thế này:

```
block(eventCheckFunction);
```

- 10. This was later renamed as Bob's Only Successful Software.
- 10. Về sau nó được đổi tên thành Bob's Only Successful Software.

This call suspended the current task, placed the eventCheckFunction in the polling list, and associated it with the newly blocked task. It then waited in the polling loop, calling each of the functions in the polling list until one of them returned true. The task associated with that function was then allowed to run.

Lời gọi này tạm dừng tác vụ hiện tại, đặt eventCheckFunction vào danh sách polling, và gắn nó với tác vụ vừa bị chặn. Sau đó nó chờ trong vòng lặp polling, gọi lần lượt từng hàm trong danh sách polling cho tới khi một hàm nào đó trả về true. Tác vụ gắn với hàm đó khi ấy sẽ được phép chạy.

In other words, as I said before, it was a simple, nonpreemptive task switcher.

Nói cách khác, như tôi đã nói ở trên, đó là một bộ chuyển tác vụ đơn giản, không trưng dụng.

This software became the basis for a vast number of projects over the following years. But one of the first was the pCCU.

Phần mềm này trở thành nền tảng cho vô số dự án trong những năm tiếp theo. Nhưng một trong những dự án đầu tiên là pCCU.

pCCU — pCCU

The late 1970s and early 1980s were a tumultuous time for telephone companies. One of the sources of that tumult was the digital revolution.

Cuối thập niên 1970 và đầu thập niên 1980 là quãng thời gian đầy biến động với các công ty điện thoại. Một trong những nguồn cơn của biến động đó là cuộc cách mạng số.

For the preceding century, the connection between the central switching office and the customer's telephone had been a pair of copper wires. These wires were bundled into cables that spread in a huge network across the countryside. They were sometimes carried on poles, and sometimes buried underground.

Suốt cả thế kỷ trước đó, kết nối giữa tổng đài trung tâm và máy điện thoại của khách hàng là một đôi dây đồng. Những sợi dây này được bó lại thành cáp, trải thành một mạng lưới khổng lồ khắp vùng nông thôn. Có chỗ chúng treo trên cột, có chỗ chôn dưới đất.

Copper is a precious metal, and the phone company had tons (literally tons) of it covering the country. The capital investment was enormous. Much of that capital could be reclaimed by transporting the telephone conversation over digital connections. A single pair of copper wires could carry hundreds of conversations in digital form.

Đồng là kim loại quý, và công ty điện thoại có hàng tấn (đúng nghĩa hàng tấn) đồng phủ khắp đất nước. Vốn đầu tư là khổng lồ. Phần lớn số vốn đó có thể thu hồi lại bằng cách truyền cuộc thoại qua kết nối số. Một đôi dây đồng duy nhất có thể tải hàng trăm cuộc thoại ở dạng số.

In response, the phone companies embarked upon the process of replacing their old analog central switching equipment with modern digital switches.

Đáp lại, các công ty điện thoại bắt tay vào quá trình thay thế thiết bị chuyển mạch analog cũ ở tổng đài trung tâm bằng các tổng đài số hiện đại.

Our 4-Tel product tested copper wires, not digital connections. There were still plenty of copper wires in a digital environment, but they were much shorter than before, and they were localized near the customer's telephones. The signal would be carried digitally from the central office to a local

Sản phẩm 4-Tel của chúng tôi đo dây đồng, chứ không đo kết nối số. Trong môi trường số vẫn còn rất nhiều dây đồng, nhưng chúng ngắn hơn trước nhiều và nằm cục bộ gần máy điện thoại của khách hàng. Tín hiệu sẽ được truyền ở dạng số từ tổng đài trung tâm tới một điểm phân phối

distribution point, where it would be converted back to an analog signal and distributed to the customer over standard copper wires. This meant that our measurement device needed to be located out where the copper wires began, but our dialing device needed to remain at the central office. The problem was that all our COLTs embodied both dialing and measurement in the same device. (We could have saved ourselves a fortune had we recognized that obvious **architectural boundary** a few years earlier!)

cục bộ, nơi nó được chuyển ngược về tín hiệu analog rồi đưa tới khách hàng qua dây đồng tiêu chuẩn. Điều đó có nghĩa là thiết bị đo của chúng tôi phải nằm ngoài kia, ở nơi dây đồng bắt đầu, còn thiết bị quay số lại phải ở nguyên trong tổng đài trung tâm. Vấn đề là mọi COLT của chúng tôi đều gói cả việc quay số lẫn việc đo vào cùng một thiết bị. (Giá mà chúng tôi nhận ra cái ranh giới kiến trúc hiển nhiên đó sớm hơn vài năm thì đã tiết kiệm được cả một gia tài!)

Thus we conceived of a new product architecture: the CCU/CMU (the COLT control unit and the COLT measurement unit). The CCU would be located at the central switching office, and would handle the dialing of the phone lines to be tested. The CMU would be located at the local distribution points, and would measure the copper wires that led to the customer's phone.

Thế là chúng tôi nghĩ ra một kiến trúc sản phẩm mới: CCU/CMU (COLT control unit và COLT measurement unit). CCU sẽ đặt ở tổng đài trung tâm và lo việc quay số các đường dây cần kiểm tra. CMU sẽ đặt ở các điểm phân phối cục bộ và đo những sợi dây đồng dẫn tới máy điện thoại của khách hàng.

The problem was that for each CCU, there were many CMUs. The information about which CMU should be used for each phone number was held by the digital switch itself. Thus the CCU had to interrogate the digital switch to determine which CMU to communicate with and control.

Vấn đề là mỗi CCU lại đi kèm rất nhiều CMU. Thông tin về việc số điện thoại nào thì dùng CMU nào lại nằm trong chính tổng đài số. Vì vậy CCU phải hỏi tổng đài số để biết cần liên lạc và điều khiển CMU nào.

We promised the phone companies that we would have this new architecture working in time for their transition. We knew they were months, if not years away, so we did not feel rushed. We also knew that it would take several man- years to develop this new CCU/CMU hardware and software.

Chúng tôi hứa với các công ty điện thoại rằng kiến trúc mới này sẽ chạy được đúng lúc họ chuyển đổi. Chúng tôi biết họ còn cách đó vài tháng, nếu không nói là vài năm, nên chẳng thấy vội. Chúng tôi cũng biết rằng phát triển phần cứng và phần mềm CCU/CMU mới sẽ tốn vài người-năm.

The Schedule Trap — Cái bẫy lịch trình

As time went on, we found that there were always urgent matters that required us to postpone development of the CCU/CMU architecture. We felt safe about this decision because the phone companies were consistently delaying the deployment of digital switches. As we looked at their schedules, we felt confident that we had plenty of time, so we consistently delayed our development.

Thời gian trôi đi, và chúng tôi nhận ra lúc nào cũng có những việc khẩn cấp buộc phải hoãn việc phát triển kiến trúc CCU/CMU. Chúng tôi thấy quyết định đó an toàn, vì các công ty điện thoại liên tục trì hoãn việc triển khai tổng đài số. Nhìn vào lịch trình của họ, chúng tôi tự tin rằng mình còn khối thời gian, nên cứ đều đặn dời việc phát triển lại.

Then came the day that my boss called me into his office and said: “One of our customers is deploying a digital switch next month. We have to have a working CCU/CMU by then.”

Rồi tới ngày sếp gọi tôi vào phòng và nói: “Một khách hàng của chúng ta sẽ triển khai tổng đài số vào tháng sau. Đến lúc đó chúng ta phải có CCU/CMU chạy được.”

I was aghast! How could we possibly do man-years of development in a month? But my boss had a plan ...

Tôi choáng váng! Làm sao có thể làm xong khối lượng công việc vài người-năm trong một tháng? Nhưng sếp tôi đã có kế hoạch...

We did not, in fact, need a full CCU/CMU architecture. The phone company that was deploying the digital switch was tiny. They had only one central office, and only two local distribution points. More importantly, the “local” distribution points were not particularly local. They actually had regular-old analog switches in them that switched to several hundred customers. Better yet, those switches were of a kind that could be dialed by a normal COLT. Better even still, the customer’s phone number contained all the information necessary to decide which local distribution point to use. If the phone number had a 5, 6, or 7 in a certain position, it went to distribution point 1; otherwise, it went to distribution point 2.

Thực ra chúng tôi không cần một kiến trúc CCU/CMU đầy đủ. Công ty điện thoại đang triển khai tổng đài số kia rất nhỏ. Họ chỉ có một tổng đài trung tâm và đúng hai điểm phân phối cục bộ. Quan trọng hơn, những điểm phân phối “cục bộ” ấy chẳng cục bộ cho lắm. Trong đó thực chất là những tổng đài analog kiểu cũ, phục vụ vài trăm khách hàng. Còn may hơn nữa, những tổng đài đó thuộc loại mà một COLT bình thường có thể quay số tới được. Và may nhất, chính số điện thoại của khách hàng đã chứa đủ thông tin để quyết định dùng điểm phân phối nào. Nếu số điện thoại có chữ số 5, 6 hoặc 7 ở một vị trí nhất định thì nó thuộc điểm phân phối 1; nếu không thì thuộc điểm phân phối 2.

So, as my boss explained to me, we did not actually need a CCU/CMU. What we needed was a simple computer at the central office connected by modem lines to two standard COLTs at the distribution points. The SAC would communicate with our computer at the central office, and that computer would decode the phone number and then relay the dialing and measurement commands to the COLT at the appropriate distribution point.

Thế nên, như sếp giải thích cho tôi, chúng tôi không thực sự cần CCU/CMU. Cái chúng tôi cần chỉ là một máy tính đơn giản đặt ở tổng đài trung tâm, nối bằng đường modem tới

hai COLT tiêu chuẩn ở hai điểm phân phối. SAC sẽ nói chuyện với máy tính đó ở tổng đài trung tâm, còn máy tính đó sẽ giải mã số điện thoại rồi chuyển tiếp các lệnh quay số và đo tới đúng COLT ở điểm phân phối tương ứng.

Thus was born the pCCU.

Và pCCU ra đời như thế.

This was the first product written in C and using BOSS that was deployed to a customer. It took me about a week to develop. There is no deep architectural significance to this tale, but it makes a nice preface to the next project.

Đây là sản phẩm đầu tiên viết bằng C, dùng BOSS, được triển khai tới khách hàng. Tôi mất khoảng một tuần để làm. Câu chuyện này chẳng có ý nghĩa kiến trúc sâu xa gì, nhưng nó là lời mở đầu đẹp cho dự án tiếp theo.

DLU/DRU — DLU/DRU

In the early 1980s, one of our customers was a telephone company in Texas. They had large geographic areas to cover. In fact, the areas were so large that a single service area required several different offices from which to dispatch craftsmen. Those offices had test craftspeople who needed terminals into our SAC.

Đầu thập niên 1980, một trong các khách hàng của chúng tôi là một công ty điện thoại ở Texas. Họ phải phủ những vùng địa lý rất rộng. Rộng đến mức một khu vực dịch vụ duy nhất cũng cần vài văn phòng khác nhau để điều thợ đi làm. Những văn phòng ấy có các kỹ thuật viên kiểm tra cần terminal nối vào SAC của chúng tôi.

You might think that this was a simple problem to solve—but remember that this story takes place in the early 1980s. Remote terminals were not very common. To make matters worse, the hardware of the SAC presumed that all the terminals were local. Our terminals actually sat on a proprietary, high-speed, serial bus.

Bạn có thể nghĩ đây là vấn đề dễ giải quyết — nhưng nhớ rằng câu chuyện này diễn ra vào đầu thập niên 1980. Terminal từ xa hồi đó chưa phổ biến lắm. Tệ hơn nữa, phần cứng của SAC mặc định rằng mọi terminal đều nằm tại chỗ. Terminal của chúng tôi thực chất treo trên một bus nối tiếp tốc độ cao, độc quyền.

We had remote terminal capability, but it was based on modems, and in the early 1980s modems were generally limited to 300 bits per second. Our customers were not happy with that slow speed.

Chúng tôi có khả năng chạy terminal từ xa, nhưng nó dựa trên modem, và đầu thập niên 1980 modem thường bị giới hạn ở 300 bit mỗi giây. Khách hàng chẳng vui gì với tốc độ rùa bò đó.

High-speed modems were available, but they were very expensive, and they needed to run on “conditioned” permanent connections. Dial-up quality was definitely not good enough.

Modem tốc độ cao thì có, nhưng rất đắt, và chúng cần chạy trên đường kết nối cố định đã được “điều kiện hoá”. Chất lượng đường quay số dứt khoát là không đủ tốt.

Our customers demanded a solution. Our response was DLU/DRU.

Khách hàng đòi một giải pháp. Câu trả lời của chúng tôi là DLU/DRU.

DLU/DRU stood for “Display Local Unit” and “Display Remote Unit.” The DLU was a computer board that plugged into the SAC computer chassis and pretended to be a terminal manager board. Instead of

controlling the serial bus for local terminals, however, it took the character stream and multiplexed it over a single 9600-bps conditioned modem link.

DLU/DRU là viết tắt của “Display Local Unit” và “Display Remote Unit.” DLU là một bo mạch máy tính cắm vào khung máy SAC và giả làm bo quản lý terminal. Nhưng thay vì điều khiển bus nối tiếp cho các terminal tại chỗ, nó lấy dòng ký tự và ghép kênh qua một đường modem 9600 bps đã điều kiện hoá.

The DRU was a box placed at the customer’s remote location. It connected to the other end of the 9600-bps link, and had the hardware to control the terminals on our proprietary serial bus. It demultiplexed the characters received from the 9600-bps link and sent them to the appropriate local terminals.

DRU là một hộp đặt tại địa điểm từ xa của khách hàng. Nó nối vào đầu bên kia của đường 9600 bps, và có phần cứng để điều khiển các terminal trên bus nối tiếp độc quyền của chúng tôi. Nó tách kênh các ký tự nhận được từ đường 9600 bps rồi gửi tới đúng terminal tại chỗ.

Strange, isn’t it? We had to engineer a solution that nowadays is so ubiquitous we never even think about it. But back then ...

Lạ nhỉ? Chúng tôi phải tự thiết kế một giải pháp mà ngày nay phổ biến đến mức chẳng ai buồn nghĩ tới nữa. Nhưng hồi đó thì...

We even had to invent our own communications protocol because, in those days, standard communications protocols were not open source shareware. Indeed, this was long before we had any kind of Internet connection.

Chúng tôi thậm chí còn phải tự phát minh giao thức truyền thông của riêng mình, vì thời ấy các giao thức truyền thông tiêu chuẩn không phải là shareware mã nguồn mở. Thực tế, chuyện này diễn ra rất lâu trước khi chúng tôi có bất kỳ kết nối Internet nào.

Architecture — Kiến trúc

The architecture of this system was very simple, but there are some interesting quirks I want to highlight. First, both units used our 8085 technology, and both were written in C and used BOSS. But that’s where the similarity ended.

Kiến trúc của hệ thống này rất đơn giản, nhưng có vài nét kỳ thú mà tôi muốn nhấn mạnh. Trước hết, cả hai khối đều dùng công nghệ 8085 của chúng tôi, đều viết bằng C và dùng BOSS. Nhưng điểm giống nhau chỉ tới đó.

There were two of us on the project. I was the project lead, and Mike Carew was my close associate. I took on the design and coding of the DLU; Mike did the DRU.

Dự án có hai người. Tôi là trưởng dự án, còn Mike Carew là cộng sự thân cận. Tôi nhận phần thiết kế và viết mã DLU; Mike làm DRU.

The architecture of the DLU was based on a dataflow model. Each task did a small and focused job, and then passed its output to the next task in line, using a queue. Think of a pipes and filters model in UNIX. The architecture was intricate. One task might feed a queue that many others would service. Other tasks would feed a queue that just one task would service.

Kiến trúc của DLU dựa trên mô hình luồng dữ liệu (dataflow). Mỗi tác vụ làm một việc nhỏ và tập trung, rồi chuyển kết quả sang tác vụ kế tiếp trong hàng, thông qua một queue. Hãy nghĩ tới mô hình pipes and filters trong UNIX. Kiến trúc ấy khá rối rắm. Một tác vụ có

thể nạp vào một queue mà nhiều tác vụ khác cùng phục vụ. Tác vụ khác lại nạp vào một queue chỉ có đúng một tác vụ phục vụ.

Think of an assembly line. Each position on the assembly line has a single, simple, highly focused job to perform. Then the product moves to the next position in line. Sometimes the assembly line splits into many lines. Sometimes those lines merge back into a single line. That was the DLU.

Hãy hình dung một dây chuyền lắp ráp. Mỗi vị trí trên dây chuyền làm đúng một việc đơn giản, tập trung cao độ. Rồi sản phẩm chuyển sang vị trí kế tiếp. Có lúc dây chuyền tách thành nhiều nhánh. Có lúc các nhánh nhập lại thành một. DLU chính là như vậy.

Mike's DRU used a remarkably different scheme. He created one task per terminal, and simply did the entire job for that terminal in that task. No queues. No data flow. Just many identical large tasks, each managing its own terminal.

DRU của Mike lại dùng một sơ đồ khác hẳn. Anh tạo một tác vụ cho mỗi terminal, và đơn giản là làm trọn công việc của terminal đó ngay trong tác vụ ấy. Không queue. Không luồng dữ liệu. Chỉ có rất nhiều tác vụ lớn giống hệt nhau, mỗi tác vụ tự quản terminal của mình.

This is the opposite of an assembly line. In this case the analogy is many expert builders, each of whom builds an entire product.

Đây là điều ngược lại với dây chuyền lắp ráp. Ở đây phép so sánh là nhiều người thợ lành nghề, mỗi người tự tay dựng nên trọn vẹn một sản phẩm.

At the time I thought my architecture was superior. Mike, of course, thought his was better. We had many entertaining discussions about this. In the end, of course, both worked quite well. And I was left with the realization that software architectures can be wildly different, yet equally effective.

Hồi đó tôi nghĩ kiến trúc của mình vượt trội hơn. Dĩ nhiên Mike nghĩ kiến trúc của anh mới hay hơn. Chúng tôi đã có vô số cuộc tranh luận thú vị về chuyện này. Và cuối cùng, tất nhiên, cả hai đều chạy rất tốt. Tôi rút ra được rằng kiến trúc phần mềm có thể khác nhau một trời một vực mà vẫn hiệu quả ngang nhau.

VRS — VRS

As the 1980s progressed, newer and newer technologies appeared. One of those technologies was the computer control of voice .

Thập niên 1980 trôi đi, hết công nghệ mới này tới công nghệ mới khác xuất hiện. Một trong số đó là điều khiển giọng nói bằng máy tính.

One of the features of the 4-Tel system was the ability of the craftsman to locate a fault in a cable. The procedure was as follows:

Một trong những tính năng của hệ thống 4-Tel là khả năng giúp người thợ định vị điểm hỏng trên một sợi cáp. Quy trình như sau:

- The tester, in the central office, would use our system to determine the
 - Kỹ thuật viên kiểm tra, ngồi ở tổng đài trung tâm, dùng hệ thống của chúng tôi để xác định

approximate distance, in feet, to the fault. This would be accurate to within 20% or so. The tester would dispatch a cable repair craftsman to an appropriate access point near that position.

khoảng cách gần đúng, tính bằng feet, tới điểm hỏng. Con số này có sai số chừng 20%. Kỹ thuật viên kiểm tra sẽ điều một thợ sửa cáp tới điểm tiếp cận thích hợp gần vị trí đó.

- The cable repair craftsman, upon arrival, would call the tester and ask to

- Thợ sửa cáp, khi tới nơi, sẽ gọi cho kỹ thuật viên kiểm tra và đề nghị

begin the fault location process. The tester would invoke the fault location feature of the 4-Tel system. The system would begin measuring the electronic characteristics of that faulty line, and would print messages on the screen requesting that certain operations be performed, such as opening the cable or shorting the cable.

bắt đầu quy trình định vị điểm hỏng. Kỹ thuật viên kiểm tra kích hoạt tính năng định vị điểm hỏng của hệ thống 4-Tel. Hệ thống bắt đầu đo các đặc tính điện của đường dây hỏng, rồi in lên màn hình những thông báo yêu cầu thực hiện một số thao tác nhất định, chẳng hạn hở mạch hoặc chập mạch sợi cáp.

- The tester would tell the craftsman which operations the system wanted,

- Kỹ thuật viên kiểm tra sẽ nói cho người thợ biết hệ thống muốn thao tác nào,

and the craftsman would tell the tester when the operation was complete. The tester would then tell the system that the operation was complete, and the system would continue with the test.

và người thợ báo lại cho kỹ thuật viên kiểm tra khi làm xong. Kỹ thuật viên kiểm tra báo cho hệ thống rằng thao tác đã hoàn tất, và hệ thống tiếp tục đo.

- After two or three such interactions, the system would calculate a new

- Sau hai hoặc ba lượt tương tác như vậy, hệ thống sẽ tính ra một

distance to the fault. The cable craftsman would then drive to that location and begin the process again.

khoảng cách mới tới điểm hỏng. Người thợ cáp lại lái xe tới vị trí đó và bắt đầu lại quy trình.

Imagine how much better that would be if the cable craftsmen, up on the pole or standing at a pedestal, could operate the system themselves. And that is exactly what the new voice technologies allowed us to do. The cable craftsmen could call directly into our system, direct the system with touch tones, and listen to the results being read back to them in a pleasant voice.

Hãy tưởng tượng mọi thứ sẽ tốt hơn bao nhiêu nếu chính người thợ cáp, đang leo trên cột hay đứng cạnh tủ cáp, có thể tự vận hành hệ thống. Và đó chính xác là điều mà các công nghệ giọng nói mới cho phép chúng tôi làm. Thợ cáp có thể gọi thẳng vào hệ thống, điều khiển nó bằng phím bấm, rồi nghe kết quả được đọc lại bằng một giọng dễ chịu.

The Name — Cái tên

The company held a little contest to select a name for the new system. One of the most creative of the names suggested was SAM CARP. This stood for

Công ty tổ chức một cuộc thi nhỏ nhỏ để chọn tên cho hệ thống mới. Một trong những cái tên sáng tạo nhất được đề xuất là SAM CARP. Nó là viết tắt của

“Still Another Manifestation of Capitalist Avarice Repressing the Proletariat.” Needless to say, that wasn’t selected.

“Still Another Manifestation of Capitalist Avarice Repressing the Proletariat.” Khỏi phải nói, cái tên đó không được chọn.

Another was the Teradyne Interactive Test System. That one was also not selected.

Một cái tên khác là Teradyne Interactive Test System. Cái đó cũng không được chọn.

Still another was Service Area Test Access Network. That, too, was not selected.

Rồi tới Service Area Test Access Network. Cái đó cũng trượt nốt.

The winner, in the end, was VRS: Voice Response System.

Rốt cuộc người thắng cuộc là VRS: Voice Response System.

Architecture — Kiến trúc

I did not work on this system, but I heard about what happened. The story I am going to relate to you is second-hand, but for the most part, I believe it to be correct.

Tôi không làm hệ thống này, nhưng tôi có nghe kể chuyện gì đã xảy ra. Câu chuyện tôi sắp kể là chuyện nghe lại, nhưng về cơ bản tôi tin nó đúng.

These were the heady days of microcomputers, UNIX operating systems, C, and SQL databases. We were determined to use them all.

Đó là những ngày hưng phấn của máy vi tính, hệ điều hành UNIX, C và cơ sở dữ liệu SQL. Chúng tôi quyết dùng bằng hết.

From the many database vendors out there, we eventually chose UNIFY. UNIFY was a database system that worked with UNIX, which was perfect for us.

Trong số rất nhiều nhà cung cấp cơ sở dữ liệu ngoài kia, cuối cùng chúng tôi chọn UNIFY. UNIFY là một hệ cơ sở dữ liệu chạy với UNIX, quá hoàn hảo với chúng tôi.

UNIFY also supported a new technology called Embedded SQL. This technology allowed us to embed SQL commands, as strings, right into our C code. And so we did—everywhere.

UNIFY còn hỗ trợ một công nghệ mới tên là Embedded SQL. Công nghệ này cho phép nhúng các câu lệnh SQL, dưới dạng chuỗi, thẳng vào mã C. Và chúng tôi đã làm đúng thế — ở khắp mọi nơi.

I mean, it was just so cool that you could put your SQL right into your code, anywhere you wanted. And where did we want to? Everywhere! And so there was SQL smeared throughout the body of that code.

Ý tôi là, chuyện đặt SQL thẳng vào mã, ở bất cứ chỗ nào bạn muốn, nghe thật quá đã. Mà chúng tôi muốn đặt ở đâu? Khắp nơi! Thế là SQL bị trét đầy khắp thân mã nguồn ấy.

Of course, in those days SQL was hardly a solid standard. There were lots of special vendor-specific quirks. So the special SQL and special UNIFY API calls were also smeared throughout the code.

Dĩ nhiên, hồi đó SQL còn lâu mới là một chuẩn vững chắc. Có cả đồng nét đặc thù riêng của từng nhà cung cấp. Thế là những câu SQL đặc thù và những lời gọi API riêng của UNIFY cũng bị trét đầy khắp mã nguồn.

This worked great! The system was a success. The craftsmen used it, and the telephone companies loved it. Life was all smiles.

Cách này chạy tuyệt vời! Hệ thống thành công. Thợ dùng nó, và các công ty điện thoại mê nó. Cuộc đời toàn nụ cười.

Then the UNIFY product we were using was cancelled.

Rồi sản phẩm UNIFY mà chúng tôi đang dùng bị khai tử.

Oh. Oh.

Ồi. Ồi.

So we decided to switch to SyBase. Or was it Ingress? I don't remember. Suffice it to say, we had to search through all that C code, find all the embedded SQL and special API calls, and replace them with corresponding gestures for the new vendor.

Thế là chúng tôi quyết định chuyển sang SyBase. Hay là Ingress nhỉ? Tôi không nhớ nữa. Chỉ cần biết rằng chúng tôi phải lục tung toàn bộ mã C, tìm mọi câu SQL nhúng và mọi lời gọi API đặc thù, rồi thay bằng các thao tác tương ứng của nhà cung cấp mới.

After three months of effort or so, we gave up. We couldn't make it work. We were so coupled to UNIFY that there was no serious hope of restructuring the code at any practical expense.

Sau khoảng ba tháng vật lộn, chúng tôi bỏ cuộc. Chúng tôi không làm nổi. Chúng tôi bị kết dính vào UNIFY chặt đến mức không còn chút hy vọng nghiêm túc nào cho việc tái cấu trúc mã với một chi phí khả thi.

So, we hired a third party to maintain UNIFY for us, based on a maintenance contract. And, of course, the maintenance rates went up year after year after year.

Thế là chúng tôi thuê một bên thứ ba bảo trì UNIFY cho mình theo hợp đồng bảo trì. Và tất nhiên, giá bảo trì cứ tăng năm này qua năm khác qua năm khác.

VRS Conclusion — Kết luận về VRS

This is one of the ways that I learned that databases are details that should be isolated from the overall business purpose of the system. This is also one of the reasons that I don't like strongly coupling to third-party software systems.

Đây là một trong những cách tôi học được rằng cơ sở dữ liệu là chi tiết (detail) và phải được cách ly khỏi mục đích nghiệp vụ tổng thể của hệ thống. Đây cũng là một trong những lý do tôi không thích kết dính chặt vào các hệ thống phần mềm của bên thứ ba.

The Electronic Receptionist — The Electronic Receptionist

In 1983, our company sat at the confluence of computer systems, telecommunications systems, and voice systems. Our CEO thought this might be a fertile position from which to develop new products. To address this goal, he commissioned a team of three (which included me) to conceive, design, and implement a new product for the company.

Năm 1983, công ty chúng tôi nằm ở giao điểm của hệ thống máy tính, hệ thống viễn thông và hệ thống giọng nói. CEO của chúng tôi cho rằng đây có thể là mảnh đất màu mỡ để phát triển sản phẩm mới. Để theo đuổi mục tiêu đó, ông lập một nhóm ba người (trong đó có tôi) để hình thành ý tưởng, thiết kế và hiện thực một sản phẩm mới cho công ty.

It didn't take us long to come up with The Electronic Receptionist (ER).

Chúng tôi không mất nhiều thời gian để nghĩ ra The Electronic Receptionist (ER).

The idea was simple. When you called a company, ER would answer and ask you who you wanted to speak with. You would use touch tones to spell the name of that person, and ER would then connect you. The users of ER could

Ý tưởng rất đơn giản. Khi bạn gọi tới một công ty, ER sẽ trả lời và hỏi bạn muốn nói chuyện với ai. Bạn dùng phím bấm để đánh vần tên người đó, rồi ER nối máy cho bạn. Người dùng ER có thể

dial in and, by using simple touch-tone commands, tell it which phone number the desired person could be reached at, anywhere in the world. In fact, the system could list several alternate numbers.

gọi vào và, bằng vài lệnh phím bấm đơn giản, cho nó biết có thể liên lạc với người cần gặp qua số điện thoại nào, ở bất cứ đâu trên thế giới. Thực tế, hệ thống có thể liệt kê nhiều số thay thế.

When you called ER and dialed RMART (my code), ER would call the first number on my list. If I failed to answer and identify myself, it would call the next number, and the next. If I still wasn't reached, ER would record a message from the caller.

Khi bạn gọi ER và bấm RMART (mã của tôi), ER sẽ gọi số đầu tiên trong danh sách của tôi. Nếu tôi không trả lời và xác nhận danh tính, nó sẽ gọi số kế tiếp, rồi số kế tiếp nữa. Nếu vẫn không tìm được tôi, ER sẽ ghi lại lời nhắn của người gọi.

ER would then, periodically, try to find me to deliver that message, and any other message left for me by anyone else.

Sau đó, ER sẽ định kỳ tìm cách gặp tôi để chuyển lời nhắn ấy, cùng mọi lời nhắn khác mà người ta để lại cho tôi.

This was the first voice mail system ever, and we 11 held the patent to it.

Đây là hệ thống thư thoại đầu tiên trên đời, và chúng tôi 11 giữ bằng sáng chế cho nó.

We built all the hardware for this system—the computer board, the memory board, the voice/telecom boards, everything. The main computer board was Deep Thought, the Intel 80286 processor that I mentioned earlier.

Chúng tôi tự làm toàn bộ phần cứng cho hệ thống này — bo mạch máy tính, bo nhớ, bo giọng nói/viễn thông, tất cả. Bo mạch máy tính chính là Deep Thought, con vi xử lý Intel 80286 mà tôi đã nhắc tới trước đó.

The voice boards each supported one telephone line. They consisted of a telephone interface, a voice encoder/decoder, some memory, and an Intel 80186 microcomputer.

Mỗi bo giọng nói phục vụ một đường điện thoại. Chúng gồm một giao diện điện thoại, một bộ mã hoá/giải mã giọng nói, một ít bộ nhớ, và một máy vi tính Intel 80186.

The software for the main computer board was written in C. The operating system was MP/M-86, an early command-line-driven, multiprocessing, disk operating system. MP/M was the poor man's UNIX.

Phần mềm cho bo mạch máy tính chính được viết bằng C. Hệ điều hành là MP/M-86, một hệ điều hành đĩa thời kỳ đầu, chạy bằng dòng lệnh và đa tiến trình. MP/M là UNIX của nhà nghèo.

The software for the voice boards was written in assembler, and had no operating system. Communication between Deep Thought and the voice boards occurred through shared memory.

Phần mềm cho các bo giọng nói được viết bằng assembler và không có hệ điều hành. Giao tiếp giữa Deep Thought và các bo giọng nói diễn ra qua vùng nhớ dùng chung.

The architecture of this system would today be called service oriented . Each telephone line was monitored by a listener process running under MP/M.

Kiến trúc của hệ thống này ngày nay hẳn sẽ được gọi là hướng dịch vụ (service oriented). Mỗi đường điện thoại được một tiến trình listener chạy dưới MP/M theo dõi.

- 11. Our company held the patent. Our employment contract made it clear that anything we invented
 - 11. Công ty chúng tôi giữ bằng sáng chế. Hợp đồng lao động nói rõ rằng mọi thứ chúng tôi phát minh ra

belonged to our company. My boss told me: “You sold it to us for one dollar, and we didn’t pay you that dollar.”

đều thuộc về công ty. Sếp tôi bảo: “Cậu đã bán nó cho chúng tôi với giá một đô la, và chúng tôi chưa trả cậu đồng đô la đó.”

When a call came in, an initial handler process was started and the call was passed to it. As the call proceeded from state to state, the appropriate handler process would be started and take control.

Khi có cuộc gọi tới, một tiến trình xử lý ban đầu được khởi động và cuộc gọi được chuyển cho nó. Cuộc gọi đi từ trạng thái này sang trạng thái khác, và tiến trình xử lý tương ứng sẽ được khởi động và tiếp quản.

Messages were passed between these services through disk files. The currently running service would determine what the next service should be; would write the necessary state information into a disk file; would issue the command line to start that service; and then would exit.

Thông điệp được truyền giữa các dịch vụ này thông qua file trên đĩa. Dịch vụ đang chạy sẽ quyết định dịch vụ kế tiếp là gì; ghi thông tin trạng thái cần thiết vào một file trên đĩa; phát dòng lệnh để khởi động dịch vụ đó; rồi thoát.

This was the first time I had built a system like this. Indeed, this was the first time I had been the principal architect of an entire product. Everything having to do with software was mine—and it worked like a champ.

Đây là lần đầu tiên tôi dựng một hệ thống kiểu này. Thực ra, đây cũng là lần đầu tiên tôi làm kiến trúc sư chính cho trọn một sản phẩm. Mọi thứ dính tới phần mềm đều là của tôi — và nó chạy ngon lành.

I would not say that the architecture of this system was “clean” in the sense of this book; it was not a “plugin” architecture. However, it definitely showed signs of true boundaries. The services were independently deployable, and lived within their own domain of responsibility. There were high-level processes and **low-level** processes, and many of the dependencies ran in the right direction.

Tôi sẽ không nói rằng kiến trúc của hệ thống này là “clean” theo nghĩa của cuốn sách này; nó không phải kiến trúc “plugin”. Tuy nhiên, nó chắc chắn có dấu hiệu của những ranh giới thực sự. Các dịch vụ có thể triển khai độc lập và sống trong phạm vi trách nhiệm của riêng mình. Có những tiến trình mức cao và những tiến trình mức thấp, và nhiều phụ thuộc chạy đúng chiều.

ER Demise — Hồi kết của ER

Unfortunately, the marketing of this product did not go very well. Teradyne was a company that sold test equipment. We did not understand how to break into the office equipment market.

Đáng tiếc là việc tiếp thị sản phẩm này không suôn sẻ cho lắm. Teradyne là công ty bán thiết bị kiểm tra. Chúng tôi không hiểu cách chen chân vào thị trường thiết bị văn phòng.

After repeated attempts over two years, our CEO gave up and— unfortunately—dropped the patent application. The patent was picked up by the company that filed three months after we filed; thus we surrendered the entire voice mail and electronic call-forwarding market.

Sau hai năm thử đi thử lại, CEO của chúng tôi bỏ cuộc và — đáng tiếc thay — rút đơn xin cấp bằng sáng chế. Bằng sáng chế rơi vào tay công ty nộp đơn sau chúng tôi ba tháng; thế là chúng tôi dâng trọn thị trường thư thoại và chuyển tiếp cuộc gọi điện tử cho họ.

Ouch!

Đau!

On the other hand, you can't blame me for those annoying machines that now plague our existence.

Nhưng mặt khác, bạn không thể trách tôi về mấy cái máy phiền toái đang hành hạ cuộc sống của chúng ta ngày nay.

Craft Dispatch System — Craft Dispatch System

ER had failed as a product, but we still had all this hardware and software that we could use to enhance our existing product lines. Moreover, our marketing success with VRS convinced us that we should offer a voice response system for interacting with telephone craftsmen that did not depend on our test systems.

ER đã thất bại với tư cách một sản phẩm, nhưng chúng tôi vẫn còn nguyên đồng phần cứng và phần mềm đó để nâng cấp các dòng sản phẩm sẵn có. Hơn nữa, thành công về mặt thị trường của VRS thuyết phục chúng tôi rằng nên đưa ra một hệ thống trả lời thoại để tương tác với thợ điện thoại mà không phụ thuộc vào các hệ thống kiểm tra của mình.

Thus was born CDS, the Craft Dispatch System. CDS was essentially ER, but specifically focused on the very narrow domain of managing the deployment of telephone repairmen in the field.

CDS — Craft Dispatch System — ra đời như thế. Về bản chất, CDS chính là ER, nhưng tập trung hẳn vào một miền rất hẹp: quản lý việc điều phối thợ sửa điện thoại ngoài hiện trường.

When a problem was discovered in a phone line, a trouble ticket was created in the service center. Trouble tickets were kept in an automated system. When a repairman in the field finished a job, he would call the service center for the next **assignment**. The service center operator would pull up the next trouble ticket and read it off to the repairman.

Khi phát hiện sự cố trên một đường dây điện thoại, trung tâm dịch vụ sẽ tạo một phiếu sự cố (trouble ticket). Các phiếu sự cố được lưu trong một hệ thống tự động. Khi một người thợ ngoài hiện trường làm xong một việc, anh ta gọi về trung tâm dịch vụ để nhận việc kế tiếp. Nhân viên trực trung tâm dịch vụ sẽ mở phiếu sự cố tiếp theo và đọc cho người thợ nghe.

We set about to automate that process. Our goal was for the repairman in the field to call into CDS and ask for the next assignment. CDS would consult the trouble ticket system, and read off the results.

CDS would keep track of which repairman was assigned to which trouble ticket, and would inform the trouble ticket system of the status of the repair.

Chúng tôi bắt tay tự động hóa quy trình đó. Mục tiêu là để người thợ ngoài hiện trường gọi vào CDS và hỏi việc kế tiếp. CDS sẽ tra cứu hệ thống phiếu sự cố rồi đọc kết quả ra. CDS theo dõi xem thợ nào được giao phiếu nào, và báo lại cho hệ thống phiếu sự cố tình trạng của ca sửa chữa.

There were quite a few interesting features of this system having to do with interacting with the trouble ticket system, the plant management system, and any automated testing systems.

Hệ thống này có khá nhiều tính năng thú vị liên quan tới việc tương tác với hệ thống phiếu sự cố, hệ thống quản lý hạ tầng, và mọi hệ thống kiểm tra tự động.

The experience with the service-oriented architecture of ER made me want to try the same idea more aggressively. The state machine for a trouble ticket was much more involved than the state machine for handling a call with ER. I set about to create what would now be called a **micro-service** architecture .

Trải nghiệm với kiến trúc hướng dịch vụ của ER khiến tôi muốn thử lại ý tưởng đó một cách quyết liệt hơn. Máy trạng thái của một phiếu sự cố phức tạp hơn nhiều so với máy trạng thái xử lý một cuộc gọi trong ER. Tôi bắt tay tạo ra cái mà ngày nay người ta gọi là kiến trúc micro-service.

Every state transition of any call, no matter how insignificant, caused the system to start up a new service. Indeed, the state machine was externalized into a text file that the system read. Each event coming into the system from a

Mỗi lần chuyển trạng thái của bất kỳ cuộc gọi nào, dù nhỏ nhất đến đâu, đều khiến hệ thống khởi động một dịch vụ mới. Thật vậy, máy trạng thái được đưa hẳn ra ngoài, vào một tệp văn bản mà hệ thống đọc. Mỗi sự kiện đi vào hệ thống từ một

phone line turned into a transition in that finite state machine. The existing process would start a new process dictated by the state machine to handle that event; then the existing process would either exit or wait on a queue.

đường dây điện thoại biến thành một bước chuyển trong máy trạng thái hữu hạn đó. Tiến trình đang chạy sẽ khởi động một tiến trình mới do máy trạng thái chỉ định để xử lý sự kiện ấy; sau đó tiến trình đang chạy hoặc thoát, hoặc chờ trên một hàng đợi.

This externalized state machine allowed us to change the flow of the application without changing any code (the Open-Closed Principle). We could easily add a new service, independently of any of the others, and wire it into the flow by modifying the text file that contained the state machine. We could even do this while the system was running. In other words we had hot-swapping and an effective BPEL (Business Process Execution Language).

Máy trạng thái nằm ngoài mã nguồn này cho phép chúng tôi thay đổi luồng chạy của ứng dụng mà không phải sửa dòng code nào (chính là Open-Closed Principle). Chúng tôi có thể dễ dàng thêm một dịch vụ mới, độc lập với mọi dịch vụ khác, rồi nối nó vào luồng bằng cách sửa tệp văn bản chứa máy trạng thái. Thậm chí làm được điều đó ngay khi hệ thống đang chạy. Nói cách khác, chúng tôi đã có hot-swapping và một thứ BPEL (Business Process Execution Language) đúng nghĩa.

The old ER approach of using disk files to communicate between services was too slow for this much more rapid flip-flopping of services, so we invented a shared memory mechanism that we called the 3DBB. 12 The 3DBB allowed data to be accessed by name; the names we used were names assigned to each state machine instance.

Cách cũ của ER là dùng tệp trên đĩa để các dịch vụ trao đổi với nhau, nhưng nó quá chậm với nhịp bật/tắt dịch vụ nhanh hơn hẳn ở đây, nên chúng tôi phát minh ra một cơ chế bộ nhớ chia sẻ gọi là 3DBB. 12 3DBB cho phép truy xuất dữ liệu theo tên; những cái tên chúng tôi dùng là tên gán cho từng thực thể (instance) của máy trạng thái.

The 3DBB was great for storing strings and constants, but couldn't be used for holding complex data structures. The reason for this is technical but easy to understand. Each process in MP/M lived in its own memory partition. Pointers to data in one memory partition had no meaning in another memory partition. As a consequence, the data in the 3DBB could not contain pointers. Strings were fine, but trees, linked lists, or any data structure with pointers would not work.

3DBB rất tuyệt để lưu chuỗi và hằng số, nhưng không dùng được cho các cấu trúc dữ liệu phức tạp. Lý do thuộc về kỹ thuật nhưng dễ hiểu. Mỗi tiến trình trong MP/M sống trong phân vùng bộ nhớ riêng. Con trỏ tới dữ liệu trong một phân vùng bộ nhớ chẳng có nghĩa gì trong phân vùng khác. Hệ quả là dữ liệu trong 3DBB không được chứa con trỏ. Chuỗi thì ổn, còn cây, danh sách liên kết, hay bất kỳ cấu trúc dữ liệu nào có con trỏ đều không chạy được.

The trouble tickets in the trouble ticket system came from many different sources. Some were automated, and some were manual. The manual entries were created by operators who were talking to customers about their troubles. As the customers described their problems, the operators would type in their complaints and observations in a structured text stream. It looked something like this:

Phiếu sự cố trong hệ thống đến từ rất nhiều nguồn khác nhau. Có cái tự động, có cái nhập tay. Các bản nhập tay do nhân viên trực tạo ra khi nói chuyện với khách hàng về sự cố của họ. Trong lúc khách mô tả vấn đề, nhân viên gõ lại lời than phiền của khách cùng những quan sát của mình thành một dòng văn bản có cấu trúc. Nó trông đại khái thế này:

/pno 8475551212 /noise /dropped-calls

- 12. Three-Dimensional Black Board. If you were born in the 1950s, you likely get this reference: Drizzle,
- 12. Three-Dimensional Black Board. Nếu bạn sinh vào thập niên 1950, hẳn bạn hiểu ngay điển tích này: Drizzle,

Drazzle, Druzzle, Drone.

Drazzle, Druzzle, Drone.

You get the idea. The / character started a new topic. Following the slash was a code, and following the code were parameters. There were thousands of codes, and an individual trouble ticket could have dozens of them in the description. Worse, since they were manually entered, they were often misspelled or improperly formatted. They were meant for humans to interpret, not for machines to process.

Bạn hình dung được rồi đấy. Ký tự / mở ra một chủ đề mới. Ngay sau dấu gạch chéo là một mã, và sau mã là các tham số. Có hàng nghìn mã như vậy, và một phiếu sự cố đơn lẻ có thể chứa hàng chục mã trong phần mô tả. Tệ hơn, vì được nhập tay nên chúng thường sai chính tả hoặc sai định dạng. Chúng sinh ra cho người đọc hiểu, chứ không phải cho máy xử lý.

Our problem was to decode these semi-free-form strings, interpret and fix any errors, and then turn them into voice output so we could read them to the repairman, up on a pole, listening with a handset. This required, among other things, a very flexible parsing and data representation technique. That data representation had to be passed through the 3DBB, which could handle only strings.

Vấn đề của chúng tôi là giải mã những chuỗi nửa tự do đó, diễn giải và sửa mọi lỗi, rồi biến chúng thành tiếng nói để đọc cho người thợ đang treo trên cột điện, tai áp vào ống nghe. Việc này đòi hỏi, cùng nhiều thứ khác, một kỹ thuật phân tích cú pháp và biểu diễn dữ liệu hết sức linh hoạt. Mà cách biểu diễn dữ liệu ấy lại phải đi qua 3DBB, thứ chỉ xử lý được chuỗi.

And so, on an airplane, flying between customer visits, I invented a scheme that I called FLD: Field Labeled Data . Nowadays we would call this XML or JSON. The format was different, but the idea was the same. FLDs were binary trees that associated names with data in a recursive hierarchy. FLDs could be queried by a simple API, and could be translated to and from a convenient string format that was ideal for the 3DBB.

Thế là, trên một chuyến bay giữa hai lần thăm khách hàng, tôi nghĩ ra một lược đồ đặt tên là FLD: Field Labeled Data. Ngày nay hẳn ta sẽ gọi nó là XML hay JSON. Định dạng thì khác, nhưng ý tưởng thì y hệt. FLD là những cây nhị phân gắn tên với dữ liệu theo một phân cấp đệ quy. Có thể truy vấn FLD qua một API đơn giản, và chuyển đổi qua lại với một định dạng chuỗi tiện lợi — lý tưởng cho 3DBB.

So, micro-services communicating through shared memory analog of sockets using an XML analog—in 1985.

Vậy đó: các micro-service giao tiếp với nhau qua bộ nhớ chia sẻ đóng vai socket, dùng một thứ tương đương XML — vào năm 1985.

There is nothing new under the Sun.

Chẳng có gì mới dưới ánh mặt trời.

Clear Communications — Clear Communications

In 1988, a group of Teradyne employees left the company to form a startup named Clear Communications. I joined them a few months later. Our mission was to build the software for a system that would monitor the communications quality of T1 lines—the digital lines that carried long- distance communications across the country. The vision was a huge monitor with a map of the United States crisscrossed by T1 lines flashing red if they were degrading.

Năm 1988, một nhóm nhân viên Teradyne rời công ty để lập startup tên là Clear Communications. Vài tháng sau tôi gia nhập họ. Nhiệm vụ của chúng tôi là xây phần mềm cho một hệ thống giám sát chất lượng truyền thông của các đường T1 — những đường truyền số tải liên lạc đường dài khắp cả nước. Viễn cảnh là một màn hình giám sát khổng lồ với bản đồ nước Mỹ chằng chịt các đường T1, nhấp nháy đỏ khi đường nào đó xuống cấp.

Remember, graphical user interfaces were brand new in 1988. The Apple Macintosh was only five years old. Windows was a joke back then. But Sun Microsystems was building Sparcstations that had credible X-Windows GUIs. So we went with Sun—and therefore with C and UNIX.

Nhớ nhé, giao diện đồ họa hồi 1988 còn mới tinh. Apple Macintosh mới ra đời được năm năm. Windows lúc ấy chỉ là trò đùa. Nhưng Sun Microsystems đang làm ra những chiếc Sparcstation với GUI X-Windows dùng được. Thế là chúng tôi chọn Sun — và kéo theo là C và UNIX.

This was a startup. We worked 70 to 80 hours per week. We had the vision. We had the motivation. We had the will. We had the energy. We had the expertise. We had equity. We had dreams of being millionaires. We were full of shit.

Đây là một startup. Chúng tôi làm 70 đến 80 giờ mỗi tuần. Chúng tôi có tầm nhìn. Chúng tôi có động lực. Chúng tôi có ý chí. Chúng tôi có sức lực. Chúng tôi có chuyên môn. Chúng tôi có cổ phần. Chúng tôi mơ thành triệu phú. Chúng tôi toàn nói phét.

The C code poured out of every orifice of our bodies. We slammed it here, and shoved it there. We constructed huge castles in the air. We had processes, and message queues, and grand, superlative architectures. We wrote a full seven-layer ISO communications stack from scratch—right down to the data link layer.

Code C tuôn ra từ mọi lỗ chân lông trên người chúng tôi. Chúng tôi tống nó vào chỗ này, nhét nó vào chỗ kia. Chúng tôi xây những lâu đài khổng lồ trên mây. Chúng tôi có tiến trình, có hàng đợi thông điệp, có những kiến trúc vĩ đại tột bậc. Chúng tôi viết trọn bộ ngăn xếp truyền thông bảy tầng ISO từ con số không — xuống tận tầng liên kết dữ liệu.

We wrote GUI code. GOOEY CODE! OMG! We wrote GOOOOOEY code.

Chúng tôi viết code GUI. GOOEY CODE! Trời ơi! Chúng tôi viết ra thứ code GOOOOOEY nhộp nháp.

I personally wrote a 3000-line C function named `gi()`; its name stood for Graphic Interpreter. It was a masterpiece of goo. It was not the only goo I wrote at Clear, but it was my most infamous.

Đích thân tôi viết một hàm C dài 3000 dòng tên là `gi()`; cái tên viết tắt cho Graphic Interpreter. Đó là một kiệt tác nhộp nháp. Nó không phải mớ nhộp nháp duy nhất tôi viết ở Clear, nhưng là mớ khét tiếng nhất.

Architecture? Are you joking? This was a startup. We didn't have time for architecture. Just code, dammit! Code for your very lives!

Kiến trúc à? Anh đùa đấy à? Đây là startup. Chúng tôi làm gì có thời gian cho kiến trúc. Code đi, quý tha ma bắt! Code như thể mạng sống phụ thuộc vào nó!

So we coded. And we coded. And we coded. But, after three years, what we failed to do was sell. Oh, we had an installation or two. But the market was not particularly interested in our grand vision, and our venture capital financiers were getting pretty fed up.

Thế là chúng tôi code. Rồi code. Rồi code nữa. Nhưng sau ba năm, thứ chúng tôi không làm được là bán hàng. Ừ thì cũng có một, hai lượt triển khai. Nhưng thị trường chẳng mấy mặn mà với tầm nhìn vĩ đại của chúng tôi, và các nhà đầu tư mạo hiểm thì bắt đầu chán ngán.

I hated my life at this point. I saw all my effort and dreams crashing down. I had conflicts at work, conflicts at home because of work, and conflicts with myself.

Giai đoạn đó tôi ghét cuộc đời mình. Tôi nhìn mọi công sức và giấc mơ của mình sụp đổ. Tôi mâu thuẫn ở chỗ làm, mâu thuẫn ở nhà vì chuyện chỗ làm, và mâu thuẫn với chính mình.

And then I got a phone call that changed everything.

Rồi tôi nhận một cuộc điện thoại làm thay đổi tất cả.

The Setup — Bối cảnh

Two years before that phone call, two things of significance happened.

Hai năm trước cuộc gọi ấy, có hai chuyện đáng kể xảy ra.

First, I managed to set up a uucp connection to a nearby company that had a uucp connection to another facility that was connected to the Internet. These connections were dial-up, of course. Our main Sparcstation (the one on my desk) used a 1200-bps modem to call up our uucp host twice per day. This gave us email and Netnews (an early social network where people discussed interesting issues).

Thứ nhất, tôi xoay xở thiết lập được một kết nối uucp tới một công ty gần đó, công ty này lại có kết nối uucp tới một cơ sở khác vốn đã nối vào Internet. Dĩ nhiên toàn là kết nối quay số. Chiếc Sparcstation chính của chúng tôi (chiếc trên bàn tôi) dùng modem 1200 bps gọi tới máy chủ uucp hai lần mỗi ngày. Nhờ vậy chúng tôi có email và Netnews (một mạng xã hội thời kỳ đầu, nơi người ta bàn luận những vấn đề thú vị).

Second, Sun released a C++ compiler. I had been interested in C++ and OO since 1983, but compilers were difficult to come by. So when the opportunity presented itself, I changed languages right away. I left the 3000-line C functions behind, and started to write C++ code at Clear. And I learned ...

Thứ hai, Sun phát hành một trình biên dịch C++. Tôi đã quan tâm tới C++ và OO từ năm 1983, nhưng trình biên dịch thì rất khó kiếm. Nên khi cơ hội tới, tôi đổi ngôn ngữ ngay lập tức. Tôi bỏ lại sau lưng những hàm C dài 3000 dòng, và bắt đầu viết code C++ ở Clear. Rồi tôi học...

I read books. Of course, I read *The C++ Programming Language* and *The Annotated C++ Reference Manual (The ARM)* by Bjarne Stroustrup. I read Rebecca Wirfs-Brock's lovely book on responsibility-driven design: *Designing Object Oriented Software*. I read *OOA and OOD and OOP* by Peter Coad. I read *Smalltalk-80* by Adele Goldberg. I read *Advanced C++ Programming Styles and Idioms* by James O. Coplien. But perhaps most significantly of all, I read *Object Oriented Design with Applications* by Grady Booch.

Tôi đọc sách. Dĩ nhiên tôi đọc *The C++ Programming Language* và *The Annotated C++ Reference Manual (The ARM)* của Bjarne Stroustrup. Tôi đọc cuốn sách tuyệt vời của Rebecca Wirfs-Brock về thiết kế hướng trách nhiệm: *Designing Object Oriented Software*. Tôi đọc *OOA and OOD and OOP* của Peter Coad. Tôi đọc *Smalltalk-80* của Adele Goldberg. Tôi đọc *Advanced C++ Programming Styles and Idioms* của James O. Coplien. Nhưng có lẽ quan trọng hơn cả, tôi đọc *Object Oriented Design with Applications* của Grady Booch.

What a name! Grady Booch. How could anyone forget a name like that. What's more, he was the Chief Scientist at a company called Rational! How I wanted to be a Chief Scientist ! And so I read his book. And I learned, and I learned, and I learned ...

Cái tên mới oách làm sao! Grady Booch. Ai mà quên nổi một cái tên như thế. Hơn nữa, ông ấy là Chief Scientist ở một công ty tên là Rational! Tôi thèm được làm Chief Scientist biết bao! Thế là tôi đọc sách của ông. Và tôi học, học nữa, học mãi...

As I learned, I also began debating on Netnews, the way people now debate on Facebook. My debates were about C++ and OO. For two years, I relieved the frustrations that were building at work by debating with hundreds of folks on Usenet about the best language features and the best principles of design. After a while, I even started making a certain amount of sense.

Càng học, tôi càng lao vào tranh luận trên Netnews, theo kiểu người ta tranh luận trên Facebook bây giờ. Các cuộc tranh luận của tôi xoay quanh C++ và OO. Suốt hai năm, tôi giải tỏa những bức dọc chất chồng nơi công sở bằng cách tranh luận với hàng trăm người trên Usenet về những tính năng ngôn ngữ tốt nhất và những nguyên tắc thiết kế tốt nhất. Sau một thời gian, tôi thậm chí bắt đầu nói ra được vài điều có lý.

It was in one of those debates that the foundations of **the SOLID principles** were laid.

Chính trong một cuộc tranh luận như thế mà nền móng của các nguyên tắc SOLID được đặt xuống.

And all that debating, and perhaps even some of the sense, got me noticed ...

Và tất cả những cuộc tranh luận đó, cộng thêm có lẽ đôi phần lý lẽ, đã khiến người ta để mắt tới tôi...

Uncle Bob — Uncle Bob

One of the engineers at Clear was a young fellow by the name of Billy Vogel. Billy gave nicknames to everyone. He called me Uncle Bob. I suspect, despite my name being Bob, that he was making an offhand reference to J. R. “Bob” Dobbs (see <https://en.wikipedia.org/wiki/File:Bobdobbs.png>).

Một trong những kỹ sư ở Clear là chàng trai trẻ tên Billy Vogel. Billy đặt biệt danh cho tất cả mọi người. Cậu ta gọi tôi là Uncle Bob. Tôi ngờ rằng, dù tên tôi đúng là Bob, cậu ta đang buột miệng nhắc tới J. R. “Bob” Dobbs (xem <https://en.wikipedia.org/wiki/File:Bobdobbs.png>).

At first I tolerated it. But as the months went by, his incessant chattering of “Uncle Bob, ... Uncle Bob,” in the context of the pressures and disappointments of the startup, started to wear pretty thin.

Ban đầu tôi cũng chịu được. Nhưng tháng này qua tháng khác, tiếng cậu ta lảm nhảm không dứt “Uncle Bob, ... Uncle Bob,” giữa áp lực và những thất vọng của một startup, bắt đầu trở nên khó nuốt.

And then, one day, the phone rang.

Rồi một ngày nọ, chuông điện thoại reo.

The Phone Call — Cuộc điện thoại

It was a recruiter. He had gotten my name as someone who knew C++ and object-oriented design. I’m not sure how, but I suspect it had something to do with my Netnews presence.

Đó là một nhà tuyển dụng. Ông ta có tên tôi trong danh sách những người biết C++ và thiết kế hướng đối tượng. Tôi không rõ bằng cách nào, nhưng tôi ngờ nó có liên quan tới sự hiện diện của tôi trên Netnews.

He said he had an opportunity in Silicon Valley, at a company named Rational. They were looking for help building a CASE 13 tool.

Ông ta nói có một cơ hội ở Thung lũng Silicon, tại một công ty tên là Rational. Họ đang tìm người phụ giúp xây dựng một công cụ CASE. 13

The blood drained from my face. I knew what this was. I don’t know how I knew, but I knew . This was Grady Booch’s company. I saw before me the opportunity to join forces with Grady Booch !

Máu dồn hết khỏi mặt tôi. Tôi biết đây là chuyện gì. Tôi không hiểu vì sao mình biết, nhưng tôi *biết*. Đây là công ty của Grady Booch. Trước mắt tôi là cơ hội được sát cánh cùng *Grady Booch*!

- 13. Computer Aided Software Engineering
 - 13. Computer Aided Software Engineering

ROSE — ROSE

I joined Rational, as a contract programmer, in 1990. I was working on the ROSE product. This was a tool that allowed programmers to draw Booch diagrams—the diagrams that Grady had written about in *Object-Oriented Analysis and Design with Applications* (Figure A.9 shows an example).

Tôi gia nhập Rational với tư cách lập trình viên hợp đồng vào năm 1990. Tôi làm việc trên sản phẩm ROSE. Đây là công cụ cho phép lập trình viên vẽ các Booch diagram — loại sơ đồ mà Grady đã viết trong *Object-Oriented Analysis and Design with Applications* (Figure A.9 là một ví dụ).

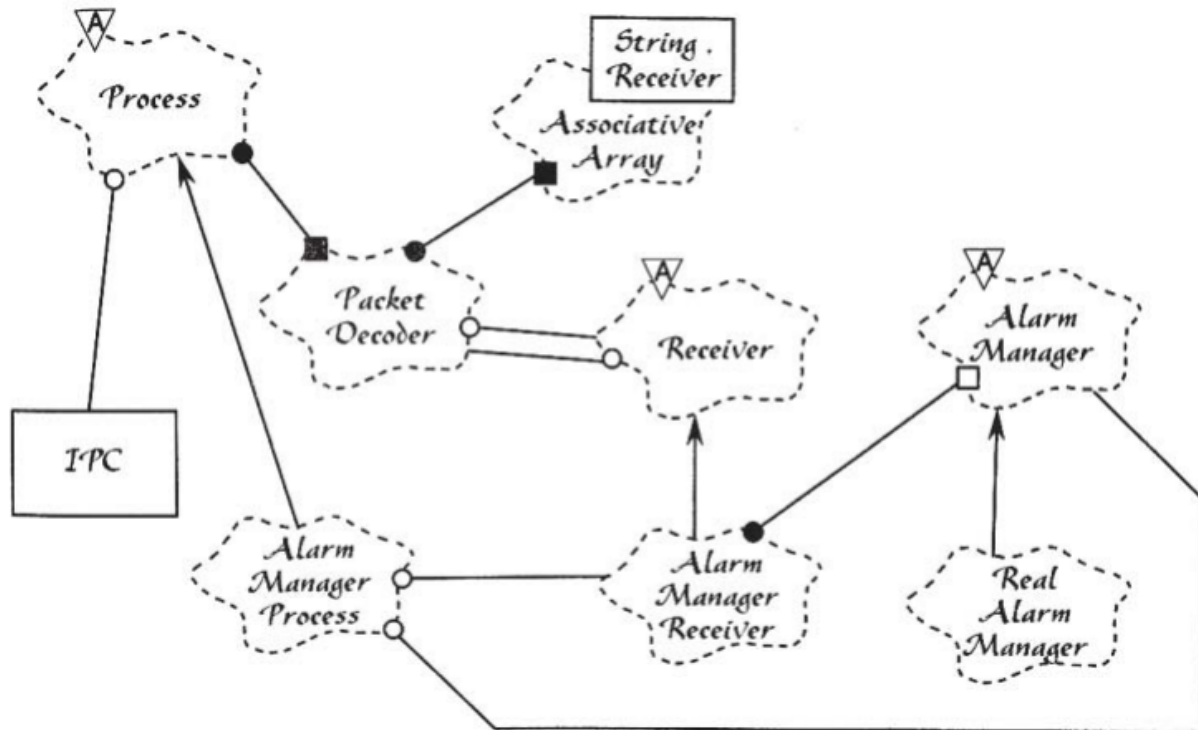


Figure A.9 A Booch diagram

Figure A.9 Một Booch diagram

The Booch notation was very powerful. It presaged notations like UML.

Ký pháp Booch rất mạnh. Nó báo trước cho những ký pháp như UML.

ROSE had an architecture—a real architecture. It was constructed in true layers, and the dependencies between layers were properly controlled. The architecture made it releasable, developable, and independently deployable.

ROSE có kiến trúc — một kiến trúc thực thụ. Nó được dựng thành các tầng đúng nghĩa, và các phụ thuộc giữa những tầng ấy được kiểm soát chặt chẽ. Kiến trúc đó khiến nó phát hành được, phát triển được, và triển khai được một cách độc lập.

Oh, it wasn't perfect. There were a lot of things we still didn't understand about architectural principles. We did not, for example, create a true plugin structure.

Ồ, nó chưa hoàn hảo đâu. Còn rất nhiều điều về các nguyên tắc kiến trúc mà hồi ấy chúng tôi chưa hiểu. Chẳng hạn, chúng tôi đã không tạo ra một cấu trúc plugin thực thụ.

We also fell for one of the most unfortunate fads of the day—we used a so-called object-oriented database.

Chúng tôi cũng dính vào một trong những mốt tai hại nhất thời đó — chúng tôi dùng cái gọi là cơ sở dữ liệu hướng đối tượng.

But, overall, the experience was a great one. I spent a lovely year and a half working with the Rational team on ROSE. This was one of the most intellectually stimulating experiences of my professional life.

Nhưng nhìn chung, đó là một trải nghiệm tuyệt vời. Tôi đã có một năm rưỡi đáng yêu làm việc cùng đội Rational trên ROSE. Đó là một trong những trải nghiệm kích thích trí tuệ bậc nhất trong đời nghề nghiệp của tôi.

The Debates Continued — Những cuộc tranh luận vẫn tiếp diễn

Of course, I did not stop debating on Netnews. In fact, I drastically increased my network presence. I started writing articles for C++ Report . And, with Grady’s help, I started working on my first book: *Designing Object-Oriented C++ Applications Using the Booch Method* .

Dĩ nhiên, tôi không ngừng tranh luận trên Netnews. Thật ra tôi còn tăng mạnh sự hiện diện của mình trên mạng. Tôi bắt đầu viết bài cho *C++ Report*. Và, với sự giúp đỡ của Grady, tôi bắt tay vào cuốn sách đầu tiên của mình: *Designing Object-Oriented C++ Applications Using the Booch Method*.

One thing bothered me. It was perverse, but it was true. No one was calling me “Uncle Bob.” I found that I missed it. So I made the mistake of putting “Uncle Bob” in my email and Netnews signatures. And the name stuck. Eventually I realized that it was a pretty good brand.

Có một điều làm tôi bận lòng. Nghe thì trái khoáy, nhưng là thật. Chẳng ai gọi tôi là “Uncle Bob” nữa. Tôi phát hiện ra mình nhớ nó. Thế là tôi phạm sai lầm khi đặt “Uncle Bob” vào chữ ký email và Netnews của mình. Và cái tên ấy dính luôn. Về sau tôi nhận ra đó là một thương hiệu khá ổn.

... By Any Other Name — ... Dù mang tên gì đi nữa

ROSE was a gigantic C++ application. It was composed of layers, with a strictly enforced dependency rule. That rule is not the rule that I have described in this book. We did not point our dependencies toward high-level policies. Rather, we pointed our dependencies in the more traditional direction of flow control. The GUI pointed at the representation, which pointed at the manipulation rules, which pointed at the database. In the end, it was this failure to direct our dependencies toward **policy** that aided the eventual demise of the product.

ROSE là một ứng dụng C++ khổng lồ. Nó gồm nhiều tầng, với một quy tắc phụ thuộc được thực thi nghiêm ngặt. Quy tắc đó không phải quy tắc tôi đã mô tả trong cuốn sách này. Chúng tôi không hướng các phụ thuộc về phía những chính sách mức cao. Thay vào đó, chúng tôi hướng phụ thuộc theo chiều truyền thống hơn: chiều của luồng điều khiển. GUI trỏ tới phần biểu diễn, phần biểu diễn trỏ tới các quy tắc thao tác, các quy tắc thao tác trỏ tới cơ sở dữ liệu. Rất cuộc, chính việc không hướng được phụ thuộc về phía chính sách đã góp phần khai tử sản phẩm.

The architecture of ROSE was similar to the architecture of a good compiler. The graphical notation was “parsed” into an internal representation; that representation was then manipulated by rules and stored in an object-oriented database.

Kiến trúc của ROSE giống kiến trúc của một trình biên dịch tốt. Ký pháp đồ họa được “parse” thành một biểu diễn nội bộ; biểu diễn ấy sau đó được các quy tắc thao tác lên và được lưu vào một cơ sở dữ liệu hướng đối tượng.

Object-oriented databases were a relatively new idea, and the OO world was all abuzz with the implications. Every object-oriented programmer wanted to have an object-oriented database in his or her system. The idea was relatively simple, and deeply idealistic. The database stores objects, not tables. The

Cơ sở dữ liệu hướng đối tượng khi ấy là ý tưởng còn khá mới, và cả thế giới OO xôn xao về những hệ quả của nó. Lập trình viên hướng đối tượng nào cũng muốn có một cơ sở dữ liệu hướng đối tượng trong hệ thống của mình. Ý tưởng khá đơn giản, và cực kỳ lý tưởng hoá. Cơ sở dữ liệu lưu đối tượng, chứ không lưu bảng.

database was supposed to look like RAM. When you accessed an object, it simply appeared in memory. If that object pointed to another object, the other object would appear in memory as soon as you accessed it. It was like magic.

Cơ sở dữ liệu lẽ ra phải trông giống như RAM. Khi bạn truy cập một đối tượng, nó cứ thế hiện ra trong bộ nhớ. Nếu đối tượng đó trỏ tới một đối tượng khác, đối tượng kia cũng sẽ hiện ra trong bộ nhớ ngay khi bạn truy cập tới. Cứ như phép màu.

That database was probably our biggest practical mistake. We wanted the magic, but what we got was a big, slow, intrusive, expensive third-party framework that made our lives hell by impeding our progress on just about every level.

Cơ sở dữ liệu ấy có lẽ là sai lầm thực tiễn lớn nhất của chúng tôi. Chúng tôi muốn phép màu, nhưng thứ nhận về là một framework của bên thứ ba: to, chậm, xâm lấn, đắt đỏ, và biến đời chúng tôi thành địa ngục bằng cách cản trở tiến độ ở gần như mọi cấp độ.

That database was not the only mistake we made. The biggest mistake, in fact, was over-architecture. There were many more layers than I have described here, and each had its own brand of communications overhead. This served to significantly reduce the productivity of the team.

Cơ sở dữ liệu đó không phải sai lầm duy nhất. Thật ra, sai lầm lớn nhất là kiến trúc quá đà. Có nhiều tầng hơn hẳn những gì tôi mô tả ở đây, và mỗi tầng lại kèm một kiểu chi phí giao tiếp riêng. Điều này khiến năng suất của cả đội giảm sút đáng kể.

Indeed, after many man-years of work, immense struggles, and two tepid releases, the whole tool was scrapped and replaced with a cute little application written by a small team in Wisconsin.

Thực tế là, sau nhiều người-năm làm việc, sau những vật lộn khổng lồ và hai lần phát hành nhạt nhẽo, toàn bộ công cụ bị vứt bỏ và thay bằng một ứng dụng nhỏ xinh do một đội nhỏ ở Wisconsin viết ra.

And so I learned that great architectures sometimes lead to great failures. Architecture must be flexible enough to adapt to the size of the problem. Architecting for the enterprise, when all you really need is a cute little desktop tool, is a recipe for failure.

Và tôi học được rằng những kiến trúc vĩ đại đôi khi dẫn tới những thất bại vĩ đại. Kiến trúc phải đủ linh hoạt để thích ứng với quy mô của bài toán. Kiến trúc kiểu doanh nghiệp, trong khi thứ bạn thực sự cần chỉ là một công cụ desktop nhỏ xinh, là công thức chắc chắn dẫn tới thất bại.

Architects Registry Exam — Architects Registry Exam

In the early 1990s, I became a true consultant. I traveled the world teaching people what this new OO thing was. My consulting was focused strongly on the design and architecture of object-oriented systems.

Đầu thập niên 1990, tôi trở thành một nhà tư vấn thực thụ. Tôi đi khắp thế giới dạy người ta cái thứ OO mới mẻ này là gì. Công việc tư vấn của tôi tập trung mạnh vào thiết kế và kiến trúc của các hệ thống hướng đối tượng.

One of my first consulting clients was Educational Testing Service (ETS). It was under contract with the National Council of Architects Registry Board (NCARB) to conduct the registration exams for new architect candidates.

Một trong những khách hàng tư vấn đầu tiên của tôi là Educational Testing Service (ETS). Họ có hợp đồng với National Council of Architects Registry Board (NCARB) để tổ chức các kỳ thi cấp chứng chỉ cho những ai muốn trở thành kiến trúc sư.

Anyone desiring to be a registered architect (the kind who design buildings) in the United States or Canada must pass the registration exam. This exam

Bất kỳ ai muốn trở thành kiến trúc sư có chứng chỉ hành nghề (loại kiến trúc sư thiết kế nhà cửa) tại Hoa Kỳ hoặc Canada đều phải vượt qua kỳ thi này. Kỳ thi đó

involved having the candidate solve a number of architectural problems involving building design. The candidate might be given a set of requirements for a public library, or a restaurant, or a church, and then asked to draw the appropriate architectural diagrams.

yêu cầu thí sinh giải một loạt bài toán kiến trúc liên quan tới thiết kế công trình. Thí sinh có thể được giao một bộ yêu cầu cho một thư viện công cộng, một nhà hàng, hay một nhà thờ, rồi phải vẽ ra các bản vẽ kiến trúc phù hợp.

The results would be collected and saved until such time as a group of senior architects could be gathered together as jurors, to score the submissions. These gatherings were big, expensive events and were the source of much ambiguity and delay.

Bài làm được thu lại và cất giữ cho tới khi tập hợp được một nhóm kiến trúc sư kỳ cựu làm giám khảo để chấm điểm. Những cuộc tụ họp này là sự kiện lớn, tốn kém, và là nguồn cơn của rất nhiều mơ hồ lẫn chậm trễ.

NCARB wanted to automate the process by having the candidates take the exams using a computer, and then have another computer do the evaluation and scoring. NCARB asked ETS to develop that software, and ETS hired me to gather a team of developers to produce the product.

NCARB muốn tự động hóa quy trình bằng cách để thí sinh làm bài trên máy tính, rồi để một máy tính khác đánh giá và chấm điểm. NCARB nhờ ETS phát triển phần mềm đó, còn ETS thuê tôi lập một đội lập trình viên để làm ra sản phẩm.

ETS had broken the problem down into 18 individual test vignettes. Each would require a CAD-like GUI application that the candidate would use to express his or her solution. A separate scoring application would take in the solutions and produce scores.

ETS đã chia nhỏ bài toán thành 18 bài thi (vignette) riêng biệt. Mỗi bài đòi hỏi một ứng dụng GUI kiểu CAD để thí sinh trình bày lời giải của mình. Một ứng dụng chấm điểm tách biệt sẽ nhận các lời giải và cho ra điểm số.

My partner, Jim Newkirk, and I realized that these 36 applications had vast amounts of similarity. The 18 GUI apps all used similar gestures and mechanisms. The 18 scoring applications all used the

same mathematical techniques. Given these shared elements, Jim and I were determined to develop a reusable framework for all 36 applications. Indeed, we sold this idea to ETS by saying that we'd spend a long time working on the first application, but then the rest would just pop out every few weeks.

Jim Newkirk — cộng sự của tôi — và tôi nhận ra 36 ứng dụng này có vô số điểm giống nhau. Cả 18 ứng dụng GUI đều dùng những thao tác và cơ chế tương tự nhau. Cả 18 ứng dụng chấm điểm đều dùng cùng những kỹ thuật toán học. Với ngân ấy điểm chung, Jim và tôi quyết tâm phát triển một framework tái dùng được cho cả 36 ứng dụng. Thật vậy, chúng tôi bán ý tưởng này cho ETS bằng cách nói rằng chúng tôi sẽ mất khá lâu cho ứng dụng đầu tiên, nhưng sau đó phần còn lại sẽ cứ thế bật ra sau mỗi vài tuần.

At this point you should be face-palming or banging your head on this book. Those of you who are old enough may remember the “reuse” promise of OO. We were all convinced, back then, that if you just wrote good clean object-oriented C++ code, you would just naturally produce lots and lots of reusable code.

Đến đây hẳn bạn đang ôm mặt hoặc đập đầu vào cuốn sách này. Ai đủ lớn tuổi hẳn còn nhớ lời hứa “tái sử dụng” của OO. Hồi đó tất cả chúng tôi đều tin rằng chỉ cần bạn viết code C++ hướng đối tượng sạch sẽ, tự khắc bạn sẽ sản sinh ra thật nhiều, thật nhiều code tái dùng được.

So we set about to write the first application—which was the most complicated of the batch. It was called Vignette Grande.

Thế là chúng tôi bắt tay viết ứng dụng đầu tiên — cái phức tạp nhất trong cả lô. Nó tên là Vignette Grande.

The two of us worked full time on Vignette Grande with an eye toward creating a reusable framework. It took us a year. At the end of that year we had 45,000 lines of framework code and 6000 lines of application code. We delivered this product to ETS, and they contracted with us to write the other 17 applications post-haste.

Hai chúng tôi làm toàn thời gian trên Vignette Grande với mục tiêu tạo ra một framework tái dùng được. Mất một năm. Cuối năm ấy chúng tôi có 45.000 dòng code framework và 6000 dòng code ứng dụng. Chúng tôi bàn giao sản phẩm này cho ETS, và họ ký hợp đồng để chúng tôi viết nốt 17 ứng dụng còn lại càng nhanh càng tốt.

So Jim and I recruited a team of three other developers and we began to work on the next few vignettes.

Thế là Jim và tôi tuyển thêm một đội gồm ba lập trình viên khác và bắt đầu làm vài vignette tiếp theo.

But something went wrong. We found that the reusable framework we had created was not particularly reusable. It did not fit well into the new applications being written. There were subtle frictions that just didn't work.

Nhưng có gì đó trục trặc. Chúng tôi phát hiện cái framework tái dùng được mà mình tạo ra lại chẳng tái dùng được là mấy. Nó không khớp tốt vào các ứng dụng mới đang viết. Có những điểm cản nhẹ khiến mọi thứ không chạy trơn.

This was deeply discouraging, but we believed we knew what to do about it. We went to ETS and told them that there would be a delay—that the 45,000-line framework needed to be rewritten, or at least readjusted. We told them that it would take a while longer to get that done.

Điều này khiến chúng tôi cực kỳ nản, nhưng chúng tôi tin mình biết phải làm gì. Chúng tôi tới gặp ETS và nói rằng sẽ có chậm trễ — rằng cái framework 45.000 dòng cần được viết lại, hoặc ít nhất là chỉnh lại. Chúng tôi nói sẽ mất thêm một thời gian nữa mới xong.

I don't need to tell you that ETS was not particularly happy with this news.

Chẳng cần tôi nói bạn cũng đoán được, ETS không mấy vui với tin này.

So we began again. We set the old framework aside and began writing four new vignettes simultaneously. We would borrow ideas and code from the old framework but rework them so that they fit into all four without modification. This effort took another year. It produced another 45,000-line framework, plus four vignettes that were on the order of 3000 to 6000 lines each.

Thế là chúng tôi bắt đầu lại. Chúng tôi gác framework cũ sang một bên và bắt tay viết bốn vignette mới cùng lúc. Chúng tôi mượn ý tưởng và code từ framework cũ nhưng chế lại sao cho chúng khớp vào cả bốn mà không cần sửa đổi. Nỗ lực này mất thêm một năm nữa. Nó cho ra một framework 45.000 dòng khác, cộng với bốn vignette mỗi cái cỡ 3000 đến 6000 dòng.

Needless to say, the relationship between the GUI applications and the framework followed **the Dependency Rule**. The vignettes were plugins to the framework. All the high-level GUI policy was in the framework. The vignette code was just glue.

Khỏi phải nói, quan hệ giữa các ứng dụng GUI và framework tuân theo Quy tắc Phụ thuộc (the Dependency Rule). Các vignette là plugin của framework. Toàn bộ chính sách GUI mức cao nằm trong framework. Code vignette chỉ là keo dán.

The relationship between the scoring applications and the framework was a bit more complex. The high-level scoring policy was in the vignette. The scoring framework plugged into the scoring vignette.

Quan hệ giữa các ứng dụng chấm điểm và framework thì phức tạp hơn một chút. Chính sách chấm điểm mức cao nằm trong vignette. Framework chấm điểm cắm vào vignette chấm điểm.

Of course, both of these applications were statically linked C++ applications, so the notion of plugin was nowhere in our minds. And yet, the way the dependencies ran was consistent with the Dependency Rule.

Dĩ nhiên, cả hai loại ứng dụng đều là ứng dụng C++ liên kết tĩnh, nên khái niệm plugin chẳng hề nằm trong đầu chúng tôi. Vậy mà chiều chạy của các phụ thuộc lại nhất quán với Quy tắc Phụ thuộc.

Having delivered those four applications, we began on the next four. And this time they started popping out the back end every few weeks, just as we had predicted. The delay had cost us nearly a year on our schedule, so we hired another programmer to speed the process along.

Sau khi giao xong bốn ứng dụng đó, chúng tôi bắt đầu bốn cái tiếp theo. Và lần này chúng bật ra ở đầu kia sau mỗi vài tuần, đúng như chúng tôi đã dự đoán. Đợt chậm trễ ngốn của chúng tôi gần một năm trong lịch trình, nên chúng tôi thuê thêm một lập trình viên nữa để đẩy nhanh tiến độ.

We met our dates and our commitments. Our customer was happy. We were happy. Life was good.

Chúng tôi đáp ứng đúng hạn và đúng cam kết. Khách hàng vui. Chúng tôi vui. Đời đẹp.

But we learned a good lesson: You can't make a reusable framework until you first make a usable framework. Reusable frameworks require that you build them in concert with several reusing

applications.

Nhưng chúng tôi rút ra một bài học hay: Bạn không thể làm ra một framework tái dùng được cho tới khi bạn làm ra được một framework dùng được đã. Framework tái dùng được đòi hỏi bạn phải xây nó song song với nhiều ứng dụng đang tái dùng nó.

Conclusion — Kết luận

As I said at the start, this appendix is somewhat autobiographical. I've hit the high points of the projects that I felt had an architectural impact. And, of course, I mentioned a few episodes that were not exactly relevant to the technical content of this book, but were significant nonetheless.

Như tôi đã nói ở đầu, phụ lục này ít nhiều mang tính tự truyện. Tôi đã điểm qua những khoảnh khắc nổi bật của các dự án mà tôi thấy có tác động về mặt kiến trúc. Và dĩ nhiên, tôi có nhắc tới vài giai đoạn không thật sự liên quan tới nội dung kỹ thuật của cuốn sách này, nhưng dù sao vẫn đáng kể.

Of course, this was a partial history. There were many other projects that I worked on over the decades. I also purposely stopped this history in the early 1990s—because I have another book to write about the events of the late 1990s.

Tất nhiên, đây chỉ là một phần lịch sử. Còn nhiều dự án khác tôi đã làm suốt mấy thập niên qua. Tôi cũng cố ý dừng câu chuyện ở đầu thập niên 1990 — bởi tôi còn một cuốn sách khác phải viết về những sự kiện cuối thập niên 1990.

My hope is that you enjoyed this little trip down my memory lane; and that you were able to learn some things along the way.

Tôi hy vọng bạn đã tận hưởng chuyến dạo nhỏ này qua ký ức của tôi; và học được đôi điều dọc đường.